

LabelKit

采集数据自动标注工具

产品设计说明书 (Product Design Specification)

文档版本	v1.16
日期	2026-08-20
状态	评审修订稿
目标读者	开发工程师、算法工程师、测试工程师
文档定位	实现级设计规格 —— 开发者依据本文档即可完成实现，无需自行补全任何设计决策

修订历史

版本	日期	修订内容
v1.0	2026-07-02	初版（评审稿）
v1.1	2026-07-02	按评审意见修订：① 各模块新增报文级输入/输出示例（3.1.6—3.11.3）并补全打分归一化公式等细节；② 新增日志系统设计——M12 日志模块（3.12）、第 7 章重写为「日志系统与可观测性」、trace 追踪日志与 rubric 优化闭环（7.5）、[trace] 配置（5.2）
v1.2	2026-07-02	按第二轮评审意见修订：① 新增「算子对输出集的影响分析」（2.3.2）；② 质量打分新增批内定量优选 <code>quality.selection = "top_ratio"</code> （3.4.3），全局精确定量与生成补齐回路列入演进路线（8.3 O6）；③ 生成算子支持多 LLM 混合与风格模板（3.6.2）；④ 算子级算法增强：多评审团投票、双顺序裁决、self-consistency 标注、SemDeDup 语义去重可选级（8.4 演进路线总表）
v1.3	2026-07-02	按第三轮评审意见修订：标注与生成拆分为两个独立模块——生成独立为 M6 generate（3.6，配置键 / Stage 类 / API 零变更），原 M6—M11 按流水线位置全量重编号为 M7—M12（全部交叉引用、图 2-1/2-2、目录同步更新）
v1.4	2026-07-02	新增纯生成模式（ <code>run.mode = "generate_only"</code> ）：无输入数据从零合成——配置种子池（Self-Instruct 形态）或无种子条件化（Persona Hub / Cosmopedia 形态）两种形态，单遍执行，产出照常走治理/标注管线（3.6.2、3.10.3、2.3.1 ④）；合成样本统一标记修正为 <code>generator ≠ null</code>
v1.5	2026-07-03	按 E2E 加固与校验回调评审修订（散注全文，标「v1.5」）：① 用户校验回调两枚——结构引擎 L2.5 <code>output.validator</code> 与生成样本过滤 <code>generate.sample_validator</code> （3.8.2、3.6.2；错误类 <code>callback_violation</code> ，7.6）；② 认证类 401/403 首错立即熔断（3.9.3、7.6）；③ dry-run 产物隔离（2.4）、trace 文件惰性打开（7.1）、 <code>per_criterion_tie_rate</code> / <code>ll_lossy</code> 等观测增强（6.4、7.2）
v1.6	2026-07-03	多 API Key 负载均衡与熔断交付（对齐决策见 1.6）：① 密钥池——profile 可声明多把 API Key（ <code>api_key_envs</code> ，5.1）：逐尝试最少在途选择、429 按密钥冷却即时轮换、认证失败按密钥禁用（最后一把存活密钥被禁才立即熔断）、全池冷却有界驻留（ <code>run.max_park_s</code> ，5.2）；观测新增三事件与报表 <code>keys</code> 子块（7.2、6.4），单密钥配置在数据产出与熔断/退出语义上与 v1.5 一致（429 等待路径修订见 3.9.3 重试行）；② 熔断交付——熔断中止改为原子交付已完成批（report 标 <code>partial_delivery</code> 、counts 增 <code>unprocessed</code> ，3.10.3、3.11.2、6.4）
v1.7	2026-07-07	分类算子与按类条件化（对齐决策见 1.6）：① 分类算子——新增 M13 classify（3.13，链序 dedup 之后、quality 之前）：LLM 封闭集分类（内部 Schema enum 词表硬校验，无 <code>uniqueItems</code> ——重复标签由代码侧确定性归一化）、单/多标签可配（ <code>classify.assignment</code> ）、可选 self-consistency 投票、失败归兜底类（ <code>classify.fallback_class</code> ）；multi 模式按标签向批尾扇出兄弟信封（Stage 契约增 ②a 例外，4.3； <code>counts.fanout</code> 与不变量扩展，6.4）；② 按类条件化—— <code>[class.<name>.*]</code> 白名单覆盖（5.2）：quality 批内按类分池（3.4.3）、annotate/verify 按类指令与评审维度（3.5.2、3.7.2）、generate 按类种子池与生成指令（3.6.2）；③ 观测面—— <code>_meta</code> 增恒在键 <code>classification</code> （6.3）、report 增 <code>classify</code> 节与 <code>quality.by_class</code> （6.4）、trace 通道增 <code>classify</code> 与新事件 <code>classify.decision</code> （7.2）、错误码增 <code>classification_invalid</code> （7.6）。默认关（ <code>classify.enabled = false</code> ），关闭时数据产出与 v1.6 逐字段一致（ <code>_meta.classification: null</code> 除外）
v1.8	2026-07-13	时序流语义分割与动作摘取（对齐决策见 1.6）：① 时间轴与会话化——[stream] 输入侧声明（5.2）：排序键与按分区键单调性校验、gap/key/上限会话规则（M2 会话装配器，3.2.8），切批改整会话装箱（3.10.3）；② 新算子——M14 segment 语义分段（3.14，滑动 LLM 边界裁决 + 噪声剔除，把成员帧收拢为序列记录 <code>episode</code> ；Stage 契约增 ②b 例外，4.3）与 M15 extract 转移/动作摘取（3.15，相邻帧对 → 结构化动作，写入 <code>item.transitions</code> ）；③ 序列级下游适配——M3/M13/M4/M5/M7 收序列记录（3.3.3、3.13.3、3.4.3、3.5.2、3.7.2/3.7.3），内置轨迹 <code>rubric default:trajectory</code> （附录 A.3）；④ 观测面——Status 增 <code>absorbed</code> / <code>dropped_noise</code> 两值（4.1）、 <code>_meta</code> 增恒在键 <code>stream</code> （6.3）、report 增 <code>stream</code> 节与守恒式扩展（6.4）、trace 通道增 <code>segment/extract</code> 与四个新事件（7.2）、错误码增 <code>segmentation_invalid</code> / <code>extraction_invalid</code> （7.6）。默认关（ <code>segment.enabled = false</code> ），关闭时数据产出与 v1.7 逐字段一致（ <code>_meta.stream: null</code> 除外）
v1.9	2026-07-16	线索缝合与层级工作单元（对齐决策见 1.6）：① 新算子——M16 stitch 线索缝合（3.16，链序 segment 之后、dedup 之前）：会话内碎片以「单调选池 LLM 判定 × 机械先验合取」保守缝合为线索 <code>thread</code> （有界二遍复评修正贪心漏缝），产出三级结构 <code>thread > fragment > step</code> ；被并 <code>episode</code> 壳置新状态 <code>stitched</code> 、幸存信封 <code>Record</code> 重绑、 <code>below_min_len</code> 短段先进候选池救援（Stage 契约增 ②c 例外，4.3）；接缝零 LLM 机械占位（ <code>extract</code> 按 <code>seam_indexes</code> 生成 T10 四键占位步，3.15）；② 下游适配——dedup 判重面 = 线索重绑配方（3.3.3）、quality/annotate/verify 以线索为单元（步行渲染 <code>thread_seam</code> 后缀、按碎片配额降采样、七段序列评审 + 缺陷词表增 <code>wrong_stitch</code> ，3.4.3/3.5.2/3.7.2）；③ 配置面——新节 [stitch]（11 键，5.2）： <code>max_open</code> 池容量 / <code>bias</code> 保守偏置 / <code>rescue_short</code> / <code>repass</code> / <code>stale_gap_steps</code> / <code>votes</code> 采样多数决（默认 1 不启用）等，M1 增 <code>stitch</code> ⇒ <code>segment</code> 等约束（3.1.4）；④ 观测面——Status 增 <code>stitched</code> （4.1）、M11 增第四路由（3.11.2）、counts 增 <code>stitched</code> / <code>threads</code> （= <code>episodes - stitched</code> 导出式）与守恒式扩项（6.4）、report 增 <code>stream.stitch</code> 子块、trace 通道 10→11（增 <code>stitch</code> ）与两事件 <code>stitch.judge</code> / <code>stitch.thread</code> （7.2）、错误码增 <code>stitch_invalid</code> （7.6）、 <code>_meta.stream</code> 增 <code>thread_id</code> / <code>fragments</code> / <code>steps</code> 行内 resumed（6.3）。默认关（ <code>stitch.enabled = false</code> ），关闭时主输出 / <code>rejects</code> / <code>report.json</code> 与 v1.8 逐字节等价（例外两处：dry-run stderr 的 <code>stitch_calls=0</code> 行与 <code>streamxverify</code> 缺陷词表 <code>wrong_stitch: 0</code> 行——3.16.4 退化锚）

v1.10	2026-07-17	Console 实时面板（对齐决策见 1.6；规格 2026-07-17 二轮定稿并实现——一轮 spec-only 定稿后经三路独立审计修订：代码可行性/亲和性 2B+5M+9m、文档清单 2B+6M+8m、deep-search refute/elevate 1 推翻 + 4 修订 + 6 成立，裁决 U19-U27 并入）：① 三态 console——<code>--console auto \ rich \ plain</code> 与 <code>[console]</code> 配置节（5.1，解析产物 <code>mode_resolved</code> 由 M1 冻结）：7.7 进度显示面重写为三态规格——rich 档为双区内联实时面板（日志上方滚动 + 底部画布 <code>asyncio tick</code> 节流原地重绘：批进度、段棋盘（<code>llm.call</code> 括号归属 × <code>estimate_run</code> 运行级分母）、状态账、LLM 用量/密钥池/熔断、键盘开关 <code>? h l e + - p q</code>；工业同型 <code>buck2 superconsole / BuildKit -progress</code>、键盘先例 <code>memray</code>），plain 档与 v1.9 <code>stderr</code> 行为等价（三层回归锚 7.8——实跑逐字节 <code>diff</code> 经审计证伪，改为固定时钟黄金快照 + <code>dry-run</code> 逐字节 + 实跑归一化结构等价；<code>heartbeat_s=0</code> 默认下），auto 按 <code>TTY ^ log_format ^ TERM ^ rich</code> 可探测判定（<code>NO_COLOR</code> 不参与——rich 原生剥色保布局，U25）；② 架构——面板为 M12 可观测性的第四个纯消费面：<code>ProgressListener</code> 五回调进程内旁路协议（3.12.3——<code>on_run_context/on_estimate/on_event/on_stage/on_stop_requested</code>；不产生 <code>TraceEvent</code>、不入 7.2 目录；<code>on_event</code> 载荷 <code>none</code> 档预脱敏 U22；<code>sink</code> 侧转发异常自吞 U23）+ <code>LLMClient.snapshot()</code> 只读快照与 p50 有界样本窗（3.9.2/3.9.3，唯一新增采集点）+ plain 行格式纯函数 <code>console_format</code> 下沉 <code>common</code> 层（U21）；渲染器归 CLI 层（依赖方向不变）；渲染异常自吞降级 plain，永不影响退出码与产出；③ 依赖面——\$2.6 白名单增 <code>rich</code>（懒 <code>import</code>，仅 CLI 层单文件触点，M1 仅 <code>find_spec</code> 探测）；④ T16 有界修订——rich 面板状态账展示 <code>stitched/threads</code>（与 <code>report.counts</code> 口径对齐），plain 进度行与文本版终版摘要键集逐字节不动（T16 固定键集约束收窄为 plain 面专属）。零 7.2 事件目录改动、<code>report.json</code> 零新键；设计裁决 U1-U27 见 <code>docs/dev/SPEC-tui-console.md</code> §2，工业调研 [C-1]-[C-20] 见 <code>docs/dev/PROPOSAL-tui-console.md</code>、审计增量 [C-21]-[C-42] 见 <code>SPEC</code> §6
v1.11	2026-07-22	上下文预算与视觉能力自动推导（对齐决策见 1.6；三路独立预实现审计折入 V22-V27，需求方裁决 A1-A7 闭合）：① 上下文预算——对每次 LLM 调用建立不变式 <code>est(prompt) + max_output_tokens + margin ≤ context_window</code>：<code>[llm.<name>]</code> / <code>[embedding.<name>]</code> 增 <code>context_window</code> 声明（5.1，0 = 未声明 = 该 profile 预算关闭、行为与 v1.10 逐字节一致；声明部署实效窗口而非厂商表——V6/V26）、<code>margin = max(256, [10%·cw])</code> 冻结；零依赖字符类估算器与装填纯函数下沉 <code>labelkit/common/runtime/budget.py</code>（<code>est_text/est_image/fit_text/min_window/TEMPLATE_HEAD_TOKENS</code> 冻结常数——V7/V8/V22）；② 动态装填——条数型参数降级为上限值、按实际内容逐项装填：<code>segment</code> 滑窗改贪心装填（重叠 1 帧与接缝语义不变，<code>w_min</code> 静态护栏 + 强制 2 帧封窗，V9/V10）、<code>quality/classify/annotate/verify</code> 树渲染与步骤行动态收缩（<code>per-judge / min-over-panel</code>——V25②）、<code>annotate</code> 关键帧 <code>k_eff</code> 收缩、<code>generate</code> 种子尾丢、<code>embedding</code> 输入截断（V15）、M8 L3 修复超预算短路（V25①）；<code>estimate_run</code> 的 <code>segment_calls</code> 报 <code>w_min</code> 上界（V12，console 零改动）；③ 图片成本测量-反应式三层——<code>default_image_px</code> 采样工作点（<code>max_image_px</code> 降为升级天花板 + 像素域硬限面，V18）→ 溢出裁帧保清重试（V20，有界 ≤2）→ <code>verify</code> 低置信裁帧升清重试（V21，修复路径专属）；<code>usage.prompt_tokens</code> 在线校准每图成本（V19，批冻结快照保确定性；<code>vendor</code> 公式降级为首批先验 ×1.2——V17 需求方向裁决）；④ M9 咽喉终检与错误分类——<code>complete()</code> 分派前预检（<code>ContextOverflowError(phase="precheck")</code>）、终止原因归一（<code>length/max_tokens</code> → 终态 <code>output_truncated</code> 永不进修复环；双协议 <code>model_context_window_exceeded</code> 200 形态 → <code>context_overflow</code> <code>reactive</code>，V11/V16/V24）、预算门控 400 错误嗅探（[C-75] 种子集，V20）；熔断矩阵 = <code>precheck/最小单元/200</code> 形态永不喂连击、嗅探 400 降级耗尽终局由终局吞点经共享 <code>budget.feed_reactive_terminal</code> 补喂恰一次（A7 + 审计盲点修订）；错误码增 <code>context_overflow / output_truncated</code>（7.6）；⑤ 视觉能力自动推导——删除 <code>segment.use_vision</code>（存量显式键定向 <code>CONFIG_ERROR</code>，V2），改为 M1 冻结 <code>parse product vision_resolved</code>（= ui 模态 ∧ <code>segment</code> 启用 ∧ <code>strategy llm/hybrid</code> ∧ <code>profile.supports_vision</code>，V1-V5；成本控制面 = <code>segment.llm</code> 指向纯文本 <code>profile</code>）；⑥ 观测面——启动预算 INFO 行、<code>report.budget</code>（<code>profiles/w_min/truncations/overflow_records/image_cost/degrade_retries/escalations</code>，计数 only）与 <code>report.stream.windows</code>（预算门控在场，6.4）、零新 <code>trace</code> 通道。默认（全 profile 未声明）与 v1.10 逐字节等价；决策 V1-V27 详表见 <code>docs/dev/SPEC-context-budget.md</code> §2（调研与审计引用 [C-1]-[C-84] 见 <code>docs/dev/PROPOSAL-context-budget.md</code>；<code>z.ai</code> 实效窗/200 形态溢出 <code>oracle</code> 实测见 <code>docs/dev/E2E-FINDINGS.md</code> #15-#21）

v1.12	2026-08-12	流模式帧级分类与标注（对齐决策见 1.6；三方预实现审计 2026-08-12 折入，凡与提案不一致处以裁决为准）：① 帧级双粒度——流模式序列信封的成员帧顺带产出帧级闭集分类与帧级按类标注：新配置节 <code>[frame.classify]</code>（帧类表 + <code>fallback_class</code>，5.2）、<code>[frame.annotate]</code>（帧级指令 + 帧级输出 JSON Schema，<code>schema_path / schema_inline</code> 恰一，5.2）与按帧类覆盖 <code>[frame.class.<name>.annotate]</code>（instruction/examples/enabled 三键白名单）；M13 帧级批量判决（—episode 一调用、预算分窗、内部 Schema 定长 enum 数组、缺项/窗失败归 <code>fallback_class</code> 永不 episode failed，3.13）+ M5 逐成员按类标注（质量门之后逐帧调用、帧 Schema 显式路由不计 <code>resolved_at</code>、幂等只补缺位、成员失败=占键 None 不写 <code>item.errors</code>，3.5）；产物为 <code>PipelineItem</code> 两个按成员 <code>record.id</code> 键控的新 dict 字段（4.1，克隆按引用共享、帧 pass 只在首标签信封执行）；成员帧状态机、链序、Stage 契约 ②a/②b/②c、守恒等式零改动；② 输出面——<code>_meta.stream</code> 增 <code>members[]</code> 块（<code>member_sources</code> 后、<code>session_split</code> 前；条目字段序 index, id[, label][, annotation, status], status 三值闭集 annotated/skipped/failed, 6.3）+ emitter 写前对每个非 null 帧标注 <code>validate_only(obj, schema=frame_schema)</code> 兜底（非法帧对象零落盘，3.11.2）；成员失败不入 rejects、不触发 —strict（rejects 面零改动）；③ 修复面第四向——verify 手术同步帧产物：收缩成员删键 / 回收成员懒加载补跑（算子间导入白名单 3→4，新增 <code>classify.classify_frames</code>；<code>annotate.annotate_member</code> 并入 <code>annotate</code> 修复面族，3.7）；④ 装箱器下沉——<code>segment._pack_windows</code> 纯函数下沉为 <code>budget.pack_windows</code>（segment 改 import 行为字节等价，帧级批量判决以零重叠调用形复用）；⑤ 观测面——<code>estimate_run</code> 增 <code>frame_classify_calls / frame_annotate_calls</code> 两键（粗上界 = 预扫描帧总数，与 <code>segment_calls</code> 同源；开关关 ⇒ 0）、dry-run 估算行改写（无条件打印；五 golden 重采 + mix 主/姊妹双工程两新 golden 共七个）、<code>report.stream</code> 增 <code>frame_classify / frame_annotate</code> 两个条件在子场块（6.4）、trace 两事件 <code>classify.frame / annotate.frame</code>（通道枚举维持 11 值、零新错误 kind，7.2/7.6）、console 段棋盘分母折入（classify 分母 = <code>classify_calls + frame_classify_calls</code>，annotate 同理，7.7）；⑥ 上手示例——<code>examples/mix/</code> 独立成套自带 <code>config.toml</code>：UI 控件树主工程（截图+控件树 17 帧对，双 profile 混合接入——DeepSeek anthropic 路由 <code>deepseek-v4-flash</code> 承担文本判决面（segment/帧级批量分类/stream 质量打分，密钥 <code>LABELKIT_DEEPSEEK_KEY</code>），<code>z.ai glm-5.2</code> 承担视觉必需面（<code>classify/annotate/frame.annotate/verify</code>）+ 文本姊妹工程 <code>project-text.toml</code>（纯 DeepSeek 最低成本形态）。帧粒度全关时与 v1.11 字节等价（唯 dry-run 估算行与 <code>estimate</code> 键表例外）；裁决详表见 <code>docs/dev/SPEC-frame-annotation.md</code> §2（问题验证与行业调研见 <code>docs/dev/PROPOSAL-frame-annotation.md</code>）
v1.13	2026-08-13	时间流生成（对齐决策见 1.6；三方预实现审计 2026-08-13 折入，凡与提案不一致处以裁决为准）：① generate_only 第三形态——新节 <code>[generate.stream]</code>（7 键，5.2）把纯生成模式从「一次一条独立样本」扩为「一次一条序列、多条序列机械交织成时间流」：LLM 只做两类内容调用（—序列一次蓝图定步数与逐步帧类、—序列一次帧实现按蓝图逐位产出帧内容，噪音帧批量实现复用平面生成模板），会话装箱、交叉、噪音插入、重复重发、时间戳铺设全部由单流 <code>Random(f"{seed}:0:generate")</code> 顺序消费的机械交织器（零 LLM 零 IO）完成（3.6.5）；② 产物一式两份——可重放的时间流工件 <code>{output_stem}.stream.jsonl</code>（M11 第五输出通道，行 = 输入格式 + <code>truth</code> 真值三件套，6.5）与直装序列信封（<code>kind="sequence"</code>、序列类与帧类两级 inherited 标签、帧类真值随 <code>member_classifications</code> 落 <code>_meta.stream.members[]</code>），信封经 dedup → <code>classify</code>（inherited 幂等零调用）→ <code>quality</code>（default:trajectory 自动解析条件扩为 <code>segment.enabled v generate_stream.enabled</code>）→ <code>annotate</code> → <code>verify</code> → <code>emit</code>；<code>segment/stitch/extract</code> 不参与，<code>stream × generate</code> 互斥条文字面维持（3.6.5、8.3 O3 核销）；③ 按序列类标注 Schema——<code>[class.<name>.annotate]</code> 白名单增 <code>schema_path / schema_inline</code>（覆盖语义、缺省回落全局 <code>output.schema</code>，兑现 v1.7 演进候选），M5 六消费点与 M11 写前终检全部改按该行类有效 Schema 取值（3.4 语义、3.5.2、3.11.2）；M8 <code>complete_validated</code> 增显式待遇参数 <code>user_treatment</code>——按类 Schema 调用显式传 schema 仍属用户待遇族（L2.5 与 <code>resolved_at</code> 记账双保留，正面丢掉 v1.12「显式 Schema = 放弃记账」的弯折，3.8.2）；④ 两个内部 Schema 构造器——<code>plan_schema(names, length)</code>（蓝图 steps 定长 + 帧类 enum 闭集）与 <code>realize_schema(step_schemas)</code>（帧实现 <code>prefixItems</code> 逐位包装 + <code>"items": false</code> 封尾，draft 2020-12 原生，L2 直接可校验，3.8.1）；⑤ 判决形序列评审——经典路径（segment 关闭）遇 <code>kind="sequence"</code> 走判决形模板（<code>VERDICT_SCHEMA</code> + 【任务指令】/【成员帧摘要】/【标注结果】三段证据，无缺陷词表/边界余量/片段结构），流式驱动器的缺陷词表面零改动（3.7.5）；⑥ 观测面——<code>report.generate</code> 增 <code>stream</code> 子块（12 键 counts-only）、<code>report.run</code> 摘要族增工件条目（路径/sha256/行数）、<code>estimate_run</code> 的 <code>generate_only</code> 分支改复用 M6 计划期纯函数精确复演（<code>classify_calls = 0</code>；估算行格式零改动，既有七个 dry-run golden 字节不动、新增第八个 <code>dryrun-synth-stream.txt</code>）、<code>TEMPLATE_HEAD_TOKENS</code> 增 <code>generate_plan / generate_realize</code> 两键；零新 trace 通道、零新事件、零新错误 kind（7.2/7.6）；⑦ 示例工程——独立第五例 <code>examples/synth-stream</code>（自含单 profile DeepSeek <code>config.toml</code>，两序列类 × 各 3 条 × 各自独立标注 Schema、三帧类含一个结构化帧类，真跑 6 条序列 / 29 行工件；工件可原样重放为 process 模式输入）。形态全关（默认）时全系统与 v1.12 字节等价；裁决详表见 <code>docs/dev/SPEC-stream-generation.md</code> §2（问题验证与行业调研见 <code>docs/dev/PROPOSAL-stream-generation.md</code>）
v1.13 修订	2026-08-14	代码规则整改（对齐决策见 1.6；无功能变更、无配置变更，规格语义零改动）：① 语言分工——生产代码的注释与 docstring 统一中文，标识符、日志、报错、异常文案与 CLI 输出统一英文（LLM 提示词模板与 spec 冻结的输出数据——<code>thread_seam</code> 步骤文本、缺陷表 <code>detail</code>、打包 rubric 准则——保持中文原样）；② 规模规则——每文件 ≤ 2000 行、每函数 ≤ 50 行且 ≤ 5 入参：四个冻结装配面收参为参数对象 <code>CallScope</code>（M8，3.8.3）/ <code>AnnotatePromptOptions</code>（M5，3.5.2）/ <code>VerifyPromptOptions</code>（M7，3.7.5）/ <code>ThreadCard</code>（M16，3.16.3），编排器收参为 <code>RunServices</code>（M10，3.10.3；自 <code>labelkit.orchestration</code> 导出）——字段名与语义逐项不变，v1.7—v1.13 的「追加式末位 kwarg」叙事退场；③ M1 拆分——<code>loader.py</code> 只留公开入口、console 模式裁定与 <code>ResolvedConfig</code> 装配，解析与校验下沉六个包内私有模块 <code>_collect / _sections / _schemas / _rubrics / _classviews / _constraints</code>（公开面 <code>load / default_rubric / ResolvedConfig</code> 不变，3.1.3）；④ M9 收敛——<code>_record_provider_result</code> 恒传关键字 <code>hard</code>（调用形嗅探分支删除，3.9.3）；⑤ 重冻结——八个 <code>tests/cli/goldens/dryrun-*.txt</code> 与 <code>console_format</code> 两条 plain 锚点（进度行、终版摘要头）整体重冻结到英文串上，键集、行结构、数值与信息集不变（7.7/7.8）。

v1.14	2026-08-18	帧类构成档位与时间字段回填（对齐决策见 1.6；三方预实现审计 2026-08-18 折入，凡与提案不一致处以裁决为准）。两个彼此正交的时间流生成增量，共用一个修订号，全部默认关闭：① 帧类构成档位——新档位表 <code>[[generate.stream.tiers]]</code>（三键 <code>tier_rank / weight / frame_classes</code>，5.2）：一个档位的定义就是该档序列的帧类构成集合（不携带质量指令、不管帧内部语义质量）；<code>tier_rank</code> 即档位身份（表内唯一、全表连续覆盖 1..N，工具不赋予序数高低任何质量方向语义），类配额按权重走整数域最大余额法零抽签分配（<code>apportion_tiers</code>，禁止任何浮点中间量；类内序数按 <code>tier_rank</code> 升序占连续区间 ⇒ <code>--limit</code> 前缀截断从每类最高档序数侧截起），构成语义为恰等——蓝图 Schema 的 <code>enum</code> 限档内子集给「<code>⊆</code>」、<code>plan_schema(cover_all=True)</code> 注入的 <code>allof</code> + 逐类 <code>contains</code> 给「<code>⊇</code>」（draft 2020-12 原生关键字，3.8.1），故档位身份可从 <code>members[]</code> 帧类集合直接反推对账；档位序数落三点—— <code>_meta.source.generator.tier_rank</code>（6.3）、工件 <code>truth.tier_rank</code>（键序重冻结：<code>sequence</code> 之后 <code>frame_class</code> 之前；噪音帧 <code>null</code>、重发帧承源，6.5）、<code>report.generate.stream.tiers.<rank></code>、<code>{planned, produced}</code>（键位在 <code>sequences</code> 之后 <code>frames</code> 之前，由编排器显式装配 ⇒ 零额档与全作废档如实在场，6.4）：M1 增档位约束簇（身份连续性 / 构成非空互异且 $\subseteq$ 帧类表 / 逐非零配额对的 <code>len_range</code> 下界 $\geq$ 档构成大小 / 分配零额与帧类未入档两条 WARN / 「每帧类生成指令必填」的检查域收窄为 U 各档构成，3.1.4、2.3.1）；② 时间字段回填——新绑定子表 <code>[frame.class.<name>.generate.time_fields]</code>（仅结构化帧合法，5.2）：把生成 Schema 内的时间语义字段绑定到闭集语义词表 <code>{ts, gap_prev_s, gap_next_s, elapsed_s}</code>，绑定即删除——被绑定字段从 LLM 面向的逐位 Schema 与逐位契约行中一并剔除（缩减 Schema 派生，层级拷贝不污染 M1 冻结产物），LLM 物理上无法生成它，值由零 LLM 零 <code>rng</code> 的机械回填尾声在 <code>ts</code> 铺设之后、直装组装之前按本序列相邻成员的 <code>ts</code> 差就地写入共享载荷（<code>round(·, 6)</code> 微秒精度，首末帧边界 0.0；重发帧与源帧引用同一对象故自动承源）；回填先于行对象与 id 计算 ⇒ 工件重放逐字节同 <code>id</code> 同会话（3.6.5）；钩子口径不变（<code>sample_validator</code> 与序列相似度过滤吃回填前载荷——时间量是机械量，不参与内容校验与内容判重）；③ 另附 v1.13 形态级缺陷修补一条——<code>frame_gap_s.lo $\geq$ 1e-6</code> 微秒地板（亚微秒 <code>lo</code> 会使 <code>timedelta</code> 取整为 0 微秒，「<code>ts</code> 严格递增」声明本就存在破口，词表的 0.0 边界哨兵更不容真零间隔；对 <code>lo $\geq$ 1e-6</code> 的一切现存工程零影响，2.6、3.1.4）；④ 零增量面（显式）——两机制零 <code>rng</code> 消费（冻结的抽签消费顺序表原文不动）、零调用数变化（<code>estimate_run</code> 与估算行格式零改动、八个 <code>dry-run golden</code> 字节不动、<code>console</code> 键集与面板零改动）、<code>TEMPLATE_HEAD_TOKENS</code> 零改动（M1 静态预检的蓝图段照旧按全帧类表计量，上界性质保持）、零新 trace 通道、零新事件、零新错误 kind（7.2/7.6；覆盖违约复用 <code>schema_violation</code> 进修复环、作废复用 <code>plan_failures</code>）。M8 另增 <code>_render_error</code> 的 <code>contains</code> 分支——覆盖违规点名缺失帧类（LO 关端点上修复提示必须可指导，3.8.4）。两开关全关（默认）时全系统与 v1.13 字节等价（例外仅微秒地板一条，属缺陷修补）；裁决详表见 <code>docs/dev/SPEC-generation-tiers.md</code> §2（问题验证与行业调研见 <code>docs/dev/PROPOSAL-generation-tiers.md</code>），示例工程 <code>examples/synth-stream</code> 就地扩展为两机制的可运行教学形态
v1.15	2026-08-19	按类档位表（对齐决策见 1.6；两路预实现审计 2026-08-19 折入，凡与提案不一致处以裁决为准）——v1.14 档位面的按类化增量，与其时间字段回填面完全正交（零触碰）：① 配置面——新增按类档位表 <code>[[class.<name>.generate.tiers]]</code>（行结构同全局表三键 <code>tier_rank / weight / frame_classes</code>，<code>[class.*.generate]</code> 白名单六键扩为七键，5.2）：表级原子覆盖（类声明了就用类的整张表，未声明回落全局 <code>[[generate.stream.tiers]]</code>，不逐行合并——行级合并会让 <code>rank</code> 身份跨表漂移）、全局表为锚（按类表要求全局表在场；档位面总开关恒 = 全局表非空 ⇒ 每个参与类恒有生效表，标识三点的在场性不逐行漂移，v1.14 的一切在场性判据零改动）、<code>tier_rank</code> 收窄为类内身份（每张生效表各自连续覆盖 1..N，N 逐类可不同；跨类同 <code>rank</code> 无任何工具语义、跨类同构成合法，行级消歧由同行的 <code>classification.label / truth.sequence_class</code> 天然承担）、空表拒收（<code>tiers = []</code> 为 <code>CONFIG_ERROR</code>，指引删键回落——面统一不存在「本类无档」态；「某类不想分档」的表达式 = 给它一张单档表）；② 取表点点点化——新增零 <code>rng</code> 纯函数 <code>effective_tiers</code>（类表，全局表）（落 <code>common</code> 层配置模型侧，M1 约束簇 / M6 计划期 / M10 报表装配三方共用），<code>apportion_tiers / tier_rank_for_ordinal</code> 本体与签名零改动，调用点改传该类生效表（3.6.5）；③ M1 侧——新增按类表前提三子款（形态门 / 全局锚 / 空表拒收，2.3.1、3.1.4），档位身份与构成的结构校验改逐生效来源表执行（构成两两互异的辖区收窄为单表之内），配额对约束与零额 WARN 逐参与类改吃本类生效表，「帧类未入档」WARN 与「每帧类生成指令必填」的检查域并集化为 U 各参与类生效表构成，零配额类声明的表结构校验不豁免；④ 观测面——<code>report.generate.stream.tiers</code> 改双形：仅全局表 ⇒ v1.14 平面形字节不变，任一按类表在场 ⇒ 类嵌套形 <code>{"<class>": {"<rank>": {planned, produced}}}</code>（外层全部声明类按声明序、内层该类生效表 <code>rank</code> 升序，键位仍冻结在 <code>sequences</code> 之后 <code>frames</code> 之前，仍由编排器按声明表显式装配 ⇒ 零额档与全作废档如实在场），计数器键族按类解冻重冻结为 <code>generate.stream.tiers.<class>.<rank>.{planned, produced}</code>（M6 恒喂类段键，平面形由编排器按 <code>rank</code> 跨类求和装配、数值与 v1.14 逐字节相等）；⑤ 零增量面（显式）——零 <code>rng</code> 消费（冻结的抽签消费顺序表原文不动）、零调用数变化（<code>estimate_run</code> 与估算行格式、八个 <code>dry-run golden</code>、<code>console</code> 键集与面板全不动）、<code>_meta.source.generator</code> 与工件 <code>truth</code> 的键、序、值语义零改动、零新 trace 通道零新事件零新错误 kind（新错误走既有 <code>CONFIG_ERROR</code> 面）。按类表全部缺省时全系统与 v1.14 逐字节等价（含报表）；裁决详表见 <code>docs/dev/SPEC-per-class-tiers.md</code> §2（问题验证与行业调研见 <code>docs/dev/PROPOSAL-per-class-tiers.md</code>），示例工程 <code>examples/synth-stream</code> 就地扩展为混合形态（一类独立表 / 一类回落全局）

v1.16	2026-08-20	时间流序列规则（以 docs/dev/SPEC-sequence-rules.md 的关闭裁决为准）：① 规则与日历配置——新增全局 <code>[[generate.stream.rules]]</code> / <code>[[generate.stream.windows]]</code>、按类同名表的独立三态整表覆盖，以及 <code>[generate] sequence_validator</code>；规则模板为 15 个标准有限迹 DECLARE 闭集，支持 occurrence、微妙量化的半开 <code>time_s</code> 与顶层同型字段相等 <code>correlation</code>，日历窗为同一自然日内半开 <code>of_day × of_week</code>；② 联合规划——精确引入 <code>ortools==9.15.6755</code>，由同一个单线程、自动搜索且固定确定性参数的 CP-SAT 模型在任何 LLM 调用前联合冻结逐序列帧类词、潜在 witness、会话归属、主任务时间戳与噪音槽；M1、<code>estimate_run</code> 与 M6 共用同一问题构造/求解入口，每个候选长度均须通过局部潜力校验，实际前缀以一次联合偏好求解选出完整可行长度向量，求解状态与模型规模上限按规格 fail closed；③ 生成与验证——约束路径以 <code>brief_schema(length)</code> 只让 LLM 生成逐位置 <code>brief</code>，帧类、时间轴与布局均由规划器给定；<code>correlation</code> 序列禁止反应式拆分，验证顺序冻结为逐帧 Schema → <code>sample_validator</code> → 声明式 <code>correlation/time</code> → <code>sequence_validator</code> → 序列相似度 → 幸存投影与噪音过滤 → 时间字段回填 → 重发深拷贝与位移 → 全局排序和组装；④ 观测面——<code>report.generate.stream</code> 按配置条件新增 <code>rules</code>、<code>sample_validator_scrapped</code>、<code>sequence_validator_scrapped</code> 与 <code>windows</code>，工件行形与 <code>truth/generator</code> 键序保持不变，零新 <code>trace</code> 通道、事件或错误 <code>kind</code>；⑤ 默认关闭锚与边界——没有生效 <code>rules/windows</code> 且未配置 <code>sequence_validator</code> 时走 v1.15 原路径并保持逐字节等价；规则只约束同一任务序列，<code>crossing</code> 仍是生成布局，帧仍是时间点事件，不生成缺帧或中断 <code>truth</code>。
-------	------------	---

目录

1. 概述
 - 1.1 背景与目标
 - 1.2 术语与缩写
 - 1.3 设计原则
 - 1.4 需求映射表
 - 1.5 算法与工程背书总表
 - 1.6 已对齐的设计决策
2. 总体设计
 - 2.1 系统定位与总体规格
 - 2.2 总体架构与模块清单
 - 2.3 端到端数据流与阶段开关
 - 2.4 CLI 规格
 - 2.5 配置体系总览
 - 2.6 非功能约束
3. 模块详细设计
 - 3.0 v1.16 时间流序列规则的跨模块落点
 - 3.1 M1 配置管理 config
 - 3.2 M2 数据接入 ingest
 - 3.3 M3 去重 dedup
 - 3.4 M4 质量打分 quality (QuRating)
 - 3.5 M5 标注 annotate
 - 3.6 M6 生成 generate
 - 3.7 M7 二次校验 verify
 - 3.8 M8 结构引擎 schema-engine
 - 3.9 M9 LLM 客户端 llm-client
 - 3.10 M10 编排器 orchestrator
 - 3.11 M11 输出器 emitter
 - 3.12 M12 日志 logging
 - 3.13 M13 分类 classify (v1.7)
 - 3.14 M14 segment——时序流语义分段
 - 3.15 M15 extract——转移/动作摘取
 - 3.16 M16 stitch——线索缝合
4. 核心数据结构与内部 API
 - 4.1 记录与信封
 - 4.2 阶段结果类型
 - 4.3 Stage 协议与异常层级
5. 配置文件完整规格
 - 5.1 config.toml (工具级静态配置)
 - 5.2 project.toml (工程级单次配置: 运行参数 + Rubric + 输出 Schema)
 - 5.3 Rubric 结构 (内联或默认包文件, 同一 TOML 结构)
6. 输入 / 输出格式规格
 - 6.1 输入: 文本模态
 - 6.2 输入: UI 模态
 - 6.3 主输出 JSONL
 - 6.4 report.json 结构
 - 6.5 时间流工件格式 (v1.13)
7. 日志系统与可观测性

7.1 日志体系总览

7.2 事件目录

7.3 记录格式规范

7.4 内容脱敏策略

7.5 rubric 优化闭环

7.6 错误分类码 (StageError.kind)

7.7 进度显示与结束摘要 (v1.10: 三态 console)

7.8 测试要求 (验收级)

8. 非目标、设计假设与演进路线

8.1 非目标

8.2 设计假设 (若不成立需回到设计层)

8.3 开放问题 (后续版本议题)

8.4 算子算法演进路线 (robustness & diversity)

9. 参考文献

附录 A. 系统默认 Rubric 全文

A.1 default:text

A.2 default:ui

A.3 default:trajectory (v1.8)

1. 概述

1.1 背景与目标

数据采集系统持续产出两类原始数据：**纯文本数据**（对话、指令、文档等）与**设备屏幕数据**（屏幕截图 + UI 控件树文件对）。这些数据在进入下游（模型训练、评测集构建、数据资产入库）之前，需要完成四类加工操作：**去重、质量打分、自动标注、（可选）数据生成与二次校验**。人工完成这些操作成本高、吞吐低、标准不一致。

LabelKit 是一个基于 LLM API 的**无状态批处理命令行工具**，目标是把上述加工操作固化为一条可配置的流水线：输入一批 JSONL 数据（或截图 + UI 树文件对），输出一批结构由用户定义、经代码规则引擎保证结构正确性的 JSONL 标注结果；v1.4 起另支持**纯生成模式**（`run.mode="generate_only"`）——无输入数据时从配置种子池或条件化提示从零合成数据集，产物照常经过全套治理与结构保证（3.6.2）。工具本身**不存储任何数据**：每一批数据的全部中间态只存在于进程内存中，运行结束即丢弃，落盘的只有显式声明的输出通道：用户输出文件、rejects 文件、不含数据内容的运行报告，显式启用时的 trace 追踪日志，以及 v1.13 时间流生成形态下的时间流工件（2.6、6.5、7.1；`rejects="full"` 与 `trace.content="full"` 档含数据内容，属用户显式选择并自担保留与清理责任）。

本文档是 LabelKit 的实现级设计规格，采用总分结构：第 2 章给出工具整体的规格、约束、架构与数据流；第 3 章逐模块给出职责、边界、输入输出、数据结构、API、算法流程与配置项；第 4—6 章给出跨模块的公共数据结构、配置文件与输入/输出格式的完整字段级定义；第 7 章定义日志系统与可观测性（含错误分类码规范，7.6）。所有功能点与算法均有顶会论文或工业级项目背书（见 1.5 节总表），不含凭空构思。

1.2 术语与缩写

术语	定义
记录 (Record)	流水线处理的最小数据单元。文本模式下为输入 JSONL 的一行；UI 模式下为一个「UI 树文件 + 截图文件」对。
批 (Batch)	一次流水线调度处理的记录集合，大小由 <code>run.batch_size</code> 决定；也是 QuRating 成对比较的采样池。
运行 (Run)	一次 <code>labelkit run</code> 进程的完整生命周期，处理一个输入路径的全部记录。
Rubric	质量评价准则集：若干条 criterion（准则），每条含 key、权重、描述、成对比较提示词与单点打分等级说明。
QuRating	Wettig et al., ICML 2024 提出的数据质量评估算法：LLM 成对比较 + Bradley–Terry 模型拟合标量质量分 [1]。
BT 模型	Bradley–Terry 配对比较概率模型 [2]： $P(i \text{ 胜 } j) = \theta_i / (\theta_i + \theta_j)$ 。
MinHash–LSH	基于最小哈希签名与局部敏感哈希的近似 Jaccard 相似检索，文本近似去重的方法学标准 [3]。
pHash	感知哈希 (perceptual hash)，对图像内容生成 64-bit 指纹，汉明距离度量视觉相似度（工业标准，imagehash 库实现）。
结构引擎	本工具中保证 LLM 输出符合用户 JSON Schema 的代码规则引擎 (M8)，含确定性修复与有界 LLM 修复环。
Profile	<code>config.toml</code> 中定义的一套 LLM API 接入参数 (provider/base_url/model/并发/重试等)，以名字被各阶段引用。
UI 树	设备屏幕的控件层级结构 (accessibility tree / view hierarchy) 导出文件，JSONL 格式，每行一个控件节点。
纯生成模式	<code>run.mode = "generate_only"</code> (v1.4)：无输入数据，M6 从配置种子池 (<code>generate.seed_examples</code>) 或无种子条件化提示从零产出样本，再走常规治理 / 标注管线 (3.6.2、3.10.3)。默认模式为 <code>"process"</code> (加工既有数据)。
Episode (序列记录 / 情节)	stream 模式的复合记录 (v1.8)：M14 把同一目标导向活动的成员帧按序键收拢为一条 <code>kind = "sequence"</code> 的序列 Record (成员经 <code>members</code> 元组引用共享持有)，作为一条普通记录走下游分类、打分、标注与评审 (3.14、4.1)。
会话 (Session)	摄取层按 <code>[stream]</code> 规则 (时间间隙 <code>gap</code> / 分区键 <code>key</code> / 长度与时长上限) 从有序记录流切出的候选窗口——流处理标准的 <code>session window</code> 原语的对应物 [55]；是 M14 语义精化的输入单元，切批改整会话装箱保证会话不跨批 (3.2.8、3.10.3)。
转移 (Transition)	序列内相邻两帧 <code><s_i, s_{i+1}></code> 之间发生的单个语义动作：M15 经 LLM 推断为结构化对象 { <code>action_type</code> , <code>target</code> , <code>value</code> , <code>description</code> } 写入 <code>item.transitions</code> ，转移数 = 成员数 - 1 (3.15、4.1)。
stream 模式	<code>segment.enabled = true</code> 的运行形态 (v1.8)：摄取按 <code>[stream]</code> 声明排序与会话化，链序为 <code>segment</code> → <code>stitch</code> → <code>dedup</code> → <code>classify</code> → <code>extract</code> → <code>quality</code> → <code>annotate</code> → <code>verify</code> (<code>stitch</code> 为 v1.9 增位，默认关)；默认关闭，关闭时数据产出与 v1.7 逐字段一致 (<code>_meta.stream: null</code> 除外) (2.3.1、3.10.3)。
线索 (Thread)	v1.9 stream 模式的顶层工作单元：M16 把同一目标导向任务被穿插切开的碎片保守缝合所得 (三级结构 <code>thread</code> > <code>fragment</code> > <code>step</code>)，承载体仍是一条 <code>kind = "sequence"</code> 的序列记录 (幸存信封 Record 重绑、id 不重算， <code>thread_id</code> = 幸存信封 <code>record.id</code>)，作为一条普通记录走下游判重、分类、打分、标注与评审 (3.16、4.1)。未被缝合的 episode 即单碎片线索； <code>stitch.enabled = false</code> 时线索与 episode 天然同值。
碎片 (Fragment)	线索的成员分段 (v1.9)：会话序上连续归属同一线索的成员帧区间——缝合前的一个 episode 或一个救援短段；以 <code>_meta.stream.fragments[]</code> { <code>order_span</code> , <code>member_count</code> , <code>cause</code> , <code>source_episode</code> } 溯源， <code>cause</code> ∈ "origin" "resumed" "rescued" (3.16、6.3)。

时间流工件 (Stream artifact)	v1.13 时间流生成形态的第二份产物： <code>{output_stem}.stream.jsonl</code> ，一行一帧、按交织序定稿——行内容 = 摄取侧输入格式（时间戳字段 + 文本字段）+ <code>truth</code> 真值对象（会话序数、序列类与类内序数、帧类、噪音标志、重发溯源）。行号即 <code>_meta.stream.member_sources[].line_no</code> ，与主输出双向可对账；配上同一份 <code>[stream]</code> 声明即可作 <code>process</code> 模式输入原样重放（M11 第五输出通道，3.6.5、3.11.2、6.5）。
蓝图 (Blueprint)	v1.13 时间流生成形态下一条序列的逐步计划。没有生效序列规则与日历窗时，LLM 一次调用产出 <code>L</code> 个步骤，每步给出所属帧类与一句话内容要点 <code>brief</code> ；v1.16 联合规划路径则由规划器预先冻结帧类词，LLM 只按 <code>brief_schema(length)</code> 产出逐位置 <code>brief</code> 。两条路径都在帧实现前冻结帧类真值，无需事后帧级判决（3.6.5、3.8.1）。
序列规则 (Sequence rule)	v1.16 时间流生成形态中约束同一 任务序列 有限迹的声明式规则。模板为 15 个标准 DECLARE 闭集，可带 <code>occurrence</code> 、半开秒区间 <code>time_s</code> 与顶层同型字段相等 <code>correlation</code> ；它不是跨业务序列的关系配置， <code>session crossing</code> 仍只属于生成布局（3.6.5、5.2）。
activation 与 witness	<code>activation</code> 是一条 DECLARE 规则中触发义务的位置； <code>witness</code> 是满足该义务的候选目标位置。同一目标可服务多个 <code>activation</code> 。联合规划器只冻结结构/时间上的潜在 <code>witness</code> ，帧实现后由运行期 <code>evaluator</code> 按 <code>correlation</code> → <code>time</code> 过滤并验证完整标准语义（3.6.5）。
日历窗 (Calendar window)	v1.16 对某个帧类的 每次 occurrence 施加的允许时间并集：同一自然日内的半开 <code>of_day</code> 时间段乘以 <code>of_week</code> 星期集合。逻辑跨午夜窗口不支持，但一个会话可跨自然日，只要每个 <code>occurrence</code> 各自在合法窗内（5.2）。
联合规划器 (Joint planner)	v1.16 的 CP-SAT 全流规划器：在任何 LLM 调用前，用同一个整数/布尔/automaton 模型联合冻结逐序列帧类词、潜在 <code>witness</code> 、会话归属、主任务时间戳与噪音槽。M1、 <code>estimate_run</code> 与 M6 共用同一问题构造和求解入口（2.2、3.1.4、3.6.5、3.10.3）。
序列校验回调 (Sequence validator)	v1.16 可选 <code>[generate] sequence_validator = "module:function"</code> ：在逐帧 Schema、 <code>sample_validator</code> 、声明式 <code>correlation/time</code> 之后，对一条已实现序列的只读深拷贝视图执行一次用户校验；返回规范化复用既有回调协议，异常按 <code>violation</code> 处理（3.6.5、4.1、5.2）。
接缝 (Seam)	多碎片线索中相邻两碎片的拼接处（v1.9）。判据：拼接对的会话序间隙含 ≥1 个归属其他线索的帧（间隙仅含噪声帧/本线索救援帧时不是接缝，该对照常摘取，3.16.4）；接缝步由 M15 零 LLM 机械占位（ <code>action_type="app_switch"</code> 、 <code>detail.kind="thread_seam"</code> 、步行内 <code>resumed=true</code> ，3.15.4），位置经 <code>seam_indexes</code> <code>duck</code> 标承载（左成员下标坐标，4.1）。

1.3 设计原则

原则	含义	来源 / 背书
无状态批处理	工具不持有跨运行状态，不落盘中间态；一次运行 = 读入 → 处理 → 写出 → 进程退出，内存即全部状态。	产品需求「工具不存储数据」；Unix 过滤器模型
算子化流水线	每个处理阶段是签名统一、可独立开关的算子（Stage），编排器只做组合与调度，不含业务逻辑。	Data-Juicer 算子体系（SIGMOD 2024）[4]；distilabel Step/DAG [5]；Dolma toolkit [6]
LLM 输出不可信	一切 LLM 输出必须经代码校验后才能进入下游：结构由 JSON Schema 校验，数值由解析器白名单校验。	结构化输出工业实践（OpenAI Structured Outputs、instructor 修复环）[7][8]
配置即契约	工具级配置（ <code>config.toml</code> ）与工程级配置（ <code>project.toml</code> ）在启动时全量校验、快速失败；运行期不再出现配置错误。	十二要素应用配置原则的文件化变体（按需求不使用环境变量，API Key 除外）
记录级隔离	单条记录的任何失败不影响其余记录：失败记录进入 <code>rejects</code> 通道并计入报告，运行继续。	Dolma / NeMo Curator 大规模管线的容错惯例 [6][9]
可复现	相同输入 + 相同配置 + 固定 <code>seed</code> + <code>temperature=0</code> 时，除 LLM 服务端非确定性外，配对采样、去重判定、流程路径完全可复现。	QuRating 开源实现的实验可复现要求 [1]

1.4 需求映射表

下表将原始需求逐条映射到本文档的承载章节，供评审时核对完整性。

原始需求	承载章节
使用 LLM 对采集数据进行自动标注 / 去重 / 打分 / 生成	M5（标注）、M6（生成）、M3（去重）、M4（打分）；总流程 2.3
输入数据为纯语言 / 设备截图+UI树 两种	M2 数据接入；输入格式规格 6.1—6.2
质量分类器使用 QuRating 算法；Rubric 用户提供 + 系统默认	M4；默认 Rubric 附录 A
LLM API 信息作为工具静态配置	M1、M9； <code>config.toml</code> 规格 5.1
工具不存储数据，批中间态标注完成后丢弃	2.1 / 2.6 非功能约束；M10 批生命周期
输出结构用户定义；LLM 输出 + 代码规则引擎保证结构正确	M8 结构引擎；输出格式 6.3
生成 / 二次校验可选，由 LLM 完成	M6 生成模式、M7 二次校验
工具配置 <code>config.toml</code> ；Rubric 与单次工程配置 <code>project.toml</code> ；不用环境变量（API Key 除外）	2.5 配置体系；5.1—5.2 完整规格

输出统一 JSONL；输入为 JSONL 路径；UI 模式为 uitree_<index>.jsonl + image_<index>.jpg/png 文件对（可不同子目录）	M2 配对算法；6.1–6.3
模块职责/功能/边界清晰	2.2 模块清单；第 3 章每模块「职责与边界」小节
功能点/算法需顶刊论文或工业项目背书	1.5 背书总表；各模块「背书」框
不清晰/多方案点与用户对齐	1.6 已对齐决策记录
（v1.1 评审补充）日志系统：行为记录/格式/打分思考支撑 rubric 优化与质量分析	M12（3.12）；第 7 章； <code>[trace]</code> 配置 5.2
（v1.2 评审补充）算子对输出集的影响分析；定量优选；多模型/多品味生成；算子算法增强	2.3.2；3.4.3 选择机制；3.6.2；8.4 演进路线总表
（v1.4 评审补充）无输入数据场景下直接生成数据（纯生成模式）	<code>run.mode</code> （5.2）；3.6.2 种子来源分支；3.10.3 纯生成行；2.3.1 组合④
（v1.7 评审补充）分类与按类条件化路由：加入分类算子，根据分类执行不同的打分、标注与生成；多类命中可流向多个管线（单/多分类开关锁定）	M13 分类（3.13）；按类条件化 3.4.3 / 3.5.2 / 3.6.2 / 3.7.2；multi 扇出 3.13.4 与契约 ②a（4.3）； <code>[classify]</code> / <code>[class.*]</code> 配置 5.2
（v1.8 评审补充）时序流分割与动作摘取：数据按时间排序输入时对流做语义分割（episode 形成与噪声帧剔除）并摘取流中的动作数据，标注算子为序列打「用户在做什么」的任务标签	M2 会话化（ <code>[stream]</code> ，3.2.8）+ M14 segment（3.14）+ M15 extract（3.15）+ 下游序列适配（M3/M13/M4/M5/M7，3.3.3 / 3.13.3 / 3.4.3 / 3.5.2 / 3.7.2）；轨迹 rubric 附录 A.3；契约 ②b（4.3）
（v1.9 评审补充）活动结构：x 小时自然使用流标注出「n 帧组成的 m 个工作单元」——同一任务被穿插切开时缝合为一个线索（串联/单交叉/多交叉/噪声四类标定），短段业务尾帧可救援，噪声帧过滤不入线索	M16 stitch 线索缝合（3.16，链序 segment 之后、dedup 之前）+ 下游线索适配（M3/M4/M5/M7/M15，3.3.3 / 3.4.3 / 3.5.2 / 3.7.2 / 3.15.4）；三级结构 thread > fragment > step 与接缝占位（3.16.4、3.15.4）；契约 ②c（4.3）； <code>[stitch]</code> 配置 5.2
（v1.10 评审补充）Console 实时面板（TUI）：交互终端下以双区内联面板实时呈现批进度、流水线段棋盘、状态账与 LLM 用量/密钥池/熔断（工业调研裁决品类：批处理工具不做全屏 TUI）；非 TTY / CI / jsonl 下保持 v1.9 行为（三层回归锚 7.8）	7.7 三态 console 规格； <code>[console]</code> 配置 5.1 与 <code>--console</code> （2.4）；ProgressListener 五回调旁路（3.12.3）与 snapshot（3.9.2）；依赖面增 rich（2.6）；裁决 U1–U27 见 docs/dev/SPEC–tui-console.md §2
（v1.13 评审补充）时间流生成：无输入数据时从零合成带时间戳的多会话请求流——一条序列一次蓝图 + 一次帧实现，多条序列机械交织出会话、交叉、噪声与重发；产物既是可直接标注的序列，也是可原样重放的时间流数据集	M6 时间流形态（3.6.5）+ <code>[generate.stream]</code> 配置 5.2 + M1 组合约束 3.1.4；时间流工件通道 3.11.2 与格式 6.5；按序列类标注 Schema 3.4/3.5.2/3.11.2 与 M8 待遇参数 3.8.2；判决形序列评审 3.7.5；估算与观测 3.10.3、6.4；裁决详表见 docs/dev/SPEC–stream-generation.md §2
（v1.16 评审补充）时间流序列规则：用户可对逐类序列声明标准有限迹控制流、occurrence 时间关系、payload correlation 与帧类日历窗，并在 LLM 后追加一次序列回调；全流排程必须在首个 LLM 调用前联合证明和冻结	<code>[generate.stream].rules/windows</code> 、 <code>[class.*.generate].rules/windows</code> 与 <code>[generate].sequence_validator</code> （5.2）；M1 每候选长度/全前缀 fail-closed 校验（3.1.4）；联合 CP–SAT 规划与运行期 evaluator（3.6.5）； <code>brief_schema</code> 与 Schema 引擎（3.8.1）；共享估算（3.10.3）；报告（6.4）；裁决详表见 docs/dev/SPEC–sequence-rules.md

1.5 算法与工程背书总表

功能点	采用方案	背书（论文 / 工业项目）
质量打分（主模式）	LLM 成对比较 + Bradley–Terry 拟合标量分	QuRating, ICML 2024, arXiv:2402.09739 [1]; BT 模型 [2]; MM 拟合算法 Hunter 2004 [10]
质量打分（低成本模式）	单点加性 rubric 打分（0–5 逐条累加）	FineWeb–Edu, NeurIPS 2024 D&B, arXiv:2406.17557 [11]
精确去重	规范化内容 SHA–256 哈希	Dolma toolkit 去重设计 [6]; 业界通行做法
近似去重	MinHash–LSH（字符 n–gram shingle, Jaccard 阈值）	Lee et al., ACL 2022, arXiv:2107.06499 [3]; 内置于 Dolma [6]、Data–Juicer [4]、NeMo Curator [9]
图像去重	pHash 感知哈希 + 汉明距离阈值	imagehash（工业标准库）；数据集治理通行做法 [9]
LLM 自动标注	提示词组装 + 结构化输出 + 多模态（截图+序列化UI树）	distilabel（Argilla，工业）[5]；Autolabel（Refuel，工业）[12]；GUI 数据 LLM 标注管线：ScreenAI [13]、GUI–360 [14]、AgentTrek, ICLR 2025 [15]
标注自一致（可选，v1.2）	self-consistency：同一记录 n 次独立采样（n≥3 奇数， <code>annotate.sc_temperature</code> ）+ 字段级多数投票，全体分歧回退首样本并计数	Self–Consistency, Wang et al., ICLR 2023, arXiv:2203.12171 [33]
UI 树 + 截图输入表示	截图图像 + accessibility–tree 线性化文本同时输入 VLM	ScreenAI screen–schema 线性化 [13]；OS–Atlas [16]；Ferret–UI [17]

数据生成（可选）	以种子记录为例的自举生成 + 相似度过滤	Self-Instruct, ACL 2023, arXiv:2212.10560 [18]; Evol-Instruct / WizardLM, ICLR 2024 [19]; distilabel 任务库 [5]
多样性生成（可选, v1.2）	多 LLM 混合（round_robin 轮转 / weighted 加权抽样） + <code>[[generate.styles]]</code> 风格模板条件化提示	Persona Hub, arXiv:2406.20094 [34]; Cosmopedia (HuggingFace, 工业) [35]; model collapse 缓解: Shumailov et al., Nature 631, 2024 [36]; distilabel 任务级 LLM 绑定 [5]
二次校验（可选）	LLM-as-a-Judge 独立评审 + 有界修复回路	Zheng et al., NeurIPS 2023, arXiv:2306.05685 [20]; Self-Refine, NeurIPS 2023 [21]; Constitutional AI 批评-修订 [22]
结构正确性保证	JSON Schema (draft 2020-12) 校验 + 确定性修复 + 有界 LLM 修复环 + 供应商原生结构化输出	OpenAI Structured Outputs (工业) [7]; Outlines 约束解码 [23]; JSONSchemaBench [24]; instructor / json-repair (工业) [8]
流水线架构	统一签名算子 + 声明式配置组合	Data-Juicer, SIGMOD 2024 [4]; distilabel DAG [5]; Dolma toolkit [6]
评审偏差缓解	成对比较随机顺序、判分与理由分离、平局处理	LLM-as-a-Judge 位置偏差/冗长偏差分析 [20]; QuRating 提示设计 [1]
API 调用容错	指数退避 + 抖动重试、并发信号量限流	AWS/Google SRE 重试规范（工业标准）; distilabel/NeMo Curator 客户端实现 [5][9]
LLM 调用追踪与结构化事件日志	双通道日志: stderr 运行日志 + trace JSONL 事件流（一行一事件、通道过滤、四档脱敏）; LLM 调用事件字段命名对齐 OTel GenAI 语义约定（仅命名对齐, 非实现依赖）	OpenTelemetry GenAI 语义约定（Development 状态, 非 stable）[27]; LangSmith (LangChain, 工业) [28]; W&B Weave (工业) [29]
评审驱动的 rubric 迭代	trace 记录逐次 pairwise 裁决与理由 → 人工审阅与指标诊断 → 修订准则 → 小样本重跑对比（7.5 闭环）	EvalGen 的 criteria drift 结论, UIST 2024, arXiv:2404.12272 [30]; CritiQ 从偏好挖掘质量准则, ACL 2025, arXiv:2502.19279 [31]
纯生成模式（无输入合成）	配置种子池自举（单遍）/ 无种子 instruction×style 条件化 + 显式量目标	Self-Instruct 以 175 条人工种子自举 [18]（种子池形态）; Persona Hub [34]、Cosmopedia [35]（无种子条件化形态）
评审鲁棒性增强	多评审团多数票（奇数个异构评审 per-criterion 投票）+ 双顺序裁决（正反两序一致才记胜）	PolL (Verga et al., 2024), arXiv:2404.18796 [32]; LLM-as-a-Judge 位置偏差分析, Zheng et al., NeurIPS 2023 [20]
数据分类（可选, v1.7）	LLM 封闭集分类: 类别表词表经内部 Schema enum 硬校验 + 可选 self-consistency 投票 + 兜底类	Autolabel classification / multilabel 任务的 labels 词表校验（工业）[12]; InsTag LLM 指令语义打标, ICLR 2024 [38]; NeMo Curator 分类器管线阶段（工业）[40]; Self-Consistency [33]
按类条件化路由（v1.7）	分类结果落记录级属性（ <code>item.classification</code> / <code>_meta.classification</code> ），下游算子按类取有效配置，管线拓扑不变	Nemotron-CC 质量分档路由由不同合成管线, arXiv:2412.02595 [37]; NeMo Curator <code>bucketed_results</code> 标签字段路由（工业）[40]; Dolma <code>tagger→attributes→mixer</code> 解耦 [6]
按类数据构造（v1.7）	按类种子池 + 按类生成指令/风格（ <code>[class.<name>.generate]</code> ），配按类 rubric 与标注指令	Tulu 3 按核心技能分治的数据构造与 per-skill 合成, arXiv:2411.15124 [39]; Nemotron-CC 每档不同改写 prompt [37]; 类内配套沿用 Persona Hub [34] / Cosmopedia [35]
多标签扇出（可选, v1.7）	<code>classify.assignment = "multi"</code> : 命中 k 类扇出 k 个单标签兄弟信封，各自独立走按类管线并各产出一行	InsTag 指令多意图的多标签打标形态 [38]; distilabel 路由函数的一批多下游并行产出（ <code>sample_n_steps</code> ）[5]; Autolabel multilabel 的「标签集合 $\subseteq$ 词表」输出契约 [12]
时序流会话化（v1.8）	<code>[stream]</code> 声明排序键 + gap/分区键/长度时长上限切候选会话，整会话装箱保证 episode 不跨批（3.2.8、3.10.3）	Apache Flink <code>EventTimeSessionWindows</code> / Apache Beam <code>Sessions</code> （工业标准）[55]——取用: session window 原语的规则层照抄（inactivity gap + 分区键 + 硬上限），纯代码零 LLM 成本
轨迹数据工程整体形态（v1.8）	转移摘取 → 任务标注 → 轨迹打分的三段式管线（extract → annotate → quality）	OS-Genesis, ACL 2025 [41]——取用: reverse task synthesis 三段式（转移标注 → 任务聚合 → trajectory reward model 打分）直接映射为本管线三工位；其 TRM 为 1–5 五级，附录 A.3 的 0–5 六级为家规改制
动作摘取范式（v1.8）	LLM zero-shot 充当运行时逆动力学模型（IDM）: 一次调用喂前后两帧，利用非因果优势推断其间动作（3.15）	VPT, NeurIPS 2022 [42]——取用:「从相邻状态推断动作」是被大规模验证的独立工序; GUI-Shift, ICLR 2026 [42] 为 IDM 范式在 GUI 域的最新自监督形态
确定性归并 + LLM 语义化分工（v1.8）	控件树 diff 代码则确定性计算作 extract 证据; 提示词锚定「动作前最后稳定帧 / 动作后首个稳定帧、多下层事件归并为单个语义动作」	OpenCUA / AgentNet [43]——取用: Action Reduction（低层事件确定性归并）与 State-Action Matching（稳定帧锚定、防未来信息泄漏）两层分工移植入 extract 模板（CONTRACTS §10.10）
流后分段再打标（v1.8）	先有记录流、事后自动识别子序列并打标（hindsight relabeling 的自动化）	AITW, NeurIPS 2023 D&B [44]——取用:「先有流、后分段再打标」是 GUI 轨迹数据工程的标准姿势，本设计为其 LLM 自动化版本
episode/step 两级结构与动作词表（v1.8）	episode 级任务标签（用户 Schema）+ step 级动作（ <code>_meta.stream.steps</code> ）; 转移数 = 成员数 - 1	AndroidControl, NeurIPS 2024 D&B [45]——取用: 两级标注结构直接采用; 8 值动作词表全集采纳（无裁剪）+ other 兜底,「动作数 = 截图数 - 1」即 extract 调用量公式
跨 App episode 一等公民（v1.8）	边界判据以「可见任务实体延续」而非换 App 判段（advances 关系值）	GUI-Odyssey, ICCV 2025 [46]——取用: 跨 App 导航流全集皆是、为此专门引入 RECENT 应用切换动作, 佐证 <code>app_switch</code> 入词表与「实体延续即同任务」判据

语义边界裁决模板 (v1.8)	三步演绎：双向上下文概括 → 五值封闭集关系分类 → 演绎查表映射边界/噪声 (LLM 不直接答边界, 3.14)	话题分割谱系 TextTiling → Embed-KCPD → Def-DTS [47]——取用：Def-DTS 消融证明半结构（仅双向概括）比裸问题差、完整三步最优，而边界信号清晰场景裸判决可胜全套（S32）；其按数据集改意图池的先例背书本设计的 5 关系词表按域定制
无词表事件边界 (v1.8)	边界判据内置、任务无关：粒度锚定「完整任务」层级 + 注意力锚定前台 App/窗口，用户零 prompt 可用	GEBD, ICCV 2021 [48]——取用：taxonomy-free 边界任务可定义、可标注、中等共识可达（5 人多评协议数据），"1 level deeper" 与 dominant subject 两原则写死在判据模板
描述后置 (v1.8)	「用户在做什么」由 annotate 在分段之后产出，任何人无需先验描述边界	Hindsight Instruction Pairing, RSS 2021 [49]——取用：先有无结构行为流、事后配指令的范式源头，回答「边界判据不需要任务描述」的需求方疑虑
UI 日志分割问题定义 (v1.8)	分段 = 边界发现 + 噪声剔除（无监督、无任务描述）；interruption → noise 词表值	RPA UI 日志分割：Marrella et al.; Leno et al. [50]——取用：「显式处理不属于任何例程的噪声事件」的问题定义直接沿用（noise_residue 准则同源）；交错例程难变体 v1 不做（8.4）
缺帧不补全 (v1.8)	verify 缺帧三级判定的「无处可寻」档仅标 capture_gap，不做补全	Repairing Event Logs, Rogge-Solti et al. [51]——取用：缺失事件修复依赖跨轨迹习得的过程模型先验，佐证缺帧补全列演进候选（8.4）而非 v1 内置
定位与描述分立 (v1.8)	segment（时序定位/分段）与 annotate（描述/打标）拆成两个工序	Vid2Seq, CVPR 2023 [52]——取用：dense video captioning 的「先 temporal localization 再 captioning」两段式先例
宁滥勿缺 + 后段精化 (v1.8)	批内不删元素、噪声帧只改状态；verify 复裁可回收（软排除而非硬删，②b）	BSN, ECCV 2018；Soft-NMS, ICCV 2017 [53]——取用：时序候选「宁滥勿缺 + 后段精化/软排除」范式对应「只改状态不删元素 + 成员回收」的谱系定位
边界余量证据 (v1.8)	verify 评审证据含段边界外前后 k=2 帧的摘要及其去向（【边界余量】段，3.7.2）	语音端点检测 hangover 惯例（ITU-T G.729 Annex B VAD / WebRTC VAD）[54]——取用：防切头切尾的工业标准手法移植为评审证据段，零额外 LLM 调用
多图请求上限 (v1.8)	annotate.sequence_frames ∈ [2,100]、默认 20；>20 联动 max_image_px > 2000 警告（3.1.4）	Anthropic Vision API 文档 [56]——取用：100 图/请求、>20 图单图任一边 >2000px 为 400 硬拒（非缩放）、32MB/请求；OpenAI 1500 图/512MB 故不设独立上限
整段单调用对照形态 (v1.8)	v1 保留 hybrid 滑窗——window ≥ 会话长时天然退化为整段单调用，长会话建议调大 window	GUIDE [57]——取用：GUI 域验证最充分的 LLM 分段形态是纯文本动作序列整段一次调用（99.4% 段可用率、50–80 步无衰减）；其整体式 judge 随轨迹变长退化的数据（>20 步降信任）入手册调优指引
extract 可靠性预算 (v1.8)	风险面明写每步 zero-shot 错误率 20–30% 的级联；缓解 = 树 diff 证据 + verify 缺陷路由 + quality 结构分 + extract.by_type 分布可观测	Watch & Learn, CVPR 2026 [58]——取用：zero-shot MLLM 动作标注 70.5% vs 专训 IDM 91.7% 的直接对照钉死可靠性预算；「噪声标注主动伤害下游」佐证 fail-closed 质量门
diff 注入可消融 (v1.8)	extract.include_diff 开关（默认开，可关做 A/B 对比）	Sharingan [59]——取用：像素 diff 显式注入实测负结果、按动作类型精度极不均衡（click 0.94 / drag 0.40）——结构化树 diff ≠ 像素 diff 且工程实践正面，但方向未定 ⇒ 做成开关而非硬编码正收益
屏幕流→轨迹的 2026 工业路线 (v1.8)	prompted LLM 滑窗仍是量产形态之一（v1 采用）；专训边界模型列本地化演进	VideoAgentTrek, ICLR 2026；Video2GUI [60]——取用：专训 7B 边界模型与 prompted Gemini 滑窗两条路线并存（滑窗未被取代）；两者均「动作先于/伴随分段」，extract–先行次序据此列演进候选（8.4）
轨迹判分信度护栏 (v1.8)	机械锚点（extract 副读数注入裁决 prompt）+ stream 默认只打分不筛 + 分数按 episode 长度可观测	Web-Shepherd, NeurIPS 2025；GUI-Shepherd；AgentRewardBench [61]——取用：zero-shot LLM 轨迹判分高方差、无单一模型通吃，检查清单分解是保命组件——机械锚点与 checklist 思想同构
统一动作空间对齐 (v1.8)	action_type 枚举 11 值 = AndroidControl 全集 ∪ UI-TARS-mobile 增量（drag、app_switch）+ other 兜底	UI-TARS（工业）；UIPro, ICCV 2025 [62]——取用：2025–2026 统一移动动作空间共识含 drag 与应用切换/recent，跨 App episode 场景频率不可忽略
树可靠性护栏 (v1.8)	帧摘要贫瘠护栏：可见文本节点为零或摘要趋零 ⇒ 计 digest_poor_frames + WARN + 手册指引为 segment.llm 配置 supports_vision=true 的 profile（v1.11/V4 改写——use_vision 键已移除，窗口附图由能力推导 vision_resolved 决定，3.14；WARN 逐字为 poor frame digest (zero visible text nodes): text-only boundary verdicts lack evidence; attach frame screenshots by pointing segment.llm at a supports_vision=true profile）	Do GUI Agents Believe Their Eyes?（引 CLAY ghost-node 统计）[63]——取用：Android 10.6% 结构节点无视觉呈现、37.4% 屏幕含 ghost node——树贫瘠不是长尾，须主动可观测而非被动抽读
线索缝合问题形态 (v1.9)	屏幕流 = 「多线索穿插 + 噪声」的形式化；缝合以线索（thread）为产出单元、错缝以乘法惩罚度量	PIRA-Bench [64]——取用：任务子轨迹 = 非连续帧子集的问题定义；噪声消融证实过连接偏差跨模型家族共享（precision 92→51 而 recall 反升，原文 "trigger-happy"）；S_final = F1 × FPS_norm 与负样本协议进真机门禁（3.16.7）。注：0 被引新基准，自报数字权重打折

保守偏置合取 (v1.9)	并入需 LLM 判 resume ^ 机械先验命中 (App 交集 / 实体重叠 / 返回同一页面, 析取三腿 + 超期降格); 口头置信度不入门槛	过连接量化: IdentifyMe / CORRECT-DETECT [76]——取用: LLM 回避「以上皆非」宁可硬连、准确与弃权此消彼长;「返回同一页面」腿机理 = cue-guided resumption (Altmann & Trafton / Trafton et al.) [80]; 置信度饱和 [79]
单调选池判定 (v1.9)	每候选一次调用: 池内开放线索摘要卡 (最近活跃降序) + 候选卡 → thread_ref \ new, 顺序贪心	GreedyDisentangle (Takada & Mori, LaCATODA@AAAI-26) [87]——取用: 同任务同形制 SOTA 先例 (簇摘要呈现 + LLM 归簇或判 new + 顺序贪心, 全指标超 per-pair); 对话解缠谱系 (贪心链接标准解码、并发线程 ≤3 占 46.4%、VI / 1-1 overlap / link-F1) [75]; 呈现序与位置扰动测试防位置偏差 [77]
有界二遍复评 (v1.9)	一遍结束后对单碎片线索逐个复评 (池 = 其他线索活视图), 修正顺序贪心漏缝; 预算 ≤ 单碎片线索数	FAMER / Gruenheid et al. [74]——取用: 无修复贪心劣于 batch 且次序依赖、n=1 局部重聚类即追平 batch; merge-only 是增量法中质量最差; subsequent-context 有效 [87]; 更重簇修复机器 (LLM-CER / GraphCR / Alper) 已评估按规模不采 [78]
池容量与时间衰减 (v1.9)	max_open = 4 (挂起窗口均值 3 + 1 活跃); stale_gap_steps 双职: 先验降格 + 池满逐出优先腿; 封闭 ≠ 终结	lqbal & Horvitz, CHI 2007 [81]——取用: 真实桌面日志挂起窗口均值 3 (S.D.≈2)、27% 挂起 >2h 才恢复; 时长分布特征 +11.36% (CASAS) [66]; 移动端仅 22.6% 任务穿插佐证宽松上界 [90]; working spheres 与手机中断/回访人因基线 [82]
短段救援 (v1.9)	below_min_len 短段按连续 run 重组先进候选池: 命中并入 + 帧翻转, 未命中维持 dropped_noise, 永不开新线索	lqbal & Horvitz [81]——取用: 切换前收尾动作密集 (段落完成率 0.78/min → 切换前 10.9-12.8/min) ——任务收尾帧天然易成短段、聚集在切换点旁的文献级机理
摘要卡证据面 (v1.9)	结构化摘要卡 (App 集合 · 任务名 · 首末帧摘要 · 变更提示 · 跨度) 替代全量帧历史; 判定对 = 线索尾帧摘要 × 候选首帧摘要	resumption 判定单元 = 「挂起尾 × 恢复首」对 (CIGAR) [65]——取用: 帧摘要级降格承载 (3.16.3); window title 最强单特征 (85.57%) 与多窗聚合 (SWISH / TaskPredictor) [83]; 精选紧凑上下文反超全量原始历史 (+10.4 pt、token 少 8x) 与 summarization drift 风险命名 (Engram / Memori / 综述) [88]; 两级任务组匹配工业近例 Log2Plan [84]
缝合稳定性 votes (v1.9)	单模型 n 采样、(verdict, thread_ref) 完整判定严格多数决 (默认 votes=1 不启用); 不采多模型评审团	Self-Consistency [33]——取用: 一致率是可靠的不确定性信号 (votes 是置信度门槛的正规替代 [79]); 边界: 高自一致处过度自信、votes 治方差不治偏差 + 评审团有效独立票仅 ≈2 [89]; 跨模型共识修不了共享偏差 (within-model 0.68 > cross-family 0.47) [86]——对照 PoLL [32] 评审团路线不采 (8.3 O8)
三级层级与身份链 (v1.9)	thread > fragment > step; 帧单一归属——交叉用「平面分段 + 线索身份」表达, 不引入帧多重归属/区间树	Ego4D Goal-Step [69]——取用: goal>step>substep 三级 + is_continued 续接标志的直接先例; GUI 域层级标配 AndroidControl [45]; 视频域层级范式 (FineGym / Breakfast) [71]; 帧多标签先例 (MultiTHUMOS / Charades) [72] 评估后否决采纳; 嵌套与区间关系形式语义底座 (HHMM / Allen) [73]
线索命名后置 (v1.9)	task_name 由池空判定自举、滚动更新 (工具内部结构, 进 trace 与判定证据; 用户任务标签仍由 annotate 产出)	OS-Genesis [41] / NNetNav [70]——取用: 自然流 → 事后反推任务标注范式;「可命名性」剪枝判据
问题域现状与护栏 (v1.9)	interleaved 解缠无域内基线: 护栏 = 保守合取 + 二遍复评 + 负样本协议 + 真机门禁 (错缝 FPS = 0 验收线)	Robotic Process Mining [67]——取用: UI 日志 interleaved 解缠 = open challenge、全局法依赖「例程重复」前提 (2025 复核不变 [85]); 学术解缠系列与三家产品均无穿插解缠 + 通信类 App 天然噪声名单 [68]; 跨 App 单目标轨迹形态 [46]
上下文预算: 窗口声明与预算公式 (v1.11)	[llm.<name>] / [embedding.<name>] 用户声明 context_window (0 = 未声明 = 该 profile 预算关闭); input_budget = context_window - max_output_tokens - margin, margin = max(256, ceil(0.10 × context_window)) (3.9)	LlamaIndex PromptHelper [91]——取用: context_window - prompt - num_output 预算式与 repack 装填; Claude Code auto-compact [92]——取用: contextWindow - min(maxOut, 20k) - 13k 的「预留输出 + 固定 buffer」结构 (各来源触发百分比不一、结构一致); OpenAI Codex CLI [93]——取用: model_context_window 用户声明 + 钳制 + 输出预留 + 比例边距的完整同构
条数上限 + 预算动态装填 (v1.11)	条数型参数 (segment.window / annotate.sequence_frames / generate.seeds_per_call) 降级为上限值, 按逐项实际 est 贪心装填; 调用次数以静态最坏值 w_min 报上界 (3.9、3.14、V9/V12)	Qwen-VL 官方评测口径 [94]——取用: 帧数上限 × 总 token 预算双约束、max_pixels = 预算 // 帧数 (帧数是上限、预算守恒); NeMo Curator Nemotron-CC DocumentJoiner [95]——取用: 按 max_segment_tokens 的 token 装填 ("maximize input utilization") 是数据管线同类算子
图片成本测量-反应式三层 (v1.11)	先验装填 (provider 公式仅作首批先验) → 溢出裁帧保清重试 → usage.prompt_tokens 在线校准 (窗口化 max 滤波 + 0.85 安全系数、批冻结快照) (3.9、V17-V20)	ABR / 拥塞控制 measure-don't-model 范式 [96]——取用: BBA 纯缓冲选档 (稳态不需容量估计、启动期必须要) 与 BBR windowed-max 测量式建模取代丢包反应——校准器滤波蓝本; Cline [97]——取用: 生产级上下文管理刻意反应式 ("accurate token counting varies by model/tokenizer") + 企业网关 usage: null 实证 (缺样本兜底的依据)
判审触发裁帧升清重试 (v1.11)	verify fail ^ policy="repair" 的修复重标注换档: 关键帧减半 + 分辨率上探一档 (≤ max_image_px), 单向有界 (3.5.2、3.7.3、V21)	置信度触发递归变焦谱系 [98]——取用: Zoom Eye 置信驱动图像树递归变焦 (训练无关)、V*/SEAL 置信度低于阈值即递归切 patch 搜索 (7B+搜索 75.4% vs GPT-4V 55.0%)、UI-Zoomer 置信门控变焦 (GUI grounding +4.2-13.4%) ——「低置信 → 定向升清重试」的学术与 GUI 域背书

时间流生成： 蓝图 → 帧实现 两阶段（v1.13）	一序列一次 蓝图 （定步数、逐步帧类与一句话要点）+ 一序列一次 帧实现 （按蓝图逐位产出帧内容， <code>prefixItems</code> 逐位契约）；帧类真值在蓝图层即确定，落盘无需再标注（3.6.5）	APIGen-MT, NeurIPS 2025 [99]——取用：「先产出并校验蓝图、再互演实现」的两阶段合成骨架直接映射为本形态两类调用；Plan-and-Write / M2M / Schema-Guided Dialogue [100]——取用：静态计划先行的收益对照，以及「先有结构真值再自然语言化 ⇒ 零再标注」的真值保留范式
时间流的结构 维度机械化 （v1.13）	会话装箱、单交叉、噪音插入、原样重发、时间戳铺设全部由 零 LLM 的机械交织器 按冻结抽签顺序完成（3.6.5）——LLM 只负责内容，结构由代码决定	PLG2 / Simod [101]——取用：过程日志生成器把「噪音逐类概率 + 按时间归并交织」与到达间隔分布族做成机械参数的成熟形态；LongMemEval / MT-Eval [102]——取用：干扰注入的位置参数化（注入位置是可控实验变量而非模型自由发挥）
噪音在计划层 注入、真值链接保留（v1.13）	两类噪音——插入型噪音帧（ <code>noise_ratio</code> ， <code>truth.noise = true</code> 三 <code>null</code> ）与原样重发序列（ <code>duplicates</code> ， <code>truth.duplicate_of</code> 指回原序列类内序号）；乱序与缺失两类不做（3.6.5、6.5）	过程挖掘真值方法 2025 [103]——取用：噪音须在计划层注入才能保留真值链接（事后加噪会切断标签溯源）；PLG2 四类噪音词表 [101]——本形态按可用性裁剪为插入与重复两类（乱序与 M2 <code>on_disorder</code> 自相矛盾、缺失对合成无意义）
合成序列的过 滤与评审 （v1.13）	交织前的 序列级 相似度过滤（成员文本按序 <code>\x1e</code> 拼接的 <code>episode</code> 配方，比对面 = 兄弟序列）+ 判决形 LLM 评审拒绝采样（fail ⇒ <code>dropped_verify</code> ，淘汰不改真值，3.7.5）	AlpaGasus / Nemotron-4 [104]——取用：「过滤优于全量」与「判定生成分离」两条合成数据纪律；MT-Bench [20]——LLM 评审即过滤器；Lee et al. [3] / SemDeDup [26]——序列级近重删除沿用既有判重配方
档位即属性条 件化（v1.14）	<code>[[generate.stream.tiers]]</code> 把「帧类构成」做成显式的档位属性，蓝图调用按档条件化并把档位序号随产物落盘（ <code>generator.tier_rank</code> / <code>truth.tier_rank</code> / <code>report</code> 三点）——档位标签是可下游消费的一等真值，而非仅仅生成期的一次性旋钮（3.6.5、6.3、6.5）。 v1.15 按类化 ：档位表可按序列类 整表覆盖 （ <code>[[class.<name>.generate.tiers]]</code> ，未声明回落全局表）， <code>tier_rank</code> 相应收窄为 类内身份 ——「哪些帧类构成算一档」本就是逐类的领域判断，跨类拉齐无意义（5.2、3.6.5）	SteerLM, EMNLP 2023 Findings [105]——取用：显式多维属性条件化生成 + 属性值随样本保留的形态（属性是可控输入而非隐式偏好）；HelpSteer2, NeurIPS 2024 D&B [105]——取用：分档属性标注的下游价值（偏好对构造、课程学习）佐证「档位随产物落盘而非用完即弃」；Nemotron-CC [37] / Cosmopedia [35]——质量分档与分桶配比贯穿生成与观测的既有引用。 v1.15 按类先例 ：Gretel Data Designer / NVIDIA NeMo Data Designer 的 <code>subcategory</code> 逐父值子配置与 <code>conditional_params</code> 整套替换 （工业）、GLAN 的分类树逐节点独立结构与配额、Schema-Guided Dialogue 的每服务一份 <code>schema</code> ([100], v1.13 已引) ——取用：条件化重资产的粒度是 整套替换 而非逐行合并；配额面另取不成比例分层抽样（逐层独立配额、多路分层零格合法）为「按类各自分档、跨类不可比」的统计学依据；逐条引用清单见 <code>docs/dev/SPEC-per-class-tiers.md §7</code>
构成恰等的双 向硬约束 （v1.14）	「不出档外类」由蓝图内部 Schema 的 <code>enum</code> 限档内子集给出，「档内每类至少一次」由 <code>allOf</code> + 逐类 <code>contains</code> 给出，合成「步帧类集合 ≡ 档声明构成」；违约进 M8 既有修复环，零新失败机制（3.8.1）	JSON Schema draft 2020-12 <code>core/validation</code> [109]——取用： <code>contains</code> / <code>allOf</code> / <code>const</code> 为原生关键字（ <code>jsonschema</code> 4.21+ L2 直接可校验），覆盖语义无需自建校验层；Autolabel 闭集纪律 [12]——取用：词表 <code>enum</code> 硬校验的既有形态，本行只是把「 <code>⊆</code> 」补上对偶的「 <code>⊇</code> 」；Schema-Guided Dialogue [100]——取用：构成声明在计划层、实现层零再标注
时间字段归时 钟所有（v1.14）	<code>[[frame.class.<name>.generate.time_fields]]</code> 把生成 Schema 内的时间语义字段绑定到时间轴：绑定即从 LLM 面剔除，值由机械回填尾声按已铺 <code>ts</code> 差算出——「时间量由引擎盖章、内容量由 LLM 产出」的分工（3.6.5）	数据驱动业务流程仿真 Camargo et al. / Simod [101] 与 BIMP/QBP 仿真引擎 [106]——取用：到达间隔与活动历时由仿真引擎按分布采样并盖章、模型侧从不自报时间量，是过程仿真领域的定式；LogGenerator [107]——取用：离散事件引擎为合成日志盖章全部时间字段的工业实现形态；AT-KDE, Process Science 2026 [108]——取用：到达时间建模是仿真侧的长期演进面（ <code>frame_gap_s</code> 的分布形扩展据此列演进候选而非本版内置）
有限迹序列规 则（v1.16）	15 个标准 DECLARE 模板直接作用于每条任务序列的帧类词； <code>activation</code> 缺席时遵守 <code>vacuity</code> ，正向规则要求存在 <code>witness</code> ，负向规则禁止命中候选；运行期按冻结顺序重放 <code>evaluator</code> （3.6.5）	De Giacomo 与 Vardi 的 LTLf/LDLf 有限迹语义 [110]；Declare4Py 模板文档 [111]——取用标准模板名和有限迹 <code>conformance</code> 语义；MP-Declare [112]——取用 <code>activation</code> 、 <code>correlation</code> 、 <code>time</code> 三类条件分工，不引入其完整 DSL
控制流、时间 与日历联合规 划（v1.16）	单个 CP-SAT 模型联合约束帧类位置、潜在 <code>witness</code> 、微秒时间、日历析取、会话装箱、真交叉与噪音槽；规则通过布尔约束与 <code>AddAutomaton</code> 施加到同一组位置变量，不构造显式乘积 DFA（3.6.5）	OR-Tools CP-SAT 与 <code>AddAutomaton</code> [115][116]；Temporal Constraint Networks [113] 与 Disjunctive Temporal Problems [114]——取用整数差分约束及日历析取的组合性；生产依赖精确锁定 <code>ortools==9.15.6755</code> [117]
payload correlation 与 序列回调 （v1.16）	声明式 <code>correlation</code> 仅支持两个结构化帧顶层必填同型字段的类型敏感相等；逐帧静态回调和声明式规则均通过后，再把 <code>JSON-compatible</code> 深拷贝的完整序列交给可选用户回调一次（3.6.5、4.1）	MP-Declare [112]——取用 <code>data correlation</code> 与 <code>activation/time</code> 的正交分工；既有 v1.5 用户校验回调协议沿用同一规范化与异常隔离纪律，不引入新的回调框架

1.6 已对齐的设计决策

以下多方案设计点已与需求方沟通对齐（对齐日期见各行，早期各轮为 2026-07-02），本文档按对齐结论展开：

设计点	候选方案	对齐结论
QuRating 实现形态	仅 pairwise+BT / 仅 pointwise / 双模式	双模式可配：默认 pairwise+Bradley-Terry（忠实 QuRating [1]），提供 pointwise 加性打分（FineWeb-Edu [11]）作为低成本模式， <code>project.toml</code> 一键切换，共用同一套 <code>rubric</code> 。

去重层级	仅精确 / 精确+MinHash / 三级含语义去重	精确 + MinHash+LSH (纯本地零 API 成本), 图像走 pHash; 语义级重复交由质量打分环节间接处理。SemDeDup 列为开放问题 (8.3)。v1.2 更新: 用户决策推翻本结论, SemDeDup 落地为可选第④级 (默认关, 3.3.3); 决策溯源见 8.3 O1。
形态与语言	Python CLI / Python 库+CLI / 其他语言	Python 3.11+ 单一 CLI 工具, 与 distilabel/Data-Juicer/Dolma/NeMo Curator 同栈 [4][5][6][9]。
输出结构描述格式	JSON Schema / TOML 简化 DSL / 两者	标准 JSON Schema (draft 2020-12), 内嵌于 project.toml 或引用外部 .json 文件; LLM 侧直接作为结构化输出约束, 规则引擎侧用 jsonschema 库校验, 零转换层 [7][23][24]。
定量优选 (v1.2 对齐, 2026-07-02)	流式批内 top_ratio / 全局两阶段精确 top-K / 双支持	仅批内 top_ratio: <code>quality.selection = "top_ratio"</code> (<code>quality.top_ratio</code> $\in$ (0,1], 与 <code>threshold</code> 互斥, M1 校验, 3.4.3) 提供流式近似定量; 全局精确定量列入演进路线 O6 (8.3)。
生成补齐回路 (v1.2 对齐, 2026-07-02)	本版实现 / 列入演进路线	列入演进路线 O6 (8.3): 设计草案已给出补齐环与三重停止条件 (含本轮合格率下限——防 model collapse [36]), 与全局定量一并立项。
多 LLM / 多品味生成 (v1.2 对齐, 2026-07-02)	单 LLM (v1.1 现状) / 多 profile 混合 + 风格模板	进规格: <code>generate.llms</code> 数组 (取代 v1.1 单值键 <code>generate.llm</code>) + <code>generate.mixture</code> ("round_robin" "weighted", <code>weighted</code> 配 <code>generate.weights</code>) + <code>[[generate.styles]]</code> 风格模板 (name、prompt), 规格见 3.6.2; 多样性思想背书 [34][35]。
算子算法增强 (v1.2 对齐, 2026-07-02)	① 多评审团投票 ② 双顺序裁决 ③ self-consistency 标注 ④ SemDeDup 语义去重	①②③④ 全部收录为默认关闭的可选配置 (总表见 8.4; 背书 [32][20][33][26]); 其中 ④ 修订 v1.0 「语义去重不做」的对齐结论 (见上文去重层级行尾注), 经 <code>[embedding.<name>]</code> profile 落地为 dedup 可选第④级。
模块拆分与重编号 (v1.3 对齐, 2026-07-02)	保持 M5 复合模块 / 拆分且编号稳定 (生成 = M12) / 拆分且按流水线位置全量重编号	拆分且全量重编号: 标注与生成职责正交 (基数保持的增列 vs 基数增加的合成), 按 2.2 模块边界准则应各自独立; 生成独立为 M6 (3.6), 原 M6—M11 顺移为 M7—M12。配置键、数据结构与 API 零变更, 属纯文档结构调整。
纯生成模式 (v1.4 对齐, 2026-07-02)	支持两种形态 / 仅种子池 / 演进路线 / 明确非目标	支持, 两种形态进规格: 配置种子池 <code>seed_examples</code> (Self-Instruct 形态 [18]) 与无种子条件化 + <code>standalone_count</code> (Persona Hub / Cosmopedia 形态 [34][35]), 单通执行不引入 O6 循环; 工具定位由「数据加工器」扩展为「亦可从零起步的数据生产者」(1.1、2.1 同步修订)。
多 API Key 负载均衡 (v1.6 对齐, 2026-07-03)	① 范围: 仅同 profile 多 key / 端点镜像池; ② 熔断中止时 .part 交付与否; ③ 全池冷却驻留超限的处置: 直接硬熔断 / 记录失败累积; ④ 配额型 403 的归类: 密钥禁用 / 冷却 / 错误误嗅探; ⑤ 报表中密钥身份: 环境变量名 / 位置别名	① 仅同 profile 多 key 、 单 endpoint (密钥池, 3.9.3) ——端点镜像池明确排除: 同模型不同部署在 <code>temperature=0</code> 下仍有数值漂移, 会翻转 <code>pairwise</code> 裁决与语义去重边界判定、污染 7.5 同种子翻转率指标 (决策溯源见 8.3 O7); ② 熔断中止交付已完成批 (熔断交付, 3.10.3、3.11.2、6.4); ③ 驻留超限 (<code>run.max_park_s</code> , 默认 3600s) 按重试耗尽记录失败并计入熔断窗口, 不直接硬熔断; ④ 配额以 403 形态出现按认证禁用该密钥处理, 不做 <code>provider</code> 特定的错误误嗅探; ⑤ 报表 / <code>trace</code> 以 环境变量名 标识密钥 (密钥值任何情况不落盘, 7.4)。
分类算子与按类条件化 (v1.7 对齐, 2026-07-07)	① 模块编号: 追加 M13 / 按流水线位置全量重编号; ② fallback 语义: 普通类成员且必填 / 隐式 <code>_unclassified</code> 特殊类; ③ <code>generate_only</code> 按类配比: 本版做 / 单独立项; ④ 白名单是否放开 <code>per-class</code> <code>quality.llm</code> / <code>annotate.llm</code> ; ⑤ 纯打标模式: 显式开关 / 零覆盖自然退化; ⑥ 多标签中间档 (仅打标不扇出): 本版加 / 留扩展位; ⑦ dry-run multi 估算口径: 乘数 1 下界 / <code>max_labels</code> 上界; ⑧ <code>enabled=false</code> 而类配置在场: <code>CONFIG_ERROR</code> / <code>warning</code> ; ⑨ 手册新章编号: 追加制 / 链序插入全书重排	① 追加 M13 (3.13, 纯新增零重排成本, v1.3 重编号先例限于模块拆分); ② <code>classify.fallback_class</code> 为 普通类成员 、 enabled 时必填 (可配 <code>per-class</code> 参数, 5.2); ③ 不做 <code>generate_only</code> 按类配比—— <code>generate_only</code> 用全局指令、产物回流被分类后按类打分/标注 (3.6.2), 按类量目标与 8.3 O6 一并立项; ④ v1 不放开 (LLM 绑定属部署与成本面, 白名单后续只增, 5.2); ⑤ 不加开关 ——不配任何 <code>[class.*]</code> 覆盖即自然退化为纯打标; ⑥ 暂不加 , <code>assignment</code> 枚举留扩展位 (8.4 演进候选); ⑦ 按标签乘数 1 报下界 + <code>stderr</code> 注明 (诚实不虚高, 3.10.3 估算行); ⑧ warning (一次、点名被忽略的表——偏离提案的 <code>CONFIG_ERROR</code> , 对齐 <code>top_ratio</code> 未生效等 no-op 键分级惯例, 3.1.4); ⑨ 追加制 <code>docs/manual/24-classify.md</code> 。另记评审裁判三则: 内部 Schema 不写 <code>uniqueItems</code> (OpenAI strict 模式与部分约束解码网关硬拒该关键字, 重复标签由 <code>classify</code> 代码在 M8 验证后确定性归一化, 3.13.3); fallback 留痕 不写 <code>item.errors</code> (rejects 归因取 <code>errors[0]</code> , 写入会在记录后续失败时污染归因——放改 <code>Classification.detail</code> + <code>error</code> 事件 + 计数器, 3.13.4); 防呆分级由提案的 <code>CONFIG_ERROR</code> 改 warning (即 ⑧)。

时序流语义分割与动作摘取 (v1.8 对齐, 2026-07-13): 提案 (docs/dev/PROPOSAL-stream-segmentation.md) §7 十四项开放决策点全部按默认裁决通过 (①追加 M14/M15; ② `[stream]` 独立节; ③噪声帧进 `rejects`; ④交错 `episode` 不做; ⑤`generate` × `stream` 互斥; ⑥序列 `dedup` ①②④级 + 跳③; ⑦`extract` 文本模态不做; ⑧超长会话硬切 + `WARN`; ⑨ `default:trajectory` 内置; ⑩`steps` 恒在; ⑪粒度旋钮不做; ⑫修复范围 = 标签重标 + 成员收缩 + 噪声池回收、跨段只标记; ⑬流式单调性校验 + `on_disorder`; ⑭`stream` $\Rightarrow$ `annotate` 必开)。在此之上, 七域 fan-out 可行性审查 (78 条发现、0 blocker) 与两路深检索 (refute: 0 条论点被推翻; elevate: 29 项外部事实钉死) 的发现收敛为三十二项设计裁决 S1—S32, 凡与提案原文不一致处以裁决为准, 详表见 docs/dev/SPEC-stream-segmentation.md §2。逐条择要:

- S1 trace 通道枚举 8→10: 增 "segment"、"extract" 两值 (通道 = stage 名), 事件名维持 segment.* / extract.*, error 事件按 stage 自动归属 (7.2)。
- S2 ClassView 增第 6 必填字段 extract, [class.<name>.extract] 白名单承诺兑现 (5.2)。
- S3 契约 ②b 补 M7 修复路径授权: 可在 absorbed ↔ dropped_noise 间双向改写成员信封状态 (回收/收缩), 禁止翻回 active (4.3)。
- S4 PipelineItem 增字段 session_id: M10 装箱时对口信封盖章, 会话边界获得批内载体 (4.1)。
- S5 build_annotate_prompt / annotate_record 增装配变体 transitions (None = 现行为, 3.5.2; 2026-08-14 起该取值是 AnnotatePromptOptions 的字段)。
- S6 序列标注模板不变量: 未 part 恒为恒在的 [成员帧摘要] text——防 repair 拼接吞末帧图 (3.5.2)。
- S7 stream 评审内部 Schema 三键全 required (critiques / defects / verdict), 可选键改可空联合 (OpenAI strict 兼容); VerificationResult 增 additive 字段 defects (3.7.2、4.1)。
- S8 成员手术两阶段批级结构: 并发评审 → 同步按批位置序执行手术 → 并发接缝重摘取/重标注——并发调度不引入额外不确定性 (3.7.3)。
- S9 extract × multi 扇出按 label 各摘 (接受 xk; 白名单承诺兑现, dry-run 报下界 + stderr 注明, 3.15)。
- S10 dedup 序列分支: 成员单条配方按序拼接 (分隔符 ASCII RS)、③pHash 自动跳过、语义层增序列 case (3.3.3)。
- S11 min_len 仅作用于 LLM 精化切出的段; 短段帧 reason = "below_min_len" (≠ "noise"), 计数独立 (3.14)。
- S12 帧摘要 = best-effort 确定性提取 (app/activity/title/salient 均自 UI 树可达面), 配摘要贫瘠护栏 (digest_poor_frames + WARN, 3.14)。
- S13 树 diff 用结构键多重集匹配 (role, bounds//quantize, depth)——node_id 非跨帧身份, 不得作匹配键 (4.2)。
- S14 extract 可靠性预算写入 §1.5 与风险面 (每步 zero-shot 错误率 20–30% 的级联 [58][59]); extract.include_diff 开关 (默认开、可 A/B); report 增按动作类型分布 extract.by_type (6.4)。
- S15 action_type 枚举 11 值 = AndroidControl 全集 U UI-TARS-mobile 增量 (drag、app_switch) + other 兜底 [45][62] (3.15)。
- S16 extract.on_error = "fallback" | "fail"; fallback 步与 LLM 确证的 other 在 quality 副读数中分列 (3.15)。
- S17 --limit 保持帧级截断; 截断视同 EOF 冲洗尾会话 + WARN 一次 (2.4)。
- S18 stream 模式 counts.unprocessed 出现条件扩为「熔断 ∨ 中断»; 守恒式两侧同步扩展 (6.4)。
- S19 单调性游标按分区键各自维护 (groupby 语义、键变即断、输入须按键成组); UI 模态增分区键来源 "source_dir" (3.2.8)。
- S20 时间戳解析规格: 数值 <1e11 判秒、[1e11, 1e14) 判毫秒、界外解析失败; 字符串先试数值再试 fromisoformat; 失败与乱序同走 stream.on_disorder (6.1)。
- S21 整会话装箱用 next-fit (顺序装箱、仅一只开口箱); 单会话超 batch_size 硬切 + WARN + session_split 标 (3.10.3)。
- S22 dry-run 估算公式修正: segment_calls = $\sum \text{ceil}((L-1)/(\text{window}-1))$; extract_calls = $\sum (L-1)$ 报上界; quality/annotate/verify 以 episodes ≈ sessions 报下界 (3.10.3)。
- S23 文本模态 dry-run 单遍融合: 一次读同时产出行数与会话空跑结果 (3.2.8)。
- S24 序列 Record 的 ref 继承首成员 line_no (文本) / pair_index (UI); 完整成员溯源由 _meta.stream.member_sources 承担 (4.1)。
- S25 rejects full 档序列载荷 = {"kind": "sequence", "member_ids": [...], "member_sources": [...]} (3.11.2)。
- S26 segment.on_error = "keep" 留痕三件套 (_meta.stream.degraded + error 事件 + 计数器), 不写 item.errors (防归因污染, 3.14)。
- S27 trace 脱敏: 新 _DATA_KEYS = {"target", "value"} none/refs 档剥除; "description" 入自由文本键集 (7.4)。
- S28 $2 \leq \text{annotate.sequence_frames} \leq 100$; >20 且引用 profile max_image_px > 2000 ⇒ WARN (Anthropic many-image 硬拒 [56]); 降采样纯整数公式、首末帧恒含 (3.5.2)。
- S29 stream 模式下 quality.rubric == "" 解析为 "default:trajectory" (两模态一致; 显式选择器恒优先; rubric 文本模态中立, 附录 A.3)。
- S30 profile 引用集四处: segment.llm 仅 strategy ∈ {llm, hybrid} 时计入; extract.llm 恒入且恒入 vision_users; stream 模式下 quality 的 supports_vision 强制校验放宽 (序列打分纯文本, 3.1.4)。
- S31 verify 收缩弃帧 rejects 行 stage = "verify"、reason = "off_task_member"; 计数器 membership_repairs / boundary_flags / defects.<kind> 入 report.stream.verify; transitions 手术后重编号 + reseamed 溯源标 (3.7.3、6.4)。
- S32 v1 保留 hybrid 滑窗 (window ≥ 会话长时天然退化为整段单调用, GUIDE 证据 [57] 建议长会话调大 window); 判据模板明文「相关但无实体延续的新流程 = context_switch (边界)」与「会话首帧恒为段首»; GEBD 措辞降级为「中等共识可达」[48]; 「extract 先行 + 动作序列上分段」次序列演进候选 (8.4), 以成本权衡论证。

活动结构——线索缝合与层级工作单元 (v1.9 对齐, 2026-07-15/16): 需求原型为「x 小时自然使用手机的时序流标注出 n 帧组成的 m 个工作单元」, 四类规范验收场景 (串联/单交叉/多交叉/噪声) 由需求方 2026-07-16 给定; 真机 E2E (2026-07-15) 证实三处结构性失效——交叉目标被切碎互不关联、短段吞业务末帧、extract/verify 编造连续性且可自洽过审。设计草案经三轮独立验证修订 (功能完整性审计 2 blocker + 9 major + 10 minor、两轮 deep-search refute/elevate 五机制无一被驳倒、定稿五路复核 2 blocker + 6 major + 5 minor——全部裁决并入), 收敛为二十二项设计裁决 T1–T22 (编号与 v1.8 的 S1–S32 区隔), 凡与草案不一致处以裁决为准, 详表见 docs/dev/SPEC-activity-structure.md §2。逐条择要:

- T1/T2/T3 交叉的表达模型：**线索身份缝合**——episode 平面互斥分区不动，M16 合并同线索碎片为线索信封、`_meta.stream.fragments` 保留碎片结构 (Goal-Step `is_continued` 同型 [69])；**帧多重归属否决** (单前台屏无真并发 [65]，帧单一 absorbed 是手术/归因/守恒公共地基，[72] 引用记录被拒方案)；层级取三级 `thread > fragment > step`、不做帧级区间树 (3.16)。
- T4 episode 内子任务跨度：**不做引擎特性** (需求方 2026-07-16 裁决) ——标注层模式 (用户 Schema `subtasks: [{label, step_range}]`) + 手册指引；下游无消费方。
- T5 算子形态与链序：新算子 M16，`_CHAIN_ORDER` 九名单一超集元组 `segment → stitch → dedup → classify → extract → quality → generate → annotate → verify` (缝合改成成员集 → 先于 dedup/extract)；默认 off，off 时主输出/rejects/report.json 与 v1.8 逐字节等价 (回归锚；例外两处见 3.16.4 退化锚——dry-run stderr 行与缺陷词表 wrong_stitch 行)。
- T6/T7 契约与守恒：Stage 契约增 ②c 例外 (被并碎片壳置 `stitched`、幸存信封 Record 重绑 id 不重算、below_min_len 来源帧 dropped_noise→absorbed 翻转，含幸存者规范句，4.3)；Status 增 `stitched` (壳仅计被并 episode 信封；救援无壳)，守恒全式 / failed 兜底 / unprocessed 残差三处同步扩 `stitched` 项，`counts.threads` 以恒等式 `threads = episodes - stitched` 由 M10 post-emit tally 导出 (6.4、3.10.3)。
- T8/T9 判定形态与保守偏置：**单调选池** (每候选一调：池内线索摘要卡按最近活跃降序 [77] + 候选卡 → `thread_ref | new`；证据面全部为帧摘要级——链序上 extract 后置，运行时无 Transition，[65] 判定单元降格为首末帧摘要对承载)；池容量 `max_open = 4` (挂起窗口均值为 3 + 1 活跃 [81]，移动域佐证 [90])；封闭仅发生于池满逐出 (stale-gap 优先 → LRU 兜底；完成感知腿撤除——无动作生产者，8.1 ⑥)、**封闭 ≠ 终结** (保留二遍目标集与产出 [64][81][66])；`bias="conservative"` 默认——并入需 LLM 判 resume ∧ 机械先验析取三腿 (App 交集 / 实体重叠 / 返回同一页面 [80]) 命中、超 `stale_gap_steps` 降格须两腿；**去除 confidence 门槛腿** (口头置信度饱和 [79]，字段仅 trace 观测)。
- T10/T20 接缝：**零 LLM 机械占位四键钉死** (`action_type="app_switch"`、`detail={kind:"thread_seam", interrupted_by:[...]}`、步行内 `resumed=true`，3.15.4)；接缝判据 = 拼接对话序列间隙含 ≥1 异线索帧 (间隙仅噪声/本线索救援帧不是接缝、照常摘取——与 v1.8 「剔噪对照摘取」单一处理)；`seam_indexes` 左成员下标坐标、与 `Transition.index` 同键空间；seam 占位不计入 `extract.transitions / by_type` (接缝唯一计量点 = `stream.stitch.seams`，6.4)。
- T11 短段救援：`below_min_len` 帧 (duck 标判别) 按连续 run 重组为救援候选先进池 (`segment.py` 零改动)；命中并入 + ②c③ 翻转计 `rescued_short` (单位 = 帧)、未命中维持 `dropped_noise`，**永不开新线索**；噪声帧 (reason="noise") 不入候选池 (3.16.4)。
- T12/T13 作用域与判重：不跨 session、不跨 batch，hard-split 边界不可缝；segment 降格 episode 照常入池；dedup 判重单元 = 线索 (重绑成员配方按序拼接，S10 机制原样)，`stitched` 壳被 `status=="active"` 过滤天然排除——dedup 代码零改动 (3.3.3)。
- T14/T15 下游适配：`quality/verify` 步行渲染对 `detail.kind=="thread_seam"` 加专用后缀 (防 trajectory rubric 把接缝当噪声残留扣分，3.4.3)；annotate 关键帧降采样升级为**按碎片配额** (每碎片保底 1 帧，3.5.2)；verify 序列 prompt 六段 → 七段 (新增 [片段结构] 节——无此节 wrong_stitch 不可判)、缺陷词表 5→6 (+ `wrong_stitch`，路由 mark-only + fail 独立分支)、`_session_episodes` 过滤 `stitched` 壳 (3.7.2/3.7.3)。
- T16/T17 观测与配置：trace 通道 10→11 (`stitch`) + 两事件 `stitch.judge / stitch.thread`、`task_name` 入脱敏自由文本集 (7.2/7.4)；`report.stream.stitch` 子块与 `batch.end` 增 `stitched/threads`、dry-run 增 `stitch_calls` 估算行 (off 恒 0 且无条件打印，3.10.3)；`_meta.stream` 增 `thread_id / fragments / steps` 行内 `resumed` (条件在场：全部 v1.9 新键仅启用时出现——off 逐字节等价的充分条件，6.3/6.4)；新节 [stitch] 11 键，M1 约束 `stitch⇒segment`、votes 偶数 = 配置错误、`stitch^rules` WARN (3.1.4、5.2)。
- T18 缝合稳定性 votes：**机制立项、默认 votes = 1 不启用** (需求方 2026-07-16 「不允许 defer」指令裁决)；>1 时 n 次采样对 (**verdict, thread_ref**) **完整判定严格多数决** (分裂一律回落保守结局)；路线选型采「单模型多次」(self-consistency [33]) 而非「多模型评审团」(PoLL [32]) ——votes 治方差 (漂移) 不治偏差 (过连接) [89]，过连接是跨家族共享偏差 [64]、评审团会把它投成多数 [86][89]；`stitch.judges` 多模型扩展列 8.3 O8。
- T19 有界二遍复评：复评候选 = 一遍结束时的单碎片线索，池 = 会话内全部其他线索活视图 (超 `max_open` 按与候选跨度最近截取)；命中方向相反——候选作壳、目标线索幸存；预算 ≤ 单碎片线索数 ([74] n=1 重聚类追平 batch；merge-only 最差 [74]；后见上下文有效 [87]；更重机器不采 [78])。
- T21/T22 输出与身份链：M11 增第四路由 (`stitched` 仅计数，壳不得落 rejects 兜底；`--strict` 补注：开 stitch 后 strict 结果可能 1→0 属预期，3.11.2/2.4)；`steps[].index` 全线索连续 0..n-2，`episode_id` = 幸存信封 `record.id` = `thread_id`、碎片原 `episode_id` 落 `fragments[].source_episode` (3.16.4)。

Console 实时面板 (v1.10 对齐，2026-07-17)：需求为「deep-search 各种工业项目，为 LabelKit 设计一个 TUI 方案」；跨四品类约 20 个工业项目的调研 (buck2 superconsole / Docker BuildKit / bazel / cargo·uv·pip / Nextflow·Snakemake / k9s·htop / tqdm / Bespoke Curator·distilabel / Claude Code·Codex CLI，引用 [C-1]–[C-20] 见 docs/dev/PROPOSAL-tui-console.md；审计增量 [C-21]–[C-42] 见 SPEC §6) 经一轮定稿 (需求方 2026-07-17: spec-only；U4 批 rich；U18 批 T16 有界修订；U14 心跳默认关；U15 键盘交互一期实施) 与三路独立审计二轮定稿 (代码可行性/亲和性 2B+5M+9m、文档清单 2B+6M+8m、deep-search refute/elevate 1 推翻 + 4 修订 + 6 成立——需求方 2026-07-17 第二指令随即实施、不允许 defer)，收敛为二十七项设计裁决 U1–U27，详表见 docs/dev/SPEC-tui-console.md §2。核心裁决择要：品类 = 双区内联面板、永不进 alternate screen (U1，批任务保 scrollbar)；面板 = M12 第四纯消费面，`ProgressListener` 五回调进程内旁路 (U19——on_run_context/on_estimate 补齐渲染器数据通路) 不产生 TraceEvent、不入 7.2 目录 (U2/U11——7.2 只增不改原则零触碰)，on_event 载荷 none 档预脱敏 (U22——U6 信息纪律由机制保证)、sink 侧转发异常自吞 (U23)；段棋盘分子 = llm.call 括号归属运行级累计、分母 = `estimate_run` 运行级估算 (U20——llm.call 事件无阶段归属的口径修复)；emitter 让位经 M1 解析产物 `mode_resolved` 静态门 + plain 行格式纯函数 `console_format` 下沉 common (U21)；回归锚三层化 (U24——实践逐字节 diff 被 refute 审计证伪：行携时间戳、温度 0

端点非确定)；NO_COLOR 不降 plain (U25——no-color.org 语义 = 禁色非禁布局，rich 原生承担)；渲染 tick = asyncio task、Live(auto_refresh=false) 钉死 (U26——rich 默认自刷新线程与事件循环内动态字典跨线程争用)；validate 通路 validate_project(..., overrides=) 修复 (U27)；渲染异常自吞降级 plain、永不影响退出码与产出 (U7 红线)；三态 auto|rich|plain，jsonl 强制 plain 不可覆盖 (U5)；批总数分母 UI 模态恒显示 (live 预扫复用、禁二次 scan)、文本模态默认不做估算扫描 (U17，console.estimate 显式换购)。

上下文预算与视觉能力自动推导 (v1.11 对齐，2026-07-22)：对每次 LLM 调用建立不变式 `est(输入 prompt) + max_output_tokens + margin ≤ context_window` —— `[llm.<name>] / [embedding.<name>]` 增 `context_window` 声明 (0 = 未声明 = 该 profile 预算关闭，行为与 v1.10 逐字节一致)；零依赖启发式估算器 + 动态装填 (条数型参数降级为上限值、按实际内容逐项装填，调用次数以静态最坏值 `w_min` 保证上界——`estimate/console` 分母共用，V9/V12)；图片成本测量-反应式三层 (默认采样装填 `default_image_px` → 溢出裁帧保清重试 → 判审低置信裁帧升清重试，`usage.prompt_tokens` 在线校准、批冻结快照——provider 文档公式降级为首批先验，V17-V21)；M9 咽喉终检与记录级 `context_overflow / output_truncated` (三形态熔断矩阵：precheck 不计连击、reactive-400 终局由属主算子补喂恰一次、reactive-200 不补喂，V16/V24/A7，7.6)；同时删除 `segment.use_vision`，改为能力推导 parse product `vision_resolved` (选 profile 即选能力；存量显式键定向 `CONFIG_ERROR`，V1-V5)。裁决 A1-A5 按推荐执行、A6 被 V17-V21 取代 (测量-反应式为需求方自提)、A7 反应态终局计入连击；决策 V1-V27 详表见 [docs/dev/SPEC-context-budget.md §2](#) (业界调研与审计引用 [C-1]-[C-84] 见 [docs/dev/PROPOSAL-context-budget.md](#))。

流模式帧级分类与标注 (v1.12 对齐，2026-08-12)：需求原型为「用户需要在流模式的一次运行中同时获得原子帧级的闭集分类 + 按类标注与序列级的意图标注——此前须对同一份数据跑两遍流水线 (帧级一遍 + 流模式一遍) 再在外部脚本合并」。设计经三方预实现审计 (代码可行性/亲和性、修改清单穷尽、对抗性反证，2026-08-12) 折入定稿，裁决详表见 [docs/dev/SPEC-frame-annotation.md §2](#)，凡与提案不一致处以裁决为准；本特性裁决以自然语言命名，不新造字母编号系列。逐条择要：

- 裁决·承载形态——帧产物 = `PipelineItem` 两个新 dict 字段 (按成员 `record.id` 键控)，成员帧保持 `absorbed`；成员帧状态机、链序、Stage 契约 ②a/②b/②c、守恒恒等式零改动 (4.1、4.3)。
- 裁决·成员失败不入 `rejects` (推翻提案) ——帧标注不可修复 ⇒ `members[]` 条目 `status:"failed" + annotation:null + report.stream.frame_annotate.failed` 计数；不写 `rejects` 行、不触发 `--strict` (emitter 四路由互斥是结构承诺、`rejects` 行键集是有序闭集，3.11.2、6.4)。
- 裁决·帧 Schema 显式路由——帧标注调用 `complete_validated(..., schema=frame_schema)`：L0-L3 四层全在、无 L2.5、不计 `resolved_at` (保住 6.4 恒等式)；`ResolvedConfig` 新增解析产物 `frame_schema` (`user_schema` 同胞：M1 元校验 + few-shot 干跑)；emitter 写前 `validate_only(obj, schema=frame_schema)` 兜底，非法帧对象永不落盘 (3.8.2、3.11.2)。
- 裁决·装箱器下沉——`segment.pack_windows` 纯函数下沉为 `budget.pack_windows`，`segment` 改 import、行为字节等价 (既有装箱测试守住)；帧级批量判决以零重叠调用形复用之 (3.13.7)。
- 裁决·修复面第四向——verify 回收成员懒加载补跑帧产物：算子间导入白名单 3→4 (新增 `classify.classify_frames` 公开直调面，v1.8 `segment.judge_window` 同款先例)；`annotate.annotate_member` 并入既有 `annotate` 修复面族；补跑幂等只补缺位、收缩成员从两 dict 删键 (3.7.3)。
- 裁决·扇出共享与首标签执行——`classify._fan_out` 克隆构造清单显式加入两字段 (按引用共享，与 `record / dedup` 同族)；帧级两 pass 只在首标签信封上执行 (克隆判据 = `classification.label != classification.labels[0]`，verify S8 同款)；手术后原/克隆行 `members` 分叉由既有 `repaired` 位消歧 (6.3)。
- 裁决·meta_mode 护栏——`frame.*` 任一启用 ⇒ `output.meta_mode != "none"` (`CONFIG_ERROR`，文案指明帧产物仅经 `_meta` 承载；sidecar 合法，3.1.4)。
- 裁决·降格会话跳过——`segment_degraded` duck 标在场的 episode 跳过两个帧 pass (`label=null`、`status="skipped"`、`frame_classify.skipped_degraded` 计数)：降格 = 噪声未剔，对垃圾帧付费反直觉 (3.13.7、3.5.5)。
- 裁决·摘要行回填砍掉 (推翻提案倾向) ——v1.12 不做「帧标签回填成员摘要行」：审计确认真实爆炸半径 = `quality/annotate/verify/extract` 四处渲染点 + `CONTRACTS §10` 多个逐字节冻结模板，收益不成比例；列 8.4 演进候选，帧标签仅经 `members[]` 输出。
- 裁决·帧级无多标签无自洽采样——`[frame.classify]` 无 `assignment` (帧单一归属地基)、`[frame.annotate]` 无 `self_consistency` (成本 $\times n$ 且投票键取自帧 Schema 需动投票主干)；两键显式书写 ⇒ 定向 `CONFIG_ERROR` (v1.11 `use_vision` 的原始节探针同款，3.1.4)。
- 裁决·vision 语义分列——`frame.classify.llm` 永不入 vision 必需集：解析产物 `FrameClassifyConfig.vision_resolved = ui ∧ enabled ∧ profile.supports_vision` (`segment V1` 同款自动推导；成本控制面 = 指向纯文本 profile，判决仅凭摘要行)；`frame.annotate.llm` 在 `ui ∧ enabled` 时无条件入 vision 必需集 (镜像序列级 `annotate`：截图是标注主证据)；两者均登记进 `referenced_profiles` (3.1.4)。
- 裁决·组链双门——factory 以或门 `classify.enabled v frame_classify.enabled` 决定 `ClassifyStage` 进链 (槽位不变)，orchestrator 组链槽位同口径；stage 内序列级判决单独受 `classify.enabled` 门控——仅帧级开启时序列记录不产生 `Classification`、帧 pass 照常 (3.10.3、3.13.7)。
- 裁决·帧类 examples 不渲染——帧级批量判决模板无 few-shot 段 (与序列级 §10.8 有意不同)：`[[frame.classify.classes]].examples` 解析合法但不渲染，M1 显名 `WARN class examples are not rendered by the batched frame-verdict template (§10.12)`，so this key is ignored，静态预算预检口径同步不计 (3.1.4、5.2)。
- 裁决·同 id 成员 first-wins——成员 id 为内容哈希 (ingest D2 已知碰撞面)，episode 内同 id 帧的帧产物 dict 落表与逐帧调用均 first-wins (首位次胜出、同 id 只调用一次、计数按唯一 id)；`members[]` 各位次行渲染同一产物 (温度 0 下同内容同判决的自然近似，3.13.7、3.5.5)。

- 裁决·扇出共享时序补丁——帧标注 pass 在 M5 才运行，M13 扇出前对将克隆的首标签序列信封把 `member_annotations` 钉为共享空容器（降格信封除外，保持 None=未运行语义）；M5 只补缺位、从不换对象；沉没成本 `discarded` 只取首标签信封视角（克隆终态不重复计共享产物，3.13.4、3.11.2）。

时间流生成 (v1.13 对齐, 2026-08-13)：需求原型为「无输入数据时直接合成一份带时间戳的多会话请求流——既要能立刻当标注结果用，也要能当输入数据重放」。设计经三方预实现审计（代码可行性/亲和性、文档观测清单穷尽、对抗性反证——反驳四条、需调整十二条全部折入）折入定稿，**裁决详表见 `docs/dev/SPEC-stream-generation.md` §2，凡与提案不一致处以裁决为准**；本特性裁决沿用 v1.12 的自然语言命名制，不新造字母编号系列。逐条择要：

- 裁决·形态与分工——`generate_only` 增**时间流形态**（`[generate.stream]`，默认关；全关时全系统与 v1.12 字节等价）：LLM 只做两类内容调用（蓝图 + 帧实现，噪音帧批量实现复用平面模板），会话装箱 / 交叉 / 噪音插入 / 重复重发 / 时间戳铺设全部归**机械交织器**——零 LLM、零 IO、纯函数族（3.6.5）。
- 裁决·抽签消费顺序表——单流 `Random(f"{seed}:0:generate")` 三段冻结顺序（测试钉住，防实现漂移）：**计划期**①配额按类名字典序展开（`--limit` 在此做前缀截断）②逐序列长度 ③逐序列 (llm, style) 预抽；**派发期**零 `rng` 消费（既有纪律）；**交织期**④重复选取 ⑤装箱洗牌 + 成对交叉 ⑥逐交叉会话切换点 ⑦逐噪音帧掷签 ⑧重发落流尾新会话 ⑨时间戳铺设。作废序列会改变交织输入 ⇒ 确定性以蓝图/实现的 LLM 输出为条件（2.6 声明链）。
- 裁决·时间流工件通道——交织产物一式两份：可重放的时间流工件（`{output_stem}.stream.jsonl`，M11 第五输出通道，与主输出同批 **finalize** fsync + 原子改名、共用 `_undeliverable` 纪律、dry-run 天然不触达）与**直装序列信封**（不再经 segment 二次分段，3.11.2、6.5）；§2.6「唯一写盘对象」清单增第五项，不放松「无中间态落盘」原则——工件是与主输出同级的数据输出通道。
- 裁决·工件行即 raw / 真值不携最终 id——成员 `Record.raw` = 工件行**全对象**（含 `truth`）、id 用 M2 公式 ⇒ 工件重放时成员 id 逐字节一致；`truth` 的序列归属只用**计划期标识**（`sequence_class` + 类内序号），**禁止携带装配后的 record id**（成员 id 依赖行内容、序列 id 依赖成员 id，携带即循环依赖），主输出与工件的对账靠 `member_sources` 行号双向可查（3.6.5、6.5）。
- 裁决·会话装箱定容——交叉并发度恒 $k \in \{1,2\}$ ：`sessions` 显式声明，交叉会话数 = $\Sigma \text{sequences} - \text{sessions}$ （M1 静态校验 `sessions \leq \Sigma \text{sequences} \leq 2 \times \text{sessions}`）——取代提案的 `cross_ratio` 比率键，消除比率 + 取整的弯折；交叉形态 = A 段 + B 段 + A 余段[+ B 余段]（保证真交叉）；重发序列恒**落流尾新会话**（避免同刻不定序）。
- 裁决·噪音只做插入与重复——插入型噪音帧（`noise_ratio`，位置掷签、尊重 `session_max_len` 容量）+ 原样重发（`duplicates`，逐字节同源 ⇒ episode 级 exact 重复演示位）；乱序与 M2 `on_disorder` 自相矛盾、缺失对合成无意义，两类不做（8.1）。重发序列**不进本次运行的信封**（其原本已进），判重演示位在工件重放。
- 裁决·量目标辖区——按类 `sequences` 是**尝试配额**（`standalone_count` 同款语义）：无输出条数保证、无补齐回路；「输出精确定量 + 补齐回路」维持 8.3 O6 辖区。
- 裁决·序列类约束按形态放宽——本形态 ⇒ `classify.enabled = true` **硬合取**（类表是配额与按类条件化的载体；标签直接继承 `inherited`，零判决调用）；同时把「≥ 2 类」放宽为 ≥ 1 类、`fallback_class` **免填**（写了仍须 ∈ 类表）——两条规则的保护对象是判决路径，本形态无判决路径；`classify.llm` 援引 S30 先例豁免密钥引用集（存在性检查照旧）。
- 裁决·按类标注 Schema——`[class.<name>.annotate]` 白名单增 `schema_path / schema_inline`（覆盖语义，缺省回落全局 `output.schema`；实现抄 rubric 的按类重资产先例），兑现 v1.7「按类输出 Schema」演进候选（8.4 M13 行核销）；M5 六消费点与 M11 写前终检统一改按**该行类有效 Schema**取值——计价的 Schema 恒等于调用的 Schema（3.4、3.5.2、3.11.2）。
- 裁决·M8 显式待遇参数——`complete_validated` 增待遇门 `user_treatment`（None = 现行 `schema is None` 推断，**全部既有调用点零改动**；2026-08-14 起它是 `CallScope` 的一个字段）：按类标注 Schema 调用显式传 `schema` 且 `user_treatment=True` ⇒ **L2.5 与 resolved_at 记账双保留**，正面修掉 v1.12「显式 Schema = 放弃回调与记账」的弯折；§6.4 恒等式重述为「resolved_at 加总 = 进入 M5 的记录级标注调用数」（3.8.2）。
- 裁决·蓝图实现内部 Schema 与 L0 待遇——新增两个模块级构造器 `plan_schema / realize_schema`（后者以 draft 2020-12 原生 `prefixItems` 逐位包装 + `"items": false` 封尾，`jsonschema` ≥ 4.21 直接可校验、无翻译层）；用户手写的帧类生成 Schema 经包装后随 L0 原样透传，**不做关键字白名单 lint**（`output.schema` 今日同款暴露面）——某些 `strict` 路由拒 `prefixItems` 的排障面是配置级 `supports_structured_output = false`（3.8.1）。
- 裁决·直装评审判决形（修正提案）——提案的「走既有非流评审路径零改动」经审计证伪（驱动器门 = `segment.enabled`、模板门 = `kind`，二者不一致导致缺陷词表模板与 `VERDICT_SCHEMA` 错配）。裁决：直装序列在 M7 走**判决形**（判决指令 `system + [任务指令] / [成员帧摘要] / [标注结果]` 三段证据，无缺陷表/边界余量/片段结构），`schema` 恒 `VERDICT_SCHEMA`，修复沿用既有 `policy` 重标注，`fail` ⇒ `dropped_verify`（拒绝采样，淘汰不改真值）；流式驱动器与缺陷词表面**零改动**（3.7.5）。
- 裁决·轨迹准则自动解析扩展（修正提案）——S29 的空 `quality.rubric` 解析条件由 `segment.enabled` 扩为 `segment.enabled v generate_stream.enabled`（loader 与 emitter 镜像两处同步）：本形态打的同样是序列/轨迹的分（3.4.3、5.2、A.3）。
- 裁决·生成能效矩阵——`llms / mixture / weights` 生效（每序列预抽一次，蓝图与实现绑定同一 `profile`；噪音批独立预抽）、`styles` 生效于实现与噪音调用（蓝图不带风格）、`temperature` 生效、`num_per_call` 仅噪音批装箱生效；`num_per_record / seeds_per_call / seed_examples / standalone_count` 显式书写 ⇒ **定向 CONFIG_ERROR**（v1.11 原始节探针机制）；`sample_validator` 生效于**逐帧文本**，任一帧违规 ⇒ 整序列作废（蓝图定长不可剔单帧，拒绝采样语义）。
- 裁决·估算精确复演与 golden 冻结锚不动——`estimate_run` 的 `generate_only` 分支改复用 **M6 计划期纯函数**（吃 `cfg + seed`，精确复演长度与噪音采样，非上界）：`generate_calls = 2 \times \Sigma \text{sequences} + [\text{噪音帧数} / \text{num\_per\_call}]`、`classify_calls = 0`（`inherited` 零调用，

v1.7 R11 幂等哲学)；估算行格式零改动（三类调用全折入 `generate_calls`，console 键表零改动），既有七个 dry-run golden 字节不动，仅新增第八个 `dryrun-synth-stream.txt` (3.10.3、7.8)。

- 裁决·观测面——`report.generate` 增 `stream` 子块 (counts-only 12 键)、`report.run` 摘要族增工件条目 (路径/sha256/行数，主输出同款)；`report.stream` 节不出现 (那是 segment 的观测面，避免混淆)、`report.classify` 直方图恒全零属预期；trace 零新通道零新事件 (蓝图/实现经 `llm.call` 可见)、§7.6 零新错误 kind (6.4、7.2)。

代码规则整改 (2026-08-14 对齐)：需求为「全库对齐仓库代码规则」——注释/docstring 中文、代码与一切用户可见输出英文，每文件 ≤ 2000 行、每函数 ≤ 50 行且 ≤ 5 入参。整改不改任何功能、配置键、事件名、错误 kind、报表结构与链序，只动语言与承载形式；离线套件全绿。逐条择要：

- 裁决·语言分工——注释与 docstring 中文；标识符、日志、报错、异常文案与 CLI 输出 (含 `--help`) 英文。两类例外保持中文原样：CONTRACTS §10 的 LLM 提示词模板 (它是模型的输入，改动即改行为)，以及 spec 冻结的输出数据——`thread_seam` 占位步文本、缺陷表 `detail` 串、打包 rubric 的准则文本 (3.15、3.7.2、附录 A)。数据内容永不翻译。
- 裁决·冻结面收参为参数对象——四个装配面与编排器构造超出「≤ 5 入参」，一律收拢为冻结 dataclass，字段名与语义逐项不变：
`CallScope{record_ids, batch_no, record, user_treatment}` (M8, 3.8.3)、`AnnotatePromptOptions{repair, temperature, label, transitions, fragment_lens, k_eff, image_px}` (M5, 3.5.2)、`VerifyPromptOptions{label, transitions, boundary_margin, fragment_structure, fit, verdict_form}` (M7, 3.7.5)、`ThreadCard{index, task_name, members, span, fragment_count}` (M16, 3.16.3)、`RunServices{llm, schema_engine, metrics, run_id, run_started_at}` (M10, 3.10.3, 自 `labelkit.orchestration` 导出)。换档一律 `dataclasses.replace`；v1.7—v1.13 逐次「追加式末位 kwarg」的叙事就此退场，缺省实例与各自引入前逐字节等价。
- 裁决·M1 拆六个包内私有模块——`loader.py` 只留公开入口、console 模式裁定与 `ResolvedConfig` 装配；解析与校验下沉 `_collect` (错误聚合器 / 类型化表读取)、`_sections` (逐节解析)、`_schemas` (Schema 元校验与 few-shot 干跑)、`_rubrics`、`_classviews`、`_constraints` (跨节组合约束与解析产物冻结)。公开面 `load / default_rubric / ResolvedConfig` 与校验顺序、错误文案、聚合反馈行为均不变 (3.1.3)。
- 裁决·M9 恒传 hard——`_record_provider_result` 恒以关键字 `hard` 调用观测汇，调用形嗅探分支删除；替代 `MetricsSink` 必须接受该关键字 (3.9.3)。
- 裁决·用户可见面重冻结——八个 `tests/cli/goldens/dryrun-*.txt` 与 `console_format` 两条 plain 锚点 (`\rlabelkit: batch ...` 进度行、`— final summary (matches report.counts item by item)` — 摘要头) 整体重冻结到英文串上：键、键序、数值、行序与信息集全不变，仅文案语言变，此后继续逐字节钉死 (7.7、7.8)。rich 面板行标签同批英文化 (键位提示、`account / stages / keys / breaker / elapsed` 等)，区块划分、数据源与布局不变。
- 裁决·测试补齐闭环 (同日随整改交付)——spec 功能面矩阵 474 项逐条对照用例，缺口一次补齐；函数覆盖率口径定为离线 + 集成合并测量 (M9 的两个 HTTP 传输函数 `_post_with_retries / _dispatch_attempt` 由真端点集成套件承保，禁止以假失败 socket 凑离线口径——mock 传输层红线不破)。补测发现的偏差按「spec 为准则、含糊处参考业界项目」裁决并记录：探测表字面格以 Text 直传 (rich 控制台 markup 吞 `profile[key]` 字面格，按 rich 官方文档与 Textualize/rich Issue #120 的处置修复，E2E 第 29 条)；HTTP-date 解析失败只经异常面 (Python ≥ 3.11 下 `parsedate_to_datetime` 失败必抛，None 兼容分支属死代码删除)；死参数直接删除 (`_collect` 三个数值读取器的零调用 `required` 形参)；校准器空桶分支删除 (`freeze_batch` 恒非空)；warn-once 守卫留在发射点 (CPython warnings 模块 registry 同形)；单文件承载 M9 单测 (`EXPECTED_TEST_PY` 冻结清单与 CONTRACTS §1.2 归属条款优先于文件长度偏好——行数上原本就只约束生产代码)；零尝试熔断中止不发事件 (`llm.call` 的 status 规范句「重试退避途中被中止」为准，为未上线调用补发事件会虚增 §7.7 控制台分子)；二遍判定失败恒按 keep (`SPEC-activity-structure.md` §3.2 实现定稿原文钉住，pass-2 是可选修复增益、其失败无权销毁一遍产出的合法线索)。

帧类构成档位与时间字段回填 (v1.14 对齐, 2026-08-18)：需求原型两条——「同一个生成工程要能按难度/复杂度档位产出不同构成的序列，并且档位身份要能随数据落盘对账」与「生成 Schema 里那些时间语义字段 (如 duration) 跟真实帧间隔对不上，LLM 没有时间轴、写出来的秒数是编的」。需求方三项方向裁决先闭 (档位即帧类构成、tier_rank 即档位身份、时间字段回填方向)，其余为本特性设计裁决与三方预实现审计折入。两机制彼此正交、共用一个修订号，全部默认关闭。裁决详见 `docs/dev/SPEC-generation-tiers.md` §2，凡与提案不一致处以裁决为准；沿用自然语言命名制，不新造字母编号系列。逐条择要：

- 裁决·档位即帧类构成 (需求方)——档位的定义就是该档序列的帧类构成集合 (「有 x 类帧为一档，有 x+y 类帧为另一档」)，不携带任何质量指令、不控制帧内部语义质量；提案初稿的档位 instruction 键删除，帧内语义照旧由帧类生成指令与温度决定。
- 裁决·tier_rank 即档位身份 (需求方)——档位表不设 `name` 键；`tier_rank` (正整数) 代表「第几个档位的要求」，表内唯一、全表连续覆盖 1..N (N = 表长；缺号/重号 = CONFIG_ERROR)，同时是配分平票与类内序数分块的确定性排序依据；工具不赋予序数高低任何质量方向语义 (方向归用户)。
- 裁决·时间字段回填方向 (需求方)——帧生成 Schema 内的时间语义字段与实际帧间隔不符的缺口，收编为「绑定即剔除 + 机械回填」：LLM 物理上无法生成该字段，值由 harness 按已铺时间轴计算——v1.13「构造既知量即真值」原则从标签面延伸到时间量面。
- 裁决·零抽签配分——每参与类的配额按 `weight` 走最大余额法确定性配分，算术冻结为整数域 (基额 = $(\text{sequences} \times \text{weight}) // \Sigma \text{weight}$ 、余额键 = $(\text{sequences} \times \text{weight}) \bmod \Sigma \text{weight}$ ，按余额键降序、平票按 tier_rank 升序逐档 +1，禁止任何浮点中间量——平票判定要一路喂给类内序数分块 → truth → 工件字节 → 成员 id，是冻结面)；类内序数按 tier_rank 升序占连续区间，推论是 `--limit` 前缀载

断在每类从最高档序数侧截起、截断与映射可交换。配分是 (sequences, tiers) 的纯函数、零 rng ⇒ 冻结的抽签消费顺序表原文不动，配分插在配额展开与长度抽取之间的零消费步 (3.6.5)。

- 裁决·蓝图双向硬约束——「不出档外类」由 plan_schema 的 enum 限档内子集保证 (调用方传子集名集，构造器零感知)；「档内每类至少一次」由 cover_all = True 注入的 allOf + 逐类 contains 保证 (draft 2020-12 原生关键字；一个 Schema 对象只有一个 contains 键位，故多类分 allOf 支)。违约进 M8 既有修复环、穷尽按既有 plan_failures 作废——零新失败机制 (3.8.1)。机制注记：L3 修复轮的提示包不带温度、生效温度回落 profile 默认，「高温首轮违约由低温修复轮收敛」是既有机制事实。
- 裁决·构成恰等——构成语义 = 恰等：enum 给「⊆」、contains 给「⊇」，合成「members[] 帧类集合 ≡ 档声明构成」，档位身份因此可从数据反推对账；「至多这些类」的宽松形态会使高低档的低配产物不可区分，列演进候选 (8.4)。
- 裁决·档位标识三点落位——① _meta.source.generator 增 tier_rank 键 (档位表在场时；rejects 侧 generator 既有携带逻辑自动跟随) ② 工件 truth 增 tier_rank (任务帧 = 本档序数、噪音帧 = null、重发帧 = 源档) ③ report.generate.stream 增 tiers 子块。§6.3 「信封只增字段」惯例遵守，档位表缺省三点全部不在场 (字节回退，6.3/6.4/6.5)。
- 裁决·报表显式装配 (审计折入)——report.generate.stream 是编排器的显式键装配而非计数器前缀树，故 tiers 必须在该装配段按档位表 tier_rank 升序零基铺开 {<rank>: {planned, produced}} (十进制字符串键；report 落盘无 sort_keys ⇒ 键序 = 装配插入序)，键位冻结在 sequences 之后、frames 之前 (配额族相邻)。零额档与全作废档的在场由装配保证 (planned 0 / produced 0 如实呈现)，不依赖计数器首触序——这是 E2E-FINDINGS 第 11 条「计数器被报表白名单静默丢弃」的同族陷阱，在规格层拦下 (3.10.3、6.4)。
- 裁决·真值键序重冻结——truth 键序为 session, sequence_class, sequence[, tier_rank], frame_class, noise[, duplicate_of]：tier_rank 仅档位表在场时出现，位于 sequence 之后 (序列身份组)、frame_class 之前；行文件字节序由此定，id 用 canonical JSON (键排序) 不受键序影响 (6.5)。
- 裁决·重发帧承源档与同源载荷——重发帧 truth.tier_rank = 源序列档位、载荷与源序列同位帧为同一回填后对象 (「内容逐字节同源」是 v1.13 冻结不变式，档位与时间字段皆内容属性)；重发帧自身的流水时间轴只体现在行 ts 字段——原样重发本就携带陈旧内容，语义自治。同源由对象同一性直接保证 (重发槽位与源槽位本就引用同一载荷对象)，回填就地写入后自动生效，无需任何回查机器。
- 裁决·配分零额告警——小配额 × 权重悬殊时某 (参与类，档) 配额为 0 是最大余额法的自然结果：WARN 一行 (值-free: 类名、tier_rank、权重表；非错误)，tiers.<rank>.planned 如实呈现 0 (3.1.4)。
- 裁决·语义词表四值——闭集 ts (本帧已铺时间戳 ISO 串——任务帧上 = 行 ts 值、重发帧承源值 ≠ 自身行 ts) / gap_prev_s (与本序列上一帧的间隔秒，首帧 0.0) / gap_next_s (与本序列下一帧的间隔秒，末帧 0.0) / elapsed_s (距本序列首帧秒，首帧 0.0)；数值 = round(ts 差秒, 6) (微秒精度与 isoforamt 对齐)。词表是冻结闭集，扩词走 spec 修订 (5.2)。
- 裁决·序内间隔口径——间隔按本序列相邻成员的 ts 差计：交叉会话夹入的外序列帧与噪音帧本就占用其间墙钟，序内差值与下游从数据实测的口径一致；不提供「会话内相邻行」口径 (那是重放侧可自行计算的流水量，非序列内容属性)。
- 裁决·绑定即剔除——被绑定字段从 LLM 面向的逐位 Schema 与逐位契约行中剔除 (properties 删键、required 减名的差集语义、其余关键字原样)，M8 按缩减 Schema 校验——不为注定被覆写的字段付 token 与修复环成本，契约无误导。工件载荷 = 缩减校验产物 + 回填字段，对用户声明的完整生成 Schema 的满足限于类型层 (ts ⇒ string、其余 ⇒ number, M1 静态保证)；绑定字段上除 type 外的约束关键字 (minimum/maximum/pattern 等) 不被强制也不被校验，M1 为此发一条 WARN——时间量的值域由时间轴决定，不受 Schema 数值约束辖制。
- 裁决·回填后计 id——回填尾声位于 ts 铺设之后、直装组装之前：行对象、成员 Record.raw / text / id、序列 id 与 session_id 全部按回填后载荷计算 ⇒ 工件重放逐字节同 id 同会话 (v1.13 裁决·工件行即 raw 原文适用)。重放侧的判重档位语义 (exact/near_text 随原会话噪音帧是否被 segment 吸收而浮动) 不受回填扰动——判重配方吃成员文本不吃 id，「逐字节一致」不得误读为「重放判重恒 exact」。
- 裁决·回填前钩子口径——sample_validator 逐帧校验与序列相似度过滤作用于回填前载荷：时间字段是机械量，不参与内容校验与内容判重 (两序列内容相同仅时间不同 ⇒ 照旧判近重，语义正确)；钩子看不到绑定字段。
- 裁决·观测零增量与冻结锚不动——档位面仅增 report.generate.stream.tiers (counts-only、条件在场)；时间字段面零观测增量 (确定性机械操作，无可计数失败模式)。两机制零调用数变化 ⇒ estimate_run 零改动、八个 dry-run golden 字节不动、console 键集与面板零改动、trace 零新通道零新事件、§7.6 零新错误 kind。
- 裁决·静态预检上界照旧——M1 静态预算预检的蓝图段照旧按全帧类表计量 (档内子集恒 ≤ 全表，上界性质保持)，realize 段在缩减后只降不升；TEMPLATE_HEAD_TOKENS 零改动 (蓝图静态脚手架四常量不动——覆盖句落在动态的 user 行)。
- 裁决·L0 待遇沿用——cover_all 产物 (allOf / contains) 随 supports_structured_output 上行，沿用 v1.13 裁决·用户生成 Schema 的 L0 待遇：不做关键字白名单 lint；strict 路由拒收的排障面是配置级 supports_structured_output = false，拒收形态是首个蓝图调用即 HTTP 400 快速失败且计入熔断连击 (非逐序列作废)。L0 关端点上结构服从性靠模板覆盖句 + 修复环兜底 (prefixItems 同款纪律，3.8.1)。
- 裁决·渲染缺类可见 (审计折入)——M8 违规渲染增 contains 分支：从 validator_value 内取 frame_class 的 const 值渲染为 steps: missing required frame_class "<名>"，不再落裸数组 repr。L0 关端点上 L3 修复提示必须点名缺失帧类，否则修复指导性趋零；渲染文本英文，值-free 纪律不变 (帧类名是配置量非数据内容，3.8.4)。
- 裁决·微秒地板 (审计折入，v1.13 缺陷修补)——M1 增约束 frame_gap_s.lo ≥ 1e-6 (微秒地板 = isoforamt 精度与 round(·, 6) 的分辨率下界)：亚微秒 lo 下 timedelta 取整为 0 微秒，v1.13 「ts 严格递增」声明本就存在破口，词表的 0.0 边界哨兵更不容真零间隔。属形态级缺陷修补而非双关面——对 lo ≥ 1e-6 的一切现存工程零影响 (全部示例 lo = 5)，亚微秒配置在 v1.13 本就产出缺陷数据 (2.6、3.1.4)。
- 裁决·指令必填域收窄 (审计折入)——档位表在场时，v1.13 「每帧类生成指令必填」的检查域收窄为 U 各档 frame_classes (蓝图可能选中的闭集)：未入档帧类豁免必填 (已写照常合法)，否则用户被迫为永不选中的类写死指令；「帧类未入档」WARN 照发并点名其生成面

(含时间字段绑定) 整体为死配置 (3.1.4)。

按类档位表 (v1.15 对齐, 2026-08-19): 需求原型为「多种意图序列混合生成是否支持每个意图独立配置档位?」——v1.14 的档位表是**全局唯一**的, 所有参与序列类共享同一套 `tier_rank / weight / frame_classes`, 而意图不同则帧类语汇与构成天然不同 (购票有确认帧、闲聊没有), 全局表达不了「每意图各一套构成与权重」。本特性是 v1.14 档位面的**按类化增量**, 与其时间字段回填面完全正交 (零触碰), 默认关闭、按类表全部缺省时与 v1.14 逐字节等价。设计经两路预实现审计 (代码可行性/亲和性、文件修改清单穷尽, 2026-08-19) 折入定稿, **裁决详见见 docs/dev/SPEC-per-class-tiers.md §2, 凡与提案不一致处以裁决为准**; 沿用自然语言命名制, 不新造字母编号系列。逐条择要:

- 裁决·表级原子覆盖——类声明了就用类的**整张表**, 不逐行合并 (行级合并会让 `rank` 身份跨表漂移, 且「补一行」「改一行」的语义无处安放)。先例是仓库里两处按类重资产的既有形态: `[class.*.quality].rubric` 的内联子表整表替换、`[class.*.annotate].schema_*` 的覆盖回落 (5.2 白名单表)。
- 裁决·全局表为锚——**按类表要求全局表在场** (缺失 = `CONFIG_ERROR`, 文案指引补全局表), 档位面总开关恒 = 全局表非空。收益是**在场性恒定不逐行漂移**: 每个参与类恒有生效表, 故 `_meta.source.generator.tier_rank / 工件 truth.tier_rank / report.generate.stream.tiers` 三点的在场判断与 v1.14 逐字不变 (噪音槽位谓词、报表在场判断等一切「档位面在场」判断零改动)。先例是 v1.13 的「`output.schema` 恰一不因按类覆盖而豁免」。「某类不想分档」的表达式 = 给它一张单档表 (`tier_rank = 1`、构成任意)——退化形态, 零机制成本。
- 裁决·rank 类内身份——每张生效表各自正整数、表内唯一、**连续覆盖 1..N** (N = 该表长, 逐类可不同); 跨类同 rank **无任何工具语义** (v1.14 本就不赋 rank 质量方向的自然收窄), 「构成两两互异」的辖区相应收窄为**单表之内**——跨类同构成完全合法 (各类都可有自己的「全类档」)。行级消歧免费: 主输出行携 `classification.label`、工件行携 `truth.sequence_class`, 与 `tier_rank` 同行相邻。
- 裁决·空表拒收——TOML 三态各有其义: 缺省 = 未声明 (回落全局)、`tiers = []` = 显式空表 (`CONFIG_ERROR`, 指引删键回落)、非空 = 覆盖。显式空表在档位面开启时没有合法语义——面统一之下不存在「本类无档」态。
- 裁决·载体 ClassView 顶层字段——按类档位表的解析产物挂 `ClassView.tiers` (None = 未声明, 回落全局), **不落 GenerateConfig**。两个理由: ① `GenerateConfig` 载体会让编排器的按类覆盖探测把纯档位覆盖误判为「估算失真型按类覆盖」; ② v1.13 按类标注 Schema 的载体先例就是 `ClassView.schema`, None = 回落语义同款。
- 裁决·note 行不因档位触发——`ClassView.tiers` **不加入 dry-run**「按类覆盖可能使估算失真」提示行的比较项: 该提示警示的是调用数估算失真, 而档位不改变任何调用数, 警示不适用 (显式写入本裁决以免检视误报遗漏)。
- 裁决·effective_tiers 下沉 common——生效表查找点是一枚零 rng 纯函数 `effective_tiers`(类表, 全局表), 与 `apportion_tiers` 同处配置模型侧 (common 层): M1 约束簇、M6 计划期、M10 报表装配三方共用同一实现; 分层纪律不许 common 导入 operators, 故 M6/M10 正向导入 (`apportion_tiers` 同款)。 `apportion_tiers / tier_rank_for_ordinal` 本体与签名零改动——v1.14 已把档位表做成参数, 调用点改传该类生效表即可。
- 裁决·计数器键按类重冻结——v1.14 冻结的计数器键族 `generate.stream.tiers.<rank>.*` **解冻重冻结**为 `generate.stream.tiers.<class>.<rank>.{planned, produced}`: M6 恒喂类段键 (单一喂数纪律, 禁双写两族键), 平面形 (仅全局表在场) 由编排器按 rank **跨类求和装配**——数值与 v1.14 逐字节相等 (v1.14 的平面计数本就是跨类聚合值), 嵌套形直铺。解冻在 `CONTRACTS §12` 显式登记 (条目 36)。
- 裁决·嵌套报表全类铺开——嵌套形外层 = **全部声明类按声明序** (与 `report.generate.stream.sequences`、`report.classify.classes` 同款零基铺开), 内层 = 该类生效表 rank 升序; 零配额类与全作废档呈现 0/0。这是 v1.14 裁决·报表显式装配的直接延伸——迭代声明表而非依赖计数器首触序; 触发谓词 = 任一类声明了按类表, `tiers` 键位仍冻结在 `sequences` 之后、`frames` 之前 (6.4)。
- 裁决·校验域并集——「帧类未入档」WARN 与「每帧类生成指令必填」两处检查域, 从「全局表构成并集」改为 U (**各参与类生效表的构成并集**) (参与类 = 有效 `sequences ≥ 1`)。推论: 若全部参与类都声明了按类表, 全局表沦为纯锚, 其独有帧类照样判死配置——精确反映「哪些帧类真会被蓝图选中」(3.1.4)。
- 裁决·零额结构校验不豁免——`sequences = 0` 的类若声明了按类表, 其**结构校验照做** (身份连续性、构成合法性、空表拒收、前提门——坏配置早报), 而配额对约束与零额 WARN **豁免** (不为永不尝试的组合抬高 `len_range` 下界, v1.14 零额对豁免语义的直接沿用), 其构成不入校验域并集 (该类不产序列)。

时间流序列规则 (v1.16 对齐, 2026-08-20): 本特性让时间流生成工程能声明「每条任务序列必须遵守什么控制流、相关帧之间隔多久、哪些 payload 字段必须相等、某类帧允许在什么日历时间出现」, 并允许在声明式校验后执行一次用户序列回调。设计以 `docs/dev/SPEC-sequence-rules.md` 的关闭裁决为唯一增量事实源; 提案中的显式乘积 DFA、独立 STN、LLM 后重排与自研求解器路径均不成立。逐条择要:

- 裁决·配置与逐类三态覆盖——全局表为 `[[generate.stream.rules]] / [[generate.stream.windows]]`; 类表为 `[[class.<name>.generate.rules]] / [[class.<name>.generate.windows]]`。两个表族彼此独立, 逐类均为未声明则继承、显式空表则清空、非空表则整表替换; 与 v1.15 tiers 不同, rules/windows 不要求全局锚, 纯按类声明合法。 `[generate] sequence_validator` 是可选 `module:function` 引用。三者只在时间流 `generate_only` 形态合法, 其他形态定向 `CONFIG_ERROR`。
- 裁决·模板与标准有限迹语义——模板闭集为 `existence`、`absence`、`exactly`、`init`、`end`、`responded_existence`、`co_existence`、`response`、`precedence`、`succession`、`alternate_response`、`chain_response`、`chain_precedence`、`not_co_existence`、`not_succession`。使用 `end`, 不提供 `last` 别名; 二元规则要求 source 与 target 不同。前三个模板独占正整数 `count`, 其他模板禁止 `count`。activation 缺席遵守 `vacuity`; 同一 witness 可服务多个 activation; identical 重复规则是 `CONFIG_ERROR`。
- 裁决·确定性 evaluator 顺序——规则按生效表声明顺序、activation 按位置顺序; `co_existence` 与 `succession` 的双向义务先 source→target 再 target→source。该顺序只冻结首个失败的计数归属, 不改变整条序列的接受集合。

- 裁决·时间关系精确口径——`time_s = [lo, hi]` 表示秒域半开区间 `[lo, hi)`，端点必须无损量化为整数微秒且 `1μs ≤ lo < hi`。有向模板用 `target_ts - source_ts`，无向模板用绝对差。正向规则要求至少一个合格 witness；负向规则禁止区间内的合格 pair。多个显式区间落在同一 pair 上时取交集。
- 裁决·默认间隔与重放护栏——每个 owner 内相邻任务帧恒满足 `1μs ≤ delta ≤ stream.gap_s`。`frame_gap_s` 只约束没有被所选正向显式时间 witness 覆盖的相邻 pair；显式时间替换默认 frame gap，但仍与重放护栏取交集。无 `time_s` 的 witness 不移除默认 gap，非相邻显式时间只约束总跨度。`frame_gap_s` 以 `Decimal(str(value))` 转换为闭合微秒整数区间，空区间是 `CONFIG_ERROR`。
- 裁决·日历 occurrence 语义——每个 window 行只对应一个帧类，同一生效表内重复帧类是 `CONFIG_ERROR`。`of_day` 为同一自然日内非空、互不重叠的半开时间段；`of_week` 缺省为周一至周日且不可重复。该帧类的每次 occurrence 都必须落在 `of_day × of_week` 并集内；逻辑跨午夜窗不支持，但会话可跨自然日。`ts_start` 带固定偏移则沿用，naive 按 UTC；不引入 IANA 时区或 DST。
- 裁决·correlation 精确口径——correlation 仅支持 `operator = "equal"`，source/target 字段必须是各自结构化帧生成 Schema 的顶层必填属性、JSON Schema 类型完全相同，且不可绑定到 `time_fields`。运行期先做类型敏感相等，再比较 canonical JSON bytes：布尔值、整数与浮点数互异，对象键序无关，数组顺序有意义；判定发生在时间字段回填前。
- 裁决·correlation 与 time 验证顺序——先枚举结构候选，再用 correlation equality 过滤，之后用 `time_s` 过滤并检查正向义务或负向 absence。正向 correlation 规则没有相等候选计 `correlation_scrapped`，有相等候选但没有时间候选计 `temporal_scrapped`；带 correlation 的负向规则命中只计 `correlation_scrapped`。无 correlation 规则若在运行期违约，说明规划器不变量失守，必须以 `InternalError` 终止，不能降为普通作废。
- 裁决·一个联合 CP-SAT 模型——精确锁定 `ortools==9.15.6755`。同一个模型在任何 LLM 调用前联合冻结逐序列帧类词、潜在 witness、会话归属、主任务时间戳与噪音槽；DECLARE 用布尔约束与 `AddAutomaton` 接到同一组位置变量，不构造显式乘积 DFA。M1、`estimate_run`、M6 必须共用 `common/runtime/sequence_planner.py` 的问题构造与求解入口。
- 裁决·求解确定性与状态——`num_search_workers = 1`、CP-SAT 自动搜索、从运行随机源抽取一个 31-bit solver seed、`max_deterministic_time = 10.0`，不设置墙钟超时；不承诺可行解均匀抽样，也不承诺跨 OR-Tools 版本相同解。模型 proto 的变量数与约束数之和超过 250,000 即 `CONFIG_ERROR`。M1 的 INFEASIBLE 表示配置不可满足，UNKNOWN 表示确定性预算内无法证明，MODEL_INVALID 是 `InternalError`；噪音最大化只接受 OPTIMAL，普通可行求解接受 FEASIBLE 或 OPTIMAL。
- 裁决·所有候选长度与前缀联合可行——M1 不只证明某个长度可行，而是校验每个候选长度的局部潜力，并对 `--limit` 后实际非零配额前缀证明全流可行。全流路径按类名与类内序数顺序，每个 attempt 恰消费一次 `rng.randrange(width)`，把偏移旋转为稳定长度偏好序；同一个全流 CP-SAT 模型保持 active 前缀长度自由并最小化偏好名次，返回的单个长度向量已与 word、日历、witness、session、crossing 与 gap 联合证明可行。不存在改偏好、改长度或重复求解的 fallback；被 `--limit` 完全截掉的约束类不单独激活规划器。
- 裁决·骨架冻结与幸存投影——首个 LLM 调用前冻结 word/session/timestamps/noise slots。某条序列内容失败只删除整个 attempt，不重排幸存者；空会话删除、其余会话重编号，时间戳不移动。`crossed_sessions` 只统计两个 owner 都幸存且仍有 A-B-A 或 B-A-B 真交替的会话。该行为是固定骨架的确定性投影，不是失败后的替代规划。
- 裁决·噪音与重发——噪音目标为 `round(noise_ratio × planned task frames)`，CP-SAT 最大化可放置槽数；不足只发一条 value-free WARN，不判全流不可满足。噪音时间戳唯一、只在会话首尾任务帧的开区间内、不扩大跨度、不参与规则或日历。幸存投影后只保留仍严格夹在幸存首尾之间的槽，既有调用返回的多余 payload 直接丢弃，不补调用。重发源排列在 LLM 前预抽取；只从幸存前缀取源，深拷贝回填后载荷并按最小正周数或微秒位移到全局尾部，绝不按重发自身时间轴重新回填。
- 裁决·约束路径 Schema 与预算——联合路径使用 `brief_schema(length)`，LLM 只输出逐位置 brief；prompt 明示固定帧类、规则、日历与 correlation，帧实现 prompt 再重复相同契约。带 correlation 的序列必须一次 realize 完整序列，禁止反应式二分；M1 必须对最坏 prompt 静态证明可装入 context window，运行期溢出或截断归 `realize_failures`，不拆分替代。无 correlation 路径保留既有有界二分。没有生效 rules/windows 时，v1.15 的提示词字节不变。
- 裁决·序列回调输入与隔离——`SequenceValidationInput` 携 `sequence_class`、`tier_rank` 与有序帧视图；每帧含 `position`、`frame_class` 和 JSON-compatible 深拷贝 payload，回调突变不得改变内部对象。返回值沿用既有回调规范化；异常只记录 hook 引用、异常类型与 violation 数，绝不记录消息、payload 或 prompt。回调是用户信任的同进程代码，不承诺屏蔽其外部非确定性。
- 裁决·验证和装配顺序——逐帧 realize Schema → 每位置 `sample_validator` → 声明式 correlation/time → `sequence_validator` 一次 → 序列相似度 → 幸存投影与噪音过滤 → 主序列 `time_fields` 回填 → 从前抽排列选择重发源 → 重发深拷贝与时间位移 → 全局排序 → 行/Record/sequence id 与装配。首个失败即停止后续验证。`validator_scrapped` 恒等于 sample、correlation、temporal、sequence 四个子计数之和，序列相似度不计入。
- 裁决·报告与工件——`report.generate.stream` 在既有 `tiers` 后、`frames` 前条件新增：生效规则在非零前缀出现时写 `rules.{sampled, correlation_scrapped, temporal_scrapped}`；v1.16 报告面（实际前缀有 rules/windows，或配置了 `sequence_validator`）在场且配置了既有 `sample_validator` 时写 `sample_validator_scrapped`，配置 `sequence_validator` 时写 `sequence_validator_scrapped`；生效日历窗时写 `windows.calendar_days_spanned`。这使旧工程只配置 sample hook、未启用任何 v1.16 机制时仍保持 v1.15 报告字节。显式块即使全零也在场，不能依赖计数器首触；天数按幸存非噪音任务帧（含重发）首尾固定偏移本地日期含首尾计算，无幸存者则为 0。时间流工件、truth、generator 和重放格式不增加逐行 rules/windows 副本。
- 裁决·默认关闭与边界——没有任何非零前缀生效 rules/windows 且未配置 `sequence_validator` 时，保持 v1.15 原调用路径、RNG 消费、提示词、调用数与全部输出字节。零配额类仍做语法、Schema、模板和逐候选长度的本地校验，但不单独激活规划器或报告。序列缺帧/中断不生成；规则只约束同一任务序列；crossing 不是跨业务序列 DECLARE；帧只建模为时间点事件，不引入 duration interval 或 Allen 区间代数。这些边界不是待实现项。

2. 总体设计

2.1 系统定位与总体规格

LabelKit 是一个**单机、单进程、无状态**的 Python CLI 批处理工具。一次运行按 project.toml 声明的阶段组合执行流水线，向一个输出路径写出 JSONL：`run.mode = "process"`（默认）读取一个输入路径、加工既有数据；`"generate_only"`（v1.4）无输入，由 M6 从零生成后走同一条治理管线（3.10.3）——v1.13 起 `generate_only` 另有**时间流形态**（`[generate.stream]`，默认关）：合成的不是一条条独立样本而是一条条**序列**，多条序列机械交织成带时间戳的多会话时间流，除主输出外另写一份可重放的时间流工件（3.6.5、6.5）。v1.16 可在该形态内声明控制流、时间、`correlation` 与日历窗；一旦实际非零配额前缀存在生效 `rules/windows`，M1、估算与 M6 通过同一个 CP-SAT 联合规划器在首个 LLM 调用前证明并冻结完整时间流骨架。工具对外只有三个交互面：**CLI 参数、两个 TOML 配置文件、输入/输出文件**；唯一外部服务是配置中声明的 LLM API。

2.1.1 功能规格总表

能力	可选性	规格
数据接入	必选	文本模式：读取 <code>.jsonl</code> 文件（或含多个 <code>.jsonl</code> 的目录），每行一条记录。UI 模式：递归扫描目录，按文件名中的 <code>index</code> 配对 <code>uitree_<index>.jsonl</code> 与 <code>image_<index>.jpg/.jpeg/.png</code> ，允许跨子目录配对。坏行/缺对策略跳过并计入报告。
去重	可选，默认开	两级：① 规范化内容 SHA-256 精确去重；② MinHash-LSH 近似去重（默认字符 5-gram、128 permutation、Jaccard 阈值 0.85）。UI 模式额外做截图 pHash（默认汉明距离 ≤ 8 ），树/图判定关系可配。作用域可选全局或批内。v1.2 增可选第④级语义去重（SemDeDup [26]，需 <code>embedding profile</code> ，默认关；余弦相似度阈值 0.95，3.3.3）。
数据分类	可选，默认关	v1.7：按用户类别表（ <code>[[classify.classes]]</code> ）对存活记录做 LLM 封闭集分类——词表经内部 Schema enum 硬校验（M8 四层防线），单/多标签可配（ <code>classify.assignment</code> ），可选 self-consistency 投票，结构失败归兜底类（3.13）。类标签进 <code>_meta.classification</code> 并驱动下游按类条件化： <code>quality</code> 按类分池与 <code>rubric</code> 、 <code>annotate/generate/verify</code> 按类指令与维度（ <code>[[class.<name>.*]]</code> 白名单覆盖，5.2）； <code>multi</code> 模式按标签扇出为多条按类管线（一条数据多份结果，3.13.4）。不配任何 <code>[[class.*]]</code> 覆盖即纯打标模式。
时序流分段与动作摘要	可选，默认关	v1.8（stream 模式， <code>segment.enabled = true</code> ）：数据按时间排序输入时，摄取层按 <code>[stream]</code> 声明排序键、做按分区键单调性校验并按 <code>gap/key/上限规则</code> 切候选会话（3.2.8），切批改整会话装箱（3.10.3）；M14 <code>segment</code> 在会话内做滑窗 LLM 边界精化与逐帧噪声剔除，把成员帧收拢为序列记录 <code>episode</code> （3.14）；M15 <code>extract</code> 对相邻帧对推断结构化动作 <code>{action_type, target, value, description}</code> 写入 <code>item.transitions</code> （仅 UI 模式，3.15）；下游 <code>dedup/classify/quality/annotate/verify</code> 按序列形态适配，内置轨迹 <code>rubric default:trajectory</code> （附录 A.3）。噪声帧进 <code>rejects</code> （ <code>reason = noise below_min_len</code> ）；关闭时数据产出与 v1.7 逐字节一致（ <code>_meta.stream: null</code> 除外）。
线索缝合	可选，默认关	v1.9（ <code>stitch.enabled = true</code> ，要求 stream 模式）：M16 <code>stitch</code> 在会话内把同一目标导向任务被穿插切开的碎片（ <code>episode</code> ）保守缝合为 线索 thread ——单调选池 LLM 判定 × 机械先验合取（App 交集 / 实体重叠 / 返回同一页面析取三腿）+ 有界二遍复修正贪心漏缝（3.16）； <code>below_min_len</code> 短段按连续 run 重组先进候选池救援；被并 <code>episode</code> 壳置 <code>stitched</code> （仅计数不落盘，3.11.2）、幸存信封 <code>Record</code> 重绑；多碎片线索机械标定接缝（ <code>seam_indexes</code> ），M15 对接缝序数零 LLM 生成占位步（ <code>detail.kind = "thread_seam"</code> ，3.15.4）。产出三级结构 <code>thread > fragment > step</code> （ <code>_meta.stream.fragments</code> 溯源，6.3）；关闭时主输出 / <code>rejects</code> / <code>report.json</code> 与 v1.8 逐字节等价（3.16.4 退化锚）。
质量打分	可选，默认开	QuRating 双模式： <code>pairwise</code> （批内 <code>k</code> 轮随机配对 → LLM 判胜负 → BT 拟合 → 百分位归一化到 [0,1]）与 <code>pointwise</code> （0–5 加性 <code>rubric</code> 打分归一化）。 <code>Rubric</code> 用户提供或用系统默认（文本/UI 各一套）。可按聚合分阈值过滤。v1.7： <code>classify</code> 启用时批内按类分池打分， <code>rubric/门槛/选择机制</code> 均可按类覆盖（3.4.3 按类分池行）。
自动标注	可选，默认开	按 project.toml 的任务指令 + <code>few-shot</code> 示例组装提示词；UI 模式附截图（base64）与序列化 UI 树；输出受用户 JSON Schema 约束，经结构引擎（M8）保证合法。v1.2 增可选 self-consistency：同一记录以 <code>annotate.sc_temperature</code> 独立采样 <code>n</code> 次（ <code>annotate.self_consistency</code> ， $n \geq 3$ 奇数）后字段级多数投票，成本 $\times n$ （3.5.2）。

数据生成	可选，默认关	仅文本模式。以通过质量门的记录为种子示例，按生成指令产出新样本（每种子 m 条），新样本与种子及彼此做 MinHash 相似度过滤后，回流经打分、标注、校验（Self-Instruct 流程 [18]）。v1.2 增多 LLM 混合（ <code>generate.llms</code> 数组 + <code>round_robin</code> 轮转 / <code>weighted</code> 加权）与风格模板条件化（ <code>[[generate.styles]]</code> ），提升产出多样性并按 <code>llm×style</code> 桶统计（3.6.2）。v1.4 增纯生成模式： <code>run.mode="generate_only"</code> 时无输入数据，种子来自配置种子池 <code>generate.seed_examples</code> 或无种子条件化（ <code>instruction × styles + generate.standalone_count</code> ），产出照常走去重 →（打分）→（标注）→ 校验（ <code>quality</code> 与 <code>annotate</code> 至少一个启用，约束①，2.3.1）。v1.13 增时间流形态（ <code>[[generate.stream]]</code> ， <code>generate_only</code> 第三形态，默认关）：按序列配额（ <code>[class.<name>.generate].sequences × len_range</code> ）合成序列——序列一次蓝图 + 一次帧实现，噪音帧批量生成；机械布局产出可重放时间流工件与直装序列信封，再经 <code>dedup</code> → <code>classify</code> （幂等零调用）→ <code>quality</code> → <code>annotate</code> → <code>verify</code> → <code>emit</code> 。v1.16 在实际前缀有生效 <code>rules/windows</code> 时把蓝图的帧类选择、会话装箱、真交叉、时间戳与噪音槽统一交给首个 LLM 调用前的 CP-SAT 联合规划器；LLM 蓝图调用收窄为逐位置 <code>brief</code> ，帧实现后依次执行逐帧回调、 <code>correlation/time evaluator</code> 、可选序列回调与相似度过滤。单条失败只投影删除，不重新规划幸存者（3.6.5）。
二次校验	可选，默认关	LLM-as-a-Judge 用独立 <code>profile</code> 评审（记录，标注）是否合格，产出 <code>verdict</code> + 批评意见；失败策略可选丢弃或有界修复（批评意见回喂标注模型，最多 N 轮）。
结构保证	必选	输出每行必然通过用户 JSON Schema 校验，四层防线：供应商原生结构化输出 → 确定性修复 → <code>jsonschema</code> 校验 → 有界 LLM 修复环；仍失败则该记录进 <code>rejects</code> 通道，绝不写入主输出。
输出	必选	主输出 JSONL（用户结构 + 可配置 <code>_meta</code> 元信息）； <code>report.json</code> 运行报告（仅统计，无数据内容）；可选 <code>rejects</code> 通道。
日志与追踪	运行日志恒有； <code>trace</code> 可选，默认关	双通道（第 7 章）： <code>stderr</code> 运行日志（ <code>tool.log_format = "text" "jsonl"</code> ，仅运维事件，绝不含数据内容与提示词）； <code>trace</code> 追踪日志（ <code>trace.enabled=true</code> 时写 <code>{output_stem}.trace.jsonl</code> ，一行一事件，记录去重判定、逐次质量裁决与理由、评审结论等，供 <code>rubric</code> 优化与标注质量分析，内容量按 <code>trace.content</code> 四档脱敏）。日志写失败绝不中断运行。

2.1.2 不做什么（工具级负边界）

工具级边界：① 不训练/托管任何本地模型（QuRater 分类器训练不在范围内，打分全部运行时经 API 完成）；② 不做数据存储、缓存、断点续跑与跨运行状态（无数据库、无 checkpoint 文件）；③ 不做数据采集与数据版本管理；④ 不提供服务化/长驻进程形态；⑤ 不做人工标注界面（人工复核请将输出导入 Argilla 等外部平台 [5]）；⑥ 不做训练数据配比/混合（属下游职责）——含跨类输出配比（v1.7）：按类参数是加工条件化而非配额/重采样，「每类输出恰好 N 条」不做（属 8.3 O6 全局定量问题域）；⑦ 除配置声明的 LLM API 外不发起任何网络请求，无遥测；⑧（v1.13）不做重放评测回路——时间流工件与主输出可自行对账（工件行的 `truth` 字段与 `_meta.stream` 经行号双向可查），但「自动重放工件并与真值比对出分」是独立工具的职责，并不入本工具（工件重放就是一次普通的 `process` 模式运行，8.1）。v1.16 的序列规则只约束一条任务序列内部：不生成缺帧或中断 `truth`，不提供跨业务序列关系配置，`session crossing` 不是跨序列 `DECLARE`，帧仍是时间点事件而非 `duration interval`。

2.2 总体架构与模块清单

系统分四层：入口层（CLI）、编排层（M10）、算子层（M2—M7、M11、M13（v1.7）、M14/M15（v1.8）、M16（v1.9），签名统一的流水线阶段）、服务层（M1 配置、M8 结构引擎、M9 LLM 客户端、M12 日志及 `common` 层共享运行时算法，被各算子与编排共享调用）。v1.16 的 `common/runtime/declare.py`、`temporal.py`、`sequence_planner.py` 与既有 `common/extensions/hooks.py` 属共享服务面：M1、M6、M10 只能正向依赖它们，`common` 不反向导入 `operators/orchestration`。算子层内部互不依赖，只依赖服务层与公共数据结构（第 4 章），这保证了阶段可独立开关、独立测试。

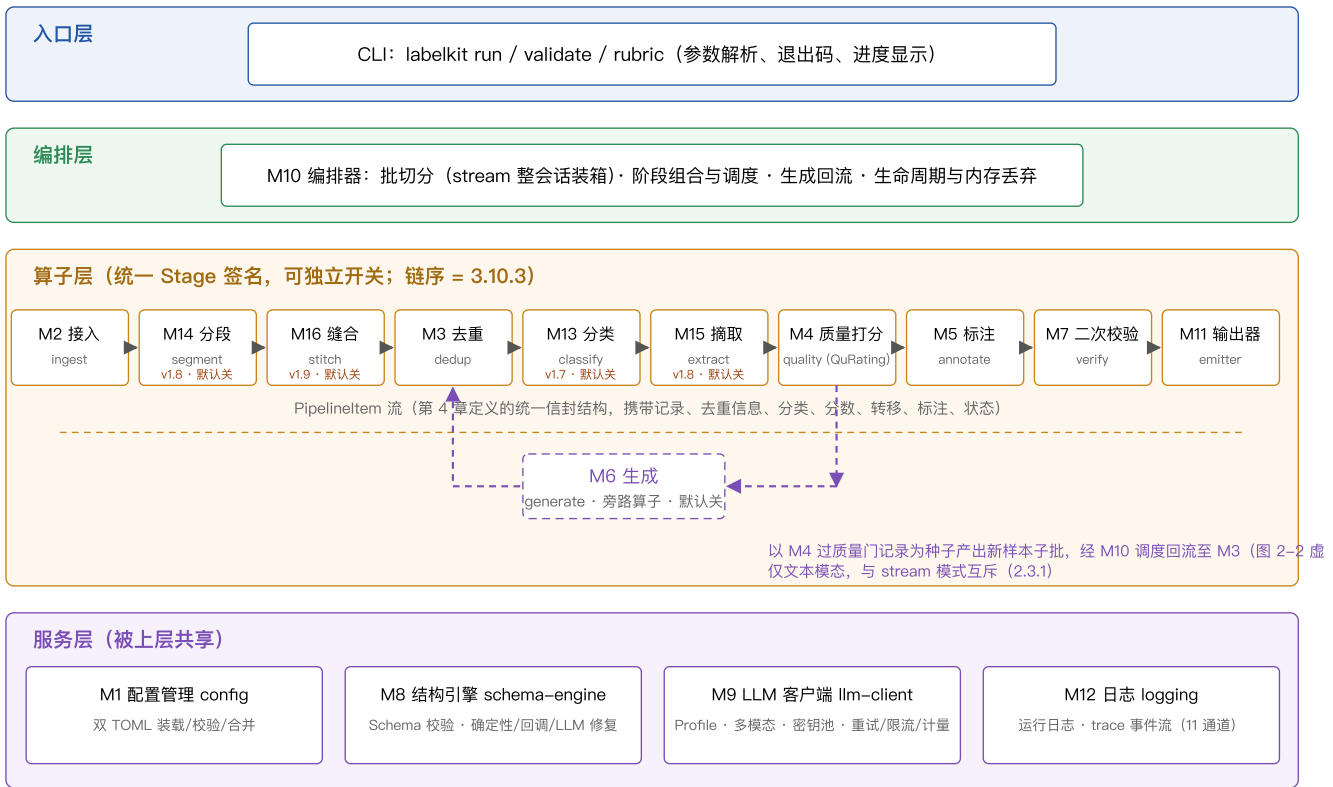


图 2-1 LabelKit 总体架构 (四层)。实线为算子层主链数据流; 紫色虚线为 M6 生成的旁路 (取种子 / 子批回流, v1.3 拆分为独立模块)。v1.7 算子层增 M13 classify (3.13, 主链工位在三 M3 与 M4 之间)。v1.8 算子层增 M14 segment 与 M15 extract (3.14、3.15, 默认关): M14 工位在主链最前 (M3 之前, 消费 M2 会话流视图产出 episode), M15 工位在三 M3 与 M4 之间 (对序列信封写入转移)。v1.9 算子层增 M16 stitch (3.16, 默认关): 工位在三 M14 与 M3 之间 (把会话内碎片缝合为线索——缝合改成员集, 先于判重与摘取)。

2.2.1 模块清单与职责边界一览

模块	职责 (做什么)	边界 (不做什么)	依赖
M1 config	装载、校验、合并两个 TOML 与 CLI 覆盖项, 产出不可变 ResolvedConfig; 解析 API Key 环境变量。v1.16 对 rules/windows/correlation/sequence hook 做聚合校验, 并用同一 seed 的独立 RNG 副本调用联合规划器, 验证每个候选长度与实际非零配额前缀。	不读输入数据; 不调用 LLM; 不修改运行期 RNG; 运行期不再被写。	common runtime
M2 ingest	解析输入路径 → 记录流; UI 文件对扫描与配对; 构造 Record (含确定性 id); 输入级合法性校验。	不做去重/打分; 不加载图像字节 (懒加载引用); 不修改原始内容。	M1
M3 dedup	精确哈希 + MinHash-LSH + pHash 判重, v1.2 增可选第④级语义判重 (经 M9 embed() 取句向量, 默认关, 3.3.3); 标记重复项并给出簇信息。	不调用对话 LLM; 语义判重仅 dedup.semantic=true 时启用 (默认零 embedding 依赖); 不物理删除 (只标记状态)。	M1 (语义级开启时另需 M9)
M4 quality	QuRating 双模式打分: 配对采样、LLM 裁决、BT 拟合、归一化; 按阈值标记低质。	不定义 rubric 内容 (来自配置/默认包); 不做标注。	M1, M8, M9
M5 annotate	组装标注提示词 (含多模态与 few-shot); 调用 LLM 获得符合用户 Schema 的标注; 可选 self-consistency 字段级投票。(v1.12) 帧粒度: frame.annotate.enabled 时在序列级标注成功后对序列信封的成员帧逐帧按帧类标注 (帧 Schema 显式路由), 结果写 item.member_annotations (3.5.5)。	不校验结构 (委托 M8); 不评审质量 (M4/M7 职责); 不产出新记录 (M6 职责)。	M1, M8, M9
M6 generate	以种子 (process 模式: 过质量门记录; generate_only 模式: 配置种子池或无种子条件化, 3.6.2) 按 llms × styles 组合产出新样本 Record (含 generator 溯源), 交 M10 回流。时间流形态改产序列与工件行; v1.16 约束路径重放共享联合计划, 按固定骨架做 brief/realize、声明式 evaluator、回调、幸存投影、噪音过滤、时间字段回填与重发装配 (3.6.5)。	仅文本模态; 单轮回流不递归; 不去重/打分/标注; 不写盘; 不在 LLM 后改变 word/session/timestamps, 也不做失败重解、重试抽样或替代排程。	M1, M8, M9, common runtime
M7 verify	LLM-as-a-Judge 评审标注; 失败按策略丢弃或驱动有界修复。	不直接改标注 (修复仍由 M5 重新标注、M8 校验)。	M1, M8, M9

M8 schema-engine	用户 Schema 装载与预校验；LLM 原始输出 → 合法 JSON 对象的四层保证；裁决输出等内部小结构的校验。v1.16 新增 <code>brief_schema(length)</code> ，固定逐位置只允许 <code>brief</code> ，不再让约束路径的 LLM 决定帧类。	不组装业务提示词；不发起首次 LLM 调用（只驱动修复调用）；不求解序列规则。	M1, M9
M9 llm-client	Profile 化的统一 LLM 访问：OpenAI 兼容/Anthropic 两类 provider、多模态消息、结构化输出参数、指数退避重试、并发信号量、token/成本计量。	不理解业务语义；不解析业务结构（返回原始文本/原生结构化结果）。	M1
M10 orchestrator	批切分、阶段组合（按开关）、生成回流调度、运行级统计聚合、生命周期与中间态丢弃。v1.16 的 <code>estimate_run</code> 复用与 M1/M6 相同的计划构造/求解入口，报告显式装配规则、回调与日历统计块。	不含任何阶段业务逻辑；不直接调用 LLM；不自行重写规划算法或推算另一套调用计划。	全部
M11 emitter	主输出/rejects/报告三通道写出； <code>_meta</code> 组装；增量落盘。（v1.13）时间流生成形态另有第五通道：把 M6 交付的工件行写出为 <code>{output_stem}.stream.jsonl</code> ，与主输出同批 <code>fsync</code> + 原子改名（3.11.2）。	不校验结构（到达此处的标注已合法）；报告不含数据内容；不生成工件内容（行由 M6 定稿，M11 只负责落盘与交付纪律）。	M1
M12 logging	进程内唯一日志设施：stderr 运行日志（标准 logging, text/jsonl 两种格式）；trace 事件流 <code>EventLog</code> （JSONL，行缓冲，每批随 M11 flush 同步 flush），由 <code>MetricsSink</code> 持有、各 Stage 经 <code>RunContext.metrics</code> 发事件。	不做跨运行聚合分析（后续 <code>analyze</code> 工具职责，8.3 O5）；不上传遥测；写失败不中断运行（warn 一次并关闭通道，计入 report）；API Key 永不落日志。	M1
M13 classify (v1.7)	按用户类别表对批内存活记录做 LLM 封闭集分类（单/多标签可配，可选 <code>self-consistency</code> 投票）；结果写 <code>item.classification</code> ；multi 模式按标签向批尾扇出兄弟信封（3.13）。（v1.12）帧粒度： <code>frame.classify.enabled</code> 时对流模式序列信封的成员帧做批量闭集判决（帧类表独立于序列列表、恒单标签），结果写 <code>item.member_classifications</code> （3.13.7）。	不淘汰记录（分类不是质量门；multi 扇出只增不减）；不定义类别语义（来自配置）；不做标注（用户 Schema 产出物属 M5）；不改链结构（扇出只改批内信封基数）。	M1, M8, M9
M14 segment (v1.8)	把批内候选会话精化为 episode：可选 LLM 滑窗边界裁决与逐帧噪声标记；成员信封置 <code>absorbed</code> 、噪声帧置 <code>dropped_noise</code> ，按序键拼装序列 Record 并尾部追加 episode 信封（契约 ②b, 4.3；3.14）。	不判重（M3）；不推断动作（M15）；不打任务标签（M5）；不改链结构。	M1, M8, M9
M15 extract (v1.8)	对每个 active 序列信封的每对相邻成员帧 <code><s_i, s_{i+1}></code> 经 LLM 产出结构化动作（内部 Schema），写入 <code>item.transitions</code> ；转移数 = 成员数 - 1（3.15）。	不重分段（M14 上游）；不产出用户 Schema 字段（M5）；不淘汰记录。	M1, M8, M9
M16 stitch (v1.9)	把会话内碎片保守缝合为线索：单调选池 LLM 判定 × 机械先验合取 + 有界二遍复评；被并 episode 壳置 <code>stitched</code> 、幸存信封 Record 重绑、 <code>below_min_len</code> 短段救援翻转（契约 ②c, 4.3）；机械标定 <code>seam_indexes</code> （3.16）。	不重分段（M14）；不摘取动作（M15）；不判重（M3）；不跨会话/跨批；不做帧多重归属。	M1, M8, M9

2.3 端到端数据流与阶段开关

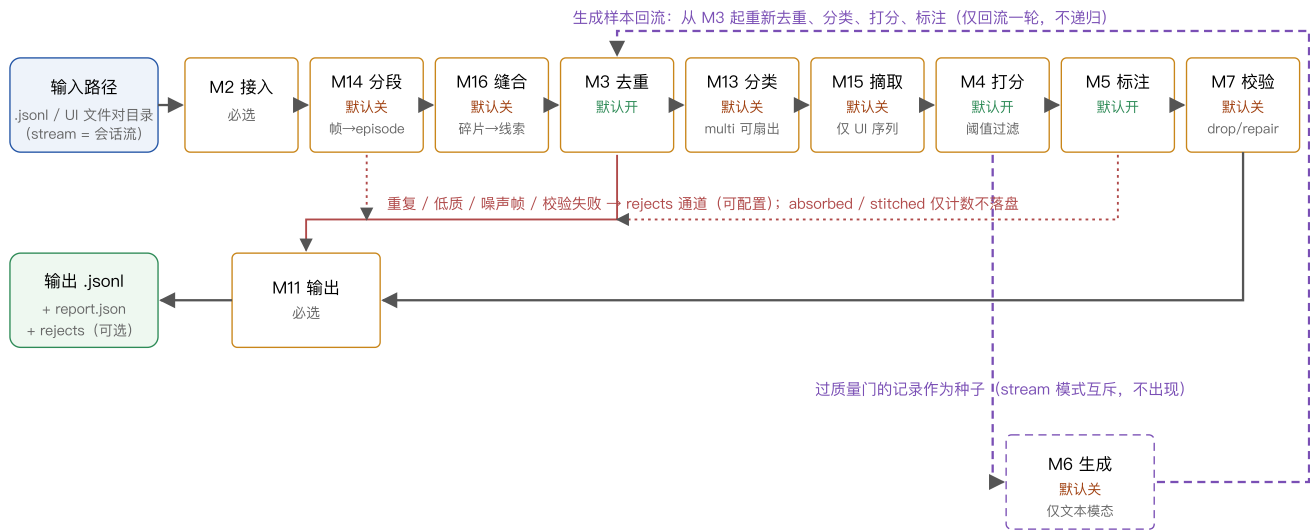


图 2-2 端到端数据流（process 模式）。实线为主路径；紫色虚线为可选的生成回流；红线为淘汰通道。generate_only 模式（v1.4）无输入与 M2：M6 为链路起点，生成样本按 batch_size 切批后自 M3 起走同一主路径（3.10.3）。v1.7：classify 启用时主路径在 M3 与 M4 之间插入 M13 分类工位（3.13）——multi 扇出在该工位内向批尾追加兄弟信封，回流子批因继承分类而幂等跳过。v1.8：segment 启用（stream 模式）时 M2 产出的是会话流、M10 重整会话装箱，主路径在批首插入 M14 分段工位（成员帧收拢为 episode 信封，②b）、在 M13 与 M4 之间插入 M15 摘取工位（对序列信封写入转移）——链序为 segment → stitch → dedup → classify → extract → quality → annotate → verify（stitch 为 v1.9 增位，默认关；3.10.3；generate 与 stream 互斥故回流旁路不出现）。v1.9：stitch 启用时主路径在 M14 与 M3 之间插入 M16 缝合工位（碎片缝合为线索——被并壳置 stitched、幸存信封重绑、短段救援翻转，②c，3.16）。v1.13：generate_only 的时间流形态下 M6 仍是链路起点，但产物是直装序列信封，不经 segment 二次分段；工件行交 M11，序列信封自 M3 起走下游。v1.16 约束路径在 M6 内容调用前增加一次全流联合规划屏障：配额前缀 → 条件长度抽样 → CP-SAT 冻结帧类词/会话/时间/噪音槽 → brief/realize → 回调与规则 evaluator → 幸存投影/重发 → 直装信封与工件；下游链与 v1.13 相同，规划不是一个新的 Stage（3.6.5、3.10.3）。

2.3.1 阶段开关矩阵

阶段组合由 project.toml 各节的 enabled 决定。合法组合与典型用法：

dedup	quality	generate	annotate	verify	典型用法
✓	✓	—	✓	—	默认：清洗 + 打分 + 标注
✓	✓	—	—	—	纯数据治理：只去重打分，输出过滤后的原始数据 + 分数
✓	✓	✓	✓	✓	全流程：治理 + 扩充生成 + 标注 + 评审（成本最高，质量最高）
—	—	—	✓	✓	纯标注：数据已治理过，只做标注与评审
✓	可选	✓	可选	—	纯生成（run.mode="generate_only"，v1.4）：无输入从零合成 → 治理 → （标注） → 输出

约束：① annotate 与 quality 至少启用一个，否则运行无产出意义，M1 在启动时报 CONFIG_ERROR；② verify.enabled=true 要求 annotate.enabled=true；③ generate.enabled=true 要求 run.modality="text"；process 模式下另要求 quality.enabled=true（种子来自质量门），generate_only 模式下 quality 可选（种子来自配置，3.6.2）；④ run.mode="generate_only"（v1.4）要求 generate.enabled=true 且 run.input 缺省（提供即报 CONFIG_ERROR，3.1.4），annotate 可选。（v1.7）classify.enabled（默认 false，5.2）与上表各开关正交：分类不改变组合合法性，任意合法组合均可叠加分类（multi 扇出后的每个信封走同一阶段组合；generate_only 链亦含 classify——生成产物被正常分类，3.10.3）；classify.enabled = false 而 [[classify.classes]] 或 [class.*] 在场不报错——M1 打 warning（一次、点名被忽略的表；「留配置、关开关」合法，3.1.4）。（v1.8）stream 组合约束：segment.enabled=true 要求 run.mode="process" ∧ generate.enabled=false（含 generate_only 的传递闭合——stream × generate 互斥） ∧ annotate.enabled=true（序列记录无 passthrough 输出形态）；extract.enabled=true 要求 segment.enabled=true ∧ run.modality="ui"；stream 与 classify 正交（episode 照常分类并驱动按类条件化）；[stream] / [segment] / [extract] 在场而 segment.enabled=false 同上打 warning（3.1.4）。（v1.9）stitch 组合约束三条：① stitch.enabled=true 要求 segment.enabled=true（缝合的输入是 episode——stream 前置约束经此传递闭合，[stitch] 名单同入上句 segment 关闭时的 no-op warning）；② stitch.votes 须为 1 或 ≥3 的奇数（偶数报 CONFIG_ERROR，多数决需破平局，3.1.4）；③ stitch.enabled=true ∧ segment.strategy="rules" ⇒ warning（非阻断：规则粗切段未经语义精化，缝合证据质量下降的组合提示）；另有单独 no-op warning——segment.enabled=true ∧ stitch.enabled=false 而 [stitch] 有 payload（annotate.sequence_frames 同形制，3.1.4）。（v1.12）帧粒度组合约束七条（本节为规范源头，3.1.4 校验规则表引用之）：① 帧粒度要求流模式——frame.classify.enabled v frame.annotate.enabled ⇒ segment.enabled=true（帧粒度仅流模式可用；报错文案指引「非流模式请改用 classify + [class.<name>.annotate] 按类标注」）；② 帧类覆盖要求帧分类——[frame.class.*] 在场 ⇒ frame.classify.enabled=true，且节名 ⊆ [[frame.classify.classes]] 类表、覆盖白名单仅 annotate 节三键（instruction/examples/enabled），白名单外键/节 ⇒ CONFIG_ERROR；③ 帧 Schema 恰一——

`frame.annotate.enabled=true` $\Rightarrow$ `schema_path / schema_inline` 恰一 + draft 2020-12 元校验 + examples 干跑 (镜像 `output.schema` 全套分支, 3.1.4); ④ **meta_mode 护栏**——`frame.*` 任一启用 $\Rightarrow$ `output.meta_mode != "none"` (帧产物仅经 `_meta.stream.members` 承载, "none" 将丢弃全部帧产物; sidecar 合法); ⑤ **fallback 合法**——`frame.classify.enabled=true` 时 `fallback_class` 必填且 $\in$ 帧类表 name 集 (传递性地要求类表非空; 帧类表与序列类表相互独立、允许重名、互不约束); ⑥ **定向探针**——`[frame.classify].assignment` 与 `[frame.annotate].self_consistency` 显式书写 $\Rightarrow$ 定向 `CONFIG_ERROR` (帧级无多标签、无自洽采样; v1.11 `use_vision` 原始节探针同款机制与文案风格); ⑦ **no-op 提示**——`[frame.*]` 节在场 $\wedge$ 均未启用 $\wedge$ `segment.enabled=false` $\Rightarrow$ 并入 `segment` 关闭时的 no-op warning 停放清单 (任一帧开关启用时由约束①的 `CONFIG_ERROR` 接管, 不再重复告警)。

(v1.13) **时间流生成组合约束** (本节为规范源头, 3.1.4 校验规则表引用之; 全部 `CONFIG_ERROR`): ① **形态前提合取**——`generate_stream.enabled = true` $\Rightarrow$ `run.mode="generate_only"` $\wedge$ `run.modality="text"` $\wedge$ `generate.enabled=true` $\wedge$ `classify.enabled=true` (序列类表是配额与按类条件化的载体, 生成侧标签直接继承) $\wedge$ `stream.order_by = "meta:<字段>"` (该字段即工件行时间戳字段, 摄取侧按同一声明可重放) $\wedge$ `output.meta_mode != "none"` (帧类真值与成员对账仅经 `_meta.stream` 承载, sidecar 合法——v1.12 帧粒度护栏的镜像) $\wedge$ **工件键守卫**——`input.text_field` 与时间戳字段名均不得含 "." (工件行以字段名为字面顶层键, 点路径在重放摄取时无法往返)、互不同名、且均不得为 "truth" (工件行三个顶层键互斥, 6.5); ② **类表与配额**——序列类表 ≥ 1 项 (放宽自 ≥ 2 , 仅本形态) $\wedge$ `classify.fallback_class` 免填 (写了仍须 $\in$ 类表; 两条放宽的保护对象是判决路径, inherited 形态无判决路径) $\wedge$ 至少一类有效 `sequences` ≥ 1 $\wedge$ 参与类 (有效 `sequences` ≥ 1) 的有效生成指令非空; 帧类表 `[[frame.classify.classes]]` 非空 $\wedge$ 每个帧类都有非空的 `[frame.class.<name>.generate].instruction` (蓝图 enum 覆盖全类表, 任一帧类都可能被选中); ③ **禁设键探针**——`[generate]` 的 `seed_examples / standalone_count / num_per_record / seeds_per_call` 与 `[class.*.generate]` 的 `num_per_record / seeds_per_call` 显式书写 $\Rightarrow$ 定向 `CONFIG_ERROR` (属平面生成形态; 配额改由 `sequences / len_range` 承载), `frame.classify.enabled / frame.annotate.enabled = true` $\Rightarrow$ 定向 `CONFIG_ERROR` (与本形态互斥——帧类真值在蓝图层已知, 无需帧级判决; 机制均为 v1.11 `use_vision` 的原始节探针); ④ **装箱一致性**——`sessions` ≥ 1 $\wedge$ `sessions` $\leq \Sigma sequences \leq 2 \times sessions$ (交叉会话数 = $\Sigma sequences - sessions$, 交叉并发度恒 $k \in \{1,2\}$) $\wedge$ `duplicates` $\in [0, \Sigma sequences]$ $\wedge$ `noise_ratio` $\in [0,1)$ ($> 0 \Rightarrow$ `noise_instruction` 非空) $\wedge$ `frame_gap_s` 满足 $0 < lo \leq hi < stream.gap_s$; ⑤ **织造上限**—— $2 \times \max(\text{各类 } len_range \text{ 上界}) \leq stream.session_max_len$ (交叉会话恒装两条序列) $\wedge$ `stream.key == []` $\wedge$ `stream.gap_steps == 0` (分区键与序差断开同生成侧铺设契约冲突) $\wedge$ `session_max_span_s > 0` 时按最坏帧间隔做静态跨度校验 $\wedge$ `ts_start` 可解析为 ISO-8601 时刻; ⑥ **Schema 元校验**——按序列类标注 Schema (`[class.<name>.annotate].schema_path / schema_inline`, 至多其一) 与帧类生成 Schema (`[frame.class.<name>.generate]`, 同至多其一) 走 `output.schema` 同款全套分支 (draft 2020-12 元校验 + 顶层 object; 按类标注 Schema 另加 `_meta` 保留键禁令 + `$ref` 可解析性遍历 + 按类 `few-shot` 干跑); ⑦ **v1.12 帧粒度约束①②的放宽**——约束①「帧粒度要求流模式」不受影响 (本形态不启用 `frame.classify / frame.annotate`); 约束②「帧类覆盖要求帧分类」放宽为 `frame.classify.enabled v generate_stream.enabled`, 且 `[frame.class.<name>.generate]` 节仅本形态合法 (非本形态出现是反向定向 `CONFIG_ERROR`, 指引改写 `[frame.class.<name>.annotate]`); ⑧ **停放豁免精确化**——本形态下 `[stream]` 节是生效面 (生成侧铺设契约)、`[frame.*]` 节是生效面 (帧类表与帧类生成契约), 两者移出 R8 no-op 停放清单; `[segment] / [stitch] / [extract]` 照旧停放告警。

(v1.14) **帧类构成档位与时间字段绑定组合约束** (本节为规范源头, 3.1.4 校验规则表引用之; 两簇均只在时间流生成形态内生效, 档位表与绑定表缺省时零新代码路径、与 v1.13 字节等价):

档位簇 (v1.15 逐生效表化注记: 下列 ②③ 的辖区是一张生效档位表、④⑤⑥ 的辖区是逐参与类的生效表, 「生效表」= 该类声明的按类表、未声明则为全局表——见下方 v1.15 段; 无任何按类表时生效表恒 = 全局表, 本簇与 v1.14 逐字同义)——① **档位表前提**: `[[generate.stream.tiers]]` 在场 $\Rightarrow$ `generate_stream.enabled = true` (定向 `CONFIG_ERROR`: 本表仅时间流形态合法); ② **档位身份**: `tier_rank` 正整数、表内唯一、每张生效表各自连续覆盖 1..N (N = 该表长, 逐类可不同; 缺号/重号 = `CONFIG_ERROR`), `weight` 整数 ≥ 1 ; ③ **档位构成**: `frame_classes` 非空、档内无重复、每名 $\in [[frame.classify.classes]] 名集, 且单表内各档构成集合两两互异 (同构成即语义重复; 跨表——即跨类——同构成合法); ④ **长度可覆盖**: 逐 (参与类, 档) 非零配额对裁定——配分是纯函数、M1 期可算——配额 ≥ 1 的每一对须满足该类 `len_range` 下界 $\geq len(\text{该档 } frame_classes)$ (档内每类至少出现一次, 步数不足即装不下; 档取自该类生效表), **零额对豁免** (不为永永尝试的组合抬高下界); ⑤ **配分零额 WARN**: 某 (参与类, 档) 按整数域最大余额法配额为 0 $\Rightarrow$ WARN (值-free: 类名 + `tier_rank` + 该类生效表的权重表; 非错误——报表 `tiers` 子块会如实呈现 planned 0); ⑥ **帧类未入档 WARN 与必填域收窄**: 档位表在场时, 某帧类不属于任何参与类生效表的任何档 `frame_classes` $\Rightarrow$ WARN (该帧类不会出现在任何蓝图中, 其 `[frame.class.<name>.generate]` 面成为死配置), 同时上文 v1.13 约束②的「每个帧类都有非空生成指令」检查域收窄为 U (各参与类生效表的 `frame_classes`) (蓝图可能选中的闭集; 未入档帧类豁免必填, 已写照常合法)。$

绑定簇——⑦ **绑定表前提**: `[frame.class.<name>.generate.time_fields]` 仅当该帧类声明了 `schema_path / schema_inline` (结构化帧) 时合法, 纯文本帧带绑定表 $\Rightarrow$ 定向 `CONFIG_ERROR`; 「载荷恒为 JSON 对象」(回填就地写入的前提) 由 v1.13 既有的帧类生成 Schema 顶层 `"type": "object"` 无条件必检承担, 绑定簇对「声明过 Schema 源键但装载失败」的帧类保持沉默、不叠加第二错; ⑧ **绑定键与类型**: 每个绑定键 $\in$ 该帧类生成 Schema 顶层 `properties`; 绑定值 $\in$ 语义词表 `{ts, gap_prev_s, gap_next_s, elapsed_s}`; 声明类型匹配 = 该属性 Schema 的 `type` 关键字字面恰等于要求值 (`ts` $\Rightarrow$ "string"、其余三值 $\Rightarrow$ "number"; 联合类型数组、缺失、经 `$ref` / 组合关键字间接声明均判不匹配, 定向 `CONFIG_ERROR`), 绑定字段上携带 `type` 以外的约束关键字 $\Rightarrow$ WARN (那些关键字既不上行也不被强制——时间量的值域由时间轴决定); ⑨ **剔除余量**: 生成 Schema 顶层 `properties` 键数 - 绑定键数 ≥ 1 (LLM 至少有一个字段可生成; 全绑定 $\Rightarrow$ `CONFIG_ERROR`)。

另附 v1.13 形态级缺陷修补一条——⑩ **微秒地板**: `frame_gap_s` 的 $0 < lo \leq hi < stream.gap_s$ 收紧为 $1e-6 \leq lo \leq hi < stream.gap_s$ 。亚微秒 `lo` 下 `timedelta` 会取整为 0 微秒, 使 v1.13 「ts 严格递增」的声明出现破口, v1.14 语义词表的 0.0 边界哨兵更不容真零间隔; 报错文案给出

这两条依据。对 $lo \geq 1e-6$ 的一切现存工程零影响（全部示例 $lo = 5$ ），亚微秒配置在 v1.13 本就产出缺陷数据。

v1.16 条件修订——微秒地板后的 v1.15 默认路径仍要求 $1\mu s \leq lo \leq hi < stream.gap_s$ ；仅当 `--limit` 后的实际非零配额前缀存在生效 rules/windows 时，v1.16 联合规划路径才允许 $hi == stream.gap_s$ ，即采用 $hi \leq stream.gap_s$ 的 replay guard。只有 `sequence_validator` 而没有该类生效 rules/windows 时，不激活此例外，仍执行严格 $<$ 。

（v1.15）**按类档位表组约束**（本节为规范源头，3.1.4 校验规则表引用之；契约侧登记为 CONTRACTS §6.3 rule 61）：`[[class.<name>.generate.tiers]]` 是 v1.14 档位面的按类化增量——行结构与逐行校验同全局表（上文档位簇 ②③ 改逐生效表执行），语义为表级原子覆盖（类声明了就用类的整张表，未声明回落全局表）。本簇只在时间流生成形态内生效，按类表全部缺省时零新代码路径、与 v1.14 字节等价。新增按类表前提三款，全部 `CONFIG_ERROR`，定位串带类名：

① **形态门**——`generate_stream.enabled = false` 时任一 `[[class.?.generate]]` 节写了 `tiers` $\Rightarrow$ 定向 `CONFIG_ERROR`（机制 = v1.11 `use_vision` 的原始节探针，不走「未知键报 warning」前向兼容路径；探针读原始节，故表内容本身非法时也照发本错），文案点明「本表仅时间流生成形态（`[[generate.stream].enabled = true`）合法，它为该类序列覆盖全局 `[[generate.stream.tiers]]`」；② **全局锚**——形态开启、任一按类表在场而全局 `[[generate.stream.tiers]]` 缺省 $\Rightarrow$ `CONFIG_ERROR`，文案点明「按类表覆盖的是全局表，而全局表缺席——请声明全局表：它既是未声明类的回落，也是整个档位面的开关」（全局表为锚：档位面总开关恒 = 全局表非空 $\Rightarrow$ 每个参与类恒有生效表，标识三点的在场性恒定不逐行漂移）；③ **空表拒收**——`tiers = []`（显式空表） $\Rightarrow$ `CONFIG_ERROR`，文案指引「删除该键以回落全局 `[[generate.stream.tiers]]`」（面统一之下不存在「本类无档」态；「本类不分档」的表达式是给它一张单档表）。②与③互斥（同一个键一条错误一个修复动作——空表的修复是删键、锚缺失的修复是补全局表，叠报会误导）；键值非数组的形状错误在解析层报出后按未声明落库，本簇任何错都不叠报（实现期裁决 2026-08-19）。

连带的逐生效表化修订（语义已在上文档位簇内就地标注，此处只列辖区）：档位身份与构成的结构校验逐生效来源表各跑一遍（全局表 + 每张已声明按类表，报错定位前缀分别为 `[[generate.stream.tiers]]` 与 `[[class.<name>.generate].tiers`；跨表同构成合法），配额对约束与零额 WARN 逐参与类改吃本类生效表，「帧类未入档」WARN 与「每帧类生成指令必填」的检查域并集化为 $\cup$ （各参与类生效表构成）。另有一条辖区分割——零额类不豁免结构校验：`sequences = 0` 的类若声明了按类表，上述结构校验照做（坏配置早报），仅配额对约束与零额 WARN 豁免，且其构成不入检查域并集（该类不产序列）。

（v1.16）**时间流序列规则组约束**（本节为总体规范源头，字段级定义见 5.2，执行语义见 3.6.5）：

- **形态门与停放规则**——任一 `[[generate.stream.rules]]`、`[[generate.stream.windows]]`、`[[class.<name>.generate.rules]]`、`[[class.<name>.generate.windows]]` 或 `[[generate].sequence_validator` 只在 `generate_stream.enabled = true` 的时间流 `generate_only` 形态合法；形态外不是 warning/no-op，而是定向 `CONFIG_ERROR`。规则与日历窗的按类表分别独立采用三态整表覆盖：未声明继承、显式空表清空、非空表替换；没有 tiers 的全局锚约束，类表可单独存在。
- **参与前缀与零配额边界**——规划激活只看 `--limit` 后实际非零配额前缀是否存在生效 rules/windows；被截断为零的约束类不激活全流规划或报告。零配额类仍执行表形、Schema、模板和每个候选长度的本地潜力校验，坏配置不得因零配额逃逸。
- **规则行闭集**——模板恰为 15 个标准有限迹 DECLARE 名称；使用 `end` 而非 `last`，无别名。二元模板要求 source 与 target 不同；`existence / absence / exactly` 必须且只能携正整数 count；其他模板禁止 count。source/target 必须属于帧类闭集；完全相同的重复规则是 `CONFIG_ERROR`。
- **时间与 correlation 静态约束**——`time_s` 可为无损量化到整数微秒的 $[lo, hi)$ ，满足 $1\mu s \leq lo < hi$ ；correlation 只允许 `operator = "equal"`，两个字段必须分别是结构化帧 Schema 顶层必填属性、类型字面完全相同，且不属于 `time_fields` 绑定字段。带 correlation 的最坏帧实现 prompt 必须能一次装入 context window，否则 M1 报 `CONFIG_ERROR`，不允许运行期拆分替代。
- **日历窗静态约束**——同一生效表中每个帧类最多一行；`of_day` 必填且非空，端点支持 `HH:MM / HH:MM:SS` / 微秒精度，逐段半开、同一自然日内 `start < end` 且互不重叠；`of_week` 缺省全周，只允许 `mon` 至 `sun` 且无重复。逻辑跨午夜窗不支持。
- **每长度与全流证明**——M1 对每个类的每个候选长度验证局部结构/时间潜力，不以「至少一个长度可行」代替；随后以同一 seed 的独立 RNG 副本，对实际前缀调用与 `M6/estimate_run` 相同的问题构造与求解入口。模型 proto 的变量数与约束数之和超过 250,000 即 `CONFIG_ERROR`。
- **求解状态 fail closed**——M1 收到 `INFEASIBLE` 报配置不可满足；`UNKNOWN` 只说明固定确定性预算内无法验证，错误不得声称 unsatisfiable；`MODEL_INVALID` 转 `InternalError`（退出码 4）。普通问题接受 `FEASIBLE/OPTIMAL`，噪音槽最大化问题只接受 `OPTIMAL`。M6 若在 M1 已通过后出现非接受状态，发 value-free `ERROR` 并以 `InternalError` 终止，不生成替代结果。
- **默认关闭锚**——没有实际非零配额生效 rules/windows 时，M1/M6/estimate 均走 v1.15 原分支；若同时没有 `sequence_validator`，其 RNG 消费、提示词、调用数、主输出、rejects、report、工件与 trace 全部保持 v1.15 逐字节等价（既有 `sample_validator` 单独配置也不得新增报告键）。仅配置 `sequence validator` 而无 rules/windows 不激活 CP-SAT，但按冻结验证顺序执行回调并出现对应报告计数。

2.3.2 算子对输出集的影响分析

设某批进入流水线的存活记录数为 N （全流程视角对应 6.4 的 counts 不变量 `emitted + dropped_* + failed + bad_input = scanned + generated`；熔断中止时左侧另加 `unprocessed`，v1.6，6.4）。每个算子对最终输出集的影响从四个维度刻画：基数（条数如何变）、内容/构成（行内容与数据分布如何变）、判定依据的落点（决策证据写入哪个 `_meta` 字段 / 哪个 trace 事件，事后可审计）、被淘汰记录的去向。总原则先行：主输出只含存活记录；`dropped_dup / dropped_lowq / dropped_verify / failed` 一律不入主输出、按 `output.rejects` 进拒绝通道

("none" | "refs" (默认) | "full", 写出规格见 3.11.2); v1.8 增两态——`dropped_noise` 同走 `rejects`, `absorbed` (成员并入 `episode`) 为第三路由: 主输出与 `rejects` 均不写、仅计数 (3.11.2); v1.9 增一态——`stitched` (被并 `episode` 壳) 为第四路由: 同 `absorbed` 仅计数 (其成员随幸存线索信封落盘, 3.11.2)。

算子	基数影响 (N→?)	对内容/构成的影响	判定依据落点 (<code>_meta</code> / <code>trace</code>)	淘汰去向
M2 接入	<code>scanned</code> → <code>ingested</code> = <code>scanned</code> - <code>bad_input</code> ; 只减不改。	按 <code>input.text_field</code> 抽取文本 / 配对 UI 文件构造 <code>Record</code> (3.2.4–3.2.5), 不改动数据内容。	<code>_meta.source</code> { <code>file</code> , <code>line_no</code> <code>pair_index</code> }; <code>trace ingest.bad_line</code> / <code>ingest.missing_pair</code> / <code>ingest.index_conflict</code> 。	坏行/缺对未构成 <code>Record</code> , 不走 <code>rejects</code> 通道——仅计 <code>report.counts.bad_input</code> (策略为 <code>fail</code> 时直接退出码 3)。
M14 分段 (v1.8)	吸收与追加: 批内 <code>N</code> 帧收拢为 <code>E</code> 个 <code>episode</code> 信封, <code>N</code> → <code>N</code> - <code>absorbed</code> - <code>dropped_noise</code> + <code>E</code> (成员帧置 <code>absorbed</code> 并入序列信封、噪声/短段帧置 <code>dropped_noise</code> ; <code>episodes</code> 由 M10 按 <code>len</code> 差计量, 守恒式扩展见 6.4)。	输出集单位从「帧」升维为「序列记录」(<code>kind="sequence"</code> , 成员经 <code>members</code> 引用共享): 主输出行承载整段任务序列而非单帧, 帧数分布变为段长分布。	<code>_meta.stream</code> { <code>episode_id</code> , <code>session_id</code> , <code>member_ids</code> , <code>member_sources</code> , ...} (未启用 = <code>null</code> , 6.3); <code>trace segment.session</code> / <code>segment.boundary</code> (7.2)。	<code>dropped_noise</code> ⇒ <code>rejects</code> (<code>stage="segment"</code> , <code>reason="noise"</code> " <code>below_min_len</code> ", 3.11.2); <code>absorbed</code> 为第三路由 (不入主输出、不入 <code>rejects</code> 、仅计数)。
M16 缝合 (v1.9)	合并与翻转: <code>E</code> 个 <code>episode</code> 缝合为 <code>T</code> 个线索, <code>E</code> → <code>E</code> - <code>stitched</code> (被并 <code>episode</code> 信封壳置 <code>stitched</code> 、幸存信封 <code>Record</code> 重绑; <code>threads</code> = <code>episodes</code> - <code>stitched</code> , M10 导出式计量, 3.10.3); 救援命中另使 <code>dropped_noise</code> 帧翻回 <code>absorbed</code> (<code>rescued_short</code> , 帧口径)。	输出集单位从「 <code>episode</code> 」升维为「线索」: 交叉任务合并为一行 (碎片结构入 <code>_meta.stream.fragments</code>)、行数下降: 救援使业务短段尾帧回归主输出; 接缝步以占位形态进入 <code>steps</code> 序列 (<code>detail.kind="thread_seam"</code> , 3.15.4)。	<code>_meta.stream</code> { <code>thread_id</code> , <code>fragments</code> [], <code>steps</code> 行内 <code>resumed</code> } (仅启用时在场, 6.3); <code>trace stitch.judge</code> / <code>stitch.thread</code> (7.2)。	<code>stitched</code> 为第四路由 (不入主输出、不入 <code>rejects</code> 、仅计数, 3.11.2); 救援未命中帧维持 <code>dropped_noise</code> 原去向; <code>on_error="fail"</code> 时 <code>episode</code> 候选信封 <code>failed</code> ⇒ <code>rejects</code> (<code>kind=stitch_invalid</code> , 7.6)。
M3 去重	<code>N</code> → <code>N</code> - <code>dup</code> ; 只减不改 (存活行内容不变)。	簇内仅留首见记录 (<code>first-writer-wins</code> , 3.3.3), 压缩高冗余来源、改变数据分布; <code>dedup.semantic = true</code> (默认 <code>false</code>) 时判重面扩至改写型语义近重 (嵌入余弦相似度 ≥ <code>dedup.semantic_threshold</code> , 默认 0.95, <code>SemDeDup</code> [26]), <code>dup</code> 增大、输出集单位条数的多样性提高。	存活者 <code>_meta.dedup.kind="unique"</code> ; 被淘汰者 <code>DedupInfo</code> { <code>kind</code> , <code>cluster_key</code> , <code>kept_id</code> } (4.2); <code>trace dedup.duplicate</code> 。	<code>dropped_dup</code> ⇒ <code>rejects</code> (<code>refs</code> 档仅 <code>_meta</code> 引用行, 3.11.2)。
M13 分类 (v1.7)	<code>single</code> : 基数不变 (每条 <code>active</code> 记录写入类标签); <code>multi</code> : 增——归一化后命中 <code>k</code> ≥ 2 类的记录向批尾扇出 <code>k-1</code> 个兄弟信封, <code>N</code> → <code>N</code> + <code>fanout</code> (3.13.4)。	不改记录内容; 类标签驱动下游按类条件化 (按类 <code>rubric</code> /门槛/指令/种子池, 3.4.3/3.5.2/3.6.2/3.7.2), 间接改变输出集组成; <code>multi</code> 下一条输入至多产出 <code>classify.max_labels</code> 行, 消费侧行唯一键变为 (<code>_meta.id</code> , <code>_meta.classification.label</code>) (6.3)。	<code>_meta.classification</code> { <code>label</code> , <code>labels</code> , <code>source</code> } (未启用 = <code>null</code> , 6.3); <code>trace classify.decision</code> (7.2)。	—— (分类不淘汰记录: 结构失败默认归兜底类仍存活; 仅 <code>classify.on_error="fail"</code> 时记录 <code>failed</code> ⇒ <code>rejects</code> , 3.13.4)。
M15 摘取 (v1.8)	基数不变: 对每个 <code>active</code> 序列信封写入 <code>item.transitions</code> (转移数 = 成员数 - 1); 仅 <code>extract.on_error="fail"</code> 时 <code>episode</code> 置 <code>failed</code> 减 (默认 <code>fallback</code> 该步记 <code>other</code> 留痕、 <code>episode</code> 存活, 3.15)。	不改记录内容; 步骤序列落 <code>_meta.stream.steps</code> 并注入下游 <code>quality/annotate/verify</code> 提示词作机械锚点, 间接决定序列标注与打分的证据质量。	<code>_meta.stream.steps</code> ; <code>trace extract.step</code> (7.2; <code>target/value</code> 属输入数据派生, 按 <code>_DATA_KEYS</code> 分档脱敏, 7.4)。	默认不淘汰 (<code>fallback</code> 留痕); <code>on_error="fail"</code> 时 <code>failed</code> ⇒ <code>rejects</code> (<code>kind=extraction_invalid</code> , 7.6)。

M4 打分	打分本身不减：每条 active 记录写入分数（各 criterion + <code>__aggregate__</code> ），基数不变；门控/选择才减——配置 <code>quality.threshold</code> （聚合分低于线 $\Rightarrow$ <code>dropped_lowq</code> ）或 <code>quality.selection = "top_ratio"</code> （保留批内前 <code>quality.top_ratio</code> 比例）时 $N \rightarrow N - \text{lowq}$ 。两者互斥（M1 校验），均缺省 = 只打分不过滤（5.2）。	不改记录内容；对质量分布截尾，直接改变输出集组成（机制见下方三路径）。 <code>pairwise</code> 模式下淘汰按批内相对位次进行（见本节 note）。	<code>_meta.scores</code> （per-criterion + <code>__aggregate__</code> + mode + <code>batch_no</code> , 6.3）；trace <code>quality.judgment</code> / <code>quality.pointwise</code> / <code>quality.bt_fit</code> / <code>quality.gate</code> 。	<code>dropped_lowq</code> $\Rightarrow$ rejects。
M5 标注	成功路径基数不变；失败减（L3 耗尽 $\Rightarrow$ <code>failed</code> , 3.8.2）。	内容增列：主输出行由原始数据变为用户 Schema 标注对象（原文仅经 <code>_meta.source</code> 溯源与 <code>output.passthrough_fields</code> 透传）； <code>annotate.self_consistency</code> ≥ 3 时标注取多数票 [33]，只增调用次数、不改基数。	<code>_meta.annotation</code> {model, attempts}；trace <code>annotate.done</code> 、 <code>schema.repair</code> 。	<code>failed</code> （如 <code>schema_violation</code> , 7.6） $\Rightarrow$ rejects。
M6 生成	增： $N \rightarrow N + G$ （ $G \leq$ 调用次数 $\times$ <code>generate.num_per_call</code> , 3.6.2）；生成子批回流 M3 起再过滤（图 2-2），实际净增 $\leq G$ 。时间流形态下 $G =$ 幸存序列条数，单位是序列而非单条文本；配额是尝试上限，内容或验证失败的 attempt 直接缺席，不产 <code>failed</code> 记录、不补齐。v1.16 联合路径中失败只从冻结骨架投影删除，绝不重排 <code>word/session/timestamps</code> ；空会话删除、幸存会话重编号而时间戳不移动，真交叉计数只保留双 owner 均幸存且仍交替者。	引入合成样本、改变输出集的「真实：合成」构成比； <code>generate.llms</code> / <code>generate.mixture</code> / <code>[[generate.styles]]</code> 决定合成子集的模型与风格分布（3.6.2）；合成占比过高有 model collapse 风险 [36]。v1.16 的 <code>rules/windows</code> 只改变合成序列的合法控制流与时间分布； <code>correlation/sample/sequence/similarity</code> 过滤依次收缩幸存集。	<code>_meta.source.generator</code> 与 <code>generated_from</code> 沿既有格式；v1.16 不把 <code>rules/windows</code> 逐行复制到产物，新增证据只在 <code>report.generate.stream.rules</code> 、回调 <code>scrap</code> 计数与 <code>windows.calendar_days_spanned</code> ，且零新 trace 事件。	内容/回调/规则/相似度作废不生成拒绝信封；回流后被下游算子淘汰者仍按所在算子的既有去向进 rejects。
M7 校验	减： <code>verify.policy="drop"</code> 直接减； <code>"repair"</code> 先修复（ $\leq$ <code>verify.max_repair_rounds</code> 轮，默认 1）仍 fail 再减（3.7.3）。	<code>repair</code> 路径会改写标注内容（批评意见回喂 M5 重标注 + M8 重校验）——M5 之后唯一还会改动主输出行内容的算子。	<code>_meta.verification</code> {verdict, rounds}；trace <code>verify.verdict</code> （每轮—事件）。	<code>dropped_verify</code> $\Rightarrow$ rejects。
M11 输出	通道分发，不新增淘汰判定（写出前 <code>validate_only</code> 终检失败属 bug 兜底，记 <code>internal_error</code> 转 rejects, 3.11.1）。	按 status 分发三通道： <code>status="active"</code> （annotate 启用时且标注成功） $\rightarrow$ 主输出； <code>dropped_*</code> / <code>failed</code> $\rightarrow$ rejects；计数/分布 $\rightarrow$ <code>report.json</code> 。组装 <code>_meta</code> （ <code>output.meta_mode = "inline"</code> \ <code>"sidecar"</code> \ <code>"none"</code> , 6.3）。	分发依据即 status 本身；trace <code>batch.end</code> / <code>run.end</code> 携带各状态计数。	——（三通道即去向本身）。

分数如何影响输出——三条独立路径。M4 产出的分数经由三条彼此独立的路径作用于输出集，约束力递减：

路径一：门控与选择（硬路径，直接改变输出集组成）。`quality.threshold` 将聚合分低于线的记录置 `dropped_lowq`（3.4.3 质量门）；v1.2 新增的 `quality.selection = "top_ratio"` 改为保留批内聚合分排名靠前的 `quality.top_ratio`（取值 (0,1]，`selection="top_ratio"` 时必填）比例——该机制的精确定义（含排序、并列与取整规则）由 3.4.3 给出，此处仅作参考。两种方式互斥（`quality.selection = "threshold"` 为默认，M1 启动时校验互斥性）。这是分数影响输出的唯一「硬」路径：直接决定哪些行存在于主输出。

路径二：_meta.scores 随行落盘（软路径，供下游后筛）。`output.meta_mode = "inline"`（默认）时分数随主输出每行落盘（6.3），因此门控可以留宽、由下游按需收紧——一行 jq 即可从主输出筛出聚合分 ≥ 0.6 的行（剥离或保留 `_meta` 自便）：

```
# inline 模式；要保留 _meta 则删去 "| del(._meta)" 一段
jq -c 'select(._meta.scores["__aggregate__"] >= 0.6) | del(._meta)' \
  out/ime-intent-0630.jsonl > out/ime-intent-0630.hq.jsonl
```

`meta_mode = "sidecar"` 时 `_meta` 在 `{output_stem}.meta.jsonl` 且与主输出行序对齐 (6.3), 可用 `paste` 或 `jq --slurpfile` 按行号连接后同法筛选; `meta_mode = "none"` 丢弃分数, 本路径不可用 (6.3 已注明不推荐)。

路径三: `trace quality.*` 事件 (诊断路径, 跨运行起效)。 `quality.judgment / quality.gate` 等事件 (7.2) 不改变本次输出的任何一行, 但它们是 7.5 rubric 优化闭环的原料: 据其修订 rubric 后重跑, 改变的是「下一次运行」的输出集。分数对输出的影响由此分三种时效——当次硬淘汰 (路径一)、当次可后筛 (路径二)、跨次可调优 (路径三)。

关键限制——pairwise 分数是批内相对量: pairwise 主模式下 $\text{score} = \log \theta$ 的批内百分位 (3.4.3 「归一化与聚合」行), 每批最低恒为 0、最高恒为 1。因此 `quality.threshold` 在 pairwise 下的语义是「**批内百分位线**」而非全局绝对质量线: $\text{threshold} = 0.3 \approx$ 每批淘汰相对最差的约 30%——即使某批整体质量极高, 仍会淘汰其相对靠后的部分; 两批各自的 0.53 也不可直接比较 (3.4.3 「比较池」行)。需要跨批绝对可比 (例如路径二想用统一分数线做全局后筛) 时应选 `quality.mode = "pointwise"` (绝对刻度); 而 `quality.selection = "top_ratio"` 本就是批内相对选择, 与 pairwise 语义天然一致 (精确定义见 3.4.3)。理解这一限制是理解「打分如何影响输出」的前提。

2.4 CLI 规格

```
labelkit run      --config <config.toml> --project <project.toml>
                  [--input PATH] [--output PATH]          # 覆盖 project.toml 中 run.input / run.output
                  [--limit N]                             # 只处理前 N 条 (试跑); v1.13 时间流生成子句: 单位 = **序列**, 截断在 M6 计划期配
额层 (类段字典序前缀; 作废序列不再生成、不进交织)  $\Rightarrow$  工件与主输出的覆盖面恒一致 (3.6.5); v1.8 stream 子句: 帧级截断不变 (islice 在 M2 解析流与会话装
配器之间), 截断视图 EOF—尾部未闭合会话按会话闭合下发并 WARN 一次「尾会话被 --limit 截断」(3.2.8)
                  [--dry-run]                             # 走完 M1/M2 校验与成本估算, 不调用 LLM; 报告写 {stem}.dryrun.report.json、tra
ce 写「trace 文件名在扩展名前插 .dryrun」(默认 {stem}.trace.dryrun.jsonl), 不覆盖上次真实运行的产物 (v1.5); generate_only 无 M2, 成本按 3.6.
2 调用次数公式静态估算; v1.7: classify 启用时估算增 classify_calls (公式与 multi/按类覆盖下的下界口径见 3.10.3 分类与扇出行); v1.8: segment 启
用时估算增 segment_calls / extract_calls—segment_calls =  $\sum \text{ceil}((L-1)/(w_{\min}-1))$  (L 为会话长; L=1 或 strategy="rules" 计 0; v1.11
注:  $w_{\min}$  = 预算最坏保证装填量, 由 budget.min_window(cfg) 导出—上界语义, 实际窗数  $\leq$  估算; 预算未声明时  $w_{\min}$  = window、与 v1.8 公式同构数值不
变, V12/3.10.3)、extract_calls =  $\sum(L-1)$  报上界, quality/annotate/verify 以 episodes  $\approx$  sessions 报下界 + stderr 注明; 批数按会话空跑实际
装箱精确得出, 文本模态行数统计与会话空跑单遍融合 (3.10.3、3.2.8); v1.9: 估算增 stitch_calls = 会话数  $\times$  votes  $\times$  (2 若 repass 否则 1) (episodes
 $\approx$  sessions 下界基数, 沿用既有 stderr 下界注; off 时恒 0 且该行无条件打印—segment_calls 先例, 3.10.3)
                  # v1.16 时间流约束形态: --limit 先裁出实际配额前缀; dry-run 以同 seed 独立 RNG 副本调用 M1/M6 共用的联合规划入口, 精确复演条
件长度、噪音槽与内容调用计划, 不推进运行期 RNG, 不发起 LLM 调用 (3.6.5、3.10.3)
                  [--strict]                             # 任何记录被拒绝即以退出码 1 结束; v1.9 补注: stitched 壳与被救援帧均不构成 rejec
ts—同输入开启 stitch 后 (短段被救援而不再落 rejects) strict 结果可能由 1 变 0, 属预期 (3.11.2、3.16.6)
                  [--log-level debug|info|warn|error]     # 默认 info
                  [--console auto|rich|plain]             # v1.10: 进度显示面三态 (7.7) —auto (默认) 按判定链选档; plain 与 v1.9 stderr
行为等价 (三层回归锚 7.8); rich 为双区内联实时面板; tool.log_format="jsonl" 时强制 plain 不可覆盖 (显式冲突 M1 WARN)
labelkit validate --config <config.toml> --project <project.toml>
                  # 仅执行 M1 全量校验 (含用户 Schema 预校验、rubric 校验、profile 连通性探测可选 --probe; v1.6: 密钥池逐密钥探测, 3.9.2 pro
be_all); v1.10: --console 同参共用 (--probe 结果表的 rich 呈现, 7.7)
labelkit rubric   [--show default:text | default:ui | default:trajectory] # 打印系统默认 rubric 的 TOML 全文, 便于用户复制修改 (defa
ult:trajectory 为 v1.8 轨迹 rubric, 附录 A.3)
```

退出码	含义
0	运行完成。可能存在被拒绝记录 (详见 report.json 与 stderr 摘要)。
1	运行完成但违反 <code>--strict</code> (存在 rejects), 或报告写出失败。
2	配置错误 (TOML 语法/字段/引用的 profile 不存在/Schema 非法/rubric 非法/环境变量缺失)。
3	输入错误, 仅 process 模式 (路径不存在、无任何合法记录、UI 模态 index 冲突且策略为 fail); generate_only 无输入不触发本码——生成产出 0 条时照常 finalize (counts.generated = 0), 以退出码 0 结束。
4	致命运行错误 (LLM 认证失败、连续不可恢复的 provider 错误超过熔断阈值、输出路径不可写)。v1.6: 熔断中止仍原子交付已完成批的主输出与 rejects 并写报告 (run.partial_delivery=true, 3.10.3 熔断交付); 输出路径不可写则无任何交付。

2.5 配置体系总览

配置分两层两个文件, 职责严格分离; 除 API Key (以环境变量名在 config.toml 中声明、值从环境读取) 外, 不使用任何环境变量:

文件	性质	内容
config.toml	工具级静态配置。随部署环境变化, 跨工程复用。	LLM API profile 列表 (provider、base_url、model、api_key_env / api_key_envs (v1.6 密钥池, 3.9.3)、并发、超时、重试、能力声明)、全局日志级别; v1.10 增 [console] 进度显示面配置 (部署环境属性: 本机终端 vs CI, 5.1)。见 5.1。

project.toml	工程级单次配置。随一次标注任务变化。	输入/输出路径与模态、批大小与 seed、各阶段开关与参数、Rubric、任务指令与 few-shot、用户输出 JSON Schema；v1.7–v1.15 依次增加分类/按类覆盖、stream/segment/extract、stitch、帧粒度、时间流生成、按类 Schema、帧类内容契约、档位与时间字段绑定。v1.16 增 <code>[[generate.stream.rules]]</code> / <code>[[generate.stream.windows]]</code> 、按类同名三态覆盖，以及 <code>[generate].sequence_validator</code> ；这些键只在时间流 generate_only 形态合法。完整字段见 5.2。
--------------	--------------------	--

参数优先级（高覆盖低）：**CLI 参数 > project.toml > config.toml 内的全局默认**。M1 在启动时完成三源合并并冻结为 `ResolvedConfig`，运行期只读。

2.6 非功能约束

维度	约束与设计
数据不落盘	全部中间态（Record、签名、LSH 索引、比较结果、BT 参数、未定稿标注）仅存于进程内存；进程退出即销毁。工具不创建任何临时文件；唯一写盘对象为显式声明的输出通道：用户输出文件、rejects 文件、report.json，显式启用（ <code>trace.enabled=true</code> ）时的 trace 日志（7.1——trace 是与主输出同级的输出通道而非中间态落盘，其保留与清理为用户责任），以及 v1.13 时间流生成形态（ <code>generate_stream.enabled=true</code> ）下的时间流工件 <code>{output_stem}.stream.jsonl</code> （第五项——同属与主输出同级的数据输出通道而非中间态落盘：行内容由 M6 定稿、经 M11 与主输出同批 fsync + 原子改名交付，dry-run 不产出；格式见 6.5，其保留与清理为用户责任）。报告只含计数/分布/耗时/token 统计，不含任何数据内容片段。v1.8 注记：stream 模式下 M2 的未闭合会话缓冲（≤ <code>session_max_len</code> 条 Record 元数据，图像仍懒加载）与 M10 的溢出会话（跨批存活封闭清单，3.10.3）同属进程内存、随装箱消费即释放，不构成新的落盘面。
隐私与网络	数据只发送至 config.toml 显式声明的 LLM API 端点；无遥测、无自动更新检查。API Key 只经环境变量进入内存，不写日志、不入报告。
规模与内存	设计目标：单次运行 ≤ 50 万条记录（默认配置下全局 LSH 索引 + 信封对象约占 2–4 GB RSS）。图像字节懒加载：仅在构造该记录的 LLM 请求时读盘并编码，用后即弃，不常驻。超过规模建议按目录分次运行，或设 <code>dedup.scope="batch"</code> 降低索引内存。 <code>dedup.semantic=true</code> 且 <code>scope=global</code> 时另需常驻向量索引，约增加 条数 × 向量维度 × 4 字节（50 万条 × 1024 维 ≈ 2 GB），须计入 RSS 预算。v1.8 注记：序列 Record 以 <code>members</code> 元组持成员 Record 的引用（frozen 对象共享、零拷贝），episode 化不改变批内存量级；懒加载不变（extract 峰值 2 图/请求、序列 annotate ≤ <code>sequence_frames</code> 图/请求）。
v1.16 规划模型规模	联合规划模型构造后检查 proto：变量数与约束数之和上限为 250,000；越界在 M1 以 <code>CONFIG_ERROR</code> 拒绝，避免把不可控模型留到运行期。规划器只保留当前问题与解，不创建缓存、checkpoint 或跨运行状态。
吞吐	瓶颈为 LLM API。并发由每 profile 的 <code>max_concurrency</code> 信号量控制；同一阶段内记录级并发，阶段之间在批内串行（屏障），保证 pairwise 打分所需的批完整性与实现简单性。
幂等与可复现	无跨运行状态 ⇒ 重跑同一输入产生独立完整输出。配对采样、生成采样均使用 <code>run.seed</code> 播种的 PRNG；temperature 默认 0。输出文件以「写临时名 + 完成后原子改名」保证不产生半截主输出（临时名位于输出同目录，属输出交付的一部分，不违反不落盘约束）。v1.7 确定性条件化声明： <code>classify</code> 启用时类池构成与 <code>multi</code> 扇出以分类输出为条件——temperature 0 下端点无逐字节保证，配对计划的可复现性由「仅依赖 seed」条件化为「以分类结果为条件」（generate 回流子批已有同类先例）。v1.8 确定性条件化声明： <code>stream</code> 模式下 <code>episode</code> 构成（边界与噪声判定）与 <code>verify</code> 成员手术以 <code>segment/verify</code> 的 LLM 输出为条件（同上先例）；其余环节全确定性（会话化规则、滑窗缝合、降采样公式均零 rng），且成员手术采用两阶段批级结构（并发评审 → 同步按批位置序执行手术，3.7.3），保证并发调度不引入额外不确定性。v1.9 确定性条件化声明： <code>stitch</code> 启用时线索构成（并入/开新/救援判定）以缝合 LLM 输出为条件（同上先例）；会话内候选流严格串行、会话间亦按批位置序串行——并发仅存在于 <code>votes>1</code> 的采样 <code>gather</code> 内（3.16.4 调用与校验行），先验合取 / 池逐出 / 接缝标定 / 按碎片配额降采样均为确定性零 rng； <code>stitch.enabled=false</code> 时主输出 / <code>rejects</code> / <code>report.json</code> 与 v1.8 逐字节等价（退化锚，3.16.4）。v1.13 确定性条件化声明：时间流生成形态下工件与信封的构成以蓝图/帧实现的 LLM 输出为条件（同上先例）——作废序列会改变交织输入，故「同 seed 双跑逐字节一致」的成立前提是本次 LLM 内容产出相同；机械交织器本身（装箱、交叉、噪音掷签、重发选取、ts 铺设）全程零 LLM，按抽签消费顺序表单流顺序消费 <code>Random(f"{seed}:0:generate")</code> ，给定同一组幸存序列即逐字节可复现（顺序表由测试钉住，3.6.5）。 <code>generate_stream.enabled=false</code> 时全系统与 v1.12 字节等价（含七个既有 dry-run golden，7.8）。v1.14 确定性注记：帧类构成档位的配额配分（整数域最大余额法）与时间字段回填尾声均零 rng 消费——抽签消费顺序表原文不动，同 seed 下有无档位表的抽签流逐字节一致；两开关全关时与 v1.13 字节等价（含八个 dry-run golden）。同批修补 v1.13 的一处形态级缺陷： <code>frame_gap_s.lo</code> 增微秒地板 <code>1e-6</code> （2.3.1 ⑩）——亚微秒 lo 会使帧间隔 <code>timedelta</code> 取整为 0 微秒，令本行「ts 严格递增」的声明出现破口；对 lo ≥ <code>1e-6</code> 的现存工程零影响。
v1.16 联合规划可复现性	无 <code>rules/windows</code> 路径继续使用 v1.15 原 RNG 顺序；联合路由 <code>Random(f"{seed}:0:generate")</code> 顺序消费一个 31-bit solver seed、每 attempt 一次长度偏好偏移、既有 LLM/style 抽样、重发源排列与噪音调用计划。CP–SAT 固定单线程、使用自动搜索并固定 <code>max_deterministic_time = 10.0</code> ，不使用墙钟超时；同 OR–Tools 精确版本、同 seed、同配置给出确定性规划，但不声明对可行解均匀抽样，也不承诺跨求解器版本同解。M1 使用同 seed 的独立 RNG 副本，不能推进运行期随机源；M6 对失败 attempt 只做冻结骨架投影，不重积幸存者。用户 <code>sequence hook</code> 的外部状态与 LLM 服务端非确定性仍不在工具可复现保证内。没有实际生效 <code>rules/windows</code> 且没有 <code>sequence hook</code> 时，全部输出与 v1.15 逐字节等价。
容错	记录级隔离（1.3 节）；LLM 调用按 profile 配置重试（指数退避+全抖动）；v1.6 密钥池：profile 可声明多把 API Key（ <code>api_key_envs</code> ，5.1），429 按密钥冷却却并即时轮换、认证失败按密钥禁用、全池冷却有界驻留（ <code>run.max_park_s</code> ，默认 3600s，超限按重试耗尽计），单密钥配置数据产出与熔断语义不变、429 等待路径有修订（3.9.3 重试行）；连续 <code>fatal_error_threshold</code> （默认 20）次不可恢复 provider 错误触发熔断（认证类 401/403 立即熔断、不计连续数，v1.5；v1.6 池化下 = 最后一把存活密钥被认证禁用时），以退出码 4 终止，写出已完成部分的报告并原子交付已完成批的主输出与 <code>rejects</code> （v1.6 熔断交付，3.10.3）。

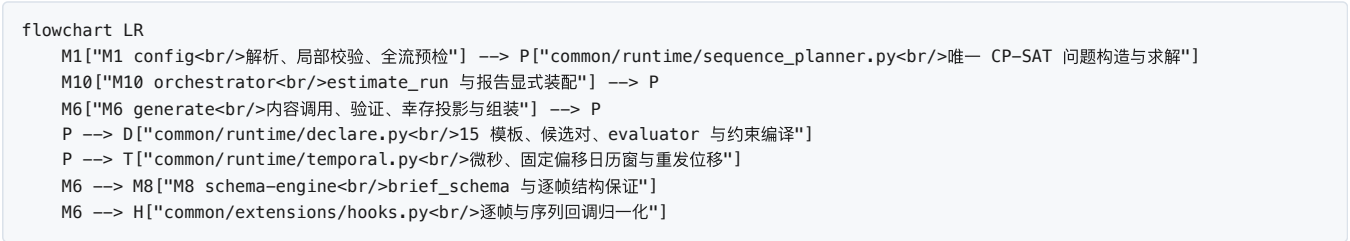
依赖面	Python ≥ 3.11 (tomllib 标准库)。第三方仅: httpx (异步 HTTP)、jsonschema (校验)、datasketch (MinHash-LSH)、Pillow + imagehash (pHash)、json-repair (确定性 JSON 修复)、numpy (BT 拟合)、rich (v1.10, 7.7 console rich 档终端呈现; 纯 Python, 传递依赖 markdown-it-py + pygments 亦纯 Python; 懒 import——仅 console 判定为 rich 档时于 CLI 层导入 (M1 只以 find_spec 探测), 导入失败自动降级 plain, operators/common 零触点; 工业背书 pip vendored (docs/dev/PROPOSAL-tui-console.md [C-9]), 对齐决策 1.6 U4)。无框架级依赖 (rich 定性为终端呈现库; 应用框架级的 textual 经调研否决—— docs/dev/SPEC-tui-console.md §2 U4/U16)。
v1.16 依赖增量	精确新增并锁定 ortools==9.15.6755。OR-Tools 是序列规则联合规划专用的成熟算法库例外, 不是应用框架; 禁止运行时替代实现或按本机版本漂移。生产触点只在 common runtime 联合规划器, M1/M6/M10 通过共享接口调用; 既有依赖及 rich 懒加载语义不变。

3. 模块详细设计

本章每个模块按统一模板描述：职责与边界 → 输入/输出 → 数据结构与 API → 算法与流程 → 配置项 → 错误处理 → 背书。所有代码签名为 Python 3.11+，公共数据结构（Record、PipelineItem 等）的完整定义集中在第 4 章。

3.0 v1.16 时间流序列规则的跨模块落点

v1.16 不增加新的流水线 Stage 或模块编号。它在时间流生成的既有 M1/M6/M8/M10 接缝上增加共享 common 运行时算法；M11、M12、CLI、M9 与其他算子不增加行为分支。约束路径的单一依赖方向如下：



责任面	规范落点	强制边界
M1 配置	3.1: 解析全局/按类 rules/windows 与 sequence_validator，校验 15 模板、correlation Schema、每个候选长度、全配额前缀和 planner 状态	用同 seed 独立 RNG 副本，不推进运行期随机源；UNKNOWN 不得伪报不可满足
M6 生成	3.6: 实际前缀重放联合计划：约束路径生成 brief/frames，依次执行逐帧回调、声明式 evaluator、序列回调、相似度、幸存投影、噪音过滤、回填、重发与组装	首个 LLM 调用前冻结 word/session/timestamps/noise slots；失败后不重解、不重织、不补齐
M8 结构引擎	3.8: 新增 brief_schema(length)；既有 plan_schema 完整保留给 v1.15 默认路径	brief Schema 只允许逐位置 brief，不让约束路径 LLM 决定 frame class
M10 编排	3.10: estimate_run 复用同一 planner；按固定键位显式装配 rules、回调 scrap 与 windows 报告块	不自行推导另一套计划；显式零值块不依赖计数器首触
common 配置/契约/扩展	第 4—5 章：冻结 SequenceRuleSpec、CorrelationSpec、SequenceWindowSpec、SequenceValidationInput、三态 effective helpers 与 hook 契约	common 不导入 operators/orchestration；回调拿 JSON-compatible 深拷贝，突变不回写
M11 与 M12	第 6—7 章：沿用既有工件、报告交付、trace 与日志纪律	工件行形和 truth/generator 键序不变；零新 trace 通道、事件或错误 kind

联合规划器不是一个新算子：它在 M1、estimate_run 与 M6 之间共享同一个问题构造与求解入口。生产依赖精确锁定 ortools==9.15.6755，模型固定单线程与确定性预算；不提供自研替代实现、运行时版本替换或失败后的 fallback。没有实际生效 rules/windows 且没有 sequence_validator 时，调用链必须直接落回 v1.15 原分支并保持逐字节等价。

3.1 M1 配置管理 config

3.1.1 职责与边界

做：装载并语法/语义校验 config.toml 与 project.toml；合并 CLI 覆盖项；解析 rubric（内联或默认包）；装载并预校验用户 JSON Schema；读取 API Key 环境变量；产出全局唯一的不可变 ResolvedConfig。不做：不接触输入数据；不发起网络请求（--probe 连通性探测委托 M9）；运行期不提供任何可変配置。

3.1.2 输入 / 输出

方向	内容
输入	config.toml 路径、project.toml 路径、CLI 参数字典（input/output/limit/strict/log_level/dry_run）、进程环境（仅读取 profile 声明的 api_key_env / api_key_envs（v1.6）所列变量）。

输出	<code>ResolvedConfig</code> (frozen dataclass 树: 第 5 章两文件全部键的类型化镜像, 另含 CLI 专属项 <code>limit/strict/dry_run</code> 与 <code>log_level</code> 覆盖, 以及 <code>load()</code> 收尾冻结的解析产物: <code>ConsoleConfig.mode_resolved</code> (v1.10, 3.1.4 console 行)、<code>SegmentConfig.vision_resolved</code> (v1.11, 3.1.4 上下文预算与视觉推导行)、<code>FrameClassifyConfig.vision_resolved</code> (v1.12, 3.1.4 帧粒度配置行); v1.12 增四字段——<code>frame_classify</code>: <code>FrameClassifyConfig</code>、<code>frame_annotate</code>: <code>FrameAnnotateConfig</code>、<code>frame_class_views</code>: <code>Mapping[str, FrameClassView]</code> (键 = 帧类名, 仅 <code>frame.classify.enabled</code> 时物化, 零覆盖类也各得一份视图——<code>class_views</code> 同款、运行期零回退)、<code>frame_schema</code>: <code>Mapping \ None</code> (帧级输出 Schema 解析产物, <code>user_schema</code> 同胞: 元校验 + <code>few-shot</code> 干跑; <code>frame.annotate</code> 关闭时恒 <code>None</code>)——四字段带默认值 (有意偏离 <code>stream/stitch</code> 的「必填无默认」惯例: 全关默认 = 字节等价 v1.11, 既有构造点零波及 <code>loader</code> 恒显式传入); v1.13 增第五个带默认字段 <code>generate_stream</code>: <code>GenerateStreamConfig</code> (时间流生成形态, 默认关 = 字节等价 v1.12, 沿用同一惯例), 另有三处既有结构的扩字段——<code>GenerateConfig</code> 增 <code>sequences / len_range</code> (按类配额与序列长度区间的载体)、<code>ClassView</code> 增 <code>schema</code> (按序列类标注 Schema 解析产物, <code>None</code> = 回落全局)、<code>FrameClassView</code> 增 <code>gen_instruction / gen_schema</code> (帧类内容契约, 后者 <code>None</code> = 纯文本帧)), 或抛出 <code>ConfigError</code> (附带全部而非首个校验错误, 一次性反馈)。
----	--

3.1.3 API

```
def load(config_path: Path, project_path: Path, cli_overrides: CliOverrides) -> ResolvedConfig:
    """三源合并 + 全量校验。失败抛 ConfigError(errors: list[str]), CLI 以退出码 2 结束。"""

def default_rubric(name: Literal["default:text", "default:ui", "default:trajectory"]) -> Rubric:
    """从包内数据文件 (labelkit/data/rubrics/*.toml) 装载系统默认 rubric。
    "default:trajectory" 为 v1.8 轨迹 rubric (default_trajectory.toml, 附录 A.3)。"""
```

模块内文件组织 (2026-08-14 代码规则整改): 公开面仍是 `labelkit/common/config/` 包导出的 `load / default_rubric / ResolvedConfig` 三个名字, 实现按职责拆为「公开入口 + 六个包内私有模块」:

```
labelkit/common/config/
├─ __init__.py      公开再导出: load / default_rubric / ResolvedConfig
├─ model.py         全部配置 dataclass (分工不变)
├─ loader.py        公开入口: 三源合并驱动 → console 模式裁定 → ResolvedConfig 装配
│   ├─ _collect      错误/警告聚合器与类型化表读取 (全程共用)
│   ├─ _sections     逐节 TOML 解析 → 各配置 dataclass
│   └─ _constraints  跨节组合约束与解析产物冻结, 内部再调用:
│       ├─ _schemas   用户 / 帧 / 按类 Schema 元校验 + few-shot 干跑
│       ├─ _rubrics   内联 rubric 解析与 default:* 包数据装载
│       └─ _classviews [class.*] / [frame.class.*] 白名单合并为按类视图
└─ (六个下划线开头的模块均为包内私有, 外部只经 loader 的公开入口进入)
```

拆分只搬代码不改语义: 校验顺序、错误文案、聚合反馈行为与 `ResolvedConfig` 结构均与拆分前一致 (`model.py` 承载全部配置 dataclass 的分工不变)。

3.1.4 校验规则 (启动时全量执行)

类别	规则
TOML 结构	两文件均须含 <code>schema_version = 1</code> ; 未知键报 <code>warning</code> (前向兼容)、缺失必填键报 <code>error</code> ; 类型逐字段核对 (第 5 章字段表即校验依据)。v1.7 例外: <code>[classify]</code> 与 <code>[class.*]</code> 为显式接管的节—— <code>[class.*]</code> 内白名单 (5.2 按类覆盖白名单表) 之外的键报 <code>CONFIG_ERROR</code> 而非 <code>warning</code> (见下方「按类覆盖合并」行)。
Profile 引用	<code>quality.llm / annotate.llm / generate.llms</code> (数组, 逐元素校验) / <code>verify.llm / output.repair_llm</code> , 以及 <code>quality.judges / verify.judges</code> (数组, 非空时须为奇数个) 引用的 <code>profile</code> 必须存在于 <code>config.toml [llm.*]</code> ; 启用视觉输入的阶段 (UI 模态的 <code>quality/annotate/verify</code>) 要求其 <code>profile supports_vision = true</code> 。v1.2: <code>dedup.semantic = true</code> 时 <code>dedup.semantic_embedding</code> 必须存在于 <code>config.toml [embedding.*]</code> 且其密钥配置通过本表「API Key」行校验 (v1.6: <code>api_key_env / api_key_envs</code> 恰其一; 5.1)。
交叉字段约束 (v1.2)	<code>quality.selection = "top_ratio"</code> 时 <code>quality.top_ratio</code> 必填且 $\in (0,1]$, 且不得再设 <code>quality.threshold</code> (互斥, 报 <code>CONFIG_ERROR</code>); <code>annotate.self_consistency</code> 为 0 或 ≥ 3 的奇数; <code>generate.mixture = "weighted"</code> 时 <code>generate.weights</code> 必填、逐项为正且长度 = <code>generate.llms</code> ; <code>[generate.styles]</code> 各项 <code>name</code> 表内唯一、 <code>prompt</code> 非空 (5.2 各行标注的 M1 校验在此汇总执行)。
运行模式 (v1.4; v1.13 三态)	<code>run.mode="generate_only"</code> 时: <code>run.input</code> 必须缺省、 <code>run.modality</code> 必须 <code>"text"</code> 、 <code>generate.enabled</code> 必须 <code>true</code> 。形态判定三态 (v1.13): <code>generate_stream.enabled = true</code> $\Rightarrow$ 时间流形态——种子池/独立计数两族键均不适用 (<code>seed_examples / standalone_count</code> 的「恰好提供其一」规则在本形态下不执行, 配额改由 <code>[class.<name>.generate].sequences $\times$ len_range</code> 承载; 显式书写该两键连同 <code>num_per_record / seeds_per_call</code> 由本表「时间流生成」行定向报错); 否则维持 v1.4 二态—— <code>generate.seed_examples</code> (非空字符串数组, 逐项非空) 与 <code>generate.standalone_count</code> (≥ 1) 恰好提供其一 (互斥, 分别对应种子池 / 无种子形态)。process 模式下这两键均不得设置。另: <code>generate.instruction</code> 的「enabled 时必填」在时间流形态下退化为可选默认——任务描述放在按类生成指令上, 「参与类 instruction 非空」由「时间流生成」行按类裁定。

分类 (v1.7)	<code>classify.enabled = true</code> 时: <code>[[classify.classes]]</code> ≥ 2 项, 每项 <code>name</code> 匹配 <code>[a-z0-9_]+</code> 且表内唯一、<code>description</code> 非空、<code>examples</code> (可选) 为字符串数组; <code>classify.fallback_class</code> 必填且 $\in$ <code>classes</code>; <code>classify.assignment</code> $\in$ <code>{"single","multi"}</code>; <code>classify.max_labels</code> 仅 <code>multi</code> 可设且 $\in [2, \text{类别数}]$ (缺省解析后回填为类别数); <code>classify.self_consistency</code> 为 0 或 ≥ 3 的奇数; <code>classify.on_error</code> $\in$ <code>{"fallback","fail"}</code>; <code>classify.llm</code> 引用的 <code>profile</code> 必须存在 (UI 模式须 <code>supports_vision = true</code>), 并计入密钥解析、<code>vision</code> 校验与 <code>--probe</code> 三处 <code>profile</code> 引用集 (本表「Profile 引用」「API Key」行同法覆盖)。<code>classify.enabled = false</code> 而 <code>[[classify.classes]]</code> / <code>[class.*]</code> 在场 $\Rightarrow$ <code>warning</code> (一次、点名被忽略的表——「留配置、关开关」合法, 对齐 <code>top_ratio</code> 未生效等 <code>no-op</code> 键分级惯例), 不报 <code>error</code>。
按类覆盖 合并 (v1.7)	<code>[class.<name>.<section>]</code> 的 <code><name></code> 必须 $\in$ <code>classes</code>; 覆盖键 $\in$ 白名单 (5.2 按类覆盖白名单表), 白名单外键报 <code>CONFIG_ERROR</code> (本表「TOML 结构」行「未知键报 <code>warning</code>」的显式例外)。合并语义 (启动时静态合并、冻结为 <code>class_views</code>, 运行期零查找): ① 逐键 <code>provenance</code> 合并——类显式提供的键覆盖全局、未提供的键继承全局; ② 选择组——类显式提供 <code>selection</code> / <code>threshold</code> / <code>top_ratio</code> 任一 $\Rightarrow$ 合并视图剔除全局侧的互斥对键, <code>threshold</code> 与 <code>top_ratio</code> 互斥校验跑在合并后视图上 (防止「全局 <code>threshold</code> + 类 <code>top_ratio</code>」逐键 <code>replace</code> 后两键并存的误报); ③ rubric——合并 <code>selector</code> 后重解析为该类有效 <code>rubric</code>, <code>pointwise</code> 6 级校验跑在 (类有效 <code>mode</code> $\times$ 类有效 <code>rubric</code>) 组合上; <code>[class.X.rubric]</code> 在场但该类 <code>selector</code> 非 <code>"inline"</code> $\Rightarrow$ 忽略并 <code>warning</code> (同全局惯例); ④ 类 <code>examples</code> 干扰该类有效 Schema与全局 <code>output.validator</code> (v1.13 修正: 此前恒过全局用户 <code>Schema</code>——类自带标注 <code>Schema</code> 时会误判; 类自带 <code>Schema</code> 时继承来的全局示例也按类 <code>Schema</code> 复跑一遍, 因为运行期就是按类 <code>Schema</code> 发出去的。错误定位写作 <code>[[class.<name>.annotate.examples]] [N]</code>, <code>Schema</code> 定位前缀相应取 <code>[class.<name>.annotate].schema_*</code>; 类自带 <code>Schema</code> 的 <code>\$ref</code> 死链只使该类停跑, 不牵连全局层)。⑤ 按类标注 Schema (v1.13) —— <code>[class.<name>.annotate]</code> 白名单增 <code>schema_path</code> / <code>schema_inline</code> (至多其一: 两者皆缺 = 未声明覆盖, 回落全局 <code>output.schema</code>; 同时声明报 <code>CONFIG_ERROR</code>), 声明了即走 <code>output.schema</code> 全套装载分支 (读取 / <code>JSON</code> 解析 / <code>draft 2020-12</code> 元校验 / 顶层 <code>type = object</code>) + <code>_meta</code> 保留键禁令 + <code>\$ref</code> 可解析性遍历; 解析产物挂 <code>ClassView.schema</code> (<code>None</code> = 无覆盖)。⑥ 按类档位表 (v1.15) —— <code>[class.<name>.generate]</code> 白名单增 <code>tiers</code> (数组表, 行结构同全局 <code>[[generate.stream.tiers]]</code> 三键), 语义为表级原子覆盖 (声明了就用类的整张表、不逐行合并, 未声明 = 回落全局表, 显式空表报 <code>CONFIG_ERROR</code>); 解析复用全局表同一实现 (<code>rank</code> 升序存放, 正整数与 <code>weight</code> ≥ 1 解析期强制), 报错定位单取 <code>[class.<name>.generate].tiers</code>; 解析产物挂 <code>ClassView.tiers</code> (<code>None</code> = 未声明), 零覆盖类经继承同得 <code>None</code>。前提三子款与逐生效表化校验见本表「按类档位表」行。
时序流 (v1.8)	组合约束: <code>segment.enabled = true</code> (<code>stream</code> 模式总开关) 要求 <code>run.mode = "process"</code> $\wedge$ <code>generate.enabled = false</code> (<code>generate_only</code> 经本表「运行模式」行传递闭合——该行要求 <code>generate.enabled = true</code>, 故 <code>stream</code> $\times$ <code>generate_only</code> 不可能同时过验) $\wedge$ <code>annotate.enabled = true</code> (序列记录无 <code>passthrough</code> 输出形态, 2.3.1); <code>extract.enabled = true</code> 要求 <code>segment.enabled = true</code> $\wedge$ <code>run.modality = "ui"</code> (文本序列 v1 不适用)。【stream】字段: <code>stream.order_by</code> $\in$ <code>{"input_order","meta:<field>"}</code> 且 <code>"meta:*"</code> 仅文本模式; 显式设置 <code>stream.session_max_span_s</code> 要求 <code>order_by = "meta:*"</code> (违反报 <code>CONFIG_ERROR</code>); 显式设置 <code>stream.gap_s</code> 而非 <code>meta</code> 序 $\Rightarrow$ <code>warning</code> 一次 (非阻断, 键不生效); <code>stream.key</code> 逐元素 $\in$ <code>{"meta:<held>"}</code> (仅文本模式), <code>"source_dir"</code> (两模式可用)}。数值界: <code>segment.window</code> ≥ 2; <code>2 \leq annotate.sequence_frames \leq 100</code> (越界报 <code>CONFIG_ERROR</code>)。引用集四处 (S30): v1.8 起 <code>profile</code> 引用集口径为四处——密钥解析 (本表「API Key」行) / <code>vision</code> 校验 / <code>--probe</code> / 存在性 (v1.7 分类行「三处」口径的显式扩展): <code>segment.llm</code> 仅 <code>segment.enabled</code> $\wedge$ <code>segment.strategy</code> $\in$ <code>{llm, hybrid}</code> 时计入密钥解析 / <code>--probe</code> / 存在性三处 (<code>rules</code> 策略零 LLM 调用, 不得强制配键), 恒不入 vision 校验集 (v1.11 修订 (V3): v1.8—v1.10 为「仅 <code>segment.use_vision = true</code> 时入」, 该键已移除——<code>segment</code> 从「要求视觉」改为「适配视觉」, 校验命题失去可失败性; 附图由解析产物 <code>vision_resolved</code> 推导, 见本表上下文预算行; 报错文案的 <code>stages</code> 集合中 <code>"segment"</code> 自 v1.11 不再可能出现); <code>extract.llm</code> 启用时恒计入四处且恒入 <code>vision</code> 集 (每转移一请求 2 图, 无纯文本档)。vision 逐阶段表 (S30): UI 模式 $\wedge$ <code>segment.enabled = true</code> 时取代本表「Profile 引用」行的整体 <code>vision</code> 规则——<code>classify</code> $\checkmark$ (首帧截图, 3.13.3)、<code>annotate</code> $\checkmark$ (多图序列模板, 3.5.2)、<code>verify</code> $\checkmark$ (首末帧截图, 3.7.2)、<code>extract</code> $\checkmark$ (恒)、<code>segment</code> $\times$ 恒不要求 (v1.11 修订 (V1/V3): 原「仅 <code>use_vision = true</code> 时 $\checkmark$」——附图改由 <code>vision_resolved</code> 能力推导自动适配, 非校验要求)、quality $\times$ (序列打分纯文本——放宽项, 3.4.3 序列行; v1.9 起 <code>stitch</code> 亦 $\times$ 恒不要求 (摘要卡纯文本, 3.16.3) ——「唯一放宽」措辞自 v1.9 失效, 见本表线索缝合行)。按类白名单: <code>[class.<name>.extract]</code> 可覆盖键仅 <code>instruction</code> (扩展本表「按类覆盖合并」行引用的 5.2 白名单表, 白名单外键同报 <code>CONFIG_ERROR</code>); <code>[class.<name>.segment]</code> 不存在——链序 <code>segment</code> 在 <code>classify</code> 之前 (3.10.3), 成段时类标签尚不存在 (链序因果, 5.2 注), 该表按白名单外键处理。rubric: <code>selector</code> 枚举扩为 <code>"default:text"</code> <code>"default:ui"</code> <code>"default:trajectory"</code> (v1.8, 包数据 <code>default_trajectory.toml</code>, 附录 A.3) <code>"inline"</code>; 空串解析 v1.8 修订 (S29): <code>segment.enabled = true</code> $\Rightarrow$ <code>""</code> 解析为 <code>"default:trajectory"</code> (两模式一致; 用户显式选择器恒优先; 按类视图经 <code>base selector</code> 自动继承) ——v1.13 条件扩为 <code>segment.enabled</code> $\wedge$ <code>generate_stream.enabled</code> (本表「时间流生成」行); 本表「Rubric」行的全部校验 (含 <code>pointwise</code> 6 级) 对 <code>trajectory rubric</code> 照常适用。【stream】在时间流生成形态的语义翻转 (v1.13): 该节由「摄取侧声明」兼作生成侧铺设契约——<code>order_by</code> 必须 <code>"meta:<字段>"</code> (声明工件行的时间戳字段名)、<code>gap_s</code> 定会话间隔下界、<code>session_max_len</code> 定织造上限, 而 <code>key</code> 与 <code>gap_steps</code> 必须取空/0 (本表「时间流生成」行 ⑤)。
线索缝合 (v1.9)	组合约束: <code>stitch.enabled = true</code> 要求 <code>segment.enabled = true</code> (缝合的输入是 <code>episode</code>; <code>stream</code> 前置约束——<code>process</code> 模式 $\wedge$ <code>generate off</code> $\wedge$ <code>annotate on</code>——经此传递闭合, 2.3.1)。数值界: <code>stitch.votes</code> 为 1 或 ≥ 3 的奇数 (偶数报 CONFIG_ERROR——(<code>verdict</code>, <code>thread_ref</code>) 严格多数决需破僵局, 3.16.4); <code>stitch.max_open</code> ≥ 1、<code>stitch.digest_max_chars</code> ≥ 1、<code>stitch.stale_gap_steps</code> ≥ 0 (越界报 <code>CONFIG_ERROR</code>); <code>stitch.bias</code> $\in$ <code>{"conservative","llm"}</code>、<code>stitch.on_error</code> $\in$ <code>{"keep","fail"}</code>。引用集: <code>stitch.llm</code> 仅 <code>stitch.enabled = true</code> 时计入密钥解析 / <code>--probe</code> / 存在性引用集, 不入 vision 校验集 (缝合判定证据为纯文本摘要卡、无视觉档, 3.16.3——<code>vision</code> 逐阶段表 (S30) 增一行: <code>stitch</code> $\times$ 恒不要求)。按类白名单: <code>[class.<name>.stitch]</code> 不存在——链序 <code>stitch</code> 在 <code>classify</code> 之前 (3.10.3), 缝合时类标签尚不存在 (链序因果, <code>[class.<name>.segment]</code> 同则, 5.2 注), 该表按白名单外键处理。

时序流警告 (v1.8, 非阻断)	同 R8 no-op 分级家族 (对齐分类行「留配置、关开关」惯例), 均 warning 一次、不报 error: ① [stream] / [segment] / [extract] / [stitch] (v1.9 增) 任一节在场而 segment.enabled = false ⇒ 点名被忽略的表; ② segment.strategy = "rules" ∧ 显式 noise_filter = true ⇒ no-op (rules 下 noise_filter / min_len 不生效, 3.14); ③ annotate.sequence_frames 显式设置而 segment.enabled = false ⇒ no-op; ④ 有效 rubric 为 trajectory (含空串解析所得) 而 extract.enabled = false ⇒ 组合提示 (rubric 模态中立、不预设步骤在场——「步骤」退化读作「帧间变化」, S29, 3.4.3 序列行); ⑤ stream.session_max_len > run.batch_size ⇒ 静态 WARN (S21: 此类会话将被 M10 硬切 + session_split 标, 3.10.3); ⑥ annotate.sequence_frames > 20 ∧ 所引 annotate profile max_image_px > 2000 ⇒ WARN (S28: Anthropic 对 >20 图请求中任一图 >2000px 返回 400 硬拒 (非缩放), 默认 max_image_px = 2048 恰在拒绝域——指引改 ≤ 2000 或降 sequence_frames; 20 图阈值按请求内全部 image block 计; openai_compatible 无此联动、不设独立上限)。(v1.9 增两条, 同分级: ⑦ segment.enabled = true ∧ stitch.enabled = false 而 [stitch] 节有 payload ⇒ 单独 no-op warning (annotate.sequence_frames 显式设置的同形制, 不落 ① 的点名名单分支——① 归属 segment 关闭分支); ⑧ stitch.enabled = true ∧ segment.strategy = "rules" ⇒ 组合提示 (规则粗切段未经语义精化, 缝合证据质量下降, 3.16)。(v1.11 增一条, 同分级: ⑨ vision_resolved ∧ segment.window > 20 ∧ 所引 segment profile max_image_px > 2000 ⇒ WARN (V5: ⑥ 的 S28 姊妹——同一 Anthropic 「>20 图 ∧ 单图 >2000px」400 硬拒域, ⑥ 只盖 annotate.sequence_frames、本条盖 segment 窗口多图; 默认 window = 20 恰在边界内侧, 不触发)。(
console (v1.10)	console.mode ∈ {"auto","rich","plain"}; console.refresh_hz ∈ [1,10] (越界 = CONFIG_ERROR); console.heartbeat_s ≥ 0 (< 0 = CONFIG_ERROR); estimate / interactive 为 bool (第 5 章字段表即校验依据)。解析产物: load() 收尾把 auto 判定链 (7.7——stderr TTY ∧ log_format ∧ TERM ∧ importlib.util.find_spec("rich") 探测, 不真 import) 冻结为 ConsoleConfig.mode_resolved ∈ {"rich","plain"}。警告 (非阻断, 独立于 R8 家族): tool.log_format = "jsonl" ∧ 显式 rich (CLI --console rich 或 config console.mode = "rich") ⇒ WARN 一次 + 强制 plain (7.7 铁律; 5.1)。
上下文预算与视觉推导 (v1.11)	context_window 校验 (V6; llm 与 embedding profile, 5.1): 0 合法 (未声明 = 该 profile 预算关闭, 行为与 v1.10 一致); 负值报 CONFIG_ERROR; > 0 时须 context_window > max_output_tokens + margin (margin = max(256, ceil(0.10 × context_window)), 3.9.5; embedding 无输出预留, 预算 = context_window - margin 须为正), 否则报 CONFIG_ERROR (预算非正)。引用 WARN (V6): 被启用阶段引用的 profile 未声明 context_window ⇒ 一次性 WARN (含建议值指引; 非阻断——该 profile 预算关闭)。default_image_px 校验 (V18, 5.1): 0 合法 (沿用 max_image_px); > 0 时须 ≤ max_image_px, 否则报 CONFIG_ERROR。移除键定向报错 (V2): [segment] 内显式出现 use_vision ⇒ CONFIG_ERROR (文案 = 5.2 移除行的迁移指引: 键已移除、附图改由 segment.llm 所指 profile 的 supports_vision 自动决定、需纯文本请指向纯文本 profile), 不走本表「TOML 结构」行「未知键报 warning」的前向兼容路径——实现机制 = loader 既有原始节探针先例 (V27②, segment_provided 同款): 解析删除后于原始 [segment] dict 上探键存在性。解析产物 (V1): load() 收尾以 dataclasses.replace 冻结 SegmentConfig.vision_resolved = (modality=="ui") ∧ segment.enabled ∧ strategy∈{llm,hybrid} ∧ llm_profiles[segment.llm].supports_vision (解析产物家族第二员, ConsoleConfig.mode_resolved 先例——本表 console 行)。segment 装填静态护栏 (V9): w_min = [(input_budget - est_static_system) / per_frame_max] (最坏保证装填量: per_frame_max = est_text(digest_max_chars 最坏串) + DIFF_MAX_TOKENS + 每图成本先验 (仅 vision_resolved 时计), 3.9.5; 未声明预算时 w_min = window、本护栏不触发); w_min < floor ⇒ CONFIG_ERROR, floor = 3 if (verify.enabled ∧ verify.policy == "repair" ∧ segment.enabled) else 2 (在先验计价下保证任意帧装得进 floor 帧窗与 verify 3 帧回收复裁窗——护栏基于每图成本先验 (3.9.5 校准值可合法超过先验 x1.2, 无钳制), 校准超先验或退化个案时装填器强制 2 帧封窗、由 M9 终检降到记录级 context_overflow (3.14.4), 永不 run 级 (v1.11 审计修订); policy="drop" 不构造复裁窗、不做三帧要求); w_min == floor ⇒ WARN (窗数放大退化警示: 每帧皆接缝、逐帧双裁决); w_min 随启动 INFO 打印 (V13①, M10 启动段)。静态系统侧预检 (V13③): 每个启用阶段的静态 prompt 部件 (模板头 + instruction + rubric/类表/schema/few-shot——模板头经 V22 冻结常数 TEMPLATE_HEAD_TOKENS 取得, 其余从 ResolvedConfig 直取, 3.9.5) est ≥ input_budget ⇒ CONFIG_ERROR (任何记录都装不下、必错无疑), > 50% ⇒ WARN (系统侧过半、单记录可用空间减半的质量退化预警)。预算类校验仅对声明了 context_window 的被引用 profile 执行。

帧粒度配置 (v1.12)	组合约束七条（规范源头 2.3.1，全部 CONFIG_ERROR，⑦ 为 warning）：① 帧粒度要求流模式——<code>frame.classify.enabled v frame.annotate.enabled ⇒ segment.enabled = true</code>（报错文案指引 <code>outside stream mode use classify + [class.<name>.annotate] per-class annotation</code>）；② 帧类覆盖要求帧分类——<code>[frame.class.*]</code> 在场 $\Rightarrow$ <code>frame.classify.enabled = true</code>，节名 $\subseteq$ 帧类表、覆盖白名单仅 <code>annotate</code> 节三键（<code>instruction / examples / enabled</code>，5.2 白名单表），白名单外键/节报 CONFIG_ERROR（「TOML 结构」行前向兼容的显式例外，<code>[class.*]</code> 同族）；③ 帧 Schema 恰——<code>frame.annotate.enabled = true ⇒ schema_path / schema_inline</code> 恰一 + draft 2020-12 元校验 + <code>[[frame.annotate.examples]]</code> 干跑（镜像 <code>output.schema</code> 全套分支；帧级无 L2.5 hook，干跑仅对帧 Schema）；④ <code>meta_mode</code> 护栏——<code>frame.*</code> 任一启用 $\Rightarrow$ <code>output.meta_mode != "none"</code>（帧产物仅经 <code>_meta.stream.members</code> 承载，sidecar 合法）；⑤ <code>fallback</code> 合法——<code>frame.classify.enabled = true</code> 时 <code>fallback_class</code> 必填且 $\in$ 帧类表 name 集（传递性地要求类表非空——v1.12 无独立 ≥ 2 类数下限，与 <code>[classify]</code> 有意不同）；⑥ 定向探针——<code>[frame.classify].assignment / [frame.annotate].self_consistency</code> 显式书写 $\Rightarrow$ 定向 CONFIG_ERROR（帧级无多标签、无自洽采样；机制 = loader 原始节探针，v1.11 <code>use_vision</code> 同款（V27②），不走「未知键报 warning」前向兼容路径）；⑦ <code>no-op</code> 提示——<code>[frame.*]</code> 节在场 $\wedge$ 均未启用 $\wedge$ <code>segment.enabled = false ⇒</code> 并入 <code>segment</code> 关闭的 <code>no-op</code> warning 停放清单（R8 家族；任一帧开关启用时由 ① 接管）。帧类表：<code>[[frame.classify.classes]]</code> 与 <code>[[classify.classes]]</code> 同构逐项校验（name 匹配 <code>[a-z0-9_]+</code> 且表内唯一、description 非空、examples 可选字符串数组——解析合法但帧级批量判决模板不渲染（§10.12 只渲染类表），任一帧类携带 examples 时显名 WARN <code>[frame.classify].classes: class examples are not rendered by the batched frame-verdict template (§10.12), so this key is ignored</code>）；帧类表与序列类表相互独立、允许重名、互不约束。必填：<code>frame.annotate.enabled = true ⇒ instruction</code> 非空（5.2 $\uparrow$ 家族的帧级镜像）。帧类覆盖合并：<code>[frame.class.<name>.annotate]</code> 逐键 provenance 合并冻结为 <code>frame_class_views</code>（键 = 帧类名；零覆盖类也各得一份视图，「按类覆盖合并」行 <code>class_views</code> 同款）；类提供的 examples 对帧级 Schema 干跑（错误定位写作 <code>[[frame.class.<name>.annotate.examples]] [N]</code>）。vision 集分列：<code>frame.annotate.llm</code> 在 ui 模式 $\wedge$ <code>enabled</code> 时无条件入 vision 必需集（镜像序列级 <code>annotate</code>——截图是标注主证据）；<code>frame.classify.llm</code> 永不入 vision 必需集——附图由解析产物 <code>FrameClassifyConfig.vision_resolved = (modality=="ui") ^ frame.classify.enabled ^ llm_profiles[frame.classify.llm].supports_vision</code> 自动推导（load() 收尾以 <code>dataclasses.replace</code> 冻结，解析产物家族第三员，<code>segment V1</code> 同款；成本控制面 = 指向纯文本 profile，判决仅凭摘要行）；两键 <code>enabled</code> 时均计入密钥解析 / <code>--probe /</code> 存在性引用集（<code>referenced_profiles</code>，预算报表经 profile 聚合自动覆盖）。静态预算预检两段（V13③ 表新增）：<code>frame.classify</code> 段 = 冻结模板头 <code>TEMPLATE_HEAD_TOKENS["frame_classify"]</code> + 帧类表文本（name/description，不计 examples——口径与渲染事实对齐，多算会误触发预检；<code>[frame.classify]</code> 无 instruction 键——提示词模板确定性内建）；<code>frame.annotate</code> 段 = 冻结模板头 <code>TEMPLATE_HEAD_TOKENS["frame_annotate"]</code> + 帧级 Schema 文本 + <code>max(全局与各帧类视图的 instruction + few-shot)</code>；<code>est ≥ input_budget ⇒</code> CONFIG_ERROR、<code>> 50% ⇒</code> WARN（「上下文预算与视觉推导」行同一判则，仅对声明预算的被引用 profile 执行）。v1.13 放宽两处：约束② 的前提改为 <code>frame.classify.enabled v generate_stream.enabled</code>（时间流生成形态经 <code>[frame.class.<name>.generate]</code> 声明帧内容契约，帧类命名空间照常物化 <code>frame_class_views</code>），白名单相应扩为两节——<code>annotate</code> 三键（<code>instruction/examples/enabled</code>）+ <code>generate</code> 三键（<code>instruction / schema_path / schema_inline</code>，仅时间流生成形态合法，非本形态出现是反向定向 CONFIG_ERROR）；约束⑤ 的 <code>fallback_class</code> 必填仅在 <code>frame.classify.enabled = true</code> 时执行（时间流形态不启用帧级判决，无兜底对象）。
时间流生成 (v1.13)	组合约束八条（规范源头 2.3.1，全部 CONFIG_ERROR）：① 形态前提合取——<code>generate_stream.enabled = true ⇒ run.mode="generate_only" ^ run.modality="text" ^ generate.enabled ^ classify.enabled ^ stream.order_by = "meta:<字段>" ^ output.meta_mode != "none"</code>，另含工件键守卫：<code>input.text_field</code> 与 <code>order_by</code> 的时间戳字段名均不得含 "."（工件行以字段名为层面顶层键，点路径在重放摄取时无法往返、整份判坏行，6.5）、两者互不同名、且均不得为 "truth"（工件行三个顶层键互斥）；② 类表与配额——序列类表 ≥ 1（放宽自 ≥ 2，仅本形态）$\wedge$ <code>fallback_class</code> 必填（写了须 $\in$ 类表）$\wedge$ <code>Σsequences ≥ 1 ^</code> 参与类（有效 <code>sequences ≥ 1</code>）的有效 <code>instruction</code> 非空 $\wedge$ 帧类表非空 $\wedge$ 每个帧类的 <code>[frame.class.<name>.generate].instruction</code> 非空；③ 禁设键探针——<code>[generate]</code> 的 <code>seed_examples / standalone_count / num_per_record / seeds_per_call</code> 与 <code>[class.*.generate]</code> 的 <code>num_per_record / seeds_per_call</code> 显式书写、<code>frame.classify.enabled / frame.annotate.enabled = true ⇒</code> 定向 CONFIG_ERROR（机制 = v1.11 <code>use_vision</code> 原始节探针，不走「未知键报 warning」前向兼容路径；文案给替代面指引）；④ 装箱一致性——<code>sessions ≥ 1 ^ sessions ≤ Σsequences ≤ 2 × sessions ^ duplicates ∈ [0, Σsequences] ^ noise_ratio ∈ [0,1)</code>（<code>> 0 ⇒ noise_instruction</code> 非空）$\wedge$ <code>frame_gap_s</code> 满足 <code>1e-6 ≤ lo ≤ hi < stream.gap_s</code>；仅当 <code>--limit</code> 后的实际非零配额前缀存在生效 <code>rules/windows</code> 时，v1.16 联合规划路径才允许 <code>hi == stream.gap_s</code>（即 <code>hi ≤ stream.gap_s</code>）；只有 <code>sequence_validator</code> 而没有实际非零 <code>rules/windows</code> 前缀时仍执行严格 <code><</code>；⑤ 织造上限与铺设契约——<code>2 × max(各类 len_range 上界) ≤ stream.session_max_len ^ stream.key == [] ^ stream.gap_steps == 0 ^ (session_max_span_s > 0 时) (session_max_len - 1) × frame_gap_s 上界 ≤ session_max_span_s ^ ts_start</code> 可 <code>datetime.fromisoformat</code> 解析；⑥ Schema 元校验——帧类生成 Schema 走 <code>_load_schema_pair</code> 全套 + <code>\$ref</code> 遍历（无 <code>_meta</code> 分支——帧内容落工件行的文本字段，与 §6.3 信封字段无冲突面），按类标注 Schema 见本表「按类覆盖合并」行 ⑤；⑦ 引用集与豁免——<code>classify.llm</code> 在本形态仅豁免密钥解析集（序列标签直接继承、classify 零判决调用，援引 S30 先例：零调用不强制配活密钥）；存在性检查照旧（拼错 profile 名仍在启动期揪出），<code>--probe</code> 引用集亦照旧（<code>referenced_profiles()</code> 按 <code>classify.enabled</code> 收录——显式探测时该 profile 仍需可连通）；⑧ 停放豁免精确化——<code>[stream]</code> 与 <code>[frame.*]</code> 在本形态是生效面，移出 R8 <code>no-op</code> 停放清单（<code>[segment] / [stitch] / [extract]</code> 照旧停放告警）。rubric 空串解析扩展（S29 扩展）：条件由 <code>segment.enabled</code> 扩为 <code>segment.enabled v generate_stream.enabled ⇒ ""</code> 解析为 <code>"default:trajectory"</code>（loader 与 M11 镜像两处同步，3.11.2；本形态打的同样是序列分）。注意「<code>trajectory rubric ^ extract.enabled = false</code>」的组合提示 WARN（本表时序流警告行 ④）在本形态下不出现——该提示归属 <code>segment.enabled</code> 分支，而本形态 <code>segment</code> 恒关；语义上也无提示对象：<code>extract</code> 是 UI 模式专属、本形态恒 <code>text</code>，「步骤」本就读作帧间变化。静态预算预检两段（V13③ 表新增）：<code>generate.stream.plan</code> 段 = 冻结模板头 <code>TEMPLATE_HEAD_TOKENS["generate_plan"]</code> + <code>max(全局与各类视图的生成 instruction)</code> + 帧类表文本；<code>generate.stream.realize</code> 段 = 冻结模板头 <code>TEMPLATE_HEAD_TOKENS["generate_realize"]</code> + 同一 instruction 项 + <code>max(len_range 上界) × max(帧类生成 Schema 文本)</code>（逐位契约把 Schema 文本按步重复）；噪音批量实现复用既有 <code>generate</code> 段。判则同「上下文预算与视觉推导」行（<code>est ≥ input_budget ⇒</code> CONFIG_ERROR、<code>> 50% ⇒</code> WARN）。annotate 段口径修订：该段的 Schema 项改按类取值——<code>max</code> 跑在「Schema + instruction + few-shot」的整份和上（无按类 Schema 时逐类回落全局，数值与 v1.12 字节等价）。

帧类构成档位与时间字段绑定 (v1.14)	档位簇六条（规范源头 2.3.1 v1.14 段，除注明 WARN 外全部 CONFIG_ERROR）——v1.15 逐生效表化注记：下列 ②③ 的辖区是「一张生效档位表」、④⑤⑥ 的辖区是「逐参与类的生效表」，生效表 = 该类声明的按类表、未声明则为全局表（详见下方「按类档位表」行；无任何按类表时生效表恒 = 全局表，本簇与 v1.14 逐字同义）：① 档位表前提——<code>[[generate.stream.tiers]]</code> 在场 ⇒ <code>generate_stream.enabled = true</code>（定向报错：本表仅时间流形态合法）；② 档位身份——<code>tier_rank</code> 正整数、表内唯一、全表连续覆盖 1..N（N = 表长；缺号/重号报错并点名缺失序数），<code>weight</code> 整数 ≥ 1；③ 档位构成——<code>frame_classes</code> 非空、档内无重复、每名 ∈ 帧类表 <code>name</code> 集，各档构成集合两两互异；④ 长度可覆盖——逐（参与类，档）非零配额对裁定（配分为纯函数，M1 期即可算出逐档配额）：配额 ≥ 1 的每对须满足该类 <code>len_range</code> 下界 ≥ <code>len(该档 frame_classes)</code>，零额对豁免；⑤ 配分零额 ⇒ WARN（值-free：类名 + <code>tier_rank</code> + 权重表；非错误）；⑥ 帧类未入档 ⇒ WARN（该帧类不会出现在任何蓝图中、其 <code>[frame.class, <name>.generate]</code> 面为死配置），同时把本表「时间流生成」行约束②的「每个帧类的生成 instruction 非空」检查域收窄为 U 各档 frame_classes（未入档帧类豁免必填，已写照常合法）。绑定簇三条：⑦ 绑定表前提——<code>[frame.class.<name>.generate.time_fields]</code> 仅结构化帧（该帧类声明了 <code>schema_path / schema_inline</code>）合法，纯文本帧带绑定表 ⇒ 定向 CONFIG_ERROR；「载荷恒为 JSON 对象」由既有的帧类生成 Schema 顶层 <code>type = object</code> 必检承担（本表「用户 Schema」行同族），对「声明过 Schema 源键但装载失败」的帧类本簇保持沉默、不叠加第二错；⑧ 绑定键与类型——每个绑定键 ∈ 该帧类生成 Schema 顶层 <code>properties</code>、绑定值 ∈ 语义词表 <code>{ts, gap_prev_s, gap_next_s, elapsed_s}</code>、该属性 Schema 的 <code>type</code> 关键字面恰等于要求值（<code>ts</code> ⇒ "string"、其余三值 ⇒ "number"；联合类型数组、缺失、经 <code>\$ref</code> / 组合关键字间接声明均判不匹配），绑定字段携带 <code>type</code> 以外的约束关键字 ⇒ WARN（值-free：帧类名 + 字段名 + 关键字名——那些关键字既不上行也不被强制）；⑨ 剔除余量——生成 Schema 顶层 <code>properties</code> 键数 - 绑定键数 ≥ 1（全绑定 ⇒ CONFIG_ERROR）。解析产物：<code>GenerateStreamConfig.tiers</code>（按 <code>tier_rank</code> 升序存放的 <code>TierSpec</code> 元组，空 = 档位面不在场）与 <code>FrameClassView.time_fields</code>（<code>None</code> = 无绑定）；<code>[frame.class, <name>.generate]</code> 的白名单由三键扩为四键（<code>instruction / schema_path / schema_inline / time_fields</code>，本表「帧粒度配置」行 v1.13 放宽段的同步修订——不扩键则子表被白名单循环判为 CONFIG_ERROR）。微秒地板（v1.13 缺陷修补）：本表「时间流生成」行约束④的 <code>0 < lo</code> 收紧为 <code>lo ≥ 1e-6</code>（isoformat 精度与 <code>round(·, 6)</code> 的分辨率下界；亚微秒 <code>lo</code> 使帧间隔 <code>timedelta</code> 取整为 0 微秒，破坏 <code>ts</code> 严格递增并使语义词表的 0.0 边界哨兵失去无歧义性——报错文案给出这两条依据）。静态预算预检零改动：「时间流生成」行的 <code>generate.stream.plan</code> 段照旧按全帧类表计量（档内子集恒 ≤ 全表，上界性质保持），<code>generate.stream.realize</code> 段在绑定剔除后只降不升，<code>TEMPLATE_HEAD_TOKENS</code> 无新键。
按类档位表 (v1.15)	按类表前提三子款（规范源头 2.3.1 v1.15 段，全部 CONFIG_ERROR，定位串带类名）：① 形态门——<code>generate_stream.enabled = false</code> 时任一 <code>[class.?.generate]</code> 节写了 <code>tiers</code> ⇒ 定向 CONFIG_ERROR（机制 = v1.11 <code>use_vision</code> 的原始节探针，与 v1.12 帧级两键同族，不走本表「TOML 结构」行的前向兼容 warning；探针在解析删除后于原始 <code>[class.<name>.generate]</code> dict 上探键存在性，故表内容本身非法时也照发本错）；② 全局锚——形态开启、任一按类表在场而全局 <code>[[generate.stream.tiers]]</code> 缺省 ⇒ CONFIG_ERROR（档位面总开关恒 = 全局表非空，文案指引补全局表并说明它兼作未声明类的回落）；③ 空表拒收——显式 <code>tiers = []</code>（解析产物为空元组）⇒ CONFIG_ERROR，指引删除该键以回落全局表。逐生效表化（修订上一行档位簇）：档位身份连续性（②）与构成合法性（③）改逐生效来源表执行——全局表 + 每张已声明按类表各跑一遍，报错定位前缀分别为 <code>[[generate.stream.tiers]]</code> 与 <code>[class.<name>.generate].tiers</code>，「构成两两互异」的辖区收窄为单表之内（跨类同构成合法）；逐（参与类，档）的长度可覆盖约束（④）与配分零额 WARN（⑤）改吃 <code>effective_tiers</code>（该类表，全局表），WARN 文案的权重清单同取该类生效表；「帧类未入档」WARN 与「每帧类生成指令必填」的检查域并集化为 U（各参与类生效表构成，⑥）——全部参与类都声明按类表时全局表沦为纯锚，其独有帧类照样判死配置。零额类不豁免结构校验：<code>sequences = 0</code> 的类声明的表照跑本行与 ②③ 的结构校验（坏配置早报），仅 ④⑤ 豁免，且其构成不入检查域并集。解析产物：<code>ClassView.tiers</code>（按 <code>tier_rank</code> 升序存放的 <code>TierSpec</code> 元组；<code>None</code> = 未声明 ⇒ 回落全局，空元组 = 显式空表 ⇒ 由 ③ 拒收）与纯函数 <code>effective_tiers</code>（类表，全局表）（<code>apportion_tiers</code> 旁的零 rng 纯函数，M1 约束簇 / M6 计划期 / M10 报表装配三方共用，3.6.5）；<code>[class.<name>.generate]</code> 白名单由六键扩为七键（增 <code>tiers</code>，5.2 按类覆盖白名单表——不扩键则该数组表被白名单循环判为 CONFIG_ERROR）。零改动确认：微秒地板（rule 59 家族）、时间字段绑定簇、<code>[frame.class.?.generate]</code> 白名单、帧类表本身的规则、静态预算预检（蓝图段照旧按全帧类表计量，上界性质在按类子集下同样保持）全部不动。
API Key	每个被引用 <code>profile</code> 的 <code>api_key_env</code> 环境变量必须存在且非空。v1.6 密钥池：<code>api_key_env</code> 与 <code>api_key_envs</code>（5.1）恰提供其一（两者皆有或皆无均报错）；<code>api_key_envs</code> 须为非空数组、逐项非空且互异；被引用 <code>profile</code> 的每个列出变量均须存在且非空（逐个缺失逐条聚合报错）。M1 归一化：标量形式解析为长度 1 的密钥池，运行时只有一条代码路径（3.9.3 密钥池行）。
用户 Schema	必须是合法 JSON 且通过 JSON Schema draft 2020-12 元 Schema 校验（<code>jsonschema</code> 库 <code>Draft202012Validator.check_schema</code>）；顶层 <code>type</code> 必须为 <code>object</code>；顶层不得声明保留键 <code>_meta</code>。同族两处：v1.12 的帧级 Schema（<code>[frame.annotate].schema_*</code>，恰一，无 <code>_meta</code> 分支）与 v1.13 的按序列类标注 Schema（<code>[class.<name>.annotate].schema_*</code>，至多其一，<code>_meta</code> 禁令与 <code>\$ref</code> 遍历照旧）走同一套装载分支——三者的解析产物分别是 <code>user_schema / frame_schema / ClassView.schema</code>（本表「按类覆盖合并」行 ⑤、「帧粒度配置」行 ③）。
Rubric	<code>criteria</code> 非空、<code>key</code> 唯一且为 <code>[a-z0-9_]+</code>、<code>weight > 0</code>；<code>pointwise</code> 模式要求每条 <code>criterion</code> 提供 <code>pointwise_levels</code>（恰好 6 级，0—5）。
阶段组合	2.3.1 节的四条组合约束（①—④；④与本表「运行模式」行联动）。
路径	<code>process</code> 模式：<code>input</code> 存在且可读，且 <code>output</code> 不得位于 <code>input</code> 目录内部（防止自吞）；<code>generate_only</code> 模式无 <code>input</code>，本行仅执行 <code>output</code> 检查（见「运行模式」行）。两种模式均要求 <code>output</code> 父目录存在且可写。

3.1.4.1 v1.16 序列规则、日历窗口与联合规划

时间流生成新增的约束面在启动期一次性冻结。全局 `[[generate.stream.rules]] / [[generate.stream.windows]]` 与 `[[class.<name>.generate.rules]] / [[class.<name>.generate.windows]]` 都只在 `generate_stream.enabled = true` 的 `generate_only` 文本工程中合法；关闭该形态时显式写入这些键是定向 **CONFIG_ERROR**，不是静默停放。类表采用三态原子语义：类节缺省为继承全局表，显式空表示清空，非空表整体替换全局表，不逐行合并；因此可以只有类表而没有全局约束表。对实际 `--limit` 前缀没有非零配额的类，约束不激活联合规划。

规则模板是有限集：`existence`、`absence`、`exactly`、`init`、`end`，以及 `responded_existence`、`co_existence`、`response`、`precedence`、`succession`、`alternate_response`、`chain_response`、`chain_precedence`、`not_co_existence`、`not_succession`。一元模板使用 `frame_class`，二元模板使用不同的 `source / target`；只有存在性三板接受正整数 `count`，只有二元模板接受半开 `time_s = [lo, hi)` 与 `correlation`。同一生效表内的完全重复声明是配置错误。时间值以无损微秒整数求解，必须满足 `1us <= lo < hi`；相邻重放边界为 `1us <= delta <= stream.gap_s`，有序链规则的显式时间区间必须与该边界相交。

`correlation` 只接受 `{operator = "equal", source_field = "...", target_field = "...}"`。两侧必须是结构化帧生成 Schema 的顶层 `properties` 且同时 `required`，不得是 `time_fields` 绑定字段，Schema 的 `type` 关键字必须字面相等。运行时在时间字段回填前比较 JSON 类型和值（对象键序不敏感、数组顺序敏感），正相关未命中与仅时间不满足分别计入 `correlation_scrapped` / `temporal_scrapped`；负相关未命中计入前者。

窗口表每个帧类最多一条。`of_day` 必须是非空的同日半开区间，接受 `HH:MM`、`HH:MM:SS` 或微秒精度，不能跨午夜且不得重叠；`of_week` 只能使用 `mon` 到 `sun` 且不得重复，缺省表示整周。窗口以 `ts_start` 的固定 UTC offset 解释，不引入 IANA 时区或夏令时。

`[generate].sequence_validator` 是可选的 `module:function`，目标函数签名为 `def validate_sequence(value: SequenceValidationInput) -> list[str]`。M1 只解析引用并确认可调用；运行时收到序列类、`tier_rank` 与按序位置/帧类/JSON-compatible 深拷贝载荷。钩子异常按违规处理，日志只允许引用、异常类型和违规数，不得泄露返回文本或载荷。

只要实际生效规则或窗口存在，M1 即对每个参与类、每个生效档位和 `len_range` 的每个候选长度做局部可行性检查，并对完整的 `--limit` 配额前缀做联合检查。M1、`estimate_run` 与 M6 共用同一个 OR-Tools CP-SAT 问题入口；单线程、固定搜索种子、最大确定性时间预算为 10 秒，模型 `proto` 条目超过 250,000 直接 `CONFIG_ERROR`。`INFEASIBLE` 或预算内无法验证的 `UNKNOWN` 是配置错误，`MODEL_INVALID` 是 `InternalError`（退出码 4）；带噪音目标时仅 `OPTIMAL` 可接受。没有任何非零规则/窗口且未配置序列钩子时，规划面完全关闭，时间流生成保持 v1.15 的默认路径与抽签顺序。

3.1.5 错误处理

所有校验错误聚合为一个 `ConfigError`，逐条打印（格式 `config.toml:[llm.default].timeout_s: expected positive integer, got "abc"`；数组元素定位写作 `[[rubric.criterial]][N]`，N 为 1 起序号），退出码 2。报错文案为英文（2026-08-14 代码规则整改；<文件>:[节].键：定位前缀机器稳定不变），不存在运行期配置错误——这是 M1 对其他模块的契约。

背书：「声明式配置 + 启动期全量校验 + 运行期只读」是 Data-Juicer 配方（recipe）体系 [4] 与 distilabel Pipeline 先验校验（在任何推理发生前校验 DAG 与列契约）[5] 的共同设计；TOML 双文件分层对应其「系统配置 / 数据配方」分离。

3.1.6 输入 / 输出示例

贯穿示例（文本模式）：对输入法采集的中文指令做意图标注，输入数据行形如 `{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}`（格式规格见 6.1）。注意：M1 不读取数据内容，仅按 3.1.4 校验 input 路径存在且可读。

示例 ①：一次成功装载 —— 三源合并与优先级生效

```
$ export LABELKIT_KEY_DEFAULT=sk-*****
$ labelkit run --config config.toml --project project.toml --limit 100
```

`config.toml` 沿用 5.1 完整示例（含 `[llm.default]` 与 `[llm.judge]` 两个 profile），并在 `[llm.default]` 中显式写入 `temperature = 0.0`；`project.toml` 节选：

```
# — project.toml (意图标注工程, 节选) —
schema_version = 1

[run]
input = "./ime-logs/2026-06-30.jsonl"
output = "./out/intent-0630.jsonl"
modality = "text"
batch_size = 128 # 覆盖内置默认 256

[input]
text_field = "instruction"

[annotate]
llm = "default"
instruction = "你是输入法指令理解标注员。判断每条用户指令的意图类别、话题与完成难度。"

[output]
schema_inline = """
{"$schema": "https://json-schema.org/draft/2020-12/schema", "type": "object",
 "properties": {
   "intent": {"type": "string", "enum": ["writing_assist", "qa", "translation", "chitchat", "other"]},
   "topic": {"type": "string"},
   "difficulty": {"type": "string", "enum": ["easy", "medium", "hard"]},
   "required": ["intent", "topic", "difficulty"], "additionalProperties": false}
  """
```


三源合并后的 `ResolvedConfig` 摘录（冻结后运行期只读；`# ←` 标注每项的生效来源）：

```
run.batch_size      = 128          # ← project.toml [run] (覆盖内置默认 256)
run.seed            = 0            # ← 内置默认 (两文件与 CLI 均未提供)
limit               = 100          # ← CLI --limit (最高优先级: CLI 参数 > project.toml > config.toml)
tool.log_level      = "info"       # ← config.toml [tool] (CLI 未传 --log-level, 不触发覆盖)
llm.default.temperature = 0.0      # ← config.toml [llm.default] (project.toml 无对应覆盖键)
llm.default.max_concurrency = 8    # ← config.toml [llm.default]
annotate.llm        = "default"    # ← 内置默认 (已校验 profile 存在于 config.toml [llm.*])
quality.rubric       = "default:text" # ← 内置默认 (缺省按 run.modality = "text" 自动选定)
```

API Key 校验按「被引用 profile」收敛：本工程 `generate/verify` 均未启用、`quality` 与 `annotate` 均引用 `default`，故 M1 只要求 `LABELKIT_KEY_DEFAULT` 存在且非空；`[llm.judge]` 未被引用，`LABELKIT_KEY_JUDGE` 缺失不报错（3.1.4）。

示例 ②：聚合校验失败 —— 3 个典型错误一次性反馈

在示例 ① 的 `project.toml` 上引入 3 处错误（节选，其余内容不变）：

```
[quality]
llm = "fast"          # 错误①: config.toml 只声明了 [llm.default] 与 [llm.judge]
rubric = "inline"

[[rubric.criteria]]
key = "intent_clarity"
weight = 2.0
description = "指令意图是否清晰可辨"
pairwise_prompt = "哪条指令的意图更清晰？"

[[rubric.criteria]]
key = "Topic-Match"    # 错误③: 含大写与连字符, 违反 [a-z0-9_]+
weight = 1.0
description = "话题是否明确可归类"
pairwise_prompt = "哪条指令的话题更明确、更易归类？"

[output]
schema_inline = """
{"type": "object",
  "properties": {
    "intent": {"type": "string", "enum": ["writing_assist", "qa", "translation", "chitchat", "other"]},
    "topic": {"type": "string"},
    "difficulty": {"type": "string", "enum": ["easy", "medium", "hard"]},
    "_meta": {"type": "object"}},
  "required": ["intent", "topic", "difficulty"], "additionalProperties": false}
"""
# ↑ 错误②: properties 顶层声明了保留键 _meta (3.1.4 禁止; 该键为 6.3 输出信封, 由工具写入)
```

M1 按 3.1.5 规定聚合为单个 `ConfigError` 逐条打印（不在首错即停，且未发起任何网络请求）：

```
$ labelkit run --config config.toml --project project.toml --limit 100
ConfigError: 3 config error(s) (all aggregated)
project.toml:[quality].llm: referenced profile "fast" does not exist in config.toml [llm.*], available: default, judge
project.toml:[output].schema_inline: user schema must not declare the reserved top-level key "_meta" (the 6.3 envelope fields are
written by the tool), got properties containing "_meta"
project.toml:[[rubric.criteria]][2].key: expected a match of [a-z0-9_]+, got "Topic-Match"
$ echo $?
2
```

三条错误分属 3.1.4 校验表的「Profile 引用」「用户 Schema」「Rubric」三类，按该行序输出。`labelkit validate --config config.toml --project project.toml` 会产生完全相同的错误清单与退出码 2（2.4），适合在提交长任务前做零成本的纯本地检查。

3.2 M2 数据接入 ingest

3.2.1 职责与边界

做：把输入路径物化为 Record 迭代器：文本模式逐行解析 JSONL 并抽取文本字段；UI 模式递归扫描、按 index 配对文件、解析 UI 树节点并构造惰性图像引用；为每条记录计算确定性 id；对坏数据执行跳过策略并计数。（v1.8）stream 模式（segment.enabled = true）下另担 [stream] 声明的输入侧排序、流式单调性校验与规则层会话化，向 M10 暴露会话流视图（3.2.8）。**不做：**不加载图像像素（只 stat 文件并记录路径/尺寸）；不截断/清洗文本（原样保留，序列化截断是消费方在构造提示词时的职责）；不判重、不打分；run.mode="generate_only" 时本模块整体不参与（3.10.3）。

3.2.2 输入 / 输出

方向	内容
输入	run.input 路径。文本模式：单个 .jsonl 文件，或目录（取其下所有 *.jsonl，按文件名字典序）。UI 模式：目录（递归扫描）。
输出	Iterator[Record]（生成器，按需产出，供 M10 切批；v1.8 stream 模式下 M10 改消费 sessions() 会话流视图——整会话装箱，3.2.8/3.10.3）；IngestReport（总行数、坏行数、缺对数、index 冲突数及其文件位置列表；v1.8 只增会话数 sessions 与乱序跳过数 disorder 两计数，3.2.8）。

3.2.3 API

```
class Ingestor:
    def __init__(self, cfg: ResolvedConfig): ...
    def scan(self) -> IngestPlan:
        """只扫描不解析：返回文件清单、配对表、预估记录数。--dry-run 与 validate 用。
        v1.8 (S23): stream 启用时文本模式的行数统计与会话空跑单遍融合（3.2.8）。"""
    def records(self) -> Iterator[Record]:
        """惰性产出 Record。解析错误按 input.on_bad_line / input.on_missing_pair 策略处理。
        非 stream 入口，v1.8 零改动。"""
    def sessions(self) -> Iterator[Session]:
        """v1.8: stream 模式的会话流视图，M10 以之取代 records() 消费（3.2.8）；产出
        Session(session_id, records, cause)——形态冻结于 CONTRACTS §7.1。"""
    @property
    def report(self) -> IngestReport: ...
```

3.2.4 算法：UI 模式文件配对

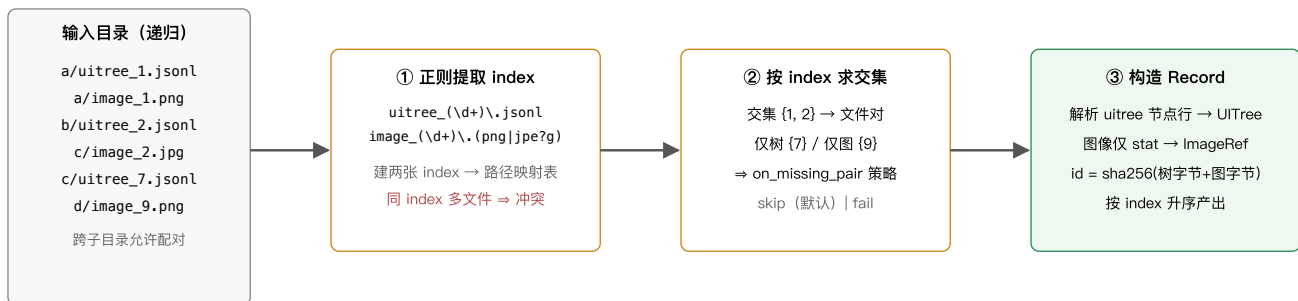


图 3-1 UI 模式文件对扫描与配对流程

配对规则的精确定义：

规则	定义
index 命名空间	整个输入目录（含全部子目录）共享一个 index 空间；index 为文件名正则捕获组按十进制解析的整数（允许前导零，image_007.png 的 index 为 7）。
冲突	同一 index 出现 ≥2 个 uitree 文件或 ≥2 个 image 文件 ⇒ 该 index 记为冲突。input.on_index_conflict = "fail"（默认，退出码 3）或 "skip"（整个 index 跳过并计数）。同 index 同时存在 .png 与 .jpg 亦视为冲突。
缺对	index 只有单侧文件 ⇒ input.on_missing_pair = "skip"（默认）或 "fail"。
UI 树文件格式	JSONL，每行一个控件节点对象（字段映射见 6.2）。空文件或全坏行 ⇒ 该记录按坏记录跳过。单文件也允许仅一行（整树作为一个节点对象给出、含 children 嵌套）——两种导出风格都支持，解析器先探测：若首行对象含 children 数组则按嵌套树解析，否则按平铺节点行解析。
图像约束	PNG/JPEG；单文件 ≤ input.max_image_mb（默认 20）；超限按坏记录跳过。仅校验魔数与尺寸，不解码全图。

3.2.5 文本模式解析

每行必须是 JSON object（非 object 的合法 JSON 视为坏行）。标注/打分所用文本由 `input.text_field`（支持点路径，如 `"conversation.turns"`；默认 `"text"`）抽取：命中字符串则直接使用；命中数组/对象则按 canonical JSON（`sort_keys=True`, `ensure_ascii=False`，紧凑分隔符）序列化为文本；未命中字段 $\Rightarrow$ 坏行。原始对象完整保留在 `Record.raw`，输出时可按 `output.passthrough_fields` 透传（见 6.3）。记录 `id = sha256(canonical_json(raw))[16]`。坏行策略 `input.on_bad_line = "skip"`（默认）| `"fail"`。**v1.13 工件可重放注记**：时间流生成形态产出的时间流工件（6.5）按本节规则逐字节可重放——M6 构造成员 `Record` 时同样取「工件行全对象」为 `raw`（含 `truth` 字段）并套用本行 `id` 公式，故把工件当输入重跑时 M2 算出的成员 `id` 与生成侧完全一致；`truth` 字段只是行上的普通字段，参与 `id` 计算、不参与任何判定，需要时经 `output.passthrough_fields` 透传出来做对照（3.6.5、6.5）。

3.2.6 配置项

见 5.2 [input] 节字段表（本模块消费其全部字段）。

背书：「截图 + accessibility tree/视图层级」文件对是 GUI 智能体数据集的标准物理形态：ScreenAI 的 screen schema 即截图与线性化控件树的配对 [13]，OS-Atlas 跨 5 平台的 1300 万控件语料同样以截图+树组织 [16]，AMEX/AndroidControl 等移动端数据集亦然。逐行 JSONL、按文件名索引的组织方式与 Dolma toolkit 的分片 JSONL 惯例一致 [6]。

3.2.7 输入 / 输出示例

① 文本模式（`run.modality = "text"`，`input.text_field = "instruction"`）

`run.input = "./ime-logs"`，目录下仅一个文件 `ime-2026-06-30.jsonl`（输入法采集的中文指令数据），共 3 行；第 3 行不含 `instruction` 字段，按 3.2.5 判为坏行：

```
{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}
{"instruction": "把这句话翻译成英文：会议改到周五下午三点", "source": "ime-log", "ts": "2026-06-30T10:15:21Z"}
{"query": "今天天气怎么样", "source": "ime-log", "ts": "2026-06-30T10:20:05Z"} ← text_field 未命中  $\Rightarrow$  坏行
```

`records()` 产出 2 个 `Record`（字段见 4.1；`id` 按 3.2.5 = `sha256(canonical_json(raw))[16]`，下列值为对 canonical JSON 实际计算的结果，可复算验证）：

```
// Record #1 (第 1 行)
{
  "id": "1cda030abc565f17",
  "modality": "text",
  "text": "帮我写一条请假条，明天上午要去医院",
  "raw": {"instruction": "帮我写一条请假条，明天上午要去医院",
    "source": "ime-log", "ts": "2026-06-30T10:12:00Z"},
  "ui_tree": null, "image": null,
  "ref": {"source_file": "ime-2026-06-30.jsonl", "line_no": 1,
    "pair_index": null, "generated_from": []}
}
// Record #2 (第 2 行)
{
  "id": "a9bbd04dca155b52",
  "modality": "text",
  "text": "把这句话翻译成英文：会议改到周五下午三点",
  "raw": {"instruction": "把这句话翻译成英文：会议改到周五下午三点",
    "source": "ime-log", "ts": "2026-06-30T10:15:21Z"},
  "ui_tree": null, "image": null,
  "ref": {"source_file": "ime-2026-06-30.jsonl", "line_no": 2,
    "pair_index": null, "generated_from": []}
}
```

第 3 行按默认 `input.on_bad_line = "skip"` 跳过并计数，迭代结束后 `report` 属性给出 `IngestReport`（计数口径与 6.4 `report.json` 的 counts 一致）：

```
{
  "scanned": 3, "ingested": 2, "bad_input": 1,
  "missing_pair": 0, "index_conflict": 0, // 仅 UI 模式会非零
  "bad_locations": [
    {"file": "ime-2026-06-30.jsonl", "line_no": 3,
      "reason": "input.text_field \"instruction\" missed"}
  ]
}
```

② UI 模态 (`run.input = "./capture/2026-07-01"` , 即 5.2 示例工程)

`b/uitree_2.jsonl` 为平铺节点行风格 (首行无 `children` 数组, 3.2.4 探测规则)。字段名取 6.2 映射表的源字段名: `id/parent/class/text/bounds/visible`; 根节点省略 `parent` $\Rightarrow$ `parent_id = null`; `hint` 不在白名单 $\Rightarrow$ 值转字符串后入 `extra`:

```
{ "id": "0", "class": "FrameLayout", "bounds": [0, 0, 1080, 2340], "visible": true }
{ "id": "1", "parent": "0", "class": "TextView", "text": "登录", "bounds": [72, 296, 264, 392], "visible": true }
{ "id": "2", "parent": "0", "class": "EditText", "bounds": [72, 520, 1008, 664], "visible": true, "hint": "请输入手机号" }
{ "id": "3", "parent": "0", "class": "EditText", "bounds": [72, 712, 672, 856], "visible": true, "hint": "请输入验证码" }
{ "id": "4", "parent": "0", "class": "Button", "text": "获取验证码", "bounds": [704, 712, 1008, 856], "visible": true }
{ "id": "5", "parent": "0", "class": "Button", "text": "登录", "bounds": [72, 952, 1008, 1096], "visible": true }
```

正则从 `b/uitree_2.jsonl` 与 `c/image_2.png` 各提取 `index = 2`, 跨子目录求交集配对成功 (图 3-1), 构造的 `Record`:

```
{
  "id": "9f2c31ab52e08d17", // sha256(树字节 + 图字节)[:16], 与 6.3 _meta 示例同源
  "modality": "ui",
  "text": null, "raw": null,
  "ui_tree": UITree(nodes = 6  $\times$  UINode, 深度优先序, 序列化见下),
  "image": { "path": "capture/2026-07-01/c/image_2.png", "format": "png", "size_bytes": 483112 },
  "ref": { "source_file": "b/uitree_2.jsonl", "line_no": null,
    "pair_index": 2, "generated_from": [] }
}
```

`UITree.serialize()` 按 4.3 规范 (深度缩进 + `role` + 引号 `text` + `[l,t,r,b]` + 非空 `extra` 的 `k=v`; 此处 `quantize_px = 0`, 即 M5 组装提示词时的形态, 3.5.2 示例复用本输出):

```
FrameLayout [0,0,1080,2340]
  TextView "登录" [72,296,264,392]
  EditText [72,520,1008,664] hint=请输入手机号
  EditText [72,712,672,856] hint=请输入验证码
  Button "获取验证码" [704,712,1008,856]
  Button "登录" [72,952,1008,1096]
```

提示: 两处 `id` 规则不同——文本模态对 `raw` 的 canonical JSON 取哈希 (与行号、文件名无关, 内容相同即 `id` 相同, 为 M3 精确判重的基础); UI 模态对树文件字节与图像文件字节拼接取哈希。图像此时仅 `stat` (`size_bytes = 483112`, 远小于 `input.max_image_mb = 20` 上限), 像素到 M5 组装提示词时才经 `ImageRef.load_base64()` 加载。

3.2.8 时序流排序与会话化 (v1.8)

`segment.enabled = true` (stream 模式, 2.3.1) 时本模块承担 `[stream]` 节声明的输入侧排序、流式单调性校验与规则层会话化 (配置键表见 5.2; 边界的语义精化属 M14, 3.14——会话化留 M2、装箱留 M10 的分工见 3.10.3)。产出面变化: M10 改消费 `sessions()` 会话流视图 (3.2.3) 而非 `records()`, 切批改整会话装箱。

排序键 (`stream.order_by`)。"input_order" (默认): 文本 = 文件名字典序 $\rightarrow$ 行号, UI = `pair_index` 升序 (即现行 `records()` 顺序)。`"meta:<field>"` (仅文本模态, M1 校验 3.1.4): `<field>` 为原始行对象上的点路径字段, 时间戳解析规格见 6.1 (S20)——数值 `v < 0` `v v  $\geq$  1e14` 解析失败、`v < 1e11` 判 epoch 秒、`[1e11, 1e14]` 判毫秒 ($\div 1000$); 字符串先试纯数字 (走数值规则) 再试 `datetime.fromisoformat` (3.11 起原生接受 `Z` 后缀), 均败 = 解析失败; aware 值换算 UTC epoch、naive 值按 UTC 解释; 内部序键 = float 秒。

流式单调性校验 (S19/S20)。不做全量重排: 单调性游标按 `stream.key` 分区键各自维护 (dict, 内存 = 键基数)——逐设备/逐来源拼接的输入不会被整体判乱序; 键变即断会话 (groupby 语义非 `keyBy`, 输入须按分区键成组; 交错流列演进候选, 8.4)。乱序记录与时间戳解析失败同走 `stream.on_disorder: "skip"` (默认) = 跳过 + 计 `bad_input` + `IngestReport.disorder` 子计数 + 每记录一条 `ingest.disorder` 事件 (7.2; `stderr` WARN 每运行仅镜像一次); `"fail"` = `InputError`, 退出码 3。

会话装配器 (规则层, 纯代码零 LLM)。在 (已 `--limit` 截断的) 解析流上按下列条件闭合候选会话, 任一触发即断 (键表 5.2):

- `stream.key` 键变即断 (`"meta:<field>"` 仅文本模态; `"source_dir"` = `ref.source_file` 父目录派生, UI 模态可用——一次采集一目录惯例例, S19); 文本模态 `order_by = "input_order"` 下源文件变更恒闭合会话 (`cause = "key"`, 即使 `stream.key = []`)——`line_no` 的语义不跨文件成立、无时间戳可桥接文件边界; `order_by = "meta:*"` 时文件边界透明 (轮转日志场景) (v1.8 D7 登记);
- `stream.gap_s` (相邻记录时间差 $>$ `gap_s` 秒即断; 仅 `order_by="meta:*" / stream.gap_steps` (序号差断开, 0 = 不启用)——两者可并用, 任一触发即断;
- `stream.session_max_len` (默认 200, 硬上限) / `stream.session_max_span_s` (时间跨度硬上限, 0 = 不启用; 仅 `meta:*`);
- 流耗尽 (eof) 或 `--limit` 截断 (limit)。

会话闭合时发一条 `segment.session` trace 事件（**属主 M2**：事件名冠 `segment` 前缀、按前缀归 `segment` 通道，S1，7.2）——payload 含 `session_id`、`first` / `last`（首末序键）、`len`、`cause`（ $\in$ `gap \setminus key \setminus max_len \setminus max_span \setminus eof \setminus limit），并计 IngestReport.sessions。会话对象 Session(session_id, records, cause) 的形态冻结于 CONTRACTS §7.1：session_id = sha256("\n".join(会话内记录 id))[:16]（会话序）、records 为按会话序的成员 Record 元组、cause 即上述闭合同表。v1.13 工件可重放注记：时间流生成形态的交织器按同一 session_id 公式给直装信封盖章——口径含会话内全部帧（噪音帧与重发帧一并计入），且铺设的会话间隔恒 $>$ stream.gap_s、会话内帧间隔恒 $<$ stream.gap_s，故把工件当输入重放时本装配器按同一份 [stream] 声明切出的会话与生成侧逐一对应、session_id 逐字节相同（3.6.5）。`

--limit 帧级截断（S17）。`--limit` 的单位不变、仍是帧（记录）：`islice` 位于解析流与会话装配器之间；截断视同 EOF——尾部未闭合会话按会话闭合下发（`cause = "limit"`）+ `WARN` 一次。`cause = "limit"` 的精确语义是「该会话在 `--limit` 预算耗尽处闭合」：预算恰好在流末耗尽（无真截断）与真截断不可区分——消歧需要多拉取并解析一条记录，会扰动 `scanned/bad_input` 台账，工具不做（v1.8 D3 裁决）；`WARN` 文案据此陈述预算耗尽而非断言截断（2.4 `--limit` 行 `stream` 子句）。

IngestReport（v1.8 只增两字段）。`sessions` = 装配器闭合的候选会话数（`stream` 模式；`report.stream.sessions` 的数据源，6.4）；`disorder` = 单调性校验跳过的记录数——`bad_input` 的子计数，经逐条 `ingest.disorder` 事件可审计，`report` 不另设键（6.4）。

dry-run 单遍融合（S23）。文本模态 `stream` 启用时，`scan()` 的行数统计与会话空跑**单遍融合**——一次读同时产出预估记录数与会话尺寸序列（供 3.10.3 `dry-run` 的 `next-fit` 装箱与调用量估算），不做第二次全量读。

背书：规则层会话化是流处理 `session window` 原语的对应物（Apache Flink `EventTimeSessionWindows` / Apache Beam `Sessions` [55]，1.5）——`inactivity gap` + 分区键 + 硬上限三件套照抄，纯代码零 LLM 成本；`gap_s` 默认偏大的结构性论证（欠分割可由 M14 的 LLM 边界精化拯救、过分割不可逆）见 5.2 `gap_s` 行。

3.2.9 v1.16 生成工件的精确重放边界

v1.16 的联合规划器以微秒整数铺设生成任务帧时间，再由 M11 写成 ISO-8601 字符串。M2 的会话状态机仍使用既有严格大于关系：相邻序键差 $\leq$ `stream.gap_s` 留在同一候选会话，差值 $>$ `stream.gap_s` 才闭合会话。生成器保证同一会话内的帧差不超过该边界，跨会话至少多出 `1us`，因此把时间流工件作为相同 `[stream]` 输入重放时，规则层会话边界与生成侧冻结的会话一致；该约束不改变 M2 对普通输入的排序、乱序或坏行处理。

规划器投影作废序列时只移除槽位并保留幸存任务帧的原始时间戳，不重新压缩时间轴；空会话被删除并重新编号，幸存帧不会跨会话搬迁。重复序列沿用源载荷（包括已经回填的时间字段），其重发会话使用规划器为流尾安排的时间。M2 只读取工件行，不识别规则、日历窗口、相关性或序列校验钩子，这些约束在 M1/M6 已完成。

3.3 M3 去重 dedup

3.3.1 职责与边界

做：对批内（或全局作用域）记录执行精确去重与近似去重，UI 模态叠加图像 `pHash`；为重复记录标记 `dropped_dup` 状态并记录簇归属；维护运行内存中的去重索引。**不做**：默认配置下不调用任何 LLM/Embedding API、不做语义级判重（v1.2 例外：`dedup.semantic = true` 时经 M9 `embed()` 调用 `embedding API` 执行可选第④级语义判重，3.3.3——仍不调用对话补全 LLM）；不物理删除记录（状态标记，由 M10/M11 决定去向）；不跨运行持久化索引（无状态约束）。

3.3.2 输入 / 输出

方向	内容
输入	<code>list[PipelineItem]</code> （一个批）；运行级 <code>DedupIndex</code> （ <code>scope=global</code> 时跨批共享，进程内存）。
输出	同一列表，重复项 <code>status="dropped_dup"</code> 、 <code>item.dedup = DedupInfo(kind, cluster_key, kept_id)</code> ；非重复项 <code>DedupInfo(kind="unique")</code> 并入索引。

3.3.3 算法

层	算法	判重条件
① 精确	去重键 = <code>sha256(dedup_text)</code> 。 <code>dedup_text</code> ：文本模态为抽取文本经 NFC 规范化、连续空白折叠为单空格、 <code>strip</code> 后的字节；UI 模态为 UI 树规范序列化（4.3 节 <code>UITree.serialize()</code> ，不含坐标抖动位——坐标按 <code>dedup.bounds_quantize_px</code> （默认 4px）量化后参与序列化）。哈希入 <code>set</code> ， $O(1)$ 查询。	哈希相等

② 近似	MinHash: <code>dedup_text</code> 的字符 n -gram (默认 $n=5$, 对中英混合文本鲁棒; 空白折叠后取滑窗) shingle 集合 $\rightarrow$ 128-permutation 签名 $\rightarrow$ <code>datasketch.MinHashLSH(threshold=0.85)</code> 查询候选 $\rightarrow$ 对候选精算签名估计 Jaccard, $\geq$ 阈值判重。首见者入索引 (first-writer-wins, 保留簇内最早记录, 与 Lee et al. 的 cluster-keep-one 策略一致 [3])。	估计 Jaccard $\geq$ <code>dedup.minhash_threshold</code>
③ 图像 (仅 UI 模态)	截图缩放解码 $\rightarrow$ 64-bit pHash (imagehash 默认 DCT 实现) $\rightarrow$ 与索引中全部已保留 pHash 求汉明距离 (64-bit 异或 popcount, 50 万条线性扫描 $< 100\text{ms}$ /查询, 可接受; 实现里按 16-bit 前缀分桶加速)。	汉明距离 $\leq$ <code>dedup.image_phash_max_distance</code> (默认 8)
④ 语义 (可选)	<code>dedup.semantic = true</code> 时启用 (默认 false, 5.2): <code>dedup_text</code> $\rightarrow$ 经 <code>config.toml [embedding.<name>]</code> profile (由 <code>dedup.semantic_embedding</code> 引用, 5.1) 取句向量 (M9 <code>embed()</code> , 3.9.2; 向量 L2 归一化后余弦 = 点积) $\rightarrow$ 与索引中全部已保留向量求余弦相似度 (批内与全局索引通查, 实现为向量矩阵化点积的线性扫描 (pHash 的 16-bit 前缀分桶依赖汉明位结构, 不适用于稠密向量), 规模上限见 2.6 注) $\rightarrow$ 达阈值判重, <code>kind = "near_semantic"</code> (4.2); 未判重者向量入索引 (first-writer-wins, 同②)。执行序在①②③之后, 仅当前级已构成判重 (含 UI 模态合成判定成立) 时才短路——UI 模态 <code>ui_dup_requires="both"</code> 下③单独命中不构成判重、不短路④; 仅对尚未判重的记录发起 embedding。UI 模态: 作用于树规范序列化文本 (与①②同一 <code>dedup_text</code>), 在合成判定中视同 <code>tree</code> 层命中 (见下段)。成本: 每条参检记录 1 次 embedding 调用 (走 M9 计量与重试, 3.9.3)。	余弦相似度 $\geq$ <code>dedup.semantic_threshold</code> (默认 0.95, SemDeDup 论文的高相似区间 [26])

UI 模态合成判定: `dedup.ui_dup_requires = "both"` (默认, 树近似重 且 图近似重才判重——最保守, 避免同一界面模板不同内容被误杀) | `"tree"` | `"image"`。精确层 ① 命中则无条件判重。生成样本回流批同样经过本模块, 天然实现 Self-Instruct 的「新样本与已有样本相似度过滤」[18] (以 MinHash-Jaccard 替代其 ROUGE-L, 二者同为 n -gram 重叠度量, MinHash 可索引化)。

第④级 (语义, v1.2) 与合成判定的关系: UI 模态下④作用于树规范序列化文本, 在 `ui_dup_requires` 合成判定中视同 `tree` 层命中——`"both"` 下需 (②或④) 与③同时命中才判重; `"tree"` 下④单独命中即判重; `"image"` 下④不参与判定。判重由④贡献时记 `kind="near_semantic"` (④与③同时命中仍记 `near_both`)。④与②的分工: ②抓 n -gram 重叠的表层改写, ④抓措辞不同语义相同的深层重复 ([26] 的动机), 两级独立可关。

序列记录 (v1.8, S10)。stream 模式下抵达本模块的判重单元是 episode (`record.kind = "sequence"`, 3.14) ——episode 级重复 = 「同样的操作流程」; 成员帧不会单独抵达 (链序 segment 在 dedup 之前, 成员帧已置 absorbed / dropped_noise, 3.10.3), 帧级判重语义在 stream 模式下有意留空 (连续 UI 帧上帧级判重本就失效)。**v1.13 适用性注记**: 本段以「链序 segment 在 dedup 之前」为前提陈述序列记录的来源, 但该前提**不是**序列分支的生效条件——时间流生成形态 (`generate_stream.enabled`, segment 关闭) 下 M6 直接产出 `kind = "sequence"` 的直装信封, 同样按下列序列配方判重 (判别式恒为 `record.kind`, 与 segment 开关无关); 本形态下**没有**成员帧信封 (噪音帧与重发帧只活在工件里, 从不构造信封), 故 absorbed / dropped_noise 两态不出现。另注: M6 在交织之前已用同一配方 (成员文本按序 `"\x1e"` 拼接) 对兄弟序列做过一次相似度过滤 (3.6.5), 本模块是该过滤之后的第二道、口径一致。四处适配, 其余零改动:

- ①② `dedup_text`: 配方增 `kind == "sequence"` 分支 (优先于模态分支) ——成员逐条按其单记录配方 (文本规则 / 树规范序列化, 随成员模态) 产出后按成员序拼接, 分隔符 `"\x1e"` (ASCII Record Separator, 0x1E: `isspace() == True`, 而成员配方输出的规范化文本已将空白折叠为单空格、不可能含该字符——拼接串与任何单记录配方输出结构性零碰撞); ①精确与②近似两级在拼接文本上照常执行。
- ③ pHash: 对序列记录自动跳过 (序列 Record 的 `image is None` ——既有跳过门, 零新增代码路径); `ui_dup_requires = "both"` 下序列记录的合成判定降级按 `"tree"` 处理 (与图像解码失败的降级路径同款, 3.3.4)。
- ④ 语义: 参检判定与判种归类两处逻辑 (实现为 `_semantic_participates` / `_semantic_verdict_kind`) 各增序列 case——`"both"` 对序列走 `tree-only` 分支 (与③的降级一致); 序列拼接文本超长导致 embedding 重试耗尽时, 走既有 `embedding_failures` 跳过路径 (3.3.4 ——该记录按①—③判定, 不增新失败通道)。

线索记录 (v1.9)。stitch 启用时抵达本模块的判重单元升维为线索 (thread——M16 缝合后的幸存序列信封, 链序 stitch 在 dedup 之前, 3.10.3/3.16): `dedup_text` 配方**机制原样** (上列 S10 序列分支零改动) ——成员逐条按其单记录配方产出后按成员序以 `"\x1e"` 拼接, 作用对象自然是**重绑后**的成员元组, 线索级重复 = 「同样的完整操作流程 (含恢复段)」; 被并 episode 壳 (`status = "stitched"`) 被既有 `status == "active"` 处理面过滤**天然排除**、不参检不入索引 (absorbed 成员帧同理) ——**本模块代码零改动** (T13, 审计核查点 6)。

嵌入输入预算截断 (v1.11, V15)。`dedup.semantic = true` 且所引 `[embedding.<name>] profile` 声明 `context_window` 时 (0 = 未声明 = 预算关闭, 行为与 v1.10 一致), 第④级的 embed 输入 (`dedup_text` 产物, 含序列/线索拼接文本) 在发起 embedding 调用前按 `embed_budget = context_window - margin` (无输出预留, 3.9) 截断——**确定性头部保留** (`keep = "head"` 行边界截断: 嵌入语义主体在文本前部), 修复该调用点完全无截断的既有缺口; 截断计入 `report.budget.truncations` (6.4)。既有 `embedding_failures` 跳过路径 (3.3.4) **保留为兜底**——截断之外的 embedding 失败仍按①—③判定、不增新失败通道。

3.3.4 API 与配置

```
class DedupStage(Stage):
    name = "dedup"
    def __init__(self, cfg: DedupConfig, index: DedupIndex): ...
    async def run(self, batch: list[PipelineItem], ctx: RunContext) -> list[PipelineItem]: ...

class DedupIndex:
    """运行内存索引: exact set[bytes] + MinHashLSH + list[(id, phash)]
    (+ dedup.semantic=true 时 list[(id, unit_vec)], v1.2). scope=batch 时每批重建。"""
    def probe_and_add(self, rec: Record) -> DedupInfo: ...
```

配置见 5.2 [dedup]。错误处理：图像解码失败 ⇒ 该记录跳过 pHash 层（按树判定）并计入 `report.dedup.image_decode_failures`；embedding 调用失败（重试耗尽，3.9.3）⇒ 该记录跳过第④级（按①—③判定）并计入 `report.dedup.embedding_failures`（仅 `dedup.semantic = true` 时可能非零，v1.2）。

背书：MinHash 近似去重由 Lee et al. (ACL 2022) 确立为 LLM 训练数据方法学标准并证明可提升模型质量 [3]；Dolma [6]、Data-Juicer [4]、NeMo Curator [9] 的内置去重算子均为「精确哈希 + MinHash-LSH」两级结构，阈值 0.8–0.9 为通行区间。pHash 图像判重为 imagehash 库承载的工业标准做法 [9]。

3.3.5 输入 / 输出示例

设定：文本模态第一批共 4 条（DedupIndex 为空），[dedup] 全部取 5.2 默认值：`dedup.scope="global"`、`dedup.minhash_threshold=0.85`、`dedup.minhash_num_perm=128`、`dedup.ngram=5`；`input.text_field="instruction"`。输入 4 行 JSONL，依行序记为 r1–r4：

```
{
  "instruction": "帮我写一条请假条，明天上午要去医院复诊，大概十点到医院，下午两点前能回公司，请按半天事假写，语气客气一点，落款写研发部小李，谢谢",
  "source": "ime-log",
  "ts": "2026-06-30T10:12:00Z"
}
{
  "instruction": "帮我写一条请假条，明天上午要去医院复诊，大概十点到医院，下午两点前能回公司，请按半天事假写，语气客气一点，落款写研发部小李，谢谢",
  "source": "ime-log",
  "ts": "2026-06-30T10:12:07Z"
}
{
  "instruction": "帮我写一条请假条，明天上午要去医院复诊，大概十点到医院，下午两点前能回公司，请按半天事假写，语气客气一点，落款写研发部小李，谢谢",
  "source": "ime-log",
  "ts": "2026-06-30T10:14:33Z"
}
{
  "instruction": "把这段会议纪要要翻译成英文，术语保留原文",
  "source": "ime-log",
  "ts": "2026-06-30T10:15:21Z"
}
```

记录 id 按 3.2.5 规则 = `sha256(canonical_json(raw))[:16]`：r1= `ff21d1c3963d17db`、r2= `350d516a359a6174`、r3= `6cd63faf65b8cfb7`、r4= `08363dbb4cd11f29`。r2 的 raw 与 r1 不同（空白、ts），id 不同——判重依据是 `dedup_text`，与 id 无关。

记录	dedup_text (NFC + 空白折叠 + strip)	sha256(dedup_text) 前 16 hex	命中层	DedupInfo	status
r1	原文已是规范形，无改动 (64 字)	57e3f858bc013a54	无（首见：精确键 + MinHash 签名入索引）	kind="unique" cluster_key="57e3f858bc013a54" kept_id=null	active
r2	去掉行首半角空格与句尾（全角空格）后，与 r1 逐字节相同	57e3f858bc013a54 (与 r1 相同)	① 精确（哈希已在 set 中，无条件判重）	kind="exact" cluster_key="57e3f858bc013a54" kept_id="ff21d1c3963d17db"	dropped_dup
r3	仅句尾多出 " 10:12" (70 字)，空白已是单空格，无改动	07aaef398c499831 ≠ r1, ① 未中	② 近似：LSH 候选 = {r1}，签名估计 Jaccard = 0.91 ≥ 0.85	kind="near_text" cluster_key="57e3f858bc013a54" kept_id="ff21d1c3963d17db"	dropped_dup
r4	原文已是规范形，无改动 (19 字)	e49758a83ce5efec	无（①② 均未中，入索引）	kind="unique" cluster_key="e49758a83ce5efec" kept_id=null	active

r3 的真实字符 5-gram Jaccard 可精确验算：r1 折叠后 64 字 → 60 个 shingle，r3 70 字 → 66 个；交集 60、并集 66， $J = 60/66 \approx 0.909$ 。128-permutation 签名估计值的标准差约 $\sqrt{(0.909 \times 0.091 / 128)} \approx 0.025$ ，本例估计值取 0.91，与阈值关系稳定。约定：`cluster_key` 取簇首（首见保留记录）的精确去重键前 16 hex，unique 记录填自身键。本批计入 `report.dedup`：`{"exact": 1, "near_text": 1, "clusters": 1}`（其余计数为 0）；r2、r3 由 M10 按 `output.rejects` 策略落 rejects 通道，不进主输出。

UI 模态合成判定示例（`dedup.ui_dup_requires = "both"`，默认）

待判记录：`capture/2026-07-01/b/uitree_2.jsonl` + `c/image_2.png`（`pair_index=2`，登录页，即 6.3 主输出示例中 `_meta.id="9f2c31ab52e08d17"` 那条）；索引中已保留同一 App 的登录页 `pair_index=1`（`a/uitree_1.jsonl` + `a/image_1.png`）。

层	计算	结果
① 精确	<code>sha256(UITree.serialize(quantize_px=4)) = 1c7a0d93e2b6f458</code> ，索引中 <code>pair_index=1</code> 的键为 <code>b2e49c05d7a1f36e</code>	不相等，未中

② 树近 似	序列化树的 MinHash 签名估计 Jaccard = 0.95 ≥ 0.85（同一登录模板，控件树几乎一致）	命中
③ 图像	pHash(c/image_2.png) = 5181a9e2bc40fcca，簇首 pHash = c3a5e17098d2b46f，汉明距离 = 21 > 8（输入框已填入手机号、软键盘弹出，画面差异大）	未命中
合成	"both" 要求 ② 且 ③ 同时命中；仅 ② 命中 ⇒ 不判重	DedupInfo(kind="unique", cluster_key="1c7a0d93e2b6f458", kept_id=null), status="active"

设计意图：若配置 `ui_dup_requires="tree"`，本条将因 ② 命中被判 `near_text` 丢弃——同一界面模板承载不同内容（不同账号、不同输入状态）的屏幕会被大量误杀。默认 "both" 即为规避该风险（3.3.3）；反之，树与图同时命中时记 `kind="near_both"`。

3.4 M4 质量打分 quality（QuRating）

3.4.1 职责与边界

做：按 rubric 对批内存活记录打质量分：pairwise 模式执行「k 轮随机配对 → LLM 裁决 → Bradley-Terry 拟合 → 批内百分位归一化」；pointwise 模式执行 0—5 加性打分归一化。计算加权聚合分，按阈值标记 `dropped_lowq`。不做：不定义 rubric 内容；不决定被过滤记录的物理去向；不做标注语义正确性评审（那是 M7）。

3.4.2 输入 / 输出

方向	内容
输入	批内 <code>status="active"</code> 的 <code>PipelineItem</code> ；Rubric；LLM profile。
输出	每条记录 <code>item.scores: dict[str, QualityScore]</code> （每 criterion 一项 + <code>"__aggregate__"</code> ）；低于阈值者 <code>status="dropped_lowq"</code> 。

3.4.3 算法：pairwise + Bradley-Terry（主模式）

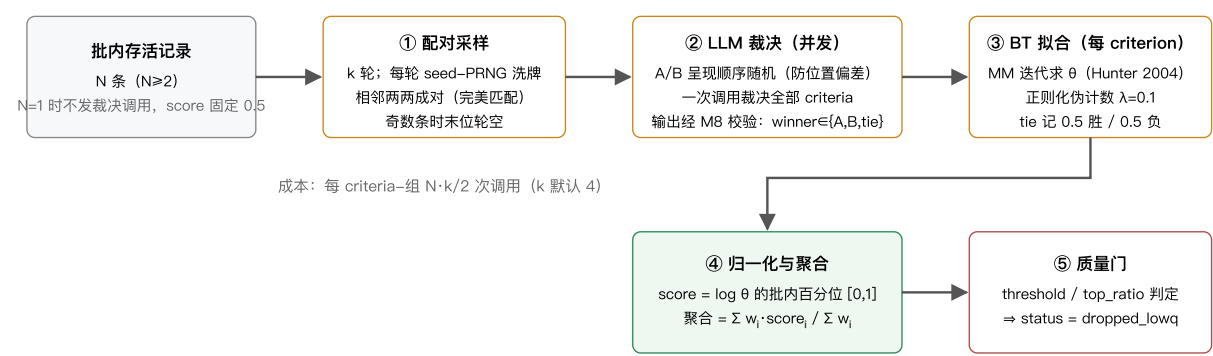


图 3-2 QuRating pairwise 模式流程

关键设计的精确定义：

设计点	定义
比较池	= 当前批（ <code>run.batch_size</code> ）。QuRating 原文在语料分片内采样成对比较 [1]；批即本工具的采样域。批间分数不可直接比较（百分位为批内相对量），报告中按批记录分布。需要跨批可比时使用 pointwise 模式（绝对刻度）。
配对方案	k 轮独立随机完美匹配（洗牌后相邻配对）。每记录恰好参与 k 次比较；k 轮随机匹配的并图为随机 k-正则图，k ≥ 3 即高概率连通，默认 k=4 兼顾成本与 BT 可辨识度；孤立分量由正则化伪计数兜底。
裁决提示词	系统提示 = rubric 全部 criteria 的 pairwise_prompt 拼接；用户消息 = 记录 A、B 内容（UI 模态为两组「截图+序列化树」，需 profile 支持多图）；要求输出 JSON：{"judgments": [{"criterion": key, "winner": "A" "B" "tie", "reason": str}]}（reason 仅当 <code>quality.judgment_reasons</code> 生效时要求——一句话裁决理由，写入 trace 日志供 rubric 优化使用，见 7.5），经 M8 内部 Schema 校验。单次调用裁决全部 criteria（QuRating 为每 criterion 独立询问 [1]；合并询问是成本优化， <code>quality.criteria_per_call = "all"</code> （默认） <code>"single"</code> 可切回原文行为）。

位置偏差	每次比较 A/B 呈现顺序由 PRNG 随机；k 轮聚合平均化残余偏差（Zheng et al. 位置偏差缓解 [20]）。
BT 拟合	每 criterion 独立：极大似然 MM 迭代 $\theta_i \leftarrow W_i / \sum_j n_{ij} / (\theta_i + \theta_j)$ （Hunter 2004 [10]），每轮后归一化 $\Pi \theta = 1$ ；收敛条件 $\max \Delta \log \theta < 1e-6$ 或 200 轮。正则化：每记录附加 $\lambda=0.1$ 次对虚拟对手（ $\theta=1$ ）的半胜半负，保证全胜/全负与孤立分量下 θ 有限且唯一。
归一化与聚合	每 criterion 独立：将批内全部 $\log \theta$ 升序排名（并列取平均秩 rank）， $\text{score} = (\text{rank} - 1) / (N - 1)$ ；N=1 时 score=0.5。得分域 [0,1]，批内最低 0、最高 1。聚合分 <code>__aggregate__</code> = $\sum w_i \cdot \text{score}_i / \sum w_i$ （ w_i 为 rubric 权重；score 为 null 的 criterion 不计入分子分母）。
选择机制	<code>quality.selection = "threshold"</code> （默认，现行为：聚合分 < <code>quality.threshold</code> $\Rightarrow$ <code>status="dropped_lowq"</code> ；threshold 缺省则只打分不筛） <code>"top_ratio"</code> （批内按聚合分降序保留 <code>ceil(top_ratio \times \text{批内存活数})</code> 条，其余 <code>dropped_lowq</code> ； <code>quality.top_ratio</code> $\in (0,1]$ 必填，与 threshold 互斥，M1 校验）。排序与并列规则：按聚合分降序，聚合分相同时按记录 id 字典序升序作确定性平局裁决（同输入同 seed 可复现）；名额基数「批内存活数」= 批内 score 非 null 的存活记录数（score=null 的 <code>on_unscored</code> 保留记录不计入基数、也不占名额）。 <code>top_ratio</code> 在 pairwise 与 pointwise 下均定义良好——两种模式的质量门输入同为批内聚合分排序，是流式场景做定量筛选的推荐姿势；需要「恰好全局 N 条」时需两阶段方案，见 8.3 O6。按 <code>on_unscored="keep"</code> 保留的未打分记录（score=null）不占名额、直接保留。
裁决失败	单次比较经 M8 修复仍非法 $\Rightarrow$ 该比较按 tie 计（对 BT 中性），计入 <code>report.quality.judgment_failures</code> ；某记录全部比较失败 $\Rightarrow$ 该记录 score 置 null 并按 <code>quality.on_unscored = "keep"</code> （默认） <code>"drop"</code> 处理。
多评审团（可选）	<code>quality.judges</code> 配置奇数个 LLM profile（默认 [] = 单评审，用 <code>quality.llm</code> ）。每次比较由各 judge 以同一呈现顺序独立裁决；per-criterion 取多数票——A/B/tie 三类计票，某类得票过半 $\Rightarrow$ 取该类，无类别过半 $\Rightarrow$ tie；BT 拟合取多数结果。trace 中每 judge 各写一条 <code>quality.judgment</code> 事件（payload 增加 <code>judge</code> 字段 = profile 名；7.2 契约只增不改）。成本 = 单评审 $\times$ judges 。背书：PoLL [32]——异构小模型评审团在三种评审设置、六个数据集上优于单一大评审，且显著降低模型内偏差。
双顺序裁决（可选）	<code>quality.both_orders = true</code> （默认 false）时，同一对记录以正反两种呈现顺序各裁决一次（多评审团下每 judge 各判两次）；per-criterion 两次结果一致（换序后仍指向同一记录） $\Rightarrow$ 记该 winner，不一致 $\Rightarrow$ tie。合成次序固定：先 per-judge 做双顺序一致性合成，再跨 judge 取多数票。相对「位置偏差」行的随机化缓解，本机制将位置偏差系统性消除（Zheng et al. 的位置一致性判定 [20]）。trace：正反两序各为一次独立裁决，每 judge 各写两条 <code>quality.judgment</code> 事件（以 <code>order</code> 字段区分两序）。成本 $\times 2$ 。
按类分池（v1.7）	classify 启用时，批内 active 项按 <code>classification.label</code> 分池，池 = 类内存活记录；classify 关闭 $\Rightarrow$ 单一匿名池 = 现行为（零变化回归锚）。 两阶段执行 ：先同步按类名字典序逐池预抽配对计划（消费 <code>ctx.rng</code> ，消费序确定），再跨池合并为一个 gather 派发 LLM 调用（跨池满并发，不损吞吐）。每池取 <code>class_views[label]</code> 的类有效（QualityConfig, Rubric）： <code>mode / rounds / rubric / threshold / selection / top_ratio</code> 池内生效； <code>judges / both_orders / criteria_per_call / llm / on_unscored</code> 恒为全局（5.2 按类覆盖白名单表）。池级 try/except 隔离——某池内部错误不波及其余池（记录级隔离原则的池级推广，1.3）。N=1 池沿用单条规则（不发裁决调用、score 固定 0.5，本表「归一化与聚合」行）； <code>top_ratio</code> 名额基数 = 池内 scored 存活数。pairwise 分数语义相应收窄为「批内类内相对」， <code>__meta.scores</code> 增 <code>pool</code> 字段（= 类名，仅 classify 启用时出现）自述比较池（6.3）。计数器与统计升维：classify 启用时 tie 计数器键为 <code>quality.tie_outcomes.<pool>.<crit></code> （ <code>tie_comparisons</code> 同），report 顶层 <code>quality.mode/rounds</code> 保留（= 全局继承基值）、增 <code>quality.by_class</code> 每池视图（每池携带有效 mode/rounds，6.4）， <code>per_criterion_tie_rate</code> 输出条件改为「存在 pairwise 池」； <code>quality.bt_fit / quality.gate / quality.judgment</code> 事件 payload 增 <code>pool</code> 字段（7.2 只增）；关闭时计数器键式与报表形状不变。
序列打分（v1.8）	stream 模式下序列信封（ <code>record.kind = "sequence"</code> ，3.14）的记录内容改走序列变体，两小节按序（逐字冻结于 CONTRACTS §10.2/§10.3——pairwise 下嵌入【记录 X】内容槽、pointwise 下替换 {record content}，标签不变）：【步骤序列】（ <code>item.transitions</code> 按 3.5.2 步骤行格式逐行文本渲染，transitions 为 None 时整段省略； fallback 步分列 —— <code>Transition.detail.kind == "extraction_invalid"</code> 的兜底步行尾加「（摘取兜底）」后缀，与 LLM 确证的 other 可区分，防兜底噪声污染连贯性锚点，S16； 接缝步分列（v1.9） —— <code>Transition.detail.kind == "thread_seam"</code> 的占位步行尾加专用后缀「（线索接缝：被 {interrupted_by} 打断）」，与 <code>extraction_invalid</code> 后缀并列——防 trajectory rubric 的 <code>noise_residue / coherence</code> 判断把接缝当噪声残留或无法解释的跳变扣分，3.15.4/3.16.4）+ 【成员帧摘要】（逐成员 <code>frame_digest</code> （4.3）按成员序每帧一行，总量有界）。 无图 ——UI 模态亦纯文本打分（vision 逐阶段表的放宽项： <code>quality.llm</code> 不因 stream 要求 supports_vision，S30，3.14/5.2；v1.9 起 <code>stitch.llm</code> 同为纯文本恒不要求，3.16.3）。transitions 与预渲染文本经 <code>_judge_once / _pointwise_once</code> 的新增私有形参下穿（私有签名，非冻结面）；trace excerpt 档对序列的摘录 = 成员摘要渲染的前 200 字符（ <code>_excerpt_payload</code> 序列分支，7.4）。 rubric ：stream 下 <code>quality.rubric</code> 空串解析为 <code>default:trajectory</code> （S29，3.14）——内置轨迹四准则（completion / coherence / purposefulness / noise_residue），全文与背书拆分注记见附录 A.3（completion/coherence 源自 OS-Genesis TRM [41]，1–5 五级改制为 0–5 六级；purposefulness 自 Coherence "toward the goal" 拆分；noise_residue 源自 RPA 日志分割噪声处理 [50]）；rubric 由既有机制消费、零改动。 <code>extract.enabled = false</code> 时步骤段缺席、 退化为帧摘要打分 ——rubric 措辞模态中立、「步骤」读作「帧间变化」（M1 对该组合发 warning 指引，3.14）。 门控 ：stream 下 <code>quality.threshold</code> 缺省 = 只打分不筛（现语义，对 stream 尤其合理——TRM 消融与 E2E 台账 #6 佐证，1.6）。 信度注记 ：长 episode（> 20 步）下整体式 LLM 判信度随长度衰减（GUIDE 长度退化数据 [57]）——建议 pairwise（批内相对比较）或对绝对分降信任、按 episode 长度分层审计。 打分单元（v1.9） ：stitch 启用时打分单元升维 episode $\rightarrow$ 线索 （thread，缝合后的幸存序列信封——被并壳被 active 过滤天然排除，3.16）；序列变体机制原样，仅步骤序列与成员摘要作用于重绑后的成员集，缝合并入使打分调用数随行数下降（3.16.4 调用与校验行）。
上下文预算装填（v1.11）	裁决 profile 声明 <code>context_window</code> 时按上下文预算装填单次裁决调用（未声明 = 预算关闭，行为与 v1.10 一致；预算/估算/校准机制见 3.9）。 pairwise ：记录侧预算 = <code>input_budget - est</code> （系统提示 + 准则文本），按 每评审各自构建 （逐对，评审）装填、取本评审 profile 的预算——与 verify 的「评审团最小预算」形态不同，V25②），两记录各半；每侧 UI 树渲染动态封顶 <code>min(input.ui_tree_max_chars, 预算折算字符)</code> （渲染后按 <code>est</code> 复核，超则按行丢尾、保留既有 <code>...(truncated N nodes)</code> 标记； <code>ui_tree_max_chars</code> 保留为绝对上限）；附图（UI 对 2 张）按校准单价计 <code>est \times 2</code> 后再分。 序列变体 ：【步骤序列】步骤行块获得预算份额，超出按「首末步恒保留、丢中段整行 + 原位标记」裁剪（与成员摘要块既有截断语义同族，V9）。 pointwise 单记录同族（树渲染动态封顶 + 步骤行同款裁剪）。 溢出反应（V20） ：识别到 provider 上下文溢出 $\rightarrow$ 收紧文本份额 重试一次 （有界降级）。 最小单元终局分粒度 （V10 的 M4 适配）：pointwise 单记录装不下 $\rightarrow$ 该记录记 <code>StageError(kind="context_overflow")</code> 、 <code>status="failed"</code> 入 <code>rejects</code> （7.6）； pairwise 2 记录装不下 $\rightarrow$ 按本表「裁决失败」行的裁决级粒度折算 tie （对局记 <code>context_overflow</code> 错误与事件、双方记录保持 active 可入其他对局）——记录级隔离（2.6）要求超大一侧不得殃及配对另一侧；全部对局皆溢出的记录落入既有 <code>on_unscored</code> 处置族。逐裁剪点计入 <code>report.budget.truncations</code> （6.4）。

3.4.4 算法：pointwise 加性打分（低成本模式）

每记录 1 次调用：提示词呈现该 criterion 的 6 级加性量表（0—5，逐级累加式描述，附录 A 给出全文），要求模型先给两句理由再给整数分（判分与理由分离缓解冗长偏差 [20]；加性量表为 FineWeb-Edu 验证的最优形式 [11]）。输出 `{"scores": [{"criterion": key, "reason": str, "score": 0..5}]}` 经 M8 校验；score 归一化为 /5。聚合与质量门同 pairwise。两种模式共用同一 rubric 的不同字段（pairwise_prompt / pointwise_levels），切换零迁移成本。

3.4.5 API 与配置

```
class QualityStage(Stage):
    name = "quality"
    async def run(self, batch, ctx) -> list[PipelineItem]: ...

def fit_bradley_terry(n_items: int, comparisons: list[tuple[int, int, float]],
                     l2_pseudo: float = 0.1, tol: float = 1e-6, max_iter: int = 200) -> np.ndarray:
    """comparisons: (winner_idx, loser_idx, weight); tie 拆为两条 weight=0.5。返回 log-theta 数组。"""
```

配置见 5.2 [quality]。

背书：算法主体逐点对应 QuRating (ICML 2024) [1]：成对判断优于绝对打分的结论、BT 标量化、rubric 四维度均出自该文；本工具以「运行时 API 裁决」替代其「训练 QuRater 分类器离线打分」（无状态约束所致，1.6 节已对齐）。pointwise 模式为 FineWeb-Edu (NeurIPS 2024 D&B) 验证的加性量表方案 [11]。BT 拟合采用 Hunter 的 MM 算法 [10]，为该模型的标准数值方法。多评审团为 PoLL [32] 的评审团方案；双顺序一致性判定为 [20] 的位置偏差消除做法。

3.4.6 输入 / 输出示例

以下用一个可手工核对的最小批完整走查 pairwise 主模式。设定：`run.modality = "text"`、`input.text_field = "instruction"`、`run.seed = 42`；`quality.mode = "pairwise"`、`quality.rounds = 2`（`k=2`，演示用，默认为 4）、`quality.threshold = 0.3`、`quality.rubric = "inline"`——rubric 仅含一条 criterion：`educational_value`（`weight=1.0`，`pairwise_prompt` 与 `description` 取自附录 A.1）。批内存活记录 `N=4`，调用成本 = `N·k/2 = 4` 次（单 criterion 下 `criteria_per_call` 取值不影响次数）。本例另设 `trace.enabled = true`（channels 默认含 quality），故 `quality.judgment_reasons = "auto"` 档生效，裁决输出携带 reason（5.2、7.5）。

① 输入记录（输入法采集的中文指令 JSONL，四行，M2 摄取后按 3.2.5 规则生成 id）：

```
{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}
{"instruction": "哈哈哈哈哈", "source": "ime-log", "ts": "2026-06-30T10:14:05Z"}
{"instruction": "解释一下二分查找为什么是 O(log n)，能不能举个在通讯录里找人的例子", "source": "ime-log", "ts": "2026-06-30T10:15:47Z"}
{"instruction": "把“会议改到周五下午三点”翻译成英文", "source": "ime-log", "ts": "2026-06-30T10:18:22Z"}
```

依次记为 r1—r4，id (sha256 前 16 hex)：r1= 1cda030abc565f17，r2= 91eaaa968c26ab62，r3= fd97f67330e81315，r4= 03dddf294e481c8。

② 配对与裁决结果（seed=42 的 PRNG 每轮洗牌后相邻配对；A/B 呈现顺序同样由 PRNG 随机）：

比较	轮	呈现 A	呈现 B	winner	BT 折算
c1	1	r3	r1	"A"	r3 胜 r1（权重 1.0）
c2	1	r2	r4	"B"	r4 胜 r2（权重 1.0）
c3	2	r3	r2	"A"	r3 胜 r2（权重 1.0）
c4	2	r1	r4	"tie"	双向各记 0.5 胜（3.4.3）

③ 单次比较的报文（以 c1 为例；提示词按 3.4.3 组装，输出经 M8 内部 Schema 校验）：

```
system:
  你将对两条记录进行成对质量比较。准则如下：
  - educational_value: 教育/训练价值：作为模型训练数据能带来多少可学习的能力。
    比较两段文本，哪一段更有教育价值、更值得用于训练语言模型？
  对每条准则给出裁决。输出必须是符合以下结构的单个 JSON 对象，不输出任何其他内容：
  {"judgments": [{"criterion": <准则 key>, "winner": "A"|"B"|"tie", "reason": <一句话理由>}]}
```

```
user:
  [记录 A] 解释一下二分查找为什么是 O(log n)，能不能举个在通讯录里找人的例子
  [记录 B] 帮我写一条请假条，明天上午要去医院
```

```
← 响应:
{"judgments": [{"criterion": "educational_value", "winner": "A",
  "reason": "A 要求讲解算法复杂度并配实例，覆盖推理与解释能力，可学习内容明显多于 B 的日常写作请求。"}]}
```

④ **BT 拟合 (MM 算法, 3.4.3)**: 胜场计入 W 后, 每记录再附加 $\lambda=0.1$ 次对虚拟对手 ($\theta=1$) 的半胜半负 (W 各 +0.05, 分母各加 $0.1/(\theta_i+1)$); 迭代 $\theta_i \leftarrow W_i / \sum_j n_{ij} / (\theta_i + \theta_j)$, 每轮归一化 $\Pi \theta = 1$ 。本例在 200 轮上限处终止 (此时 $\max |\Delta \log \theta|$ 约 4×10^{-6} , 尚未达 1×10^{-6} 收敛线, $\log \theta$ 前 3 位小数早已稳定)。

⑤ **归一化与 ⑥ 质量门**: $\text{score} = (\text{rank}-1)/(N-1)$, rank 为 $\log \theta$ 升序排名 (并列取平均秩); 单准则 $\text{weight}=1.0$ 时聚合分 = 该准则分:

记录	comparisons	wins / ties	log θ	升序 rank	score	__aggregate__	质量门 (threshold=0.3)
r2	2	0 / 0	-3.021	1	0.000	0.000	< 0.3 $\Rightarrow$ status="dropped_lowq"
r1	2	0 / 1	-0.082	2	0.333	0.333	active
r4	2	1 / 1	0.082	3	0.667	0.667	active
r3	2	2 / 0	3.021	4	1.000	1.000	active

r1 写入 item.scores 的内容 (4.2 QualityScore 的 JSON 形态):

```
"scores": {
  "educational_value": {"criterion": "educational_value", "score": 0.333, "mode": "pairwise_bt",
    "detail": {"comparisons": 2, "wins": 0, "ties": 1, "log_theta": -0.082}},
  "__aggregate__": {"criterion": "__aggregate__", "score": 0.333, "mode": "pairwise_bt",
    "detail": {}}
```

r2 被标记 dropped_lowq, 按 output.rejects 策略进入 rejects 通道并计入 report.counts.dropped_lowq; r1、r3、r4 保持 active 进入 M5。

⑦ **pointwise 低成本模式对照** (quality.mode = "pointwise", 对 r1 仅 1 次调用; 量表取附录 A.1 educational_value 的 pointwise_levels):

```
system:
按以下 0-5 加性量表为记录的 educational_value (教育/训练价值) 打分, 先给两句理由再给整数分:
0: 无学习价值 (噪声、纯广告、无意义重复)。
1: 学习价值极低, 内容浅表。
2: 在 1 的基础上, 有一定可学习内容但组织松散。
3: 在 2 的基础上, 内容系统、有清晰的知识或任务示范价值。
4: 在 3 的基础上, 示范性强, 覆盖推理/解释/结构化表达等能力。
5: 在 4 的基础上, 训练价值突出, 属稀缺的高质量样本。
输出 JSON: {"scores": [{"criterion": <准则 key>, "reason": <两句理由>, "score": 0..5]}

user:
[记录内容] 帮我写一条请假条, 明天上午要去医院

← 响应:
{"scores": [{"criterion": "educational_value",
  "reason": "该指令是意图明确的写作示范任务, 包含时间与事由等具体要素。但任务简单, 不涉及推理或专业知识, 可学习内容有限。",
  "score": 3}]}
```

```
→ 归一化 3/5 = 0.6: {"criterion": "educational_value", "score": 0.6, "mode": "pointwise",
  "detail": {"raw_score": 3, "reason": "该指令是意图明确的写作示范任务....."}}
```

提示: r1 在 pairwise 下得 0.333、在 pointwise 下得 0.6, 二者不矛盾——前者是批内相对百分位 (本批恰有 r3 这类强样本压秩), 后者是绝对刻度 (3.4.3 「比较池」行)。需要跨批可比时选 pointwise。

⑧ **top_ratio 选择示范**

沿用本例 4 条记录的最终得分 (r2=0.000、r1=0.333、r4=0.667、r3=1.000), 改设 quality.selection = "top_ratio"、quality.top_ratio = 0.5 (此时不设 quality.threshold, 二者互斥, M1 校验): 批内按聚合分降序保留 $\text{ceil}(0.5 \times 4) = 2$ 条 $\Rightarrow$ r3、r4 保持 active, r1、r2 置 status="dropped_lowq"。对比 ⑥ 中 threshold = 0.3 只丢 r2 一条: threshold 按绝对分数线筛、每批淘汰数随分布浮动, top_ratio 按批内名次筛、每批保留比例恒定。

3.5 M5 标注 annotate

3.5.1 职责与边界

做: 为每条存活记录组装标注提示词 (任务指令 + few-shot + 记录内容, UI 模态含截图与序列化树), 经 M9 调用 LLM、经 M8 获得符合用户 Schema 的标注对象; 可选 self-consistency 多次采样字段级投票 (3.5.2)。**不做:** 不校验结构 (全部委托 M8); 不评审标注质量 (M4/M7 职责); 不产出新记录 (生成属 M6); 不做提示词内容的「智能改写」——提示词组装是确定性模板拼接。

3.5.2 标注提示词组装（确定性模板）

```
system:
{annotate.instruction}                # project.toml, 必填
输出必须是符合以下 JSON Schema 的单个 JSON 对象，不输出任何其他内容：
{user_schema_json}                    # M8 提供的规范化 Schema 文本
user（对每条 few-shot 示例，依次）：
[示例输入] {example.input}
[示例输出] {example.output}          # M1 启动时已校验示例输出符合用户 Schema
user（当前记录）：
文本模态：[待标注数据] {record.text}
UI 模态：[屏幕截图] <image: base64>
[UI 控件树] {record.ui_tree.serialize(max_chars=input.ui_tree_max_chars)}
```

UI 树序列化格式（UITree.serialize()，4.3 节）：深度缩进的每节点一行 <role> "text" [l,t,r,b] {关键属性}，只保留可见节点与非空属性，超出 ui_tree_max_chars 时按深度优先截断并追加 ... (truncated N nodes) 标记。此「图 + 线性化结构文本」双通道输入是 ScreenAI 的 screen-schema 表示 [13] 与 GUI 智能体输入惯例 [16][17]。

装配变体参数对象（2026-08-14 代码规则整改）。build_annotate_prompt(record, cfg, schema_text, opts) 与 annotate_record(record, ctx, opts) 的全部装配变体取值集中在一个冻结 dataclass AnnotatePromptOptions 上：

字段	语义	引入
repair	修复上下文（RepairContext）；None = 首次标注	v1.1
temperature	采样温度；None = profile 默认（M5 内部设定，调用方置值忽略）	v1.1
label	分类标签；v1.13 起同时选定按类标注 Schema	v1.7
transitions	[动作序列] 步骤源；None = 整段省略	v1.8
fragment_lens	逐碎片成员数（按碎片配额降采样）；None = 全局均匀降采样	v1.9
k_eff	生效关键帧上限（V20 折半 / V21 修复梯）	v1.11
image_px	升档后的图像采样边长	v1.11

缺省实例即引入这些取值之前的全局无变体装配（逐字节等价）；换档一律以 dataclasses.replace(opts, ...) 产出新对象。字段名与语义与逐次引入时完全一致——变的只是承载形式，v1.7–v1.11 的「追加式末位 kwarg」叙事就此退场。

按类取值（v1.7）。classify 启用且记录带类标签时，本节模板的 {annotate.instruction} 与 few-shot examples 取该类有效配置（class_views[label].annotate，3.1.4 按类覆盖合并行）——模板结构不变，仅取值来源变化。取值载体为 opts.label（None = 全局配置）；stage 层传 item.classification.label if item.classification else None。trace annotate.done 事件 payload 增 label 字段（仅 classify 启用时携带，7.2 只增不改）。

按类标注 Schema（v1.13，裁决·按类标注 Schema）。label 自 v1.13 起同时选定标注 Schema：类有效 Schema = class_views[label].schema ?? cfg.user_schema（[class.<name>.annotate].schema_path / schema_inline 的解析产物；类未声明覆盖、label 缺失或类表外的未知类一律回落全局，3.1.4 按类覆盖合并行 ⑤）。取值经单点取值函数实现，本模块的每个 Schema 消费点都读它——保证「计价的 Schema 恒等于调用的 Schema」，六处：

消费点	v1.13 取值
本节模板的 {user_schema_json}	类有效 Schema 的 canonical 单行 dump（无覆盖时直取 M8 的 user_schema_text 属性，字节等价）
标注调用（首次 / 修复重标注两处）	有覆盖 ⇒ 显式 schema=类有效Schema 且 scope.user_treatment=True（3.8.2 待遇参数：L2.5 与 resolved_at 记账双保留）；无覆盖 ⇒ schema=None 的既有推断路径
self-consistency 字段级投票	可投票字段取自类有效 Schema（按类 Schema 的字段集可与全局不同）
v1.11 预算装填的 schema 计价项	按类有效 Schema 计价
M7 修复路径的重标注与 V21 试装	传同一 label 自然穿透（无修复侧改动，3.7.3）
M11 写前终检	按该行类标签取有效 Schema（3.11.2；multi 扇出的兄弟信封各带自己的标签，按行天然对齐）

未配置任何按类 Schema 时全部调用形与 v1.12 逐字节一致。

标注鲁棒性：self-consistency（可选，v1.2）。annotate.self_consistency = n（默认 0 = 关；启用须 n ≥ 3 且为奇数，5.2）时，M5 对每条记录按本节模板独立采样 n 次（temperature 统一取 annotate.sc_temperature，默认 0.7——采样多样性的来源），每次输出都各自经 M8 走完完整结构保证后才参与投票。字段级投票：enum / boolean / integer 字段逐字段取 n 个样本中的众数；自由文本 / 数组字段不逐字投票，取「与

众数字段组合一致的样本」中第一个的对应字段值。其余类型字段（number、嵌套 object 等）与自由文本/数组同法处理（不逐字段投票，随众数字段组合整体取值）。全体分歧（众数组合不存在或无样本与其完全一致）时整体采用第一个样本，并计入 `report.annotate.sc_disagreements`。某次采样经 M8 修复仍失败（SchemaViolation）⇒ 该样本弃权、由其余合法样本投票（`agreement_ratio` 分母仍为 `n`）；`n` 次全部失败才置 `status="failed"`。`_meta.annotation.attempts` 记 `n` 次采样 `attempts` 之和。`_meta.annotation` 增 `sc = {n, agreement_ratio}`（`agreement_ratio` 与最终众数字段组合完全一致的样本数 / `n`；6.3 只增字段）；`trace annotate.done` 事件 payload 增同构 `sc` 字段（7.2「只增不改」契约内扩展）。该机制对分类型 Schema 收益最大——如统一示例的 `intent` / `difficulty` 枚举字段：多路径采样 + 多数投票显著优于单次贪心解码（Self-Consistency, Wang et al., ICLR 2023 [33], GSM9K +17.9%）。成本：标注调用与 token $\times n$ 。

序列标注模板 (v1.8, S5/S6/S28)。stream 模式下序列信封（`record.kind = "sequence"`，3.14）的「当前记录」user 消息改走序列变体——system 与 few-shot 消息不变，**段序与步骤行格式逐字冻结**（CONTRACTS §10.1 序列变体），单条 user 消息内 Part 恰按此序：

- ① text part: [动作序列] ← `item.transitions` 为 None 时整段省略
 {index}. {action_type} (对象: {target|-}; 值: {value|-}) {description}
 ← 每 Transition 一行, index 升序;
 target/value 为 null 时渲染为字符「-」
- ② 每保留关键帧（关键帧序数 `i/k`, 成员序数 `m`——标签显式携带成员序数）：
 text part: [关键帧 {i}/{k}·成员 {m}]
 image part: `member.image` (M9 调用时编码, 3.9.2)
- ③ text part: [成员帧摘要] ← 恒在收尾段
 {全体成员逐帧 `frame_digest` (4.3), 每成员一行、按成员序, 总量有界}

模板不变量 (S6)：user 消息末 Part 恒为 ③ 恒在的 text 段——M7 修复后缀（3.7.3）拼接在末 Part 的 text 之上，若消息以图收尾页会静默产出 "None\n..." 并丢失末帧图；恒在收尾摘要段以零修复代码改动保证该不变量（repair 拼接路径不动）。

关键帧降采样 (S28)：成员数 $n > \text{annotate.sequence_frames} = k$ 时确定性均匀降采样 $\text{idx}_i = \lfloor i \cdot (n-1) / (k-1) \rfloor$, $i = 0..k-1$ ——首末帧恒含、严格递增无重复、纯整数零 rng（种子豁免面不变, 2.6）； $n \leq k$ 取全量。 $k \in [2, 100]$ 与 $> 20 \wedge \text{max_image_px} > 2000$ 联动警告由 M1 校验（3.1.4、5.2）。

按碎片配额降采样 (v1.9)：stitch 启用且信封为多碎片线索（ $F \geq 2$ 个碎片，3.16）时，上式升级为**按碎片配额**——全局均匀采样会把小碎片整段抽空（恢复段可能仅 2—3 帧，恰是缝合语义的关键证据），每碎片须至少保底 1 帧（T14）。确定性公式（纯整数零 rng； n_f = 碎片 `f` 的成员数， $\Sigma n_f = n > k \geq F$ ）：

```
配额:   q_f = 1 + base_f + tip_f           # 每碎片保底 1 帧,  $\Sigma q_f = k$ 
        base_f = \lfloor (n_f - 1) \cdot (k - F) / (n - F) \rfloor   # 剩余  $k - F$  个名额按  $(n_f - 1)$  加权
                                                # （保底帧已计入, 按剩余成员数分摊）
        tip_f ∈ {0, 1}: 余名额  $(k-F) - \Sigma \text{base\_f}$  个, 按余数  $(n_f-1) \cdot (k-F) \bmod (n-F)$ 
            降序逐碎片 +1（平局取碎片序小者）——最大余数法
碎片内: 以  $q_f$  对碎片成员局部套均匀公式 ( $q_f \geq 2$  时碎片首末帧恒含;  $q_f = 1$  取碎片首帧,
        唯一例外: 未碎片  $q_F = 1$  时取其末帧), 选中下标映射回线索成员元组
不变量: 全局首帧（经首碎片）与末帧（经末碎片）恒含; 跨碎片严格递增无重复
退化:   碎片划分缺席 / 单碎片 / 与成员数不一致、或  $k < F$ （保底不可行）⇒ 静默回退
        全局均匀公式（单碎片线索由此逐字节退化为 v1.8 行为——零变化回归锚）
```

穿参义务 (v1.9)：碎片划分在信封 duck 标上而 `build_annotate_prompt` / `annotate_record` 只收 Record——载体是 `opts.fragment_lens`（= 线索各碎片的成员数，按碎片序；None = 全局均匀降采样）。stage 层自信封碎片跨度表取各碎片 `member_count` 传入；M7 修复重标注调用同步穿参（3.7.3——两处调用点一并穿参，否则修复重标丢配额、退回全局均匀采样）。

transitions 取值 (S5)：载体是 `opts.transitions`（CONTRACTS §7.4）。stage 层传 `item.transitions`；M7 修复路径在成员手术后传**重建值**（3.7.3）。self-consistency 与 L2.5 路径不动——序列记录的 L2.5 回调收到 `record = None`（Record.raw 对序列恒 None，4.1；文档声明的既有局限，富载荷形参列演进候选）。

上下文预算装填与修复升级换挡 (v1.11)。标注 profile 声明 `context_window` 时按上下文预算装填单次标注调用（未声明 = 预算关闭，行为与 v1.10 一致；预算/估算/校准机制见 3.9）。**份额定序**（确定性, V9）：① 系统侧静态部件（instruction / 用户 Schema / few-shot）**不裁剪**——用户语义资产，由 M1 静态预检把关（3.1.4, V13③）；② 文本块（步骤行 + 成员摘要块；单记录 UI 树渲染同 3.13.4 的动态封顶语义）按其既有绝对上限渲染并计 est；③ 图片吃剩余： $k_{\text{eff}} = \min(\text{sequence_frames}, \max(2, \lfloor \text{剩余} / \text{est_image} \rfloor))$ ——**首末帧恒保留**、中间均匀下采样（本节降采样公式与按碎片配额语义不变，仅 k 收缩；图片单价读校准器, 3.9）；④ $k = 2$ 仍超预算 → 回头按「首末恒保留、丢中段」裁文本块直至装下；⑤ 仍不下 → 该记录记 `context_overflow` 入 `rejects`（V10, 7.6）。**图片先于文本让步**：文本摘要是裁决兜底证据、token 效率高于像素。溢出反应（V20）：识别到 provider 上下文溢出 → 关键帧减半重试（ k 减半， $\min 2$ ；有界 ≤ 2 次降级），耗尽仍溢出按 V10 处置。**修复轮升级换挡 (V21)**：`verify.policy = "repair"` 的修复重标注按质量阶梯换挡——关键帧数减半（ $k \rightarrow \max(2, \lfloor k/2 \rfloor$ ），首末恒保）+ 分辨率上探一档（`default_image_px × 1.5n/维`, $\leq \text{max_image_px}$ ），预算约束经校准估算复核后不变；阶梯参数经 `opts.k_eff` / `opts.image_px` 两字段传入（M7 以 `dataclasses.replace` 换挡, F3）；单向有界——升级仅发生于修复路径、每记录 $\leq \text{verify.max_repair_rounds}$ 次（触发条件见 3.7.3）。**不可裁剪动态块 (V25③)**：修复轮追加的【上一版标注】/【审核意见】尾注为语义资产——计入 est、永不裁剪；全部可裁份额耗尽仍超 → V10。逐裁剪点计入 `report.budget.truncations`（6.4）。

3.5.3 API 与错误处理

```
class AnnotateStage(Stage):
    name = "annotate"
    async def run(self, batch, ctx) -> list[PipelineItem]:
        """对每条 active 记录: prompt = build_prompt(rec); item.annotation = await ctx.schema_engine
        .complete_validated(profile, prompt, user_schema) # M8 全责保证结构
        SchemaViolation(不可修复) => item.status='failed', 错误入 item.errors."""
```

背书:「指令 + few-shot + 结构化输出」的 LLM 标注器是 distilabel (Argilla) [5] 与 Autolabel (Refuel) [12] 两个工业框架的核心抽象; UI 模态输入表示见 3.5.2 背书 [13][16][17]; self-consistency 字段级投票为 Wang et al. (ICLR 2023) 的多路径采样多数决 [33]。

3.5.4 输入 / 输出示例

① 文本模态标注 (输入法中文指令 → 意图标注)

工程配置: `input.text_field = "instruction"`, `annotate.llm = "default"`, `annotate.examples` 含 1 条 few-shot 示例, 用户 Schema 经 `output.schema_inline` 内嵌 (M1 启动时已校验示例输出符合该 Schema)。输入记录 (id 规则见 3.2.5):

```
{"instruction": "帮我写一条请假条, 明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}
=> Record(id="1cda030abc565f17", modality="text", text="帮我写一条请假条, 明天上午要去医院", ...)
```

按 3.5.2 模板逐字组装的完整提示词 (`{user_schema_json}` 为 M8 提供的规范化 Schema 文本):

```
system:
  你是输入法中文用户指令的意图标注员。判断给定指令属于哪类意图、其主题是什么、
  以及完成该指令对语言模型的难度。
  输出必须是符合以下 JSON Schema 的单个 JSON 对象, 不输出任何其他内容:
  {"type": "object",
   "properties": {
     "intent": {"type": "string", "enum": ["writing_assist", "qa", "translation", "chitchat", "other"]},
     "topic": {"type": "string"},
     "difficulty": {"type": "string", "enum": ["easy", "medium", "hard"]}},
   "required": ["intent", "topic", "difficulty"], "additionalProperties": false}
user (few-shot 示例 1):
  【示例输入】NBA 总决赛什么时候开始
  【示例输出】{"intent": "qa", "topic": "体育赛事时间查询", "difficulty": "easy"}
user (当前记录):
  【待标注数据】帮我写一条请假条, 明天上午要去医院
```

LLM 响应文本经 M8 (L1 直得平衡花括号子串, L2 一次通过, 无 L3 修复) 返回合法对象, M5 构造 `Annotation` (4.2 节):

```
响应: {"intent": "writing_assist", "topic": "请假条代写", "difficulty": "easy"}

item.annotation = Annotation(
  output = {"intent": "writing_assist", "topic": "请假条代写", "difficulty": "easy"},
  model = "qwen2.5-vl-72b-instruct",          # llm.default 的 model
  attempts = 1,                               # 1 + 0 次 L3 修复
  usage = Usage(prompt_tokens=312, completion_tokens=31))
```

② UI 模态标注 (§5.2 登录页工程)

提示词骨架与 ① 完全相同 (system = §5.2 的 `annotate.instruction` + 规范化用户 Schema), 差别仅在「当前记录」user 消息由两个 Part 组成 (3.9.2): `【屏幕截图】` 为 `kind="image"` 的 Part (`capture/2026-07-01/c/image_2.png`, M9 调用时缩放并 base64 编码); `【UI 控件树】` 为 `kind="text"` 的 Part, 内容即 `record.ui_tree.serialize(max_chars=30000)` 的输出——实例见 3.2.7, 此处不重复。对 `capture/2026-07-01/b/uitree_2.jsonl` 该记录 (id `9f2c31ab52e08d17`) 的响应 JSON (即 §6.3 主输出行剥除 `_meta` 后的用户结构, `annotation.model / attempts` 与该行 `_meta.annotation` 一致):


```
{
  "screen_category": "login",
  "page_title": "登录",
  "interactive_elements": [
    {
      "role": "EditText",
      "label": "请输入手机号",
      "bounds": [72, 520, 1008, 664]},
    {
      "role": "EditText",
      "label": "请输入验证码",
      "bounds": [72, 712, 672, 856]},
    {
      "role": "Button",
      "label": "获取验证码",
      "bounds": [704, 712, 1008, 856]},
    {
      "role": "Button",
      "label": "登录",
      "bounds": [72, 952, 1008, 1096]}],
  "description": "手机号+验证码登录页"
}
```

3.5.5 帧级逐帧标注 (v1.12)

流模式帧粒度的标注面（`frame.annotate.enabled`，默认关，5.2）：对序列信封的成员帧逐帧产出符合**帧级 Schema**（`cfg.frame_schema`，3.1.4 帧粒度配置行）的标注对象，产物写 `item.member_annotations`（键 = 成员 `record.id`，4.1），随序列行落盘 `_meta.stream.members[]`（3.11.2）。链位 = 序列级标注成功之后（同一 `_annotate_item` 内追加帧 `pass`）——质量门之后、被淘汰记录与序列级标注失败的信封永不付帧标注费（裁决·链位与成本）。序列级 `annotate_record` / `build_annotate_prompt` 冻结签名零改动。

公开面（修复面族新成员，签名冻结）：

```
async def annotate_member(member: Record, ctx: RunContext,
                          label: str | None = None) -> Annotation | None:
    """对单个成员 Record 做一次帧级标注。M7 verify 回收成员补跑经懒加载直调本面，
    与 annotate_record / segment.judge_window / extract.extract_transition 同列
    （算子间导入白名单第四向，3.7.3）。label 非 None => 指令/few-shot 取
    cfg.frame_class_views[label]（帧类覆盖视图）；None => 全局 [frame.annotate]
    (frame.classify 关闭时的全员形态)。类视图 enabled=false 的跳过判定归调用方
    (M5 帧 pass / M7 回收)，本面不重复判定。失败返回 None，永不抛记录级异常
    (CircuitBreakerTripped / KeyboardInterrupt / CancelledError 运行级控制流除外)。"""

def build_frame_annotate_prompt(member: Record, cfg: ResolvedConfig, schema_text: str,
                                label: str | None = None) -> PromptBundle:
    """帧级标注提示词的确定性装配 (verbatim 捕于 CONTRACTS §10.13)。"""
```

要点（规格与理由）：

- **执行门**：`active` $\wedge$ `record.kind == "sequence"` $\wedge$ 首标签信封（`classification.label == labels[0]`，无 `classification` 视为首标签——非首标签克隆不执行，裁决·扇出共享与首标签执行） $\wedge$ 非降格（`segment_degraded` `duck` 标在场即跳，裁决·降格会话跳过） $\wedge$ 序列级标注成功后。逐成员并发（gather），成员级隔离由 `annotate_member` 不抛保证。
- **模板段序（冻结）**：`system` = [任务] + 生效指令（类覆盖或全局，含 `few-shot` 各一条 `user` 消息、配置序）+ Schema 约束句 + 帧 Schema 文本嵌入（`cfg.frame_schema` 的 canonical 单行 dump，镜像序列级 `build_annotate_prompt` 的 `schema_text` 嵌入手法）→ 成员内容 `user` 消息：`text` 模态 = [成员帧] {行文本}；`ui` 模态 = [屏幕截图] + `image part` + [UI 控件树] + 树摘要三段形（本模块单记录 `ui` 标注同款；树渲染绝对上限 `input.ui_tree_max_chars`，预算声明时动态帽收缩——树是唯一可裁块，生效指令 / `few-shot` / 帧 Schema 文本是静态语义资产只计不裁，3.1.4 V13③ 预检领地）。
- **Schema 显式路由**（裁决·帧 Schema 显式路由）：`complete_validated(frame.annotate.llm, prompt, schema=cfg.frame_schema, ...)`——内部 Schema 待遇：L0–L3 四层全在、无 L2.5、不计 `resolved_at`（6.4 恒等式「`resolved_at` 加总 = 进入 M5 的记录数」不被帧调用污染，3.8.2）。
- **幂等只补缺位**：帧 `pass` 一旦运行即把 `member_annotations` 初始化为 {}（区别于「未运行」的 `None`——emitter 在场规则的单一真相，4.1；multi 扇出场景下该容器由 M13 在扇出前预先钉住共享，裁决·扇出共享时序补丁，§1.6）；已有 dict 只补缺位、从不换对象（扇出克隆按引用共享同一 dict 的前提）；仅当成员 `id` 不在 dict 时才调用（M7 回收补跑同款）；同 `id` 成员只调用一次（first-wins，裁决·同 `id` 成员 first-wins——防同键并发双付费与计数虚高，3.13.7 同口径）。
- **跳过与失败语义**：帧类视图 `enabled=false` ⇒ 该成员 `skipped`、不占键、计 `frame_annotate.skipped`；修复穷尽/不可恢复 ⇒ 该成员占键 `None`（failed）、计 `frame_annotate.failed` + WARN 运行日志（只含成员 `id` / 错误 `kind` / 异常类型，绝不含数据内容或提示词），`item.errors` 不写（成员失败非信封失败——写入会污染 `rejects` 归因，v1.7 fallback 留痕同理）；episode 照常发射。成功 ⇒ 占键 `Annotation`、计 `frame_annotate.annotated`。单一真相 = dict 形态本身（占键 `None` = failed / 缺键 = `skipped` / dict 为 `None` = `pass` 未运行），emitter 三值判定见 3.11.2。
- **无降级梯（最小单元）**：帧 `prompt` 本身就是最小单元——单成员、至多单图，无窗可分、无关键帧可减；预算装填裁树后仍超限 ⇒ `ContextOverflowError(phase="precheck")`，按成员失败处置（注定失败的请求永不发出）、永不喂熔断；反应式终端镜像本模块 A7 纪律（仅 `http_400` 反应式恰一次喂）。图像成本恒取 `profile` 工作点（`default_image_px`——校准器按 `profile` 聚合的前提，帧调用不设独立尺寸）。
- **事件载荷纪律**：`annotate.frame` 每成员一发（`ids=(episode_id,)`，3.12.4）；payload 仅 `member_id` / `status` / `attempts`——标注内容只经既有 `excerpt` 键按档位截断（`excerpt/full` 档 200 字，7.4），不新增任何承载数据内容的 `payload` 键（none 档预脱敏载荷直通 console 面板的红线）。

3.6 M6 生成 generate

3.6.1 职责与边界

做：组装生成提示词——种子按运行模式取自：process 模式 = 当前批过质量门的记录；generate_only 模式（v1.4）= 配置种子池 generate.seed_examples，或无种子（仅 instruction × style 条件化）。按 generate.llms ×（可选）[[generate.styles]] 组合产出新样本文本，构造为携带 generated_from 与 generator 溯源的新 Record，组成生成子批交 M10 回流调度。提示词组装同为确定性模板拼接（含「风格要求」追加）。**不做：**仅文本模态（无法生成截图，2.3.1 约束③）；单轮回流、不递归；不去重 / 不打分 / 不标注 / 不校验（回流后由 M3 / M4 / M5 / M7 完成）；生成调用失败仅损失该调用的样本，不产生 failed 记录（种子记录状态不变，记录级隔离，1.3）。

3.6.2 生成模式

步骤	定义
种子选取	当前批内 status="active" 且聚合分 ≥ generate.seed_min_score（默认取 quality.threshold，未设阈值时取批内中位数）的记录（process 模式）。generate_only 模式（v1.4）：种子 = generate.seed_examples 字符串数组——Self-Instruct 的人工种子池形态（原文以 175 条人工种子自举 [18]）；数组缺省则为无种子条件化，提示词不含示例段、仅由 generate.instruction ×（可选）styles 驱动（Persona Hub / Cosmopedia 形态 [34][35]），须显式给出量目标 generate.standalone_count。
生成调用	每次调用随机不放回抽 min(generate.seeds_per_call（默认 3），可用种子数）条种子作为示例（种子池小于抽样数时取全池）（Self-Instruct 的 in-context 自举结构 [18]），system = generate.instruction，要求输出 {"samples": [str, ...]}（恰 generate.num_per_call 条，默认 4），经 M8 校验。调用次数 = [种子数 × generate.num_per_record / num_per_call]（generate_only 种子池形态同式，种子数 = len(seed_examples)，seeds_per_call 从种子池抽；无种子形态 = [standalone_count / num_per_call]，提示词省去示例段）。
多模型混合（v1.2）	generate.llms 为 profile 引用数组（默认 ["default"]），取代 v1.1 的单值键 generate.llm，5.2 配置表已同步修订）。每次生成调用按 generate.mixture 选定 1 个 profile: "round_robin"（默认）按调用序轮转；"weighted" 按 generate.weights 加权随机抽样。抽样 PRNG 用 ctx.rng（3.10.3，随 run.seed 派生）；实现须在并发派发前按调用序号 0..C-1 一次性预抽全部 (llm, style) 对（round_robin 的轮转序同样以调用序号为准），使结果与并发调度顺序无关，逐调用可复现。动机：单一模型自生成会使产出分布收窄、尾部逐渐消失（model collapse, Shumailov et al., Nature 2024 [36]），异构生成器混合是公认缓解；distilabel 的「任务级绑定任意 LLM」为同构工业设计 [5]。
风格条件化（可选）	[[generate.styles]] 子表（每项 name + prompt）非空时，每次生成调用经 ctx.rng 均匀抽取 1 个 style，其 prompt 以「[风格要求] ...」格式追加在 generate.instruction 之后（仍为确定性模板拼接，3.6.1 边界不变）。persona / 受众条件化提升合成多样性的背书：Persona Hub 以 10 亿 persona 条件化提示 [34]；Cosmopedia 按受众 × 风格分桶派生提示 [35]。溯源与可观测（v1.2 只增）：新记录 _meta.source 增 generator = {"llm": <profile>, "style": <name>\[null]}（未配置 styles 时 style 为 null；6.3 信封只增字段）；report.generate 增 buckets 统计——每 llm×style 桶的调用数 / 产出条数 / 去重存活数，令多样性可观测：某桶去重命中率显著偏高 ⇒ 该桶贡献的多样性低，应调整其权重或 style prompt。
样本回调过滤（v1.5，可选）	generate.sample_validator 配置时，每条样本文本在相似度过滤之前先过用户回调 fn(text) -> list[str]：非空 ⇒ 剔除该样本（过滤语义，与相似度过滤同性质：不重试、不产生 failed 记录），桶统计增 rejected_by_validator（6.4 只增字段）。回调抛异常 ⇒ 该样本按违规剔除并 stderr warn 一次性提示。
新记录构造	每条样本文本构造 Record: raw = {input.text_field: sample}, id 规则同 M2, ref.generated_from = [种子id列表]；generate_only 模式下 generated_from = []（种子非记录、无记录 id，种子本身在 project.toml 中可审计），generator 照常携带。
回流	新记录组成「生成子批」交回 M10，从 M3 起走 去重 → 打分 → 标注 → 校验；去重索引含全部原始记录与先前生成样本，即 Self-Instruct 的相似度过滤 [18]（3.3.3 节）。子批不再触发生成（单轮回流，不递归）。generate_only 模式下生成子批即唯一数据来源：按 run.batch_size 切批走同一链路 M3→M4→M5→M7→M11（3.10.3），单遍不递归同样适用。
按类种子池（v1.7）	classify 启用时（process 模式）：种子按 classification.label 分组为按类种子池，每类用该类有效 instruction / styles / num_per_record / temperature（class_views[label].generate）独立走「生成调用」行的量公式：llms / mixture / weights / seeds_per_call / num_per_call 恒为全局（5.2 按类覆盖白名单表）。类段字典序拼接调用序：参与类（有种子的类）按类名字典序占据连续的全局调用序号区间，每类预算 C_c = [len(seeds_c) × num_per_record_c / num_per_call]；单遍 i = 0..C-1 预抽——llm 照旧按全局序号选定（round_robin 零 rng 消耗 / weighted 逐 i 一次 choices，「多模型混合」行机制不变），style 从该 i 所属类的有效 styles 中均匀抽，种子抽样按全局序号升序逐调用执行；classify 关闭 ⇒ 单一匿名段 = 现行为。种子门槛按类默认链：每类取全局 generate.seed_min_score → 缺省取该类有效 quality.threshold → 再缺省取该类种子池聚合分中位数；select_seeds 按 label 分组返回。规划产物 CallPlan 增 class_name 字段，one_call 按 plan.class_name 取类有效 instruction / temperature；postprocess_samples 返回 list[tuple[Record, str \[None]], run() 构造 PipelineItem 时新样本继承种子类——带 Classification(label, (label,), "inherited", {})（零额外分类调用；回流子批经 M13 幂等跳过，3.13.4）。桶统计 key 在 classify 启用时扩展为 <class>×<llm>×<style>（6.4；关闭时格式不变）。generate_only 模式：generate_all 扁平路径不变——生成用全局指令（无输入无从按类），产物由链上 classify 正常分类后按类打分/标注；不支持 generate_only 按类生成配比（1.6 v1.7 对齐决策 ③，与 8.3 O6 一并立项）。
时间流形态（v1.13）	generate_stream.enabled = true（generate_only 第三形态，默认关）时本模块改产序列而非独立样本：种子概念退场——量目标由按序列表的 sequences（尝试配额）× len_range（步数区间）承载；LLM 只做两类内容调用（一序列一次蓝图 + 一次帧实现，噪声帧批量实现复用「生成调用」行的既有模板与 Schema），会话装箱 / 交叉 / 噪声插入 / 原样重发 / 时间戳铺设全部由零 LLM 的机械交织器完成；产物一式两份（可重放的时间流工件 + 直装序列信封）。全文见 3.6.5；本表其余各行在本形态下的效力见该节「生成键效力矩阵」。

上下文预算装填 (v1.11)	本次调用的目标 profile 声明 <code>context_window</code> 时按上下文预算装填生成调用（未声明 = 预算关闭，行为与 v1.10 一致；预算/估算/校准机制见 3.9）： <code>seeds_per_call</code> 降为 上限值 ——按 <code>rng</code> 采样序从 尾部丢弃种子 直到装下（确定性；被丢种子不递补）， <code>min 1</code> ：连 1 条种子都装不下 → 该调用按 <code>context_overflow</code> 处置（V10, 7.6）。 <code>generate.llms</code> 多 profile mixture（ <code>round_robin</code> / <code>weighted</code> ）下按本次调用的目标 profile 的预算装填——（ <code>llm, style</code> ）预抽序与轮转序不变（确定性），装填结果仍逐调用可复现。系统侧静态部件（ <code>instruction</code> / <code>styles</code> / 生成输出 Schema）不动态裁剪，由 M1 静态预检把关（V13③, 3.1.4）。
-----------------	---

3.6.3 API

```
class GenerateStage(Stage):
    name = "generate"
    async def run(self, batch, ctx) -> list[PipelineItem]:
        """返回值为新生成记录的子批（原批不修改）；M10 负责回流调度。
        单次生成调用经 M8 修复仍非法或重试耗尽 → 该调用作废并计入
        report.generate.buckets (calls 计入、produced 为 0)，不影响其他调用与原批。"""
```

背书：生成模式为 Self-Instruct（ACL 2023）的种子自举 + 相似度过滤流程 [18]，指令可按 Evol-Instruct 风格写深化/扩展变体 [19]；多模型混合与风格条件化的多样性背书：Persona Hub [34]、Cosmopedia [35]、model collapse 的多生成器缓解 [36]；「任务级绑定任意 LLM」为 distilabel 的同构工业设计 [5]。

3.6.4 输入 / 输出示例

沿用统一文本示例（意图标注工程，仅文本模态），`project.toml` 追加：

```
[generate]                                # project.toml 追加片段
enabled = true
llm = "default"
instruction = """你是中文输入法的真实用户。模仿示例指令的口吻与场景，生成全新的一句话中文指令：
日常场景、口语化、诉求明确；只借鉴风格与题材范围，不得复述示例内容。"""
num_per_record = 2
seeds_per_call = 3
num_per_call = 4                          # temperature 取默认 0.9
```

v1.2 键名迁移与多样性设定补充：上述片段中的单值键 `llm = "default"` 在 v1.2 写作数组键 `llms`（5.2 配置表），本示例取双模型轮转 + 两个风格模板（种子选取、调用次数与下方样本文本均不变）：

```
llms = ["default", "judge"]                # v1.2：取代 llm = "default"；元素须为 [llm.*] profile
mixture = "round_robin"                    # 第 1 次调用走 llms[0]="default"，第 2 次走 llms[1]="judge"

[[generate.styles]]                        # 可选；每次调用经 ctx.rng 均匀抽 1 个 style
name = "concise"
prompt = "指令务求简短口语化，一句话直接给出诉求，不加铺垫。"

[[generate.styles]]
name = "scenario"
prompt = "指令中须带出一个具体的生活或工作场景（对象、事由或时间）。"
```

抽中 `style` 的 `prompt` 以「[风格要求] ...」追加在 `generate.instruction` 之后（3.6.2 风格条件化行）。本示例第 1 次调用轮转到 `"default"`、`ctx.rng` 抽中 `"concise"`，`system` 末尾追加「[风格要求] 指令务求简短口语化，一句话直接给出诉求，不加铺垫。」；第 2 次调用走 `"judge"`。

设 `quality.threshold = 0.5`（故 `generate.seed_min_score` 默认取 0.5），本批过门槛种子恰 3 条；调用次数 = $[3 \times 2 / 4] = 2$ 。第 1 次调用抽全部 3 条种子入提示词（`system = generate.instruction` + M8 持有的内部生成输出 Schema，要求 `{"samples": [str, ...]}` 恰 4 条）：

```
种子：1cda030abc565f17 "帮我写一条请假条，明天上午要去医院"
      d5ad41d6357f8a55 "写一份周报模板"
      7ed3a60f4714c33f "帮我把这段话翻译成英文....."

响应(经 M8 校验)：{"samples": [
    "帮我写一段给客户的道歉话术，快递发错货了",
    "把'会议改到下周三下午三点'翻译成英文",
    "帮我编一条朋友圈文案，晒周末爬山的照片",
    "写一个辞职信的开头，语气委婉一点"]}]
```

每条样本按 3.6.2 构造新 `Record`（`raw = {input.text_field: sample}`，id 规则同 M2）。第 1 条：

```
Record(
  id      = "31dae67e9b295e34",          # sha256(canonical_json(raw))[:16]
  modality = "text",
  text    = "帮我写一段给客户的道歉话术，快递发错了货了",
  raw     = {"instruction": "帮我写一段给客户的道歉话术，快递发错了货了"},
  ui_tree = None, image = None,
  ref     = RecordRef(source_file="", line_no=None, pair_index=None,
                      generated_from=("1cda030abc565f17", "d5ad41d6357f8a55", "7ed3a60f4714c33f"),
                      generator={"llm": "default", "style": "concise"}))
```

v1.2 设定下，该 Record 出自第 1 次生成调用（"default" × style "concise"），主输出中其 `_meta.source` 片段（6.3；generator 为 v1.2 只增字段，计入 `report.generate.buckets` 的 `default×concise` 桶）：

```
"source": {"file": "", "pair_index": null,
           "generated_from": ["1cda030abc565f17", "d5ad41d6357f8a55", "7ed3a60f4714c33f"],
           "fields": {}},
"generator": {"llm": "default", "style": "concise"}} // 未配置 styles 时 style 为 null
```

4 条新 Record 组成生成子批交回 M10，从 M3 起回流（单轮，不递归）：与全部原始记录及先生成样本做去重，MinHash-Jaccard ≥ 0.85 者标记 `dropped_dup`（3.3.3 的 Self-Instruct 相似度过滤）。

纯生成模式变体（v1.4，无输入数据）

```
# project.toml (纯生成工程；无 [run].input)
[run]
mode = "generate_only"
output = "./out/synth-ime-0702.jsonl"
modality = "text"

[generate]
enabled = true
llms = ["default", "judge"]
mixture = "round_robin"
instruction = "" (同上例生成指令) ""
seed_examples = [
  "帮我写一条请假条，明天上午要去医院",
  "写一份周报模板",
  "帮我把这段话翻译成英文....."
]
num_per_record = 2
# 无种子形态则改为：省去 seed_examples，设 standalone_count = 500 (调用数 = {500/4} = 125)
```

与上例（process 模式）的差异仅在入口与种子来源：无 M2 接入（IngestReport 全零、`report.counts.scanned = 0`），生成样本按 `run.batch_size` 切批走 M3→M4→M5→M7→M11；新 Record 的 `generated_from = []`、`generator` 照常携带（如 `{"llm": "default", "style": null}`）。6.4 计数不变量退化为 `emitted + dropped_* + failed = generated`，仍成立。

时间流形态静态走查（v1.16）

`examples/synth-stream` 是 v1.16 的可运行教学配置，不在本节写入任何一次运行的产出数量、耗时、哈希或调用统计。它固定展示规则、日历窗口、类型敏感关联、时间字段回填、按类档位表和序列验证钩子如何共同进入联合规划与下游校验。

工具级 profile 明确关闭 DeepSeek thinking，并为 L0 关闭的文本 JSON 路径保留结构结果预算；`max_output_tokens` 不是关闭 thinking 的替代方案：

```
[llm.default]
provider = "anthropic"
base_url = "https://api.deepseek.com/anthropic"
model = "deepseek-v4-flash"
api_key_env = "LABELKIT_DEEPSEEK_KEY"
supports_structured_output = false
supports_vision = false
max_output_tokens = 8192
thinking = "disabled"
```

当前工程的流铺设与配额面为：

```
[stream]
order_by = "meta:ts"
gap_s = 3600
session_max_len = 12
session_max_span_s = 3000

[generate.stream]
sessions = 5
noise_ratio = 0.1
duplicates = 1
frame_gap_s = [5, 60]
ts_start = "2026-01-05T09:00:00+08:00"

[class.ticket_booking.generate]
sequences = 3
len_range = [4, 5]

[class.smart_home.generate]
sequences = 3
len_range = [4, 5]
```

帧类闭集是 `task_request`、`acknowledgement`、`followup`、`progress`、`confirmation`。`task_request` 和 `acknowledgement` 是结构化帧，均含 `subject_id`；其余三类是纯文本帧。`task_request` 的 `duration` 绑定 `gap_next_s`，从 LLM 面向的 Schema 中剔除，并在时间轴 定稿后按同一序列的下一成员回填。

声明式规则固定为：`task_request` 初始化且恰有一次；它以 `chain_response` 在 `[1200, 2400)` 秒内响应到 `acknowledgement`，两帧的 `subject_id` 必须按运行时类型敏感规则相等；`acknowledgement` 恰有一次；它响应到 `confirmation`；`confirmation` 是序列末帧且恰有一次。`task_request` 必须落在工作日的 `[08:00, 11:00)` 或 `[14:00, 17:00)` 窗口；`ticket_booking` 的按类窗口整表覆盖为 `[09:00, 11:00)` 或 `[14:00, 16:00)`，`smart_home` 继承全局窗口。序列级 `generate.sequence_validator` 再校验位置连续、以 request 开始并以 confirmation 收尾。

全局档位表和 `ticket_booking` 的按类档位表分别定义帧类构成；`smart_home` 使用全局表，`ticket_booking` 使用自身整表。`tier_rank` 只在所属序列类的生效表内有意义，读取工件 真值或主输出元数据时必须同时读取 `sequence_class`。

当实际 `--limit` 配额前缀包含有效 rules/windows 时，M6 在任何内容调用前调用共享 planner：先为每个 attempt 恰抽一次长度偏好，再由一个 CP-SAT 联合模型以偏好名次和为 主目标、可行 noise 为次目标，一次冻结长度、帧类 word、session、timestamp、crossing 和 noise 槽。它不逐候选重求，不按候选长度分别求解，不重抽、不放宽约束，也不 fallback。冻结 后每条 attempt 依次调用一次 brief 和一次 realize；LLM 作废只删除该 attempt 并投影幸存 布局，不重规划。M1、dry-run estimate 和 M6 复用同一问题构造与求解入口。

v1.16 最终真实验收事实（2026-08-20）

使用上述当前配置和 DeepSeek profile 的最终完整真实生成 exit 0。报告观测为：`generated = 6`、`emitted = 5`、`dropped_verify = 1`、`failed = 0`；`planned = 6`，`ticket_booking = 3/3`、`smart_home = 3/3`；`tiers.ticket_booking.1 = 1/1`、`tiers.ticket_booking.2 = 2/2`、`tiers.smart_home.1 = 2/2`、`tiers.smart_home.2 = 1/1`；`sessions = 5`、`crossed_sessions = 1`、`frames = 27`、`noise_frames = 3`、`duplicates = 1`、`calendar_days_spanned = 8`、工件 34 行，`sha256:927e469e16df3f007f057357a267b8f8228506a5dfb279dc83bdfa1f1da672bf`。

调用计数为 `plan_calls = 6`、`realize_calls = 6`、`noise_calls = 1`；规则计数为 `sampled = 6`、`correlation_scrapped = 0`、`temporal_scrapped = 0`、`sequence_validator_scrapped = 0`。LLM 用量为 53 calls、12173 prompt、4487 completion、0 retries，耗时 35.214s。本次保留 planner 规划的真实 crossing；此前作废投影后 `crossed_sessions = 0` 的运行属于历史证据，不是当前验收值。

正式 `project-replay.toml` 重放上述 34 行工件也 exit 0：`scanned = ingested = 34`、`episodes = 6`、`absorbed = 31`、`dropped_noise = 3`、`dropped_dup = 1`、`emitted = 5`、`failed = 0`；`sessions = 6`、`mean_episode_len = 5.17`、`windows = 7`。LLM 用量 12 calls、4542 prompt、721 completion、0 retries，耗时 5.475s。

3.6.5 时间流形态（v1.13—v1.16）

`generate_stream.enabled = true` 时（M1 硬合取 `generate_only ∧ text ∧ generate.enabled ∧ classify.enabled ∧ stream.order_by = "meta:*" ∧ meta_mode != "none"`，3.1.4 时间流生成行），M10 的 `generate_only` 分支改调本节的时间流入口恰一次（`ctx.batch_no == 0`、`ctx.rng == Random(f"{seed}:0:generate")`，3.10.3），3.6.2 的平面路径（`generate_all`）不参与、代码面零改动。本形态的产物是一式两份：按交织序定稿的时间流工件行（交 M11 第五通道落盘，6.5）与直装序列信封（`kind = "sequence"`，带序列类与帧类两级 `inherited` 标签，按 `run.batch_size` 切批自 M3 起走链，3.10.3）。`segment / stitch / extract` 不参与——流本就是生成出来的，不需要再切分；`stream × generate` 互斥条文字面维持（2.3.1、8.3 O3 核销）。

公开面（签名冻结）：


```

async def generate_stream_all(ctx: RunContext) -> StreamGenerateProduct:
    """时间流形态入口 (M10 分支调用一次)。计划期抽签 → 派发 (蓝图 → 帧实现逐序列
    作业与噪音批并发) → 逐帧钩子与序列相似度过滤 → 机械交织 → 直装组装。
    作废序列只缺席, 不产 failed 记录、不写 item.errors。"""

@dataclass(frozen=True)
class StreamGenerateProduct:
    envelopes: list[PipelineItem]    # 直装序列信封 (计划序)
    artifact_lines: list[str]        # 工件行 (交织序定稿; 行号 = 列表序 + 1)

def plan_stream(cfg: ResolvedConfig, rng: random.Random) -> StreamPlan:
    """计划期纯函数 (吃 cfg + rng, 零 IO 零 LLM): 配额展开 → 逐序列长度 →
    逐序列 (llm, style) 预抽 + 噪音批预抽。M10 的 estimate_run 复用之做
    精确复演 (非上界估算, 3.10.3)。"""

def weave_stream(survivors, noise_payloads, cfg, rng) -> tuple[list[list[Slot]], dict]:
    """机械交织器入口 (纯函数族, 零 LLM 零 IO): 重复选取 → 装箱洗牌与成对交叉 →
    逐交叉会话切换点 → 逐噪音帧掷签 → 重发落流尾新会话 → ts 铺设。"""

def assemble_stream(sessions, survivors, cfg) -> tuple[list[str], list[PipelineItem]]:
    """直装组装: 逐槽位构造工件行对象与成员 Record, 逐序列构造序列 Record 与信封。"""

def stream_artifact_path(cfg: ResolvedConfig) -> str:
    """工件路径 = 输出路径去末级后缀 + ".stream.jsonl" (M11 以同一规则各自推导,
    算子间不互导; 两侧等式由测试钉住)。"""

# — v1.14 增补 (两枚零 rng 纯函数) —————

def apportion_tiers(sequences: int, tiers) -> tuple[int, ...]:
    """档位配分 (落点在 common 层的 M1 配置模型侧, M6 反向导入): 把一个序列类的
    配额按 weight 走整数域最大余额法配到各档, 按 tier_rank 升序返回逐档配额。
    纯函数、零 rng—M1 的逐非零配额对约束与 M6 计划期共用同一实现。
    v1.15: `tiers` 传该类的**生效表** (effective_tiers 的返回值)——本体与签名
    零改动, 改的只是调用点; 「全表连续覆盖 1..N」相应读作「每张生效表各自」。"""

def tier_rank_for_ordinal(sequences: int, tiers, ordinal: int) -> int | None:
    """类内序数 → 所属档位序数 (配分结果按 tier_rank 升序切成连续分块的前缀和查表)。
    sequences 须传该类**全量**配额而非 --limit 截断后的条数; 档位表为空时 None。
    v1.15: `tiers` 同取该类生效表, 故序数分块是**类内**的一返回值是类内档序数,
    跨类组不比 (本体与签名零改动)。"""

def backfill_time_fields(sessions, cfg) -> None:
    """机械回填尾声 (零 rng 零 LLM 零 IO): 调用点 = weave_stream 之后、
    assemble_stream 之前, 按已铺时间轴把绑定字段就地写入共享载荷对象。"""

# — v1.15 增补 (生效档位表的单点查找, 零 rng 纯函数) —————

def effective_tiers(class_tiers, global_tiers) -> tuple[TierSpec, ...]:
    """取一个序列类的**生效档位表** (裁决·表级原子覆盖 + 裁决·全局表为锚): 类声明了
    就用类的整张表 (`class_tiers` 非 None), 未声明则回落全局表。落点同
    apportion_tiers—common 层的 M1 配置模型侧, M6 与 M10 反向导入; M1 约束簇、
    M6 计划期、M10 报表装配三方共用同一实现, **档位取表点全库只此一处**。
    全局表为锚 ⇒ 档位面在场 ⇐ 全局表非空 ⇒ 每个参与类恒有非空生效表。"""

```

要点 (规格与理由):

- **抽签消费顺序表 (按形态冻结)**: 无有效 rules/windows 的实际前缀沿用 v1.15 默认路径: 一条 `Random(f"{seed}:0:generate")` 先按类名字典序展开配额 (`--limit` 在此做前缀截断, 零 rng), 再抽每条序列长度、(llm, style), 交织期再按既有顺序处理重复、装箱、交叉、噪音、重发与 ts。档位配分和时间字段回填仍是零 rng 纯函数, 不改变该默认路径。实际前缀含有效 rules/windows 时切换 v1.16 受约束顺序: 配额展开 → 一个 31-bit solver seed → 每个 attempt 恰一次 `randrange` 循环长度偏好 → (llm, style) 预抽 → duplicate source 顺序预抽 → noise 内容调用计划; 一个 CP-SAT 联合模型以长度偏好名次和为主目标、可行 noise 为次目标, 一次冻结长度、frame-class word、session、timestamp、crossing 和 noise 槽, 之后才派发内容调用。求解失败不触发重抽、逐候选重求、放宽约束或 fallback; 内容作废只改变后续投影输入, 确定性以 LLM 内容产出为条件 (2.6 声明链)。
- **档位构成 (v1.14 全局表 `[[generate.stream.tiers]]`; v1.15 增按类表 `[[class.<name>.generate.tiers]]`; 默认均不在场)**: 一个档位的定义就是该档序列的帧类构成集合, 不携带质量指令、不控制帧内部语义质量 (那归帧类生成指令与温度)。**生效表 (v1.15)**——档位表分两级, 一个序列类的**生效表** = `effective_tiers`(该类表, 全局表): 类声明了就用类的整张表 (**表级原子覆盖**, 不逐行合并——行级合并会让 rank 身份跨表漂移), 未声明则回落全局表。全局表是**锚** (按类表要求其在场, 3.1.4), 故「档位面在场」⇐ 全局表非空, 且每个参与类恒有非空生效表; `tier_rank` 是**类内身份**——每张生效表各自连续覆盖 1..N (N 逐类可不同), 跨类同 rank **无任何工具语义**、跨类同构成合法。下文凡「档」与「档位表」均读作该序列类的生效表; 无任何按类表时生效表恒 = 全局表, 本行与 v1.14 逐字同义。**配分**——每个参与

类的 `sequences` 配额按本类生效表各档 `weight` 走整数域最大余额法配到各档 (基额 = $(\text{sequences} \times \text{weight}) // \Sigma \text{weight}$ 、余额键 = $(\text{sequences} \times \text{weight}) \bmod \Sigma \text{weight}$ ，按余额键降序、平票按 `tier_rank` 升序逐档 +1 至配满；禁止任何浮点中间量——平票判定要一路喂给类内序数分块 $\rightarrow \text{truth} \rightarrow$ 工件字节 $\rightarrow$ 成员 `id`，是冻结面)；类内序数按 `tier_rank` 升序占连续区间，(类，类内序数) $\rightarrow \text{tier_rank}$ 即配分结果的前缀和查表。配分是 (sequences, 生效表) 的纯函数、零 `rng`：它插在抽签消费顺序表的 ① 配额展开与 ② 长度抽取之间作为零消费步，顺序表原文不动，同 `seed` 下有无档位表 (含按类表) 的抽签流逐字节一致。推论：`--limit` 的前缀截断只切掉尾部序数，即在每个类内从最高档序数侧截起 (低档序数在前)，截断与档位映射可交换——按类表下各类的分块各不相同，本推论逐类原文成立。构成恰等——蓝图 Schema 的 `enum` 限档内子集给「`⊆`」、`cover_all` 注入的逐类 `contains` 给「`⊇`」，合成「一条序列的 `members[]` 帧类集合 $\equiv$ 其档声明的构成」，故档位身份可从数据反推对账 (可审计性；按类表下反推须先按行的序列类取其生效表，行内 `classification.label` 与工件 `truth.sequence_class` 天然给出该类名)。标识三点——`_meta.source.generator.tier_rank` (6.3)、工件 `truth.tier_rank` (6.5)、`report.generate.stream.tiers` (6.4)；三点的在场判据是全局表非空 (全局表为锚 $\Rightarrow$ 在场性恒定，不因某类未声明按类表而逐行漂移)，全局表缺省时三点全部不在场。M1 侧的身份连续性、构成互异 (逐生效表执行)、逐非零配额对的长度可覆盖、两条 `WARN` 与按类表前提三子款见 3.1.4。

- 蓝图调用 (一计划期配额序列一次——存活与否是此调用之后的事，故估算基数恒为 $2 \times \Sigma \text{sequences}$)：`system` = 计划器指令 + [任务] 类有效生成 `instruction` + [帧类表] (`name: description` 行) + 结构句；`user` = 「请为一条「{类名}」序列产出 {L} 步蓝图。」；`schema` = `plan_schema`(帧类名集, L) (内部待遇, 3.8.1)。【帧类表】的取值与 `user` 行随档位表条件化 (v1.14)：档位表缺省 $\Rightarrow$ 表为全类表、`user` 行取上述原形 (与 v1.13 逐字节一致)；档位表在场 $\Rightarrow$ 表只渲染档内子集 (帧类表按声明序过滤本序列所属档的 `frame_classes`；v1.15：所属档 = 该序列类的生效表内 `tier_rank` 对应的那一行——`effective_tiers`(类表, 全局表)[`tier_rank - 1`]，生效表连续覆盖 1..N 故下标法照旧)、`user` 行取冻结变体「请为一条「{类名}」序列产出 {L} 步蓝图，且 [帧类表] 中每个帧类都至少出现一次。」、`schema` 取 `plan_schema`(档内名集, L, `cover_all=True`)。修复穷尽 / 不可装填 $\Rightarrow$ 该序列作废、计 `plan_failures` (覆盖违约不新增失败机制——它是普通的 L2 违规，进 M8 既有修复环)。模板 `verbatim` 冻结于 `CONTRACTS §10.14`——L0 关闭的端点 (如 `DeepSeek anthropic` 路由硬拒强制工具调用) 上，结构服从性靠模板内嵌的结构契约与覆盖句兜底，这是硬要求不是兜底优化。
- 帧实现调用 (一蓝图一次)：`system` = [任务] 类有效 `instruction` + [风格要求] (预抽 `style`，可缺——蓝图不带风格) + 结构句 + 逐位契约行「第 i 帧 ({帧类}) 须符合：{该帧类生成 Schema 文本 | 自由文本契约}」；`user` = 蓝图逐步行 i. [帧类] {brief} + 「请实现全部 {L} 帧内容。」；`schema` = `realize_schema`(逐步 Schema 序列) (`prefixItems` 逐位包装，纯文本帧位取 {`"type": "string"`})。默认 v1.15 路径的纯文本契约字面量仍为「自由文本一段」；受约束 v1.16 路径改为「JSON 字符串 (如 "...")，不得用对象包裹」，只明确 JSON string 的表示方式，输出语义仍是自由文本，不改变调用次数或 Schema。逐位 Schema 与契约行取的是缩减 Schema (v1.14)——声明了时间字段绑定的帧类，其绑定键从「该帧类生成 Schema」中剔除后才进这两处 (见下方「时间字段回填」行；无绑定帧类与纯文本帧位逐字节不变)。结构化帧的帧内容以对象原样落工件行的文本字段 (纯文本帧落字符串)，成员 `Record.text` 取其 M2 语义投影 (对象 $\Rightarrow$ canonical JSON，字符串 $\Rightarrow$ 直取——与重放时 M2 的点路径抽取产出同一投影, 6.5)。修复穷尽 / 降级穷尽 $\Rightarrow$ 序列作废、计 `realize_failures`。模板 `verbatim` 冻结于 `CONTRACTS §10.15`。
- 噪音批量实现：复用 3.6.2 的既有生成模板与 `samples_schema`，调用数 = [噪音帧数 / `generate.num_per_call`] (噪音帧数 = `round(noise_ratio \times \Sigma \text{任务帧数})`)；单批作废 $\Rightarrow$ 缺额帧从交织中缺席 (不补生成)。
- 逐帧钩子与序列相似度过滤：`generate.sample_validator` 对帧实现产物逐帧文本执行——任一帧违规 $\Rightarrow$ 整序列作废 (蓝图定长不可剔单帧，拒绝采样语义) 并计 `validator_scrapped` 与桶 `rejected_by_validator`；随后 M6 内置的相似度过滤单元上移为序列级：判重文本 = 成员 `text` 按序 "`\x1e`" 拼接 (M3 序列配方式)、比对面 = 兄弟序列 (本形态无种子)、参数取 [dedup] 三键，淘汰以桶 `survived_dedup` 差呈现 (桶键在本形态为 `<class>x<llm>x<style>` 三段式——`generate_only` 首现类段)。
- 机械交织器 (零 LLM 零 IO)：`sessions_eff` = $\min(\text{sessions}, \Sigma \text{幸存})$ ，交叉对数 = $\Sigma \text{幸存} - \text{sessions_eff}$ (M1 已静态保证 `sessions \leq \Sigma \text{sequences} \leq 2 \times \text{sessions}`，故交叉并发度恒 $k \in \{1, 2\}$)；单个交叉会话形态 = A 段 + B 段 + A 余段 + B 余段 (切点 `cut_a` $\in [1, |A| - 1]$ 、`cut_b` $\in [1, |B|]$ 保证真交叉；一方不足 2 帧时与另一方互换，两方都不足则退化为顺次拼接——纯长度条件、零 `rng`)；噪音帧逐帧掷签 (会话、槽位)，满员会话 (`len \geq \text{stream.session\_max\_len}`) 退出签池，签池耗尽 $\Rightarrow$ 余帧缺席 + `WARN`；重发序列 (`duplicates`，超出幸存数时钳制 + `WARN`) 取自幸存集、帧内容逐字节同源、恒落流尾新会话；`ts` 铺设自 `ts_start` 起严格递增——会话内帧间隔 `uniform(frame_gap_s)`、跨会话间隔 `uniform(gap_s + lo, gap_s + hi)` (恒 $> \text{stream.gap_s} \Rightarrow$ 摄取则按同一 `gap_s` 复演出相同会话切分)，微秒精度 ISO-8601 写出。交织尾声统一回填 `truth.session` (全流会话序数 0 基)。
- 时间字段回填 (v1.14, [frame.class.<name>.generate.time_fields]；默认不在场)：帧生成 Schema 里的时间语义字段与实际帧间隔本就对不上——LLM 没有时间轴，写出来的秒数是编的。处置是「绑定即剔除 + 机械回填」两步。绑定即剔除：被绑定字段从 LLM 面向的逐位 Schema 与逐位契约行中一并剔除 (缩减 Schema = 生成 Schema 删该 `properties` 键、`required` 取差集、其余关键字原样；派生须重建顶层与 `properties` 两层，绝不就地改动 M1 冻结的帧类生成 Schema——静态预算预检与契约行渲染同源读它)，M8 按缩减 Schema 校验，LLM 越权输出绑定字段则按 `additionalProperties` 违规进修复环。不为注定被覆写的字段付 token 与修复环成本，契约也不误导。机械回填：新的回填尾声 (零 `rng` 零 LLM 零 IO) 位于 ⑨ `ts` 铺设之后、直装组装之前，只遍历任务帧槽位并按所属序列归组 (会话序即序内成员序——交叉切片不改序内次序)，对绑定帧类逐帧按语义词表算值就地写入共享载荷对象——`ts` = 该槽位已铺 ISO 串；`gap_prev_s / gap_next_s / elapsed_s` = 本序列相邻成员 / 首帧的 `ts` 差 `round(·, 6)` (微秒精度与 `isoformat` 对齐)，首帧的 `gap_prev_s / elapsed_s` 与末帧的 `gap_next_s` 恒 0.0。序内口径的理由：交叉会话夹入的外序列帧与噪音帧本就占用其间墙钟，序内差值才与下游从数据实测的口径一致。重发槽位不遍历也不触碰——它与源槽位引用同一载荷对象，回填自动生效且其 `ts` 绑定值承源、 $\neq$ 自身行 `ts` (原样重发本就携带陈旧内容，语义自治)；噪音帧与无绑定帧类不触碰；每个载荷对象恰被写入一次。位次的两条推论：① 回填先于行对象与 `id` 计算 $\Rightarrow$ `Record.raw / text / 成员 id`、序列 `id` 与 `session_id` 全部含回填值，工件重放逐字节同 `id` 同会话 (重放侧的判重档位

仍随分段判决浮动,「逐字节一致」≠「重放判重恒 exact」); ② `sample_validator` 逐帧校验与序列相似度过滤在交织之前完成, 故二者吃的是回填前载荷——时间量是机械量, 不参与内容校验与内容判重 (两序列内容相同仅时间不同 ⇒ 照旧判近重, 语义正确)。绑定字段上除 `type` 外的约束关键字不被强制也不被校验 (M1 为此发 WARN, 3.1.4) ——时间量的值域由时间轴决定。

- **直装组装**: 逐槽位构造工件行对象 {<ts字段>: ..., <text_field>: ..., "truth": {...}} (真值键集见 6.5) ——成员 `Record.raw` = 该行全对象、`id` = `sha256(canonical_json(raw))[:16]` (M2 公式 ⇒ 工件重放同 id)、`text` = `text_field` 值的 M2 语义投影、`ref` = `RecordRef(source_file=工件路径, line_no=行号, pair_index=None, generated_from=(), generator={"llm", "style"})` (v1.14: 档位表在场时 `generator` 取 {"llm", "style", "tier_rank"} 三键——v1.15 下键、序与在场判据均零改动, 只是值来源为本行序列类生效表内的档序数, 跨类不可比); `session_id` = `sha256("\n".join(会话内全部帧 id))[:16]` (M2 公式, 含噪音帧与重发帧 ⇒ 重放一致); 逐序列构造 `Record(kind="sequence", members=..., text/raw/ui_tree/image=None, ref=首成员 ref, id=sha256("\n".join(member ids))[:16])` (M14 公式, S24 字段惯例) 与信封——`classification` = `Classification(label=序列类, labels=(label,), source="inherited")`、`member_classifications` = {成员 id: `Classification(帧类, (帧类,), "inherited")`} (帧类真值随 `members[]` 落盘, 3.11.2)。噪音帧与重发帧只活在工件 (不构造信封、不进守恒账), 重发序列本体早已在 `envelopes` 中、不重复入列。
- **--limit 与量目标**: 单位 = 序列, 截断在计划期配额层 (类段字典序前缀) ——作废序列不再生成、不进交织 ⇒ 工件与主输出的覆盖面恒一致; M10 尾部另有一次 `belt & braces` 截断。按类 `sequences` 是尝试配额 (`standalone_count` 同款语义): 无输出条数保证、无补齐回路 (8.3 O6 辖区不变); `counts.generated` = 进链序列条数。
- **作废语义** (3.6.1 边界的序列版): 蓝图 / 帧实现 / 逐帧钩子任一环节失败 ⇒ 该序列缺席, 不产 `failed` 记录、不写 `item.errors` (种子概念不存在, 无「原批」可损); 留痕 = 计数器 + 一行值-free `stderr` WARN (`seq=` 序号 / `class=` / `llm=` / `call=` / `kind=`)。
- **预算与溢出纪律 (v1.11 家族)**: 三类调用发出前都做 `est(system) + est(user) + 2 × 消息包封 ≤ input_budget` (`supports_structured_output` 时另扣 `response schema` 文本) 的预检——不可装填即从不发出 (V10 先例, `precheck` 永不喂熔断); 帧实现的反应式溢出走序列对半分 (`schema` 与蓝图概要随切片同步减半, ≤ 2 级 AIMD, 每次计 `budget.degrade_retries`; 单步跨度或级数耗尽 ⇒ 序列作废), `reactive-400` 终局在作废吞点经共享 `budget.feed_reactive_terminal` 补喂熔断恰一次 (A7 纪律, 7.6)。
`TEMPLATE_HEAD_TOKENS` 增 `generate_plan` / `generate_realize` 两键 (噪音批复用 `generate` 键值), M1 静态预检增对应两段 (3.1.4)。
- **计数与观测**: `report.generate.buckets` 照常 (`calls` / `produced` / `survived_dedup` / `rejected_by_validator`); 新增 `report.generate.stream` 子块 (`counts-only` 12 键: `sessions` (交织出的会话数, 不含重发尾会话; 作废序列会使其低于声明的 `sessions` ——交织按 `sessions_eff = min(sessions, Σ幸存)` 装箱) / `crossed_sessions` / `sequences.<class>.{planned, produced}` (`produced` = 该类最终进链的序列数, 即过了蓝图、帧实现、逐帧钩子与序列相似度过滤四关的条数——`planned - produced` 的缺口按环节分摊在 `plan_failures` / `realize_failures` / `validator_scrapped` 与相似度淘汰上, 后者只以桶 `survived_dedup` 差呈现) / `frames` (幸存序列的任务帧总数, 不含噪音帧与重发帧) / `noise_frames` (实际织入数) / `duplicates` / `plan_calls` / `realize_calls` (含对半降级后的分次调用) / `noise_calls` (三个调用计数在派发前递增, 故含被预算预检拦下、从未发出的调用——平面路径同款口径) / `plan_failures` / `realize_failures` / `validator_scrapped`, 6.4); `report.run` 摘要族增工件条目 (路径 / `sha256` / 行数)。v1.14 档位面增量: `report.generate.stream` 增 `tiers` 子块 (`counts-only`, 条件在场 = 全局档位表非空——全局表为锚故这一判据在 v1.15 下零改动; 键位冻结在 `sequences` 之后、`frames` 之前 (配额族相邻)) ——口径与 `sequences.<class>` 同款: `planned` 计于计划期逐序列、`produced` 数最终进链 (过了蓝图、帧实现、逐帧钩子与序列相似度过滤四关) 的条数。该子块由 M10 在报表装配时按声明档位表显式铺开 (3.10.3), 故零额档与全作废档也如实在场 (`planned 0` / `produced 0`), 不依赖计数器首触序。v1.15 双形 (裁决·嵌套报表全类铺开): 全部序列类都未声明按类表 ⇒ 平面形 {"<tier_rank>": {`planned`, `produced`}}, 按全局表 `rank` 升序铺开, 与 v1.14 报表逐字节相等; 任一按类表在场 ⇒ 类嵌套形 {"<class>": {"<tier_rank>": {`planned`, `produced`}}, 外层 = 全部声明序列类按声明序零基铺开 (`sequences.<class>` 同款)、内层 = 该类生效表 `rank` 升序, 键均为十进制字符串 (落盘无 `sort_keys` ⇒ 键序 = 装配插入序), 零配额类与全作废档同样如实呈现 0/0。两形下键位都仍冻结在 `sequences` 与 `frames` 之间。计数器键按类重冻结 (裁决·计数器键按类重冻结): M6 恒喂类段键 `generate.stream.tiers.<class>.<rank>.{planned, produced}` (单一喂数纪律, 禁双写两族键), 平面形由 M10 按 `rank` 跨类求和装配——数值与 v1.14 逐字节相等 (v1.14 的平面计数本就是跨类聚合值)。时间字段面零观测增量 (确定性机械操作, 无可计数的失败模式)。零新 `trace` 通道、零新事件 (两类调用经 `llm.call` 可见; `generate` 专属通道列 8.4 演进候选)、零新错误 `kind` (7.6; 覆盖违约复用 `schema_violation` 进修复环、作废复用 `plan_failures`; v1.15 的按类表前提错误走既有 `CONFIG_ERROR` 面)。零调用数变化——档位 (含按类化) 与时间字段三个机制均不改调用次数, `estimate_run`、估算行格式、八个 `dry-run golden` 与 `console` 键集全部零改动 (3.10.3、7.8)。

3.6.6 v1.16 序列规则与联合规划

时间流生成的生效规则或日历窗口在实际 `--limit` 配额前缀中出现时, M6 在任何 LLM 调用 之前进入全流联合规划路径。没有生效规则或窗口时, 继续使用 v1.15 默认路径; 序列级 `generate.sequence_validator` 可以独立运行, 不改变帧类词的规划开关。联合路径的入口、问题构造和求解与 M1、`estimate_run` 共用 `labelkit/common/runtime/sequence_planner.py`, 不在 M6 内复制一套约束实现。

规划期先展开按类名字典序的尝试配额, 取一个 31-bit solver seed, 再为每个 attempt 恰抽一次循环长度偏好。同一个模型以偏好名次和为主目标、最大可行 `noise` 为次目标, 联合定稿 长度、帧类 `word`、owner `session`、任务时间戳和可用 `noise` 槽; 所有 `task timestamp` 全局唯一, `session` 间隔至少为 `stream.gap_s + 1us`, `session` 内相邻任务帧满足 `replay guard`。双 owner `session` 必须有真实的 A-B-A 或 B-A-B 交替; `noise` 只放在存活任务帧首尾之间的开区间, 不参与规则和窗口。求解器单线程、固定 `search seed`、最大确定性时间预算 10 秒; `model proto` 超过 250,000 项、`INFEASIBLE` 或无法在预算内验证的 `UNKNOWN` 都由启动期 M1 拒绝, 运行期不靠重抽长度或重规划补救。

规划器先冻结 frame class，因此 LLM brief 调用不再返回类名。M8 使用 `brief_schema(length)`，只接受固定长度的逐位 `{brief: string}` 对象；prompt 同时带入固定 类词、生效 rules/windows/correlation 与按类 instruction。M6 将 brief 与固定类词合并，再用既有 `realize_schema(prefixItems)` 完成 payload。realize prompt 必须再次携带规则和 correlation 说明；有 correlation 的序列禁止 reactive halving，溢出或截断直接沿既有 `realize_failures` 作废整条序列，无 correlation 时保留既有有界对半降级。

每条序列的验证顺序冻结为：realize Schema、逐帧 `generate.sample_validator`、声明式 correlation/time、一次 `generate.sequence_validator`、序列相似度过滤，然后投影幸存 primary、过滤 noise 槽、回填 primary `time_fields`，最后复制已回填 payload 给 duplicate 并完成全局排序和组装。任一序列失败都只让该 attempt 缺席，不创建 failed 信封或写入 `item.errors`；hook 输入是 `SequenceValidationInput` 的深拷贝，hook 异常视同违规。

规则验证在冻结 skeleton 上重新枚举标准 occurrence 候选，不把 planner 的 potential witness 当作 i 对 i 的承诺。correlation 先按 JSON 运行时类型 and canonical bytes 相等过滤，再按半开 `time_s` 过滤；正规则则分别计 `correlation_scrapped` 或 `temporal_scrapped`，负规则的相关性失败统一计前者。无 correlation 的运行期失败表示 planner 不变量破坏，记录 ERROR 并抛 `InternalError`。`validator_scrapped` 恒等于 `'sample_validator_scrapped + correlation_scrapped + temporal_scrapped + sequence_validator_scrapped'`，相似度淘汰不进入该等式。

作废后的布局是已冻结 skeleton 的确定性投影：删除整条 attempt，移除空 session，按时间重新编号但绝不移动幸存时间戳；crossed 只有两位 owner 都幸存且仍有真实交替时保留。重复从调用前预抽的 source 排列中选取，复制 source 的 tier、frame class word 和已回填时间字段，按规则允许的最小时间平移放到结尾。noise 目标是最大化已规划槽位的目标值；目标无法全放下只记录一次值无关 WARN，不追加 LLM 调用。

M6 不改变 artifact 的 truth 键集、主输出 Schema、Record/session/id 公式或既有 trace 通道。规则和窗口不复制到工件；report 仅在实际非零配额类的生效面出现 `rules`、可选 validator 计数与 `windows.calendar_days_spanned`，并置于既有 tiers 后、frames 前。`MODEL_INVALID` 或 M6 发现已通过 M1 的模型不变量被破坏时走既有 `InternalError` / 退出码 4。

3.7 M7 二次校验 verify

3.7.1 职责与边界

做：用独立 judge profile 对每条（记录，标注）评审：输出 verdict (pass/fail) + 逐项批评意见；fail 时按策略丢弃，或将批评意见回喂 M5 重新标注（有界修复环）。**不做：**不自己改写标注（修复 = M5 重标注 + M8 重校验，M7 只供给批评意见）；不评审结构合法性（到达此处的标注必已合法）；不做打分（M4 职责）。

3.7.2 评审调用

```
system: 你是标注质量审核员。给定任务指令、原始数据与标注结果，独立判断标注是否合格。
        评审维度：① 是否遵循任务指令 ② 与原始数据的事实一致性 ③ 字段语义是否正确填写
        {verify.extra_criteria} # 可选，用户追加维度
        先逐维度给出简短意见，再给结论。
user:    【任务指令】 {annotate.instruction}
        【原始数据】 {record 内容, UI 模态含截图+树}
        【标注结果】 {annotation.output 的 JSON}
输出(经 M8 校验): {"critiques": [{"aspect": str, "opinion": str}], "verdict": "pass"|"fail"}
```

「先意见后结论」的顺序固定，利用自回归生成让结论以意见为条件（chain-of-thought 评审，Zheng et al. [20]）。judge profile 应配置为与标注 profile 不同的模型（自我评审存在自增强偏差 [20]），M1 在两 profile 的 model 字段相同时打印 warning（不阻断）。

按类取值 (v1.7)。classify 启用且记录带类标签时，本节模板的【任务指令】段与 `{verify.extra_criteria}` 均取该类有效值（分别为 `class_views[label]` 的 `annotate.instruction` 与 `verify.extra_criteria`，3.1.4 按类覆盖合并行）——按类标注配全局评审指令是语义错位，故两处同步取类值。`build_verify_prompt` 经 `options.label` 取值，`_judge_round` / `_reannotate` 透传（repair 重标注调 `annotate_record(record, ctx, AnnotatePromptOptions(label=..., ...))`，3.5.2 按类取值段）；`policy` / `max_repair_rounds` / `llm` / `judges` 恒为全局（5.2 按类覆盖白名单表）。trace `verify.verdict` 事件 payload 增 `label` 字段（仅 classify 启用时携带，7.2 只增不改）。

多评审团（可选，v1.2）：`verify.judges`（array，默认 []，与 `quality.judges` 语义一致）非空时启用评审团：空 = 单评审走 `verify.llm`，本节既有行为完全不变；非空须为奇数个 profile 引用（M1 校验，不满足报错退出码 2）。各 judge 按本节同一模板各自独立评审（互不可见对方意见），最终 `verdict` 取多数票；各方 `critiques` 全部合并保留进 `VerificationResult.critiques`（4.2），每条标注来源——条目增加 `judge` 字段（= profile 名）。trace 事件 `verify.verdict` 相应改为每 judge 一条，payload 新增 `judge` 字段（字段只增不改，7.2 事件契约向后兼容）。`policy = "repair"` 回喂 M5 时，【审核意见】段 = 全部投 fail 的 judge 的 `critiques` 合并（各条前缀来源 judge 名）。成本为单评审的 `|judges|` 倍，宜配置 3 个异构小模型 profile 而非加倍调用同一大模型。**背书：**多个较小模型组成的评审团（PoLL）在三种评审设置、六个数据集上优于单一大模型评审，因跨模型家族的多样性显著降低单模型自增强偏差，且成本比单一大评审低 7 倍以上（Verga et al. [32]）——与本节「judge 独立于标注模型」是同一去偏原则的推广。

stream 序列评审与缺陷表 (v1.8, S7)。stream 模式下序列信封 (episode, `record.kind = "sequence"`, 3.14) 的评审改走序列变体；非 **stream 路径零改动**——本节既有模板与评审 Schema 是回归锚，序列信封由 stage 层旁路驱动器承载。输出经 `schema_engine.defect_verdict_schema()` 校验 (3.8.1 内部 Schema 清单；与既有评审 Schema **并存**, S7)：三顶键 `{critiques, defects, verdict}` **全 required** (意见/缺陷在前、结论在后——本节「先意见后结论」同理)；`critiques` 形态与既有评审 Schema 逐字节一致 (原样走既有合并/回喂链路)；`defects` 逐项 `{kind, members, position, detail}` 四子键全 required, 可选性以可空联合 `["array", "null"]` / `["string", "null"]` 表达 (OpenAI strict 兼容, 3.8.1)。缺陷 `kind` 六值封闭词表 (v1.8 五值 + v1.9 增 `wrong_stitch`——defect Schema、`DEFECT_KINDS`、`report _DEFECT_KINDS`、`_route_defects` 四处同步扩值, 3.8.1/6.4)：

kind	语义
<code>label_mismatch</code>	标注的任务标签与序列证据不符。
<code>off_task_members</code>	段内混入与任务无关的成员帧 (<code>members</code> 列出这些成员帧 id)。
<code>missing_head</code> / <code>missing_tail</code>	段首缺少任务起点帧 / 段尾缺少任务终点帧 (结合边界余量判断)。
<code>missing_members</code>	段中缺失成员帧 (<code>members</code> 列出可指认的帧 id, 无从指认则为 null)。
<code>wrong_stitch</code>	v1.9: 线索的某处缝合是错误的——某碎片与线索其余部分不属同一目标导向任务 (结合 [片段结构] 判断； <code>position</code> 指向可疑碎片的线索内序数)。仅 <code>stitch</code> 启用时可判 (3.16)。

评审证据为单条 user 消息**七段序** (v1.8 为六段, v1.9 插入 [片段结构]；system 含六类缺陷说明 (v1.9 起)；全文逐字冻结于 `CONTRACTS §10.5` 序列变体)：[任务指令] (classify 启用时取类有效值, 同本节按类取值段) → [动作序列] (`item.transitions` 按 3.5.2 步骤行格式渲染, 接缝占位步带 `thread_seam` 后缀 (3.4.3 序列行同款)；`transitions` 为 None 时整段省略) → [片段结构] (v1.9, **仅 `stitch` 启用时在场**——关闭时整段省略、退回六段形态即 v1.8 回归锚：每碎片一行「线索内序数 / 帧跨度 / 首帧摘要」+ 接缝位置表 (`seam_indexes` 的文字化)；无此节 `wrong_stitch` 不可判, 3.16) → [边界余量] (段边界外前后 `k = 2` 帧的 `frame_digest` (4.3) 及其去向标注：noise / 相邻段序数 / 无——防切头切尾的证据段, 零额外 LLM 调用, 语音端点检测 hangover 惯例的移植 [54]；多碎片线索的边界 = 线索两端, 维持**首碎片头 / 尾碎片尾**邻帧 (接缝不是边界, v1.9)；「相邻段序数」的会话内清单过滤 `status="stitched"` 壳 (实现 `_session_episodes`, 壳非产出单元、不得占用「第 n 段」序数, v1.9) → [首帧截图] → [末帧截图] (judge profile 须 `supports_vision`, 3.1.4 vision 逐阶段表) → [标注结果]。产物增量：`VerificationResult` 增 `additive` 字段 `defects` (4.2, 非 stream 恒 ())；`_meta.verification` 在 stream 模式下携带恒在 `defects` 键 (无缺陷 = [], 6.3)；缺陷摘要随 `verify.verdict` 事件 payload (受 `trace.content` 分级, 7.4, S31)。 `verdict = "fail"` 而 `defects` 为空数组 ⇒ 代码侧归一化为一条默认 `label_mismatch` 缺陷 (S7——修复路由建立在缺陷表之上, fail 必有路由依据)。

上下文预算装填 (v1.11)。评审 profile 声明 `context_window` 时按上下文预算装填评审调用 (未声明 = 预算关闭, 行为与 v1.10 一致；预算/估算/校准机制见 3.9)：记录侧可裁份额 = 单记录 UI 树渲染动态封顶 (`min(input.ui_tree_max_chars`, 预算折算字符), `marker` 与绝对上限语义同 3.13.4) 与序列 [动作序列] 步骤行块的「首末步恒保留、丢中段整行 + 原位标记」裁剪 (V9)。**多评审团共用单 prompt** (本节多评审团行与 stream 序列评审均为一次构建广播全团) ⇒ 记录侧份额按评审团**最小 `input_budget`** 装填 (V25②；对照：quality pairwise 逐(对, 评审)各自构建、按本评审预算装填, 3.4.3)。**不可裁剪动态块 (V25③)**：[标注结果] 的标注 JSON 为 per-record 语义资产——计入 **est**、**永不裁剪**；全部可裁份额耗尽仍超 → 该记录记 `context_overflow` 入 `rejects` (V10, 7.6)。逐裁剪点计入 `report.budget.truncations` (6.4)。

3.7.3 失败策略与修复环

策略	行为
<code>verify.policy = "drop"</code> (默认)	fail ⇒ <code>status="dropped_verify"</code> , 批评意见摘要入 <code>_meta.verification</code> 与 <code>rejects</code> 通道。
<code>verify.policy = "repair"</code>	fail ⇒ 将批评意见追加进标注提示词 ([上一版标注] ... [审核意见] ... 请修正后重新输出), M5 重标注、M8 重校验、M7 重评审；最多 <code>verify.max_repair_rounds</code> (默认 1) 轮, 仍 fail 按 drop 处理。评审轮数记入 <code>_meta.verification.rounds</code> (含首评, 一次通过 =1; 修复后复评 =2), 各轮意见按序累积于 <code>VerificationResult.critiques</code> (4.2), 实例见 3.7.4。

stream 修复路由：两阶段批级成员手术 (v1.8, S8/S31)。 `policy = "repair"` 下序列信封的修复不止重标注——按缺陷表 (3.7.2) 路由三类动作：**标签重标** (`label_mismatch`：常规批评意见回喂重标注)、**成员收缩** (`off_task_members`) 与 **成员回收** (`missing_head` / `missing_tail` / `missing_members`)。为保证并发 `gather` 下的确定性 (相邻 episode 争抢同一噪声帧、multi 兄弟互撕共享成员集)，每轮修复为**两阶段批级结构** (`classify` 扇出「先同步后并发」先例)：

- 1. **并发评审全部待审 episode**；
- 2. **同步按批位置序执行全部成员手术** (「先到先得」变为确定性「位次得」)——**收缩**：`defect.members` 指认帧 `absorbed` → `dropped_noise` + duck-typed `off_task_member` 标 (`rejects` 归因 `stage="verify"`、`reason="off_task_member"`, 3.11.2)；**回收** (三级判定)：同 `session_id` 的批内 `dropped_noise` 噪声池帧经 `segment.judge_window` 直调复裁 (3.14.3；`relation ∈ {continues, advances}`) 即回收 `dropped_noise` → `absorbed`、按序键插回 `members`) → 缺帧在相邻 episode 手中：**只标记、不跨段夺帧** (`boundary_flags` 计数) → 无处可寻：缺陷条目增顶层键 `suspected = "capture_gap"` (代码侧标注, 非 LLM 输出——`detail` 在 Schema 中为字符串, 故 `suspected` 以

兄弟键落在缺陷条目上；所属会话曾被 batch_size 硬切的帧改标 "session_split" ——判定依据即 `_meta.stream.session_split`，S21，3.10.3)；

3. 并发接缝重摘取：手术触点经 `extract.extract_transition` 直调重摘 (3.15.3；1—2 次/手术，重建 Transition 带 `detail.reseamed = true` 溯源)；
4. 同步重建：record 以新成员集重建 (替换 `members`；序列 id 不重算，3.14.4 拼装行)、transitions 重编号——`Transition.index` 恒 = 元组下标、不变量 `len(transitions) = len(members) - 1` 恒真 (4.2，S31)；
5. 并发重标注与复审：`annotate_record(record, ctx, AnnotatePromptOptions(transitions=重建值, ...))` (3.5.2 transitions 取值段) → 下一轮评审。

帧产物同步 (v1.12)：第 4 步同步重建之后、第 5 步重标注之前，对本轮执行了成员手术的 episode 同步两个帧产物 dict (`member_classifications / member_annotations`，4.1) ——手术改了成员集，帧产物必须随成员集走，否则 `members[]` 落盘时出现无主条目或缺帧：

- **收缩删键**：不再属于 `record.members` 的成员 id 从两 dict 删键，含值为 `None` 的 `failed` 占位键 (不留无主条目)；仅当对应 dict 非 `None` 时操作；dict 对象本身从不更换 (扇出克隆按引用共享同一 dict 的前提，4.3 补注)。
- **回收补跑** (懒加载，幂等只补缺位)：新入 `record.members` 且键缺位的成员先经 `classify.classify_frames([member], ctx)` 单元素调用补帧分类 (`frame.classify.enabled` $\wedge$ dict 非 `None` 时；窗口失败在 `classify_frames` 内落 `fallback_class`，契约上不抛记录级异常，3.13.7)，再经 `annotate.annotate_member(member, ctx, label)` 补帧标注 (`frame.annotate.enabled` $\wedge$ dict 非 `None` 时；帧标注补跑必须在帧分类补跑落键之后——帧类取新鲜判决；帧类视图 `enabled=false` $\Rightarrow$ 跳过类不占键 (emitter 按缺键推导 `skipped`)；`frame.classify` 关 $\Rightarrow$ `label=None` 走全局指令；不可修复返回 `None` $\Rightarrow$ 占键 `None`，同 3.5.5 失败语义)。并发形态与记录级隔离镜像接缝重摘取 (`gather + dead` 集合)。
- **dict None 全程不触碰**：dict 为 `None` = 帧 pass 未运行 (降格会话 / 帧粒度关闭 / 非首标签)，收缩与补跑均不触碰——降格语义保持、永不无中生有。
- **克隆无同步分支**：克隆信封永不手术 (既有 S8——multi 扇出克隆的 `membership` 类手术只标记，仅原信封可执行)，故帧产物同步不需要克隆分支；懒加载直调面由三个扩为四个 (`classify.classify_frames` 为算子间导入白名单第四向；`annotate_member` 并入既有 `annotate` 修复面族——CONTRACTS §1.1)。

wrong_stitch 路由 (v1.9，独立分支)：只标记、不拆线——自动拆线手术是 v1.9 非目标 (8.1)，本缺陷不进上述三类手术路由，尤其不得落入 `missing\_*` 的噪声池回收扫描 (错缝的修复方向是移除碎片而非补帧，回收扫描会反向加重错缝)；`repair` 轮内不为其执行任何成员手术 (重标注亦不能修复错缝)，持续 fail 按 drop 收尾 (`dropped_verify`——错缝线索 `fail-closed` 不入主输出)；计数入 `verify.defects.wrong_stitch` (6.4)。成员手术的回收扫描语义不变 (异线索 `absorbed` 帧按既有 D5 邻域判定已是 `neighbor mark-only`，缝合不改变其结论)。

修复轮数计入 `verify.max_repair_rounds` (含首评，与本节非 stream 语义一致)。状态改写授权：手术在 `absorbed` 与 `dropped_noise` 间双向改写成员信封状态——4.3 契约 ②b 的 M7 修复路径豁免 (契约①的唯一一反向豁免)，**禁止翻回 active** (帧与其 episode 不得双写主输出)。其余裁决：multi 扇出克隆兄弟的 `membership` 类手术只标记——仅原信封 (首标签) 可执行 (S8，3.13.4 multi $\times$ episode 行)；多评审团下 `defects` = 投 fail 的 judge 的并集，按 (kind 枚举序, position, members) 确定性去重排序，同成员的互斥手术取先序 (S31)；修复后不重打分——沿用修复前质量分 + `_meta.stream.repaired = true` 标记 (6.3；multi 下亦用于消歧同 id 兄弟行)。观测面 (M7 属主，`report.stream.verify` 子块，6.4)：`verify.membership_repairs` (执行的手术数)、`verify.boundary_flags` (只标记的边界判定数)、`verify.defects.<kind>` (逐缺陷类型计数)。

修复路径与上下文预算的交互 (v1.11)。① **升级触发 (V21)**：`verdict = "fail"  $\wedge$  policy = "repair"` 是修复重标注质量阶梯换挡 (关键帧减半 + 分辨率上探 $\leq$ `max_image_px`，3.5.2 v1.11 段) 的唯一触发面——升级只发生在修复路径、每记录 $\leq$ `verify.max_repair_rounds` 次，阶梯参数经 `AnnotatePromptOptions` 的 `k_eff / image_px` 两字段传入 (实现为 `dataclasses.replace(opts, ...)`，F3)。② **回收复裁的静态保证 (V9/F14)**：成员回收的固定 [前成员, 候选, 后成员] 三帧复裁窗 (直调 `segment.judge_window`) 由 M1 预算护栏静态覆盖——`w_min` 护栏下限 `floor = 3` 仅当 `verify.enabled  $\wedge$  verify.policy = "repair"  $\wedge$  segment.enabled` (`policy = "drop"` 不构造复裁窗、不做三帧静态要求)，`w_min < floor` $\rightarrow$ `CONFIG_ERROR` (3.1.4、3.9)，由此在先验计价下保证修复路径运行期复裁不 `context_overflow` (v1.11 审计修订：校准值超先验的个案降为复裁失败的既有 `mark-only` 处置——记录级，永不 run 级；reactive-400 形态在该吞点补喂熔断恰一次，7.6 熔断矩阵)。③ 缺陷词表与预算无交互：`wrong_stitch` 的无条件闭合同义语义不变 (3.7.2 四处同步闭集，`stitch off` 亦在场)。

背书：LLM-as-a-Judge 的可靠性、偏差类型 (位置/冗长/自增强) 与缓解手段出自 Zheng et al. (NeurIPS 2023) [20]；「批评意见回喂原模型迭代修正」是 Self-Refine (NeurIPS 2023) 的 FEEDBACK $\rightarrow$ REFINE 循环 [21]，有界轮数与其停机设定一致；批评-修订两阶段结构同 Constitutional AI [22]。GUI-360 以同构的「LLM 质量过滤」环节筛选 GUI 轨迹数据 [14]。

3.7.4 输入 / 输出示例

沿用全文文本模态贯穿示例 (输入法中文指令意图标注工程，`input.text_field = "instruction"`)。配置：`verify.enabled = true`、`verify.llm = "judge"`、`verify.policy = "repair"`、`verify.max_repair_rounds = 1` (默认)、`verify.extra_criteria = ""` (默认，未追加

维度)。记录 `id = "1cda030abc565f17"`，原始行 `{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}`，M5 首版标注（已过用户 Schema）为 `{"intent": "writing_assist", "topic": "请假条写作", "difficulty": "easy"}`。

① 首次评审调用（第 1 轮）

按 3.7.2 模板组装，judge 走 `[llm.judge]` profile（`claude-sonnet-5`，独立于标注模型）：

```
system: 你是标注质量审核员。给定任务指令、原始数据与标注结果，独立判断标注是否合格。
        评审维度：① 是否遵循任务指令 ② 与原始数据的事实一致性 ③ 字段语义是否正确填写
        先逐维度给出简短意见，再给结论。 # extra_criteria 为空，无追加行
user:   【任务指令】你是输入法中文指令的意图标注员。判断每条用户指令的意图类别（intent）、
        主题（topic）与完成难度（difficulty）。
        【原始数据】帮我写一条请假条，明天上午要去医院 # 文本模式 = record.text
        【标注结果】{"intent": "writing_assist", "topic": "请假条写作", "difficulty": "easy"}
```

judge 响应（经 M8 按评审内部 Schema 校验合法）：

```
{"critiques": [{"aspect": "字段语义",
                 "opinion": "difficulty 标为 easy，但该指令涉及正式文书格式与措辞得体性，应为 medium"}],
 "verdict": "fail"}
```

② 修复轮：批评意见回喂 M5

`verdict = "fail"` 且 `policy = "repair"`、已用修复轮数 `0 < max_repair_rounds = 1`，触发修复。按 3.7.3 格式将下述片段追加进 3.5.2 组装的标注提示词末尾（system / few-shot / 当前记录各段与首次标注调用逐字相同）：

```
【上一版标注】{"intent": "writing_assist", "topic": "请假条写作", "difficulty": "easy"}
【审核意见】字段语义：difficulty 标为 easy，但该指令涉及正式文书格式与措辞得体性，应为 medium
请修正后重新输出
```

M5（`[llm.default]`，`qwen2.5-vl-72b-instruct`）重新输出，经 M8 通过用户 Schema（L0 直出即合法，`attempts = 1`）：

```
{"intent": "writing_assist", "topic": "请假条写作", "difficulty": "medium"}
```

③ 二次评审（第 2 轮）

以修正版标注按 ① 相同模板重新组装（仅 `【标注结果】` 段更换），judge 响应：

```
{"critiques": [{"aspect": "字段语义",
                 "opinion": "difficulty = medium 与正式文书的格式及措辞要求相符，intent 与 topic 填写正确"}],
 "verdict": "pass"}
```

`verdict = "pass"`，记录保持 `status = "active"`，流转至 M11 写出。

④ 最终结果对象

`PipelineItem.verification`（4.2 `VerificationResult`；`critiques` 为各评审轮意见按轮次顺序累积）：

```
VerificationResult(
    verdict = "pass",
    rounds = 2, # 评审轮数：首评 fail + 修复后复评 pass；一次通过时为 1（对照 6.3 示例）
    critiques = ({ "aspect": "字段语义",
                   "opinion": "difficulty 标为 easy，但该指令涉及正式文书格式与措辞得体性，应为 medium"},
                 {"aspect": "字段语义",
                   "opinion": "difficulty = medium 与正式文书的格式及措辞要求相符，intent 与 topic 填写正确"}))
```

主输出行中 `_meta` 的相关片段（形态见 6.3）：

```
"_meta": {
  "id": "1cda030abc565f17", ...,
  "annotation": {"model": "qwen2.5-vl-72b-instruct", "attempts": 1}, // 修复轮的 M5 输出，结构一次合法
  "verification": {"verdict": "pass", "rounds": 2}
}
```

对照分支：若二次评审仍 fail，此时已达 `max_repair_rounds`（默认 1），按 drop 收尾——`status = "dropped_verify"`，批评意见摘要入 `_meta.verification` 与 rejects 通道（3.7.3）；rejects 行（`output.rejects = "refs"`）不含数据内容本体。

3.7.5 直装序列评审：判决形（v1.13）

序列信封自 v1.13 起有两个来源：M14 分段产出的 episode（stream 模式）与 M6 时间流生成产出的直装序列（`generate_stream.enabled`，segment 关闭，3.6.5）。前者由 stage 层的流式驱动器承载、走 3.7.2 的缺陷词表变体；后者走经典路径，但不能照搬非序列模板——非流模板的 [原始数据] 段对 `kind = "sequence"` 的记录无内容可渲染（序列 Record 的 `text/raw/ui_tree/image` 恒 None，4.1），且缺陷词表变体的 `defects` 键被经典路径的 `VERDICT_SCHEMA` 禁止。裁决为判决形：调用方按「流式驱动器在场与否」选择模板变体，两条路径互不干扰。

装配面（选项对象，2026-08-14 收参）：

```
@dataclasses.dataclass(frozen=True)
class VerifyPromptOptions:
    """一次评审提示词的全部可选装配项；缺省实例 = v1.7 之前的单记录经典调用形。"""
    label: str | None = None          # 类标签；None = 取全局指令与准则（v1.7）
    transitions: tuple | None = None  # 序列步表；None = 整段省略 [动作序列]（v1.8）
    boundary_margin: str = ""        # [边界余量] 段正文（驱动器预渲染）
    fragment_structure: str = ""     # [片段结构] 段正文；空串 = 整段省略（v1.9）
    fit: "_PromptFit | None" = None  # 面板最小预算装填状态；None = 预算关（v1.11）
    verdict_form: bool = False       # 序列走 §10.16 判决形变体（v1.13 直装序列）

def build_verify_prompt(record, output, cfg,
                       options: VerifyPromptOptions | None = None) -> PromptBundle:
    """options.verdict_form = True 且 record.kind == "sequence" => §10.16 判决形序列变体；
    其余组合逐字节维持既有分支（流式驱动器从不置该字段）。"""

def verify_verdict_sequence_system_text(extra_criteria: str) -> str:
    """判决形 system 段：三维评审 + 结论指令 + VERDICT_SCHEMA 结构句；无缺陷词表。"""
```

要点（规格与理由）：

- **选择判据 = 驱动器在场**：经典路径遇 `record.kind == "sequence"` 时置 `options.verdict_form = True`（流式驱动器在场时序列永不进入该函数）；`schema` 恒为既有 `VERDICT_SCHEMA`，与模板天然配对——修掉「驱动器门 = `segment.enabled` 而模板门 = `kind`」的错配。
- **模板形态**（verbatim 冻结于 CONTRACTS §10.16）：`system` = 判决指令（三维评审：遵循任务指令 / 与成员帧摘要证据的事实一致性 / 字段语义）+ 类有效 `extra_criteria`（为空整行省略，非流规则同款）+ 「先意见后结论」句 + `VERDICT_SCHEMA` 结构句；`user` = [任务指令] → [成员帧摘要] → [标注结果] 三段。**无缺陷表、无 [边界余量]、无 [片段结构]、无截图段**（直装序列为 text 模态）。
- **成员摘要渲染**：逐成员 {m}. {`frame_digest(member, 400)`}（m 1 基、成员序），总量受 `input.ui_tree_max_chars` 约束——**首末行恒保留、中段整行丢弃**并以 `...(truncated N members)` 标记闭合（镜像 M5 的序列摘要渲染；算子间不互导，M7 自持同式副本）。
- **预算装填**：`fit` 非 None 时成员摘要块是**唯一可裁槽位**（edges 裁剪计 `report.budget.truncations`），[标注结果] 与任务指令是记录级语义资产恒计不裁（V25③）；不可裁地板仍超预算 => `fit.overflow`（V10——调用方拒绝，请求从不发出）。
- **失败与修复**：修复 = 既有 policy 重标注（透传同一 `label` => 自然穿按序列类标注 Schema，3.5.2）；持续 fail => `dropped_verify`——对合成数据而言这就是**拒绝采样**：淘汰不合格序列，**不改真值**（工件行照常保留该序列的帧与 truth，主输出少一行，6.5）。
- **零改动面（显式声明）**：`VerificationResult.defects` 在本形态恒空、emitter 的 `_verification_block` `defects` 门**零改动**（该键仅 stream 模式在场，6.3）；缺陷词表六值闭集、四处同步登记、流式驱动器与成员手术路径**全部零改动**。

3.8 M8 结构引擎 schema-engine

3.8.1 职责与边界

做：持有经预校验的用户 Schema 与各内部小 Schema（裁决、评分、评审、生成输出；v1.7 增分类 `classification_schema(class_names, assignment, max_labels, with_reason)`——按 `classify.assignment` 二态、类名词表以 `enum` 硬约束，关键字集 $\subseteq$ 既有内部 Schema 关键字集且无 `uniqueItems`：该关键字会被 OpenAI strict 模式与部分约束解码网关硬拒，重复标签由 `classify` 代码在 M8 验证后确定性归一化，全文见 3.13.3；v1.8 增三项——分段窗口 `segment_window_schema(frame_count, with_reason)`（M14，全文见 3.14.3）、动作 `action_schema()`（M15，11 值动作词表 `enum` 硬约束，全文见 3.15.3）、stream 缺陷评审 `defect_verdict_schema()`（M7 stream 分支，三顶键 {critiques, defects, verdict} + 缺陷词表——v1.8 五值，v1.9 起六值 (+ `wrong_stitch`, 3.7.2)，全文见 3.7.2)；v1.9 增一项——缝合判定 `stitch_schema()`（M16，五键 {verdict, thread_ref, task_name, reason, confidence} 全 required、thread_ref 可空联合，全文见 3.16.3)；v1.12 增一项——帧级批量判决 `frame_classify_schema(names, n)`（M13 帧粒度，单键 {"labels": [enum×n]}、minItems = maxItems = n 钉死窗内成员数组长度、帧标签词表 `enum` 硬约束，同族无 `uniqueItems`（帧标签本就允许重复）；长度/索引对齐后校验在代码侧（first-wins，缺项 $\Rightarrow$ 该帧 `fallback_class`），全文见 3.13.7)。四者逐字 JSON 冻结于 CONTRACTS §10.7，规则同族：关键字集 $\subseteq$ 既有内部 Schema 关键字集、同样无 `uniqueItems`（重复 index / 标签由调用方代码在 M8 验证后确定性收窄——3.14.4 的 first-wins 建表、3.13.4 的归一化行）；可选性一律以可空联合 `type` 数组 ["array", "null"] / ["string", "null"] 表达、全键 required（OpenAI strict 模式硬拒可选属性，L0 无条件透传 Schema）；minItems = maxItems 钉死窗口数组长度（`judgment_schema` 同款）。`defect_verdict_schema` 与既有评审 Schema 并存——非 stream 评审路径继续用后者（回归锚，S7）。四者与其余内部 Schema 同级：不计入 `report.schema_engine.resolved_at`、不经过 L2.5)；v1.13 增两项——时间流生成的蓝图 `plan_schema(names, length, cover_all=False)`（M6，单键 {"steps": [{frame_class: enum, brief: string}]}、minItems = maxItems = length 钉死步数、帧类名词表 `enum` 硬约束，同族无 `uniqueItems`（同一帧类在一条序列里本就可重复出现）；v1.14 增第三入参 `cover_all`——True 时在 steps 数组对象上追加 `allOf` + 逐名一项 `contains`（子模式形如 {"properties": {"frame_class": {"const": <名>}}, "required": ["frame_class"]}，按传入名集序；一个 Schema 对象只有一个 `contains` 键位，故多类分 `allOf` 支)，与 `enum` 合成帧类构成的恰等约束（`enum` 给 $\subseteq$ 、`contains` 给 $\supseteq$ ，3.6.5 档位构成行）；names 取值由调用方决定（档位表在场即存档内子集），构造器零感知；缺省 False 的输出与 v1.13 逐字节一致。全文见 3.6.5) 与帧实现 `realize_schema(step_schemas)`（M6，逐位包装器：单键 {"frames": [...]}，第 i 位服从蓝图第 i 步帧类的用户生成 Schema——纯文本帧位取 {"type": "string"}；以 draft 2020-12 原生关键字 `prefixItems` 表达逐位约束（`jsonschema` $\geq$ 4.21 直接可校验、无翻译层）、"items": false 封尾禁超长、minItems = maxItems 再钉一次长度）；两者逐字 JSON 冻结于 CONTRACTS §10.7，与前述内部 Schema 同族同级。用户生成 Schema 的 L0 待遇（v1.13）：`realize_schema` 的逐位子模式是用户手写的帧类生成 Schema（`[frame.class.<name>.generate].schema_*`，M1 元校验），包装后随 L0 原样透传——不做关键字白名单 lint（与 `output.schema` 今日同款暴露面）；某些 strict 路由拒 `prefixItems` 的排障面是配置级 `supports_structured_output = false`，不新增调用级参数）；提供「LLM 调用 $\rightarrow$ 合法 JSON 对象」的唯一入口 `complete_validated()`，内部实现四层结构保证；统计各层修复命中率。不做：不组装业务提示词（调用方传入完整 prompt）；不解释业务语义；不放行任何未通过校验的对象——这是它对全系统的硬契约。

3.8.2 四层保证与修复环

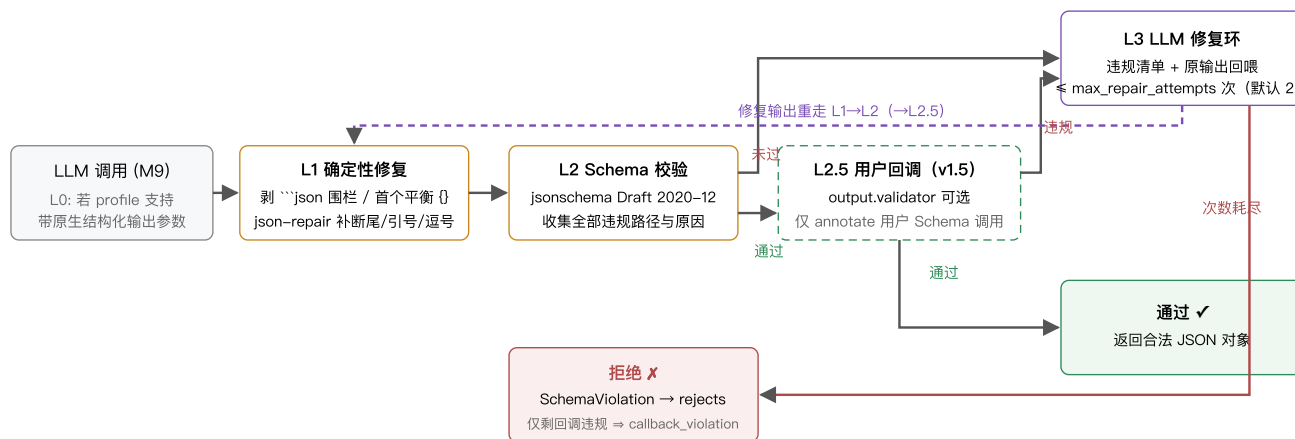


图 3-3 结构引擎四层保证。任何写入主输出的对象必然经过 L2 通过分支。

层	精确行为
L0	profile <code>supports_structured_output=true</code> 时：OpenAI 兼容 provider 传 <code>response_format={"type":"json_schema", "json_schema":{"...strict:true}}</code> ；Anthropic provider 以单工具 <code>tool_choice</code> 强制工具调用、Schema 作为工具入参。L0 只是「使 L1/L3 少触发」的优化，不豁免 L2——供应商实现存在覆盖缺口（JSONSchemaBench 实测各引擎均有不支持的 Schema 特性 [24]），校验永远执行。
L1	顺序执行：① 剥离 Markdown 代码围栏；② 取首个花括号平衡子串；③ <code>json_repair.loads()</code> （工业库，处理截断/单引号/尾逗号/裸换行 [8]）。全部失败 $\Rightarrow$ 直接进 L3。L1 为纯函数，无副作用、可单测穷举。
L2	<code>Draft202012Validator.iter_errors()</code> 收集全部违规（非首个），每条含 JSON Pointer 路径、期望与实际。通过 $\Rightarrow$ 返回；未通过 $\Rightarrow$ L3。违规渲染的两个显名分支（渲染文本进 L3 修复清单，也进 <code>StageError</code> / <code>rejects</code> 错误报告面，故统一英文）： <code>enum</code> 违规按「期望/实际」措辞自行渲染（3.8.4 逐字实例）； <code>contains</code> 违规（v1.14）点名缺失的帧类——蓝图覆盖约束的 <code>contains</code> 子模式形如 {"properties": {"frame_class": {"const": <名>}}, ...}，直接取该 const 值渲染为 <code>steps: missing required frame_class "<名>"</code> （不是该形时——用户 Schema 也可以写 <code>contains</code> ——回落 <code>jsonschema</code> 原始消息）。理由：L0 关闭的端点上，L3 修复提示不点名缺失帧类则修复指导性趋零（裸数组 repr 无信息量）；帧类名是配置量非数据内容，值-free 纪律不变。其余关键字直接携带 <code>jsonschema</code> 原始消息。

L2.5 (v1.5, 可选)	<code>output.validator</code> 配置时、且仅对用户 Schema 调用: L2 通过后执行用户回调 <code>fn(obj, record)</code> 。返回非空违规列表 $\Rightarrow$ 违规以 <code>(validator) < 消息></code> 形式并入违规清单、与 Schema 违规同路进入 L3 修复环 (回调意见回喂模型自我修正——回调既是门卫也是修复环的教练); 返回空 $\Rightarrow$ 通过。L3 每轮修复输出重走 L1 $\rightarrow$ L2 $\rightarrow$ L2.5。预算耗尽且剩余违规全部来自回调 $\Rightarrow$ <code>SchemaViolation(callback_only=True)</code> , 记录 <code>kind = callback_violation</code> (7.6), 否则仍为 <code>schema_violation</code> 。回调抛异常不吞: 向上传播、按记录级 <code>internal_error</code> 收敛 (3.5.3)。内部 Schema (裁决/评分/评审/生成/分类 (v1.7) /分段窗口/动作/缺陷评审 (v1.8) /缝合判定 (v1.9)) 不经过 L2.5。
L3	修复提示词 = 单条 user 消息, 按 [原始输出] / [违规清单] 分节标签组织, 末尾指令「只输出修正后的 JSON」(逐字实例见 3.8.4)。使用 <code>output.repair_llm</code> (默认同调用方 profile)。每次修复输出重走 L1 $\rightarrow$ L2。尝试次数耗尽 $\Rightarrow$ 抛 <code>SchemaViolation(errors, raw_last_output)</code> 。修复调用计入 token 计量, 命中层级分布计入报告 (<code>report.schema_engine.resolved_at = {l0_or_clean, l1, l3_1, l3_2, rejected}</code>)。上下文预算交互 (v1.11, V25①): 修复调用经 M9 终检 (3.9) 抛出 <code>ContextOverflowError(phase="precheck")</code> 时捕获并记该轮修复失败——修复 prompt 恒定 $\Rightarrow$ 余轮必然同败, 短路至预算耗尽; <code>reject</code> 归因维持既有 <code>schema_violation</code> / <code>callback_violation</code> (不新增 <code>reject</code> 值、不计 <code>report.budget.overflow_records</code> , 7.6 注); [原始输出] 修复原文永不截断——截断即破坏修复语义: 该吞点即异常终局—— <code>reactive-400</code> 形态在此共享 <code>budget.feed_reactive_terminal</code> 补喂熔断恰一次 (7.6 熔断矩阵: <code>_breaker_fed</code> duck 标防重喂, <code>precheck/200</code> 形态永不喂, v1.11 审计修订)。

帧级两类调用的路由声明 (v1.12): 帧粒度引入的两类 LLM 调用都走本引擎既有能力, `complete_validated` 与 `validate_only` 的显式 schema 参数路径零改动——

- 帧分类 (M13 批量判决, 3.13.7): 传入模块级构造器 `frame_classify_schema(names, n)` 产出的内部 Schema, 待遇与裁决/评分/评审等内部 Schema 完全同族——L0–L3 全在、不经过 L2.5、不计入 `report.schema_engine.resolved_at`。
- 帧标注 (M5 逐帧标注, 3.5.5): 传入用户声明的帧级 Schema `cfg.frame_schema` (显式 schema 参数, 裁决·帧 Schema 显式路由)——虽为用户 Schema 的同胞 (M1 元校验 + few-shot 干跑, 3.1.4), 但按内部 Schema 待遇路由: L0–L3 四层全在、无 L2.5 (`output.validator` 仅约束序列级用户 Schema 调用; 帧级回调列 8.4 演进候选)、不计 `resolved_at`——保住 6.4 恒等式「resolved_at 加总 = 进入 M5 的记录数」不被帧调用污染。
- 写前兜底 (M11, 3.11.2): `emitter` 对每个非 null 帧标注对象跑 `validate_only(obj, schema=cfg.frame_schema)`——通过 $\Rightarrow$ `status="annotated"`, 不通过 $\Rightarrow$ 翻 "failed" + annotation 置 null + 计数, 非法帧对象永不落盘 (主输出 `validate_only` 终检的帧级镜像)。

显式待遇参数与三类路由声明 (v1.13, 裁决·M8 显式待遇参数): v1.12 的路由把「显式 schema 参数」与「内部 Schema 待遇」绑死, 导致 v1.13 的按序列类标注 Schema 无处安放——它是用户 Schema 的另一份实例 (记录级标注调用), 却必须显式传参。裁决: `complete_validated` 增待遇门 `user_treatment: bool | None = None` 把待遇与传参方式解耦 (2026-08-14 代码规则整改后它是 `CallScope` 的一个字段, 与 `record_ids` / `batch_no` / `record` 同乘一个参数对象; 语义逐字不变) ——

路由	schema 参数	scope.user_treatment	L2.5	resolved_at 记账
用户 Schema (全局 <code>output.schema</code>)	None (引擎持有)	None $\Rightarrow$ 推断为真	✓ (配置了 <code>output.validator</code> 时)	✓
按序列类标注 Schema (v1.13)	显式传该类 Schema	True	✓	✓
帧级 Schema 与全部内部 Schema	显式传	None $\Rightarrow$ 推断为假 (帧级/内部调用不显式传 True)	✗	✗

None 即现行 `schema is None` 推断 $\Rightarrow$ 既有全部调用点零改动; `stats` 的口径句相应重述为「用户待遇族调用」而非「schema 参数为 None 的调用」, §6.4 恒等式重述为「resolved_at 加总 = 进入 M5 的记录级标注调用数」(按类 Schema 的记录级标注照常计入, 帧级标注仍不计——恒等式不被帧调用污染, 6.4)。

3.8.3 API


```

@dataclass(frozen=True)
class CallScope:
    """一次调用的记账与追踪范围（2026-08-14 收参：原四个关键字入参的参数对象形，
    字段名与语义逐项不变）。"""
    record_ids: tuple[str, ...] = () # 本次调用覆盖的记录 id，仅用于 trace 事件
    batch_no: int = 0 # 批次号，仅用于 trace 事件与日志 extra
    record: Mapping | None = None # L2.5 回调第二入参 (Record.raw)，无则 None
    user_treatment: bool | None = None # 显式待遇门；None ⇒ 按 schema is None 推断

class SchemaEngine:
    def __init__(self, user_schema: dict, llm: LLMClient, cfg: OutputConfig): ...
    async def complete_validated(self, profile: str, prompt: PromptBundle,
                                schema: dict | None = None, *,
                                scope: CallScope = CallScope()) -> dict:
        """schema=None 时用用户 Schema；内部 Schema（裁决/评分/评审/生成/分类（v1.7）/
        分段窗口/动作/缺陷评审（v1.8）/缝合判定（v1.9）/帧级判决（v1.12）/
        蓝图与帧实现（v1.13））由各 Stage 传入。
        v1.13 scope.user_treatment: None = 按 schema is None 推断（既有调用点零改动）；
        True = 用户待遇（计 resolved_at + 启 L2.5）—按序列类标注 Schema 即此形；
        False = 内部待遇。成功返回已通过 L2 的 dict；失败抛 SchemaViolation。"""
    def validate_only(self, obj: dict, schema: dict | None = None) -> list[str]:
        """M1 校验 few-shot 示例输出、M11 写出前终检用；v1.12: M11 帧标注写前
        校验经显式 schema=cfg.frame_schema 走同一入口（3.8.2 路由声明）。"""

```

设计考量：“Let Me Speak Freely?” (arXiv:2408.02442) 报告了严格格式约束可能损失推理质量 [25]。缓解按各内部 Schema 的实际字段序落地：评审输出的 `critiques` 置于 `verdict` 之前（3.7.2「先意见后结论」）、pointwise 打分的 `reason` 置于 `score` 之前（3.4.4「先给两句理由再给整数分」），让模型先推理后作答。例外是成对裁决：字段序为 `criterion` → `winner` → `reason`，`reason` 在结论之后且仅当 `quality.judgment_reasons` 生效时才要求（3.4.3）——它的用途是落入 trace 供 rubric 优化（7.5），不承担「先推理后作答」的缓解职责（生成输出 `{“samples”: [...]}` 则不含自由文本字段）。用户 Schema 若需同类缓解，可自行加 `reasoning` 字段并在下游忽略。背书：「Schema 约束生成 + 机器校验 + 修复重试」为工业标准三件套：OpenAI Structured Outputs [7]、约束解码框架 Outlines [23]、JSONSchemaBench 对 6 家引擎的评测 [24]、instructor 的 validation-retry 循环与 json-repair 库 [8]。四层纵深（供应商能力不被信任、校验不可豁免）是对 [24] 所示覆盖缺口的直接工程回应。

3.8.4 输入 / 输出示例

以贯穿示例的文本模式工程为例：输入行 `{“instruction”: “帮我写一条请假条，明天上午要去医院”, “source”: “ime-log”, “ts”: “2026-06-30T10:12:00Z”}`（`input.text_field = “instruction”`，记录 `id = 1cda030abc565f17`）。M5 组装标注提示词后调用 `complete_validated(profile=“default”, prompt, user_schema)`。用户 Schema（`output.schema_inline`）：

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "intent": {"type": "string",
      "enum": ["writing_assist", "qa", "translation", "chitchat", "other"]},
    "topic": {"type": "string"},
    "difficulty": {"type": "string", "enum": ["easy", "medium", "hard"]}
  },
  "required": ["intent", "topic", "difficulty"],
  "additionalProperties": false
}

```

① **L0（本例未启用）**：本走查假设该 profile 未声明 `supports_structured_output`（默认 false，5.1），M9 `complete()` 忽略 `response_schema` 参数（3.9.2），不注入任何原生结构化输出参数——结构保证完全落在 L1–L3。

② **LLM 原始输出**（`LLMResponse.text`，原文照贴；叠加三个问题：Markdown 围栏、尾逗号、`intent` 取值在枚举之外）：

```

```json
{
 "intent": "writing",
 "topic": "请假条写作",
 "difficulty": "easy",
}
```

```

③ **L1 确定性修复**：按 3.8.2 顺序执行——① 剥离 ````json` 围栏；② 取首个花括号平衡子串；③ `json_repair.loads()` 修掉 `"easy"` 后的尾逗号。解析成功，得到对象 `{“intent”: “writing”, “topic”: “请假条写作”, “difficulty”: “easy”}`。L1 只保证「可解析」、不看 Schema——枚举

违规原样进入 L2。

④ L2 校验：`Draft202012Validator.iter_errors()` 收集全部违规，本例共 1 条：

```
JSON Pointer: /intent
期望：枚举 ["writing_assist", "qa", "translation", "chitchat", "other"] 之一
实际："writing"
(jsonschema 原始消息：'writing' is not one of ['writing_assist', 'qa',
'translation', 'chitchat', 'other'])
```

违规清单非空 ⇒ 进入 L3。

⑤ L3 修复调用（第 1 次，预算 `output.max_repair_attempts = 2`）：本工程未配置 `output.repair_llm` ⇒ 使用调用方 `profile default`。修复提示词按 3.8.2 逐字组装 = 原始输出全文 + 违规清单 + 「只输出修正后的 JSON」，作为单条 user 消息发出：

```
[原始输出]
```json
{
 "intent": "writing",
 "topic": "请假条写作",
 "difficulty": "easy",
}
```

[违规清单]

1. /intent: expected one of enum ["writing\_assist", "qa", "translation", "chitchat", "other"], got "writing"

只输出修正后的 JSON。

修复响应：

```
{"intent": "writing_assist", "topic": "请假条写作", "difficulty": "easy"}
```

\*\*\*⑥ 重走 L1→L2：\*\*\*L1 无围栏可剥、直接解析成功；L2 ``iter_errors()`` 返回空清单 ⇒ 通过。``complete_validated()`` 返回该对象，M5 写入 ``item.annotation``：``Annotation.attempts = 2`` (= 1 + 1 次 L3 修复，4.2 定义)，首次调用与修复调用的 token 均计入 ``Annotation.usage`` 与 `profile` 计量 (3.9.3)；本次解决计入 ``report.schema_engine.resolved_at`` 的 ``l3_1`` 桶（首次 L3 修复即通过），主输出 ``_meta.annotation.attempts = 2``。

层	输入	动作	输出
L0	<code>`supports_structured_output = false`</code>	不注入原生结构化输出参数	提示词原样发出，保证责任交给 L1-L3
L1	带围栏 + 尾逗号的原始文本	剥围栏 → 平衡花括号子串 → <code>`json_repair.loads()`</code>	可解析对象 ( <code>`intent`</code> 仍为 <code>`"writing"`</code> )
L2 (第 1 次)	L1 产物	<code>`iter_errors()`</code> 全量收集违规	1 条违规 ( <code>`/intent`</code> 枚举) ⇒ 转 L3
L3 (第 1 次)	原始输出全文 + 违规清单	经 <code>`output.repair_llm`</code> (默认同调用方) 发起修复调用	修正后的 JSON 文本
L1-L2 (重走)	修复输出	同 L1 / L2	通过 ⇒ 返回对象； <code>`attempts = 2`</code> ； <code>`resolved_at.l3_1`</code> 计 1

若第 2 次 L3 修复后仍未通过 L2，则预算耗尽，抛 ``SchemaViolation(errors, raw_last_output)``：该记录 ``status = "failed"``、错误码 ``schema_violation`` (7.6) 入 `rejects` 通道，并计入 ``resolved_at.rejected``。

### 3.8.5 v1.16 联合规划的 brief Schema

v1.16 为 M6 的 sampled brief 增加内部构造器 ``brief_schema(length)``。它返回一个顶层 object，唯一属性为 ``steps``；``steps`` 是长度恰为 ``length`` 的数组，每个位置的 object 只有必填字符串属性 ``brief``，并以 ``additionalProperties = false`` 封闭。该 Schema 不返回 ``frame_class``，因为帧类词已经被 CP-SAT planner 冻结，LLM 只能补充每个位置的自然语言概要。它是内部 Schema 待遇：经过 L0-L3，但不经过 ``output.validator``，不计入 ``report.schema_engine.resolved_at``。

```
```python
def brief_schema(length: int) -> dict:
    """构造 v1.16 联合规划的定长 brief 内部 Schema。"""
```

M6 将 brief 数组按规划器的固定 frame class 位置配对，再传给已有的 `realize_schema(step_schemas)`；`realize` 的 `prefixItems` 仍逐位约束帧类生成 Schema，时间字段绑定后的缩减 Schema 仍由 M6 传入。默认无 rules/windows 的 v1.15 路径继续使用 `plan_schema` 并返回 `frame_class + brief`，因此 brief Schema 不改变默认路径的 prompt 或 Schema 字节。M8 不解析规则、窗口或 correlation，也不参与长度可行性判断；这些语义由 M1/M6 的共享 planner 与声明式 evaluator 负责。

3.9 M9 LLM 客户端 llm-client

3.9.1 职责与边界

做：按 profile 提供统一异步调用接口：消息构造（文本/多图多模态）、provider 适配（OpenAI 兼容 / Anthropic 原生）、结构化输出参数透传、超时/重试/限流、token 与成本计量、--probe 连通性探测。**不做：**不解析业务结构（返回原始文本或原生结构化载荷，解析归 M8）；不缓存响应（无状态）；不做模型路由/降级等「智能」行为——模型与 profile 选择完全由配置决定。v1.6 注：同 profile 内的密钥池轮换（3.9.3）是传输层容错而非模型路由——池内各密钥打同一 base_url、同一 model，密钥选择不改变产出数据的任何内容语义。

3.9.2 API 与数据结构

```

@dataclass(frozen=True)
class Part:
    kind: Literal["text", "image"]; text: str | None; image: ImageRef | None
@dataclass(frozen=True)
class Message:
    role: Literal["system", "user", "assistant"]; parts: tuple[Part, ...]
@dataclass(frozen=True)
class PromptBundle:
    messages: tuple[Message, ...]; temperature: float | None = None
    image_px: int | None = None # v1.11 追加字段 (V23①, 3.9.5): 本次调用的图片生效像素档
    # (V21 判审升级档的唯一载体; None = 用 profile 工作点。
    # px 必须随 bundle 而非算子可变状态—build_body 每
    # attempt 重编码图片, 载体在 bundle 才保重试确定性)

@dataclass(frozen=True)
class LLMResponse:
    text: str; structured: dict | None; usage: Usage; model: str; latency_ms: int
    finish: str | None = None # v1.11 追加字段 (V23③, 3.9.5): 规范化终止原因 (openai
    # finish_reason / anthropic stop_reason 原值), 供 V11/V24
    # 终局化判定; _result_usage 的 len==4 分派随元组形状同步适配

class LLMClient:
    def __init__(self, profiles: dict[str, ProfileConfig]): ...
    async def complete(self, profile: str, prompt: PromptBundle,
        response_schema: dict | None = None) -> LLMResponse:
        """response_schema 仅在 profile 声明 supports_structured_output 时转为 L0 参数, 否则忽略。
        重试后仍失败: 抛 ProviderRetryableError / ProviderFatalError。"""
    async def embed(self, profile: str, texts: list[str]) -> list[list[float]]:
        """v1.2 新增。profile 须为 config.toml [embedding.*] 子表名 (5.1), 不接受 [llm.*]。
        openai_compatible: POST {base_url}/embeddings (3.9.3)。返回向量与 texts 顺序一一对应;
        profile 配置了 dims 时逐条校验返回维度, 不匹配抛 ProviderFatalError。
        token 计量入 usage_by_profile (键 = embedding profile 名); 每次调用发 llm.call
        trace 事件, payload 增可选字段 operation="embedding" (事件目录只增不改, 7.2)。
        重试/限流/超时规则与 complete 一致 (3.9.3)。"""

    async def probe(self, profile: str) -> ProbeResult # validate --probe: 1-token 试调用 (池化 profile 探测首密钥)
    async def probe_all(self, profile: str) -> list[ProbeResult]
        """v1.6: 逐密钥探测—按声明序每密钥一个 ProbeResult (增 key_env 字段: 所用密钥的环境变量名,
        单密钥 profile 为 None); 单密钥 profile 退化为 [await probe(profile)]. llm 与 embedding
        profile 均适用。validate --probe 使用本方法 (2.4), 成本 = 各被引用 profile 池大小之和 次探测调用。"""

    @property
    def usage_by_profile(self) -> dict[str, Usage] # 报告用累计量 (v1.6: 池化 profile 另含逐密钥
    # calls/rate_limited/disabled 与驻留统计, 6.4)

    @property
    def calibrator(self) -> ImageCostCalibrator # v1.11 (V19/V23②, 3.9.5): 图片成本在线校准器—
    # LLMClient 自持构造 (零 factory 改动); M9 每响应
    # 喂样本、算子经 ctx.llm.calibrator.cost(profile)
    # 读数、M10 批边界 freeze_batch()

    def snapshot(self, now: float | None = None) -> tuple[ProfileSnapshot, ...]
        """v1.10 (7.7 console 面板数据源, U19/U26)。纯读、无 await、无锁; 仅从渲染 tick
        (事件循环线程内) 调用—同线程下与并发 gather 无争用。now 注入供离线测试
        (KeyPool 同风格)。全量枚举 [llm.*] 与 [embedding.*] profile; 密钥池未物化时
        从声明构造零值 KeySnapshot (snapshot 不物化池—读操作不改状态)。"""

@dataclass(frozen=True)
class KeySnapshot:
    env: str # v1.10
    # 环境变量名—唯一可展示身份 (密钥值任何面均不出现)
    state: Literal["ok", "cooldown", "disabled"]
    cooldown_remaining_s: int = 0
    calls: int = 0 # 逐密钥用量镜像 (KeyUsage) —面板 'l' 展开视图
    rate_limited: int = 0 # 数据源 (7.7); 池未物化时为 0

@dataclass(frozen=True)
class ProfileSnapshot:
    name: str # v1.10
    kind: Literal["llm", "embedding"] # usage 按 name 合桶的既有口径由 kind 消歧
    in_flight: int # Σ 密钥 in_flight (在线 HTTP 请求数, 不含驻留/退避)
    max_concurrency: int
    calls: int
    retries: int
    prompt_tokens: int
    completion_tokens: int
    est_cost_usd: float | None # 未配价目为 None (面板显示 "--")
    p50_latency_ms: int | None # 有界样本窗中位数; 无样本为 None
    keys: tuple[KeySnapshot, ...] # 池 = 1 时单元素

```

3.9.3 行为规格

机制	定义
Provider 适配	<code>openai_compatible</code> ：POST <code>{base_url}/chat/completions</code> ，图像为 <code>image_url: data:image/png;base64,...</code> ； <code>anthropic</code> ：POST <code>{base_url}/v1/messages</code> ，图像为 <code>source.type="base64"</code> 。任一 LLM profile 显式设置 <code>thinking = "enabled" "disabled"</code> 时，按对应协议在请求顶层追加 <code>{"thinking": {"type": <value>}}</code> ；缺省不追加。两者的结构化输出映射见 3.8.2 L0。v1.2 <code>embedding: embed()</code> 仅支持 <code>openai_compatible</code> ：POST <code>{base_url}/embeddings</code> ，请求体含 <code>model</code> 与 <code>input</code> (texts 数组)，响应取 <code>data[*].embedding</code> (顺序与输入对齐)； <code>[embedding.*]</code> profile 的 provider 无 "anthropic" 取值 (5.1)。
重试	可重试错误 = 网络错误、超时、HTTP 408/409/429/5xx。第 i 次等待 = $\text{random}(0, \text{retry_base_delay_s} \times 2^i)$ (全抖动指数退避，工业标准)，封顶 60s，最多 <code>max_retries</code> 次——v1.6 起该退避公式仅适用于网络错误/超时/408/409/5xx；一切 429 等待 (含与不含 <code>Retry-After</code>) 一律落于密钥冷却而非调用内休眠，时长以密钥池行为唯一规范 (含 <code>Retry-After</code> 遵从其全时长；缺失按密钥计数指数冷却、封顶 300s)。池内有其他可用密钥时下一次尝试立即轮换、零等待；池大小 1 时驻留至冷却结束——含 <code>Retry-After</code> 时净等待与 v1.5 相同但受 <code>run.max_park_s</code> (默认 3600s) 约束，超长 <code>Retry-After</code> (如小时级配额信号) 在单密钥配置下不再无界等待，而按重试耗尽让该记录失败 (v1.6 行为修订)。不可重试错误 (401/403/400/404) 直接抛 <code>ProviderFatalError</code> ；其中认证类 (401/403) 在抛出的同时立即打开熔断器——凭据/权限故障不会自愈，按连续计数只是烧钱 (v1.5)；v1.6 密钥池下认证失败先按密钥禁用，仅当禁用的是最后一把存活密钥时才立即熔断 (密钥池行)。重试耗尽 (<code>provider_retryable_exhausted</code> ，v1.6 含驻留超限) 同样计入熔断窗口 (7.6)。v1.11 修订 (V16/V20/V24, 3.9.5)：所用 profile 预算开启 (<code>context_window > 0</code>) 时，400 响应先于 <code>ProviderFatalError</code> 分类在完整响应体上匹配超窗 pattern 集 (3.9.5 pattern 行)——命中抛 <code>ContextOverflowError(phase="reactive")</code> ，M9 不喂熔断连击 (降级重试与终局补喂归属主算子，熔断交互矩阵见 3.9.5)；未命中或预算关闭走本行 fatal 老路 (零回归)。另两类记录级终局同不入本行分类： <code>complete()</code> 分派前终检抛 <code>ContextOverflowError(phase="precheck")</code> (零 provider 交互，V16)、成功响应按终止原因归一终局化 (<code>OutputTruncatedError</code> ；200 形态 <code>model_context_window_exceeded</code> 同抛 <code>ContextOverflowError(phase="reactive")</code> ——V11, 3.9.5)——上述异常均不喂 <code>_record_provider_result(fatal=True)</code> 、不烧常规重试。
限流	每 profile 一个 <code>asyncio.Semaphore(max_concurrency)</code> ；全部调用 (含修复、评审) 共享该信号量。v1.6：池化 profile 仍是一个信号量—— <code>max_concurrency</code> 为池内全部密钥的总在途上限，不随密钥数放大。
密钥池 (v1.6)	profile 以 <code>api_key_envs = [...]</code> (5.1, 与 <code>api_key_env</code> 恰提供其一) 声明多把密钥，同 <code>base_url</code> 、同 <code>model</code> 构成同构池；单密钥配置 = 池大小 1：数据产出、重试记账与熔断/退出语义与 v1.5 一致，429 等待路径为 v1.6 行为修订 (见重试行： <code>Retry-After</code> 等待受 <code>run.max_park_s</code> 约束；无 <code>Retry-After</code> 冷却封顶 300s 且按密钥跨调用计数；驻留发 WARN 与事件)。选择：每次请求尝试发出前选「在途请求数最少」的可用密钥 (并列取声明序靠前者；确定性算法、无 RNG——密钥选择只影响时序不影响数据内容，与重试抖动同属 seed 豁免，2.6 可复现性不受影响)，请求头按所选密钥逐次构造。每密钥 429 冷却：含 <code>Retry-After</code> 时冷却其全时长；缺失时按 $\text{random}(0, \text{retry_base_delay_s} \times 2^c)$ 全抖动冷却、封顶 300s (c = 该密钥连续 429 计数，跨逻辑调用累计，该密钥自身任一成功清零)——无 <code>Retry-After</code> 的持续限流由此以每密钥 $\leq$ 每 5 分钟一次的探测频率自愈。冷却不改重试记账：该次尝试照常消耗一次重试预算，但下次尝试立即换可用密钥重发 (<code>llm.key_cooltdown</code> 事件, 7.2)。认证禁用：密钥 401/403 $\Rightarrow$ 本运行内永久禁用 (<code>stderr WARN</code> 一次 + <code>llm.key_disabled</code> 事件，携环境变量名)；池内尚有存活密钥 $\Rightarrow$ 同一尝试立即换密钥重发，不消耗重试预算、不计入熔断 (认证失败是密钥级确定性故障，每密钥至多发生一次，轮换次数以池大小为界)；禁用的是最后一把存活密钥 $\Rightarrow$ 等价 v1.5 认证首错：立即熔断、退出码 4 (3.10.3)。配额以 403 形态出现的 provider 同按认证禁用处理，不做错误体嗅探 (1.6 对齐决策 ④)。驻留：全部存活密钥均在冷却 $\Rightarrow$ 调用驻留至最早冷却结束 (<code>llm.pool_parked</code> 事件 + <code>stderr WARN</code> ； $\leq 60s$ 分片休眠，每片重查熔断器——v1.5 排队调用熔断复查语义保持)；驻留不消耗重试预算，单次逻辑调用累计驻留 (跨多段驻留求和，不含信号量排队等待) $> \text{run.max_park_s}$ (5.2, 默认 3600, 0 = 不驻留) $\Rightarrow$ 按重试耗尽抛 <code>ProviderRetryableError</code> (记录 failed、计入熔断窗口, 1.6 对齐决策 ③)；最早冷却结束时刻已可证超出剩余驻留预算时立即按同路径失败，不空耗墙钟。驻留发生在已获取的信号量槽内并持有该槽——全池冷却时吞吐本为零，放槽只会放入更多注定驻留的调用。降级阶梯：轮换 (零等待) $\rightarrow$ 驻留 (有界) $\rightarrow$ 记录失败累积 $\rightarrow$ 熔断退出 4。400/404 等请求形错误与密钥无关：不轮换，行为同 v1.5。 <code>embed()</code> 与 <code>[embedding.*]</code> profile 适用同一机制。
图像编码	调用时读盘 $\rightarrow$ 若长边 $>$ 生效像素档按比例缩小再编码 (Pillow) $\rightarrow$ base64 $\rightarrow$ 请求发出后即释放字节 (懒加载契约, 2.6 节)。v1.11 生效档链 (V18/V21/V23①, 3.9.5)：生效 <code>px = bundle.image_px or profile.default_image_px or profile.max_image_px</code> ，再钳 <code>min(·, profile.max_image_px)</code> —— <code>default_image_px</code> 为采样默认工作点 (0/缺省 = 沿用 <code>max_image_px</code> ，行为与 v1.10 逐字节一致, 5.1)、 <code>bundle.image_px</code> 为 V21 判审升级档载体、 <code>max_image_px</code> 恒为升级天花板。
计量	从响应 <code>usage</code> 字段累计 <code>prompt/completion token</code> ： <code>profile.price_per_mtok_in/out</code> (可选配置) 存在时折算成本入报告。
校准采样 (v1.11)	每个含图成功响应向图片成本校准器 (V19, 3.9.5) 喂一个样本： $(\text{usage.prompt_tokens} - \text{est_text}(\text{本请求文本})) / \text{本请求图数}$ ；响应无可用 <code>usage</code> (企业网关有 <code>usage: null</code> ，[C-64]) $\Rightarrow$ 不记样本、WARN 一次/profile ("image-cost calibration inactive")——校准器停留先验 $\times 1.2$ 。
快照 (v1.10)	<code>snapshot()</code> (3.9.2) 为 console 面板的只读拉取面 (7.7, 每 tick 一次)： <code>in_flight</code> = Σ 密钥在途 (在线 HTTP 请求数口径)、密钥三态 (ok / cooldown 携剩余秒 / disabled)、用量镜像 <code>usage</code> 、p50 延迟 = 每 (kind, profile) 有界样本窗 <code>deque(maxlen=256)</code> 的中位数 (成功逻辑调用口径， <code>_post_with_retries</code> 成功返回前喂入——v1.10 唯一新增采集点)；窗口与中位数不入 <code>report.json</code> (报告零新键, 7.7)、不入任何事件。

背书：「统一多 provider 异步客户端 + 信号量并发 + 指数退避」与 distilabel 的 LLM 抽象层 [5]、NeMo Curator 的服务客户端 [9] 同构；全抖动退避为 AWS 架构规范确立的工业标准。

3.9.4 输入 / 输出示例

以统一 UI 示例贯穿：5.1 的 `[llm.default]` profile + 5.2 的 `project.toml`，记录为文件对 `capture/2026-07-01/b/uitree_2.jsonl` + `c/image_2.png` (`pair_index=2`，即 6.3 中 id 为 `9f2c31ab52e08d17` 的登录页记录)。M5 按 3.5.2 模板组装 `PromptBundle`，M8 因 `supports_structured_output=true` 启用 L0，M9 适配为如下 `openai_compatible` 请求。

① 标注请求报文 (`openai_compatible`) 与响应要点


```
POST https://llm-gw.example.com/v1/chat/completions HTTP/1.1
Authorization: Bearer ${LABELKIT_KEY_DEFAULT}      ← api_key_env 所指环境变量的值
Content-Type: application/json

{
  "model": "qwen2.5-vl-72b-instruct",
  "temperature": 0.0,
  "max_tokens": 4096,
  "messages": [
    {
      "role": "system",
      "content": "你是移动端 UI 理解标注员。根据屏幕截图与 UI 控件树，\n标注该屏幕的功能类别、页面标题、可交互元素列表与一句话页面描述。输出必须是符合以下 JSON Schema 的单个 JSON 对象，不输出任何其他内容：\n{...5.2 用户 Schema 全文，与下方 response_format.json_schema.schema 相同，此处从略...}"
    },
    {
      "role": "user", "content": [
        {
          "type": "text", "text": "[屏幕截图]"
        },
        {
          "type": "image_url", "image_url": {
            "url": "data:image/png;base64,iVBORw0KGgoAAAANSU... (截断示意；c/image_2.png 读盘后长边≤2048px 等比缩放再编码，3.9.3)"
          }
        },
        {
          "type": "text", "text": "[UI 控件树]\nFrameLayout [0,0,1080,2340]\n  TextView \"登录\" [72,296,264,392]\n  EditText [72,520,1008,664] hint=请输入手机号\n  EditText [72,712,672,856] hint=请输入验证码\n  Button \"获取验证码\" [704,712,1008,856]\n  Button \"登录\" [72,952,1008,1096]"
        }
      ]
    },
    {
      "response_format": {
        "type": "json_schema", "json_schema": {
          "name": "user_schema", "strict": true,
          "schema": {
            "$schema": "https://json-schema.org/draft/2020-12/schema",
            "type": "object",
            "properties": {
              "screen_category": {
                "type": "string", "enum": ["login", "home", "list", "detail", "form", "settings", "dialog", "other"]
              },
              "page_title": {
                "type": "string"
              },
              "interactive_elements": {
                "type": "array", "items": {
                  "type": "object",
                  "properties": {
                    "role": {
                      "type": "string", "label": {
                        "type": "string",
                        "bounds": {
                          "type": "array", "items": {
                            "type": "integer", "minItems": 4, "maxItems": 4
                          },
                          "required": ["role", "label", "bounds"], "additionalProperties": false
                        },
                        "description": {
                          "type": "string", "maxLength": 200
                        },
                        "required": ["screen_category", "page_title", "interactive_elements", "description"],
                        "additionalProperties": false
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  ],
  "id": "chatcmpl-7d31a9c04b5e82f6",
  "choices": [
    {
      "index": 0, "finish_reason": "stop", "message": {
        "role": "assistant",
        "content": "{\n  \"screen_category\": \"login\", \"page_title\": \"登录\", ... (与上方 message.content 逐字相同)"
      }
    }
  ],
  "usage": {
    "prompt_tokens": 3184, "completion_tokens": 156, "total_tokens": 3340
  }
}
```

② M9 返回的 LLMResponse (3.9.2)

```
{
  "text": "{\n  \"screen_category\": \"login\", \"page_title\": \"登录\", ... (与上方 message.content 逐字相同)",
  "structured": null,
  "usage": {
    "prompt_tokens": 3184, "completion_tokens": 156,
    "model": "qwen2.5-vl-72b-instruct",
    "latency_ms": 4820
  }
}
```

structured 仅在 anthropic provider 以 tool_choice 强制工具调用 (3.8.2 L0) 返回原生结构化载荷时非空；openai_compatible 的 json_schema 模式产物是文本，M9 原样放入 text 不做解析 (3.9.1 负边界)，解析与校验由 M8 L1→L2 完成——本例 content 已是纯 JSON，将计入 report.schema_engine.resolved_at.l0_or_clean。

③ 重试时间线 (同一次 complete() 调用，profile 默认值：timeout_s=120, max_retries=5, retry_base_delay_s=1.0)

尝试	时刻	结果	等待	依据 (3.9.3 重试规则)
attempt 1	t=0.0s 发出	HTTP 429, 响应头 Retry-After: 5	5.0s	429 属可重试错误集 (408/409/429/5xx)；「429 响应含 Retry-After 时优先遵从」，直接等 5s，回避公式不参与。

attempt 2	t=5.0s 重发	t=125.0s 达 timeout_s=120 超时	2.3s	超时属可重试错误；全抖动指数退避「第 i 次等待 = <code>random(0, retry_base_delay_s × 2^i)</code> 」, i=2: <code>random(0, 1.0×2^2)=random(0, 4.0)</code> ，本次抽得 2.3s (< 60s, 封顶不生效)。
attempt 3	t=127.3s 重发	t=132.1s 收到 HTTP 200 (即 ① 的响应, 耗时 4.82s → <code>latency_ms=4820</code>)	—	成功返回 LLMResponse。已用重试 2 次 < <code>max_retries=5</code> ；若第 5 次重试后仍失败则抛 <code>ProviderRetryableError</code> (3.9.2)；本次调用 <code>retries+=2</code> 计入该 profile 计量。

注 (v1.6)：本表为 v1.5 单密钥基线，⑤ 引用其作对比。v1.6 起 attempt 1 的 5s 等待经**密钥冷却 + 驻留**实现（发 `llm.key_cooldown` / `llm.pool_parked` 事件，计入 `run.max_park_s`）——净时序与本表相同，重试记账不变（3.9.3 密钥池行）。

④ `usage_by_profile` 累计结果示例

```
>>> client.usage_by_profile          # 快照：完成 3 次标注调用（含上表那次）与 1 次评审调用后
{
  "default": {"calls": 3, "prompt_tokens": 9552, "completion_tokens": 486,
              "retries": 2, "est_cost_usd": 0.0066},
  "judge":   {"calls": 1, "prompt_tokens": 2210, "completion_tokens": 118, "retries": 0}
}
```

数值自洽性：3 次标注请求 prompt 各 3184 token， $3 \times 3184 = 9552$ ；completion 为 $156 + 162 + 168 = 486$ 。[`llm.default`] 配置了 `price_per_mtok_in=0.6` / `price_per_mtok_out=1.8`，故 `est_cost_usd = 9552 ÷ 106 × 0.6 + 486 ÷ 106 × 1.8 = 0.0057312 + 0.0008748 = 0.006606 ≈ 0.0066`；[`llm.judge`] 未配置单价，不产生成本字段（3.9.3 计量）。`retries=2` 来自上表时间线。该字典在 `finalize` 时由 M10/M11 落入 `report.json` 的 `llm_usage` (6.4)。

⑤ 密钥池轮换时间线 (v1.6；`api_key_envs = ["LABELKIT_KEY_A", "LABELKIT_KEY_B"]`，其余 profile 参数同 ③)

尝试	时刻	密钥	结果	等待	依据 (3.9.3 密钥池行)
attempt 1	t=0.0s 发出	KEY_A (两密钥在途数相同, 取声明序靠前者)	HTTP 429, 响应头 <code>Retry-After: 30</code>	0s	KEY_A 冷却至 t=30.0s (<code>llm.key_cooldown</code> , <code>cooldown_s=30</code> , <code>retry_after=true</code>)；本次尝试消耗 1 次重试预算；KEY_B 可用 ⇒ 下次尝试立即发出，限流等待为零。
attempt 2	t=0.0s 重发	KEY_B	t=4.6s 收到 HTTP 200	—	成功返回。对比 ③ 时间线：v1.5 单密钥同场景须原地等 30s——轮换把限流等待压为零。本调用 <code>retries+=1</code> 计入计量； <code>llm.call</code> 携 <code>key_env="LABELKIT_KEY_B"</code> (7.2)。

边界情形：若 t=0.0s 时 KEY_B 也在冷却（如至 t=12.0s），则调用**驻留**至 t=12.0s 再以 KEY_B 重发（`llm.pool_parked`；驻留不消耗重试预算，累计受 `run.max_park_s` 约束）；若 KEY_B 此前已被 401 禁用，则 KEY_A 即最后一把存活密钥——其 429 冷却照常驻留等待，而其 401 将立即熔断（3.10.3）。

3.9.5 上下文预算、估算器与图片成本校准 (v1.11)

v1.11 新增（决策 V1—V27 见 `docs/dev/SPEC-context-budget.md`，调研引用 [C-1]—[C-84] 见 `docs/dev/PROPOSAL-context-budget.md`）。**不变式**：对每一次 LLM 调用，`est(输入 prompt) + max_output_tokens + margin ≤ context_window`。预算按 profile **声明制**开启（5.1 `context_window` 行：0 = 未声明 = 本节机制对该 profile 整体关闭，行为与 v1.10 一致；声明实效窗口教义见 5.1/V26）。预算与估算原语落新文件 `labelkit/common/runtime/budget.py`（全部纯函数、零第三方依赖；margin/密度/阶梯常数冻结于代码、不开配置面——业界不存在权威保守系数（V7/V8 调研负结果），用户逃生门 = 声明更小的 `context_window`；修改常数即 spec 修订）。数据自适应的贪心装填器属算子逻辑、落在 M14 (3.14.4) ——`budget.py` 只提供估算与预算原语 + 校准器（依赖方向 `operators` → `common` 不变）。

机制	定义
预算公式 (V7)	<code>margin = max(256, ceil(0.10 × context_window))</code> ； <code>input_budget = context_window - max_output_tokens - margin</code> （ <code>context_window == 0</code> ⇒ 0 = 预算关）。embedding 预算 = <code>context_window - margin</code> （ 无输出预留 , V15）——embed 输入超预算按确定性头部保留截断（嵌入语义主体在前部，3.3.3 第④级）。margin 承担：估算残差 + 消息封装 + provider 侧计数偏差。预算非正 ⇒ M1 CONFIG_ERROR (3.1.4)。
零依赖估算器 (V8 v3)	<code>est_text(s) = ceil(ascii/3 + cjk×1.0 + other/2)</code> ——ASCII 取 /3 非 /4 = 对 JSON 膨胀的保守化；CJK×1.0 覆盖 GLM 0.67 / o200k 0.8—1.0 / Qwen·DeepSeek 0.77—0.9 (t/字)； 已知局限 ：c100k 旧词表中文 1.25—1.4 t/字不被覆盖（记载 + 逃生门同上，per-profile 密度旋钮列 roadmap）。CJK 判定 = Unicode 块 CJK Unified Ideographs 及扩展 + 全角标点。消息封装 +4 t/消息；结构化输出 schema 文本计入 est（它随请求发送）。 <code>est_image_prior(profile, px)</code> = provider 文档公式于 生效工作点 px 的最坏纵横比 求值： <code>anthropic = min([px/28]², 1568)</code> (28px patch 制最坏正方形)； <code>openai_compatible = tile 制按 2048→短边 768 归一化后的最坏纵横比求值</code> ——@2048 长边竖屏 = $85 + 8 \times 170 = 1445$ （正方形 765 是特例，UI 截图纵横比下系统性低估 36%，[C-60]）。图片估算仅作 首批先验 （校准器先验种子 = 本值 × 1.2 保守放大，PRIOR_INFLATION）——正确性由在线校准（下行）与溢出反应（V20 行）承担，公式准确度只影响首批装填效率（V17 测量-反应式范式；est 的语义自始是 预留上界而非精确记账 ）。不引 tokenizer 依赖（对主力 GLM 不给真值）。

在线图片成本校准器 (V19/V23②)	<code>ImageCostCalibrator</code> (<code>budget.py</code> ; 运行内存、零持久化——跨运行冷启动是 <code>stateless</code> 约束的固有代价) 实例由 <code>LLMClient</code> 自持 (构造器内建, 零 <code>factory</code> 改动), 公开面 <code>llm.calibrator</code> (3.9.2)。每 <code>profile</code> 维护每图实际成本估计: 样本 = <code>(usage.prompt_tokens - est_text(该请求文本)) / 本请求图数</code> (M9 每含图响应喂入, 3.9.3 校准采样行); 滤波 = 窗口化最大值, 窗口单位 = 批 ——样本以 <code>asyncio</code> 完成序到达, M10 于批边界调 <code>freeze_batch()</code> 聚合 本批 样本的 <code>max</code> (对无序集取 <code>max</code> , 序无关) 压入 <code>deque(maxlen=8)</code> 批最大值窗口; 装填读数 <code>cost(profile) = max(批最大值窗口) ÷ 0.85</code> 取整 (装填安全折扣); 累计样本 <code>< 8</code> ⇒ 先验 <code>× 1.2</code> (样本不足不做主动升档)。 确定性护栏: 校准快照按批冻结 ——第 <code>N</code> 批装填只读 <code>< N</code> 批聚合值 (批序串行 ⇒ 同输入同配置可复现; 逐响应更新 + 逐调用读取会让内容依赖 <code>asyncio</code> 完成序, 禁止)。usage 缺失兜底见 3.9.3 校准采样行。校准终值入 <code>report.budget.image_cost</code> (6.4, V13⑤)。
<code>complete()</code> 终检 (V16)	<code>complete()</code> 于 <code>provider</code> 分派前执行不变式终检: <code>est_prompt + max_output_tokens + margin > context_window</code> ⇒ 抛 <code>ContextOverflowError(phase="precheck")</code> (记录级; 不喂熔断、不烧重试)。归属理由: <code>complete()</code> 是全部调用——含 M8 L3 修复调用与 <code>--probe</code> ——的唯一咽喉: <code>probe</code> 经一次性子客户端同走 <code>complete()</code> , <code>max_output_tokens=1 + V6</code> 正预算校验使其平凡通过, 无须豁免工程 (F13)。 装填层正确时终检永不触发 ——它是防御性不变式, 不是第二套装填逻辑。
终止原因归一 (V11)	响应终止原因 (<code>LLMResponse.finish: openai finish_reason / anthropic stop_reason</code> 原值) 按闭合映射 终局化 , 不再把截断 JSON 送 L1–L3 修复循环硬修: ① <code>finish_reason=length (openai) / stop_reason="max_tokens" (anthropic)</code> ⇒ <code>output_truncated</code> (输出触到 <code>max_output_tokens</code> 上限; 记录级 <code>reject</code> , 不喂熔断——预算已为 <code>max_output_tokens</code> 预留完整空间, 模型自然写满属输出侧事件); ② 双协议 <code>model_context_window_exceeded</code> (anthropic 4.5+ 系 <code>stop_reason [C-57]</code> ; z.ai openai 协议 <code>finish_reason [C-58]</code>) ⇒ <code>context_overflow</code> 反应态—— <code>input+max_tokens > cw</code> 时新款后端不再 400 而是接受请求、生成触墙截断, 此值即溢出 oracle 的 200 形态, 同抛 <code>ContextOverflowError(phase="reactive")</code> (预算开启时可触发属主算子降级重试, 不依赖 400 嗅探); ③ z.ai 扩展值 <code>sensitive / network_error</code> 及其他未知值 ⇒ v1 不做专项处置, 沿现行管线流转 (内容进 M8 校验, 垃圾输出自然走修复/拒收)。 显式拒绝厂商「加大 <code>max_tokens</code> 重试」建议 ([C-61]) ——逐调用抬升输出上限即破坏预算不变式与确定性; 正确的用户补救 = 配置层提高 <code>max_output_tokens</code> 。
超窗错误体 pattern 集 (V20)	按 <code>profile</code> 预算门控 (<code>context_window == 0</code> 时嗅探不启用, 400 走 v1.10 原路): 400 响应在完整 <code>resp.text</code> 上 (先于任何截断) 匹配溢出 pattern 初始集 ([C-75] 实证种子)——OpenAI/Azure <code>code == "context_length_exceeded"</code> ∨ 消息含 <code>"maximum context length"</code> ; vLLM 同消息族 (<code>type=BadRequestError</code> 、无该 <code>code</code> ——只匹 <code>code</code> 会漏); anthropic 协议 <code>invalid_request_error</code> ∧ 消息含 <code>"prompt is too long"</code> ; z.ai 业务码 <code>"1261"</code> / 消息含 <code>"Prompt too long"</code> ; OpenRouter <code>error_type == "context_length_exceeded"</code> 。命中 ⇒ 抛 <code>ContextOverflowError(phase="reactive")</code> , M9 不喂 <code>_record_provider_result(fatal=True)</code> ——降级重试机会与终局补喂归属主算子 (下行矩阵); 未命中或预算关闭 ⇒ 走 3.9.3 重试行 fatal 老路 (零回归)。嗅探只是「是否给降级机会」的优化门, 不构成熔断豁免面 (A7): 漏判 = 老路零回归, 误判 = 浪费 ≤ 2 次有界重试。z.ai 端点错误体样本列入集成测试采集, pattern 集可渐进扩充 (E2E-FINDINGS 记录)。
熔断交互矩阵 (V16/V20/V24/A7)	<code>ContextOverflowError</code> 带 <code>phase: Literal["precheck","reactive"]</code> (V24)。三形态 × 熔断连击: precheck 不计连击 (发生在任何 <code>provider</code> 交互之前, 属客户端决策, 与 <code>provider</code> 健康无关; 含 V10 最小单元不装——由算子在装填处直接记录, 无异常穿越); reactive-400 降级耗尽的终局计入连击 (A7 裁决——由属主算子经 <code>ctx.metrics.record_provider_result(fatal=True)</code> 补喂恰一次, M9 抛出时不喂; 成功降级或任何成功调用清零连击——只有「连续 <code>fatal_error_threshold</code> 条记录都无法靠降级挽救」的系统性失配才停机); reactive-200 (<code>model_context_window_exceeded</code>) 终局 不补喂 (HTTP 交互本身成功、 <code>streak</code> 已被该次 <code>ok</code> 清零, <code>llm.call</code> 事件维持 <code>status="ok"</code> , 实现者不得「修正」为 fatal)。output_truncated 同不喂熔断 (交互成功)。两异常均不烧常规重试预算 (V20 降级重试独立计数、有界 ≤ 2 次/调用)。降级机会仅在属主算子有降级机制且预算开启时消费 (segment 窗对半改切 3.14.4 / annotate 减帧 / quality-pointwise 文本收紧); 无降级面的调用点 (<code>extract / verify</code> 回收复裁 / <code>probe / L3</code> 修复) 直接按最小单元语义处置 (V10/V25)。

背书: 预算不变式为业界共识形态 (`LlamaIndex context_window - prompt - num_output [C-5]`、Claude Code `contextWindow - min(maxOut,20k) - 13k [C-15]`、OpenRouter 「输入+补全」合并判定 [C-17]); 「首批先验 + 稳态测量校准」的两段结构对标 ABR 的 `measure-don't-model` 谱系 (BBR `windowed-max [C-54]`、dash.js `DYNAMIC` 双阶段 [C-37]、FESTIVE `p=0.85` 装填折扣 [C-32])。

3.10 M10 编排器 orchestrator

3.10.1 职责与边界

做: 把 M2 记录流切批; 按配置组装阶段链并逐批驱动; 调度生成子批回流; 聚合运行级统计; 控制批生命周期——批完成 (其输出行已写盘) 后释放该批全部中间态; 熔断监测。不做: 零业务逻辑 (不知道任何算法细节); 不直接调用 LLM; 不写文件 (M11 职责)。

3.10.2 主循环与批生命周期

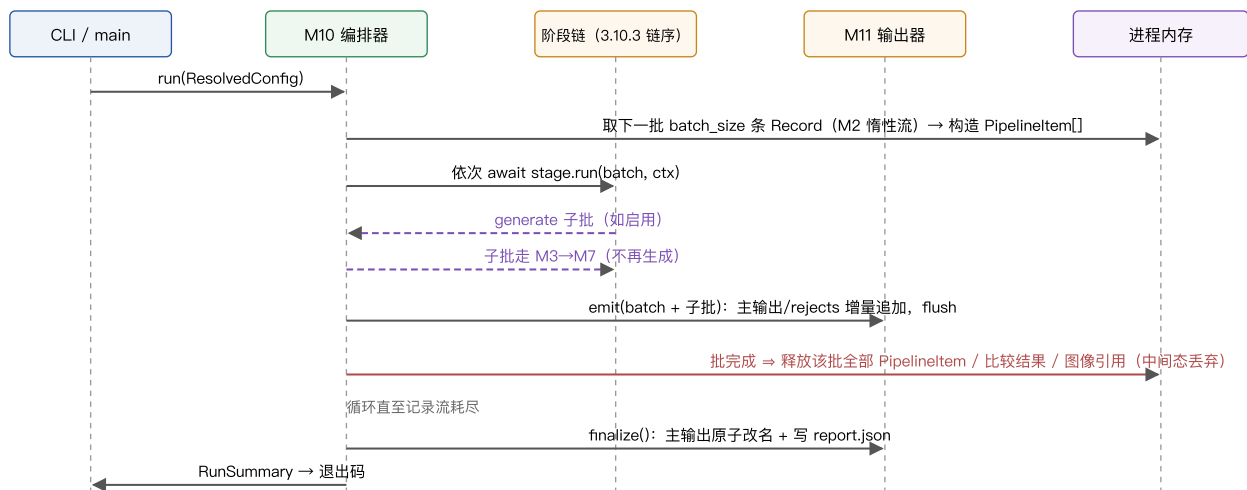


图 3-4 批生命周期时序。全局仅去重索引与统计计数器跨批存活，其余中间态随批释放。

3.10.3 API 与行为规格

```
@dataclass(frozen=True)
class RunServices:
    # 2026-08-14 收参：运行期共享服务与运行身份的参数对象
    llm: LLMClient # M9 客户端（用量 / 校准器读取面）
    schema_engine: SchemaEngine # M8 结构引擎（resolved_at 统计面）
    metrics: MetricsSink # M12 计数与事件汇（含 console 旁路）
    run_id: str # 本次运行标识
    run_started_at: datetime # 运行起点（带时区）

class Orchestrator:
    def __init__(self, cfg: ResolvedConfig, stages: list[Stage],
                 ingestor: Ingestor | None, emitter: Emitter,
                 services: RunServices): ...
    async def run(self) -> RunSummary: ...

@dataclass
class RunContext:
    # 传入每个 stage.run 的上下文
    cfg: ResolvedConfig; llm: LLMClient; schema_engine: SchemaEngine
    rng: random.Random # seed 派生：Random(f"{run.seed}:{batch_no}:{stage.name}")
    batch_no: int; metrics: MetricsSink
```

RunServices 由 labelkit.orchestration 导出（与 Orchestrator / RunSummary / build_stages 并列），调用方按名构造；RunContext 仍是规范的六字段（3.12.3 禁改签名），run_id / run_started_at 照旧只经构造参数传递、不进 RunContext。

规格	定义
跨批存活状态	仅三项：① DedupIndex（scope=global 时）；② MetricsSink 计数器；③ M9 用量累计。均不含数据内容本体（哈希/签名/计数），运行结束随进程销毁。v1.8（stream 模式）封闭清单增两项：④ M2 未闭合会话缓冲（≤ session_max_len 条 Record 元数据，图像仍懒加载，3.2.8）；⑤ M10 待装箱溢出会话（next-fit 唯一开口箱，见下方时序流行）——两者均属进程内存、随装箱/消费即释放，不构成新的落盘面（2.6 注记）。
尾批	最后一批不足 batch_size 照常处理；批内仅 1 条时 M4 不发裁决调用，各 criterion score 固定 0.5（3.4.3 归一化行）。
熔断	MetricsSink 维护连续致命计数（ProviderFatalError 与重试耗尽 provider_retryable_exhausted 均计入，7.6），达 run.fatal_error_threshold（默认 20），或 401/403 认证类首错立即（v1.5, 3.9.3；v1.6 密钥池下「认证首错」= 该 profile 最后一把存活密钥被认证禁用——池内尚有存活密钥时单密钥认证失败仅禁用该密钥、不计入熔断）⇒ 取消在飞任务、finalize。熔断交付（v1.6, 1.6 对齐决策 ②）：已完成批的主输出与 rejects 照常 fsync + 原子改名交付（v1.5 及以前为「.part 不交付」——长跑未段配额死亡不再丢弃全部已完成产出），报告写 run.circuit_broken=true 与 run.partial_delivery=true、counts 增列 unprocessed（6.4），退出码 4 不变。「运行完整处理了全部输入」的判定信号由此从「目标文件名出现」改为「report.run.interrupted=false 且 circuit_broken=false」——退出码 0/1 不足以判定：被 SIGINT 优雅中断的运行同样交付且以 0 退出（本表中断行）（3.11.2 主输出行、3.11.3 ④、6.4）。
中断（SIGINT/SIGTERM）	停止取新批 ⇒ 等待当前批完成或 30s 超时取消 ⇒ finalize（报告标记 interrupted=true）。已 flush 的输出行有效。
--limit N	M2 流截断在前 N 条记录，其余全流程不变（试跑）；generate_only 模式下作用于生成样本流的前 N 条：仅执行预抽序前 [N / generate.num_per_call] 次生成调用（(llm, style) 预抽不受影响），产出再截断到 N 条——本表下行「执行全部生成调用」带 --limit 时按此截断。

纯生成模式 (v1.4)	<code>run.mode="generate_only"</code> 时跳过 M2 (IngestReport 全零)：启动后先按 3.6.2 的量公式执行全部生成调用（并发受 profile 信号量限流, (llm, style) 组合按调用序号预抽保证可复现——生成先于切批、尚无批号, 预抽 PRNG 固定取 Random(f"{run.seed}:0:generate")），即 3.10.3 派生式中 batch_no 恒取 0），产出构造为 Record 后按 <code>run.batch_size</code> 切批, 逐批走 M3→M4→M5→M7→M11, 批生命周期与内存释放同 process 模式；不触发二次生成（单遍）。规模建议同 2.6 (≤ 50 万条)。
分类与扇出 (v1.7)	规范链序 <code>_CHAIN_ORDER = ("dedup", "classify", "quality", "generate", "annotate", "verify")</code> (v1.8 起扩展为含 segment/extract、v1.9 起含 stitch 的九名单一起集元组, 见下方时序流行——三者关闭时逐字节退化回本六名形)；<code>_compose_chain</code> 的 enabled 表增 classify——主链、生成回流链、generate_only 链均含（回流子批带 <code>source="inherited"</code> 继承分类, 经 M13 幂等跳过, 零额外调用, 3.13.4）。multi 扇出只改变批内信封基数、不改链结构（4.3 契约 ②a）：<code>counts.fanout = classify</code> 阶段执行前后 <code>len(batch)</code> 的差值, 由 M10 在批链循环处计量（<code>counts.*</code> 所有权属 M10, 与从 generate 返回值计 generated 同构）；<code>batch.end</code> 事件 payload 增 <code>fanout</code> 字段（7.2 只增；<code>batch.start.size</code> 语义 = 批入口信封数, 即扇出前基数）；熔断交付的 unprocessed 残差公式右侧同步 + <code>fanout</code>（6.4 不变量扩展）。--dry-run 估算（<code>_estimate</code>）增 <code>classify_calls</code>：process 模式 = <code>ingested × max(1, self_consistency)</code>, generate_only 模式 = 生成记录数 × <code>max(1, self_consistency)</code>（回流子批继承分类、不计入）；存在 <code>[class.*]</code> 覆盖或 <code>assignment="multi"</code> 时, quality/annotate/verify 估算按全局继承配置、multi 按标签乘数 1 报下界, 并在 stderr 注明口径——注记逐字为 dry-run: note: estimated with global config / multi reports a lower bound at label multiplier 1（1.6 v1.7 对齐决策 ⑦）。
时序流与整会话装箱 (v1.8, 仅 <code>segment.enabled = true</code>)	链序： <code>_CHAIN_ORDER = ("segment", "stitch", "dedup", "classify", "extract", "quality", "generate", "annotate", "verify")</code> ——九名单一起集元组（stitch 为 v1.9 增位, <code>_compose_chain</code> 的 enabled 映射同步增 <code>"stitch": cfg.stitch.enabled</code>）；segment/stitch/extract 默认关, 三者关闭时有效链逐字节退化为 v1.7 六名链序（generate 与 stream 互斥 (2.3.1), 故 generate 顺位与三新工位永不同链）。装箱 (S21)：M10 改消费 <code>ingestor.sessions()</code> 会话流视图 (3.2.8) 而非 <code>records()</code>, 按 next-fit (顺序装箱, 仅一只开口箱) 整会话装箱——会话按到达序装入, 装不下即封批开新箱；批容量 = <code>run.batch_size</code> 帧。单会话 > batch_size ⇒ M10 硬切 + WARN 一次 + 对切分会话的帧信封打 duck-typed <code>session_split</code> 标 (M7 缺帧判定的降级依据与 <code>_meta.stream.session_split</code>, 3.7.3/6.3)；M1 对 <code>stream.session_max_len > run.batch_size</code> 发静态 warning (3.1.4)。待装箱溢出会话是唯一新增跨批存活项（本表跨批存活行 ⑤, 装箱即释放）。session_id 盖章 (S4)：M10 构造帧信封时盖章 <code>PipelineItem.session_id</code>（簿记非业务逻辑, 4.1；M14 对其追加的 episode 信封盖章）。计量与记账：<code>counts.episodes = segment</code> 阶段执行前后 <code>len(batch)</code> 差值（fanout 同构计量, M10 属主——M14 不碰 <code>counts.*</code>）；post-emit 状态 tally 增 absorbed / dropped_noise；failed 兜底公式扩展为 <code>failed = max(len(batch) - emitted - dropped_dup - dropped_lowq - dropped_verify - absorbed - dropped_noise, 0)</code>（不扩展则 absorbed 成员被误计 failed）。<code>batch.end</code> 事件 payload 增 <code>episodes / absorbed / dropped_noise</code> 三可选键（仅 segment 启用时携带, fanout 的 R20 形制, 7.2 只增）；stderr 进度/摘要行不增键（fanout 先例——报表与 batch.end 可见）。守恒与中断 (S18)：守恒式全展开形见 6.4（左侧新增 dropped_noise 与 absorbed、右侧新增 episodes）；stream 模式下 <code>counts.unprocessed</code> 的出现条件扩为「熔断 V interrupted」——SIGINT 叠加会话缓冲会产生未走完流水线的在飞残差, 残差公式两侧同步扩展（右侧 + episodes、左侧 + absorbed + dropped_noise）；非 stream 中断残差恒 0、不加键（回归锚不动）。dry-run 估算 (S22/S23; v1.11/V12 修订)：<code>_estimate</code> 增 <code>segment_calls = Σ ceil((L-1)/(w_min-1))</code>（对长度 L ≥ 2 的会话求和；L = 1 或 <code>strategy="rules"</code> 计 0；window 实参自 v1.11 起替换为 <code>budget.min_window(cfg)</code> 导出的最坏保证装填量 w_min——上界语义：实际每窗装填 ≥ w_min 帧 ⇒ 实际窗数 ≤ 估算, 与 M1 预算护栏共用单一事实源；预算未声明时 w_min = window, 公式与数值与 v1.8 同构不变）与 <code>extract_calls = Σ(L-1)</code>（报上界）；quality/annotate/verify 估算以 <code>episodes ≈ sessions</code> 报下界 + stderr 注明口径（R28 式；注记逐字为 dry-run: note: stream estimate: downstream reports a lower bound at episodes=sessions (LLM refinement only adds segments)），stream stderr 注行在 w_min < window 时增补一句；segment reports an upper bound at worst-case budget packing（v1.11/V12）；批数由会话尺寸空跑 next-fit 装箱精确得出（文本模态行数统计与会话空跑单遍融合, 3.2.8）；两新键无条件打印 <code>segment_calls=...</code> <code>extract_calls=...</code>（classify 先例：默认关闭恒 0）。
线索缝合 (v1.9, 仅 <code>stitch.enabled = true</code>)	计量 (T7)：<code>counts.stitched</code> 由 post-emit 状态 tally 归集（壳终态；仅计被并 episode 信封壳——救援短段无信封形态、不产生壳, 3.16.6）；<code>counts.threads</code> 在 post-emit tally 处以恒等式 threads = episodes - stitched 导出（单点上报, 不设第二落点——救援只并入不开新线索、壳一对一抵扣、降格段照常入池、fanout 后置无交互, 恒等经审计验算；<code>counts.*</code> 属主仍归 M10, M16 不碰）。公式三处同步 (T7)：<code>failed</code> 兜底公式终态减项同步增 - stitched（<code>failed = max(len(batch) - emitted - dropped_dup - dropped_lowq - dropped_verify - absorbed - dropped_noise - stitched, 0)</code>）——不扩展则壳被误计 failed）；熔断/中断的 unprocessed 残差公式减项同步增 - stitched；守恒式左侧增 stitched 项（6.4）。batch.end：payload 增 <code>stitched / threads</code> 两可选键（仅 stitch 启用时携带, episodes 的 R20 形制, 7.2 只增）；stderr 进度/摘要行不增键（固定键集, stitched 经报表与 batch.end 可见——有意为之；v1.10 U18：固定键集约束收窄为 plain 面专属, rich 面板状态账展示 stitched/threads, 7.7）。report：stream 节增 <code>stitch</code> 子块 {<code>stitched, rescued_short, seams, judgments, repass_judgments, failures</code>}（M16 属主, 6.4）。dry-run 估算 (T16)：<code>_estimate</code> 增 <code>stitch_calls = len(session_lens) × votes × (2 若 repass 否则 1)</code>（<code>episodes ≈ sessions</code> 下界基数, 沿用既有 stderr 下界注；救援候选调用不计入估算——池非空才发生的 +ε 项）；该行无条件打印 <code>stitch_calls=...</code>（off 时恒 0, segment_calls 先例）。

console 旁路 (v1.10)	stage 信号: 批链循环内每 stage run() 之前调 metrics.stage_begin(stage.name, batch_no) ——进程内旁路仅转发 ProgressListener, 不产生 TraceEvent、不入 7.2 目录 (3.12.3/U11); _request_stop 内加一行 metrics.stop_requested() (中断横幅通路)。估算导出 (U20): 静态估算公式抽出为纯函数 estimate_run(cfg, plan) (_estimate() 改薄封装; dry-run 与渲染器批级分母共用); live 路径在 P2-4 预扫后经 metrics.run_estimate(...) 发送 ——process 模式复用该次 scan (UI 模态翻 estimate=True, 配对表零额外 I/O; 文本模态仅 console.estimate = true 时做行数估算, U17), 禁二次 scan; generate_only 走 3.6.2 静态公式无 scan。dry-run 呈现 (U13; v1.11/V12 修订): rich 档下估算四行 print 让位于渲染器表格 (数值逐项一致); plain 档行式输出为逐字节锚—— segment_calls / stitch_calls 维持无条件打印, 其中 segment_calls 行的含义自 v1.11 起改为按 w_min 报预算最坏装填上界 (本表时序流行; 预算未声明或 w_min ≥ window 时数值与 v1.10 逐字节不变——examples 声明保守实效窗下当时的五个黄金文件不动, V26; v1.12 起黄金文件为七个且因估算行插入两帧粒度键全部重采, 见本表帧粒度行)。listener = None 时以上全部为 no-op (v1.9 行为逐字节一致)。
上下文预算 (v1.11, 仅预算启用时)	批边界校准冻结 (V19): 每批处理完成、下一批装填开始前调用 self.llm.calibrator.freeze_batch() ——聚合本批图片成本样本的 max (对无序集取 max, 序无关) 压入批最大值窗口、刷新可读快照 (第 N 批装填只读 < N 批的聚合值, 确定性护栏——批序串行 ⇒ 同输入同配置可复现; 校准器由 LLMClient 自持, 公开面 llm.calibrator, 3.9)。启动期预算 INFO 行 (V13①): M10 于运行起点打印预算参数 (如 segment: w_min=6 window=20 (budget) ——数据无关、仅计数与参数; 归属 M10 启动段而非 loader——加载期 logging 尚未按 CLI 覆盖定级, 7.1)。报表汇总: finalize 时组装 report.budget = {profiles, w_min, truncations, overflow_records, image_cost, degrade_retries, escalations} (counts-only 键义见 6.4; truncations 由各算子逐裁剪点计数、overflow_records 按 7.6 词表归集、image_cost/degrade_retries/escalations 为 V17 三层的校准终值与反应频度对账, V13②⑤) + report.stream.windows (segment 实际窗数, M14 属主计数、随 stream 节落盘——供用户对账 V12 上界估算, V13④, 6.4)。
帧粒度 (v1.12)	组链或门 (裁决·组链双门): factory (build_stages) 或以门 classify.enabled v frame_classify.enabled 决定 ClassifyStage 进链 (链序与槽位不变——仅帧级开启时 ClassifyStage 仍须进链执行帧 pass; 组链的 classify 槽位判定与该或门同口径); stage 内序列级判决单独受 classify.enabled 门控——仅帧级开启时序列记录不产生 Classification、_meta.classification 维持 null (3.13.7)。estimate_run 两键: frame_classify_calls / frame_annotate_calls = 粗上界 = 预扫描帧总数 Σ session_lens (数据源与 segment_calls 完全同源, 复用同一次预扫描; 帧分类实际按窗批量、帧标注跳过噪声成员与跳过类, 实际调用数均 ≤ 帧总数); 对应开关关闭 ⇒ 0 (帧粒度要求流模式 (3.1.4), 非流分支恒 0); total_calls 扩项; 键序冻结—— frame_classify_calls 紧跟 classify_calls、frame_annotate_calls 紧跟 annotate_calls (返回键表冻结注释同步, CONTRACTS §7.13)。dry-run 估算行改写: 估算行 (stderr 第 2 行) 按冻结键序插入两键, 无条件打印 (非流工程恒 = 0, v1.9 stitch_calls 先例) ——是改第 2 行而非加行: 五个既有 dry-run golden 重采 + examples/mix 主/姊妹双工程的 dryrun-mix.txt / dryrun-mix-text.txt 两个新 golden, 共七个 (7.8 回归锚, tests/cli/goldens/)。
时间流生成 (v1.13, 仅 generate_stream.enabled = true)	驱动分支: generate_only 分支在 _run_generate 入口按本开关分流——时间流形态调 generate_stream_all(ctx0) 恰一次 (同为 guarded task 形态: SIGINT 的 30 s 计时器可取消, 耗时照记 timing.per_stage.s.generate; 中断即无产物、无工件、无批次, finalize 照常标 interrupted=true), 平面路径 (种子池 / 无种子两形态) 代码零改动。成功返回后依次: ① 工件先落盘 (本表工件属主行) ② counts.generated = 进链序列条数 (—limit 已在 M6 计划期配额层截断, 此处 belt & braces 再截一次) ③ 直装信封按 run.batch_size 切批走 _compose_chain(include_generate=False) 的链 (M3→M13→M4→M5→M7→M11) ——信封整只交付, 绝不 PipelineItem(record=r) 裸构造重建 (会丢 session_id / classification / member_classifications 三项)。工件属主: M6 定稿行内容、M11 持有通道与交付纪律、M10 只在两者之间转手一次 (emitter.write_stream_artifact(lines), 先于任何批派发; finalize 时与主输出同批 fsync + 原子改名, 3.11.2); report.run 的工件条目 (路径 / sha256 / 行数) 由 M10 在组报告时自 emitter 读取——仅工件实际写入时在场 (dry-run 与形态关闭恒缺席)。estimate_run 精确复演: generate_only 分支改复用 M6 计划期纯函数 plan_stream(cfg, Random(f"{seed}:0:generate")) (吃 cfg + seed, 精确复演长度与噪音采样, 非上界) —— records = Σsequences (limit 后)、generate_calls = 2 × records + [噪音帧数 / num_per_call] (蓝图 + 实现 + 噪音批三类全折入既有键)、classify_calls = 0 (标签 inherited, 零判决调用, v1.7 R11 幂等哲学)、quality/annotate/verify 基数 = records、batches = ceil(records / batch_size); 估算行格式零改动 (不加键; console 的 _ESTIMATE_CALL_KEYS / _STAGE_CALL_KEYS 与面板行零改动) ——既有七个 dry-run golden 字节不动, 仅新增第八个 dryrun-synth-stream.txt (7.8 回归锚)。report 子块: report.generate 增 stream (counts-only 12 键, 键集与键序冻结; sequences 直方图按 [[classify.classes]] 声明序零基铺开——report.classify 同款; 计数面由 M6 以 generate.stream.* 前缀供给); report.stream 节不出现 (那是 segment 的观测面); report.classify 直方图恒全零属预期 (inherited 不经判决路径计数)。
序列规则与日历窗口联合规划 (v1.16, 仅时间流生成的实际约束面)	M10 只负责选择同一个 M6 入口与接收富返回, 不实现规则、窗口或 CP-SAT 业务算法。estimate_run 与 M6 共用 planner 的问题构造; 每个 attempt 恰一次 randrange 循环长度偏好, 全流由单个联合 CP-SAT 模型恰一次 solve。因此 dry-run 与真实运行共享配额前缀、solver seed 和长度冻结语义。规则/窗口在实际非零配额类生效时, M6 在 LLM 派发前冻结全部 word、owner session、任务 timestamp 与 noise 槽, 随后返回直装信封与时间流工件; 无约束时保留 v1.15 默认路径。M10 继续先写工件、再以原信封切批回流下游链, 不重建信封。M1 的 INFEASIBLE / UNKNOWN 是启动期 CONFIG_ERROR, MODEL_INVALID 或运行期 planner 不变量破坏是 InternalError / 退出码 4; M10 不新增 stage、trace channel 或错误 kind。report 装配只在规则/窗口实际生效面插入 v1.16 counts-only 子块, 不复制配置内容。

背书: 「编排器只做组合调度、算子无相互依赖」是 Data-Juicer 配方执行器 [4] 与 distilabel Pipeline 运行时 [5] 的共同架构; 批式流转 + 增量写出与 Dolma toolkit 的并行分片处理模型一致 [6]。

3.10.4 运行走查示例

走查前提（贯穿示例·文本模式）：输入为输入法采集的中文指令 JSONL 共 1000 行、无坏行（首行 `{"instruction": "帮我写一条请假条，明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}`），`input.text_field = "instruction"`；`run.output = "./out/ime-intent-0630.jsonl"`；`run.batch_size = 256`、`run.seed = 0`；阶段为 2.3.1 的默认组合（`dedup.enabled` ✓、`quality.enabled` ✓、`annotate.enabled` ✓，`generate.enabled = false`、`verify.enabled = false`）。`quality` 取默认 `mode="pairwise"`、`rounds=4`、`criteria_per_call="all"`、`rubric="default:text"`（4 条 criteria，附录 A.1），并显式设 `quality.threshold = 0.3`；`annotate` 输出用户 Schema 为意图标注对象 `{intent, topic, difficulty}`（`additionalProperties:false`、三字段全 required）；`output.rejects = "refs"`（默认）、`dedup.scope = "global"`（默认）。M2 惰性流被 M10 切为 4 个批：256 / 256 / 256 / 232（尾批不足 `batch_size` 照常处理，3.10.3）。

批	取入 →PipelineItem	M3 dropped_dup	M4 dropped_lowq	M5 failed	写出 emitted	批未释放的中间态	DedupIndex 签名 累计
1	256	18	21	3	214	256 个 PipelineItem；476 次裁决 / 1904 条比较结果	238
2	256	24	19	4	209	256 个 PipelineItem；464 次裁决 / 1856 条比较结果	470
3	256	27	20	5	204	256 个 PipelineItem；456 次裁决 / 1824 条比较结果	699
4（尾批）	232	28	18	2	184	232 个 PipelineItem；408 次裁决 / 1632 条比较结果	903
合计	1000	97	78	14	811	—	903

口径与自洽性：每批 `emitted = 取入 - dropped_dup - dropped_lowq - failed`（批 1：256 - 18 - 21 - 3 = 214）。M4 比较池 `N = 去重后存活数`（批 1 为 238），裁决调用数 = `k · [N/2]` = 4×119 = 476，`criteria_per_call="all"` 下每次调用裁决 4 条 criteria ⇒ 1904 条 criterion 级比较结果；批 3 的 `N=229` 为奇数，每轮末位轮空（3.4.3），故为 4×114 = 456 而非 458。进入 `annotate` 的记录数 = `N - dropped_lowq`：217 / 213 / 209 / 186（合计 825 = 811 emitted + 14 failed；14 条 failed 中 `schema_violation` 11 条、`provider_retryable_exhausted` 3 条）。批 1 `quality` 阶段的 `ctx.rng = Random("0:1:quality")`（3.10.3 派生式代入 `run.seed=0`、`batch_no=1`）。

批生命周期（图 3-4 的逐批实例）：每批 `emit()` 后 M11 向 `ime-intent-0630.jsonl.part` 追加 214 / 209 / 204 / 184 行并 flush，同时向 `ime-intent-0630.rejects.jsonl` 追加 42 / 47 / 52 / 48 行（= 该批 `dup+lowq+failed`，`refs` 模式每行仅 `_meta` 引用）；随后 M10 释放该批全部中间态——上表「批未释放」列的 PipelineItem 与比较结果，外加 4 组 BT log 0 数组（每 criterion 一组、长度 N）；文本模式无图像引用，释放数为 0。跨批存活的仅 3.10.3 所列三项，其规模变化：

跨批状态	批 1 → 2 → 3 → 4 末的规模
① DedupIndex	签名条目 238 → 470 → 699 → 903（= 1000 - 97；每条目为 exact sha256 + 128-perm MinHash 签名，文本模式无 pHash 表）。注意被 lowq/failed 淘汰的记录也已入索引——M3 在先，first-writer-wins。
② MetricsSink 计数器	emitted 214 → 423 → 627 → 811；dropped_dup 18 → 42 → 69 → 97；dropped_lowq 21 → 40 → 60 → 78；failed 3 → 7 → 12 → 14。仅整数计数，无数据内容。
③ M9 用量累计	LLM 调用 693 → 1370 → 2035 → 2629（每批 = 裁决 + 标注调用，不含重试与 L3 修复）。

流耗尽后 `finalize()`：`.part` fsync 并原子改名为 `ime-intent-0630.jsonl`（811 行），写 `ime-intent-0630.report.json`，其 `counts` 节：

```
"counts": {"scanned": 1000, "ingested": 1000, "bad_input": 0,
           "dropped_dup": 97, "dropped_lowq": 78, "dropped_verify": 0,
           "failed": 14, "generated": 0, "emitted": 811}
```

按 6.4 不变量 `emitted + dropped_* + failed + bad_input = scanned + generated` 验算：811 + (97 + 78 + 0) + 14 + 0 = 1000 = 1000 + 0，等式成立。`run()` 返回的 `RunSummary` 与上述 counts 一致：4 批全部完成、主输出 811 行、rejects 189 行（97+78+14）、`interrupted = false`——CLI 以退出码 0 结束（2.4：存在被拒绝记录不影响退出码；若本次带 `--strict`，则因 189 条 rejects 返回退出码 1）。

提示：本走查中 `dropped_lowq = 78` 依赖显式设置 `quality.threshold = 0.3`；该键默认缺省 = 不过滤只打分（5.2），若不设阈值则 `dropped_lowq = 0`，903 条唯一记录将全部进入标注，emitted 相应变为 903 - failed。

3.11 M11 输出器 emitter

3.11.1 职责与边界

做：三通道写出——主输出 JSONL（增量追加、终检、原子改名交付）、rejects 通道、report.json；组装 `_meta`；stderr 进度与结束摘要。**不做：**不修改标注内容；不做结构校验之外的任何数据加工（写出前调用 `SchemaEngine.validate_only` 做最后一道终检，失败即 bug——fail loudly，转 rejects 并记 `internal_error`）；报告不含数据内容。

3.11.2 写出规格

通道	规格
主输出	运行期写 <code>{output}.part</code>，每批 flush；finalize 时 fsync + 原子 rename 为目标名。行格式见 6.3。仅 <code>status="active"</code> 且（annotate 启用时）标注成功的记录写入。v1.6：熔断中止（退出码 4）的 finalize 同样执行交付（3.10.3 熔断交付）——已交付文件中每一行恒完整合法，运行是否完整处理了全部输入以 <code>report.run</code> 判定（<code>interrupted=false</code> 且 <code>circuit_broken=false</code>，3.11.3 ④）。v1.8：<code>status = "absorbed"</code>（成员帧已并入 episode，3.14）为第三路由——主输出与 rejects 均不写、仅计数（成员内容以 <code>members</code> 引用随其 episode 的序列行落盘）；分发规则变为 <code>active</code> → 主输出、<code>absorbed</code> → 仅计数、其余非 <code>active</code> 状态 → rejects。v1.9：<code>status = "stitched"</code>（被并 episode 壳，3.16）为第四路由——同 <code>absorbed</code> 仅计数（其成员随幸存线索信封的序列行落盘）；分发规则变为 <code>active</code> → 主输出、<code>absorbed</code> / <code>stitched</code> → 仅计数、其余非 <code>active</code> 状态 → rejects——壳不得落入兜底 <code>else</code> → rejects 分支（否则以 <code>internal_error</code> 之名污染 rejects 且 <code>--strict</code> 必退 1）。<code>_meta</code> 增恒在键 <code>stream</code>（未启用 <code>segment = null</code>；键位在 <code>source</code> 之后、<code>scores</code> 之前——链序镜像；结构见 6.3）；<code>stream</code> 模式下 <code>_meta.verification</code> 另含恒在 <code>defects</code> 键（无缺陷 = [], 6.3）。v1.9：stitch 启用时 <code>_meta.stream</code> 另含 <code>thread_id</code> / <code>fragments</code>（含 <code>order_span</code> 包络规范句）/ <code>steps</code> 行内 <code>resumed</code>（三者仅启用时在场——<code>off</code> 时行内容与 v1.8 逐字节等价，6.3、3.16.4 退化锚）。stderr 进度与结束摘要不增键（fanout 先例——<code>episodes</code> / <code>absorbed</code> / <code>dropped_noise</code>（及 v1.9 <code>stitched</code> / <code>threads</code>）经 <code>report</code> 与 <code>batch.end</code> 事件可见，3.10.3；v1.10 U18：不增键约束限 plain 面——rich 面板状态账展示 <code>stitched/threads</code>，7.7）。v1.10 console 让位（U21）：<code>_progress</code> / <code>_print_summary</code> 的行格式改经 common 层纯函数 <code>console_format</code>（输出与 v1.9 硬编码逐字节一致，黄金快照钉死）；两方法加静态门 <code>console.mode_resolved == "rich"</code> 时直接 return——plain 档原样执行（回归锚），rich 档信息由 CLI 层面板超集覆盖（终版摘要换渲染器表格版，数值来源不变，7.7）；中途降级 / q 脱离后的 plain 行由渲染器以同一 <code>console_format</code> 续打。v1.12 <code>members</code> 块（任一帧开关启用时在场）：<code>_meta.stream</code> 增 <code>members[]</code>——冻结位 = <code>member_sources</code> 之后、<code>session_split</code> 之前（两处有序精确断言测试同步该位置）；逐成员按 <code>rec.members</code> 序各一条目，条目字段序冻结 <code>index, id[, label][, annotation, status]</code>（<code>index</code> 0 基 = 成员在序列中的位次，<code>id</code> = 成员 <code>record.id</code>）。在场规则：<code>label</code> 键仅 <code>frame.classify.enabled</code> 时在场（<code>member_classifications</code> dict 为 <code>None</code> 或缺键 ⇒ <code>null</code>——覆盖降格跳过）；<code>annotation</code> / <code>status</code> 两键仅 <code>frame.annotate.enabled</code> 时在场。status 闭集三值判定（单一真相 = <code>member_annotations</code> dict 形态，3.5.5）：dict 缺键（或 dict 为 <code>None</code>）⇒ <code>"skipped"</code>（<code>annotation=null</code>）；值 <code>None</code> ⇒ <code>"failed"</code>（<code>annotation=null</code>）；对象 ⇒ 写前 <code>validate_only(obj, schema=frame_schema)</code> 兜底（3.8.2）——通过 ⇒ <code>"annotated"</code>，不通过 ⇒ <code>"failed"</code> + <code>annotation</code> 置 <code>null</code> + <code>frame.annotate.failed</code> 计数（非法帧对象零落盘；成员失败不改信封状态、不写 <code>item.errors</code>）。沉没成本记账：终态非 <code>active</code> 的序列信封仍携带 <code>member_annotations</code> ⇒ 按非 <code>None</code> 条目数累计 <code>frame.annotate.discarded</code>（已产出未交付，仅计数、不落盘，6.4）。<code>_meta</code> 顶层键序、四路由互斥、守恒恒等式零改动。v1.13（时间流生成形态，<code>generate_stream.enabled</code>）三处修订、其余零改动：① 写前终检按行取 Schema——<code>validate_only</code> 的 Schema 实参改取该行的类有效 Schema（<code>item.classification.label</code> → 该类声明的标注 Schema 覆盖，未分类 / 未知类 / 该类未声明 ⇒ 走全局 <code>output.schema</code> 的既有缺省路径，字节等价 v1.12；multi 扇出的兄弟信封各带自己的标签，按行天然对齐）；本模块不跨算子导入 M5 的取值函数，按 §2.2 依赖方向在内部持一份最小镜像（两侧取值语义须一致，测试钉住）。② <code>_meta.stream</code> 块的门扩为 <code>segment.enabled v generate_stream.enabled</code>（<code>members[]</code> 在场门与其中的 <code>label</code> 列门同步扩）——直装序列行原样复用该块：<code>order_span</code> 与 <code>member_sources</code> 指向工件路径与行号（<code>order_span</code> 的两端渲染为 <code>"<工件路径>:<行号>"</code>，<code>member_sources</code> 每项为 <code>{file: <工件路径>, line_no: <行号>}</code>——既有渲染器直接可用）、<code>members[]</code> 条目 = <code>{index, id, label}</code>（<code>label</code> 取 <code>member_classifications</code> 的帧类真值；本形态与 <code>frame.annotate</code> 互斥 ⇒ 无 <code>annotation</code> / <code>status</code> 两列，v1.12 的条件列规则相容）、<code>session_split=false</code>、<code>repaired=false</code>、<code>degraded=null</code>、<code>steps=null</code>、无 <code>thread_id</code> / <code>fragments</code>。③ rubric 镜像——<code>_meta.run.rubric</code> 的空选择器解析镜像同步扩为 <code>segment.enabled v generate_stream.enabled ⇒ "default:trajectory"</code>（与 loader 两处同改，3.1.4 S29 扩展）。零改动（显式声明）：<code>_meta</code> 顶层键序、四路由互斥（本形态只出现 <code>active</code> 与 <code>dropped_*</code> / <code>failed</code> 两类去向——无 <code>absorbed</code> / <code>stitched</code>：成员帧根本不构造信封）、守恒恒等式（取 <code>generate_only</code> 退化形，6.4）。
时间流工件 (v1.13)	第五输出通道，仅时间流生成形态出现：<code>write_stream_artifact(lines)</code> 把 M6 交付的、按交织序定稿的工件行写入 <code>{output_stem}.stream.jsonl.part</code>（写入 + flush；路径规则 = 主输出路径去末级后缀 + <code>.stream.jsonl</code>，与 M6 各自推导同一值——算子间不互导，两侧等式由测试钉住），finalize 时与主输出同批 fsync + 原子改名（3.11.3 ④）的时间线对工件同样成立：目标名出现即每行完整合法）。共用 <code>_undeliverable</code> 纪律——通道不可写或写失败即置位，此后 finalize 一律不改名（半截工件永不冒充成品，exit 4 家族）。dry-run 天然不触达（<code>_run_dry</code> 不驱动生成、不开 emitter 通道）。写入同时冻结 <code>report.run.artifact = {path, sha256, lines}</code>（sha256 按落盘字节计，<code>config_digest</code> 同款前缀形态；M10 组报告时读取，6.4）。行内容由 M6 定稿——本模块不生成、不改写、不校验工件行（格式规范见 6.5）。

rejects	<code>output.rejects = "none" \ "refs" (默认) \ "full"</code>。refs: 每行仅 <code>{"_meta": {id, source, stage, reason, errors}}</code> ——不含数据内容 (source 亦不含 <code>passthrough_fields</code>, 其值属数据内容), 贴合不存储原则; full: 额外含记录内容与最后一版非法输出 (调试用, 用户显式选择)。文件名 <code>{output_stem}.rejects.jsonl</code>。v1.7: <code>classify</code> 启用时 <code>_meta</code> 增 <code>label</code> 键 (= 该信封路由标签; multi 扇出下同 <code>id</code> 的兄弟信封由此消歧, 行唯一键 (<code>_meta.id, label</code>), 6.3), refs / full 两档均携带。v1.8: <code>dropped_noise</code> 行按翻转方留下的 duck-typed reason 标记归因分流 (此类帧不写 <code>item.errors</code>, 「归因取 <code>item.errors[0]</code>」规则无从服务), (stage, reason) 组合恰增三种——(<code>"segment"</code>, <code>"noise"</code>) (LLM 判噪声帧)、(<code>"segment"</code>, <code>"below_min_len"</code>) (短段丢弃帧, 独立于 noise, S11)、(<code>"verify"</code>, <code>"off_task_member"</code>) (修复收缩弃帧, S31); absorbed 信封永不入本文件 (第三路由)。--strict 交互: stream 工程的噪声帧属预期产物, 会因 rejects 非空退出 1 (6.4)。v1.9: (stage, reason) 组合再增一种——(<code>"stitch"</code>, <code>"stitch_invalid"</code>) (仅 <code>stitch.on_error = "fail"</code> 时出现: 判定修复耗尽的 episode 候选信封, 3.16.6); stitched 壳与被救援帧 (<code>dropped_noise</code> → absorbed 翻转) 永不入本文件——--strict 补注: 同输入开启 stitch 后 (短段被救援而不再落 rejects) strict 结果可能由 1 变 0, 属预期 (2.4、3.16.6)。full 档对序列 Record 的 <code>record</code> 载荷输出 <code>{"kind": "sequence", "member_ids": [...], "member_sources": [...]}</code> (S25; <code>kind="single"</code> 载荷形态不变); <code>raw_last_output</code> 仍仅 <code>schema_violation</code> 行携带——<code>classification_invalid</code> / <code>segmentation_invalid</code> / <code>extraction_invalid</code> 失败行不带原始输出 (<code>classify</code> 起的既有缺口, 明文接受)。v1.11: reason 词表再增两值——<code>context_overflow</code> (上下文预算三形态: 预检/最小单元不装/反应态降级耗尽, V10/V16/V24) 与 <code>output_truncated</code> (响应以输出上限截断收尾终局化, V11); stage = 产生该错误的属主算子 (任何 LLM 调用阶段皆可出现, 语义与熔断矩阵见 7.6); refs / full 档行形态不变, <code>raw_last_output</code> 门不扩 (两 kind 均不携带原始输出, 同上缺口口径)。v1.12: rejects 面零改动——行键集维持既有闭集 (refs 五键 / full 六键, 有序精确断言), (stage, reason) 组合、reason 词表均不增; 帧标注失败的成员不产生 rejects 行、不触发 --strict (成员失败非信封失败——只落 <code>members[]</code> 条目 <code>status="failed" + frame_annotate.failed</code> 计数, 裁决·成员失败不入 rejects; --strict 读信封状态计数, 帧失败不改信封状态)。
report.json	<code>{output_stem}.report.json</code>。结构见 6.4: 运行参数摘要 (脱敏, 无 key)、各阶段计数、分数分布直方图、去重簇统计、结构引擎各层命中、token/成本、耗时、失败分类计数。v1.7: <code>classify</code> 启用时增 <code>classify</code> 节 (assignment / 逐类分布 / <code>fallback_count</code> / <code>failures</code>, multi 另含 <code>multi_label_records</code>) 与 <code>quality.by_class</code> 按池视图, multi 时 counts 增 <code>fanout</code> (6.4)。v1.8: <code>segment</code> 启用时 counts 增 <code>episodes</code> / <code>absorbed</code> / <code>dropped_noise</code>, counts 之后新增 <code>stream</code> 节 (<code>sessions</code> / <code>episodes</code> / <code>mean_episode_len</code> / <code>absorbed</code> / <code>dropped_noise</code> / <code>below_min_len</code> / <code>digest_poor_frames</code> / <code>segment_failures</code> + <code>extract</code> / <code>verify</code> 可选子块, 6.4)。v1.9: <code>stitch</code> 启用时 counts 增 <code>stitched</code> / <code>threads</code> (= <code>episodes</code> - <code>stitched</code> 导出式, 3.10.3), <code>stream</code> 节增 <code>stitch</code> 子块 (<code>stitched</code> / <code>rescued_short</code> / <code>seams</code> / <code>judgments</code> / <code>repass_judgments</code> / <code>failures</code>, 6.4); 全部 v1.9 新键仅启用时在场——off 时本模块三通道产物与 v1.8 逐字节等价 (3.16.4 退化锚; 唯一报告面例外 = <code>streamxverify</code> 缺陷词表 <code>wrong_stitch: 0</code> 行, 词表为 3.7.2 四处同步闭集、不随开关条件化)。v1.11: 上下文预算启用时增 <code>report.budget</code> 节 (<code>profiles</code> / <code>w_min</code> / <code>truncations</code> / <code>overflow_records</code> / <code>image_cost</code> / <code>degrade_retries</code> / <code>escalations</code>——counts-only, M10 汇总, 3.10.3、6.4), <code>stream</code> 节增 <code>windows</code> (segment 实际窗数, M14 属主——供对账 V12 上界估算, 6.4); 预算全体未声明时两处不在场, 报告与 v1.10 逐字节等价。
v1.16 规则/ 窗口联合面	M11 不解释或复制规则、窗口、correlation、planner 状态和 sequence-validator 结果。M6 交付的时间流 artifact 仍由第五通道原样写入, 主输出仍按既有类有效标注 Schema 写前 <code>validate_only</code>; 整条 attempt 的 planner、realize、sample-validator、declarative 或 sequence-validator 作废都不会构造 failed 信封, 因此不会被 M11 重新归因。report.generate.stream 的 rules、可选 validator 计数和 windows 子块由 M10 装配; M11 只写 report, 不新增 truth、_meta.stream、Record id 或 artifact 行字段, 规则配置不进入任何输出。

背书: 「主数据 + 拒绝通道 + 统计报告」三分法是 NeMo Curator / Dolma 管线产物的通行组织 [6][9]; 原子改名交付为数据工程防半载文件的标准手法。

3.11.3 输出示例

贯穿示例沿用 6.1 的输入法中文指令数据 (`input.text_field = "instruction"`, 用户 Schema 为意图标注三字段: `intent` / `topic` / `difficulty`, 全部 required、`additionalProperties:false`), 运行设定与数字全部沿用 3.10.4 走查 (`run.output = ".out/ime-intent-0630.jsonl"`、`run.batch_size = 256`、`run.seed = 0`、`quality.threshold = 0.3`、`verify.enabled = false`; 其 1000 行输入文件此处记为 `ime-2026-06.jsonl`)。

① 主输出: meta_mode = "sidecar" 的一对行

`meta_mode = "inline"` 的完整行示例见 6.3, 此处不重复。当 `output.meta_mode = "sidecar"` 且 `output.passthrough_fields = ["source"]` 时, 主输出行为纯用户结构, `_meta` 逐行写入 `out/ime-intent-0630.meta.jsonl`, 两文件以 `_meta.id` 与行序对齐 (主输出第 k 行 ↔ meta 第 k 行)。下例对应输入 `ime-2026-06.jsonl` 首行 `{"instruction": "帮我写一条请假条, 明天上午要去医院", "source": "ime-log", "ts": "2026-06-30T10:12:00Z"}`; 每行写出前均经 `SchemaEngine.validate_only` 终检 (3.11.1)。


```
# — out/ime-intent-0630.jsonl 第 1 行（纯用户结构，无 _meta 键，剥无可剥）—
{"intent": "writing_assist", "topic": "请假条", "difficulty": "easy"}

# — out/ime-intent-0630.meta.jsonl 第 1 行（实际为单行 JSONL，此处折行排版）—
{"_meta": {
  "id": "1cda030abc565f17",
  "run": {"tool": "labelkit/1.0.0", "started_at": "2026-07-02T10:27:41+08:00",
    "project_file": "project.toml", "rubric": "default:text", "seed": 0},
  "source": {"file": "ime-2026-06.jsonl", "line_no": 1,
    "generated_from": [], "fields": {"source": "ime-log"}}, // passthrough_fields 落点
  "stream": null, // v1.8 恒在键：未启用 segment = null (6.3)
  "scores": {"writing_style": 0.72, "facts_trivia": 0.44, "educational_value": 0.61,
    "required_expertise": 0.35, "__aggregate__": 0.53, // 等权均值 = 2.12/4
    "mode": "pairwise_bt", "batch_no": 1},
  "dedup": {"kind": "unique"},
  "annotation": {"model": "qwen2.5-vl-72b-instruct", "attempts": 1}, // attempts=1: 未触发 L3
  "verification": null // verify 未启用
}}
```

v1.8 序列行说明：stream 工程（`segment.enabled = true`）的主输出行以 episode 为单位，其 `_meta.stream` 携带完整成员溯源与步骤序列（`episode_id / session_id / member_ids / member_sources / steps` 等，结构与样例见 6.3），行仍以 `_meta.id`（= 序列 Record id）对齐。

v1.12 members 块示例（摘自 `examples/mix` UI 主工程真跑主输出 `out/mix-labels.jsonl` 的一条 episode 行，实际为单行 JSONL，此处折行排版、长值截断；该工程 `frame.classify`（DeepSeek digest-only）与 `frame.annotate`（z.ai 视觉）双开、`[frame.class.transition.annotate].enabled = false`——index 4 成员即跳过类的 skipped 形态）：

```
{
  "task_label": "在美食外卖App上点一份黄焖鸡米饭", "app": "com.example.food",
  "summary": "在美食外卖App搜索并下单金牌黄焖鸡大份微辣米饭，支付¥38后等待送达。",
  "_meta": {
    "id": "eccc814c0694fb41", ...,
    "stream": {
      "episode_id": "eccc814c0694fb41", "session_id": "eccc814c0694fb41",
      "order_span": [1, 6], "member_count": 6,
      "member_ids": ["7cfb0c25f855b2d7", "164b7480ab098de5", ...],
      "member_sources": [{"file": "s1-food-order/uitree_1.jsonl", "pair_index": 1}, ...],
      "members": [ // ← member_sources 后、session_split 前（冻结结）
        {
          "index": 0, "id": "7cfb0c25f855b2d7", "label": "list_screen",
          "annotation": {"screen_role": "美食外卖首页",
            "key_widgets": ["搜索美食", "搜索", "推荐餐厅", "金牌黄焖鸡 4.9 分", ...]},
          "status": "annotated"},
        {
          "index": 1, "id": "164b7480ab098de5", "label": "detail_screen",
          "annotation": {"screen_role": "菜品详情页",
            "key_widgets": ["金牌黄焖鸡", "黄焖鸡米饭 ¥38", "月售 1200+ 好评率 99%", ...]},
          "status": "annotated"},
        {
          "index": 2, "id": "25ce67ce53d5f1d7", "label": "form_screen",
          "annotation": {"screen_role": "商品规格选择/加入购物车", // [frame.class.form_screen.annotate]
            "key_widgets": ["商品：黄焖鸡米饭 ¥38", "份量：大份", "辣度：微辣", ...]},
          "status": "annotated", // 覆盖指令：抽取表单字段与取值
        },
        {
          "index": 3, "id": "d77a51064a52f91e", "label": "confirm_screen",
          "annotation": {"screen_role": "订单确认页",
            "key_widgets": ["确认订单", "黄焖鸡米饭 大份 ×1", "提交订单 ¥38", ...]},
          "status": "annotated"},
        {
          "index": 4, "id": "96cb96ed666583b1", "label": "transition",
          "annotation": null, "status": "skipped", // 帧类视图 enabled=false ⇒ 缺键推导 skipped
        },
        {
          "index": 5, "id": "347864af1bc54006", "label": "confirm_screen",
          "annotation": {"screen_role": "支付成功结果页",
            "key_widgets": ["支付成功", "订单号 FD20260812001", ...]},
          "status": "annotated"}
      ],
      "session_split": false, "repaired": false, "degraded": null, "steps": null}}
}
```

② rejects = "refs"（默认）的一行

场景：第 213 行的标注输出经 L3 两次修复（`output.max_repair_attempts = 2`）仍未通过用户 Schema，M8 抛 `SchemaViolation`，记录置 `failed`（`kind = schema_violation`，7.6）转入 `out/ime-intent-0630.rejects.jsonl`。errors 即 M8 L2 `iter_errors()` 收集的全部违规（JSON Pointer 路径 + 期望 + 实际，3.8.2；违规描述为英文——枚举类由 M8 自渲染，其余直接携带 jsonschema 的原始消息，2026-08-14 起统一）；行内无任何数据内容——记录原文与 `raw_last_output` 仅在 `rejects = "full"` 时才写出。


```
# — out/ime-intent-0630.rejects.jsonl 中的一行（折行排版） —
{"_meta": {
  "id": "c47d09e2b8a1f350",
  "source": {"file": "ime-2026-06.jsonl", "line_no": 213, "generated_from": []},
  "stage": "annotate",
  "reason": "schema_violation",
  "errors": [
    "/difficulty: expected one of enum [\"easy\", \"medium\", \"hard\"], got \"非常难\"",
    "/: Additional properties are not allowed ('confidence' was unexpected)"
  ]
}}
```

v1.7: classify 启用的工程中该 `_meta` 另含 `"label"` 键 (3.11.2 rejects 行); 本例工程未启用 classify, 无此键。v1.8: stream 工程另见三种 (stage, reason) 组合的 rejects 行——segment/noise、segment/below_min_len、verify/off_task_member (3.11.2) ——refs 档形态同本例 (仅 `_meta` 引用行; 此类帧不写 item.errors, `errors` 恒 [])。

③ 运行结束 stderr 摘要（逐字样例）

非 TTY、`--log-level info` 下的运行尾部。行格式为 7.3 的 `ts level stage batch msg` (日志正文自 2026-08-14 起统一英文, 键名、结构与信息集不变); 数字即 3.10.4 走查: 1000 条无坏行入流水线 (4 批: $256 \times 3 + 232$), 尾批写出 184 行、失败 2 条; rejects 通道含重复 / 低质 / 失败三类 (`output.rejects = "refs"`, 3.11.2)。

```
2026-07-02T10:41:22+08:00 INFO emitter batch=4 batch 4 flushed: main output +184 line(s) (total 811), rejects +48 (total 189)
2026-07-02T10:41:23+08:00 INFO emitter batch=- finalize: fsync + rename out/ime-intent-0630.jsonl.part -> out/ime-intent-0630.jsonl (811 lines)
2026-07-02T10:41:23+08:00 INFO emitter batch=- wrote out/ime-intent-0630.rejects.jsonl (189 lines) and out/ime-intent-0630.report.json
2026-07-02T10:41:23+08:00 INFO run batch=0 run.end exit_code=0
— final summary (matches report.counts item by item) —
scanned=1000 ingested=1000 bad_input=0 generated=0
dropped_dup=97 dropped_lowq=78 dropped_verify=0 failed=14 emitted=811
```

不变量自查 (6.4): $\text{emitted } 811 + \text{dropped } (97+78+0) + \text{failed } 14 + \text{bad_input } 0 = 1000 = \text{scanned } 1000 + \text{generated } 0$ 。尾批自查: $184 + 28(\text{dup}) + 18(\text{lowq}) + 2(\text{failed}) = 232$; 尾批 $\text{rejects} = 28+18+2 = 48$, 四批累计 $189 = 97+78+14$ 。

④ 原子改名交付时间线

运行全程只向 `out/ime-intent-0630.jsonl.part` 追加 (每批 flush); finalize 时 fsync 后一次 rename 为 `out/ime-intent-0630.jsonl`。因此目录中任一时刻要么只有 `.part` (运行中, 或未走到 finalize 的硬崩溃 / 输出路径不可写), 要么只有最终文件——目标文件名出现即保证已交付的**每一行完整且合法**, 永远不会读到半截行。v1.6 起熔断中止同样交付 (3.10.3 熔断交付), 「目标文件出现」因此不再等价「全部输入处理完毕」: 消费方判定运行完整性须看 `report.run: interrupted=false 且 circuit_broken=false` (退出码 0/1 不足——被 SIGINT 优雅中断的运行同样交付且以 0 退出, 3.10.3 中断行); 熔断交付的主输出是「已完成批的完整前缀」, 缺口可由 `counts.unprocessed` 核对 (6.4 不变量扩展)。

3.12 M12 日志 logging

3.12.1 职责与边界

做: 进程内唯一日志设施, 承载两条通道: ① **运行日志**——写 stderr, 级别 debug/info/warn/error, 行格式由 `tool.log_format = "text"` (默认) | `"jsonl"` 决定, 只记运维事件, **绝不含数据内容与提示词**; ② **trace 追踪日志** (可选, `trace.enabled=true` 时)——JSONL 文件 (默认 `{output_stem}.trace.jsonl`), 一行一事件的结构化事件流, 供 rubric 优化 (7.5) 与标注质量分析。事件目录与格式规范见第 7 章。**不做**: 不做跨运行聚合分析 (下游 / 后续 `labelkit analyze` 工具职责, 8.3 O5); 不上传任何遥测 (2.1.2 边界⑦); 写失败**绝不中断运行**——首次失败 warn 一次并关闭该通道, 此后事件丢弃并计入 `report.trace.dropped_events`; API Key 永不落日志 (任一通道、任一脱敏档位)。进度显示 (console, 7.7) 不属于日志——M12 仅以 3.12.3 的 `ProgressListener` 旁路向其**转发** (v1.10), 不承载其渲染。

3.12.2 输入 / 输出

方向	内容
输入	各模块经标准 <code>logging</code> 记录器提交的运行日志; 各 Stage 经 <code>EventLog.emit()</code> 提交的 <code>TraceEvent</code> ; <code>ResolvedConfig</code> 中的 <code>tool.log_level</code> / <code>tool.log_format</code> 与 [trace] 节。

输出	stderr 字节流: <code>trace.path</code> JSONL 文件 (首行恒为 <code>run.start</code> header 事件); <code>report.json</code> 的 <code>"trace"</code> 统计块 (6.4)。
----	--

3.12.3 API 与数据结构

```
@dataclass(frozen=True)
class TraceEvent:
    ts: str                # ISO8601, 毫秒精度, 含时区
    run_id: str            # 本次运行标识: 启动时 secrets.token_hex(6) 生成的随机 12 hex
    batch_no: int          # 运行级事件 (run.*) 为 0
    stage: str             # 发出事件的 stage 名; run.*/batch.* 固定 "run"
    ev: str                # 事件名 (7.2 事件目录)
    record_ids: tuple[str, ...] # 涉及的记录 id (0/1/2 条)
    payload: Mapping       # 事件负载: 7.2 逐事件定义, 经 7.4 脱敏

class EventLog:
    def emit(self, ev: TraceEvent) -> None:
        """行缓冲写入 trace 文件; 通道未启用或已因写失败关闭时为 no-op (调用方无需判断)。"""
```

接入方式: `EventLog` 由 `MetricsSink` 持有并转发——各 `Stage` 通过既有的 `RunContext.metrics` (3.10.3) 发事件, 不改 `RunContext` 签名。
stderr 侧直接使用标准 `logging` 模块, handler 由 M12 在启动时按 `log_format` 安装。

v1.10 增: `ProgressListener` 进程内旁路 (console 面板的唯一数据通路, 7.7; 实现归 CLI 层 `labelkit/cli/console.py`, 协议归本层——依赖方向 `cli` → `orchestration` → `operators` → `common` 不变):

```
class ProgressListener(Protocol):
    # v1.10 (7.7 console 的订阅协议, 五回调)
    def on_run_context(self, cfg, snapshot, counters, fatal_streak) -> None: ...
        # execute_run 装配完成后、asyncio.run 之前调用一次 (U19): cfg = ResolvedConfig;
        # snapshot = LLMClient.snapshot (3.9.2); counters / fatal_streak = MetricsSink 只读闭包。
        # 渲染器以「惰性壳」形态传入 (CLI 在 load 前无 cfg, 本回调完成激活)。
    def on_estimate(self, est: Mapping) -> None: ...
        # M10 预扫后经 MetricsSink.run_estimate 转发的 estimate_run() 静态估算 (3.10.3);
        # 文本模式未开 console.estimate 时不发 (U17)。
    def on_event(self, ev: TraceEvent) -> None: ...
        # MetricsSink.event() 旁路转发; payload 经 redact_payload(payload, "none") 预脱敏 (U22) —
        # 无 LLM 自由文本、无输入内容, U6 红线由机制保证; record_ids 保留 (结构字段)。
    def on_stage(self, stage: str, batch_no: int) -> None: ...
        # M10 stage 循环经 MetricsSink.stage_begin 转发 (每 stage run() 之前一次)。
    def on_stop_requested(self) -> None: ...
        # SIGINT/SIGTERM 经 MetricsSink.stop_requested 转发 (优雅中断横幅, 3.10.3)。
```

四条纪律: ① 旁路不属于 `trace` 面——五回调均不产生 `TraceEvent`、不受 `trace.channels` 过滤 (7.2 事件目录零改动的充分条件); `on_event` 的 `payload` 按 `none` 档预脱敏后转发 (仅 listener 非 `None` 时执行, 浅递归 `strip` 成本可忽略——U22)。② 全部回调必须 O(1)、无 I/O、无锁等待——重绘由消费方自己的节流 `tick` 驱动 (渲染与事件源解耦)。③ sink 侧异常防护 (U23): `MetricsSink` 每次转发 `try/except Exception` ——首次异常打一条 `WARN` 并置 listener 为 `None` (`EventLog` 写失败「warn 一次 + 关通道」同款纪律, 3.12.4), listener 异常永不进入记录级/批级失败路径。④ `listener = None` (`validate` / 全部既有调用路径) 时行为与 v1.9 逐字节一致。

API 增量 (均只增): `MetricsSink.__init__` 增可选尾参 `listener`; 增仅转发方法 `stage_begin(stage, batch_no)` / `run_estimate(est)` / `stop_requested()` 与两只读属性 `fatal_streak` (熔断行数据源)、`has_listener` (旁路在挂探针——M10 `dry-run` 让位门读之; U23 跳闸后永久 `False`)。plain 行格式 (进度行/终版摘要) 下沉为本层纯函数模块 `labelkit/common/observability/console_format.py` (U21——emitter 与 CLI 渲染器共用的单一事实源, 两串由 `golden` 快照逐字节钉死; 2026-08-14 全库英文化把这两串重冻结到英文文案上: 进度行 `\rlabelkit: batch {batch_no} emitted={emitted_total} dropped_dup=N dropped_lowq=N dropped_verify=N failed=N`、终版摘要头 — `final summary (matches report.counts item by item)` —; 键集、行结构与信息集不变, 仅语言变)。配套的 `LLMClient.snapshot()` 只读快照 (密钥池三态 + 逐密钥用量镜像 + p50 有界样本窗) 规格见 3.9.2; 全部签名已入册于 `CONTRACTS §7.8/§7.11/§7.12` (签名) 与 `§7.9/§7.10/§8.4` (行为与措辞)。

3.12.4 行为规格

机制	定义
通道过滤	事件按 7.2 归属通道, 不在 <code>trace.channels</code> 中的事件不写; <code>run.*</code> / <code>batch.*</code> 生命周期事件不受过滤。
schema 版本	<code>trace_schema_version = 1</code> 只写在文件首行 <code>run.start</code> header 事件的 <code>payload</code> 中 (避免每行冗余); 事件目录即稳定契约, 后续版本只增不改。
flush	行缓冲; 每批随 M11 flush (3.11.2) 同步 flush——保证主输出已落盘的行, 其 <code>trace</code> 事件必已落盘。

写失败	首次 <code>OSError</code> ⇒ <code>stderr</code> warn 一次 + 关闭该通道 + 后续事件计入 <code>report.trace.dropped_events</code> ；运行绝不因日志中断。
run_id	启动时生成、写入本次运行全部事件；用于多次运行的 <code>trace</code> 文件合并分析时区分来源。
文件语义	首个事件写出时 若 <code>trace.path</code> 已存在则截断覆盖并 <code>stderr</code> warn 一次 (v1.5 惰性打开：构造不碰文件，死于配置/输入校验的运行不触碰旧 <code>trace</code> ；保留历史请改名或另配 <code>trace.path</code>)。 <code>trace</code> 不做 <code>.part</code> 原子改名——它是逐批 <code>flush</code> 的流式日志，异常终止时已 <code>flush</code> 的行即有效前缀（首行仍为本次 <code>run.start</code> ）。
帧粒度事件 (v1.12)	两个新事件（7.2 目录两行；事件名前缀自动路由由既有 <code>classify</code> / <code>annotate</code> 通道， 通道枚举维持 11 值 ）：① <code>classify.frame</code> —— 每 episode 一发 (<code>ids = (episode_id,)</code>)， <code>payload = members</code> （判决成员数） / <code>windows</code> （分窗数） / <code>fallback</code> （兜底帧数）三计数（3.13.7）；② <code>annotate.frame</code> —— 每成员一发 (<code>ids = (episode_id,)</code>)， <code>payload = member_id / status</code> (<code>annotated skipped failed</code>) / <code>attempts</code> ，标注内容仅经既有 <code>excerpt</code> 键按档位分级（ <code>excerpt/full</code> 档 200 字截断，7.4）， 不新增任何承载数据内容的 <code>payload</code> 键 （ <code>none</code> 档预脱敏载荷直通 <code>console</code> 面板的红线，3.5.5）。

序列规则与联合规划 (v1.16)：规划器、声明式 `evaluator` 与序列钩子沿用既有运行日志 通道，不新增 `trace channel`、事件名或 `StageError.kind`。 `planner` 状态异常使用既有英文、 值无关的 `ERROR/WARN`： `INFEASIBLE` / `UNKNOWN` 由 M1 聚合为配置错误， `MODEL_INVALID` 或通过 M1 后的不变量破坏记录 `ERROR` 并交给既有 `InternalError` 退出面； `noise` 目标短缺只打一条值无关 `WARN`。序列钩子异常日志只含 `hook` 引用、异常类型，违规日志只含序列索引、类名和违规数；规则作废日志只含序列索引和类名。所有日志禁止载荷、输入文本、`prompt`、违规 `message`、时间表、相关字段值和 `API key`。规划器的 `CP-SAT` 求解经过已有 `llm.call` 统计面以外不产生新 `trace` 记录，`report` 计数由 M6/M10 供给。

3.12.5 配置项

见 5.1 `tool.log_format` 与 5.2 `[trace]` 节、`quality.judgment_reasons`。

背书：对每次 LLM 调用与判定做结构化追踪 (`trace`) 并以其驱动评测迭代，是 LLM 工程的工业标准形态：LangSmith (LangChain) [28] 与 W&B Weave [29] 均以「逐步 `trace` LLM 调用 + 数据集评测」为核心能力。`llm.call` 等事件的字段命名对齐 OpenTelemetry GenAI 语义约定 [27]——该约定截至 2026-07 处于 Development（实验性、非 `stable`）状态，本工具仅做命名对齐、不依赖其 SDK 实现（7.3）。

3.13 M13 分类 `classify` (v1.7)

3.13.1 职责与边界

做：按用户类别表（`[classify.classes]`，5.2）对批内 `status="active"` 且 `classification is None` 的记录做 LLM 封闭集 (`closed-set`) 分类：单/多标签可配（`classify.assignment = "single" | "multi"`），可选 `self-consistency` 投票（3.13.4）；分类结果写入 `item.classification`（4.1），供下游算子按类取有效配置（3.4.3、3.5.2、3.6.2、3.7.2）；`multi` 模式下按标签向批尾扇出兄弟信封（3.13.4 `multi` 扇出行）。链序位于 `dedup` 之后、`quality` 之前（3.10.3）——重复记录不消耗分类调用；「按类打分」要求类归属与按类 `rubric` 在打分前就绪。**不做**：不淘汰记录（分类不是质量门；`multi` 扇出只增不减）；不定义类别语义（类别表来自配置，同 `rubric` 信任级）；不做标注（分类是工具内部结构——内部 `Schema`、驱动管线行为、进 `_meta`；用户 `Schema` 产出物属 M5）；不改变链结构（扇出改变的只是批内信封基数，4.3 契约 ②a）。

3.13.2 输入 / 输出

方向	内容
输入	批内 <code>status="active"</code> 且 <code>classification is None</code> 的 <code>PipelineItem</code> （ <code>classification is not None</code> 者幂等跳过，3.13.4）；类别表与分类参数（ <code>[classify]</code> ，5.2）；LLM profile（ <code>classify.llm</code> ）。
输出	每条处理记录 <code>item.classification: Classification</code> （4.1： <code>label =</code> 本信封路由标签， <code>labels =</code> 命中全集， <code>source ∈ {"llm","fallback","inherited"}</code> ）； <code>assignment="multi"</code> 且归一化后命中 <code>k ≥ 2</code> 类时批尾追加 <code>k-1</code> 个兄弟信封；返回值 = 传入的同一列表对象（4.3 契约 ②a）。 <code>on_error="fail"</code> 且结构修复耗尽时该记录 <code>status="failed"</code> 、 <code>StageError</code> 入 <code>item.errors</code> 。

3.13.3 分类提示词与内部 Schema（确定性模板）

```
system:
  single: 你是数据分类员。阅读待分类数据，判断它属于以下类别中的哪一类。类别表：
  multi:  你是数据分类员。阅读待分类数据，判断它适用于以下哪些类别（至少 1 类，至多 {max_labels} 类）。类别表：
  - {name}: {description}                                ← 按 [[classify.classes]] 声明序逐类一行
  {classify.instruction}                                ← 可选补充说明；缺省省略此行
  输出必须是符合以下结构的单个 JSON 对象，不输出任何其他内容：
  single: {"class": <类名>[, "reason": <一句话理由>]}
  multi:  {"classes": [<类名>, ...][, "reason": <一句话理由>]}    ← reason 仅请求时出现于两式
user (对每条配置了 examples 的类，按声明序；类内按数组序)：
  [类别示例·{name}] {example}
user (当前记录)：
  文本模式：[待分类数据] {record.text}
  UI 模式：  [屏幕截图] <image: base64>
              [UI 控件树] {record.ui_tree.serialize(max_chars=input.ui_tree_max_chars)}
```

UI 模式的「当前记录」Part 组装与 3.5.2 相同（[屏幕截图] 为 kind="image" 的 Part，[UI 控件树] 为 kind="text" 的 Part，3.9.2），所引 profile 须 supports_vision = true（M1 校验，3.1.4）。类别示例 examples 仅输入侧——分类的输出格式由内部 Schema 钉死，无「示例输出」段。reason 的请求条件见 3.13.4 调用与校验行。

序列记录分支 (v1.8)。stream 模式下 episode（record.kind = "sequence"，3.14）作为普通记录被分类（对序列形态零崩溃），仅「当前记录」user 消息改走序列变体——system 与类别示例消息不变，段标签与截断标记行逐字冻结于 CONTRACTS §10.8 序列变体：text part [待分类数据·序列] = 成员逐帧 frame_digest（4.3）按成员序每帧一行拼接的 episode 摘要，**总量封顶 input.ui_tree_max_chars**——**首尾成员恒保留**、中段按**整条截断**、被截断时以标记行 ... (truncated N members) 收尾；UI 模式另附**首帧截图**（text part [首帧截图] + 首成员的 image part，M9 调用时编码，3.9.2）——classify 保持在 vision 引用集，所引 profile 仍须 supports_vision = true（3.1.4 vision 逐阶段表）；文本模式序列仅摘要段。

分类输出经 M8 内部 Schema 校验（schema_engine.classification_schema，3.8.1 内部 Schema 清单）。关键字集 ⊆ 既有内部 Schema 关键字集，**不写 uniqueItems**——该关键字会被 OpenAI strict 模式与部分约束解码网关硬拒（L0 无条件透传 Schema），重复标签改由本模块在 M8 验证后确定性归一化（3.13.4 归一化行）：

```
def classification_schema(class_names: list[str], assignment: str,
                           max_labels: int, with_reason: bool) -> dict:
    if assignment == "single":
        props: dict = {"class": {"type": "string", "enum": list(class_names)}}
        required = ["class"]
    else:
        props = {"classes": {"type": "array",
                              "items": {"type": "string", "enum": list(class_names)},
                              "minItems": 1, "maxItems": max_labels}}
        required = ["classes"]
    if with_reason:
        props["reason"] = {"type": "string"}
        required += ["reason"]
    return {"type": "object", "properties": props,
            "required": required, "additionalProperties": False}
```

3.13.4 算法规格

关键设计的精确定义：

设计点	定义
调用与校验	每记录 1 次调用（self-consistency 启用时 xn），经 complete_validated(schema=classification_schema(...))（3.8.3）——内部 Schema：不计入 report.schema_engine.resolved_at、不经过 L2.5（3.8.2）。reason 仅当 trace.enabled = true 且 trace.channels 含 "classify" 时请求（零额外 token 原则，对齐 quality.judgment_reasons 的 "auto" 语义；7.2）。temperature 恒 0（sc 采样取 classify.sc_temperature）。批内记录级并发（asyncio.gather + profile 信号量，骨架同 M5）。
归一化 (M8 之后，确定性，顺序固定)	① 标签映射到类别表声明序并 去重 ；② 兜底类与具体类同现 ⇒ 剔除兜底类（纯兜底保留——「其余」与具体类命中矛盾）。归一化只收窄已验证集合（词表合法性由内部 Schema enum 保证，3.13.3）。
sc 投票	classify.self_consistency = n（0 关；≥3 奇数，M1 校验）：n 次独立采样（某次采样 SchemaViolation ⇒ 该样本弃权，分母仍为 n）。single：多数票，无过半 ⇒ 归兜底类；multi：逐标签保留出现于 > n/2 个采样集合者，全落选 ⇒ 归兜底类。Classification.detail.sc = {"n", "agreement_ratio"}（single = 胜出类票占比；multi = 保留标签中最低票占比）。本投票不复用 3.5.2 的字段级投票——其「全体分歧回退首样本」语义不适用于分类（无过半应归兜底而非取首样本）。

失败与兜底	M8 修复耗尽: <code>classify.on_error = "fallback"</code> (默认) $\Rightarrow$ 归兜底类 <code>classify.fallback_class</code> , <code>source="fallback"</code> , 留痕写 <code>Classification.detail</code> (含 kind 与消息) ——不写 <code>item.errors</code> (记录存活; rejects 归因取 <code>item.errors[0]</code>), 写入会在该记录后续阶段失败时污染归因, 3.11.2) + error 事件 (kind = <code>classification_invalid</code> , 7.6) + 计数器 <code>classify.fallback</code> ; <code>on_error = "fail"</code> $\Rightarrow$ <code>status="failed"</code> 、 <code>StageError</code> 入 <code>item.errors</code> $\Rightarrow$ rejects。
multi 扇出	归一化后 $k \geq 2$: 原信封取首标签 (声明序), 其余 $k-1$ 标签各克隆一个兄弟 <code>PipelineItem</code> 原地追加到传入批列表尾部——克隆共享 <code>record</code> 与 <code>dedup</code> (引用共享, 零内容复制; 保证兄弟行 <code>_meta.dedup</code> 一致) 并继承 <code>session_id</code> (v1.8——兄弟序列信封对 M7 边界余量/邻域查询保持可寻址, 3.7.3) 以及 <code>thread_id</code> 与 <code>seam_indexes</code> (v1.9, <code>stitch</code> 启用时在场——复制清单增两项: <code>thread_id</code> 为真字段进克隆构造、 <code>seam_indexes</code> 为 duck 标进复制循环; 兄弟线索信封对 M15 接缝占位与 M7 [片段结构] 证据保持可用, 3.16/3.15.4/3.7.2), <code>classification</code> 换 label (labels 同为全集), <code>status="active"</code> , <code>scores / annotation / verification / errors</code> 为全新默认容器。追加序 = (原元素批内位置 $\rightarrow$ 标签声明序), 逐字节可复现。返回值 = 传入的同一列表对象 (4.3 契约 ②a)。此后每个信封与普通单标签记录完全同构——进各自类池打分、按各自类参数标注/评审、独立淘汰、独立产出一行 (行唯一键 = (<code>_meta.id</code> , label), 6.3), 下游算子对扇出零感知; 扇出净增数由 M10 计入 <code>counts.fanout</code> (3.10.3、6.4)。
multi x episode (v1.8, S9)	stream 模式下 multi 扇出照常作用于序列信封, 两点增量语义: ① 克隆兄弟的 <code>transitions</code> 恒为 <code>None</code> ——extract 链序在 <code>classify</code> 之后 (3.10.3), 每个兄弟按各自 label 的有效 [<code>class.<label>.extract</code>] instruction 独立摘取 (per-label 白名单承诺兑现; <code>transitions</code> 每信封自持, $\times k$ 摘取成本接受——episode 命中多类应属罕见: M14 边界判据即「单一目标导向活动」, 3.14.4; dry-run 沿本表 multi 惯例按乘数 1 报下界, 3.10.3)。② 共享语义边界声明: 本表「multi 扇出」行的「克隆共享 <code>record</code> 引用」不变量仅维持到 M7 成员手术为止——被修复兄弟的 <code>record</code> 分叉 (以新成员集重建, 3.7.3 stream 修复路由), 同 <code>_meta.id</code> 的兄弟输出自此 <code>member_ids</code> 可不同, 以 <code>_meta.stream.repaired</code> 消歧 (6.3); <code>membership</code> 类手术仅原信封 (首标签) 可执行、克隆兄弟降为只标记 (S8, 3.7.3)。
幂等	<code>classification is not None</code> 的项跳过——覆盖生成样本的 <code>source="inherited"</code> 继承 (3.6.2 按类种子池行) 与任何重入: 回流子批经过 <code>classify</code> 时零额外调用。v1.13 时间流生成形态 (3.6.5): 直装序列信封在 M6 出厂即携 <code>source="inherited"</code> 的序列类标签 (类表本就是配额与按类条件化的载体, 生成期已知归属), 故本模块对其零判决调用—— <code>classify.enabled = true</code> 在该形态下只作类表载体, <code>report.classify</code> 直方图恒全零属预期 (M1 因此豁免 <code>classify.llm</code> 的密钥引用集, 3.1.4); <code>estimate_run</code> 的 <code>classify_calls</code> 相应恒 0 (3.10.3)。
事件与计数	每记录一条 <code>classify.decision trace</code> 事件 (<code>classify</code> 通道 / trace-only, payload: <code>label</code> 、 <code>labels</code> (multi 携带全集)、 <code>source</code> 、 <code>reason</code> †、 <code>sc</code> †; 条件与字段定义见 7.2); 计数器 (M13 属主): <code>classify.classes.<name></code> (逐标签计)、 <code>classify.fallback</code> 、 <code>classify.failures</code> 、 <code>classify.multi_label_records</code> ; <code>counts.fanout</code> 由 M10 计量 (<code>counts.*</code> 所有权属 M10, 3.10.3)。report <code>classify</code> 节见 6.4。
上下文预算装填 (v1.11)	分类 profile 声明 <code>context_window</code> 时按上下文预算装填分类调用 (未声明 = 预算关闭, 行为与 v1.10 一致; 预算/估算/校准机制见 3.9): 「当前记录」的单记录 UI 树渲染动态封顶—— <code>UITree.serialize(max_chars=...)</code> 实参从固定 <code>input.ui_tree_max_chars</code> 改为 <code>min(ui_tree_max_chars, 预算折算字符)</code> (渲染后按 <code>est</code> 复核, 超则按行丢尾、保留既有 <code>...(truncated N nodes)</code> 标记; <code>ui_tree_max_chars</code> 保留为绝对上限, V9; 序列分支的摘要段封顶为同族语义)。类别示例是系统侧静态部件 (V13③): [类别示例·{name}] 段与类表、 <code>classify.instruction</code> 一律静态裁剪 (用户语义资产) ——由 M1 静态预检把关 (<code>est \geq input_budget \rightarrow CONFIG_ERROR</code> 、 $> 50\% \rightarrow WARN$, 3.1.4)。连最小单元 (单记录) 都装不下 $\rightarrow$ 该记录记 <code>context_overflow</code> 入 rejects (V10, 7.6)。逐裁剪点计入 <code>report.budget.truncations</code> (6.4)。

3.13.5 API 与配置

```
class ClassifyStage(Stage):
    name = "classify"
    def __init__(self, cfg: ResolvedConfig): ...
    async def run(self, batch, ctx) -> list[PipelineItem]: ...    # 返回传入的同一列表 (multi 可尾部追加)

def build_classify_prompt(record: Record, cfg: ResolvedConfig,
                          with_reason: bool) -> PromptBundle    # 3.13.3 模板的确定性组装
async def classify_record(record: Record, ctx: RunContext) -> Classification
```

配置见 5.2 [classify] 键表与 [class.<name>.<section>] 按类覆盖白名单表; 按类合并语义与全量校验清单见 3.1.4 (分类与按类覆盖行)。

背书:「先分类、按类走不同加工/合成策略」经 Nemotron-CC 在 6.3T token 级生产中验证——质量分档路由不同合成管线 [37], 并已产品化为 NeMo Curator 的分类器管线阶段 (DomainClassifier / QualityClassifier 等, 记录级标签字段驱动路由) [40]; LLM 对指令数据做语义打标并据标签驱动数据决策的可靠性见 InsTag [38]; 按类条件化的数据构造是 Tülu 3 按核心技能分治的标准姿势 [39]。LLM 封闭集分类的「标签 $\in$ 词表」硬校验形态同 Autolabel 的 `classification / multilabel` 任务 [12]——本模块以 M8 内部 Schema 的 enum 约束实现 (供应商结构化输出 + 修复环全套复用, 3.8); self-consistency 投票为 Wang et al. 的多路径采样多数决 [33] (分类恰是该机制收益最大的分类型 Schema 场景, 3.5.2)。与 NeMo Curator 跑本地 GPU 分类模型不同, 本模块走运行时 LLM API——与 M4 以「运行时 API 裁决」替代 QuRater 离线分类器是同一既有决策的延伸 (2.1.2 ①、3.4.5 背书行)。

3.13.6 输入 / 输出示例

沿用统一文本示例 (输入法中文指令意图工程, `input.text_field = "instruction"`), `project.toml` 追加:


```
[classify]                                # project.toml 追加片段
enabled = true
llm = "default"
fallback_class = "other"                  # 必填, 须 ∈ classes; LLM 亦可主动选择它

[[classify.classes]]
name = "writing"
description = "写作协助类指令: 代写、改写、文案、模板"
examples = ["帮我写一条请假条, 明天上午要去医院"]  # 可选类别示例 (仅输入侧)

[[classify.classes]]
name = "qa"
description = "知识问答与解释类指令"

[[classify.classes]]
name = "other"
description = "不属于以上任何一类的指令"
```

对记录 `fd97f67330e81315` ("解释一下二分查找为什么是 $O(\log n)$, 能不能举个在通讯录里找人的例子", 3.4.6 的 r3) 按 3.13.3 模板逐字组装 (single 模式; 本例 trace 未启用 $\Rightarrow$ 不请求 reason):

```
system:
  你是数据分类员。阅读待分类数据, 判断它属于以下类别中的哪一类。类别表:
  - writing: 写作协助类指令: 代写、改写、文案、模板
  - qa: 知识问答与解释类指令
  - other: 不属于以上任何一类的指令
  输出必须是符合以下结构的单个 JSON 对象, 不输出任何其他内容:
  {"class": <类名>}
user (类别示例·writing):
  [类别示例·writing] 帮我写一条请假条, 明天上午要去医院
user (当前记录):
  [待分类数据] 解释一下二分查找为什么是  $O(\log n)$ , 能不能举个在通讯录里找人的例子

← 响应(经 M8 classification_schema 校验): {"class": "qa"}

item.classification = Classification(label="qa", labels=("qa",), source="llm", detail={})
```

multi 扇出变体: 改设 `assignment = "multi"`、`max_labels = 2`, 对双意图记录「写一段讲解二分查找原理的教程」(写作 + 问答双命中):

```
响应: {"classes": ["writing", "qa"]}      # 归一化: 映射声明序、去重、无兜底类同现  $\Rightarrow k=2$ 
原信封:      classification = Classification("writing", ("writing", "qa"), "llm", {})  # 首标签 (声明序)
批尾追加兄弟信封: classification = Classification("qa",      ("writing", "qa"), "llm", {})
                # 共享同一 record 与 dedup 引用; scores/annotation/verification/errors 为全新默认容器
```

两个信封各自进 writing / qa 类池打分、按各自类有效 instruction 标注, 各产出一行 (行唯一键 (`_meta.id`, label), 6.3); 本批 `counts.fanout` 增 1 (3.10.3)。**fallback 分支**: 若某记录的分类输出经 M8 修复耗尽仍非法, 默认 `on_error="fallback"` 下归兜底类——`Classification(label="other", labels=("other",), source="fallback", detail={"kind": "classification_invalid", "message": "..."}),` 记录保持 active、不写 `item.errors` (3.13.4 失败与兜底行)。

3.13.7 帧级批量判决 (v1.12)

流模式帧粒度的分类面 (`frame.classify.enabled`, 默认关, 5.2): 对序列信封的成员帧按**帧类表** (`[[frame.classify.classes]]`, 与序列类表相互独立、允许重名、互不约束) 做一次批量闭集判决, 产物写 `item.member_classifications` (键 = 成员 `record.id`, 4.1)。链位住 M13 的成本结构 (裁决·链位与成本): dedup 之后——重复 episode 不付费; quality 之前的少量批量调用可接受 (帧标注贵在逐帧, 故住质量门之后的 M5, 3.5.5)。

公开面 (算子间导入白名单第四向, 签名冻结):

```
async def classify_frames(members: Sequence[Record], ctx: RunContext) -> dict[str, Classification]:
    """对给定成员 Record 序列做帧级闭集批量判决, 返回 {member.id: Classification}
       (label = 帧类名, labels = (label,) 恒单元素, source ∈ {"llm", "fallback"})。
       M7 verify 的成员回收补跑直接调用 (单成员回收即单元素调用)——v1.8
       segment.judge_window 同款契约地位的 sanctioned import exception (CONTRACTS §1.1)。
       本函数永不抛出记录级异常 (运行级控制流大三样除外)。"""

def build_frame_classify_prompt(members: Sequence[Record], cfg: ResolvedConfig,
                                digests: Sequence[str]) -> PromptBundle:
    """帧级批量判决模板的确定性装配 (verbatim 捕于 CONTRACTS §10.12)。"""
```

要点（规格与理由）：

- **执行门**（stage 内帧 pass，序列级判决写完之后、multi 扇出之前）：`active` $\wedge$ `record.kind == "sequence"` $\wedge$ 首标签信封（`classification.label == labels[0]`；`classification` 为 None 视同非克隆——克隆恒携 `classification`） $\wedge$ 幂等门（`item.member_classifications` is None $\wedge$ 非降格（`segment_degraded` duck 标在场 $\Rightarrow$ 计 `frame_classify.skipped_degraded` 并跳过，裁决·降格会跳过）。
- **组链双门**（裁决·组链双门）：`factory` 以或门 `classify.enabled` $\vee$ `frame_classify.enabled` 决定 `ClassifyStage` 进链（槽位不变，3.10.3）；stage 内序列级判决单独受 `classify.enabled` 门控——仅帧级开启时序列记录不产生 `Classification`（`_meta.classification` 维持 null）、帧 pass 照常。
- **调用形态**：— episode 一调用；预算声明时按 `budget.pack_windows`（v1.12 自 `segment._pack_windows` 下沉的同一纯函数，裁决·装箱器下沉）对成员摘要行成本贪心分窗——**零重叠调用形**：`pack_windows` 跨度链自带的 1 帧重叠（M14 缝帧语义）不适用于帧分类，自第二窗起丢弃与前窗重叠的首帧（前窗持有缝帧判决），所得跨度两两不交且完整覆盖；帧分类无窗口上限键 $\Rightarrow$ 预算是唯一分力；**预算关 $\Rightarrow$ 单窗全成员**。
- **提示词**（确定性模板，house 风格，verbatim 冻结 CONTRACTS §10.12）：`system = [任务]` 逐帧闭集分类指令（{N} 代入窗内成员数）+ 帧类表行 - `name: description`（声明序）+ 结构句 + 输出契约；`user = [会话成员帧]` 1-based 摘要行（`frame_digest`，每行上限 `segment.digest_max_chars`，会话级预计算复用——装配器自身永不计算摘要）；`FrameClassifyConfig.vision_resolved` 时每成员追加 [成员 i 截图] 标签 + `image part`（工作点 = `profile.default_image_px`，不另设尺寸——校准器按 `profile` 聚合的前提）。
- **内部 Schema**：`schema_engine.frame_classify_schema(names, n) = {"labels": {"type": "array", "items": {"enum": [...]}}, "minItems": n, "maxItems": n}` + `additionalProperties: false`（`segment_window_schema` 同款先例，3.8.1）；**对齐后校验在代码侧**（first-wins 家族）：`labels` 数组按位置对齐窗内成员序，**超长截断**（保留前 n 项）、**缺项 $\Rightarrow$ 该帧 `fallback_class`**（`source="fallback"`）；**同 id 成员 first-wins**（裁决·同 id 成员 first-wins，§1.6）——成员 id 为内容哈希，episode 内同 id 帧落表首位次胜出、不重复计数，各位次 `members[]` 行渲染同一产物。
- **失败语义**（v1.7 fallback 哲学下推）：单窗修复穷尽或调用不可恢复 $\Rightarrow$ 该窗**全部成员落 `fallback_class`**，计 `frame_classify.fallback += N`、`frame_classify.window_failures += 1`；**永不使 episode 信封 failed**、不写 `item.errors`。窗口跨度零重叠 $\Rightarrow$ 各窗写入互不覆盖，折叠结果与调度顺序无关。
- **溢出纪律**：`precheck`（含强制最小窗仍不可装填）= 最小单元失败，**永不喂熔断**；反应式溢出 $\Rightarrow$ 窗口**对半重切 ≤ 2 级**（`segment V20` 同款镜像；帧分类窗口零重叠，切分不保缝帧；每次对半计 `budget.degrade_retries`），级数耗尽/单帧窗不可再切 $\Rightarrow$ 按窗失败兜底；`reactive-400` 终局在窗失败吞点经共享 `budget.feed_reactive_terminal` 补喂熔断恰一次（A7 纪律，7.6 熔断矩阵）。
- **扇出共享**（裁决·扇出共享与首标签执行）：`_fan_out` 克隆构造清单显式加入 `member_classifications` / `member_annotations` 两字段——与 `record` / `dedup` 同族**按引用共享**（帧产物描述成员帧本身而非信封路由，克隆行渲染同一 dict）；帧 pass 在扇出之前执行，克隆自身永不重跑。
- **产物**：`item.member_classifications = {member_id: Classification(label=帧类名, labels=(label,), source="llm"|"fallback")}`（恒单标签——帧级无 assignment，3.1.4 定向探针）。
- **与时间流生成互斥（v1.13）**：`frame.classify.enabled = true` 与 `generate_stream.enabled = true` 互斥（M1 定向 `CONFIG_ERROR`，3.1.4）——时间流生成形态的帧类真值在**蓝图层**即确定，M6 直接把 `source="inherited"` 的帧级 `Classification` 写进同一个 `member_classifications` 字段（值形态与本节完全一致，仅 `source` 取第三值），本模块的帧 pass 对其不再执行（该形态下本模块整体零调用）。`members[]` 的渲染面因此对两种来源无差别（3.11.2）。
- **事件与计数**：`classify.frame` 每 episode 一发（`ids=(episode_id,)`，`payload = members/windows/fallback` 三计数，不携带任何数据内容，3.12.4）；计数器 `frame_classify.calls` / `fallback` / `window_failures` / `skipped_degraded`（report 子块见 6.4）。

3.14 M14 segment——时序流语义分段

3.14.1 职责与边界

做：（v1.8 新增算子）把批内候选会话精化为 episode：对 `status="active"` 的帧信封按 `session_id` 重组会话（M10 装箱时盖章，3.10.3），可选 LLM 滑动边界裁决与逐帧噪声标记（3.14.4）；成员信封置 `absorbed`、噪声帧置 `dropped_noise`，按序键拼装序列 `Record` 并向批尾追加 episode 信封（4.3 契约 ②b）。链序位于链首（`_CHAIN_ORDER` 首位、`dedup` 之前，3.10.3）——episode 形成先于一切逐条算子：帧级判重语义在连续 UI 帧上失效，重复判定改为 episode 级（3.3）；类标签、质量分、标注全部以 episode 为单位。`segment.enabled = false`（默认）时本算子不入链，工具行为与 v1.7 一致（输出仅多 `_meta.stream: null` 恒在键，6.3）。**不做：**不排序、不会话化（`Istream` 规则层属 M2，3.2；M14 收到的是已按整会话装箱的批）；不判重（M3）；不推断动作（M15）；不打任务标签（M5）；不改链结构（②b 改变的只是批内信封基数与状态，与 ②a 同为受控例外）。

模块	职责	边界	依赖
M14 segment	把批内候选会话精化为 episode：可选 LLM 滑动边界裁决与逐帧噪声标记；成员信封置 <code>absorbed</code> 、噪声帧置 <code>dropped_noise</code> ，按序键拼装序列 <code>Record</code> 并尾部追加 episode 信封（②b）	不判重（M3）；不推断动作（M15）；不打任务标签（M5）；不改链结构	M1, M8, M9

3.14.2 输入 / 输出

方向	内容
输入	批内 <code>status="active"</code> 且 <code>record.kind="single"</code> 的帧信封 (M10 已按整会话装箱并盖章 <code>session_id</code> ——同会话帧在批内连续、批内位置序即会话序, 3.10.3; 本算子追加的序列信封 <code>kind="sequence"</code> , 不落入处理面——天然幂等); <code>[segment]</code> 参数 (5.2); <code>strategy ∈ {llm, hybrid}</code> 时 LLM profile (<code>segment.llm</code>)。
输出	成员信封 <code>status → "absorbed"</code> ; 噪声帧 <code>status → "dropped_noise"</code> (携带 <code>noise / below_min_len</code> 二值之一的 duck-typed reason 标记, 3.14.4 成段流程; M11 rejects 归因据此分流, 3.11.2); 每段一个序列信封原地追加到传入批列表尾部 (<code>status="active"</code> 、 <code>record.kind="sequence"</code> 、盖章 <code>session_id</code>); 返回值 = 传入的同一列表对象 (4.3 契约 ②b)。 <code>on_error="fail"</code> 且窗口修复耗时该会话成员全部 <code>status="failed"</code> 、 <code>StageError</code> 入 <code>item.errors</code> (3.14.6)。

信封变化示例 (5 帧点外卖会话 `sess-0003`: `f0` 首页 → `f1` 搜索结果页 → `f2` 弹窗噪声 → `f3` 餐厅页 → `f4` 下单确认页; ②b 状态写入 = 只改既有元素状态 + 尾部追加, 无删除/重排/替换; 3.15.2 的摘取示例沿用本 episode):

```
段前批 (5 信封, 均 active, session_id="sess-0003", pair_index 3..7) :
#0 f0(b3a1c4e29d70f512) #1 f1(4c8e02d9a1b6f374) #2 f2(9a7d33c8b1e4f062)
#3 f3(e07b94a3c25d18f6) #4 f4(61f8d0b4a9c3e725)
逐帧 rel 定案 (3.14.4) : [continues, continues, interruption, advances, continues]
段后批 (6 信封) :
#0 absorbed #1 absorbed #2 dropped_noise(noise) #3 absorbed #4 absorbed
#5 episode 信封 (尾部追加) : status="active", session_id="sess-0003", transitions=None,
    record = Record(kind="sequence", id="7655568d2c485c43",          ← sha256("\n".join(成员 id))[:16]
        members=(f0, f1, f3, f4),                                ← 序键升序
        modality="ui", text/raw/ui_tree/image=None,
        ref=RecordRef(source_file="a/uitree_3.jsonl", line_no=None,
            pair_index=3,                                          ← 继承首成员
            generated_from=(), generator=None))
M10 计量: counts.episodes += 1 (segment 阶段 len 差, fanout 同构);
          absorbed += 4, dropped_noise += 1 (状态 tally, 3.10.3)
```

②b 的完整契约文本见 4.3 (含 M7 修复路径豁免: verify 缺陷修复可在本批内将成员状态在 `absorbed` 与 `dropped_noise` 间双向改写, 禁止翻回 `active`; 每个成员信封至多被一个序列信封吸收, 3.7)。

3.14.3 数据结构与 API

```
class SegmentStage(Stage):
    name = "segment"
    def __init__(self, cfg: ResolvedConfig): ...
    async def run(self, batch, ctx) -> list[PipelineItem]: ... # 返回传入的同一列表 (②b 尾部追加)

def build_segment_prompt(frames: Sequence[Record], diffs: Sequence[Mapping | None],
    digests: Sequence[str],          # v1.11 增形参 (V9, CONTRACTS 同步修订): 会话级
                                     # 预计算的逐帧摘要 (3.14.4 装填伪代码), 不再窗内现算
    cfg: ResolvedConfig, with_reason: bool) -> PromptBundle
    # 3.14.4 模板的确定性组装; 帧摘要与相邻帧 diff 由代码侧预组装;
    # 附图判据 = seg.vision_resolved (v1.11, V1—原 use_vision 键已移除)
async def judge_window(frames: Sequence[Record], ctx: RunContext) -> list[str]
    # 一窗一调用: 经 complete_validated(schema=
    # segment_window_schema(len(frames), with_reason)); 校验后
    # 按 index first-wins 建表、缺席帧缺省 "continues", 返回与
    # frames 对齐的逐帧 relation; 每窗发一条 segment.boundary
    # 事件 (3.14.6)。M7 成员回收复裁直调本函数 (3.7)
```

帧摘要与相邻帧树 diff 由共享 helper 提供——`frame_digest(record, max_chars)` 与 `tree_diff(a, b, quantize_px)`, 规范定义见第 4 章 (落位 `types.py`、毗邻 `UITree.serialize()`; M13/M4 序列分支复用同一实现——算子层不得互相依赖, 共享渲染下沉共享层)。摘要 = best-effort 确定性提取: `app` (`extra` 键 `package/package_name/pkg` 首个非空) · `activity` (`extra` 键 `activity/activity_name/window_title`, 可缺省) · `title` (DFS 首个可见非空 `text`) · `salient` (可见 `text/content_desc` 按序去重, 交互角色加前缀), 整体截断至 `max_chars`; 可见文本节点数为 0 或摘要长度 < 8 ⇒ 判贫瘠 (调用方计数, 3.14.4 贫瘠护栏)。diff = 结构键 (`role`, `bounds//quantize`, `depth`) 多重集匹配, 输出 `added/removed/text_changed/change_ratio/app_changed/title_changed`—— $O(n_1+n_2)$ 纯统计, 只提供变化幅度与类型证据, 不做语义归因 (不越 M15 界); `node_id` 不作跨帧匹配键 (同 `id` 可承载不同控件)。

窗口内部 Schema (`schema_engine.segment_window_schema`, 3.8.1 内部 Schema 清单: 不计入 `report.schema_engine.resolved_at`、不经过 L2.5)。关键字集 ⊆ 既有内部 Schema 关键字集、不写 `uniqueItems` (OpenAI strict 模式硬拒, 3.13.3 同教训) ——`index` 对齐由代码侧后校验保证 (3.14.4 缝合行); `minItems = maxItems = N` 钉死数组长度 (`judgment_schema` 同款):

```
def segment_window_schema(frame_count: int, with_reason: bool) -> dict:
    relations = ["continues", "advances", "returns_to_entry", "context_switch", "interruption"]
    item_props = {"index": {"type": "integer", "minimum": 0, "maximum": frame_count - 1},
                  "relation": {"type": "string", "enum": relations}}
    required = ["index", "relation"]
    if with_reason:
        item_props["reason"] = {"type": "string"}
        required = ["index", "relation", "reason"]
    return {"type": "object",
            "properties": {"frames": {"type": "array",
                                      "items": {"type": "object", "properties": item_props,
                                                "required": required, "additionalProperties": False},
                                      "minItems": frame_count, "maxItems": frame_count}},
            "required": ["frames"], "additionalProperties": False}
```

`with_reason` 条件 = `trace.enabled = true` 且 `trace.channels` 含 `"segment"` (零额外 token 原则, 3.13.4 调用与校验行同款)。

3.14.4 算法与流程

策略三态 (`segment.strategy`):

值	行为
<code>"rules"</code>	候选会话同样成 episode, 零 LLM 调用; <code>noise_filter / min_len</code> 不生效 (M1 对 <code>rules</code> ^ 显式 <code>noise_filter=true</code> 发 no-op warning, 3.1.4)。
<code>"llm" / "hybrid"</code> (默认 hybrid)	滑窗裁决 (下述)。两值在 M14 内行为一致——规则层会话化恒在 (M2), M14 收到的必是规则粗切后的候选会话, hybrid 命名即声明「规则粗切 + LLM 精化」的组合形态。窗上限 <code>segment.window</code> (默认 20, M1 校验 ≥ 2); v1.11 修订 (V9, 3.9.5): 所引 profile 声明 <code>context_window</code> 后按预算贪心装填切窗 (实际每窗帧数 $\leq$ window, 溢出即封窗——下述装填伪代码), 未声明预算时逐字节退化为固定窗、步长 = window-1; 两形态均保持重叠 1 帧、接缝帧整帧判决归后窗 (3.14.4 缝合); <code>len(session) == 1</code> 走 rules 退化 (零 LLM)。

三步演绎判据模板 (确定性拼接, 逐字冻结于 CONTRACTS §10.9; 判据内置且任务无关——用户零 prompt 可用, `segment.context` 只是可选域上下文、不是边界定义):

```
system:
  你是屏幕操作流的分段审核员。下面给出同一会话中按时间顺序排列的 {N} 帧状态摘要
  (含相邻帧的确定性变更提示)。按三步作业:
  一、双向上下文概括: 通读全窗, 把握每帧之前若干帧正在进行的活动与之后若干帧的走向, 再判断该帧。
  二、逐帧关系分类: 对每一帧, 判断它相对进行中活动的功能角色, 只能从以下封闭词表中取恰一值:
  - continues: 同一流程的推进。
  - advances: 屏幕或 App 变了, 但可见的任务实体延续 (验证码、订单号、餐厅名等跨屏出现) ——
    跨 App 的同一任务属此值, 不是边界。
  - returns_to_entry: 回到入口/搜索/桌面后开启新流程 (同 App 背靠背任务的断点)。
  - context_switch: 交互对象与环境不连续且无实体延续——相关但无实体延续的新流程也取此值。
  - interruption: 与前后活动均无关的短暂插入 (通知、弹窗、误触)。
  三、只输出逐帧关系, 不判断边界 (边界由既定规则从关系推导)。
  锚定约定: 分段粒度取「完整任务」层级 (整段录屏之下一层); 只看前台 App/前台窗口,
  忽略状态栏、后台通知等背景变化。
  {segment.context} ← 可选域上下文; 缺省省略此行
  输出必须是符合以下结构的单个 JSON 对象, 不输出任何其他内容:
  {"frames": [{"index": <窗内帧序号>, "relation": <词表值>[, "reason": <一句话理由>]}, ...]} (恰 {N} 项)
user (窗内逐帧, 一帧一段):
  [帧 {i}] {frame_digest(frame_i, segment.digest_max_chars)}
  [帧 {i} 变更] {tree_diff(frame_{i-1}, frame_i) 的文字化摘要} ← i ≥ 1; 窗首帧无此行
  (vision_resolved = true 时 (v1.11, V1 解析产物—3.1.4; 原 segment.use_vision 键已移除):
  每帧摘要 text Part 前附该帧 kind="image" 的 Part, 3.9.2)
```

两个锚定写死在模板文本 (不随配置变化): 粒度锚定 = 「完整任务」层级 (GEBD "1 level deeper" 原则); 注意力锚定 = 只看前台 App/窗口 (GEBD dominant subject 原则)。关系词表固定且域无关; `advances` 与 `context_switch` 的分界钉死为**实体延续**——相关但无实体延续的新流程 = `context_switch` (边界)。LLM 不直接回答边界问题, 只做封闭集分类 (M8 enum 硬校验, 与 `classify` 同款防线); `boundary / noise` 是代码侧查表结果 (演绎映射):

relation	演绎结果 (代码侧查表, LLM 不可见)
<code>continues / advances</code>	非边界 (帧归入当前段)
<code>returns_to_entry / context_switch</code>	边界——该帧是新段第一帧
<code>interruption</code>	noise (<code>noise_filter=true</code> 时剔除; <code>false</code> 时按非边界成员保留在所属段内)

会话首帧恒为首段: `rel[0]` 的边界值不参与判决 (无论 LLM 输出何值, 帧 0 都开启首段); `noise[0]` 照常生效。

调用与校验：每窗 1 次调用，经 `complete_validated(schema=segment_window_schema(...))` (3.8.3)；temperature 恒 0；批内全部窗口（跨会话）并入一个 `asyncio.gather`（profile 信号量约束，3.4.3 骨架同款）——缝合是判决收齐后的同步 pass，结果按窗口位置定位、与完成顺序无关；无 rng 消耗（种子豁免面不变，2.6）。episode 构成以 LLM 输出为条件（classify 分池同款条件化声明，2.6 幂等行）；同输入同 seed 逐字节可复现。

滑窗缝合（确定性；全窗判决收齐后执行。v1.11 (V9)：切窗从定长改为**预算贪心装填**——预算未声明时逐字节退化为 v1.10 定长窗；窗口边界由此成为（输入，配置）的确定函数——digest/diff 是记录内容纯函数、本算子零 rng，同输入同配置逐字节可复现不破）：

```
def refine(session: list[Record]) -> list[str]:
    if strategy == "rules" or len(session) == 1:
        return ["continues"] * len(session)
    digests = [frame_digest(f, digest_max_chars)
               for f in session]

    rel = [None] * len(session)
    start = 0
    while start < len(session):
        end = pack_window(digests, start)

        verdicts = judge_window(session[start:end])

        for i in range(end - start):
            rel[start + i] = verdicts[i]

        if end == len(session): break
        start = end - 1

    return rel

# 返回逐帧 relation (长度 = len(session))
# rules / 孤帧退化：原样成段，零 LLM

# v1.11 (V9)：会话级逐帧摘要预计算——前移到
# 切窗之前、每会话恰一次 (v1.8–v1.10 在切窗后
# 逐窗现算：接缝帧双算——前移是净改善；贫瘠
# 护栏计算路径独立保持不动)；build_segment_prompt
# 增 digests 形参消费本值 (3.14.3)

# v1.11 (V9)：窗 = [start, end) 贪心装填——自 start
# 逐帧累加 c_i = est_text(digests[i]) + DIFF_MAX_TOKENS
# + (每图成本 if vision_resolved else 0)
# (diff 在切窗后才算、输出结构有界故取最坏常数；
# 每图成本读校准器快照，3.9.5)，装填条件
# est_static_system + Σ c_i ≤ input_budget
# Λ 窗内帧数 ≤ window，溢出即封窗 (M1 静态护栏
# w_min ≥ floor 在**先验计价**下保证任意帧装得进
# floor 帧窗，3.1.4；校准值超先验 (3.9.5 无钳制，
# 合法) 或退化个案：装填器**强制 2 帧封窗** (V10
# 语义最小单元)，真实 est 超预算由 M9 终检以
# 记录级 context_overflow 走既有单窗失败路径——
# 永不 run 级、永不死循环，v1.11 审计修订)；
# 预算未声明 ⇒ end = min(start + window,
# len(session))，逐字节退化为定长窗
# 一窗一调用（并发下收齐后按位缝合）；
# 返回值已在 judge_window 内完成后校验：
# 按 index first-wins 建表（同窗重复 index
# 取首个出现者）、缺席帧缺省 "continues"
# （保守中性，quality「缺席准则-tie」同款）

# 无条件覆写 ⇒ 接缝帧（前窗末帧 = 后窗首帧）
# 整帧判决归后窗

# 后窗自前窗末帧起（重叠 1 帧；定长退化态下
# 等价于 v1.10 的步长 = window-1)
```

溢出降级重试 (v1.11, V20/V24)：某窗调用被识别为 provider 上下文溢出（统一溢出信号，3.9.5：预算开启下 400 错误体嗅探命中，或双协议 `model_context_window_exceeded` 200 形态——`ContextOverflowError(phase="reactive")`）⇒ 该窗对半分裂为两个子窗改切重试（维持重叠 1 帧与接缝归后窗语义，帧一个不丢——多花调用；乘性减、有界：每调用至多 2 次降级）；到最小单元仍溢出 ⇒ 按 V10 最小单元语义记录级 `context_overflow reject` (7.6)；**reactive-400 降级耗尽的终局由本算子经 `ctx.metrics.record_provider_result(fatal=True)` 补喂熔断恰一次** (A7；reactive-200 终局不补喂——3.9.5 熔断交互矩阵)，降级重试独立计数入 `report.budget.degrade_retries` (6.4)。未识别的 400 走现行 fatal 老路（零回归）。

成段流程（rel 定案后，逐会话确定性执行）：

1. 剔噪：noise_filter = true 时，rel[i] == "interruption" 的帧置 dropped_noise、reason 标记 "noise"（含帧 0）；false 时跳过本步。
2. 切段：剩余帧按演绎映射切段——rel[i] ∈ {returns_to_entry, context_switch} 的帧开启新段（会话首帧恒为段首）。
3. min_len 检查：仅作用于本步（LLM 边界精化）切出的段 (S11) ——段长 < segment.min_len ⇒ 该段全部帧置 dropped_noise、reason 标记 "below_min_len"（≠ "noise"：未经噪声声明裁决，不得污染噪声审计口径；计数独立，report.stream.below_min_len，6.4）。规则层孤帧/短会话（strategy="rules" 与 len(session)==1 退化）不经 min_len，原样成 episode。v1.9 注：帧信封上的 duck 标 noise_attribution == ("segment", "below_min_len") 即 M16 短段救援的判别载体（reason="noise" 帧不入救援候选池）——M16 救援命中时按 ②c③ 将此类帧 dropped_noise → absorbed 翻转 (4.3；本模块零改动——重组与翻转全在 M16 侧，3.16.4 救援行)；below_min_len 计数器为发生计数（帧口径），救援不回退（救援量另计 rescued_short，6.4）。
4. 拼装：每段成员按序键升序 → 成员信封置 absorbed → 构造序列 Record：kind="sequence"；id = sha256("\n".join(成员 id))[:16]（形成时定死，后续成员手术不重算，3.7）；text/raw/ui_tree/image = None；modality = 成员模态；members = 成员 Record 元组

(序键升序)；`ref` 继承首成员 (`source_file`、`line_no` (文本) / `pair_index` (UI)，`generated_from=()`、`generator=None`，4.1) → 尾部追加 `episode` 信封并盖章 `session_id` (②b)。完整成员溯源由 `_meta.stream.member_sources` 承担 (6.3)。

失败语义：单窗 M8 修复耗尽按 `segment.on_error` 处置 (3.14.6) —— "keep" (默认)：该会话放弃全部窗口判决，整体原样成一个 `episode` (零剔噪、零切分)，留痕三件套 = `_meta.stream.degraded` = {`kind`: "segmentation_invalid", `windows_failed`: k} + `error` 事件 + 计数器 `segment.failures`，不写 `item.errors` (记录存活；`rejects` 归因取 `item.errors[0]`，写入会污染后续阶段失败的归因，3.13.4 失败与兜底行同则)；"fail"：会话成员全部 `failed` → `rejects`。

摘要贫瘠护栏：某帧 `frame_digest` 判贫瘠 (可见文本节点为 0 或摘要长度 < 8——无文本 UI 树在真实采集中常见：ghost nodes、画布类屏幕 [63]) ⇒ 计 `digest_poor_frames` (`report.stream`，6.4) + 每运行至多一次 `WARN`；帧摘要保真度是纯文本裁决的第一瓶颈 (摘要没抓到的实体 LLM 看不见)，手册指引为 `segment.llm` 配置 `supports_vision = true` 的 `profile` 补偿 (v1.11 改写 (V4)：原「开 `segment.use_vision`」指引随该键移除失效——附图由 `profile` 能力自动推导，选 `profile` 即选能力)。

3.14.5 配置项

[`segment`] 键表 (与 5.2 一致，5.2 为配置规范属主；[`stream`] 排序与会话化键属 M2 消费，3.2、5.2)：

键	类型 / 默认	说明与约束
<code>enabled</code>	<code>bool</code> / <code>false</code>	<code>stream</code> 模式总开关。 <code>false</code> = 不入链、行为与 v1.7 一致 (输出仅多 <code>_meta.stream: null</code>)。启用要求 <code>run.mode="process" ^ generate.enabled=false</code> (含 <code>generate_only</code> 传递闭合) ^ <code>annotate.enabled=true</code> (2.3.1)。[<code>stream</code>] / [<code>segment</code>] / [<code>extract</code>] 在场而本键为 <code>false</code> ⇒ M1 no-op warning (3.1.4)。
<code>strategy</code>	"rules" "llm" "hybrid" / "hybrid"	三态语义见 3.14.4 策略表。
<code>llm</code>	<code>str</code> / "default"	LLM <code>profile</code> 引用；仅 <code>strategy ∈ {llm, hybrid}</code> 时计入密钥解析 / <code>probe</code> / 存在性引用集 (S30，3.1.4)。v1.11 (V1/V3)：恒不入 <code>vision</code> 校验集——窗口是否附图由本 <code>profile</code> 的 <code>supports_vision</code> 能力自动决定 (<code>vision_resolved</code> 行)；选 <code>profile</code> 即选能力 (省钱形态 = 指向纯文本 <code>profile</code>)。
<code>window</code>	<code>int</code> / 20	单窗帧数上限 (v1.11 语义修订，V9)：所引 <code>profile</code> 声明 <code>context_window</code> 后按预算贪心装填 (实际每窗帧数 ≤ <code>window</code> 、溢出即封窗，3.14.4 装填伪代码)，未声明时为固定窗大小 (v1.10 行为逐字节一致)；M1 校验 ≥ 2；两形态均重叠 1 帧、接缝帧整帧判决归后窗。会话不长且预算装得下时可调大以趋近整段单调用形态 (3.14.7)。
<code>digest_max_chars</code>	<code>int</code> / 400	单帧摘要 (<code>frame_digest</code>) 截断上限。
<code>noise_filter</code>	<code>bool</code> / <code>true</code>	仅 <code>llm/hybrid</code> 生效； <code>rules</code> ^ 显式 <code>true</code> ⇒ M1 no-op warning。
<code>min_len</code>	<code>int</code> / 2	段长下限；仅作用于 LLM 边界精化切出的段 (3.14.4 成段流程 ③， <code>below_min_len ≠ noise</code>)。
<code>vision_resolved</code>	<code>bool</code> (parse product)	v1.11 (V1)：非用户键——M1 于 <code>load()</code> 收尾冻结的解析产物 (3.1.4)： <code>vision_resolved = (modality=="ui") ^ enabled ^ strategy∈{llm,hybrid} ^ llm_profiles[segment.llm].supports_vision</code> ； <code>true</code> 时窗内逐帧附图 (3.14.4 模板)。原用户键 <code>use_vision</code> 已于 v1.11 移除 (V2：显式出现 → <code>CONFIG_ERROR</code> 定向迁移指引，5.2/3.1.4)。
<code>context</code>	<code>str</code> / ""	可选域上下文 (如「这是手机屏幕操作流」)，注入模板可选行；不是边界定义——判据内置于固定模板，零配置可用。
<code>on_error</code>	"keep" "fail" / "keep"	窗口修复耗尽处置 (3.14.6)。

[`class.<name>.segment`] 不存在：`segment` 在 `classify` 之前执行，类标签尚不存在 (链序因果；5.2 按类覆盖白名单表注明缘由)。

3.14.6 错误处理

错误码 `segmentation_invalid` (7.6，v1.8 增行) 两形态：

<code>segment.on_error</code>	行为
"keep" (默认)	会话整体降级为一个 <code>episode</code> (原样、零剔噪) 并存活；留痕三件套 = <code>_meta.stream.degraded</code> = { <code>kind</code> : "segmentation_invalid", <code>windows_failed</code> : k} (6.3) + <code>error</code> 事件 (<code>kind = segmentation_invalid</code> ， <code>segment</code> 通道) + 计数器 <code>segment.failures</code> 。不写 <code>item.errors</code> (S26；归因保护同 3.13.4)。
"fail"	该会话成员信封全部 <code>status="failed"</code> 、 <code>StageError(stage="segment", kind="segmentation_invalid")</code> 入各 <code>item.errors</code> ⇒ <code>rejects</code> 。

事件 (7.2，v1.8 增行；通道 "segment" = `stage` 名——`_TRACE_CHANNELS` 8→10 之一，事件名前缀即通道、`error` 事件按 `stage` 自动归属，零路由代码)：

事件名	通道 / <code>stderr</code> 级别	触发点	payload 字段
-----	-----------------------------	-----	------------

segment.session	segment / — (trace-only, 无 stderr 镜像)	M2 会话装配器闭合会话时 (属主 M2, 3.2; 事件名冠 segment 前缀归本通道); record_ids = ()。	session_id、first、last (首末序键)、len、cause (∈ gap key max_len max_span eof limit)。
segment.boundary	segment / — (trace-only)	M14 每窗裁决经 M8 校验通过后; record_ids = ()。	session_id、window: [s, e]、member_ids、relations: [{index, relation}]]、model、reason†。

† reason 请求条件 = with_reason (3.14.3); 作为 LLM 自由文本受 7.4 分级——none 档剔除、refs 起携带 (键已在 _FREE_TEXT_KEYS 集合); 其余 payload 字段均为结构字段, 全档保留。

计数与归属: segment.failures (M14 属主); counts.episodes = segment 阶段 len 差 (M10 计量, fanout 同构); absorbed / dropped_noise 由状态 tally 归集 (M10, 3.10.3); below_min_len、digest_poor_frames 入 report.stream (M14 属主, 6.4); v1.11 增 report.stream.windows (V13④, M14 属主, 6.4) = 实际窗函数 (含 V20 分裂产生的子窗)——供用户对账 estimate_run 的 w_min 上界估算 (3.10.3); sessions 数据源 = IngestReport (M2 属主)。rejects 归因 (3.11.2): dropped_noise 行按 duck-typed 标记分流为 stage="segment", reason="noise" 或 reason="below_min_len"。--strict 交互: stream 工程的噪声帧属预期产物, --strict 会因 rejects 非空退出 1 (手册明示); v1.9 注: stitch.rescue_short 命中的 below_min_len 帧翻回 absorbed、不再落 rejects——同输入开启 stitch 后 strict 结果可能由 1 变 0, 属预期 (3.16.6、3.11.2)。

3.14.7 背书

边界判据内置、无需任务词表的依据是 GEBD [48]: 其把认知科学「人类无需预定义事件类别即自然分割连续活动」形式化为可标注、可评测的基准 (Kinetics-GEBD), 并给出两条使之可操作的标注锚定——固定相对粒度 ("1 level deeper") 与 dominant subject 注意力, 本模块模板的两条锚定即其移植; 对其共识强度的准确措辞是中等共识可达 (每视频 5 人多评的协议数据支撑「可标注」, 而非「天然高度一致」)——这正是本模块在判据之外仍保留 trace 审计闭环 (segment.boundary reason 抽读 → 调 context/window/gap → 同 seed 重跑对比) 的原因。三步演绎结构照抄 Def-DTS [47] 的可操作化手法 (双向上下文概括 → 封闭集意图分类 → enforced 演绎查表, LLM 不自由判断边界); 其消融证据的精确读法: 半结构形态 (仅保留双向概括、去掉关系分类) 比裸问题更差, 完整三步最优; 而在边界信号清晰的数据集上, 裸判决可胜过全套结构——结构收益集中于边界模糊场景, GUI 流的跨 App 延续与弹窗插入恰属此类; 其意图词表逐数据集改池 (Dialseg711 删值、SuperDialseg 换表) 证明关系词表按域定制是该方法的预期用法, 本模块五值词表即 GUI 域定制: advances / context_switch 以实体延续重划对话域的「相关新话题」, returns_to_entry 是对话域没有的 GUI 入口态线索, interruption 噪声维则来自 RPA UI 日志分割对「不属于任何例程的噪声事件」的显式处理 (Marrella; Leno et al. [50])——「分段 = 边界发现 + 噪声剔除」的问题定义同源, 其难点变体「交错例程」v1 明确不做 (8.4); 「会话首帧恒为段首」对应 Def-DTS 「对话首句恒判 YES」。滑窗 LLM 逐帧裁决是 2026 年 GUI 轨迹量产管线仍在使用的形态之一 (VideoAgentTrek / Video2GUI [60]); 对照形态是整段单调用——GUIDE [57] 报告该形态 99.4% 的段可用率, v1 保留滑窗 (有界上下文、有界单请求规模——v1.11 措辞修订 (V9): 预算装填下窗帧数随内容浮动但恒有上界 window ∧ input_budget), window ≥ 会话长且预算装得下整段 (v1.11 补预算前提: 所引 profile 未声明 context_window, 或整段装填 est 不超 input_budget) 时滑窗天然退化为整段单调用, 故两形态是同一旋钮的两端, 会话不长时建议调大 window 以贴近该证据形态 (S32; 「extract 先行 + 在动作序列上分段」的次序变体以成本权衡列演进候选, 8.4)。「定位/分段」与「描述/标注」拆为两道工序 (segment 与 M5 分立) 沿 dense video captioning 的两段式先例 (Vid2Seq [52])。

3.15 M15 extract——转移/动作摘取

3.15.1 职责与边界

做: (v1.8 新增算子) 对批内每个 status="active" 的序列信封 (episode), 逐对相邻成员帧 <s_i, s_{i+1}> 经 LLM 产出结构化动作 (内部 Schema, 3.15.3), 写入 item.transitions; 转移数 = 成员数 - 1 (动作发生在相邻帧之间 [45])。链序位于 classify 之后、quality 之前 (3.10.3)——类标签先就位使 [class.<name>.extract] 按类 instruction 生效 (3.15.4 multi 行); 轨迹 rubric 与序列标注在下游消费步骤序列。仅 UI 模态序列 (M1 强制 extract.enabled 要求 segment.enabled ∧ run.modality="ui", 2.3.1; 文本序列的「转移」语义弱, v1 不适用, 列演进候选 8.4)。不做: 不重分段 (边界属 M14 上游; 成员集是给定输入); 不产出用户 Schema 字段 (步骤序列是工具内部结构——进 _meta.stream.steps 并注入下游提示词; 用户 Schema 产出物属 M5); 不淘汰记录 (修复耗尽的默认路径是兜底留痕而非丢弃, 3.15.6); 不打分、不评审。

模块	职责	边界	依赖
M15 extract	对每个 active 序列信封的每对相邻成员帧 <s_i, s_{i+1}> 经 LLM 产出结构化动作 (内部 Schema), 写入 item.transitions; 转移数 = 成员数 - 1	不重分段 (M14 上游); 不产出用户 Schema 字段 (M5); 不淘汰记录	M1, M8, M9

3.15.2 输入 / 输出

方向	内容
输入	批内 status="active" 且 record.kind="sequence" 且 transitions is None 的 PipelineItem (transitions is not None 者需等跳过, 3.15.4); [extract] 参数 (5.2); LLM profile (extract.llm, 须 supports_vision = true——请求 2 图, 3.1.4)。

输出	每个处理信封 <code>item.transitions: tuple[Transition, ...]</code> (长度恒 = <code>len(record.members) - 1</code> , 按成员对位次升序; 单条转移失败不破坏该不变量——fallback 占位, 3.15.6); 批基数不变 (不追加、不淘汰), 返回值 = 传入的同一列表对象。 <code>on_error="fail"</code> 且结构修复耗尽时该 <code>episode</code> <code>status="failed"</code> 、 <code>StageError</code> 入 <code>item.errors</code> (唯一改状态路径)。
----	--

接缝序数机械占位 (v1.9)。 `stitch` 启用且信封为多碎片线索时 (3.16), `seam_indexes` duck 标所列序数 (接缝对左成员下标, 与 `Transition.index` 同坐标) 的转移不发 LLM 调用——代码侧生成 T10 占位 `Transition` (四键与 detail 见 3.15.4 占位行); 其余成员对照常摘取 (含「间隙仅噪声/本线索救援帧」的拼接对与会话位置紧邻的救援拼接对——真实转移, 与 v1.8「剔噪对照常摘取」惯例单一处理, 3.16.4 接缝判据)。 `transitions` is None 幂等门与 `len(transitions) = len(members) - 1` 不变量不动 (占位步占据其 `index` 位次); 实现注记: 跳过 `seam` 序数需重构本模块平铺 `gather` 的记账结构 (既有 `spans` / 切片假设每成员对一协程) ——属实质改动, 非行内小补。

信封变化示例 (沿用 3.14.2 的点外卖 `episode` 7655568d2c485c43, 成员 f0 首页 → f1 搜索结果页 → f3 餐厅页 → f4 下单确认页; f1↔f3 在采集流中原不相邻——噪声帧 f2 已被 M14 剔除, 转移以成员相邻为准):

```

段前: item.transitions = None
段后: item.transitions = (
    Transition(index=0, action={"action_type": "input_text", "target": "搜索框",
                                "value": "麻辣烫", "description": "在首页搜索框键入「麻辣烫」并搜索"},
              model="glm-5.2", attempts=1, detail={}),
    Transition(index=1, action={"action_type": "click", "target": "老王麻辣烫",
                                "value": None, "description": "点击搜索结果中的餐厅进入餐厅页"},
              model="glm-5.2", attempts=1, detail={}),
    Transition(index=2, action={"action_type": "click", "target": "去结算",
                                "value": None, "description": "点击「去结算」进入下单确认页"},
              model="glm-5.2", attempts=1, detail={}),
)

```

步骤序列随后由 M11 写入 `_meta.stream.steps` (verbatim, 6.3), 并经新形参注入 M4 序列打分与 M5 序列标注提示词 (3.4、3.5)。

3.15.3 数据结构与 API

```

class ExtractStage(Stage):
    name = "extract"
    def __init__(self, cfg: ResolvedConfig): ...
    async def run(self, batch, ctx) -> list[PipelineItem]: ... # 返回传入的同一列表 (不增不减)

def build_extract_prompt(prev: Record, curr: Record, cfg: ResolvedConfig,
                        label: str | None) -> PromptBundle
    # 3.15.4 模板的确定性组装; label 非空时 instruction 取
    # class_views[label].extract 有效值 (3.1.4 按类覆盖合并)

async def extract_transition(prev: Record, curr: Record, index: int,
                            ctx: RunContext, label: str | None = None) -> Transition
    # 一转移一调用: 经 complete_validated(schema=action_schema());
    # 修复耗尽按 on_error 兜底/抛出。M7 成员手术后的接缝重摘取
    # 直调本函数 (1-2 次/手术; 重建的 Transition 带
    # detail.reseamed=true 溯源, index 重编号后不变量
    # len(transitions) = len(members) - 1 恒真, 3.7)

```

产物类型 `Transition` (完整定义见 4.2): `index` (重建后位次, 恒 = 在 `transitions` 元组中的下标)、`action` (过 `Schema` 的动作对象)、`model`、`attempts` (1 + L3 修复次数)、`detail` (干净摘取 {} ; fallback {kind: "extraction_invalid", message}; 手术重摘取 {reseamed: true})。

动作内部 `Schema` (`schema_engine.action_schema()`), 3.8.1 内部 `Schema` 清单: 不计入 `report.schema_engine.resolved_at`、不经过 L2.5)。
全键 required + 可空联合: OpenAI strict 模式硬拒可选属性 (L0 无条件透传 `Schema`, 3.13.3 `uniqueItems` 同型教训), 可空字段以 `["string", "null"]` 类型联合表达而非从 `required` 摘除; 关键字集 $\subseteq$ 既有内部 `Schema` 关键字集:

```

def action_schema() -> dict:
    actions = ["click", "long_press", "input_text", "scroll", "drag", "open_app",
               "app_switch", "navigate_back", "navigate_home", "wait", "other"] # 11 值 (S15)
    return {"type": "object",
            "properties": {"action_type": {"type": "string", "enum": actions},
                           "target": {"type": ["string", "null"]},
                           "value": {"type": ["string", "null"]},
                           "description": {"type": "string"}},
            "required": ["action_type", "target", "value", "description"],
            "additionalProperties": False}

```

3.15.4 算法与流程

提示词模板（确定性拼接，逐字冻结于 CONTRACTS §10.10；一请求 2 图 = pairwise quality 既有形态，3.4.3）：

```
system:
  你是屏幕操作流的动作摘取员。给定同一操作流中相邻的前后两帧屏幕状态，推断用户在两帧之间
  执行的动作。action_type 只能取以下值：
  - click / long_press / drag: 点击 / 长按 / 拖拽某控件
  - input_text: 在输入框键入文本
  - scroll: 滚动屏幕或列表
  - open_app: 打开一个应用；app_switch: 切换到另一已打开的应用
  - navigate_back / navigate_home: 系统返回 / 回到桌面
  - wait: 无用户交互，仅等待界面加载或变化
  - other: 无法归入以上任何一类（把语义写进 description）
  锚定约定：前一帧是动作发生前最后一个稳定状态，后一帧是动作完成后的首个稳定状态；推断
  二者之间发生的单个语义动作；若变化由多个低层事件构成（连续滚动、连续键入），归并为一个
  语义动作。
  {instruction}                                     ← 可选补充说明（per-label 有效值）；缺省省略此行
  输出必须是符合以下结构的单个 JSON 对象，不输出任何其他内容：
  {"action_type": <词表值>, "target": <目标控件文本引用或 null>,
   "value": <动作参数或 null>, "description": <一句话动作描述>}
user (单条消息多 Part——「text 标签 + image」组装惯例同 3.5.2/3.13.3；一请求 2 图)：
  text part:  [前一帧截图]
  image part: s_i.image                               (M9 调用时编码, 3.9.2)
  text part:  [后一帧截图]
  image part: s_{i+1}.image
  text part:  [树变更摘要] {tree_diff(s_i.ui_tree, s_{i+1}.ui_tree) 的文字化}
                                     ← include_diff = true 时; false 整段省略
                                     [前后帧树摘要] {frame_digest(s_i)} → {frame_digest(s_{i+1})}
```

锚定句（「前一帧是动作发生前最后一个稳定状态……归并为一个语义动作」）移植自 OpenCUA 的 State–Action Matching 与 Action Reduction 约定 [43] (3.15.7)。「树变更摘要」= tree_diff（第 4 章 helper, M14 同源）输出的确定性文字化——增/删/文本变化节点数、变化比例、App/标题是否变更；零额外 LLM 调用。

字段语义（逐字冻结于 CONTRACTS §10.10 表；词表合法性由 Schema enum 保证，字段语义由模板文字锚定）：

action_type	target 语义	value 语义
click / long_press / drag	目标控件的 文本引用 ，取值优先级 text → content_desc → 类名+序号；不可辨识时 null	null
input_text	被键入输入框的文本引用（同上优先级）	键入的文本—— 聚合语义 ：同一相邻对之间的「聚焦点击 + 键入」归并为一步 input_text，聚焦点击不单独记步
scroll	滚动容器引用；不可辨识时 null	方向，限 up / down / left / right（模板文字锚定四值；代码侧小写归一）
open_app / app_switch	null	应用名
navigate_back / navigate_home / wait	null	null
other	尽力而为的对象引用或 null	null（语义全落 description）

两条设计注记：① target 用**文本引用**、**不用坐标**——extract 是事后标注而非执行器，元素文本引用与中心坐标对 LLM 等效 [45]，且 max_image_px 降采样会破坏坐标系与原始截图的对应；② include_diff = true（默认开，可关做 A/B）——注入的是**结构化树 diff 而非像素 diff**：像素 diff 注入在 Sharingan 中报告为负结果 [59]，结构化 diff 则是确定性归并证据（OpenCUA Action Reduction 同族 [43]），缩短视觉推断距离、降低幻觉（S14）。

其余关键设计的精确定义：

设计点	定义
调用与校验	每对相邻成员帧 1 次调用（extract_calls = Σ(len(members) - 1)），经 complete_validated(schema=action_schema())（3.8.3）。temperature 恒 0。
并发	批内全部转移（跨 episode）并入一个 asyncio.gather（profile 信号量约束）——骨架同 M4 pairwise phase2（3.4.3）； 无 rng 消耗 （种子豁免面不变，2.6）；结果按（episode 批内位置, 对位次）定位回写，与完成顺序无关，逐字节可复现。
幂等	transitions is not None 的信封跳过——任何重入零额外调用。M7 修复路径不重跑本 stage：接缝重摘取经 extract_transition 函数直调（3.15.3、3.7）。
multi 扇出 （按 label 各摘，S9）	classify assignment="multi" 扇出的兄弟信封克隆时 transitions 恒 None（classify 在前、extract 在后，3.13.4 multi 扇出行）——每个兄弟按各自 label 的有效 extract.instruction 独立摘取（per-label 白名单承诺兑现；transitions 每信封自持，接受 xk 调用成本）。episode 命中多类应属罕见——M14 边界判据即「单一目标导向活动」（3.14.4）。dry-run 估算按乘数 1 报下界 + stderr 注明（3.13 R28 口径，2.4）。

fallback 语义	单转移 M8 修复耗尽且 <code>on_error="fallback"</code> (默认): 该步写入代码侧构造的兜底 Transition—— <code>action = {"action_type": "other", "target": null, "value": null, "description": ""}</code> + <code>detail = {kind: "extraction_invalid", message}</code> 留痕; episode 存活、后续转移照常摘取; 不写 <code>item.errors</code> (rejects 归因取 <code>item.errors[0]</code> , 写入会在该记录后续阶段失败时污染归因——3.13.4 失败与兜底行同则)。留痕使兜底步与 LLM 确证的 other 对下游可区分 (detail.kind 在场与否)。
接缝占位 (v1.9, T10 四键钉死)	<code>seam_indexes</code> 所列序数 (3.15.2 占位段) 写入代码侧构造的占位 Transition, 零 LLM: <code>action = {"action_type": "app_switch", "target": null, "value": null, "description": "线索接缝: 被<打断者>打断后恢复"}</code> (<打断者> = interrupted_by 各任务名顿号连接) + <code>detail = {kind: "thread_seam", interrupted_by: [...]}</code> (按接缝判据恒非空, 3.16.4); steps 步行 <code>resumed = true</code> 落接缝步自身 (emitter 由 detail.kind 推导, 6.3)。语义备注: 占位 <code>action_type="app_switch"</code> 对同 App 内穿插 (返回同页型) 语义不贴——占位类型不承诺语义, 下游以 detail.kind 判别 (与 extraction_invalid 留痕同法)。计数器口径: seam 占位不计入 <code>report.stream.extract.transitions</code> 与 <code>extract.by_type.*</code> (非摘取产物——零 LLM 的 app_switch 会灌污 by_type 分布; 接缝唯一计量点 = <code>report.stream.stitch.seams</code> , 6.4); 相邻救援不占位: 会话位置紧邻的救援拼接对是真实转移, 照常送 LLM 摘取并正常计数 (3.16.4 救援行)。
上下文预算 (v1.11)	摘取 profile 声明 <code>context_window</code> 时 (未声明 = 预算关闭, 行为与 v1.10 一致; 机制见 3.9): 恒定 2 帧 + 2 图的单转移调用无可收缩项 (图不可减帧、diff / 摘要段结构有界) ——不做装填裁剪, 由 M9 咽喉终检兜底 (V16); 溢出 (终检命中或反应态, 本调用点无降级面) → 该步走既有 <code>on_error="fallback"</code> 机械回退语义不变 (<code>action_type="other"</code> 兜底步留痕, 3.15.6; <code>"fail"</code> 时错误分类按 7.6 词表精确记 <code>context_overflow</code>)。接缝占位步零 LLM、不受预算影响 (本表接缝占位行)。

3.15.5 配置项

[extract] 键表 (与 5.2 一致, 5.2 为配置规范属主):

键	类型 / 默认	说明与约束
enabled	bool / false	启用要求 <code>segment.enabled ^ run.modality="ui"</code> (2.3.1); [extract] 在场而 <code>segment.enabled=false</code> ⇒ M1 no-op warning (3.1.4)。
llm	str / "default"	LLM profile 引用: 恒计入密钥解析 / vision / probe / 存在性四处引用集且恒入 vision_users (S30) ——每请求 2 图, 无纯文本档。
instruction	str / ""	可选域提示, 注入模板可选行; [class.<name>.extract] 按类覆盖白名单的唯一键 (5.2 白名单表; per-label 生效见 3.15.4 multi 行)。
include_diff	bool / true	【树变更摘要】注入开关 (S14); 关闭可做 A/B 对照 (手册调用账章给指引)。
on_error	"fallback" "fail" / "fallback"	单转移修复耗尽处置 (3.15.6)。

3.15.6 错误处理

错误码 extraction_invalid (7.6, v1.8 增行) 两形态:

extract.on_error	行为
"fallback" (默认)	该步记兜底动作 <code>action_type="other"</code> + <code>Transition.detail = {kind: "extraction_invalid", message}</code> 留痕 (3.15.4 fallback 行); episode 存活、不写 <code>item.errors</code> ; + error 事件 (kind = extraction_invalid, extract 通道) + 计数器 <code>extract.fallback_steps</code> 。
"fail"	该 episode 信封 <code>status="failed"</code> 、 <code>StageError(stage="extract", kind="extraction_invalid")</code> 入 <code>item.errors</code> ⇒ rejects; + 计数器 <code>extract.failures</code> 。

事件 (7.2, v1.8 增行; 通道 "extract" = stage 名——_TRACE_CHANNELS 8→10 之一, error 事件按 stage 自动归属):

事件名	通道 / stderr 级别	触发点	payload 字段
extract.step	extract / — (trace-only, 无 stderr 镜像)	每转移定案后 (含 fallback 步); <code>record_ids = (s_i.id, s_{i+1}.id)</code> 。	<code>episode_id</code> 、 <code>index</code> 、 <code>action_type</code> 、 <code>description</code> †、 <code>target</code> ‡、 <code>value</code> ‡。

脱敏分级 (S27; 7.4): `target` / `value` 是输入数据派生 (可能含用户键入文本) ——纳入 `_DATA_KEYS = {"target", "value"}, none / refs` 档剔除 (refs 档「无输入数据内容」红线)、`excerpt` 起携带 (†); `description` 是 LLM 自由文本——纳入 `_FREE_TEXT_KEYS`, `none` 档剔除、`refs` 起携带 (†)。逐档 payload: `none = {episode_id, index, action_type}`; `refs = + description`; `excerpt = + target、value`。

计数与归属 (M15 属主; `report.stream.extract` 子块, 6.4):

计数器	语义
extract.transitions	产出转移总数 (含 fallback 步)。
extract.fallback_steps	兜底步数 (fallback 路径)。
extract.failures	fail 路径失败 episode 数。

<code>extract.by_type.</code> <code><action_type></code>	逐动作类型分布（S14 ③）——系统性劣化（如 other 占比异常升高、某类型塌缩）可观测，供 <code>include_diff</code> A/B 与模型/提示词迭代用。
---	--

3.15.7 背书

「从相邻状态对推断中间动作」是被大规模验证过的独立工序：OpenAI VPT 用逆动力学模型（IDM）给 70k 小时无标签视频打伪动作标签——因可同时看过去与未来帧，反演环境动力学远比行为克隆易学 [42]；LabelKit 的负边界「不训练/托管本地模型」（2.1.2 ①）决定本模块用 **LLM zero-shot 充当运行时 IDM**（与 M4 以运行时 API 裁决替代 QuRater 离线分类器是同一既有决策的延伸，3.4.5 背书行），且同样利用非因果优势（一次调用喂 `s_i` 与 `s_{i+1}` 两图）。「三元组 `<s_pre, a, s_post>` → 结构化动作/低层指令」正是 OS-Genesis reverse task synthesis 的第一道工序（ACL 主会级验证）[41]——差异在 OS-Genesis 的动作是采集时自带，本模块输入只有状态流故动作须推断。「确定性归并 + LLM 语义化」的两层分工出自 OpenCUA 的标注基础设施 [43]：其 Action Reduction 把海量低层事件确定性归并为语义动作（鼠标移动序列→click、滚轮单方向累计、连续键击→文本串）、State-Action Matching 把每个动作锚定到动作前最后一个视觉稳定帧（防未来信息泄漏）——对应本模块「树 diff（代码侧确定性）+ LLM 动作裁决」的分工与模板锚定句（3.15.4）。动作词表对齐口径 = **AndroidControl 词表全集** ∪ **UI-TARS-mobile 增量 + other 兜底**：AndroidControl 的 8 动作（click, long_press, input_text, scroll, navigate_back, navigate_home, open_app, wait）全集采纳、无裁剪 [45]（被有意排除的仅其论文 agent 侧人工插入的 terminate/status——终态判定属分段边界与标注语义层；「转移数 = 截图数 - 1」计数恒等式即本模块调用量公式）；drag 与 app_switch 取自 2025–2026 统一动作空间共识（UI-TARS / UIPro [62]）——跨 App episode 是本设计一等公民（GUI-Odyssey 全集皆跨 App [46]），应用切换须是词表内一步而非边界；other 兜底优于原表（人工采集不产词表外动作，LLM 推断会）。**可靠性预算**（S14，风险表与手册调优章同源）：LLM zero-shot 动作推断的可靠性钉在 **70–80%/步**（Watch & Learn 实测 70.5% [58]；Sharingan 70–80% 且按动作类型不均衡 [59]）——每步 20–30% 错误率会沿 episode 级联，本模块不承诺单步正确性，承诺**缓解链**：树 diff 证据注入（`include_diff`，3.15.4 注记 ②）+ verify 缺陷路由兜底（步骤↔标签一致性硬门，3.7）+ quality 结构分软门（连贯性/噪声残留维度压分缺陷段，3.4）+ `extract.by_type` 分布可观测（3.15.6）。

3.16 M16 stitch——线索缝合

3.16.1 职责与边界

做：（v1.9 新增算子）对批内候选会话执行线索缝合：以「单调选池 LLM 判定 × 机械先验合取」把同一目标导向任务被穿插切开的碎片（episode）保守缝合为**线索（thread）**，有界二遍复评修正顺序贪心的漏缝（3.16.4）；被并 episode 信封壳置 `status="stitched"`、幸存信封 Record 重绑（4.3 契约 ②c）；`below_min_len` 短段按连续 run 重组为救援候选先进候选池，命中时成员帧 `dropped_noise` → `absorbed` 翻转（②c③）；对多碎片线索机械标定接缝（`seam_indexes` duck 标，零 LLM——接缝转移由 M15 按 T10 四键占位，3.15.4）。产出三级结构 `thread > fragment > step`（对齐 Ego4D Goal-Step `goal>step>substep` [69] 与 AndroidControl `goal>instruction>action` [45]）；帧永远单一归属——交叉用「平面分段 + 线索身份」表达（Goal-Step `is_continued` 同型 [69]；PIRA 把任务子轨迹形式化为**非连续帧子集** [64]），不引入帧多重归属。链序位于 segment 之后、dedup 之前（3.10.3）——缝合改变成员集，必须先于判重（线索判重面 = 重绑后成员配方，3.3.3）与摘取（接缝序数占位，3.15）。`stitch.enabled = false`（默认）时本算子不入链，**主输出、rejects、report.json 与 v1.8 逐字节等价**（退化锚，3.16.4 退化锚行；例外恰两处——dry-run stderr 的 `stitch_calls=0` 行与 `streamxverify` 缺陷词表的 `wrong_stitch: 0` 行，见退化锚行）。**不做：**不重分段（episode 边界属 M14 上游；本算子只合并、不切分）；不推断接缝动作内容（接缝是已知中断，零 LLM 机械占位属 M15 消费面，3.15.4）；不判重（M3）；不打任务标签（`task_name` 是摘要卡滚动线索名——工具内部结构，进 trace 与判定证据，用户 Schema 产出物属 M5）；不跨会话、不跨批缝合（hard-split 边界不可缝，3.16.4 作用域行）；不做真并发/帧多重归属（单前台屏无真并发 [65]，2.1.2 / 8.1）。

模块	职责	边界	依赖
M16 stitch	把会话内碎片保守缝合为线索：单调选池 LLM 判定 × 机械先验合取 + 有界二遍复评；被并 episode 壳置 <code>stitched</code> 、幸存信封 Record 重绑、 <code>below_min_len</code> 短段救援翻转（②c）；机械标定 <code>seam_indexes</code>	不重分段（M14）；不摘取动作（M15）；不判重（M3）；不跨会话/跨批；不做帧多重归属	M1, M8, M9

3.16.2 输入 / 输出

方向	内容
输入	按会话独立执行（ <code>session_id</code> 分组，批内位置序即会话序——M10 整会话装箱保证，3.10.3）。候选流 = 会话内 <code>status="active"</code> 且 <code>record.kind="sequence"</code> 的 episode 信封 + （ <code>stitch.rescue_short = true</code> 时） <code>noise_attribution == ("segment", "below_min_len")</code> 的帧信封按 连续 run 重组 的救援候选（3.16.4 救援行； <code>reason="noise"</code> 的噪声帧不入候选池），按会话序合流； <code>[stitch]</code> 参数（5.2）；LLM profile（ <code>stitch.llm</code> ，纯文本判定——摘要卡无图）。
输出	命中并入的候选：幸存信封 Record 重绑（成员按序键升序拼接： <code>record.id</code> 不重算——M7 手术先例，3.7.3）、被并 episode 信封 <code>status</code> → <code>"stitched"</code> （壳终态，M11 第四路由仅计数，3.11.2）；救援命中：成员帧 <code>dropped_noise</code> → <code>absorbed</code> 翻转 + 计 <code>rescued_short</code> （单位 = 帧）；每个幸存线索信封盖章 <code>PipelineItem.thread_id = record.id</code> （单碎片线索亦然，4.1）并挂 <code>seam_indexes</code> duck 标（无缝隙 = 空元组；坐标语义见 3.16.4 接缝行）与碎片跨度表 duck 标（供 M11 组装 <code>_meta.stream.fragments</code> 与 M5 按碎片配额，6.3/3.5.2）；返回值 = 传入的同一列表对象（4.3 契约 ②c）。 <code>on_error="fail"</code> 且判定修复耗尽时仅 episode 候选信封置 <code>status="failed"</code> （3.16.6）。

信封变化示例（规范验收场景 V2「单交叉」：任务 A 被任务 B 打断——会话 `sess-0012` 内 segment 已产出 3 个 episode；②c 状态写入 = 只改既有元素状态 + 幸存信封 Record 重绑，无删除/重排/替换）：

```
缝合前 (3 个 episode 信封均 active, session_id="sess-0012", 会话序 = 批内位置序) :
#12 e0 (点外卖·前半, 4 成员, order_span=[0,3])      id=9c31f5a2d84e07b6
#13 e1 (回复微信消息, 3 成员, order_span=[4,6])      id=4e8a02c97d15f3b0
#14 e2 (点外卖·提交订单, 2 成员, order_span=[7,8]) id=b57d1e04a92c86f3
一遍逐候选 (3.16.4) :
e0: 池空 → 判定照常发起 (固定「(当前无开放线索)」行) → verdict=new → 开线索 T0
    (task_name 自举「点外卖」)
e1: 池=[T0] (卡 1) → verdict=new → 开线索 T1
e2: 池=[T1, T0] (最近活跃降序: T1=卡 1、T0=卡 2) → verdict=resume, thread_ref=2;
    机械先验腿 app_overlap (App 交集) 与 entity_overlap (实体「老王麻辣烫」跨碎片
    重叠) 命中 → 并入 T0: 幸存信封 #12 Record 重绑 members = e0 u e2 成员
    (序键升序, 6 成员, id 不重算); #14 壳置 status="stitched"
二遍 (T19 有界复评) : 复评候选 = T1 (单碎片线索) → 维持 new → 零并入
接缝标定 (T20) : T0 拼接对 (成员下标 3 与 4) 的会话序间隙 [4,6] 含 T1 的 3 帧
    ⇒ seam_indexes = (3,), interrupted_by = ["回复微信消息"]
缝合后:
#12 active (线索 T0: 2 碎片, thread_id=9c31f5a2d84e07b6, seam_indexes=(3,))
#13 active (线索 T1: 1 碎片, thread_id=4e8a02c97d15f3b0)
#14 stitched (壳, 不写主输出、不写 rejects、仅计数, 3.11.2)
M10 计量: stitched += 1; counts.threads = episodes - stitched = 3 - 1 = 2 (post-emit tally 导出式, 3.10.3)
```

②c 的完整契约文本见 4.3 (含幸存者规范句：一遍幸存信封恒为线索创始信封、二遍方向相反——目标线索幸存、候选信封作壳；救援翻转是②b 双向豁免的 M16 延伸，仅限救援命中)。

3.16.3 数据结构与 API

```
class StitchStage(Stage):
    name = "stitch"
    def __init__(self, cfg: ResolvedConfig): ...
    async def run(self, batch, ctx) -> list[PipelineItem]: ... # 返回传入的同一列表 (②c 状态改写
                                                                # + 幸存信封 Record 重绑)

@dataclasses.dataclass(frozen=True)
class ThreadCard:
    index: int # 2026-08-14 收参: 一张线索摘要卡的线索侧取值
    task_name: str # 卡片在池中的 1 基编号 (= 提示词里的线索编号)
    members: Sequence[Record] # 线索当前任务名; 为空渲染占位符
    span: tuple[int, int] # 线索当前的全部成员帧, 按会话序
    fragment_count: int # 线索的会话序跨度 [首, 尾]
    # 线索当前的碎片数

def render_thread_card(card: ThreadCard, candidate_head: Record | None,
    cfg: ResolvedConfig) -> str
    # 渲染一张开放线索摘要卡 (逐行结构见下); candidate_head
    # 非 None 时追加「接续对」变更证据行 (E5 判定对)

def build_stitch_prompt(thread_cards: Sequence[str], candidate_card: str,
    cfg: ResolvedConfig) -> PromptBundle
    # 3.16.4 模板的确定性组装; 线索卡由调用方按最近活跃降序
    # 编号呈现 (1 起, 缓解位置偏差 [77]), 候选卡恒居末;
    # 池空时渲染固定「(当前无开放线索)」行

async def judge_stitch(thread_cards: Sequence[str], candidate_card: str,
    ctx: RunContext, record_ids: tuple[str, ...] = ()) -> Mapping | None
    # 一候选一次判定: 经 complete_validated(schema=stitch_schema());
    # stitch.votes > 1 时同一提示词 n 次采样、(verdict, thread_ref)
    # 完整判定严格多数决 (3.16.4 votes 行) ——不足严格多数返回
    # None (调用方回落保守结局); 每候选定案后由 stage 发一条
    # stitch.judge 事件 (3.16.6)
```

摘要卡 (digest card)：判定证据的确定性结构化载体，全部字段自 episode 成员的 `frame_digest` / `tree_diff` (4.3 共享 helper) 与信封簿记可达——链序上 `extract` 后置，缝合运行时批内无任何 **Transition**，证据面全部为帧摘要级 (resumption 判定单元「挂起目标尾动作 × 候选恢复首动作」[65] 相应降格为**线索尾帧摘要** × **候选首帧摘要**对的帧级承载)。线索卡逐行：[线索 {i}] 任务名：{task_name} (i = 呈现序 1 起编号；未命名渲染「(未命名)」) → App 集合：(成员 App 集排序顿号连接，空集渲染「(未知)」) → 序号跨度：[first, last] | 帧数 n | 碎片数 F → 首帧摘要：→ 尾帧摘要：→ 接续对 (线索尾帧 → 候选首帧) 变更：(该线索尾帧与候选首帧的 `tree_diff` 确定性文字化——增/删/文本变化节点数、变更比例、应用切换/标题变化, E5 判定对的变更证据行)；候选卡逐行：[候选碎片] 类型：分段产出|短段救援 → App 集合：→ 序号跨度：[first, last] | 帧数 n → 首帧摘要：→ 末帧摘要：。卡内嵌入的每个帧摘要沿用 `segment_digest_max_chars` 同名键语义截断 (`stitch.digest_max_chars`, 5.2)；卡的结构化字段有界由构造保证。App / activity / title 提取循环与 `diff` 文字化由本模块自带副本 (先例 `extract` 的 `_diff_text` 副本

——算子互不依赖，共享渲染仅 `frame_digest` / `tree_diff` 两枚下沉第 4 章，其余模块内自持）；**数据依赖声明**：activity 依赖采集侧 dump 将其写入 UI 树 `extra`（该字段常缺席，4.1 注），缺失时先验腿③静默失效——析取降格可接受（3.16.4 先验行）。

判定内部 Schema（`schema_engine.stitch_schema()`，3.8.1 内部 Schema 清单：不计入 `report.schema_engine.resolved_at`、不经过 L2.5）。规则同族：关键字集 $\subseteq$ 既有内部 Schema 关键字集、无 `uniqueItems`、可空以类型联合表达、**全键 required**（OpenAI strict 兼容，3.8.1）；`thread_ref` = 池内线索卡的 **1 起呈现序编号**（Schema 不设界——Schema 看不到池大小，域校验由代码侧执行）；`reason` **恒请求**（判定量级小——每会话 $\approx$ episode 数次调用，零额外 token 原则的成本面不适用；votes 聚合亦需按多数簇取 `task_name` / `reason`，3.16.4 votes 行）；`confidence` 仅作 trace 观测、**不进判定门槛**（口头置信度饱和且系统性过高 [79]，去 `confidence` 腿见 3.16.4 先验行）：

```
def stitch_schema() -> dict:
    return {"type": "object",
            "properties": {"verdict": {"type": "string", "enum": ["resume", "new"]},
                           "thread_ref": {"type": ["integer", "null"]},
                           "task_name": {"type": "string"},
                           "reason": {"type": "string"},
                           "confidence": {"type": "string", "enum": ["high", "medium", "low"]}},
            "required": ["verdict", "thread_ref", "task_name", "reason", "confidence"],
            "additionalProperties": False}
```

代码侧后校验（M8 之后，确定性收窄——3.14.4 first-wins 建表、3.13.4 归一化同则）：`thread_ref` 非整数、越界（ $\in [1, \text{池大小}]$ ）或与 `verdict` 组合矛盾（resume 而 null） $\Rightarrow$ 按保守结局收窄：episode 候选判 new、救援候选按未命中。

3.16.4 算法与流程

按会话独立执行的两遍结构（候选分两型，处理规则不对称）：

- 一遍（单调贪心选池）**：候选按会话序逐个处理，每候选一次调用，提示词呈现池内全部开放线索摘要卡（按最近活跃降序编号 [77]）+ 候选摘要卡，输出 `thread_ref` | new。- **episode 候选**：池空时判定照常发起（呈现零张线索卡，`verdict` 恒 new、`thread_ref` 恒 null——`task_name` 由此自举，是线索命名的唯一来源）；判 new 或先验全不命中 $\rightarrow$ 开新线索；命中（LLM 判 resume $\wedge$ 机械先验合取命中） $\rightarrow$ 并入：幸存信封 Record 重绑、候选壳置 `stitched`、线索摘要卡滚动更新（尾帧摘要/跨度/任务名）。- **救援短段候选**：永不开新线索。池空 $\rightarrow$ 跳过判定（零调用），维持 `dropped_noise`；池非空 $\rightarrow$ 判定，命中 $\rightarrow$ 并入 + 成员帧 `dropped_noise` $\rightarrow$ absorbed 翻转（②c③）计 `rescued_short`；未命中（含判定失败） $\rightarrow$ 维持 `dropped_noise` 原 reason。- **池满且需开新**（仅 episode 候选触发） $\rightarrow$ 按逐出优先级挑一条封闭（封闭仅发生于池满逐出，无主动封闭——完成感知封闭腿已撤除：extract 后置使收尾动作模式不可判，记入 8.1 非目标 ⑥ 与 8.4 演进注记）：① 挂起跨度超 `stale_gap_steps` 者优先（0 = 该腿失效） $\rightarrow$ ② LRU 兜底。**封闭 $\neq$ 终结**：被逐出线索不再出现在一遍卡集中，但保留在二遍复评目标集并照常产出（PIRA 顺序线程记忆基线 PIRF 靠反思删除控规模的批处理对应物 [64]；27% 的挂起超过 2 小时才恢复 [81]、时长分布特征使重叠活动识别 +11.36% [66]；同形制 SOTA 先例的顺序贪心亦不设终结 [87]）。
- 二遍（有界复评）**：复评候选 = 一遍结束时的单碎片线索，按其碎片会话序逐个处理；每候选一次调用（同一遍选池形制），池 = 该会话全部其他线索（最近活跃降序，超 `max_open` 张时按与候选跨度最近截取）；命中（判定 $\wedge$ 合取） $\rightarrow$ 并入，**方向相反**：候选信封作壳、目标线索信封幸存（幸存者规范句见 ②c，4.3；fragments 按会话序重排、episode_id / thread_id 随幸存信封）。**目标集取活视图**（复评中的并入即时更新各线索跨度与卡片）；已并入者不再作候选；无其他线索时跳过（零调用）。预算 $\leq$ 单碎片线索数（自然流每小时约 +10—15 次调用，全链占比 <5%）。修正顺序贪心的漏缝自增殖（无修复贪心劣于 batch 且次序依赖、n=1 局部重聚类即追平 batch [74]；merge-only 是增量法中质量最差 [74]；后见上下文显著有效 [87]）。**残差声明**：池截取排除目标、双多碎片线索误分裂不在修复面内——由真机实测门禁兜底（错缝 FPS = 0 验收线，3.16.7）；更重的簇修复机器（图度量分类器 / 全局演化图标签传播）已评估、按规模不采——会话内碎片 $\leq$ 数十，n=1 复评够用且零训练 [78]。
- 接缝标定**：两遍定案后，对每条线索计算 `seam_indexes: tuple[int, ...]` 并挂 duck 标（M16 盖章，4.1；单碎片线索/无缝 = 空元组）。**接缝判据**：拼接对构成接缝 $\Leftrightarrow$ 两成员的会话序间隙内含 ≥ 1 个归属其他线索的帧（absorbed 于异线索碎片）；间隙仅含噪声帧 / 本线索救援帧时不是接缝——该对照常送 M15 摘取，与 v1.8「转移以成员相邻为准、剔噪对照常摘取」惯例（3.15.2）完全一致，同一物理情形单一处理。推论：接缝的 `interrupted_by` 恒非空（T10 占位 `detail.interrupted_by`，3.15.4）。**坐标规范句**：元素 = 接缝对左成员在重绑成员元组中的下标，与 `Transition.index` / `steps[].index` 同坐标，值域 $[0, \text{len(members)} - 2]$ ；与 `_meta.stream` 的 `order_span` 会话序键空间无换算关系（下游切片依据见 6.3 包络规范句）。`seams` 计数 = 满足判据的拼接处数（接缝唯一计量点，6.4）。

缝合判定模板（确定性拼接，逐字冻结于 CONTRACTS §10.11；保守偏置写在模板文本，不随配置变化）：

```
system:
你是屏幕操作流的线索缝合审核员。下面给出当前会话中 {P} 条开放线索的摘要卡
（按最近活跃降序排列）与一张候选碎片摘要卡。
判断该候选碎片是恢复其中某条线索（用户切回了之前挂起的同一任务），还是开启一个新任务：
- resume: 候选与某条线索是同一任务的延续—任务实体跨碎片延续（订单号、地点、商品、
  联系人等再次出现）、返回同一页面继续操作、或 App 与操作语境明确承接；给出该线索编号。
- new: 候选是一个新任务。
保守偏置：仅在证据明确指向同一任务时判 resume；证据不足、模糊或仅有表面相似
（同 App 不同任务、同类页面不同对象）时一律判 new—错缝的代价高于漏缝。
若当前无开放线索，恒判 new。
task_name 用一句话概括任务：resume 时给出该线索合并候选后的任务名（滚动更新），
new 时给出新任务名。
{stitch.context}                                ← 可选域上下文；缺省省略此行
输出必须是符合以下结构的单个 JSON 对象，不输出任何其他内容：
{"verdict": "resume"|"new", "thread_ref": <线索编号|null>,
 "task_name": <一句话任务名>, "reason": <一句话理由>,
 "confidence": "high"|"medium"|"low"}
```

```
user（单条消息多 text Part：线索卡在前—按最近活跃降序、【线索 {i}】以 1 起编号；
  候选卡恒居末；池空时以固定行「(当前无开放线索)」替代线索卡段）：
【线索 {i}】{线索摘要卡（逐行结构见 3.16.3）}
【候选碎片】{候选摘要卡（逐行结构见 3.16.3）}
```

其余关键设计的精确定义：

设计点	定义
保守偏置合取 (<code>bias="conservative"</code> , 默认)	并入需 LLM 判 resume $\wedge$ 机械先验命中 。LLM 面对杂乱 GUI 流的系统性偏差方向是 过连接 （PIRA 噪声消融 precision 92→51 而 recall 反升，原文 "trigger-happy" [64]；LLM 回避「以上皆非」宁可硬连、准确与弃权此消彼长 [76]），故 LLM 单腿不足为凭。先验白名单（析取三腿，任一命中即过；trace 记名 <code>app_overlap</code> / <code>entity_overlap</code> / <code>same_page</code> ，3.16.6）：① App 交集非空；② 候选首帧摘要 $\times$ 线索尾帧摘要 实体重叠 （可见文本实体跨碎片出现）；③ 返回同一页面 （候选首帧页面标识 == 线索某 碎片尾帧 页面标识，页面标识 = <code>app</code> + <code>activity</code> (+ <code>title</code>)——cue-guided resumption：「返回同一页面」是任务恢复的强前兆线索 [80][81]）。③ 补齐两个失效面：同 App 恒真使①失去区分度时②③提供正交证据、跨 App 时①为空由②兜底；activity 缺席时③ 静默失效 （3.16.3 数据依赖声明）。 去除 confidence 腿 ：口头置信度门槛饱和且系统性过高 [79]， <code>confidence</code> 字段保留仅作 trace 观测。 时间衰减降格 ：候选与线索尾的会话序跨度超 <code>stale_gap_steps</code> （0 = 不启用）时先验降格为 须两腿命中 （挂起时长分布特征显著助益重叠活动识别 [66]）。 <code>bias="llm"</code> 档跳过先验合取（纯 LLM 判，审计/消融用）。
调用与校验	每候选 1 次调用 (<code>votes > 1</code> 时 $\times n$)，经 <code>complete_validated(schema=stitch_schema())</code> （3.8.3）；temperature 不另设（取 profile 默认—— votes 采样亦同 ，同温同前缀吃 prompt 缓存，无 <code>sc_temperature</code> 键）。会话内候选流 严格串行 （单调选池的池状态是逐候选演进的）， 会话间同样按批位置序（= 会话序）串行 ——确定性事件序、零 RNG；与 M14 跨会话窗口并发不同，M16 的并发仅存在于 votes > 1 的采样 gather 内 （profile 信号量约束）；无 rng 消耗（种子豁免免面不变，2.6）。线索构成以 LLM 输出为条件（classify 分池 / segment 边界同款条件化声明，2.6 幂等行）；同输入同 seed 逐字节可复现。调用量（1 小时自然流 $\approx$ 2400 帧 / 40 episode / 25 thread 口径）：一遍 $\approx$ 40（每 episode 候选一调；救援候选仅池非空时 + ϵ ）、二遍 $\approx$ 10–15；缝合合并后下游 quality/annotate/verify 以线索为单元、调用下降，全链行和 $\approx$ 1.1 $\times$ 帧数 + 3 $\times$ threads，stitch 全口径占比 $\approx$ 2.0%（<3%；votes=3 时判定 $\times 3$ 、占比 $\approx$ 5.8% <8%）。
救援短段 (<code>rescue_short=true</code> ，默认)	判别载体 = 帧信封的 <code>noise_attribution == ("segment", "below_min_len")</code> duck 标（M14 剔噪时盖章，3.14.4）； <code>reason="noise"</code> 的噪声帧不入候选池。 连续 run 重组 ：会话序上连续的 <code>below_min_len</code> 帧（中间无任何其他帧）重组为一个救援候选，与 segment 原切分不再一一对应（相邻两短段合为一候选；混合任务 run 因先验难命中而维持 dropped——保守面兜底）。机理：用户在切换前密集执行收尾动作（段落完成率基线 0.78/min $\rightarrow$ 切换前 10.9–12.8/min [81]），任务收尾帧天然易成短段、聚集在切换点旁。命中 $\rightarrow$ 并入 + 帧翻转（② $\circ$ ③）、 rescued_short 累加翻转帧数 （单位 = 帧，非救援事件数）；未命中 $\rightarrow$ 维持 <code>dropped_noise</code> 原 reason 落 rejects。 <code>below_min_len</code> 计数器为发生计数（帧口径），救援 不回退 （3.14.4 ③）。 相邻救援不盖接缝 ：会话位置紧邻的拼接对是真实转移，照常送 M15 摘取（接缝判据行）。
votes 多数决（ <code>votes=1</code> 默认不启用）	votes = n (≥ 3 奇数) 时同判定 n 次采样， 聚合键 = (verdict, thread_ref) 完整判定的严格多数 ($> n/2$)；任何不足严格多数的分裂（含 verdict 多数但 thread_ref 分裂）一律回落保守结局——episode 候选 = <code>new</code> 、救援候选 = 未命中； <code>task_name</code> / <code>reason</code> 取多数簇内首个采样。定位：votes 是 口头置信度门槛的正规替代 （采样一致性是可靠的不确定性信号 [33]，口头置信度被证不可靠 [79]）；边界：自一致高 $\neq$ 对——votes 治方差（漂移）不治偏差（过连接） ，与机械先验合取不可互替 [89]。 路线选型 （业界两路线对照）：采「单模型多次」（self-consistency [33]）而非「多模型评审团」（PoLL [32]）——过连接是 跨家族共享偏差 （PIRA 消融 GPT 系与 Gemini 系同向 trigger-happy [64]），异构裁判会把共享偏差投成多数、且评审团有效独立票仅 ≈ 2 [86][89]；部署纪律为单端点单模型。若第二模型家族进场， <code>stitch.judges</code> 可镜像既有 <code>verify.judges</code> 模式作纯配置扩展（8.3 O8）。成本 = 判定调用 $\times n$ （同模型同前缀吃 prompt 缓存）。
重绑与身份链	幸存信封 Record 重绑： <code>members</code> = 两方成员按序键升序拼接的新元组， <code>record.id</code> 不重算 （M7 手术先例，3.14.4 拼装行）； <code>episode_id</code> = 幸存信封 <code>record.id = thread_id</code> （stitch on 时语义为线索 id；off 时二者天然同值——概念性陈述，off 时 <code>_meta.stream.thread_id</code> 键不在场）；碎片原 <code>episode_id</code> 记录于 <code>_meta.stream.fragments[].source_episode</code> （cause $\in$ "origin" "resumed" "rescued"，6.3）。 steps 编号 ：线索的 <code>steps[].index</code> 全线索连续 0.. $n-2$ ——重绑先于 M15， <code>Transition.index</code> 恒等元组下标、 <code>len(transitions) = len(members) - 1</code> 不变量与 emitter 渲染三处约束的唯一解（3.15、4.2）。

作用域	不跨 session、不跨 batch；hard-split 边界不可缝（ <code>session_split</code> 标照旧 + WARN 提示调大 <code>batch_size</code> ，3.10.3）； <code>segment_on_error="keep"</code> 的整会话降格 episode 照常入池（合法 episode，3.14.6）。
幂等	已盖章 <code>thread_id</code> 的信封跳过（重入零额外调用）；M7 修复路径不重跑本 stage（ <code>wrong_stitch</code> 缺陷 mark-only，不拆线，3.7.3）。
退化锚	单碎片会话 / 全 new 判定 → 产出 = v1.8 形态（thread = 单碎片、fragments 长度 1、零接缝）； <code>stitch.enabled = false</code> → 主输出、 <code>rejects</code> 、 <code>report.json</code> 逐字节等价 v1.8——依赖条件在场规则：counts.stitched/threads、 <code>report.stream.stitch</code> 子块、 <code>batch.end</code> 新字段、 <code>_meta.stream</code> 新键（ <code>thread_id / fragments / resumed</code> ）仅启用时在场（6.3/6.4）。例外恰两处（均无条件设计）：① dry-run stderr 的 <code>stitch_calls=0</code> 行（3.10.3，v1.8 <code>segment_calls</code> 先例）；② <code>streamxverify</code> 报告的缺陷词表行 <code>verify.defects.wrong_stitch: 0</code> 与序列评审 system 词表行——缺陷词表是 3.7.2 四处同步的单一闭集，不随本开关条件化（真机复验：stitch off 重跑 examples/stream，counts / rejects 全等，仅该一行新增）。
上下文预算（v1.11）	判定 profile 声明 <code>context_window</code> 时（未声明 = 预算关闭，行为与 v1.10 一致；机制见 3.9）：缝合判定 prompt 的卡池结构静态有界（≤ max_open + 1 张卡 × digest 项，3.16.3 构造保证）⇒ 不做运行期动态裁剪，改由 M1 静态最坏预检把关——最坏 est > <code>input_budget</code> → WARN（不自动缩 <code>max_open</code> ——改语义须用户动手：调大 <code>context_window</code> / 缩 <code>digest_max_chars</code> / 缩 <code>max_open</code> ，3.1.4）。运行时兜底 = M9 咽喉终检（V16）+ 既有 <code>on_error="keep"</code> 保守结局（3.16.6：episode 候选开新线索存活、救援候选维持 <code>dropped_noise</code> ），无专属降级重试面。

3.16.5 配置项

[stitch] 键表（与 5.2 一致，5.2 为配置规范属主）：

键	类型 / 默认	说明与约束
<code>enabled</code>	bool / <code>false</code>	总开关：true ⇒ <code>segment.enabled = true</code> （M1 约束，3.1.4）。false = 不入链、主输出/rejects/report.json 与 v1.8 逐字节等价（3.16.4 退化锚行）。[stitch] 有 payload 而本键 false ⇒ M1 no-op warning（3.1.4）。
<code>llm</code>	str / <code>"default"</code>	判定 profile 引用；启用时计入密钥解析 / <code>--probe</code> / 存在性引用集，不入 vision 校验集（纯文本证据，无视觉必需，3.1.4）。
<code>max_open</code>	int / <code>4</code>	开放线索池容量。锚点：真实桌面日志的挂起窗口均值 3（S.D.≈2）+ 1 条活跃 [81]；移动域穿插深度更浅（仅 22.6% 任务存在穿插 [90]），4 为宽松上界。
<code>bias</code>	<code>"conservative" "llm" / "conservative"</code>	判定偏置：默认 LLM × 机械先验合取（3.16.4 保守偏置行）； <code>"llm"</code> = 纯 LLM 判（审计/消融用）。
<code>rescue_short</code>	bool / <code>true</code>	below_min_len 短段按连续 run 重组先进候选池（3.16.4 救援行）；false = 短段维持 <code>dropped_noise</code> （v1.8 行为）。
<code>repass</code>	bool / <code>true</code>	有界二遍复评（3.16.4 ②）；false = 纯一遍贪心。
<code>stale_gap_steps</code>	int / <code>0</code>	时间衰减阈值（会话序号差；0 = 不启用）。双职：① 先验降格（超限须两腿命中，3.16.4 保守偏置行）；② 池满逐出优先腿（3.16.4 ①）。与 <code>stream.gap_steps</code> 语义区分——后者是会话切分规则（M2），本键是会话内线索挂起跨度。
<code>digest_max_chars</code>	int / <code>400</code>	卡内嵌入的每个帧摘要截断上限（沿用 segment 同名键语义，3.16.3）。
<code>context</code>	str / <code>""</code>	可选域上下文（何为「同一任务」的领域提示），注入模板可选行；不是判据定义——保守偏置内置于固定模板，零配置可用。
<code>votes</code>	int / <code>1</code>	判定稳定化采样数：1 = 不启用（单调用）；> 1 须为奇数（偶数 = CONFIG_ERROR，3.1.4），n 次采样对（ <code>verdict</code> , <code>thread_ref</code> ）严格多数决（3.16.4 votes 行）。成本 ×n。
<code>on_error</code>	<code>"keep" "fail" / "keep"</code>	单判定修复耗尽处置（3.16.6）；fail 仅施于 episode 候选信封。

[class.<name>.stitch] 不存在：stitch 在 classify 之前执行，类标签尚不存在（链序因果，segment 同则；5.2 按类覆盖白名单表注明缘由）。

3.16.6 错误处理

错误码 `stitch_invalid`（7.6，v1.9 增行）两形态——候选两型不对称：

<code>stitch.on_error</code>	行为
<code>"keep"</code> （默认）	该判定放弃：episode 候选开新线索存活（保守缺省结局；线索 <code>task_name=""</code> ，摘要卡渲染为「(未命名)」），留痕两件 = error 事件（kind = <code>stitch_invalid</code> ， <code>stitch</code> 通道）+ 计数器 <code>stitch.failures</code> ，不写 <code>item.errors</code> （S26 归因防污染同则，3.14.6）。与 segment S26 三件套的差异：条件在场规则（3.16.5 退化锚）封闭了 <code>_meta.stream</code> 的 v1.9 键清单（ <code>thread_id/fragments/resumed</code> ），无 <code>stitch</code> 降格 <code>_meta</code> 腿—— <code>degraded</code> 键保持 segment 专属；救援候选维持 <code>dropped_noise</code> 原 reason + 同款两件留痕。
<code>"fail"</code>	仅 episode 候选信封置 <code>status="failed"</code> 、 <code>StageError(stage="stitch", kind="stitch_invalid")</code> 入 <code>item.errors</code> ⇒ rejects——成员帧维持 <code>absorbed</code> （M7 fail 先例）；②c 授权面不含 <code>absorbed / dropped_noise</code> → <code>failed</code> 的帧迁移）；救援候选不适用 <code>fail</code> 路径，判定失败一律按未命中处理（维持 <code>dropped_noise</code> ）。

事件（7.2，v1.9 增行；通道 "stitch" = stage 名——_TRACE_CHANNELS 10→11，事件名前缀即通道、error 事件按 stage 自动归属，零路由代码）：

事件名	通道 / stderr 级别	触发点	payload 字段
stitch.judge	stitch / — (trace-only, 无 stderr 镜像)	每候选判定定案后 (votes 聚合之后；一遍与二遍均发)； record_ids = [候选碎片首成员 id]。	session_id、candidate ("episode" "rescue")、repass (bool, false = 一遍 / true = 二遍)、verdict (votes 分裂回落时记保守结局 "new")、thread_ref、confidence (仅观测, 3.16.3)、priors (机械先验命中腿列表, ⊆ {app_overlap, entity_overlap, same_page})、merged (bool, 是否实际并入——LLM 判 resume 而先验未过时 verdict 与 merged 可分离)；条件字段: votes_split (= true, 仅 votes 严格多数不成立回落时携带)、task_name ↑↑、reason ↑↑ (votes 分裂时二者不携带)、target_thread_id (仅 merged 时携带 = 目标线索 id)。
stitch.thread	stitch / — (trace-only)	会话缝合定案后每线索一条；record_ids = [幸存信封 record.id]。	session_id、thread_id、task_name ↑↑、fragments []{ order_span, member_count, cause, source_episode } (碎片跨度表)、seam_indexes。

↑↑ task_name / reason 为 LLM 自由文本——入 _FREE_TEXT_KEYS 脱敏集（7.4，v1.9 增 task_name）：none 档剥除、refs 档起携带；其余 payload 字段均为结构字段，全档保留。

计数与归属：report.stream.stitch = {stitched, rescued_short, seams, judgments, repass_judgments, failures}（M16 属主，6.4——judgments / repass_judgments 计一遍/二遍**逻辑判定数**：每候选一判、失败不计；votes > 1 放大调用数不放大判定数）；counts.stitched（壳终态 tally）与 counts.threads（= episodes - stitched 导出式）由 M10 计量（counts.* 属主不变，3.10.3）——threads 仅落 counts.threads 单点，避免双落点；batch.end payload 增 stitched / threads（仅启用时携带，7.2）。**壳的范围规范句**：stitched 仅计被并 **episode 信封壳**；救援短段无信封形态（由帧重组），命中只计 rescued_short 帧翻转、**不产生壳**。stderr 进度条与终版摘要为固定键集，stitched 不显示（fanout / episodes 先例，报表与 batch.end 可见——有意为之；v1.10 U18：本约束收窄为 **plain 面专属**，rich 面板状态账展示 stitched/threads、与 report.counts 口径对齐，7.7）。--strict 交互（3.11.2 / 2.4 补注）：stitched 壳与 rescued 帧均不构成 rejects——同输入开启 stitch 后（短段被救援而不再落 rejects）strict 结果可能 1→0，**属预期**。

3.16.7 背书

问题形态与偏差方向的形式化出自 PIRA-Bench [64]：其把屏幕流定义为「多线索穿插 + 噪声」（T = U任务子轨迹 ∪ 噪声，任务子轨迹为**非连续帧子集**），并以噪声消融证实 LLM 的系统性偏差方向是**过连接**（PIRF 框架下 GPT 系 precision 92.23→50.52 而 recall 反升、Gemini 系同向，原文 "trigger-happy"）——这是保守偏置（LLM ∧ 机械先验合取，3.16.4）的直接依据，与指代消解域「回避以上皆非宁可硬连」及弃权研究「默认过度承诺」的同向量化 [76] 三方互证；其线索级综合指标（S_final = F1 × FPS_norm，错缝以乘法惩罚合成）与负样本协议（纯噪声会话必须 0 缝合）为真机实测门禁提供度量（注：0 被引新基准，自报数字权重打折，故充分性只能实测——错缝 FPS = 0 为验收线）。单调选池形制有**同任务同形制的直接 SOTA 先例** [87]：簇摘要呈现 + LLM 归簇或判 new + 顺序贪心（GreedyDisentangle）在对话解缠标准基准全指标超 per-pair 与非 LLM 方法，其 subsequent-context 显著有效结论亦是二遍复评的第三依据（风险注记：小参数量开源模型上该形制性能骤降的报告在案，实测由门禁兜底）；对话解缠谱系的贪心链接标准解码、并发线程 ≤3 占 46.4% 与 VI / 1-1 overlap / link-F1 指标三件套见 [75]。有界二遍复评的机制依据来自增量实体消解：无修复贪心劣于 batch 且次序依赖、**n=1 局部重聚类即追平 batch**，merge-only 是增量法中质量最差 [74]——会话内碎片 ≤ 数十的规模下，更重的簇修复机器（图度量分类器 + 主动学习 / 全局演化图概率标签传播）与全局 LLM 聚类（自身需防幻觉护栏、记录集构成显著影响质量）已评估、按规模不采 [78]。resumption 判定单元「挂起目标尾证据 × 候选恢复首证据」出自 interleaving/concurrent 活动建模的经典工作 [65]（链序上动作后置，本模块降格为首末帧摘要对承载，3.16.3）；时间衰减先验（挂起时长分布特征使重叠活动识别 +11.36%）出自 interleaved ADL 数据集系列 [66]；「返回同一页面」先验腿的机理是 cue-guided resumption [80]。**max_open = 4** 的实证锚点是真实桌面日志的**挂起窗口均值 3 (S.D.≈2) + 1 条活跃** [81]——同文献「27% 的挂起超过 2 小时才恢复」支撑「封闭 ≠ 终结」、「切换前收尾动作密集（0.78 → 10.9–12.8/min）」支撑短段救援的机理；移动域佐证：人工标注 1414 个真实手机任务仅 22.6% 存在穿插，桌面锚在移动域为宽松上界 [90]；working spheres 多任务粒度与手机中断/回访的人因基线见 [82]。摘要卡证据面的正面证据：window title 是任务识别最强单特征（85.57%）且多窗证据聚合优于单窗 [83]；精选紧凑上下文准确率**反超**全量原始历史（+10.4 pt 且 token 少 8 倍）、结构化摘要以 ~5% token 胜过其它记忆系统 [88]——反向风险的正式命名 **summarization drift**（每次压缩静默丢弃低频细节）同出 [88]，trace 全量留判定证据供审计即为此设；embedding 召回 + LLM 精判的两级任务组匹配工业近例见 [84]。votes 机制（默认关）的出处是 self-consistency [33]（单模型 n 采样多数决；一致率是可靠的不确定性信号），其边界由 2026 年两则审计钉死 [89]：前沿模型高自一致处**过度自信**（高自一致条目近半仍错——votes 治方差不治偏差，与机械先验合取不可互替）、9 裁判 7 家族评审团有效独立票仅 ≈2——加上跨模型共识研究「self-consistency 更好校准、跨模型信号更准确，但自一致性修不了系统性偏差（错误相关性 within-model 0.68 > cross-family 0.47）」[86]，共同构成拒绝多模型评审团路线（PoLL [32]）的论证：过连接是跨家族**共享偏差** [64]，评审团会把它投成多数（选型记录见 8.3 O8）；多候选选择式判定的位置/上下文结构敏感性 [77] 决定池卡按最近活跃降序的固定呈现序与候选位置扰动测试要求。层级与产出形态的先例：thread > fragment > step 对齐 Ego4D Goal-Step 的 goal>step>substep 三级 + is_continued 续接标志（平面分段 + 线索身份的直接先例）[69] 与 GUI 域层级标配 AndroidControl [45]；跨 App 单目标轨迹形态见 GUI-Odyssey [46]；「自然流 → 事后反推任务标注」范式与可命名性剪枝判据出自 OS-Genesis [41] 与 NNetNav [70]；视频域层级时间标注范式（FineGym / Breakfast）[71] 为三级结构的域外印证；帧多标签先例（MultiTHUMOS / Charades）[72] 经评估后**否决采纳**（帧单一归属是手术/归因/守恒的公共地基），引用记录被拒方案；嵌套结构与 during/overlaps 区间关系的

形式语义底座 (HHMM / Allen 区间代数) [73] 界定「线索包含碎片、碎片穿插」的语义而不引入区间树。问题域现状：UI 日志 interleaved 解缠是 Robotic Process Mining 的公开难题——全局法 (trace alignment / 频繁模式) 依赖「例程重复」前提，对一次性自然任务不可迁移 [67]，无分段 UI 日志例程识别的「重复例程」前提亦经 2025 年工作复核 [50][85]；学术解缠系列之外，三家工业任务挖掘产品均**无穿插解缠**（采集纪律回避 / 时间邻近分组 / 按任务录制），通信类 App「天然噪声/穿插高发」名单同出其产品文档 [68]——产品空白区，缝合质量无域内基线，护栏 = 保守合取 + 二遍复评 + 负样本协议 + 真机门禁四层 (8.1/8.4 注记)。

4. 核心数据结构与内部 API

本章是全部模块共享的类型契约。除 `PipelineItem` 的状态字段外全部为不可变（frozen dataclass）；模块间只通过这些类型与第 3 章列出的类签名交互。

4.1 记录与信封

```

Status = Literal["active",      # 存活, 继续流转
                "dropped_dup",  # M3 判重
                "dropped_lowq", # M4 低于质量门
                "dropped_verify", # M7 评审失败且策略为 drop
                "failed",       # 处理异常 (结构不可修复 / provider 错误耗尽重试等)
                "absorbed",     # v1.8 只增: 成员帧已被序列信封吸收 (M14 ②b, 3.14);
                                # 第三路由—不写主输出也不写 rejects, 仅计数 (3.11.2)
                "dropped_noise", # v1.8 只增: 噪声帧 / 短段帧 (M14: reason=noise / below_min_len, 3.14)
                                # 或 verify 修复收缩弃帧 (M7: off_task_member) → rejects (3.11.2)
                "stitched"]     # v1.9 只增: 被并 episode 信封壳 (M16 ②c, 3.16) —壳终态, 仅计
                                # 被并 episode 信封 (救援短段无信封形态、不产生壳); 第四路由
                                # —不写主输出也不写 rejects, 仅计数 (absorbed 同款, 3.11.2)

@dataclass(frozen=True)
class RecordRef:
    source_file: str          # 相对 run.input 的路径
    line_no: int | None       # 文本模态: 1-based 行号
    pair_index: int | None    # UI 模态: 文件对 index
    generated_from: tuple[str, ...] # process 模式生成样本: 种子记录 id 列表; 其余 (含 generate_only 生成样本) 为空元组—合成判据用 generator (v1.4)
    generator: Mapping | None = None # v1.2: 生成记录的 {"llm": profile 名, "style": name|None} 溯源 (3.6.2); 非生成记录为 None
    # 键集条件形 (v1.14): 恒含 llm 与 style 两键; 时间流生成的**任一生效
    # 档位表**在场时增第三键 tier_rank (该序列所属档位序号, 正整数),
    # 全缺省时维持两键—「信封只增字段」惯例 (6.3)。v1.15 注: 按类档位表
    # 要求全局表在场 (全局表为锚), 故「任一生效表在场」⇐ 全局
    # [[generate.stream.tiers]] 非空—判据与 v1.14 逐字同义, 只是
    # 值取自本行序列类的生效表 (跨类不可比, 3.6.5)
    # v1.13 时间流生成侧构造约定 (3.6.5): 成员帧的 ref =
    # RecordRef(source_file=时间流工件路径, line_no=工件行号 (1 基),
    # pair_index=None, generated_from=(), generator={"llm","style"})
    # —任一生效档位表在场 (= 全局表非空) 时为
    # {"llm","style","tier_rank"} 三键)
    # —工件是该帧的**真实来源文件**, 故走 source_file/line_no 而非
    # 空源; 序列 Record 的 ref 照 S24 继承首成员 ref (下方 Record 注)

@dataclass(frozen=True)
class ImageRef:
    path: Path; format: Literal["png", "jpeg"]; size_bytes: int
    def load_base64(self, max_px: int) -> tuple[str, str]: # (media_type, b64) 用后即弃

@dataclass(frozen=True)
class UINode:
    node_id: str; parent_id: str | None; depth: int
    role: str          # class/type 归一后的控件角色
    text: str; content_desc: str
    bounds: tuple[int, int, int, int] # (l, t, r, b) 像素
    visible: bool; extra: Mapping[str, str] # 白名单外字段原样保留

@dataclass(frozen=True)
class UITree:
    nodes: tuple[UINode, ...] # 深度优先序
    def serialize(self, max_chars: int | None = None, quantize_px: int = 0) -> str

@dataclass(frozen=True)
class Record:
    id: str          # sha256 前 16 hex (M2 定义的确定性规则)
    modality: Literal["text", "ui"]
    text: str | None # 文本模态: 抽取文本; UI 模态: None
    raw: Mapping | None # 文本模态: 原始行对象
    ui_tree: UITree | None; image: ImageRef | None
    ref: RecordRef
    kind: Literal["single", "sequence"] = "single" # v1.8 只增 (尾部追加、带默认—既有构造点零改动):
    # "sequence" = M14 拼装的 episode 序列记录 (3.14)
    members: tuple["Record", ...] = () # v1.8 只增: sequence 时为成员帧按序键升序; single 恒 ()
    # 序列 Record 字段约定 (S24): text/raw/ui_tree/image = None;
    # modality = 成员模态; id = sha256("\n".join(member_ids))[:16]
    # (拼装时定格, 成员手术不重算; v1.9: M16 缝合重绑同样不重算
    # —episode_id = 幸存信封 record.id = thread_id, 碎片原
    # episode_id 落 _meta.stream.fragments[].source_episode,
    # 3.16.4/6.3); ref = RecordRef(source_file=首成员源,
    # line_no=首成员 line_no, pair_index=首成员 pair_index,
    # generated_from=(), generator=None)—完整成员溯源由
    # _meta.stream.member_sources 承担 (6.3)
    # v1.13 生成侧构造约定 (时间流形态, 3.6.5): 序列 Record 的第二
    # 来源—M6 直装组装 (非 M14 拼装), 字段约定与公式**逐条同 S24**

```

```

# (id = sha256("\n".join(member ids))[:16], text/raw/ui_tree/
# image = None, ref = 首成员 ref); 成员 Record 的 raw = 工件行
# **全对象** (含 truth 字段)、id = M2 公式 sha256(canonical_json
# (raw))[:16], text = text_field 值的 M2 语义投影 (字符串直取 /
# 对象 canonical JSON) —工件重放时成员 id 逐字节一致 (6.5)

@dataclass(frozen=True)
class Classification:
    label: str # v1.7: M13 分类结果 (3.13) # 本信封路由标签
    labels: tuple[str, ...] # 该记录命中全集 (声明序; single 恒单元素)
    source: Literal["llm", "fallback", "inherited"]
    detail: Mapping # reason / sc 统计 / fallback 留痕 (kind, message)

@dataclass(frozen=True)
class SequenceValidationFrame:
    position: int # v1.16: M6 序列级生成钩子的单帧输入 (3.6.6) # 序列内零基位置
    frame_class: str # planner 冻结的帧类名
    payload: object # JSON-compatible 深拷贝; 钩子修改不得回写内部载荷

@dataclass(frozen=True)
class SequenceValidationInput:
    sequence_class: str # v1.16: generate.sequence_validator 的输入 (3.6.6) # 生成序列类名
    tier_rank: int | None # 该类生效档位序号; 无档位面时为 None
    frames: tuple[SequenceValidationFrame, ...] # 按序列位置排列的成员帧

@dataclass
class PipelineItem:
    record: Record # 唯一可变信封; 生命周期 = 一个批
    status: Status = "active"
    classification: Classification | None = None # v1.7: 未启用 classify 恒为 None
    dedup: DedupInfo | None = None
    scores: dict[str, QualityScore] = field(default_factory=dict)
    annotation: Annotation | None = None
    verification: VerificationResult | None = None
    errors: list[StageError] = field(default_factory=list)
    transitions: tuple[Transition, ...] | None = None # v1.8 只增: M15 写入 (3.15);
    # None = 未启用 extract / 未到站 (幕等门: is not None 跳过)
    session_id: str | None = None # v1.8 只增: 会话边界的批内载体 (S4) —M10 装箱时对帧信封
    # 盖章、M14 对追加的 episode 信封盖章 (簿记非业务逻辑);
    # M7 修复邻域查询 = session_id 过滤 + 批列表位置序
    thread_id: str | None = None # v1.9 只增: 线索身份 (3.16) —M16 对幸存线索信封盖章
    # (= record.id, 单碎片线索亦盖); 未启用 stitch 恒 None;
    # classify multi 扇出克隆复制 (3.13.4)。另有 duck 标
    # seam_indexes: tuple[int, ...] (M16 对幸存线索信封盖章,
    # 无缝隙 = 空元组; 非 dataclass 字段): 元素 = 接缝对左成员在重绑成员元组
    # 中的下标, 与 Transition.index / steps[].index 同坐标、
    # 值域 [0, len(members)-2], 与 _meta.stream.order_span 的
    # 会话序键空间无换算关系 (3.16.4; _fan_out 同复制)
    member_classifications: dict[str, Classification] | None = None
    # v1.12 只增: M13 帧级批量判决写入 (首标签序列信封, 3.13.7);
    # 键 = 成员 record.id、值恒单标签 (labels = (label, ),
    # source ∈ {"llm", "fallback"}; v1.13 增第三值 "inherited"
    # —时间流生成的帧类真值在蓝图层已知, 由 M6 直装写入,
    # 值形态与帧级判决产物完全一致, 3.6.5/3.13.7); None = 帧 pass 未运行
    # (帧分类关闭 / 降格会话 / 非首标签克隆) —幕等门 is None;
    # 扇出克隆按引用共享同一 dict (record/dedup 同族, 3.13.7)
    member_annotations: dict[str, Annotation] | None = None
    # v1.12 只增: M5 帧级逐帧标注写入 (同一执行门, 3.5.5);
    # 键 = 成员 record.id; 值语义 = 单一真相 (emitter 三值判定
    # 直读 dict 形态, 3.11.2): 占键 Annotation = annotated、
    # 占键 None = failed (成员标注不可修复)、缺键 = skipped
    # (跳过类)、dict 本身 None = 帧 pass 未运行; 克隆按引用
    # 共享 (同上); M7 手术同步随成员集删键/补跑 (3.7.3)

```

4.2 阶段结果类型


```

@dataclass(frozen=True)
class DedupInfo: kind: Literal["unique", "exact", "near_text", "near_image", "near_both", "near_semantic"]
                 cluster_key: str; kept_id: str | None      # 重复时指向被保留记录

@dataclass(frozen=True)
class Transition:
    index: int          # v1.8 只增: M15 对一对相邻成员帧的摘取产物 (3.15),
                        # 经 PipelineItem.transitions 承载 (4.1)
    action: Mapping      # 重建后位次 (恒 = 在 transitions 元组中的下标); 成员手术后
                        # 重编号—不变量 len(transitions) = len(members)-1 恒真 (S31)
    model: str           # 过 action_schema 的对象: {action_type, target, value,
    attempts: int        # description} (字段语义见 3.15)
    detail: Mapping      # 摘取 profile 的模型名
                        # 1 + L3 修复次数
                        # fallback 留痕: {kind:"extraction_invalid", message} (S16);
                        # 手术接缝重摘取: {reseamed: true} (S31); 干净摘取为 {};
                        # v1.9 只增保留键: 线索接缝占位 {kind:"thread_seam",
                        # interrupted_by:[...]} (与 extraction_invalid 并列, 零 LLM
                        # 机械占位, 3.15.4; emitter 据此推导 steps 行内 resumed=true)

@dataclass(frozen=True)
class QualityScore: criterion: str; score: float          # [0,1] 归一化
                    mode: Literal["pairwise_bt", "pointwise"]
                    detail: Mapping      # pairwise: {comparisons, wins, ties, log_theta}
                                         # pointwise: {raw_score(0-5), reason}

@dataclass(frozen=True)
class Annotation: output: Mapping      # 已通过用户 Schema (L2) 的对象
                  model: str; attempts: int      # 1 + L3 修复次数
                  usage: Usage

@dataclass(frozen=True)
class VerificationResult: verdict: Literal["pass", "fail"]
                           rounds: int; critiques: tuple[Mapping, ...]
                           defects: tuple[Mapping, ...] = ()
                           # ↑ v1.8 additive (S7): stream 缺陷表 (3.7 stream 分支), 每项
                           #   {"kind", "members", "position", "detail"} (kind 枚举见 3.7—v1.8
                           #   五值, v1.9 起六值: +wrong_stitch, 只标记不拆线);
                           #   非 stream 路径恒 (); 随信封入 _meta.verification.defects (6.3)

@dataclass(frozen=True)
class StageError: stage: str; kind: str                # 错误分类码 (7.6)
                  message: str; retryable: bool

```

4.3 Stage 协议与异常层级

```

class Stage(Protocol):
    name: str
    async def run(self, batch: list[PipelineItem], ctx: RunContext) -> list[PipelineItem]:
        """契约：① 只处理 status=='active' 的项；② 不删除列表元素（只改 status）；
        ②a (v1.7) classify 例外（仅 assignment="multi"）——可向传入列表尾部追加派生信封；
        追加物视同批内普通元素、同受 ①③④ 约束；不得删除、重排或替换任何既有元素对象
        （既有元素的 status / classification / errors 字段写入属 ①④ 的正常行为）；
        返回值仍须是传入的同一列表对象（调用方依赖列表身份）；
        ②b (v1.8) segment 例外（仅 stream 模式）——segment 可将批内既有 active 成员信封的
        status 置为 `absorbed` 或 `dropped_noise`（属①④的正常状态写入），并向传入列表
        **尾部**追加以这些成员拼装的序列信封；追加物视同批内普通元素、同受①③④约束；
        每个成员信封至多被一个序列信封吸收；不得删除、重排或替换任何既有元素对象；
        返回值仍须是传入的同一列表对象。**M7 修复路径豁免**：verify 的缺陷修复可在本批内
        将成员信封状态在 `absorbed` 与 `dropped_noise` 间双向改写（成员回收/收缩），
        此为契约①的唯一反向豁免；禁止将成员信封翻回 `active`；
        ②c (v1.9) stitch 例外（仅 stitch 启用）——授权恰三件事：其一，将批内既有 active
        episode 信封（被并方）的 status 置为 `stitched`（壳终态，属①④的正常状态写入）；
        其二，对幸存线索信封执行 Record 重绑（`members` 替换为两方成员按序键升序拼接的
        新元组；`record.id` 不重算—M7 手术先例，thread_id == 幸存信封 episode_id）；
        其三，将 below_min_len 来源信封 `dropped_noise` → `absorbed` 翻转（②b 双向豁免的
        M16 延伸，**仅限救援命中**）。**幸存者规范句**：一遍中幸存信封恒为**线索创始信封**
        （开线索者），被候选信封作壳；二遍复评方向相反—单碎片线索作候选方并入目标
        线索，**目标线索信封幸存**、候选信封作壳（fragments 按会话序重排，episode_id /
        thread_id 随幸存信封，3.16.4）。不得删除、重排或替换任何既有元素对象（重绑改写的
        是幸存信封自身的 record 字段，非元素替换）；返回值仍须是传入的同一列表对象；
        禁止将 `stitched` 壳翻回 `active`；授权面不含 absorbed / dropped_noise → failed
        的帧迁移（`on_error="fail"` 仅施于 episode 候选信封，3.16.6）；
        ③ generate 例外——返回新增子批（原批元素不修改）；④ 单条失败不得抛出到批层面，
        必须落入 item.errors 并置 status='failed'."""

LabelKitError
├─ ConfigError(errors: list[str])          # M1, 退出码 2
├─ InputError                             # M2 fail 策略触发，退出码 3
├─ ProviderRetryableError / ProviderFatalError # M9
├─ SchemaViolation(errors, raw_last_output) # M8, 记录级
└─ InternalError                          # 不变量破坏（如 M11 终检失败）

```

帧粒度与 Stage 契约 (v1.12 零改动声明)：帧级分类/标注 (3.13.7、3.5.5) 对本契约**零改动**——契约例外维持 ②a/②b/②c 三条原文，不新增例外条款：帧产物只写入信封自身的 member_classifications / member_annotations 两字段（属 ①④ 的正常字段写入），不改成员帧状态机（成员保持 absorbed）、不增删列表元素、不改链序与守恒恒等式 (6.4)。**克隆共享语义补注** (②a 的 v1.12 侧注)：classify multi 扇出克隆对两字段**按引用共享**（与 record / dedup 同族，3.13.7 扇出共享行）——帧产物描述成员帧本身而非信封路由，原/克隆行渲染同一 dict；帧级两 pass 只在首标签信封上执行（克隆判据 = classification.label != classification.labels[0]，verify S8 同款），M7 帧产物同步亦无克隆分支（克隆信封永不手术，3.7.3）；手术后原/克隆行 members 分叉由既有 repaired 位消歧 (6.3 补注)。

时间流生成与 Stage 契约 (v1.13 零改动声明)：时间流形态 (3.6.5) 对本契约**零改动**——契约例外维持 ②a/②b/②c 三条原文，不新增例外条款。理由逐条：① M6 仍遵守既有的 generate 例外（「返回新子批而非原地改状态」）——返回值只是从 list[Record] 升格为「信封列表 + 工件行列表」的富返回（PipelineItem(record=r) 裸构造无法携带 session_id / classification / member_classifications，故必须整只交付），平面路径 generate_all 的冻结签名与行为不动；② 直装序列信封自出厂即是普通 active 信封，下游六个算子按既有规则处理，无任何序列专属的状态迁移；③ 成员帧从不构造信封（噪音帧与重发帧只活在工件里），故 absorbed / dropped_noise / stitched 三态在本形态下不出现，②b/②c 的适用面为空；④ 状态机、链序与守恒恒等式（取 generate_only 退化形，6.4）全部不动。

UITree.serialize() 的规范定义 (M3 去重与 M5 提示词共用，M3 传 quantize_px=dedup.bounds_quantize_px)：深度优先遍历可见节点；每行 = " " * depth + role + (' ' + text + ' ' if text) + (' desc=' + content_desc + ' ' if content_desc) + ' [' + l, t, r, b + ']' + 非空 extra 的 k=v 列表；坐标除以 quantize_px 取整 (0 = 不量化)；超长截断规则见 3.5.2。该线性化即 ScreenAI 的 screen-schema 表示思想 [13]。

共享帧 helper (v1.8 只增，S12/S13)：frame_digest 与 tree_diff 为 labelkit/common/contracts/types.py 模块级函数（与 UITree.serialize 同处的共享渲染层，签名入 CONTRACTS §3），供 M14 分段 (3.14)、M15 摘取 (3.15)、M13 序列分支 (3.13) 与 M4 序列打分 (3.4) 共用——算子模块互不依赖，共享渲染逻辑一律落本章类型层：

```

def frame_digest(record: Record, max_chars: int) -> str
# best-effort 确定性帧摘要 (S12—UINode 封闭九字段, 包名/activity 仅经 extra 兜底可达):
# UI 模态: app      = extra 键 package|package_name|pkg 首个非空 (可见节点)
#           activity = extra 键 activity|activity_name|window_title 首个非空 (可缺省)
#           title    = DFS 首个可见非空 text
#           salient   = 可见 text/content_desc 按序去重; Button/EditText/CheckBox 类
#                   交互角色加 "*" 前缀
#           整体截断至 max_chars (serialize 截断惯例)。
# 文本模态: record.text 截断至 max_chars。
# v1.11 (V9): M14 的 digest 计算前移为会话级预计算—一切窗前每会话一次求出逐帧
# digest (预算贪心装填以逐帧成本为输入, 3.14); 接缝帧不再随相邻两窗双算
# (前移是净改善; 贫瘠护栏计算路径独立、保持不动)。本函数为记录内容纯函数,
# 签名与语义零改动。
# 摘要贫瘠判定: 可见文本节点数为 0 或摘要长度 < 8 ⇒ 贫瘠—调用方计入
# digest_poor_frames (6.4 report.stream) + 每运行一次 WARN, 指引为 segment.llm
# 配置 supports_vision=true 的 profile (v1.11/V4 改写—use_vision 键已移除,
# 窗口附图由能力推导 vision_resolved 决定, 3.14)。

def tree_diff(a: UITree | None, b: UITree | None, quantize_px: int) -> Mapping
# 结构键 (role, bounds//quantize_px, depth) 多重集匹配 (S13—node_id 非跨帧身份,
# 不得作匹配键); 仅可见节点: 0(n1+n2); 纯统计不做语义归因 (归因属 M15)。返回:
# {added:int, removed:int, text_changed:int, change_ratio:float,
#  app_changed:bool, title_changed:bool}

```

预算原语契约引 (v1.11): 上下文预算的估算与装填原语 (`margin` / `input_budget` / `embed_budget` / `est_text` / `est_image_prior` / `est_prompt` / `fit_text` / `min_window` / `classify_stage_error` 与 `ImageCostCalibrator`) 为 新 共 享 模 块 `labelkit/common/runtime/budget.py` 的模块级纯函数与类 (common 层运行时, **非本章类型层**——签名与冻结常数以 CONTRACTS 的 budget 新节为准, 机制见 3.9); 本章共享渲染层 (`serialize` / `frame_digest` / `tree_diff`) 签名零改动, 装填器 (贪心切窗等) 属算子逻辑、落各算子模块。

序列规则与 Stage 契约 (v1.16 零改动声明): v1.16 新增的 `SequenceValidationFrame` / `SequenceValidationInput` 只描述 M6 调用用户序列钩子时传入的 只读视图, 不是新的 `PipelineItem` 或 `Stage` 例外。M6 仍返回富的生成子批 (序列信封与 工件行), 不修改原批; 每个序列信封出厂即为普通 `active` 信封, 继续走既有下游链。规则、窗口、correlation、planner 状态和钩子违规不会写进 `Record`、`PipelineItem.errors` 或新的状态值; 整条 attempt 作废只表现为缺席与既有计数器。钩子取得 payload 的深拷贝, 任何用户修改都不能污染内部载荷、时间字段回填或 duplicate source。

5. 配置文件完整规格

5.1 config.toml（工具级静态配置）

键	类型	默认	说明
schema_version	int	必填	本版本固定 1。
tool.log_level	str	"info"	debug info warn error；被 CLI --log-level 覆盖。
llm.<name>	table	≥1 个	每个子表定义一个 profile，<name> 为被 project.toml 引用的名字。
llm.?.provider	str	必填	"openai_compatible" "anthropic"。
llm.?.base_url	str	必填	API 根地址。
llm.?.model	str	必填	模型名，原样透传。
llm.?.api_key_env	str	必填*	持有 API Key 的环境变量名（API Key 是唯一的环境变量用途，2.5）。* v1.6：与 api_key_envs 恰提供其一（互斥，M1 校验 3.1.4）。
llm.?.api_key_envs	array	无	v1.6 密钥池（3.9.3）：持有 API Key 的环境变量名数组（≥1 项，逐项非空且互异），与 api_key_env 互斥。池内密钥共享本 profile 其余全部字段（同 base_url、同 model——同构池，密钥选择不改变产出数据内容）；被引用 profile 的每个列出变量都须存在且非空（M1 校验）。单元素数组与 api_key_env 等价；max_concurrency 仍为池内总在上限。
llm.?.max_concurrency	int	8	该 profile 并发上限（信号量）。
llm.?.timeout_s	int	120	单次请求超时。
llm.?.max_retries	int	5	可重试错误的最大重试次数。
llm.?.retry_base_delay_s	float	1.0	全抖动指数退避基数（3.9.3）。
llm.?.supports_structured_output	bool	false	true 时结构引擎启用 L0（3.8.2）。
llm.?.supports_vision	bool	false	UI 模式所引用 profile 必须为 true（M1 校验）。
llm.?.max_output_tokens	int	4096	透传给 API。
llm.?.context_window	int	0	v1.11 新增（V6/V26，3.9.5）：模型上下文窗口（token）。0 = 未声明：该 profile 上下文预算关闭（行为与 v1.10 一致），被启用阶段引用时 M1 WARN 一次（3.1.4）。> 0 时必须满足 context_window > max_output_tokens + margin，否则 CONFIG_ERROR（预算非正）；margin = max(256, ceil(0.10 × context_window))。声明部署实效窗口，勿照抄文档（V26/[C-59]，docs/dev/PROPOSAL-context-budget.md）：同名模型随部署差数倍（Together 版 glm-5.2 为 256K、vLLM 由 --max-model-len 决定），且文档说法可能与端点实况相悖（z.ai anthropic 路由实测：裸 glm-5.2 实效窗即 input+max_tokens ≤ 2^20，官方博客的 [1m] 后缀反被拒——E2E-FINDINGS #16）——窗口只能按部署实测或保守欠声明；欠声明恒安全（只多裁不溢出）。
llm.?.temperature	float	0.0	profile 级默认；生成阶段建议在 project.toml 用 generate.temperature 调高。
llm.?.thinking	str	缺省	v1.16：可选 "enabled" "disabled"；显式值在 OpenAI 兼容与 Anthropic 两种请求的顶层写为 {"thinking": {"type": <value>}}，缺省不写该字段以保持既有请求体形态。
llm.?.max_image_px	int	2048	图像长边上限，超出等比缩小（3.9.3）。v1.11 语义升格（V18/V27③，3.9.5）：升级天花板 + provider 像素制硬限制域——V21 判审升级路径的分辨率上探以本键封顶；像素是运载意图与 provider 硬限制（带宽/载荷；Anthropic 的 8000px 与 >20 图 ^ >2000px 硬拒本身是像素制）的控制面。[C-62] 记载：gpt-5.6 级 openai 后端默认 detail 等效 original（服务端不再隐式钳制图片 token），本键与 default_image_px 因此成为该类后端唯一的客户端本闸。
llm.?.default_image_px	int	0	v1.11 新增（V18，3.9.5）：图片采样默认工作点（长边 px）。0 = 沿用 max_image_px（v1.10 行为为逐字节不变）。> 0 时须 ≤ max_image_px（CONFIG_ERROR，3.1.4）；V21 升级路径可上探至 max_image_px。
llm.?.price_per_mtok_in / _out	float	可选	每百万 token 单价；配置后报告输出成本估算。
embedding.<name>	table	可选	v1.2 新增：每个子表定义一个 embedding profile，<name> 为被 project.toml dedup.semantic_embedding 引用的名字（5.2；3.3.3 第④级）。
embedding.?.provider	str	"openai_compatible"	本版唯一取值：POST {base_url}/embeddings（3.9.3）。

<code>embedding.*.base_url</code>	str	必填	API 根地址。
<code>embedding.*.model</code>	str	必填	embedding 模型名，原样透传。
<code>embedding.*.api_key_env</code>	str	必填*	持有 API Key 的环境变量名；被 <code>dedup.semantic_embedding</code> 引用时须存在且非空（M1 校验，3.1.4）。* v1.6：与 <code>embedding.*.api_key_envs</code> 恰提供其一。
<code>embedding.*.api_key_envs</code>	array	无	v1.6：同 <code>llm.*.api_key_envs</code> ——embedding profile 的密钥池，机制一致（3.9.3 密钥池行）。
<code>embedding.*.max_concurrency</code>	int	8	该 profile 并发上限（信号量，与 <code>llm.*</code> 同机制，3.9.3）。
<code>embedding.*.timeout_s</code>	int	60	单次请求超时。
<code>embedding.*.max_retries</code>	int	5	可重试错误的最大重试次数（重试规则同 3.9.3）。
<code>embedding.*.retry_base_delay_s</code>	float	1.0	全抖动指数退避基数（与 <code>llm.*</code> 同名键同机制，3.9.3）。
<code>embedding.*.dims</code>	int	可选	返回向量维度校验：配置后 <code>embed()</code> 逐条比对返回维度，不匹配抛 <code>ProviderFatalError</code> （3.9.2）。
<code>embedding.*.context_window</code>	int	0	v1.11 新增（V15，同 <code>llm.*.context_window</code> 声明制，3.9.5）： <code>0</code> = 未声明 = 该 embedding profile 预算关闭。> 0 时预算 = <code>context_window - margin</code> （无输出预留）； <code>embed</code> 输入超预算按确定性头部保留截断（3.3.3 第④级语义嵌入）。声明实效窗口指引同 <code>llm</code> 行（V26）。
<code>tool.log_format</code>	str	"text"	"text" "jsonl"：stderr 运行日志行格式（7.3）；"jsonl" 时强制 console plain 档以保证 stderr 逐行可解析（7.7，显式 <code>rich</code> （CLI <code>--console rich</code> 或 <code>console.mode="rich"</code> ）冲突时 M1 WARN）。
<code>console.mode</code>	str	"auto"	v1.10（7.7）："auto" "rich" "plain"——进度显示面三态；被 CLI <code>--console</code> 覆盖。auto 判定链：stderr TTY $\wedge$ <code>log_format="text"</code> $\wedge$ TERM 非 dumb/空 $\wedge$ rich 可导入（M1 以 <code>find_spec</code> 探测），全真取 rich，否则 plain（TERM 定性为终端能力探测，与 <code>isatty</code> 同级，非配置通道；NO_COLOR 不参与判定——rich 原生剥色保布局，U25）。判定产物由 M1 冻结为解析字段 <code>mode_resolved</code> （3.1.4）。plain 档 stderr 与 v1.9 行为等价（ <code>heartbeat_s=0</code> 时——三层回归锚 7.8）。
<code>console.refresh_hz</code>	int	5	v1.10：rich 画布重绘频率（asyncio 节流 tick，7.7），1—10，越界 = <code>CONFIG_ERROR</code> 。
<code>console.heartbeat_s</code>	int	0	v1.10：仅 plain 且非 TTY 生效——每 N 秒一行数据无关汇总心跳（固定键集 <code>heartbeat batch= stage= llm_calls= elapsed=</code> ，7.7）；0 = 关（默认，保回归锚；对齐决策 1.6 U14）；< 0 = <code>CONFIG_ERROR</code> 。
<code>console.estimate</code>	bool	false	v1.10：仅文本模式生效——启动时做估算扫描（ <code>Ingestor.scan(estimate=True)</code> ），全量多读一遍输入）换取批总数分母与 ETA（7.7；对齐决策 1.6 U17）；UI 模式分母天然廉价（live 预扫复用）、恒显示，本键无效。
<code>console.interactive</code>	bool	true	v1.10：rich $\wedge$ stdin TTY $\wedge$ <code>termios</code> 可用时启用键盘开关（封闭键集 <code>? l e + - p q</code> （ <code>h</code> 为 <code>?</code> 同义键），7.7）；false = 纯渲染（stdin 完全不被占用—— <code>buck2 --no-interactive-console</code> 对应物）。


```
# —— config.toml 完整示例 ——
schema_version = 1

[tool]
log_level = "info"
log_format = "text"                                # "jsonl" 供日志采集系统消费 (7.3)

[console]
mode = "auto"                                     # v1.10 进度显示面 (7.7); 整节可缺省
refresh_hz = 5                                   # auto | rich | plain
heartbeat_s = 0                                  # rich 画布重绘频率 (1-10)
estimate = false                                  # plain 非 TTY 心跳; 0 = 关 (默认)
interactive = true                                # 文本模式批总数分母 (多读一遍输入)
                                                # rich 档键盘开关 (? l e + - p q; h=?)

[llm.default]                                     # 多模态主力模型
provider = "openai_compatible"
base_url = "https://llm-gw.example.com/v1"
model = "qwen2.5-vl-72b-instruct"
api_key_env = "LABELKIT_KEY_DEFAULT"
# api_key_envs = ["LABELKIT_KEY_DEFAULT", "LABELKIT_KEY_DEFAULT_2"]  # v1.6 密钥池: 与上行互斥 (3.9.3)
max_concurrency = 8
timeout_s = 120
max_retries = 5
supports_structured_output = true
supports_vision = true
context_window = 131072                          # v1.11 上下文预算 (0/缺省 = 关; 声明部署实效窗口而非厂商表值, 3.9.5)
# default_image_px = 1092                                # v1.11 图片采样工作点 (0/缺省 = 沿用 max_image_px; 须 ≤ max_image_px)
price_per_mtok_in = 0.6
price_per_mtok_out = 1.8

[llm.judge]                                       # 独立评审模型 (避免自增强偏差, 3.7.2)
provider = "anthropic"
base_url = "https://api.anthropic.com"
model = "claude-sonnet-5"
api_key_env = "LABELKIT_KEY_JUDGE"
# thinking = "disabled"                                # v1.16: provider 顶层 thinking 开关; 缺省不发送
max_concurrency = 4
supports_structured_output = true
supports_vision = true

[embedding.default_emb]                          # v1.2: 语义去重句向量 profile (被 dedup.semantic_embedding 引用, 5.2)
provider = "openai_compatible"                   # 本版唯一取值: POST {base_url}/embeddings (3.9.3)
base_url = "https://llm-gw.example.com/v1"
model = "bge-m3"
api_key_env = "LABELKIT_KEY_EMB"
max_concurrency = 8
timeout_s = 60
max_retries = 5
dims = 1024                                       # 可选: 返回向量维度校验
```

5.2 project.toml (工程级单次配置: 运行参数 + Rubric + 输出 Schema)

键	类型	默认	说明
schema_version	int	必填	固定 1。
run.input	str	process 必填*	输入路径 (* 可被 CLI --input 覆盖); run.mode="generate_only" 时必须缺省, 提供 (含 --input) 即报 CONFIG_ERROR (3.1.4)。
run.output	str	必填*	主输出 .jsonl 路径 (* 可被 CLI --output 覆盖)。
run.modality	str	必填	"text" "ui"。
run.mode	str	"process"	"process" (读取 run.input 加工既有数据) "generate_only" (v1.4 纯生成: 无输入从零合成, 3.6.2 / 3.10.3; 组合与互斥约束见 2.3.1 ④、3.1.4)。
run.batch_size	int	256	批大小 = QuRating 比较池大小 (3.4.3)。v1.11 语义句 (V14): 决定内存生命周期、QuRating 对比池基数与 stream 装箱容量; 从不影响单次 prompt 体积——单次调用容量由各算子条数上限与上下文预算 (3.9.5) 共同决定。
run.seed	int	0	PRNG 种子 (配对采样/顺序随机/种子抽样)。

<code>run.fatal_error_threshold</code>	int	20	熔断阈值（3.10.3）。
<code>run.max_park_s</code>	int	3600	v1.6 驻留上限（3.9.3 密钥池行）：单次逻辑 LLM 调用因「所引 profile 全部存活密钥均在冷却」而驻留等待的累计秒数上限；超限按重试耗尽处理（记录 failed、计入熔断窗口，1.6 对齐决策 ③）。0 = 不驻留（全池冷却即按重试耗尽失败）——注意：0 与单密钥 profile 组合意味着 任何 429（含短 Retry-After）都立即按重试耗尽失败 ，仅建议在多密钥池上设 0。运维容忍度参数，不影响产出内容；单密钥配置下亦约束超长 <code>Retry-After</code> 等待（3.9.3 重试行）。
<code>input.text_field</code>	str	"text"	文本模态取文内容的点路径（3.2.5）。
<code>input.on_bad_line / on_missing_pair / on_index_conflict</code>	str	skip / skip / fail	"skip" "fail"（3.2.4–3.2.5）。
<code>input.max_image_mb</code>	int	20	单图大小上限。
<code>input.ui_tree_max_chars</code>	int	30000	提示词中树序列化长度上限。v1.11（V9，3.9.5）：升格为 绝对上限 ——所引 profile 声明 <code>context_window</code> 后，单记录树渲染实参取 <code>min(ui_tree_max_chars, 预算折算份额)</code> 动态收缩（超预算按行丢尾、保留既有 truncated marker）；未声明预算时即固定上限（现行行）。
<code>stream.order_by</code>	str	"input_order"	v1.8 新增（ <code>[stream]</code> 节 = stream 模式输入侧排序与会话化声明，M2 消费，3.2/3.14； v1.13 措辞修订 ：本节在 <code>segment.enabled</code> <code>V generate_stream.enabled</code> 时生效——后者下本节兼作 生成侧铺设契约 （ <code>order_by</code> 声明工件行的时间戳字段名、 <code>gap_s</code> 定会话间隔下界、 <code>session_max_len</code> 定织造上限；此时必须 <code>order_by = "meta:<字段>" ^ key == [] ^ gap_steps == 0</code> ，3.1.4 时间流生成行 ⑤），两开关皆关时本节入 R8 停放清单）。"input_order"（默认：文本 = 文件名字典序→行号，UI = pair_index 升序） "meta:<field>"（ 仅文本模态 ，M1 校验；时间戳解析规格见 6.1——数值秒/毫秒判定、ISO 字符串、时区归一；解析失败与乱序同走 on_disorder）。
<code>stream.on_disorder</code>	str	"skip"	v1.8: "skip"（默认：乱序/时间戳解析失败记录跳过——计 bad_input + IngestReport.disorder 子计数 + <code>ingest.disorder</code> 事件 + WARN 一次） "fail"（InputError，退出码 3）。单调性游标 按分区键各自维护 （S19；键变即断语义保留，输入须按键成组，6.1）。
<code>stream.key</code>	array	[]	v1.8: 分区键列表，键变即断会话（groupby 语义非 keyBy）。元素 = "meta:<field>"（仅文本模态） "source_dir"（= ref.source_file 父目录派生，UI 模态可用——一次采集一目录惯例，S19）；元素合法性 M1 校验（3.1.4）。
<code>stream.gap_s</code>	int	300	v1.8: 相邻记录时间差 > gap_s 秒即断开会话；仅 <code>order_by="meta:*"</code> 时生效——显式设置而非 meta 序 ⇒ M1 warning 一次（非阻断，键不生效；对照 <code>session_max_span_s</code> 行的 CONFIG_ERROR 级）。默认偏大的结构性论证：欠分割可由 LLM 边界精化拯救、过分割不可逆（3.14）。
<code>stream.gap_steps</code>	int	0	v1.8: 相邻记录序号差 > gap_steps 即断开（0 = 不启用）；与 gap_s 可并用，任一触发即断。
<code>stream.session_max_len</code>	int	200	v1.8: 会话硬上限（帧），到限即断。 <code>session_max_len > run.batch_size</code> ⇒ M1 静态 WARN（S21: 单会话超批容量将被 M10 硬切 + <code>session_split</code> 标，3.10.3）。
<code>stream.session_max_span_s</code>	int	0	v1.8: 会话时间跨度硬上限（秒，0 = 不启用）；仅 <code>order_by="meta:*"</code> 时可设（M1 校验，违反报 CONFIG_ERROR）。
<code>segment.enabled</code>	bool	false	v1.8 新增：语义分段算子 / stream 模式总开关（M14，3.14）。默认关——工具行为与 v1.7 逐字节一致（ <code>_meta.stream: null</code> 除外，6.3）。启用要求（3.1.4）： <code>run.mode = "process"</code> <code>^ generate.enabled = false</code> （generate_only 经 2.3.1 ④ 传递闭合） <code>^ annotate.enabled = true</code> 。no-op warning（R8 家族）： <code>[stream] / [segment] / [extract]</code> 任一节在场而 <code>segment.enabled = false</code> 。
<code>segment.strategy</code>	str	"hybrid"	"rules"（候选会话原样成 episode，零 LLM；noise_filter / min_len 不生效） "llm" "hybrid"（默认：滑窗 LLM 边界精化 + 逐帧噪声标记；len(session)==1 走 rules 退化，3.14）。

<code>segment.llm</code>	str	"default"	profile 引用；仅 strategy ∈ {llm, hybrid} 时计入密钥解析 / <code>--probe</code> / 存在性引用集 (S30, 3.1.4) ——rules 策略零调用不强制配键。v1.11 (V1/V3)：不再入 vision 校验集——segment 从「要求视觉」改为「适配视觉」，窗口是否附图由本 profile 的 <code>supports_vision</code> 能力自动决定 (parse product <code>vision_resolved</code> ，见下)；选 profile 即选能力，需纯文本裁决指向纯文本 profile。
<code>segment.window</code>	int	20	滑窗帧数/调用上限；M1 校验 ≥ 2 。v1.11 语义修订 (V9, 3.9.5)：单窗帧数上限——所引 profile 声明 <code>context_window</code> 后按预算贪心装填 (溢出即封窗，实际每窗帧数 $\leq$ window)，未声明时为固定窗大小 (v1.10 行为逐字节一致)。步长 = 重叠 1 帧 (接缝帧整帧判决归后窗)；window $\geq$ 会话长且预算装得下时天然退化为整段单调用 (S32, 3.14.7)。
<code>segment.digest_max_chars</code>	int	400	单帧摘要 (frame_digest, 4.3) 长度上限。
<code>segment.noise_filter</code>	bool	true	逐帧噪声标记 (interruption $\rightarrow$ dropped_noise, reason="noise")；仅 llm/hybrid 生效—— <code>strategy = "rules" $\wedge$ noise_filter = true $\Rightarrow$ no-op</code> warning (3.1.4)。
<code>segment.min_len</code>	int	2	段最短帧数；仅作用于 LLM 边界精化切出的段 (S11) ——规则层孤帧/短会话 (含 <code>strategy="rules"</code>) 原样成 episode、不受本键约束；被丢弃帧 reason = "below_min_len" ($\neq$ "noise")，独立计数 <code>report.stream.below_min_len</code> (6.4)。
<code>segment.context</code>	str	""	可选域上下文，注入判据模板；非边界定义——边界判据内置于模板 (3.14)，零配置可用。
<code>segment.on_error</code>	str	"keep"	单窗结构修复耗尽的处置："keep" (默认：该会话整体成一个 episode 存活 + 留痕三件套 <code>_meta.stream.degraded = {kind:"segmentation_invalid", windows_failed}</code> / error 事件 / <code>segment.failures</code> 计数，不写 <code>item.errors</code> ——S26 归因防污染) "fail" (会话成员全部 failed $\rightarrow$ rejects, kind = segmentation_invalid, 7.6)。
<code>~~ segment.use_vision ~~</code>	—	— (v1.11 移除)	v1.11 移除键 (V1/V2)：显式出现 $\rightarrow$ CONFIG_ERROR： [segment].use_vision: segment.use_vision was removed in v1.11: whether a window carries images is derived automatically from supports_vision of the profile named by segment.llm; point segment.llm at a text-only profile for text-only judgments (V2) (3.1.4) ——不走「未知键忽略」前向兼容警告。
<code>segment.vision_resolved</code>	bool	parse product	v1.11 (V1)：非用户键——M1 于 load() 收尾冻结的解析产物 (<code>mode_resolved</code> 同款, 3.1.4)： <code>vision_resolved = (modality=="ui") $\wedge$ segment.enabled $\wedge$ strategy∈{llm,hybrid} $\wedge$ llm_profiles[segment.llm].supports_vision</code> ；运行期窗口是否附图的唯一判据 (3.14.4 模板)。
<code>stitch.enabled</code>	bool	false	v1.9 新增：线索缝合算子开关 (M16, 3.16；链序 segment 之后、dedup 之前, 3.10.3)。默认关——主输出 / rejects / report.json 与 v1.8 逐字节等价 (例外两处：dry-run stderr 的 <code>stitch_calls=0</code> 行、streamxverify 缺陷词表 <code>wrong_stitch: 0</code> 行——3.16.4 退化锚)。启用要求 <code>segment.enabled = true</code> (M1 约束, 3.1.4——stream 前置约束经此传递闭合)。no-op warning：[stitch] 在场而 <code>segment.enabled = false</code> 入 R8 点名名单； <code>segment.enabled = true $\wedge$ 本键 false</code> 而节内有 payload $\Rightarrow$ 单独 warning (3.1.4 ⑦)。
<code>stitch.llm</code>	str	"default"	判定 profile 引用；仅启用时计入密钥解析 / <code>--probe</code> / 存在性引用集，不入 vision 校验集 (判定证据为纯文本摘要卡，无视觉必需, 3.1.4 / 3.16.3)。
<code>stitch.max_open</code>	int	4	开放线索池容量 (挂起窗口均值 3 + 1 活跃 [81]；移动域佐证 [90])；池满且需开新线索时按逐出优先级封闭一条 (stale-gap 优先 $\rightarrow$ LRU 兜底；封闭 $\neq$ 终结, 3.16.4)。M1 校验 ≥ 1 。
<code>stitch.bias</code>	str	"conservative"	"conservative" (默认：并入需 LLM 判 resume $\wedge$ 机械先验合取命中——App 交集 / 实体重叠 / 返回同一页面析取三腿, 3.16.4) "llm" (纯 LLM 判，审计/消融用)。

stitch.rescue_short	bool	true	below_min_len 短段按连续 run 重组先进候选池救援 (3.16.4 救援行; 命中翻转计 rescued_short、未命中维持 dropped_noise、永不开新线索); false = 短段维持 dropped_noise (v1.8 行为)。
stitch.repass	bool	true	有界二遍复评 (3.16.4 ②: 一遍结束后对单碎片线索逐个复评, 修正顺序贪心漏缝; 预算 ≤ 单碎片线索数); false = 纯一遍贪心。
stitch.stale_gap_steps	int	0	时间衰减阈值 (会话序号差; 0 = 不启用)。双职: ① 先验降格——候选与线索尾跨度超限时先验须两腿命中 (3.16.4 保守偏置行); ② 池满逐出优先腿 (3.16.4 ①)。与 stream.gap_steps 语义区分: 后者是 M2 会话切分规则, 本键是会话内线索挂起跨度。
stitch.digest_max_chars	int	400	摘要卡内嵌入的每个帧摘要截断上限 (沿用 segment 同名键语义, 3.16.3)。
stitch.context	str	""	可选域上下文 (何为「同一任务」的领域提示), 注入判定模板可选行; 非判据定义——保守偏置内置于固定模板 (3.16.4), 零配置可用。
stitch.votes	int	1	判定稳定化采样数: 1 (默认) = 不启用 (单调用); > 1 须为 ≥3 的奇数 (偶数 = CONFIG_ERROR, M1 校验, 3.1.4) ——同判定 n 次采样、对 (verdict, thread_ref) 完整判定严格多数决 (> n/2; 任何分裂回落保守结局, 3.16.4 votes 行)。成本 = 判定调用 xn。
stitch.on_error	str	"keep"	单判定结构修复耗尽的处置: "keep" (默认: episode 候选开新线索存活 + 留痕两件 (事件+计数器); 救援候选维持 dropped_noise + 同款留痕) "fail" (仅施于 episode 候选信封——failed → rejects, kind = stitch_invalid, 7.6; 救援候选不适用 fail 路径, 3.16.6)。
dedup.enabled	bool	true	—
dedup.scope	str	"global"	"global" "batch" (2.6 内存权衡)。
dedup.minhash_threshold	float	0.85	Jaccard 判重阈值 (工业通行 0.8–0.9 [3][6])。
dedup.minhash_num_perm / ngram	int	128 / 5	签名精度 / 字符 shingle 宽度。
dedup.image_phash_max_distance	int	8	64-bit pHash 汉明距离阈值。
dedup.ui_dup_requires	str	"both"	"both" "tree" "image" (3.3.3)。
dedup.bounds_quantize_px	int	4	树去重时坐标量化粒度。
dedup.semantic	bool	false	v1.2 新增: 可选第④级语义去重开关 (3.3.3; SemDeDup [26])。默认关——零 embedding 依赖, 默认行为与 v1.0 一致 (8.3 O1)。
dedup.semantic_embedding	str	必填†	† dedup.semantic = true 时必填: 引用 config.toml [embedding, <name>] profile (5.1); 存在性与密钥配置 (api_key_env / api_key_envs 恰其一且逐项非空, v1.6) 由 M1 校验 (3.1.4)。
dedup.semantic_threshold	float	0.95	余弦相似度判重阈值 (SemDeDup 论文的高相似区间 [26]; 3.3.3 第④级)。
classify.enabled	bool	false	v1.7 新增: 分类算子开关 (3.13)。默认关——工具行为与 v1.6 完全一致 (_meta.classification: null 除外, 6.3)。
classify.llm	str	"default"	profile 引用; UI 模态须 supports_vision (M1 校验); 计入密钥解析 / vision 校验 / —probe 三处 profile 引用集 (3.1.4 分类行)。
classify.assignment	str	"single"	"single" (锁定一条一类) "multi" (允许多类命中并按标签扇出, 3.13.4)。
classify.max_labels	int	类别数	仅 multi 可设; ∈ [2, 类别数]; 缺省由 M1 解析后回填为类别数 (扇出成本上界旋钮)。
classify.instruction	str	""	可选补充说明, 追加进 system 类别表之后 (3.13.3 模板)。
classify.fallback_class	str	必填†	† enabled 时必填且 ∈ classes (3.13.4 失败与兜底行; LLM 亦可主动选择它)。
classify.self_consistency	int	0	0 或 ≥3 的奇数 (M1 校验); sc 投票语义见 3.13.4 (single 多数票 / multi 逐标签投票, 无过半兜底类)。
classify.sc_temperature	float	0.7	sc 各次采样的 temperature, 仅 self_consistency ≥ 3 生效 (与 annotate.sc_temperature 同机制)。
classify.on_error	str	"fallback"	"fallback" (结构修复耗尽归兜底类, 记录存活) "fail" (记录 failed → rejects) (3.13.4)。

<code>[[classify.classes]]</code>	array	必填†	† enabled 时 ≥ 2 项。每项: <code>name</code> (<code>[a-z0-9_]+</code> , 表内唯一)、 <code>description</code> (非空)、 <code>examples</code> (字符串数组, 可选, 仅输入侧, 3.13.3)。
<code>extract.enabled</code>	bool	false	v1.8 新增: 转移/动作抽取算子开关 (M15, 3.15; 链序位于 <code>classify</code> 之后、 <code>quality</code> 之前, 3.10.3)。启用要求 <code>segment.enabled = true ^ run.modality = "ui"</code> (M1 校验, 3.1.4; 文本序列 v1 不适用)。
<code>extract.llm</code>	str	"default"	profile 引用; 恒计入密钥解析 / vision / <code>--probe</code> / 存在性四处引用集且恒入 vision 校验集 (每转移—请求 2 图, S30, 3.15)。
<code>extract.instruction</code>	str	""	可选抽取补充说明, 追加进 system 抽取指令之后 (3.15 模板); <code>[class.<name>.extract]</code> 可按类覆盖 (白名单 仅此键 , 见按类覆盖表)。
<code>extract.include_diff</code>	bool	true	【树变更摘要】注入开关 (S14): true (默认) 时向抽取提示词注入 <code>tree_diff</code> (4.3) 输出的文字化——结构化树 diff 证据 (≠ 像素 diff, 工程实践正面); false 关闭注入, 供 A/B 消融对比抽取质量 (<code>report.stream.extract.by_type</code> 可观测, 6.4)。
<code>extract.on_error</code>	str	"fallback"	单转移结构修复耗尽的处置: "fallback" (默认, S16: 该步记 <code>action_type="other" + Transition.detail = {kind:"extraction_invalid", message}</code> 留痕, 不写 <code>item.errors</code> ; quality 副读数注入时 fallback 步与 LLM 确证的 other 分列——防污染连贯性锚点) "fail" (episode failed → rejects, kind = <code>extraction_invalid</code> , 7.6)。
<code>quality.enabled</code>	bool	true	—
<code>quality.mode</code>	str	"pairwise"	"pairwise" "pointwise" (1.6 对齐决策)。
<code>quality.llm</code>	str	"default"	profile 引用。v1.8 只增注: stream 模式下序列打分为纯文本 ([步骤序列] + 帧摘要, 无图, 3.4.3 序列行) ——UI 模态亦不因 stream 要求本 profile supports_vision (vision 逐阶段表的放宽项, S30, 3.1.4; v1.9 起 <code>stitch.llm</code> 同为纯文本恒不要求, 「唯一放宽」不再成立)。
<code>quality.rounds</code>	int	4	pairwise 轮数 k。
<code>quality.criteria_per_call</code>	str	"all"	"all" "single" (3.4.3)。
<code>quality.threshold</code>	float	无	聚合分过滤线 [0,1]; 缺省 = 不过滤只打分。
<code>quality.selection</code>	str	"threshold"	"threshold" "top_ratio" (3.4.3 选择机制行)。 "threshold" = 现行行为: 聚合分 < <code>quality.threshold</code> ⇒ <code>dropped_lowq</code> , threshold 缺省则只打分不筛; "top_ratio" = 批内按聚合分降序保留 <code>ceil(top_ratio × 批内存活数)</code> 条。selection="top_ratio" 时不得再设 <code>quality.threshold</code> (互斥, M1 报 <code>CONFIG_ERROR</code>)。
<code>quality.top_ratio</code>	float	无	(0,1; <code>selection="top_ratio"</code> 时必须填, 与 <code>threshold</code> 互斥 (M1 校验); selection 为默认 "threshold" 时设置本键无效——M1 打 warning 提示 (v1.5)。保留条数 = <code>ceil(top_ratio × 批内存活数)</code> ; <code>on_unscored="keep"</code> 保留的未打分记录不占名额 (3.4.3)。
<code>quality.judges</code>	array	[]	评审团 profile 引用数组。空 = 单评审 (用 <code>quality.llm</code>); 非空须为奇数个且每项存在于 <code>config.toml [llm.*]</code> (M1 校验), 每次比较各 judge 独立裁决、per-criterion 多数票 (3.4.3 多评审团行, PoLL [32])。成本 <code>x judges </code> 。
<code>quality.both_orders</code>	bool	false	true 时间一对正反两种呈现顺序各裁决一次 (每 judge), 两次一致才记 winner、不一致按 tie (3.4.3 双顺序裁决行 [20])。成本 <code>×2</code> 。
<code>quality.on_unscored</code>	str	"keep"	"keep" "drop" (3.4.3 裁决失败行)。

quality.rubric	str	自动	"default:text" "default:ui" "default:trajectory" (v1.8 增: 轨迹四准则 rubric, 包数据 default_trajectory.toml, 附录 A.3) "inline"。缺省 (空串) 按模态选默认; v1.8 空串解析规则: segment.enabled = true ⇒ 解析为 "default:trajectory" (两模态一致; 用户显式选择器恒优先; 按类视图经 base selector 自动继承, S29) ——v1.13 条件扩为 segment.enabled v generate_stream.enabled (时间流生成形态打的同样是序列/轨迹的分; loader 与 M11 的 _meta.run.rubric 镜像两处同步, 3.11.2)。trajectory rubric 与 extract.enabled = false 组合 ⇒ M1 warning 提示 (rubric 模态中立、不预设 steps 在场——「步骤」退化读作「帧间变化」, S29)。写 inline 时必须提供 [[rubric.criteria]]。
quality.judgment_reasons	str/bool	"auto"	"auto" true false。生效时 pairwise 裁决 Schema 增加 reason 字段 (3.4.3), 写入 trace 供 rubric 优化 (7.5); "auto" = trace.enabled=true 且 trace.channels 含 "quality" 时开 (trace 关闭则不请求 reason, 零额外 token)。成本: 每次裁决约增加 30—60 输出 token。
rubric.criteria	array	可选	内联 rubric, 字段见 5.3。
generate.enabled	bool	false	仅文本模态 (2.3.1 约束)。
generate.llms / instruction	array/str	["default"] / 必填†	† enabled 时必填。llms 为 profile 引用数组 (v1.2, 取代 v1.1 单值键 generate.llm), 每个元素须存在于 config.toml [llm.*]; 每次生成调用按 generate.mixture 从中选 1 个 (3.6.2 多模型混合行)。
generate.mixture	str	"round_robin"	"round_robin" (按调用序轮转) "weighted" (按 generate.weights 加权抽样, PRNG 用 ctx.rng, 随 run.seed 可复现)。llms 仅 1 个元素时二者等价 (即 v1.1 行为)。
generate.weights	array	[]	正数权重; mixture = "weighted" 时必填且长度须等于 llms (M1 校验), round_robin 下忽略。
[[generate.styles]]	array	[]	风格子表 (可选): 每项含 name (str, 表内唯一) 与 prompt (str, 非空); 非空时每次生成调用经 ctx.rng 均匀抽 1 个 style, 其 prompt 追加进生成指令 (3.6.2 风格条件化行)。
generate.num_per_record	int	2	每种子期望产出条数。
generate.seeds_per_call / num_per_call	int	3 / 4	3.6.2. v1.11 (V9, 3.9.5): seeds_per_call 升格为上限——所引 profile 声明 context_window 后按 rng 采样序从尾部丢弃种子直到装下 (确定性收缩, min 1); 未声明预算时即固定条数 (现行行为)。
generate.seed_min_score	float	自动	种子门槛, 默认取 quality.threshold 或批中位数。
generate.temperature	float	0.9	生成需要多样性, 覆盖 profile 默认。
generate.sample_validator	str	无	v1.5 校验回调 (方案 A): "module:function", 签名 fn(text: str) -> list[str] (空 = 通过)。对每条生成样本在相似度过滤之前执行, 违规样本剔除 (过滤语义, 不触发重试、不产生 failed 记录), 计入桶统计 rejected_by_validator (3.6.2/6.4)。M1 校验同 output.validator (无 few-shot 干跑)。回调抛异常 ⇒ 该样本按违规剔除并 stderr warn (过滤器不失败)。
generate.seed_examples	array	[]	generate_only 专用 (process 模式不得设置, 3.1.4): 字符串数组种子池, 非空即种子池形态 (3.6.2)。
generate.standalone_count	int	无	generate_only 无种子形态必填 (与 seed_examples 互斥): 目标产出条数, 调用数 = [standalone_count / num_per_call]。
generate.sequences	int	0	v1.13 新增 (时间流形态): 序列 尝试配额 的全局默认, 按类经 [class.<name>.generate].sequences 覆盖: 0 = 该类不参与生成。M1 要求 $\sum \text{sequences} \geq 1$ (各类有效值求和, 3.1.4 时间流生成行)。语义同 standalone_count —— 尝试配额 , 无输出条数保证、无补齐回路 (8.3 O6 辖区)。

<code>generate.len_range</code>	array	[3, 6]	v1.13 新增（时间流形态）：单序列步数的均匀采样区间 <code>[lo, hi]</code> （整数， $1 \leq lo \leq hi$ ），按类可覆盖；逐序列 <code>L = randint(lo, hi)</code> （计划期第②步抽签，3.6.5）。织造上限： $2 \times \max(\text{各类 } hi) \leq \text{stream.session_max_len}$ （M1 校验）。v1.14 长度可覆盖交叉引用（v1.15 措辞按类化）：档位表在场时，逐（参与类，档）非零配额对另须满足 $lo \geq \text{len}(\text{该档 frame_classes})$ ——其中「档」取自本类生效表（ <code>[[class.<name>.generate.tiers]]</code> 声明了就用它，否则回落全局 <code>[[generate.stream.tiers]]</code> ）；档内每类至少出现一次，步数不足即装不下，零额对豁免（见下方两节档位表与 3.1.4）。
<code>generate.stream.enabled</code>	bool	false	v1.13 新增：时间流生成形态总开关（generate_only 第三形态，M6，3.6.5）。默认关——全关时全系统与 v1.12 字节等价（含七个既有 dry-run golden）。启用要求（M1 硬合取，3.1.4 时间流生成行）： <code>run.mode="generate_only" ^ run.modality="text" ^ generate.enabled ^ classify.enabled ^ stream.order_by="meta:<字段>" ^ output.meta_mode != "none"</code> ；与 <code>frame.classify.enabled / frame.annotate.enabled</code> 互斥（帧类真值在蓝图层已知，定向 CONFIG_ERROR）。
<code>generate.stream.sessions</code>	int	0	v1.13：会话数（ ≥ 1 ）。交叉会话数 = $\Sigma \text{sequences} - \text{sessions}$ ，故 M1 要求 $\text{sessions} \leq \Sigma \text{sequences} \leq 2 \times \text{sessions}$ （交叉并发度恒 $k \in \{1,2\}$ ；更高并发度列 8.4 演进候选）。重发序列另落流尾新会话、不计入本键（无作废时工件实际会话数 = $\text{sessions} + \text{duplicates}$ ；有序列作废或被相似度淘汰时按 $\text{sessions_eff} = \min(\text{sessions}, \Sigma \text{幸存})$ 装箱，实际会话数相应减少——见 <code>report.generate.stream.sessions</code> ，3.6.5）。
<code>generate.stream.noise_ratio</code>	float	0.0	v1.13：噪音帧 / 任务帧 比例， $\in [0,1]$ ；噪音帧数 = <code>round(noise_ratio × Σ任务帧数)</code> ，调用数 = <code>⌈噪音帧数 / generate.num_per_call⌉</code> 。> 0 时 <code>noise_instruction</code> 必填。噪音帧逐帧掷签（会话，槽位），满员会话（ $\text{len} \geq \text{stream.session_max_len}$ ）退出签池（3.6.5）。
<code>generate.stream.noise_instruction</code>	str	""	v1.13：噪音帧的生成指令（ <code>noise_ratio > 0</code> 时必填非空，M1 校验）；批量实现复用 3.6.2 的既有生成模板与输出 Schema。
<code>generate.stream.duplicates</code>	int	0	v1.13：原样重发的序列条数（0 = 无；M1 要求 $\in [0, \Sigma \text{sequences}]$ ，运行期另按幸存数钳制 + WARN）。取自幸存序列、帧内容逐字节同源、恒落流尾新会话（避免同刻不定序）——重发帧只活在工件（不构造信封、不进本次运行的守恒账），判重演示位在工件重放（6.5）。
<code>generate.stream.frame_gap_s</code>	array	[5, 60]	v1.13：会话内帧间隔的均匀采样区间（秒，数值）；M1 默认 v1.15 路径（以及仅有 <code>sequence_validator</code> 、无实际非零 <code>rules/windows</code> 前缀的路径）要求 $1e-6 \leq lo \leq hi < \text{stream.gap_s}$ ；仅当 <code>--limit</code> 后实际非零配额前缀存在生效 <code>rules/windows</code> 时，v1.16 联合规划路径才允许 $hi \leq \text{stream.gap_s}$ （上界等于阈值时由 <code>replay guard</code> 接管；下界为 v1.14 补的微妙地板——isoformat 精度与 <code>round(·, 6)</code> 的分辨率下界，亚微妙 lo 下帧间隔 <code>timedelta</code> 取整为 0 微妙，破坏「ts 严格递增」并使时间语义词表的 0.0 边界哨兵失去无歧义性）。会话间隔取 <code>uniform(gap_s + lo, gap_s + hi)</code> 恒 > <code>gap_s</code> $\Rightarrow$ 摄取侧按同一 <code>gap_s</code> 复演出相同会话切分（3.6.5）。
<code>generate.stream.ts_start</code>	str	"2026-01-01T00:00:00Z"	v1.13：时间流起点（ISO-8601，M1 以 <code>datetime.fromisoformat</code> 校验可解析；无时区视为 UTC，与 <code>meta:<字段></code> 摄取规则一致）。恒不取墙钟——同 <code>seed</code> 双跑工件逐字节一致的前提之一（2.6 可复现行）。
<code>annotate.enabled</code>	bool	true	—
<code>annotate.llm / instruction</code>	str	default / 必填†	† enabled 时必填。
<code>annotate.examples</code>	array	[]	few-shot: <code>[[input, output]]</code> ，output 须过用户 Schema（M1 校验）。
<code>annotate.self_consistency</code>	int	0	0 = 关（单次标注，v1.1 行为）；启用须 ≥ 3 且为奇数（M1 校验）：每条记录独立采样 n 次后字段级投票（3.5.2 note 框）。成本：标注调用与 token $\times n$ 。
<code>annotate.sc_temperature</code>	float	0.7	self-consistency 各次采样的 temperature（采样多样性来源 [33]），覆盖 profile 默认；仅 <code>self_consistency ≥ 3</code> 时生效。

<code>annotate.sequence_frames</code>	int	20	v1.8 新增：序列 (episode) 标注单请求最大关键帧数， $\in [2, 100]$ (越界 <code>CONFIG_ERROR</code> , M1 校验)。成员数 $n > k$ 时确定性均匀降采样 $idx_i = \lfloor i \cdot (n-1)/(k-1) \rfloor$, $i=0..k-1$ (首末帧恒含、严格递增、纯整数零 <code>rng</code> ; $n \leq k$ 取全量, 3.5.2 序列行)。 <code>sequence_frames > 20</code> 且所引 <code>profile max_image_px > 2000</code> $\Rightarrow$ M1 WARN (S28: Anthropic 对 >20 图请求单图 $>2000px$ 为 400 硬拒非缩放, 现默认 <code>max_image_px=2048</code> 恰撞拒——指引改 ≤ 2000 或降帧; 20 图阈值按请求内全部 image block 计)。非 stream 模式显式设置 $\Rightarrow$ no-op warning (3.1.4)。v1.11 (V9, 3.9.5): 升格为上限——所引 <code>profile</code> 声明 <code>context_window</code> 后关键帧数按预算剩余动态收缩 <code>k_eff = min(sequence_frames, max(2, \lfloor 剩余 / 每图成本 \rfloor)) (首末帧恒保留、中间均匀下采样, 既有降采样语义不变); 未声明预算时即固定上限 (现行为)。</code>
<code>frame.classify.enabled</code>	bool	false	v1.12 新增：帧级闭集分类开关 (M13 帧粒度, 3.13.7) ——对流模式序列信封的成员帧做批量闭集判决, 产物落 <code>_meta.stream.members[].label</code> (6.3)。默认关; 帧粒度全关时全系统与 v1.11 字节等价 (唯 dry-run 估算行与 <code>estimate</code> 键表例外, 3.10.3)。启用要求 <code>segment.enabled = true</code> (帧粒度仅流模式, 2.3.1 帧粒度约束; 非流模式请改用 <code>classify + [class, <name>.annotate]</code>)。
<code>frame.classify.llm</code>	str	"default"	<code>profile</code> 引用; <code>enabled</code> 时计入密钥解析 / <code>--probe</code> / 存在性引用集; 永不入 vision 必需集——附图由解析产物 <code>FrameClassifyConfig.vision_resolved</code> ($= ui \wedge enabled \wedge profile.supports_vision$) 自动推导 (3.1.4 帧粒度配置行; 成本控制面 = 指向纯文本 <code>profile</code> , 判决仅凭摘要行)。
<code>frame.classify.fallback_class</code>	str	必填†	† <code>enabled</code> 时必填且 $\in$ 帧类表 <code>name</code> 集 (2.3.1 帧粒度约束): 修复穷尽 / 窗口失败的兜底类 (3.13.7 失败语义; LLM 亦可主动选择它)。
<code>[[frame.classify.classes]]</code>	array	必填†	† <code>enabled</code> 时经「 <code>fallback_class \in</code> 类表」传递性要求非空 (无独立 ≥ 2 类数下限, 与 <code>[[classify.classes]]</code> 有意不同, 3.1.4)。每项: <code>name</code> (<code>[a-z0-9_]+</code> , 表内唯一)、 <code>description</code> (非空)、 <code>examples</code> (字符串数组, 可选——解析合法但帧级批量判决模板不渲染 (§10.12 只渲染类表, 与序列级 <code>few-shot</code> 有意不同), 在场时 M1 显名 WARN <code>class examples are not rendered by the batched frame-verdict template (§10.12)</code> , so this key is ignored, 静态预算预检口径同步不计, 3.1.4)。帧类表与序列类表相互独立、允许重名、互不约束 (计数命名空间同分离: <code>frame_classify.*</code> vs <code>classify.*</code> , 6.4)。
<code>~ frame.classify.assignment ~</code>	—	— (不提供)	v1.12 定向探针键: 显式书写 $\rightarrow$ <code>CONFIG_ERROR</code> ——帧分类恒为单一归属 (帧多标签/帧级扇出为 v1.12 非目标, 8.1); 多标签扇出请用序列级 <code>[classify].assignment</code> (机制 = v1.11 <code>use_vision</code> 原始节探针同款, 3.1.4)。
<code>frame.annotate.enabled</code>	bool	false	v1.12 新增：帧级逐帧标注开关 (M5 帧粒度, 3.5.5) ——序列级标注成功后对成员帧逐帧产出符合帧级 Schema 的标注, 产物落 <code>_meta.stream.members[].annotation/status</code> (6.3)。默认关。启用要求 <code>segment.enabled = true</code> ; <code>frame.*</code> 任一启用 $\Rightarrow$ <code>output.meta_mode != "none"</code> (帧产物仅经 <code>_meta.stream.members</code> 承载, <code>sidecar</code> 合法——2.3.1 帧粒度约束)。
<code>frame.annotate.llm</code>	str	"default"	<code>profile</code> 引用; <code>enabled</code> 时计入密钥解析 / <code>--probe</code> / 存在性引用集; <code>ui</code> 模态 $\wedge$ <code>enabled</code> 时无条件入 vision 必需集 (镜像序列级 <code>annotate</code> ——截图是标注主证据, 3.1.4 帧粒度配置行)。
<code>frame.annotate.instruction</code>	str	必填†	† <code>enabled</code> 时必填 (非空, M1 校验)。全局帧标注指令; <code>[frame.class.<name>.annotate]</code> 可按帧类覆盖 (见按类覆盖表 v1.12 注)。
<code>frame.annotate.examples</code>	array	[]	<code>few-shot</code> : <code>[[input, output]]</code> , <code>output</code> 须过帧级 Schema (M1 干跑校验, 3.1.4; 帧级无 L2.5 hook); 形态镜像 <code>annotate.examples</code> 。
<code>frame.annotate.schema_path</code>	str	二选一	外部 <code>.json</code> 的帧级输出 Schema; 与 <code>schema_inline</code> 恰一 (2.3.1 帧粒度约束; 解析产物 <code>ResolvedConfig.frame_schema</code> , <code>user_schema</code> 同胞——draft 2020-12 元校验, 镜像 <code>output.schema</code> 全套分支)。
<code>frame.annotate.schema_inline</code>	str	二选一	TOML 多行字符串内嵌的帧级 Schema JSON 文本 (同上)。

~~ frame.annotate.self_consistency ~~	—	—（不提供）	v1.12 定向探针键：显式书写 → CONFIG_ERROR——自洽采样成本 $\times n$ 且投票键须取自帧 Schema（v1.12 非目标，8.1）；自洽采样请用序列级 [annotate].self_consistency（机制同上）。
verify.enabled	bool	false	—
verify.llm	str	"judge"†	† verify.enabled = true 且 verify.judges 为空时该 profile 须存在于 config.toml [llm.*]（judges 非空时被评审团替代、不参与运行也不要求存在，v1.5）；建议独立于 annotate.llm（3.7.2）。
verify.judges	array	[]	多评审团 profile 列表（v1.2, 3.7.2；与 quality.judges 语义一致）：空 = 单评审用 verify.llm；非空须为奇数个（M1 校验），verdict 取多数票，critiques 合并并标注来源 judge，成本 $\times \text{judges} $ 。背书 PoLL [32]。
verify.policy / max_repair_rounds	str/int	"drop" / 1	3.7.3。
verify.extra_criteria	str	""	追加评审维度的自由文本。
output.schema_path	str	二选一	外部 .json 的用户 Schema；与 schema_inline 恰一。
output.schema_inline	str	二选一	TOML 多行字符串内嵌的 Schema JSON 文本。
output.max_repair_attempts	int	2	结构引擎 L3 次数（3.8.2）。
output.repair_llm	str	同调用方	L3 修复用 profile。
output.validator	str	无	v1.5 校验回调（方案 A）："module:function" 形式的 Python 可调用引用，签名 fn(obj: dict, record: dict None) -> list[str]（返回违规描述列表，空 = 通过；record = Record.raw：文本/生成记录为该行原始对象，UI 记录为 None）。挂接为结构引擎 L2.5（3.8.2）：仅作用于用户 Schema 的标注调用，违规并入 L3 修复环、共享 max_repair_attempts 预算，耗尽 ⇒ 记录 failed（kind = callback_violation，7.6）。M1 启动校验：格式、可导入、可调用，且逐条 few-shot 示例 output 须过回调（干跑）。回调以运行者同权限执行任意用户代码（信任边界与配置文件一致）；回调内抛异常按记录级 internal_error 处理。
output.meta_mode	str	"inline"	"inline" "sidecar" "none"（6.3）。
output.passthrough_fields	array	[]	从 Record.raw 透传进 _meta.source.fields 的字段名列表。
output.rejects	str	"refs"	"none" "refs" "full"（3.11.2）。
trace.enabled	bool	false	启用 trace 追踪日志（第 7 章）。
trace.path	str	自动	默认 {output_stem}.trace.jsonl，与主输出同目录。
trace.channels	array	["quality","verify","schema"]	可选值 ingest segment（v1.8 增） stitch（v1.9 增） dedup classify（v1.7 增） extract（v1.8 增） quality annotate verify schema llm（十一个，7.2 事件目录；通道 = stage 名，S1）；默认值不变——分类事件须用户显式加 "classify"、分段/摘取/缝合事件须显式加 "segment" / "extract" / "stitch" 才写；run.*/batch.* 生命周期事件不受此过滤。
trace.content	str	"refs"	"none" "refs" "excerpt" "full" 内容脱敏四档（7.4）。

[[generate.stream.tiers]] 帧类构成档位表（v1.14 新增，可选；数组表，仅时间流生成形态合法）。一个档位的定义就是该档序列的帧类构成集合：它不携带质量指令、不控制帧内部语义质量（那归各帧类的生成指令与温度）。缺省（表不在场）⇒ 档位面整体不在场，与 v1.13 字节等价。

字段	类型	默认	说明
tier_rank	int	必填	档位序号（「第几个档位的要求」，即档位身份——本表不设 name 键）。正整数、表内唯一、全表连续覆盖 1..N（N = 表长；缺号/重号 = CONFIG_ERROR）。它同时是配分平票与类内序数分块的确定性排序依据：工具不赋予序数高低任何质量方向语义——方向由用户在各档构成上自行赋予。
weight	int	必填	配额权重（整数 ≥ 1）。每个参与类的 sequences 配额按该类生效表（v1.15：本类的按类表或回落的全全局表）各档权重走整数域最大余额法零抽签配分（3.6.5 档位构成行）；配分为（sequences，权重表）的纯函数、不消费任何随机数。某（参与类，档）配额为 0 是小配额 × 悬殊权重的自然结果 ⇒ M1 发 WARN（非错误，报表 tiers 子块如实呈现 planned 0）。
frame_classes	array	必填	档位构成：该档序列恰用这些帧类（每类至少出现一次、不出现档外类）。非空、档内无重复、每名 ∈ [[frame.classify.classes]] 名集，同一张表内各档构成集合两两互异（同构成即语义重复；v1.15：跨表——即跨类——同构成合法）。恰等语义由蓝图内部 Schema 双向保证——enum 限档内子集给「⊆」、逐类 contains 给「⊇」（3.8.1），故档位身份可从 _meta.stream.members[] 的帧类集合直接反推对账。长度前提：配额非零的（类，档）对须满足该类 len_range 下界 ≥ 本档构成大小（M1 校验）。

档位序数在三处落地（同一个值三个面，档位表缺省时三处全部不在场）：主输出每行的 `_meta.source.generator.tier_rank`（6.3）、时间流工件行的 `truth.tier_rank`（6.5）、报表的 `report.generate.stream.tiers`（6.4——v1.15 起该子块有平面与类嵌套两形，见下节）。另一条推论：`-limit` 在计划期配额层做前缀截断，而类内序数按 `tier_rank` 升序占连续区间，故截断是在每个类内从最高档位序数侧截起。

[[class.<name>.generate.tiers]] 按类档位表（v1.15 新增，可选；数组表，仅时间流生成形态合法）。行结构与上表逐字段相同（`tier_rank` / `weight` / `frame_classes` 三键，逐行校验同款），语义是**表级原子覆盖**——本类声明了就用本类的整张表，**不逐行合并**（行级合并会让 `rank` 身份跨表漂移；先例是仓库既有的两处按类重资产：`[class.*.quality].rubric` 内联子表整表替换、`[class.*.annotate].schema_*` 覆盖回落）。缺省（所有类都不声明）⇒ 生效表恒 = 全局表，与 v1.14 字节等价。四条语义要点：

要点	规格
覆盖与回落	一个序列类的 生效表 = 该类声明的表；未声明（键缺省）⇒ 回落全局 <code>[[generate.stream.tiers]]</code> 。配分、蓝图【帧类表】档内子集、 <code>generator.tier_rank</code> 、工件 <code>truth.tier_rank</code> 与报表逐类分账，全部按 本类生效表 取值（3.6.5）。
全局表为锚	按类表 要求全局表在场 （缺省 = <code>CONFIG_ERROR</code> ，指引补全局表）。档位面总开关恒 = 全局表非空 ⇒ 每个参与类恒有生效表，上文「三处落地」的在场性因此恒定、不因某类未声明按类表而逐行漂移。「本类不想分档」的表达式 = 给它一张单档表（ <code>tier_rank = 1</code> + 任意构成）——退化形态，零机制成本。
rank 为类内身份	<code>tier_rank</code> 在 每张生效表内 各自正整数、表内唯一、连续覆盖 1..N（N = 该表长， 逐类可不同 ）；跨类同 <code>rank</code> 无任何工具语义 （工具本就不赋序数以质量方向），「各档构成两两互异」的辖区相应收窄为 单表之内 ——跨类同构成完全合法（各类都可有自己的「全类档」）。消费侧行级消歧免费：主输出行携 <code>_meta.classification.label</code> 、工件行携 <code>truth.sequence_class</code> ，与 <code>tier_rank</code> 同行相邻。
空表拒收	三态互斥：键缺省 = 未声明（回落全局）； <code>tiers = []</code> = 显式空表 ⇒ <code>CONFIG_ERROR</code> （指引删键回落——档位面统一之下不存在「本类无档」态）；非空 = 覆盖。

```
[[generate.stream.tiers]]                                # 全局表：锚 + 面开关 + 未声明类的回落
tier_rank = 1
weight = 2
frame_classes = ["task_request", "followup"]

[[generate.stream.tiers]]
tier_rank = 2
weight = 1
frame_classes = ["task_request", "followup", "confirmation"]

[[class.ticket_booking.generate.tiers]]                 # 按类表：本类整表取代全局表（构成与权重双差异）
tier_rank = 1
weight = 1
frame_classes = ["task_request", "followup"]

[[class.ticket_booking.generate.tiers]]
tier_rank = 2
weight = 2
frame_classes = ["task_request", "confirmation"]
# smart_home 不声明 ⇒ 回落全局两档表（混合形态：一类独立表 / 一类回落）
```

M1 校验（3.1.4 「按类档位表」行，规范源头 2.3.1 v1.15 段）：**按类表前提三子款**——形态门（`generate_stream.enabled = false` 时书写 = 定向 `CONFIG_ERROR`，原始节探针）、全局锚、空表拒收；上表的**身份连续性与构成合法性改逐生效来源表执行**（全局表 + 每张已声明按类表各跑一遍）；逐（参与类，档）的长度可覆盖约束与配分零额 `WARN` 逐类改吃本类生效表；「帧类未入档」`WARN` 与「每帧类生成指令必填」的检查域**并集化为 U**（各参与类生效表构成）。零配额类（`sequences = 0`）声明的表照跑结构校验，仅豁免上述两项配额相关检查、其构成不入并集。观测面：`report.generate.stream.tiers` 在任一按类表在场时改**类嵌套形**（6.4），`_meta.source.generator.tier_rank` 与工件 `truth.tier_rank` 的键、序、在场判据**全部零改动**（值 = 本行序列类生效表内的档位序数，跨类不可比）。

[[class.<name>.<section>] 按类覆盖（v1.7）。`classify` 启用时可按类覆盖下游算子参数：`<name>` 必须 ∈ `classes`；未出现的键一律继承全局节（不配任何覆盖即纯打标模式）。可覆盖键白名单（M1 强校验，白名单外的键报 `CONFIG_ERROR` ——3.1.4 「未知键报 `warning`」行的显式例外；白名单后续只增）：

节	可覆盖键	不可覆盖（保持全局）及理由
<code>[class.*.quality]</code>	<code>mode</code> , <code>rounds</code> , <code>rubric</code> （含 <code>[class.*.rubric]</code> 内联子表，结构同 5.3）， <code>threshold</code> , <code>selection</code> , <code>top_ratio</code>	<code>llm</code> / <code>judges</code> / <code>both_orders</code> / <code>criteria_per_call</code> / <code>on_unscored</code> ——LLM 绑定属部署与成本面，类差异先用 <code>rubric</code> 表达（1.6 v1.7 对齐决策④）

[class.?.annotate]	instruction, examples, schema_path / schema_inline (v1.13 增, 至多其一)	llm / self_consistency / sc_temperature. v1.13 按类标注 Schema 语义: 覆盖 (类声明了就用类的, 两键皆缺 = 回落全局 output.schema ; 同时声明报 CONFIG_ERROR) ——装载走 output.schema 全套分支 + _meta 保留键禁令 + \$ref 遍历 + 按类 few-shot 干跑 (3.1.4 按类覆盖合并行 ⑤); 运行期由 M5 的单点取值函数供给全部消费点、M11 按行终检同口径 (3.5.2、3.11.2)
[class.?.generate]	instruction, styles, num_per_record, temperature, sequences / len_range (v1.13 增, 时间流形态的按类配额与步数区间), tiers (v1.15 增第七键: 按类帧类构成档位表, 数组表 [[class.<name>.generate.tiers]] , 表级原子覆盖语义——见下方按类档位表节)	llms / mixture / weights / seeds_per_call / num_per_call / sample_validator. v1.13 时间流形态下 num_per_record / seeds_per_call 从本行白名单语义中除名——显式书写是定向 CONFIG_ERROR (属平面生成形态, 3.1.4 时间流生成行 ③)。v1.15 注: tiers 同为仅时间流生成形态合法—— generate_stream.enabled = false 时书写是定向 CONFIG_ERROR (原始节探针, 3.1.4 按类档位表行), 另要求全局 [[generate.stream.tiers]] 在场 (全局表为锚)
[class.?.verify]	extra_criteria	llm / judges / policy / max_repair_rounds
[class.?.extract]	instruction (v1.8 增)	llm / include_diff / on_error——LLM 绑定与失败策略属部署与成本面 (与 quality 行同理)
[frame.class.?.annotate] (v1.12)	instruction, examples, enabled (enabled = false ⇒ 该帧类成员跳过帧标注——省成本面, members[] 呈现 status="skipped", 3.11.2)	llm / schema——LLM 绑定属部署与成本面; 帧级标注 Schema 按粒度唯一 (8.4 M13 行)。v1.12 时白名单仅此一节, v1.13 增 generate 节 (下行); 两节之外的节名 ⇒ CONFIG_ERROR (3.1.4 帧粒度配置行)
[frame.class.?.generate] (v1.13; v1.14 增第四键)	instruction (必填非空——v1.13 为「每个帧类», v1.14 起: 档位表在场时检查域收窄为 U 各档 frame_classes , 未入档帧类豁免必填并另发一条 WARN 提示其整个生成面为死配置), schema_path / schema_inline (至多其一: 声明 = 结构化帧 (帧内容以对象原样落工件行的文本字段, 成员 Record.text 取其 canonical JSON 投影, 6.5); 均缺 = 纯文本帧), time_fields (v1.14 增, 子表——见下方时间字段绑定表)	llm / temperature / styles——生成侧 profile 与采样参数属序列级 (蓝图与实现绑定同一 profile, 3.6.5)。本节仅时间流生成形态合法: generate_stream.enabled = false 时出现是反向定向 CONFIG_ERROR (指引改写 [frame.class.<name>.annotate], 3.1.4); 帧类生成 Schema 走 _load_schema_pair 全套 + \$ref 遍历, 无 _meta 分支 (帧内容落工件行文本字段, 与 §6.3 信封字段无冲突面)
——	——	run.* / input.* / stream.* (v1.8) / dedup.* / segment.* (v1.8) / stitch.* (v1.9) / classify.* / trace.* 全部不可按类; output.* 中除标注 Schema 外全部不可按类——v1.13 修订: output.schema 自本版起可按序列类覆盖 ([class.<name>.annotate].schema_*, 上表 annotate 行; 兑现 8.4 M13 行「按类输出 Schema」演进候选), output.validator (L2.5 回调) 与 meta_mode / rejects / passthrough_fields 等其余输出面仍全局唯一, 帧级标注 Schema (frame.annotate.schema) 亦维持按粒度唯一

v1.8 注: segment.* 不入白名单是链序因果而非取舍——链序为 segment → stitch → dedup → classify → extract →... (3.10.3), segment 在 classify 之前执行, 成段时类标签尚不存在,「按类分段」无从谈起; extract 在 classify 之后, 故其 instruction 可按类覆盖 (multi 扇出下兄弟信封各按其标签的有效 instruction 摘取, S9, 3.15)。v1.9 注: stitch.* 不入白名单同为链序因果——stitch 亦在 classify 之前 (3.10.3), [class.<name>.stitch] 不存在 (3.1.4 线索缝合行)。v1.12 注: [frame.class.<name>.annotate] 按帧类覆盖 (键控 [[frame.classify.classes]] 类表, 要求 frame.classify.enabled = true , 3.1.4 帧粒度配置行②), 与 [class.<name>.*] 的序列类覆盖是两个独立命名空间——帧类表与序列类表相互独立、允许重名、互不约束 (重名类各自作用于各自粒度, 互不干扰); 合并产物为 frame_class_views (零覆盖类也各得一份视图, class_views 同款, 运行期零回退)。

[frame.class.<name>.generate.time_fields] 时间字段绑定子表 (v1.14 新增, 可选; 仅结构化帧合法) ——把该帧类生成 Schema 里的时间语义字段绑定到时间轴: 键 = 生成 Schema 顶层字段名, 值 = 语义词表取值。绑定即剔除——被绑定字段从 LLM 面向的逐位 Schema 与逐位契约行中一并删掉 (LLM 物理上生不出它), 值由机械回填尾声在时间戳铺好之后按本序列相邻成员的 ts 差算出写回 (3.6.5 时间字段回填行)。语义词表是冻结闭集, 扩词走 spec 修订:

语义词	要求的字段类型	取值
ts	"string"	本帧已铺时间戳的 ISO-8601 串。任务帧上 = 该行的时间戳字段值; 重发帧承源值 (≠ 自身行 ts——原样重发本就携带陈旧内容)。
gap_prev_s	"number"	与本序列上一帧的间隔秒; 本序列首帧恒 0.0 。

gap_next_s	"number"	与本序列下一帧的间隔秒；本序列末帧恒 0.0。
elapsed_s	"number"	距本序列首帧的秒数；首帧恒 0.0。

口径与约束：间隔一律按**序内相邻成员**计——交叉会话夹入的外序列帧与噪音帧本就占用其间墙钟，序内差值才与下游从数据实测的口径一致；不提供「会话内相邻行」口径（那是重放侧可自行计算的流水量）。数值取 `round(·, 6)`（微秒精度，与 `isoformat` 写出的分辨率对齐；0.0 边界的无歧义性由 `frame_gap_s` 的微秒地板保证）。M1 校验（3.1.4 帧类构成档位与时间字段绑定行）：绑定表仅结构化帧合法（纯文本帧带绑定表 = 定向 `CONFIG_ERROR`）；每个绑定键 $\in$ 生成 Schema 顶层 `properties`；绑定值 $\in$ 上表四词；该属性 Schema 的 `type` 关键字**字面恰等于**上表要求（联合类型数组、缺失、经 `$ref` / 组合关键字间接声明均判不匹配）；顶层 `properties` 键数 $-$ 绑定键数 ≥ 1 （全绑定 = `CONFIG_ERROR`）。绑定字段上再写 `minimum` / `maximum` / `pattern` 等约束关键字 $\Rightarrow$ WARN——那些关键字既不上行也不被强制，**时间量的值域由时间轴决定，不受 Schema 数值约束辖制**。

合并优先级：`[<class.<name>].<sect>.<key> > project.toml [<sect>].<key> > 内置默认`——这是 `project.toml` **内部**的条件化合并，不改变「CLI `> project.toml > config.toml`」三源优先级（2.5）。M1 启动时按逐键 `provenance` 静态合并、冻结为 `class_views`，运行期零查找成本；选择组互斥对剔除、`per-class rubric` 重解析、类 `examples` 干跑等精确语义见 3.1.4 按类覆盖合并行。

```
# —— project.toml 完整示例 (UI 模态标注工程) ——
schema_version = 1

[run]
input = "./capture/2026-07-01"
output = "./out/ui-labels-0701.jsonl"
modality = "ui"
batch_size = 128
seed = 42

[dedup]
ui_dup_requires = "both"

[quality]
mode = "pairwise"
rounds = 4
threshold = 0.3
rubric = "default:ui"

[annotate]
llm = "default"
instruction = ""
你是移动端 UI 理解标注员。根据屏幕截图与 UI 控件树，
标注该屏幕的功能类别、页面标题、可交互元素列表与一句话页面描述。
""

[verify]
enabled = true
llm = "judge"
policy = "repair"
max_repair_rounds = 1

[trace]                                # 追踪日志 (第 7 章): 调优期开启, 用于 rubric 诊断 (7.5)
enabled = true
channels = ["quality", "verify"]

[output]
meta_mode = "inline"
schema_inline = ""
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "screen_category": {"type": "string",
      "enum": ["login", "home", "list", "detail", "form", "settings", "dialog", "other"]},
    "page_title": {"type": "string"},
    "interactive_elements": {"type": "array", "items": {
      "type": "object",
      "properties": {"role": {"type": "string"}, "label": {"type": "string"},
        "bounds": {"type": "array", "items": {"type": "integer"},
          "minItems": 4, "maxItems": 4}},
      "required": ["role", "label", "bounds"], "additionalProperties": false}},
    "description": {"type": "string", "maxLength": 200}
  },
  "required": ["screen_category", "page_title", "interactive_elements", "description"],
  "additionalProperties": false
}
```

5.2.1 v1.16 序列规则、日历窗口与序列级校验钩子

v1.16 的约束表属于时间流 `generate_only` 形态。规则和窗口缺省时均为空，不改变 v1.15 的默认生成路径；任一实际非零配额类的生效表非空时，M1、`estimate_run` 与 M6 共同启用全流 CP-SAT planner。`--limit` 先在按类配额前缀上截断，完全截掉的类不激活 planner 或对应 report 面，但零配额类的声明仍接受完整静态校验。

键	类型	默认	说明
<code>generate.sequence_validator</code>	str	缺省	可选 <code>module:function</code> 。函数签名为 <code>def validate_sequence(value: SequenceValidationInput) -> list[str]</code> ；返回空列表表示通过，非空列表表示整条序列作废。钩子收到深拷贝，不得依赖工具隔离其同权限执行。

<code>generate.stream.rules</code>	array table	<code>[]</code>	全局有限迹规则表；每行 <code>template</code> 必填，按模板使用 <code>frame_class</code> 或 <code>source / target</code> ，可选 <code>count</code> 、半开秒区间 <code>time_s = [lo, hi]</code> 与 <code>typed correlation</code> 。
<code>generate.stream.windows</code>	array table	<code>[]</code>	全局帧类日历窗口表；每个 <code>frame_class</code> 最多一行， <code>of_day</code> 为同日半开区间数组， <code>of_week</code> 缺省为整周。
<code>class.<name>.generate.rules</code>	array table	<code>None</code>	类规则表的三态覆盖：键缺省继承全局，显式 <code>rules = []</code> 清空，非空表整体替换；允许只有类表而没有全局表。
<code>class.<name>.generate.windows</code>	array table	<code>None</code>	与类规则表独立的同样三态覆盖；不与 <code>rules</code> 共享清空或继承状态。

规则模板闭集为 `existence`、`absence`、`exactly`、`init`、`end`、`responded_existence`、`co_existence`、`response`、`precedence`、`succession`、`alternate_response`、`chain_response`、`chain_precedence`、`not_co_existence`、`not_succession`。存在性三模板要求正整数 `count`，其他模板禁止 `count`；二元模板要求不同的 `source / target`，且只有二元模板可写 `time_s` 与 `correlation`。同一生效表中 完全重复的规则声明是 `CONFIG_ERROR`。规则的 `occurrence` 语义、activation/target 候选 枚举和正负规则的时间方向见 3.1.4 与 `docs/dev/SPEC-sequence-rules.md`，`planner witness` 不是运行期的 `i` 对 `i` 配对承诺。

```
[[generate.stream.rules]]
template = "init"
frame_class = "task_request"

[[generate.stream.rules]]
template = "chain_response"
source = "task_request"
target = "confirmation"
time_s = [1.2, 4.8]
correlation = { operator = "equal", source_field = "subject_id", target_field = "subject_id" }

[[generate.stream.windows]]
frame_class = "task_request"
of_day = [["08:00", "11:00"], ["14:00:00", "17:00:00.000001"]]
of_week = ["mon", "tue", "wed", "thu", "fri"]

[generate]
sequence_validator = "hooks:validate_sequence"

[[class.ticket_booking.generate.rules]]
template = "exactly"
frame_class = "confirmation"
count = 1

[[class.smart_home.generate.windows]]
frame_class = "task_request"
of_day = [["09:00", "18:00"]]
```

`time_s = [lo, hi]` 的实际语义是半开 `[lo, hi)` 秒，端点必须可无损量化为微秒且满足 `1us <= lo < hi`；有序模板使用 `target` 减 `source`，`responded_existence`、`co_existence` 与 `not_co_existence` 使用绝对时间差。联合路径把 `frame_gap_s` 量化为 `ceil(lo * 1e6)` 到 `floor(hi * 1e6)` 的整数区间，空区间是配置错误；显式链区间必须与 相邻 `replay guard [1us, stream.gap_s]` 相交。

`correlation` 只接受 `operator = "equal"`。字段必须位于两侧结构化生成 Schema 的顶层 `properties` 与 `required`，两侧 `type` 关键字字面相等，且不能是 `time_fields` 绑定 字段。运行期先按 JSON 类型敏感的 canonical bytes 比较，再按 `time_s` 过滤；对象键序 不影响相等，数组顺序影响相等，`true`、`1`、`1.0` 不视为同值。纯文本帧不能参与 `correlation`。

窗口的 `of_day` 只接受 `HH:MM`、`HH:MM:SS`、微秒精度的同日半开区间，不能跨午夜且 同一行内不得重叠；`of_week` 只能使用 `mon` 到 `sun`，不能重复，省略即整周。所有 窗口以 `ts_start` 的固定 offset 解释，naive `ts_start` 按 UTC，不使用 IANA 时区或 DST。

M1 对每个类、生效 `tier`、`len_range` 的每个候选长度执行局部潜在可满足性检查，并对 实际配额前缀执行完整问题检查。`planner` 使用 OR-Tools 9.15.6755，CP-SAT 单线程、固定 31-bit seed、`max_deterministic_time = 10.0`，模型 proto 超过 250,000 项直接报错；无 `noise objective` 时接受 `FEASIBLE / OPTIMAL`，有 `noise objective` 时必须 `OPTIMAL`。`INFEASIBLE` 或 `UNKNOWN` 是 `CONFIG_ERROR`，`MODEL_INVALID` 是 `InternalError`，不做 运行期重抽、fallback 或重规划。

`generate.sequence_validator` 的 `hook` 异常按违规处理；日志只允许引用、异常类型和违规 数量。M6 的验证顺序为 `realize Schema` → `generate.sample_validator` → `declarative correlation/time` → `sequence hook` → `sequence similarity`。报告只在实际非零配额类的生效面 加入 `rules / validator` 计数与 `windows.calendar_days_spanned`，规则、窗口、hook 返回 文本和 `planner` 细节永不写入 `artifact`、`truth`、主输出或 `report`。

5.3 Rubric 结构（内联或默认包文件，同一 TOML 结构）

键	类型	默认	说明
rubric.name	str	必填	rubric 标识，入 _meta 与报告。
rubric.criteria[].key	str	必填	[a-z0-9_]+，全局唯一。
rubric.criteria[].weight	float	1.0	聚合权重 (>0)。
rubric.criteria[].description	str	必填	准则含义（进入两种模式的提示词）。
rubric.criteria[].pairwise_prompt	str	必填	成对比较问句，如「哪段文本的写作水平更高？」。
rubric.criteria[].pointwise_levels	array[6]	pointwise 必填	0—5 六级加性描述（附录 A 示例）。

6. 输入 / 输出格式规格

6.1 输入：文本模态

UTF-8 编码 JSONL；每行一个 JSON object；行分隔符 `\n`；空行跳过（不计坏行）。示例（`input.text_field = "instruction"`）：

```
{"instruction": "帮我把这段话翻译成英文.....", "source": "app-feedback", "ts": "2026-06-30T10:12:00Z"}
{"instruction": "写一份周报模板", "source": "ime-log", "ts": "2026-06-30T10:15:21Z"}
```

时间戳字段语义（v1.8 只增：**stream 模式** `stream.order_by = "meta:<field>"` 的解析规格，仅文本模态，S20）—— `<field>` 为原始行对象上的点路径字段（上例 `ts`），M2 按以下规则解析（3.2）：

- 数值：`v < 0` 或 `v ≥ 1e14` ⇒ 解析失败；`v < 1e11` 判 epoch 秒；`1e11 ≤ v < 1e14` 判 epoch 毫秒（÷1000）。
- 字符串：先试纯数字 → 按上述数值规则；再试 `datetime.fromisoformat`（Python 3.11 起原生接受 `Z` 后缀）；均败 = 解析失败。
- 时区：aware 值换算为 UTC epoch；naive 值按 UTC 解释。内部序键 = float 秒。
- 解析失败与乱序同走 `stream.on_disorder`（"skip" 默认：跳过并计 `bad_input + IngestReport.disorder`；"fail"：`InputError`，退出码 3；5.2）。
- 流式单调性校验不做全量重排：单调性游标按 `stream.key` 分区键各自维护（S19，内存 = 键基数）——逐设备/逐来源拼接的输入不会被整体判乱序；键变即断会话，输入须按分区键成组（交错流为演进候选，8.4）。
- 与时间流工件的对应（v1.13）：时间流生成形态产出的工件（6.5）就是本节格式的一份实例——工件行的时间戳字段名即该工程 `stream.order_by = "meta:<field>"` 声明的 `<field>`、文本字段名即 `input.text_field`，写出值为微秒精度的 ISO-8601 字符串（本节字符串分支）；工件另带一个 `truth` 对象，对摄取侧而言只是行上的普通字段（参与 id 计算、不参与任何判定，可经 `output.passthrough_fields` 透传做对照）。配同一份 `[stream]` 声明重放时，M2 切出的会话与生成侧逐一对应（3.2.8 可重放记注）。

6.2 输入：UI 模态

目录递归扫描与配对规则见 3.2.4。`uitree<index>.jsonl` 节点行的字段映射（平铺风格；嵌套风格为同字段 + `children` 数组）：

Record 字段	接受的源字段名（按序取首个存在者）	缺省行为
node_id	id, node_id	行号字符串
parent_id	parent, parent_id	null（根）
role	class, className, type, role	"unknown"
text	text, label	""
content_desc	content_desc, contentDescription, desc	""
bounds	bounds（[l,t,r,b] 数组或 "[l,t][r,b]" 字符串两种形式）	[0,0,0,0]
visible	visible, visible_to_user	true
extra	其余全部字段（值转字符串）	—

该映射覆盖 Android uiautomator dump、accessibility 服务导出与主流 GUI 数据集（AndroidControl/AMEX 风格）的常见字段名；不匹配的导出格式可在采集侧做一次字段重命名。

6.3 主输出 JSONL

每行一个 JSON object。`meta_mode` 三种形态：

meta_mode	行结构
inline（默认）	用户 Schema 的全部字段平铺在顶层 + 保留键 <code>_meta</code> 。校验语义：剥除 <code>_meta</code> 后的对象必须通过该行的类有效 Schema（v1.13 修订原「用户 Schema」措辞——= <code>[class.<name>.annotate].schema_*</code> 声明的按序列类覆盖，未声明 / 未分类 / 未知类则为全局 <code>output.schema</code> ；M1 已禁止两者顶层声明 <code>_meta</code> ，3.1.4）。M11 写出前的 <code>validate_only</code> 终检按同一口径逐行取 Schema（3.11.2）。
sidecar	主输出行 = 纯用户结构； <code>_meta</code> 逐行写入 <code>{output_stem}.meta.jsonl</code> ，以 <code>_meta.id</code> 与行序对齐。
none	只写用户结构，不产出元信息（丢弃分数与溯源，不推荐）。

`_meta` 的完整结构：

```

{
  "screen_category": "login",                                // ← 用户 Schema 字段 (示例)
  "page_title": "登录",
  "interactive_elements": [ ... ],
  "description": "手机号+验证码登录页",
  "_meta": {
    "id": "9f2c31ab52e08d17",
    "run": { "tool": "labelkit/1.0.0", "started_at": "2026-07-02T09:30:00+08:00",
      "project_file": "project.toml", "rubric": "default:ui", "seed": 42},
    "source": { "file": "capture/2026-07-01/b/uitree_2.jsonl", "pair_index": 2,
      "generated_from": [], "fields": {},                                // passthrough_fields 落点
      "generator": null}, // v1.2 只增: 生成记录为 {"llm", "style"} (3.6.2), 否则 null;
    // v1.14 键集条件形: 时间流生成的档位表在场时增第三键
    // tier_rank (该序列所属档位序数), 档位表缺省时维持两键
    // v1.15 零改动: 在场判据仍是**全局**档位表非空 (全局表为锚), 键与序
    // 不动; 仅值来源条件化—= 本行序列类**生效表**内的档序数
    // (生效表 = 该类的 [[class.<name>.generate.tiers]] 或回落全局表),
    // 跨类同序数不可比, 类名见同行 _meta.classification.label
    "stream": null, // v1.8 恒在键 (位置: source 之后、scores 之前—链序镜像);
    // = null 当 segment 与 generate_stream 均未启用 (v1.13 门扩:
    // segment.enabled v generate_stream.enabled, 3.11.2)。启用时 (3.14/3.10.3):
    // {"episode_id", "session_id", "order_span": [first, last], "member_count",
    // "member_ids": [...], "member_sources": [{file, pair_index|line_no}, ...],
    // "members": [{index, id[, label][, annotation, status]}, ...],
    // // v1.12 (任一帧开关启用时在场; 冻结位 = member_sources 后、
    // // session_split 前): 逐成员按序一条目, 字段序冻结;
    // // label 仅 frame.classify 启用时在场 (降格跳过 = null);
    // // annotation/status 仅 frame.annotate 启用时在场, status
    // // 闭集 "annotated"|"skipped"|"failed" (三值判定与写前
    // // 校验兜底见 3.11.2; 真实示例见 3.11.3); 帧粒度全关时
    // // 本键不在场—本块与 v1.11 逐字节等价 (退化锚);
    // // 手术后原/克隆行 members 分叉由 repaired 位消歧 (4.3)
    // // v1.13 时间流生成形态: 本块与 label 列的在场门同步扩
    // // `v generate_stream.enabled`一条目恒 {index, id, label}
    // // (label = 蓝图定下的帧类真值, source="inherited"; 本形态
    // // 与 frame.annotate 互斥 → 无 annotation/status 两列,
    // // v1.12 条件列规则相容, 3.6.5/3.11.2)
    // "session_split": false, // 所属会话曾被 batch_size 硬切 (S21, M7 缺帧判定降级依据)
    // "repaired": false, // verify 缺陷修复改写过成员集 (3.7 stream 分支;
    // multi 扇出下消歧同 id 兄弟行的成员分叉, 3.13)
    // "degraded": null | {kind, windows_failed}, // segment.on_error="keep" 留痕 (S26; segme

nt 专属—

    // // stitch keep 路径留痕为事件+计数器两件, 无 _meta 腿, 3.16.6)
    // "steps": null | [{index, action_type, target, value, description}, ...]
    // // extract 关闭时恒 null; 启用 = transitions 逐步摘要 (3.15);
    // // v1.9 (仅 stitch 启用): 步行内另含 "resumed": true—仅接缝
    // // 占位步携带 (emitter 由 Transition.detail.kind=="thread_sea

m"

    // // 推导, 3.15.4; 非接缝步不携带该键)
    // v1.9 增两键 (仅 stitch.enabled=true 时在场—off 时本块与 v1.8
    // 逐字节等价, 3.16.4 退化锚):
    // "thread_id": "9c31f5a2d84e07b6", // = 幸存信封 record.id = episode_id (T22, 3.16.4)
    // "fragments": [{ "order_span": [first, last], "member_count", "cause",
    // "source_episode"}, ...]
    // // 每碎片一项、按会话序; cause ∈ "origin"|"resumed"|"rescued";
    // // source_episode = 碎片缝合前的 episode_id (救援碎片 = null)。
    // // 包络规范句: 多碎片线索的顶层 order_span 为包络 (区间内含
    // // 异线索帧) —下游切片必须用 fragments[].order_span,
    // // 不得按顶层跨度切片 (3.16.4)
    "scores": { "screenshot_readability": 0.81, "tree_screen_consistency": 0.66,
      "state_completeness": 0.74, "interaction_richness": 0.52,
      "__aggregate__": 0.68, "mode": "pairwise_bt", "batch_no": 3},
      // scores v1.7 只增: classify 启用时另含 "pool" (= 类名, 比较池自述, 3.4.3 按类分池行)
    "dedup": { "kind": "unique"},
    "classification": null, // v1.7 恒在键: classify 启用时为 {"label", "labels", "source"} (labels = 命中全集, single 恒
单元素; 3.13);
    // 未启用 = null。multi 模式下行唯一键 = (_meta.id, classification.label)—同 id 可有多行 (3.1
3.4 扇出行)
    "annotation": { "model": "qwen2.5-vl-72b-instruct", "attempts": 1}, // v1.2 只增: self-consistency 启用时另含 "sc": {"n", "agr
reement_ratio"} (3.5.2)
    "verification": { "verdict": "pass", "rounds": 1} // verify 未启用则为 null
    // verification v1.8 只增: stream 模式下另含 "defects" (该键恒在,
    // 无缺陷 = []; 缺陷项 {kind, members, position, detail}, S7, 3.7 stream 分支);
    // 非 stream 行不携带该键

```

```
}  
}
```

时间流生成形态的 `_meta.stream` (v1.13)：直装序列行原样复用上述 `stream` 块，取值面如下——`episode_id = session_id` 之外的序列 `record.id`、`order_span = ["<工件路径>:<首行号>", "<工件路径>:<末行号>"]`、`member_sources` 每项 = `{file: <工件路径>, line_no: <工件行号>}` (工件路径即 `run.output` 同款相对写法，行号 1 基、与工件行序一一对应 ⇒ 主输出与工件双向可对账)、`members[]` 见上、`session_split = false`、`repaired = false`、`degraded = null`、`steps = null` (`extract` 不参与)、无 `thread_id` / `fragments` (`stitch` 不参与)。`_meta.run.rubric` 在本形态下的空选择器解析为 `"default:trajectory"` (3.11.2 rubric 镜像)。

6.4 report.json 结构

```

{
  "run": {"tool_version": "1.0.0", "started_at": "...", "finished_at": "...",
    "interrupted": false, "circuit_broken": false, "exit_code": 0, // circuit_broken: v1.5 只增 "modality": "ui", "see
d": 42,
    "config_digest": "sha256:...", "project_digest": "sha256:..."}, // 配置指纹 (脱敏后)
  // run 节 v1.6 只增: "partial_delivery": true — 仅熔断交付 (3.10.3) 时出现, 恒伴随 circuit_broken=true
  // run 节 v1.13 只增: "artifact": {"path", "sha256", "lines"} — 时间流工件条目 (主输出摘要同款形态:
  // sha256 按落盘字节计, 带 "sha256:" 前缀); **仅工件通道实际写入时在场** (形态关闭与
  // dry-run 恒缺席, 3.11.2 工件通道行)
  "counts": {"scanned": 5000, "ingested": 4987, "bad_input": 13,
    "dropped_dup": 412, "dropped_lowq": 305, "dropped_verify": 41,
    "failed": 9, "generated": 0, "emitted": 4220},
  // counts v1.6 只增: 熔断中止时增列 "unprocessed" (已入流水线但因中止未走完的记录数, 见本节尾注不变量扩展)
  // counts v1.7 只增: classify.assignment="multi" 时增列 "fanout" (扇出净增信封数, M10 计量, 3.10.3; 见尾注不变量扩展)
  // counts v1.8 只增: segment 启用时增列 "episodes" (segment 阶段 len 差, M10 计量, fanout 同构) /
  // "absorbed" / "dropped_noise" (post-emit tally, 3.10.3); 且 stream 模式下 "unprocessed"
  // 的出现条件扩为「熔断 v interrupted」(S18; 见尾注不变量扩展)
  // counts v1.9 只增: stitch 启用时增列 "stitched" (壳终态 tally—仅计被并 episode 信封壳) /
  // "threads" (= episodes - stitched, M10 post-emit tally 导出式单点上报, 3.10.3);
  // 两键仅启用时在场 (off 时 counts 与 v1.8 逐字节等价, 3.16.4 退化锚)
  // v1.8 可选节 (segment 启用时出现, 位于 counts 之后):
  // "stream": {"sessions", "episodes", "mean_episode_len", "absorbed", "dropped_noise",
  // "below_min_len", "digest_poor_frames", "segment_failures",
  // [预算启用, v1.11] "windows", // segment 实际窗数 (M14 属主, V13④) — 供对账 dry-run/估算的
  // // w_min 上界 (V12; 预算未声明时不在场)
  // [stitch 启用, v1.9] "stitch": {"stitched", "rescued_short", "seams", "judgments",
  // "repass_judgments", "failures"},
  // [frame.classify 启用, v1.12] "frame_classify": {"calls", "fallback", "window_failures",
  // "skipped_degraded"}, // 位置冻结: stitch 后、extract 前
  // [extract 启用] "extract": {"transitions", "fallback_steps", "failures", "by_type": {<action_type>: n, ...}},
  // [frame.annotate 启用, v1.12] "frame_annotate": {"annotated", "skipped", "failed",
  // "discarded"}, // 位置冻结: extract 后、verify 前
  // [verify 启用] "verify": {"membership_repairs", "boundary_flags", "defects": {<kind>: n, ...}}
  // — sessions 数据源 = IngestReport (M2 属主, 3.2); below_min_len 独立于 noise 计数 (S11,
  // 发生计数、v1.9 救援不回退—救援量另计 rescued_short, 3.14.4); digest_poor_frames =
  // 摘要贫瘠帧数 (4.3 frame_digest 贫瘠判定); stitch 子块 M16 属主 (rescued_short 单位 = 帧、
  // seams = 满足 T20 判据的拼接处数—接缝唯一计量点、judgments / repass_judgments =
  // 一遍/二遍逻辑判定数 (每候选一判、失败不计; votes>1 放大调用不放大判定), 3.16.6); extract.by_type 为按动作类型分布 (系统性劣化可观测, S1
4;
  // v1.9 注: 接缝占位步不计入 extract.transitions 与 by_type—非摘取产物, 3.15.4);
  // verify 子块见 3.7 stream 分支 (S31; defects 计数键 v1.9 起含 wrong_stitch, 3.7.2);
  // v1.12 两子块**条件在场** (对应开关开启才出现—off 时 stream 节与 v1.11 逐字节等价,
  // 退化锚; 两处精确集合断言测试同步): frame_classify 键义见 3.13.7 (calls = 实际派发窗数、
  // fallback = 兜底帧数、window_failures = 失败窗数、skipped_degraded = 降格跳过 episode 数);
  // frame_annotate 键义见 3.5.5 / 3.11.2 (annotated / skipped / failed 按成员计,
  // discarded = 终态非 active 信封携带的已产出未交付帧标注条目数—沉没成本记账)
  "dedup": {"exact": 118, "near_text": 201, "near_image": 46, "near_both": 47,
    "clusters": 366, "image_decode_failures": 2}, // v1.2: dedup.semantic 开启时另含 near_semantic 与 embedding_failure
s
  // v1.7 可选块 (classify 启用时出现): "classify": {"assignment": "single", "classes": {<name>: n, ...}, // 逐标签计数 (multi 下多标签
记录逐标签计)
  // "fallback_count": n, "failures": n [, "multi_label_records": n - 仅 multi]} (3.13.4 事件与计数行)
  "quality": {"mode": "pairwise_bt", "rounds": 4, "judgment_failures": 17,
    "aggregate_histogram": {"0.0-0.1": 12, "...": 0}, // 10 桶
    "per_criterion_mean": {"screenshot_readability": 0.61},
    "per_criterion_tie_rate": {"screenshot_readability": 0.31}}, // v1.5 只增: 仅 pairwise; 分母为拿到裁决的比较数 (调用级
失败不计入, 见 judgment_failures)
  // quality v1.7 只增: classify 启用时另含 "by_class": {<pool>: {"mode", "rounds", "aggregate_histogram",
  // "per_criterion_mean", "per_criterion_tie_rate"}}—每池携带有效 mode/rounds; 顶层 mode/rounds 保留 = 全局继承基值 (3.
4.3 按类分池行)
  "schema_engine": {"resolved_at": {"l0_or_clean": 4141, "l1": 87, "l3_1": 30,
    "l3_2": 3, "rejected": 9}},
  // v1.11 可选节 (上下文预算启用时出现—任一被启用阶段引用的 profile 声明 context_window):
  // "budget": {"profiles": {<profile>: {"context_window", "input_budget"}}, // 声明与预算终值
  // "w_min": {"segment.window": [cap, w_min]}, // 静态最坏装填量 (V9/V12)
  // "truncations": {<stage>: n}, // 各算子逐裁剪点计数
  // "overflow_records": n, // context_overflow 记录数 (7.6)
  // "image_cost": {<profile>: n}, // 每图成本校准终值 (V19)
  // "degrade_retries": n, "escalations": n} // V20 降级重试数 / V21 升级数
  // — counts-only (计数/统计, 不含数据内容, 2.6); M10 汇总 (3.10.3), 键义 V13②⑤
  // v1.2 可选块: "annotate": {"sc_disagreements": 0} (self-consistency 启用时);
  // "generate": {"buckets": {"default×concise": {"calls", "produced", "survived_dedup" [, "rejected_by_validator" -
v1.5, 仅配置 generate.sample_validator 时]]} (generate 启用时)
  // generate.buckets v1.7: classify 启用时桶 key 扩展为 "<class>×<llm>×<style>" (3.6.2 按类种子池行; 关闭时格式不变)
  // v1.13 可选子块 (generate.stream 启用时出现, 位于 generate 节内、buckets 之后; counts-only, 键集与键序冻结):

```

```

// "generate": {"buckets": {...},
//             "stream": {"sessions",          // 交织出的会话数 (**不含**重发尾会话)
//                       "crossed_sessions",    // v1.15 默认固定—/二 owner 装箱可由
//                                               // Σ幸存 - sessions_eff 导出;
//                                               // v1.16 幸存者投影后按剩余 owner 时间序列的
//                                               // 真实 A-B-A / B-A-B 交替重算, 不沿用代数公式
// "sequences": {<class>: {"planned", "produced"}}, // 按 [[classify.classes]] 声明序零基础铺开
// "tiers": {...}, // v1.14, **条件在场**：仅**全局** [[generate.stream.tiers]]
//                // 非空时出现 (全局表为锚, v1.15 判据零改动); 键位冻结在
//                // sequences 之后、frames 之前 (配额族相邻); 口径同
//                // sequences (planned 计于计划期、produced 数最终进链的
//                // 条数)。由 M10 **显式铺开** ⇒ 零额档与全作废档也如实
//                // 在场 (planned 0 / produced 0), 不依赖计数器首触序。
//                // **v1.15 双形** (按任一按类档位表是否在场二选一):
//                // ① 平面形 (全部序列类均未声明按类表) —
//                //    {"<tier_rank>": {"planned", "produced"}},
//                //    按全局表 rank 升序铺开; 与 v1.14 报表**逐字节相等**
//                //    (M6 恒喂类段计数器, 本形由 M10 按 rank 跨类求和)
//                // ② 类嵌套形 (任一 [[class.<name>.generate.tiers]] 在场) —
//                //    {"<class>": {"<tier_rank>": {"planned", "produced"}}},
//                //    外层 = 全部声明序列类按**声明序**零基础铺开
//                //    (sequences.<class> 同款)、内层 = 该类**生效表**
//                //    rank 升序; 零配额类与全作废档同呈 0/0
//                //    两形的键均为十进制字符串, 落盘无 sort_keys ⇒ 键序 =
//                //    装配插入序; 键位在两形下都仍冻结于 sequences 与 frames 之间
//                //    任务帧总数 (幸存序列的步数之和)
//                //    实际织入的噪音帧数 (签池耗尽时 < 目标数)
//                //    实际重发的序列条数 (按幸存数钳制后)
//                //    "plan_calls", "realize_calls", "noise_calls", // 三类调用数 (realize 含对半降级的分次调用)
//                //    "plan_failures", "realize_failures", // 作废序列数 (按失败环节归类)
//                //    "validator_scrapped"}} // sample_validator 逐帧违规作废的序列数
// — M6 属主 (3.6.5); 工件行数不在本子块, 登记于 run.artifact.lines (上文 run 节);
// `report.stream` 节在本形态下**不出现** (那是 segment 的观测面, 避免混淆);
// `report.classify` 直方图恒全零属预期 (标签 inherited、零判决调用, 3.13.4)
"trace": {"enabled": true, "path": "./out/ui-labels-0701.trace.jsonl",
          "events": 18342, "dropped_events": 0},
"llm_usage": {"default": {"calls": 31240, "prompt_tokens": 8.1e7,
                          "completion_tokens": 3.2e6, "est_cost_usd": 54.3, "retries": 210},
              "judge": {"...": 0}},
// llm_usage v1.6 只增: profile 对象另含
// "keys": {"<api_key_env 名>": {"calls", "rate_limited", "disabled"}} (仅密钥池 >1 时出现; 池内每把密钥各一项, 未用到的密钥为零计数;
密钥以环境变量名标识, 1.6 对齐决策 ⑤)
// 与 "parked_calls" / "parked_ms" (驻留统计, 3.9.3 密钥池行; 池 >1 或数值非零时出现—单密钥驻留亦须留痕)
"timing": {"wall_s": 5400, "per_stage_s": {"dedup": 40, "quality": 2900,
                                           "annotate": 1800, "verify": 620}}
}

```

不变量: $\text{emitted} + \text{dropped_}* + \text{failed} + \text{bad_input} = \text{scanned} + \text{generated}$ 。熔断中止 (v1.6 熔断交付, 3.10.3) 时扩展为 $\text{emitted} + \text{dropped_}* + \text{failed} + \text{bad_input} + \text{unprocessed} = \text{scanned} + \text{generated}$ —— unprocessed 仅此时出现, = 已扫描/已生成但因中止未走完流水线的记录数 (M10 在 finalize 时按差额计算)。v1.7: $\text{classify.assignment} = \text{"multi"}$ 时右侧另加 fanout —— $\text{emitted} + \text{dropped_}* + \text{failed} + \text{bad_input} = \text{scanned} + \text{generated} + \text{fanout}$; 与熔断中止叠加时两项扩展并存 (左侧 + unprocessed 、右侧 + fanout , 熔断残差公式同步, 3.10.3 分类与扇出行)。v1.8/v1.9: segment 启用时守恒式为全展开形 (3.10.3; stitched 为 v1.9 增项, 仅 stitch 启用时非零在场) ——

$$\text{emitted} + \text{dropped_dup} + \text{dropped_lowq} + \text{dropped_verify} + \text{dropped_noise} + \text{failed} + \text{bad_input} + \text{absorbed} + \text{stitched} = \text{scanned} + \text{generated} + \text{fanout} + \text{episodes}$$

(左侧新增 dropped_noise 与 absorbed (v1.8) 及 stitched (v1.9 壳终态; fanout (右侧) 计信封存在、 stitched (左侧) 计壳终态, 二者分别记账无双记——经审计数值验证)、右侧新增 episodes ; 未启用的项恒 0, 退化为上式)。 counts.threads 不入守恒式——它是恒等式 $\text{threads} = \text{episodes} - \text{stitched}$ 的导出量 (M10 post-emit tally 单点上报, 3.10.3; rescued_short 帧的 $\text{dropped_noise} \rightarrow \text{absorbed}$ 翻转发生在 emit 前、账目在路由时已定格, 不破坏两侧平衡)。且 **stream 模式下 $\text{counts.unprocessed}$ 的出现条件扩为「熔断 V interrupted」** (S18: SIGINT 中断叠加会话缓冲会产生未走完流水线的残差; 此时左侧另加 unprocessed , 残差公式右侧 + episodes 、左侧 + $\text{absorbed} + \text{dropped_noise}$ (v1.9 另 + stitched) 同步扩展, failed 兜底公式减项同步——三处同步见 3.10.3 线索缝合行); 非 stream 模式中断残差恒 0、不加键 (回归锚不动)。 $\text{schema_engine.resolved_at}$ 仅统计用户 Schema 的标注调用, 加总 = 进入 M5 的记录数 (4141+87+30+3+9 = 4270 = ingested 4987 - dropped_dup 412 - dropped_lowq 305); 裁决/评审/生成等内部 Schema 解析不计入。v1.12: **守恒恒等式与全部计数不变量零改动**——帧产物挂信封字段、不改任何信封状态 (成员帧保持 absorbed , 4.3 零改动声明), frame_classify.* / frame_annotate.* 是独立命名空间的新增计数、不进 counts 与守恒式; **resolved_at 恒等式不受帧标注影响**——帧标注走 $\text{complete_validated}$ 的显式 **schema 参数** (= 内部 Schema 待遇, 3.8.2 路由声明), 与裁决/评审/生成同列不入桶, 「加总 = 进入 M5 的记录数」在帧粒度开启时依然逐数成立。v1.13: **守恒恒等式零改动**——时间流生成形态取 generate_only 的退化形 $\text{emitted} + \text{dropped_dup} + \text{dropped_lowq} + \text{dropped_verify} + \text{failed} = \text{generated}$ (generated = 进链的直装序列条数; 成员帧从不构造信封 ⇒ absorbed / dropped_noise / stitched / episodes 四项恒 0 不出现, 噪音帧与

重发帧只活在工件、不进任何账)，`report.generate.stream.*` 与 `run.artifact.*` 同属独立命名空间的新增计数、不进 counts 与守恒式。
resolved_at 恒等式口径重述 (3.8.2 待遇参数)：「加总 = 进入 M5 的记录级标注调用数 (用户待遇族)」——按序列类标注 Schema 的调用虽显式传 schema，仍属记录级标注、照常计入 (`user_treatment=True`)；帧级标注等内部待遇调用仍不计。报告中无任何数据内容字段。

rejects 通道 v1.8 增量 (完整格式规范属 3.11.2，此处登记 IO 面变化)：rejects 行的 (stage, reason) 组合新增三种——`segment / noise` (LLM 判噪声帧)、`segment / below_min_len` (短段丢弃帧，独立于 noise, S11)、`verify / off_task_member` (修复收缩弃帧, S31)；`--strict` 交互注意：stream 工程下噪声帧属预期产物，会触发退出码 1。**rejects 通道 v1.9 增量**：(stage, reason) 组合再增一种——`stitch / stitch_invalid` (仅 `stitch.on_error = "fail"` 时出现, 3.16.6)；stitched 壳与被救援帧永不入 rejects (第四路由 / 翻转回 absorbed, 3.11.2) ——`--strict` 补注：同输入开启 stitch 后 (短段被救援不再落 rejects) strict 结果可能由 1 变 0，属预期 (2.4)。`output.rejects = "full"` 档对序列 Record 的原始载荷输出 `{"kind": "sequence", "member_ids": [...], "member_sources": [...]}` (S25——单记录 `_raw_payload` 假设的序列分支；`raw_last_output` 的 reason 门维持 schema_violation 现状，既有缺口明文接受)。**rejects 通道 v1.11 增量**：reason 词表再增两值——`context_overflow` (上下文预算三形态：预检 / 最小单元不装 / 反应态降级耗尽, V10/V16/V24) 与 `output_truncated` (响应以输出上限截断收尾的终局化, V11)；stage = 产生该错误的属主算子 (任何 LLM 调用阶段皆可出现)，语义、处置与熔断矩阵见 7.6；refs / full 档行形态不变 (两 kind 均不携带 `raw_last_output`)。**rejects 通道 v1.12 零增量声明**：帧粒度对本通道零改动——(stage, reason) 组合不增、reason 词表不增、行键集闭集不动；**帧级失败的成员不产生 rejects 行** (帧分类失败落 `fallback_class`、帧标注失败落 `members[]` 条目 `status="failed"`，均为成员级留痕非信封失败, 3.13.7/3.5.5/3.11.2)，`--strict` 判定读信封状态计数，不受帧失败影响 (裁决·成员失败不入 rejects)。**rejects 通道 v1.13 零增量声明**：时间流生成对本通道同样零改动——(stage, reason) 组合不增、reason 词表不增、行键集闭集不动；生成期作废的序列 (蓝图/帧实现失败、逐帧钩子违规、序列相似度过滤淘汰) **不产生 rejects 行**——它们从未成为记录，留痕在 `report.generate.stream.*` 计数与值-free 的 stderr WARN (3.6.5 作废语义，`--strict` 不受影响)；进链后被淘汰的序列 (`dropped_dup` / `dropped_lowq` / `dropped_verify` / `failed`) 照既有规则入 rejects——判决形评审的 fail 收尾即 `verify / dropped_verify` 的既有形态 (3.7.5)。

report.generate.stream v1.16 增量：当实际非零配额类的生效规则非空时，在既有 `tiers` (若在场) 之后、`frames` 之前出现 ``rules = {sampled, correlation_scrapped, temporal_scrapped}``；仅当 v1.16 report face 已由实际配额前缀的生效 `rules/windows` 或 `generate.sequence_validator` 激活、且配置了 `generate.sample_validator` 时出现 `sample_validator_scrapped`；`sample_validator` 单独配置不改变 v1.15 report bytes。配置了 `generate.sequence_validator` 时出现 `sequence_validator_scrapped`。当实际非零配额类的生效窗口非空时，同一位置出现 `windows = {calendar_days_spanned}`。这些块是 counts-only，条件在场时即使值为零也显式写出，零配额类的约束不会单独激活 report 面。

`sampled` 是完成机械 word 规划并进入 brief 调用的 attempt 数。冻结不变量为 ``validator_scrapped = sample_validator_scrapped + correlation_scrapped + temporal_scrapped + sequence_validator_scrapped``；序列相似度淘汰不进入该等式。约束阶段每条序列只在首个失败阶段递增一个子计数和一次总计数。`calendar_days_spanned` 按 `ts_start` 的固定 offset 计算幸存非噪音任务帧从首日到末日的自然日跨度，首尾包含，没有 survivor 时为零。规则、窗口、correlation 与 hook 返回文本不复制到 report 之外的任何输出面。

v1.16 守恒与 rejects 面：时间流生成仍使用 generate-only 退化守恒式；planner、规则、窗口、sample-validator 或 sequence-validator 作废发生在 Record 构造前，不增加 `failed`、`item.errors` 或 rejects 行。进链后的 `dropped_dup`、`dropped_lowq`、`dropped_verify` 与 `failed` 继续沿既有信封路由记账。`MODEL_INVALID` 和已通过启动校验后发现的不变量破坏仍走既有 `InternalError` / 退出码 4，不产生新错误 kind。

6.5 时间流工件格式 (v1.13)

时间流生成形态 (`generate_stream.enabled`, 3.6.5) 的第二份产物，路径 `{output_stem}.stream.jsonl` (M11 第五输出通道, 3.11.2)。UTF-8 JSONL，一行一帧，**行序 = 交织序** (时间戳严格递增)，行号 (1 基) 即 `_meta.stream.member_sources[].line_no`。

行结构 (三键，键序冻结)：

```
{ "<stream.order_by 的 meta 字段名>": "<ISO-8601 微秒精度时间戳>",  
  "<input.text_field>": "<帧内容: 纯文本帧为字符串 | 结构化帧为对象>",&br/>  "truth": {...}}
```

truth 键集 (冻结，counts 与标识，不含任何工具内部 id)：

键	类型	语义
session	int	全流会话序数，0 基 (含重发尾会话)。
sequence_class	str null	所属序列类名；噪音帧为 null。
sequence	int null	该序列在其类内的序数，0 基 (= 计划期标识)；噪音帧与重发副本为 null。

tier_rank	int null	v1.14 新增 ，仅档位表（全局 <code>[[generate.stream.tiers]]</code> ）在场时出现：该序列所属档位的序数。任务帧 = 本档序数；噪音帧为 null；重发帧承源（= 被重发序列的档位）。键位在 <code>sequence</code> 之后（序列身份组）、 <code>frame_class</code> 之前—— 键序重冻结 ，行文件的字节序由此定（id 用 canonical JSON 键排序计算，不受键序影响）。 v1.15 零改动 ：在场判据（全局表非空，全局表为锚）、键、键序、三类帧的取值规则全部不动；仅值来源条件化——= 本行序列类生效表内的档序数（生效表 = 该类的 <code>[[class.<name>.generate.tiers]]</code> 或回落全局表），故 跨类同序数不可比 ，逐行反推对账须先按同行 <code>sequence_class</code> 取该类生效表。
frame_class	str null	该帧的帧类（蓝图定下的真值）；噪音帧为 null。
noise	bool	插入型噪音帧标志；任务帧与重发帧恒 false。
duplicate_of	int	仅重发序列的帧在场 ：值 = 被重发的原序列的类内序数（重发副本无自身计划期身份，归属经本键对账）。

真值不携最终 id（封死循环依赖）：序列归属只用计划期标识（`sequence_class + sequence[+ tier_rank]`），**禁止携带装配后的 record id**——成员 id 依赖行内容、序列 id 依赖成员 id，携带即成环；主输出 `_meta.stream` 与工件的对账靠 `member_sources` 的行号双向可查（3.6.5）。

回填字段注记（v1.14）：声明了时间字段绑定（`[frame.class.<name>.generate.time_fields]`）的帧类，其行内文本字段对象里的绑定键不是 LLM 产出而是 harness 按已铺时间轴回填的机械量（值 = `round(序内相邻成员 ts 差, 6)` 等，词表见 5.2）。回填发生在时间戳铺设之后、行对象与 id 计算之前，故这些值同样进 `Record.raw`、进成员 id 与序列 id ⇒ 工件重放逐字节同 id 同会话（下方重放契约①）。对账口径两条：① 值按**本序列相邻成员计**——交叉会话里夹进来的外序列帧与噪音帧不参与差值，逐行对账时须先按 `truth.sequence_class + truth.sequence` 归组；② **duplicate_of 在场的重发行除外**——其绑定值与 `tier_rank` 同为承源量，不与自身所在会话的时间轴对账。

重放契约：工件本身就是一份合法的 6.1 文本模态输入。可往返性由 M1 工件键守卫在启动期保证（3.1.4 时间流生成行）：`input.text_field` 与 `stream.order_by` 的时间戳字段名均须为**平坦字段名**（工件行以该字符串原样作键，点路径在重放侧按 6.1 抽取时取不到、整份判坏行——含 `"."` 即 `CONFIG_ERROR`），两者互不同名、且均不得为 `"truth"`（工件行三个顶层键互斥）。把它拷为某工程的 `[run].input`、配同一份 `[stream]` 声明（`order_by = "meta:<同名字段>"`、同一 `gap_s`）并开 `segment`，即可原样重放——① 成员 `Record.id` 逐字节一致（M2 的 `sha256(canonical_json(raw))[:16]` 作用于同一份行对象，生成侧的成员 `raw` 就是工件行全对象，3.2.5）；② 会话切分一致（交织器铺设的会话间隔恒 > `gap_s`、会话内间隔恒 < `gap_s`），`session_id` 亦逐字节一致（M2 公式的输入含会话内**全部**帧，3.2.8）；③ `truth` 对摄取侧只是普通字段——参与 id 计算、**不参与任何判定**，需要时经 `output.passthrough_fields` 透传出来与重放结果比对（自动化的重放评测回路是明确非目标，2.1.2 ⑧）。

真实样例（`examples/synth-stream` 2026-08-20 成功真跑工件的前三行摘录，`order_by = "meta:ts"`、`text_field = "text"`，档位表与时间字段绑定均在场；实际为单行 JSONL）：

```
// 第 1 行: task_request 结构化帧; ticket_booking 的类内 tier_rank = 1, sequence = 0。
//      duration 是回填字段，值为同序列下一成员的 gap_next_s。
{"ts": "2026-01-05T10:20:10.000000+08:00", "text": {"subject_id": "A1B2C3", "utterance": "你好，我想买一张明天从北京到上海的高铁票。",
"entities": ["北京", "上海", "明天"], "duration": 2399.999998}, "truth": {"session": 0, "sequence_class": "ticket_booking", "sequence": 0, "tier_rank": 1, "frame_class": "task_request", "noise": false}}

// 第 2 行: 另一个 ticket_booking task_request; 与第 1 行在同一 session 内交织。
{"ts": "2026-01-05T10:20:10.000001+08:00", "text": {"subject_id": "trip001", "utterance": "你好，我想买一张高铁票，从北京到上海。", "entities": ["北京", "上海", "高铁票"], "duration": 2399.999999}, "truth": {"session": 0, "sequence_class": "ticket_booking", "sequence": 1, "tier_rank": 2, "frame_class": "task_request", "noise": false}}

// 第 3 行: sequence=0 的 acknowledgement; subject_id 与第 1 行按类型敏感规则相等。
{"ts": "2026-01-05T11:00:09.999998+08:00", "text": {"subject_id": "A1B2C3", "utterance": "好的，我来帮您查询。请问您想乘坐哪个时间段的车次呢？"}, "truth": {"session": 0, "sequence_class": "ticket_booking", "sequence": 0, "tier_rank": 1, "frame_class": "acknowledgement", "noise": false}}
```

三条可直接在工件上读出来的事实：① 第 1、2 行都是 `ticket_booking` 的结构化 `task_request`，但属于同一 `session = 0` 中交织的 `sequence = 0` 与 `sequence = 1`；② 第 3 行回到 `sequence = 0`，是与第 1 行 `subject_id` 相等的 `acknowledgement`，其时间差为 2399.999998 秒，落在 [1200, 2400]；③ 三行均为任务帧（`noise = false`），第 1、3 行是同一序列的前两步，第 2 行则证明了真实 crossing 交织，而不是旧版噪音或重发样例。

v1.16 工件不变与对账规则：规则/窗口约束只影响 planner 的 word、owner、timestamp 与 noise 槽，不改变三顶层键、`truth` 键序、`tier_rank` 归属、`duplicate_of` 语义、成员 id 或序列 id 公式。planner 投影作废 attempt 后删除空 session、按时间重编号但不移动幸存 timestamp；crossed 只有两个 owner 都幸存且仍有真实交替时保留。回填发生在 primary timestamp 铺设之后、行对象与 id 计算之前；duplicate 复制已经回填的 source payload，按固定规则平移到流尾，绝不按 duplicate 自身 timestamp 重算时间字段。

规则、日历窗口、correlation、sequence-validator 结果和 CP-SAT witness 都不写入 `truth` 或 artifact。对 artifact 的可验证事实仍是：同一 `[stream]` 重放时，session 内相邻 timestamp 差不超过 `gap_s`，跨 session 至少多出 1us，M2 严格大于 gap 的切分规则因此得到相同会话边界；`truth` 仅作为普通输入字段参与 Record id 的 canonical JSON 计算，不参与规则或分段判定。

7. 日志系统与可观测性

本章定义 LabelKit 的日志体系（承载模块为 M12，见 3.12）：两条相互独立的输出通道，外加一个不属于日志的进度显示面。本章核心是 7.2 的事件目录——它与第 5 章配置表、第 6 章输出格式同级，属对外稳定契约。

7.1 日志体系总览

输出面	形态与开关	内容	消费方
① 运行日志	stderr, 恒开。级别 debug/info/warn/error (tool.log_level / --log-level); 行格式 tool.log_format = "text" (默认) "jsonl" (7.3)。	仅运维事件：生命周期、警告、错误、LLM 调用摘要 (debug 级)；v1.11 增启动期上下文预算参数 INFO 行 (如 segment: w_min=6 window=20 (budget) ——数据无关、仅计数与参数, M10 启动段属主, V13①, 3.10.3)。绝不含数据内容与提示词。	人工排障；日志采集系统。
② trace 追踪日志	JSONL 文件, 默认 {output_stem}.trace.jsonl (trace.path); 默认关 (trace.enabled = false)。文件在首个事件写出时才打开/截断 (v1.5)：死于配置或输入校验的运行不会碰上一一次的 trace；dry-run 写 {name}.dryrun{suffix} 独立文件。	结构化事件流，一行一事件 (7.2)；按 trace.channels 过滤通道，按 trace.content 控制内容量 (7.4)。	rubric 优化 (7.5)；标注质量分析与审计；后续 labelkit analyze (8.3 O5)。
③ 进度显示 (v1.10 起称 console)	三态 console.mode = auto \ rich \ plain (5.1/ CLI - --console), 恒开。rich = 双区内联实时面板；plain = v1.9 行为逐字节保留 (heartbeat_s=0 默认下)；auto 按 TTY 等判定链选档 (7.7)。	批进度、流水线段棋盘、各状态计数、LLM 用量/密钥池/熔断、瞬时成本累计 (7.7 区块表)。	交互终端前的人。不属于日志 (7.7)。

与「工具不存储数据」原则的关系：trace 是用户显式启用的输出通道，与主输出、rejects 同级 (2.6「数据不落盘」行已将其列入唯一写盘对象)，而非中间态落盘——它记录的是「处理判定及其依据」而非批中间态本身。trace 文件的保留、脱敏档位选择 (7.4) 与清理均为用户责任；content="full" 档含数据内容，风险见 7.4 明示。

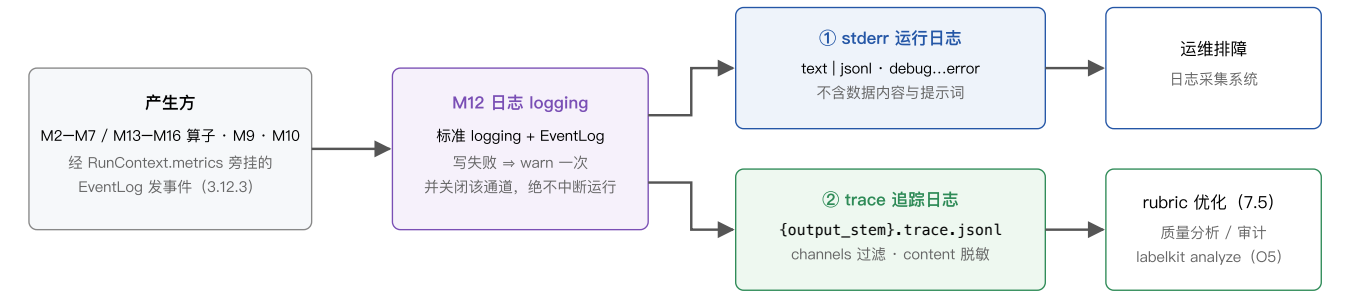


图 7-1 双通道日志架构。进度显示走 stderr 但不经日志设施 (7.7)。

7.2 事件目录

通道归属规则：事件名前缀即通道名 (ingest. / segment. (v1.8) / stitch. (v1.9) / dedup. / classify. (v1.7) / extract. (v1.8) / quality. / annotate. / verify. / schema. / llm. , 对应 trace.channels 的十一个可选值——v1.7 增 "classify", v1.8 增 "segment"、"extract", v1.9 增 "stitch" (通道 = stage 名，与 classify 同构, S1)，默认值不变, 5.2)；run.* 与 batch.* 生命周期事件由 M10 发出、stage="run"、record_ids 为空，不受 channels 过滤 (trace.enabled=true 即写)；error 事件按产生它的 stage 归属通道 (segment/extract 阶段的 error 事件自动按此归属，零路由代码改动, S1)。本目录为稳定契约： trace_schema_version = 1 , 后续版本只增不改 (可新增事件与 payload 字段，不改既有字段语义)。

事件名	通道 / stderr 级别	触发点	payload 字段
run.start	恒写 / info	M1 校验通过、首批开始前；trace 文件首行 header 事件。	tool_version、config_digest、project_digest (同 6.4 run 节)、trace_schema_version (=1, 仅此事件携带，避免逐行冗余)。
run.end	恒写 / info	finalize 完成后 (trace 末行)。	counts (与 report.json counts 同构的摘要对象)、exit_code。
batch.start	恒写 / debug	批构造完成 (PipelineItem[] 就绪)。	size。

batch.end	恒写 / info	批 emit 并释放中间态后。	active、dropped_dup、dropped_lowq、dropped_verify、failed、duration_ms；v1.7 增可选字段 fanout（仅 classify 启用时携带；batch.start.size 语义 = 批入口信封数即扇出前基数，扇出后各状态计数和 = size + fanout，3.10.3 分类与扇出行）；v1.8 增可选字段 episodes、absorbed、dropped_noise（仅 segment 启用时携带，fanout 同形制，3.10.3 时序流行）；v1.9 增可选字段 stitched、threads（仅 stitch 启用时携带，同形制，3.10.3 线索缝合行）。
ingest.bad_line	ingest / warn	M2 坏行跳过（3.2.5）。	file、line_no、reason。
ingest.missing_pair	ingest / warn	M2 缺对跳过（3.2.4）。	index、present（"tree" "image"，实际存在的一侧）、file。
ingest.index_conflict	ingest / warn (fail 策略时 error)	M2 index 冲突（3.2.4）。	index、files（冲突文件路径列表）。
ingest.disorder	ingest / — (trace-only，无逐事件 stderr 镜像；v1.8)	M2 流式单调性校验拒绝一条记录时（乱序或时间戳解析失败，stream.on_disorder，3.2/6.1）；skip 策略下每记录一事件，M2 自身另打一条全运行仅一次的数据-free stderr WARN (reason 含时间戳/游标值故不入 stderr——§7.1 ①)；fail 策略经 InputError 以退出码 3 终止。	file、line_no（文本） index (UI)、reason（out of order: ... timestamp parse failure: ... 类文案，含违规值——仅 trace 通道）。
segment.session	segment / — (trace-only，无 stderr 镜像；v1.8)	M2 会话装配器闭合一个候选会话时（3.2 会话化行；--limit 截断视同 EOF 冲洗尾会话，S17）——发出方是 M2，但按前缀归 segment 通道 (S1)；record_ids 为空。	session_id、first / last（会话首/末帧）、len、cause（"gap" "key" "max_len" "max_span" "eof" "limit"）。
segment.boundary	segment / — (trace-only，无 stderr 镜像；v1.8)	M14 每个滑窗裁决经 M8 校验通过后（3.14）；record_ids 为空（成员溯源在 payload）。	session_id、window（=[s, e] 窗口帧位次区间）、member_ids、relations [{ index, relation }（五值封闭词表，3.14）、model、reason †（逐帧关系理由，条件见表下注）。
stitch.judge	stitch / — (trace-only，无 stderr 镜像；v1.9)	M16 每候选判定定案后（votes 聚合之后；一遍与二遍均发，3.16.6）；record_ids = [候选碎片首成员 id]。	session_id、candidate（"episode" "rescue"）、repass (bool, false = 一遍 / true = 二遍)、verdict（votes 分裂回落时记保守结局 "new"）、thread_ref、confidence（仅观测，不进门槛，3.16.3）、priors（机械先验命中腿列表，⊆ {app_overlap, entity_overlap, same_page}）、merged (bool)；条件字段：votes_split（= true，仅严格多数不成立回落时携带）、task_name †、reason †（votes 分裂时不携带）、target_thread_id（仅 merged 时携带）。
stitch.thread	stitch / — (trace-only，无 stderr 镜像；v1.9)	会话缝合定案后每线索一条（3.16.6）；record_ids = [幸存信封 record.id]。	session_id、thread_id、task_name †、fragments [{ order_span, member_count, cause, source_episode }（碎片跨度表）、seam_indexes。
dedup.duplicate	dedup / —	M3 判重时；record_ids = [被判重记录 id]。	kind、cluster_key、kept_id（同 DedupInfo，4.2）、jaccard（near_text: 实测估计值）或 hamming（near_image: 实测距离）或 cosine（near_semantic: 实测余弦相似度，v1.2 增）；精确重复三者皆无。
classify.decision	classify / — (trace-only，无 stderr 镜像，同 quality.judgment；v1.7)	M13 每记录分类定案后（3.13.4）；record_ids = [记录 id]。	label（本信封路由标签）、labels（multi 时携带命中全集）、source（"lm" "fallback" "inherited"）、reason †、sc（={n, agreement_ratio}，仅 classify.self_consistency 启用时携带）。
classify.frame	classify / — (trace-only，无 stderr 镜像；v1.12)	M13 每 episode 帧级批量判定定案后（含窗失败兜底，3.13.7）；record_ids = [episode id]。	members（判决成员数）、windows（实际派发窗数）、fallback（兜底帧数）——仅三个计数，不携带任何数据内容（trace 载荷纪律，3.12.4）。
extract.step	extract / — (trace-only，无 stderr 镜像；v1.8)	M15 每对相邻成员帧的转移摘取定案后（含 fallback，3.15）；record_ids = [s_i.id, s_{i+1}.id]（前后帧记录 id）。	episode_id、index、action_type、description †（LLM 产出文本，refs 档起）、target § / value §（输入数据派生字段，excerpt 档起——S27，7.4 _DATA_KEYS 行）。

quality.judgment	quality / —	M4 每次 pairwise 裁决经 M8 校验通过后；record_ids = [记录甲, 记录乙] (采样顺序)。rubric 优化的核心事件。	order ({ "A": id, "B": id }, 随机化后的呈现顺序)、model、judgments [{ criterion, winner ("A" "B" "tie"), reason t }]; v1.2 增可选字段 judge (= 评审 profile 名, 仅 quality.judges 非空时携带, 3.4.3); v1.7 增可选字段 pool (= 类名, 仅 classify 启用时携带, 3.4.3 按类分池行)。
quality.pointwise	quality / —	M4 pointwise 每记录每 criterion 打分后。	criterion、score (0—5 原始分)、reason。
quality.bt_fit	quality / —	M4 每批每 criterion BT 拟合结束 (批级, record_ids 为空)。	criterion、iterations、converged、comparisons (参与拟合的比较数); v1.7 增可选字段 pool (= 类名, 仅 classify 启用时携带——分池后拟合可归因, 3.4.3 按类分池行)。
quality.gate	quality / —	M4 质量门判定 (配置了 quality.threshold, 或 quality.selection = "top_ratio" 时, 3.4.3)。	aggregate、decision ("keep" "drop")、threshold (threshold 选择时); v1.2 增可选字段 selection、top_ratio、rank (top_ratio 选择时携带, payload 只增不改); v1.7 增可选字段 pool (= 类名, 仅 classify 启用时携带, 3.4.3 按类分池行)。
annotate.done	annotate / —	M5 标注经 M8 通过后。	attempts (同 Annotation.attempts, 4.2); v1.2 增可选字段 sc = {n, agreement_ratio} (self-consistency 启用时, 3.5.2); v1.7 增可选字段 label (= 信封路由标签, 仅 classify 启用时携带, 3.5.2 按类取值段)。
annotate.frame	annotate / — (trace-only, 无 stderr 镜像; v1.12)	M5 帧级逐帧标注每成员定案后 (annotated / skipped / failed 三种结局均发, 3.5.5); record_ids = [episode id]。	member_id、status ("annotated" "skipped" "failed")、attempts (skipped/failed = 0); 可选 excerpt (= {member_id: 帧标注对象 dump}, 仅 "excerpt"/"full" 档携带并截 200 字, 7.4) —— 不新增任何承载数据内容的 payload 键。
verify.verdict	verify / —	M7 每轮评审后 (round 从 1 计, repair 策略下每轮一事件)。	verdict、round、critiques [{ aspect, opinion }]; v1.2 增可选字段 judge (verify.judges 非空时每 judge 一事件, 3.7.2); v1.7 增可选字段 label (仅 classify 启用时携带, 3.7.2 按类取值段); v1.8 增可选字段 defects [{ kind, members, position, detail } (仅 stream 缺陷表评审时携带, 受 7.4 分级——detail 属自由文本, 3.7.2 缺陷表段/S31)。
schema.repair	schema / —	M8 任何非 clean 路径出结果时 (L1 修复命中 / L3 各轮 / 拒绝)。	resolved_at ("l1" "l3_1" "l3_2" "rejected", 同 6.4 命名)、violations (JSON Pointer 路径 + 违反的 Schema 关键字摘要, 不含数据值); 违规清单中来自 output.validator 回调的条目以 (validator) 前缀标识 (v1.5, 3.8.2 L2.5); v1.5 增可选字段 l1_lossy (=true, 仅当 L1 修复疑似截断内容——json-repair 对未转义内引号的截断故障启发式, 命中时另发 stderr warn, payload 只增不改)。
llm.call	llm / debug (stderr 摘要行恒有)	M9 每次调用返回后 (含失败)。不含提示词与响应内容 (full 档例外, 7.4)。	profile、gen_ai.request.model、latency_ms、gen_ai.usage.input_tokens、gen_ai.usage.output_tokens、retries、status ("ok" "retryable_exhausted" "fatal" "breaker_aborted"——v1.5 只增: 重试退避途中熔断打开、该逻辑调用被中止时发出, retries 携带已消耗次数); v1.2 增可选字段 operation (= "embedding", 仅 M9 embed() 调用携带, 缺省即对话补全; payload 只增不改); v1.6 增可选字段 key_env (= 本次逻辑调用最后一次尝试所用密钥的环境变量名, 成功失败同义; 零尝试即终止的调用——如入口即驻留超限/breaker_aborted——不携带; 仅密钥池 >1 的 profile 携带; payload 只增不改)。
llm.key_cooldown	llm / —	v1.6: M9 密钥进入 429 冷却时 (每次冷却一事件, 3.9.3 密钥池行)。任意池大小 (含 1) 均发出——单密钥 429 的等待在 v1.6 亦经冷却/驻留路径。	profile、key_env、cooldown_s (本次冷却秒数)、retry_after (bool, 冷却时长是否来自 Retry-After 头)。
llm.key_disabled	llm / warn	v1.6: M9 密钥 401/403 认证禁用时 (每密钥每运行至多一次; 任意池大小含 1——单密钥场景该事件先于立即熔断发出)。	profile、key_env、status_code。
llm.pool_parked	llm / warn	v1.6: M9 某 profile 全部存活密钥均在冷却、调用开始驻留时 (每次驻留一事件; 任意池大小含 1)。	profile、wait_s (预计驻留秒数)、live_keys (存活密钥数)。
error	随产生 stage 的通道 / warn (记录级) · error (运行级)	StageError 构造时。	stage、kind (取值见 7.6)、message、retryable ——即 StageError 全字段 (4.2); v1.7 增可选字段 label (仅 classify 启用时携带——multi 扇出下消歧同 id 兄弟信封, 3.13.4)。

不经事件目录的 CLI 顶层 `stderr` 行 (2026-08-14 明示): `labelkit/cli/main.py` 的异常收口在事件目录之外另打三条 ERROR 级运行日志 (它们发生在 M12 事件生命周期结束之后或之外, 故不入本目录、不进 `trace`):

```
command <cmd> aborted: <message> (exit <n>)      # 已归类的 LabelKitError
command <cmd> interrupted by user (exit <n>)      # KeyboardInterrupt
command <cmd> failed: <message> (exit <n>)        # 未预期异常
```

配置错误另经 `stdout/stderr` 打印聚合抬头 `ConfigError: {n} config error(s) (all aggregated)` 加逐条错误行 (3.1.5), `validate` 通过时打印一行 `configuration valid`。

v1.13 零增量声明 (时间流生成): 本目录对时间流生成形态零改动——**零新通道** (通道枚举维持 11 值; 蓝图、帧实现与噪音批三类调用经既有 `llm.call` 完全可见, `generate` 专属通道列 8.4 演进候选)、**零新事件** (不新增任何事件名, 既有事件的 `payload` 亦不增键)、**零新错误 kind** (7.6 词表不动: 蓝图/实现的结构失败复用 `schema_violation`、预算面复用 `context_overflow` / `output_truncated`, 且这些失败不产生记录级 **StageError**——序列直接作废, 留痕在 `report.generate.stream.*` 计数与值-free 的 `stderr` WARN, 3.6.5)。

v1.14 零增量声明 (帧类构成档位与时间字段回填): 两机制对本目录同样零改动——**零新通道** (通道枚举维持 11 值)、**零新事件** (不新增事件名, 既有事件的 `payload` 亦不增键——档位面的全部观测经 `report.generate.stream.tiers` 承载、时间字段面零观测增量)、**零新错误 kind** (7.6 词表不动: 蓝图覆盖违约是普通的 L2 违规, 复用 `schema_violation` 进 M8 修复环, 穷尽后复用既有的序列作废语义与 `plan_failures` 计数)。M1 侧新增的三条 WARN (配分零额、帧类未入档、绑定字段携带额外约束关键字) 走既有的配置告警面 (3.1.5), 值-free 纪律不变——类名、帧类名、字段名与关键字名均为配置量而非数据内容。

v1.15 零增量声明 (按类档位表): 按类化对本目录同样零改动——**零新通道** (通道枚举维持 11 值)、**零新事件** (不新增事件名, 既有事件的 `payload` 亦不增键: 档位面的全部观测仍只经 `report.generate.stream.tiers` 承载, v1.15 改的是该子块的形状与其计数器键名, 两者都不进事件目录)、**零新错误 kind** (7.6 词表不动)。M1 侧新增的按类表前提三子款 (形态门 / 全局锚 / 空表拒收) 走既有 `CONFIG_ERROR` 面, 逐生效表化后的两条既有 WARN (配分零额、帧类未入档) 走既有配置告警面 (3.1.5), 值-free 纪律不变——序列类名与帧类名同为配置量而非数据内容。

v1.16 零增量声明 (序列规则与联合规划): 本目录继续维持既有通道枚举、事件名与 `StageError.kind` 闭集。CP-SAT planner 只使用值无关的英文 `stderr` WARN/ERROR, M1 的 `INFEASIBLE` / `UNKNOWN` 属于配置错误聚合而不是 `trace` 事件; `MODEL_INVALID` 与通过 M1 后发现的 planner 不变量破坏复用 `internal_error` 退出面。noise 目标未完全放置只发一次值无关 WARN。sequence-validator 异常日志只含 hook 引用与异常类型, 违规日志只含 序列索引、类名和违规数; declarative 作废日志只含序列索引与类名。任何 planner witness、时间表、correlation 值、规则文本、窗口文本、hook message、payload、prompt 和 API key 都不得进入 `stderr`、`trace`、`report` 或 `artifact`。所有生成调用仍经既有 `llm.call` 事件可见, 不建立 `generate` 专属通道。

† `reason` 仅当 `quality.judgment_reasons` 生效时存在 (5.2); `classify.decision` 的 `reason` 条件独立 (v1.7) = `trace.enabled = true` 且 `trace.channels` 含 `"classify"` (零额外 token 原则, 3.13.4 调用与校验行); `segment.boundary` 的 `reason` 条件同款 (v1.8) = `trace.enabled = true` 且 `trace.channels` 含 `"segment"` (对应窗口内部 Schema 的 `with_reason` 参数, 零额外 token, 3.14)。¶ `stitch.judge` / `stitch.thread` 的 `task_name` 与 `reason` (v1.9) 无请求条件——`stitch_schema()` 恒含两键 (判定量级小、votes 聚合需按多数簇取值, 3.16.3), 但作为 LLM 自由文本受 7.4 分级: `none` 档剥除、`refs` 档起携带 (`task_name` 为 v1.9 新增自由文本键, 7.4)。‡ / § 为 `extract.step` 的内容分档标记 (v1.8, S27): `description` 自 `"refs"` 档起、`target` / `value` 自 `"excerpt"` 档起携带 (7.4)。全部自由文本字段 (`reason` / `critiques` / `violations` 文本) 受 7.4 脱敏档位控制。密钥相关事件 (v1.6) 只携环境变量名——密钥值在任何档位、任何通道均不落日志 (7.4 规则不变)。

7.3 记录格式规范

`stderr` 两种格式承载同一信息, 由 `tool.log_format` 选择; `trace` 事件行是 `TraceEvent` (3.12.3) 的 JSON 序列化, 恒为 UTF-8 单行 (下例因排版自动折行)。日志正文语言 (2026-08-14 代码规则整改): `stderr` 运行日志、报错消息与 CLI 输出统一英文 (数据内容原样, 不翻译); 中文只出现在源码注释/docstring 与 LLM 提示词模板中。

```
# — stderr, tool.log_format = "text" (行格式: ts level stage batch msg) —
2026-07-02T09:31:04+08:00 INFO   quality batch=3 pairwise done items=128 comparisons=256 judgment_failures=1
2026-07-02T09:31:12+08:00 WARN   ingest batch=4 bad_line file=ime-2026-06-30.jsonl line=217 reason=missing_text_field

# — stderr, tool.log_format = "jsonl" (同一事件) —
{"ts":"2026-07-02T09:31:04+08:00","level":"info","stage":"quality","batch":3,"msg":"pairwise done items=128 comparisons=256 judgment_failures=1"}
```

```
# — trace 事件一: quality.judgment (文本模式意图标注工程; content="refs", judgment_reasons 生效) —
# 记录甲 1cda030abc565f17 = {"instruction": "帮我写一条请假条, 明天上午要去医院", ...}
# 记录乙 d5ad41d6357f8a55 = {"instruction": "写一份周报模板", ...}; 本次呈现顺序随机为 A=乙, B=甲
{"ts":"2026-07-02T09:31:04.482+08:00","run_id":"f3a9c04b7d21","batch_no":3,"stage":"quality","ev":"quality.judgment","record_ids":["1cda030abc565f17","d5ad41d6357f8a55"],"payload":{"order":{"A":"d5ad41d6357f8a55","B":"1cda030abc565f17"},"model":"qwen2.5-vl-72b-instruct","judgments":[{"criterion":"writing_style","winner":"tie","reason":"两条指令表达均通顺完整, 写作水平相当。"}, {"criterion":"facts_trivia","winner":"B","reason":"B 含明确时间与事由 (明天上午去医院), 具体信息更多。"}, {"criterion":"educational_value","winner":"B","reason":"B 是带场景约束的写作任务, 示范价值更高。"}, {"criterion":"required_expertise","winner":"tie","reason":"两者均为日常任务, 无专业门槛差异。"}]}}

# — trace 事件二: verify.verdict (UI 模式登录页工程; content="refs") —
{"ts":"2026-07-02T10:02:17.905+08:00","run_id":"0c47d9e2b8a5","batch_no":3,"stage":"verify","ev":"verify.verdict","record_ids":["9f2c31ab52e08d17"],"payload":{"verdict":"pass","round":1,"critiques":[{"aspect":"任务指令遵循","opinion":"四个字段均按指令填写, screen_category=login 与截图一致。"}, {"aspect":"事实一致性","opinion":"interactive_elements 与控件树可交互节点逐一对应, bounds 未见编造。"}, {"aspect":"字段语义","opinion":"page_title 取自屏幕顶部标题控件文本, 正确。"}]}}
```

LLM 相关 payload 的字段命名 (`gen_ai.request.model`、`gen_ai.usage.input_tokens / output_tokens`, full 档的 `gen_ai.input.messages / gen_ai.output.messages`) 对齐 OpenTelemetry GenAI 语义约定 [27], 便于 OTel 生态的采集与分析工具直接消费。注意两点: 这是**命名对齐而非实现依赖** (不引入 OTel SDK); 且该约定截至本文档日期处于 Development (实验性、非 stable) 状态 [27]——若其后续更名, 本工具事件目录按「只增不改」原则保持自身稳定。

7.4 内容脱敏策略

`project.toml` 的 `trace.content` 四档决定 trace 中的内容量, 逐档递增; **API Key 在任何档位、任何通道都不落日志** (M9 不将认证头传入日志路径):

档位	trace 事件 payload 中出现的内容	典型用途
"none"	仅记录 id、枚举与数值等结构化字段; <code>reason</code> 、 <code>critiques</code> 、 <code>violations</code> 等 LLM 产出文本一律不写。	最严格合规场景; 7.5 的全部比率类指标仍可计算。
"refs" (默认)	另含 LLM 产出的 <code>reason / critique / violations</code> 文本, 但不含任何输入数据内容 (与 <code>output.rejects="refs"</code> 同一语义, 3.11.2)。	rubric 优化常规档 (7.5)。
"excerpt"	另含输入内容前 200 字符: <code>quality.judgment / quality.pointwise / annotate.done / verify.verdict</code> 的 payload 增加 <code>excerpt</code> 字段 (<code>record_id</code> : 前 200 字符) 逐条给出, <code>quality.judgment</code> 含两条; 文本模式取 <code>Record.text</code> 、UI 模式取 <code>UITree.serialize()</code> 输出, 不含图像)。	免于回原始文件比对的快速人工审查。
"full"	另含完整提示词与响应: <code>llm.call</code> 事件 payload 增加 <code>gen_ai.input.messages / gen_ai.output.messages</code> (须同时启用 <code>llm</code> 通道)。	调试与审计。

v1.8 只增 (S27): `extract.step` 的 `target / value` 是输入数据派生字段 (目标控件文本引用、键入文本——可能含用户输入内容), 归入新增剥除集 `_DATA_KEYS = {"target", "value"}: "none"` 与 `"refs"` 档一律剥除 (守住 `refs` 档「不含任何输入数据内容」红线), 自 `"excerpt"` 档起携带; `description` 为 LLM 产出文本, 计入既有自由文本集 `_FREE_TEXT_KEYS: "none"` 档剥除、自 `"refs"` 档起携带 (与 `reason / critiques` 同级); `verify.verdict` 的 v1.8 可选字段 `defects` (缺陷表含自由文本 `detail`) 同入 `_FREE_TEXT_KEYS` —— `"none"` 档整键剥除 (与 `critiques` 同级)。**v1.12 只增:** 帧标注内容仅经既有 `excerpt` 键按档位分级——`annotate.frame` 的可选 `excerpt` 字段 (`{member_id: 帧标注对象 dump}`) 与其余 `excerpt` 同族: `"none" / "refs"` 档剥除、自 `"excerpt"` 档起携带并截 200 字 (3.5.5 事件载荷纪律; 脱敏集与键家族零扩充)。逐事件分级速查: `extract.step none = {episode_id, index, action_type}`、`refs = +description`、`excerpt = +target/value`; `segment.boundary none = 结构字段 (session_id / window / 逐帧 relation)`、`refs = +reason` (`reason` 键已在自由文本集)。三个 v1.8 事件的 `stderr` 镜像均无 (`trace-only`, 7.2)。

v1.9 只增: `task_name` (线索任务名——`stitch.judge / stitch.thread` payload 及缝合判定输出的滚动线索名, 3.16) 为 LLM 产出文本, 计入 `_FREE_TEXT_KEYS: "none"` 档剥除、自 `"refs"` 档起携带 (与 `reason / description` 同级)。逐事件分级速查: `stitch.judge none = {session_id, candidate, repass, verdict, thread_ref, confidence, priors, merged[, votes_split, target_thread_id]}`、`refs = +task_name/reason`; `stitch.thread none = {session_id, thread_id, fragments, seam_indexes}`、`refs = +task_name`。两个 v1.9 事件的 `stderr` 镜像均无 (`trace-only`, 7.2)。

风险明示: `content="full"` 时 `trace` 文件包含全部经手数据与模型输出, 体积可达主输出的数十倍, 且构成一份完整的数据副本——仅在调试/审计时短期启用, 用后由用户负责清理。

7.5 rubric 优化闭环

trace 的首要用途：把「LLM 依据 rubric 做出的每一次裁决及其理由」暴露给人，驱动准则迭代。流程：① `project.toml` 设 `trace.enabled=true`、`channels` 含 `"quality"`、`quality.judgment_reasons=true`（或保持 `"auto"`）；② `--limit` 小样本运行；③ 从 `trace` 计算下表诊断指标并抽读 `reason`；④ 修订 `rubric`（改写 `pairwise_prompt`、合并/删除/新增 `criterion`、调 `weight`）；⑤ 同一 `run.seed` 小样本重跑，对比指标是否收敛；⑥ 定稿后全量运行（可关 `judgment_reasons` 省 `token`）。

信号	计算（基于 7.2 事件）	诊断	处置
tie 率过高	某 <code>criterion</code> 的 <code>winner="tie"</code> 占比（ <code>quality.judgment</code> ）；参考线 > 0.4 。	准则区分度不足。	把 <code>pairwise_prompt</code> 的比较问句改为更具体的判据；或降低该 <code>criterion</code> 权重。
同 <code>seed</code> 重跑翻转率	同一 <code>run.seed</code> 、同一输入重跑两次（配方案逐对相同，1.3 可复现性——流程 ⑤ 的重跑天然产生第二份 <code>trace</code> ），按无序对齐两次 <code>quality.judgment</code> ，统计 <code>winner</code> 相反的对占比；参考线 > 0.3 。	准则表述含糊 / 模型不稳定。	细化 <code>description</code> 与判据；确认 <code>temperature=0</code> ；必要时更换裁决 <code>profile</code> 。
<code>criteria</code> 间胜负相关性过高	两 <code>criterion</code> 在同一次比较中 <code>winner</code> 一致的占比；参考线 > 0.9 。	准则冗余，可合并。	合并为一条（权重相加）或删除其一。
<code>reason</code> 关键词缺口	<code>reason</code> 文本聚类/高频词中反复出现 <code>rubric</code> 未覆盖的评价维度（需 <code>judgment_reasons</code> 生效且 <code>content ≥ refs</code> ）。	准则缺口。	新增 <code>criterion</code> ——这正是 <code>criteria drift</code> 的预期表现 [30]。
<code>judgment_failures</code> 率	<code>error</code> 事件中 <code>kind="judgment_invalid"</code> 数 / 总比较数（亦见 <code>report.quality.judgment_failures</code> ）；参考线 > 0.05 。	裁决提示词或内部 <code>Schema</code> 问题。	缩短/消歧 <code>rubric</code> 文案；确认 <code>profile</code> 的结构化输出能力（L0）。

jq 单行示例——统计文本工程中 `facts_trivia` 准则的 tie 率：

```
jq -s '.[.] | select(.ev=="quality.judgment") | .payload.judgments[]
| select(.criterion=="facts_trivia")
| (map(select(.winner=="tie")) | length) / length' ./out/ime-labels-0630.trace.jsonl
# 输出示例：0.4375 → 高于 0.4 参考线，该准则对输入法指令数据区分度不足，应改写判据
```

背书：EvalGen (UIST 2024) 通过混合主动式实验确认了 **criteria drift**：评估准则无法先验完全确定——人需要看到 LLM 评审的具体输出才能修订、新增准则，准则部分依赖于观察到的输出 [30]；因此评审的中间产物（逐次裁决与理由）必须暴露给人，这正是 `quality.judgment` 事件的存在理由。CriticQ (ACL 2025) 进一步证明「成对偏好信号 → 自然语言质量准则」方向可行：约 30 对人工偏好即可自动挖掘出可解释准则 [31]。本节闭环即两文献结论的工程化：`trace` 承担 EvalGen 的对齐界面职责，人工（或后续 `labelkit analyze`，8.3 O5）承担 CriticQ 的准则挖掘角色。

7.6 错误分类码 (StageError.kind)

kind	级别	产生点与处置
<code>bad_input_line</code> / <code>missing_pair</code> / <code>index_conflict</code> / <code>image_too_large</code>	记录级	M2；按 <code>input.*</code> 策略 <code>skip</code> （计数）或 <code>fail</code> （退出码 3）。
<code>image_decode_error</code>	记录级	M3 跳过 <code>pHash</code> 层；M5/M7 遇到时该记录 <code>failed</code> 。
<code>segmentation_invalid</code>	窗口级	v1.8：M14 单窗边界裁决经 M8 修复耗尽—— <code>segment.on_error="keep"</code> （默认）时该会话整体成一个 <code>episode</code> 存活，留痕三件套 <code>_meta.stream.degraded = {kind, windows_failed}</code> + <code>error</code> 事件 + <code>segment.failures</code> 计数（不写 <code>item.errors</code> ——归因防污染，S26）； <code>"fail"</code> 时会话成员全部 <code>failed</code> → <code>rejects</code> (3.14)。
<code>stitch_invalid</code>	判定级	v1.9：M16 单候选缝合判定经 M8 修复耗尽—— <code>stitch.on_error="keep"</code> （默认）时 <code>episode</code> 候选开新线索存活 / 救援候选维持 <code>dropped_noise</code> ，留痕两件 = <code>error</code> 事件 + <code>stitch.failures</code> 计数（不写 <code>item.errors</code> ——S26 同则；无 <code>_meta</code> 腿， <code>degraded</code> 键保持 <code>segment</code> 专属，3.16.6）； <code>"fail"</code> 时仅 <code>episode</code> 候选信封 <code>failed</code> → <code>rejects</code> （救援候选不适用 <code>fail</code> 路径，3.16.6）。
<code>classification_invalid</code>	记录级	v1.7：M13 分类输出经 M8 修复耗尽—— <code>cclassify.on_error="fallback"</code> （默认）时归兜底类并留痕于 <code>Classification.detail</code> （不入 <code>rejects</code> ，记录存活）； <code>"fail"</code> 时记录 <code>failed</code> → <code>rejects</code> (3.13.4 失败与兜底行）。
<code>extraction_invalid</code>	转移级	v1.8：M15 单转移动作摘取经 M8 修复耗尽—— <code>extract.on_error="fallback"</code> （默认）时该步记 <code>action_type="other"</code> 并留痕于 <code>Transition.detail = {kind, message}</code> （ <code>episode</code> 存活，不写 <code>item.errors</code> ，S16）； <code>"fail"</code> 时 <code>episode</code> <code>failed</code> → <code>rejects</code> (3.15)。

judgment_invalid	比较级	M4；按 tie 计入 BT (3.4.3)。
schema_violation	记录级	M8 L3 耗尽；记录 failed → rejects。
callback_violation	记录级	v1.5: M8 L3 耗尽且剩余违规全部来自 output.validator 回调 (3.8.2 L2.5)；记录 failed → rejects。
context_overflow	记录级	v1.11: 三形态 (V24) ——precheck = M9 终检命中 (V16) / 最小单元不装 (V10)；reactive-400 = 预算开启下嗅探命中且 V20 降级耗尽；reactive-200 = model_context_window_exceeded 且降级耗尽 (或无降级面)。处置：记录级 failed → rejects；计 report.budget.overflow_records。熔断矩阵：precheck 不计连击 (无 provider 交互)；reactive-400 终局计入连击 (A7 已裁——由属主算子经 ctx.metrics.record_provider_result(fatal=True) 补喂恰一次，M9 抛出时不喂；成功降级/任何成功调用清零连击)；reactive-200 终局不补喂 (HTTP 交互成功、streak 已被该次 ok 清零——llm.call 事件维持 status="ok"，实现者不得"修正"为 fatal, F9)。均不烧常规重试 (V20 降级重试独立计数、有界)。
output_truncated	记录级	v1.11: 响应以输出上限截断收尾 (finish_reason=length / stop_reason="max_tokens", V11) ——输入合窗、输出写满 max_output_tokens。处置：记录级 failed → rejects 归因独立成桶；不喂熔断 (交互成功，llm.call 维持 ok)；不进 L1-L3 修复循环。
provider_retryable_exhausted	记录级	M9 重试耗尽 (v1.6 含驻留超限 run.max_park_s, 3.9.3 密钥池行)；记录 failed，计入熔断窗口。
provider_fatal	运行级	M9 不可重试错误。400/404 等计入连续熔断计数，连续达阈值 ⇒ 退出码 4；401/403 认证类立即熔断 (v1.5, 3.9.3)。v1.6 密钥池：认证失败先按密钥禁用 (llm.key_disabled)，池内尚有存活密钥时不产生本错误、不计入熔断——仅当禁用的是该 profile 最后一把存活密钥时才抛出并立即熔断 (3.9.3 密钥池行)。
internal_error	记录级	任何未预期异常 (含 M11 终检失败)；记录 failed，堆栈入日志 (debug 级)。

v1.11 两注：① M8 L3 修复调用内的 precheck 溢出不落本词表 (V25①：该轮记修复失败并短路至耗尽，reject 归因维持 schema_violation / callback_violation, 3.8.2)；② z.ai 扩展终止值 sensitive / network_error 及未知值不做专项处置 (V11③——沿现行管线流转，垃圾输出由 M8 校验兜住)。v1.12 注：帧粒度零新错误 kind——帧分类失败落 fallback_class (不产生 StageError 入信封)、帧标注失败复用既有词表分类 (schema_violation / provider_* / image_decode_error / context_overflow / output_truncated / internal_error) 仅用于 WARN 日志与计数归因，成员失败不写 item.errors、不入 rejects (3.5.5/3.13.7；帧 Schema 走内部 Schema 待遇、无 L2.5 ⇒ callback_violation 在帧路径不可达)；熔断矩阵零改动——帧调用的 precheck/最小单元不喂连击、reactive-400 终局由属主算子补喂恰一次 (A7 同则)。v1.13 注：时间流生成同样零新错误 kind——蓝图/帧实现/噪音批三类调用的失败不产生记录级 StageError (此时尚无记录：整条序列作废、直接缺席)，失败按既有词表分类仅用于值-free 的 stderr WARN 与 report.generate.stream.{plan_failures, realize_failures} 计数归因；熔断矩阵零改动——两类调用的 precheck / 不可装填不喂连击 (V10 先例)，reactive-400 终局在作废吞点经共享 budget.feed_reactive_terminal 补喂恰一次 (A7 同则，3.6.5 预算与溢出纪律)。

7.7 进度显示与结束摘要 (v1.10: 三态 console)

进度显示不属于日志：直接写 stderr 而不经 logging 模块、无级别概念、不产生 trace 事件。v1.10 将本面重写为三态 console (设计裁决 U1-U27 见 docs/dev/SPEC-tui-console.md；工业调研 [C-1]-[C-20] 见 docs/dev/PROPOSAL-tui-console.md，审计增量 [C-21]-[C-42] 见 SPEC §6)。

三态与判定 (console.mode, 5.1; CLI --console 覆盖)：

```
auto → rich 当且仅当: stderr.isatty() ∧ tool.log_format == "text"
                        ∧ TERM 非 "dumb"/空 ∧ rich 可导入 (M1 以 find_spec 探测, CLI 层懒 import)
其余一律 plain。NO_COLOR 不参与判定 (U25) —rich 档下由 rich 原生剥色保布局 (no-color.org
语义 = 禁颜色而非禁布局; BuildKit/uv/cargo 同实践)。判定产物由 M1 于 load() 收尾冻结为
ConsoleConfig.mode_resolved ∈ {"rich", "plain"} (U21—emitter 让位的静态门)。
显式 --console rich 尊重显式档 (CI 录 ANSI 场景);
tool.log_format = "jsonl" 强制 plain 且不可被显式 rich (CLI 或 config) 覆盖
(stderr 逐行可 json.loads 铁律; 显式冲突 M1 WARN 一次)。
```

plain 档 (回归锚)：与 v1.9 行为等价 (console.heartbeat_s = 0 默认下；判据 = 三层回归锚，7.8 console 行/U24) ——TTY 单行 \r 批级进度 (批号 + 五状态固定键集，T16 键集约束在本档继续成立：stitched 不入行)；非 TTY 每批一行摘要 (即 batch.end 的 stderr info 行)；运行

结束打印与 report.json counts 完全一致的文本版终版摘要表。行格式的单一事实源为 common 层纯函数 console_format (U21——emitter 与渲染器共用，字符串逐字节钉死于黄金快照)。两条锚串 (2026-08-14 重冻结到英文文案上——键集、行结构与信息集不变，仅语言变)：

```
\rlabelkit: batch {batch_no} emitted={emitted_total} dropped_dup=N dropped_lowq=N dropped_verify=N failed=N
— final summary (matches report.counts item by item) —
```

可选心跳： console.heartbeat_s > 0 且非 TTY 时每 N 秒一行数据无关汇总 heartbeat batch= stage= llm_calls= elapsed= (固定 deadline 自续不漂移；CI 长批「像死机」缓解，默认关；所有权 = CLI 层 plain 监听器， run.start 起 run.end 止)。

rich 档 (双区内联实时面板)：日志在上方滚动区照常输出 (行文本与 plain 逐字节一致——渲染器接管 labelkit logger 的 handler 流 (setStream 返回值记账)，退出/降级时恢复)，终端底部画布按 console.refresh_hz 节流原地重绘 (渲染 tick = asyncio task，Live(auto_refresh=false) 钉死——rich 默认自刷新线程与事件循环内动态字典有跨线程争用，U26；除 rich 自身外零新线程)；永不进入 alternate screen (批任务保 scrollback, U1)。画布六区块 (数据源全部为既有结构字段，唯一新增采集点 = M9 的 p50 延迟有界样本窗)：

区块	内容	数据源
标头	run_id、mode/modality (stream/stitch 徽标)、seed、耗时、ETA (仅批总数分母可得时显示，EMA 外推标 ~)	on_run_context(cfg,...) (3.12.3)；run_id 自 run.start
批进度	UI 模态 batch i/N + scanned (live 预扫复用：M10 既有 P2-4 预扫在 UI 模态翻 estimate=True，配对表零额外 I/O、stream 批数 = next-fit 仿真精确，禁二次 scan——U17)；文本模态默认 batch i 无分母——行数估算需全量多读一遍输入，默认不做 (M10 现状)，console.estimate = true 显式换购 (U17)	batch.start/end、on_estimate (estimate_run 导出，3.10.3)
段棋盘	仅启用 stage 按链序；✓ 本批已过 / ► 进行中 / · 待走 (三符号反映当前批位置)。► 后缀进度数：分子 = 运行级累计 llm.call 完成数按括号归属记账 (批内阶段串行屏障下，落在本 stage on_stage 与下一次 on_stage 之间的调用记入本 stage——U20)；分母 = on_estimate 送达的运行级 estimate_run 各 *_calls 估算 (标「估算」；未送达估算——文本模态未开 console.estimate ——则仅显示分子计数)。v1.12 分母口径：_STAGE_CALL_KEYS 改为可多键求和的分母映射——classify 分母 = classify_calls + frame_classify_calls、annotate 分母 = annotate_calls + frame_annotate_calls (帧调用的 llm.call 括号归属落在同名 stage 内，分子分母同栏对齐)；_ESTIMATE_CALL_KEYS 加两键 (rich 估算表与 dry-run 键表等价性)，面板零新行	stage_begin 旁路 (3.12.3) + llm.call 括号归集 + on_estimate
状态账	九态计数，stream/stitch 键仅启用时在场、同 report.counts 口径——stitched/threads 的展示为对 T16 的有界修订 (仅本档；对齐决策 1.6 U18)；emitted 分量 = batch.end.active (post-emit 恒等)；批内随批未更新	batch.end + counters 拉取
LLM	每 profile：在途/并发上限 (在途 = Σ 密钥 in_flight，口径 = 在线 HTTP 请求数)、calls、retries、tokens $\uparrow\downarrow$ 、成本 (未配价目 -)、p50 延迟 (成功调用口径，有界窗 256)；密钥池行 (环境变量名 + ok/冷却剩余秒/禁用)；熔断 streak/threshold，打开时红色横幅	LLMClient.snapshot() 每 tick 只读拉取 (3.9.2/3.12.3)
键位提示 / 中断态	交互键提示一行 (下表)；SIGINT 后画布顶部横幅 graceful interrupt in progress ($\leq 30s$)...	3.10.3 中断路径旁路转发

面板行的标签文案自 2026-08-14 起统一英文 (键集、区块划分、数据源与布局不变)：抬头 labelkit run · {run_id} · {badge} · seed {n} · elapsed {mm:ss}、批进度 batch {i}/{N} ... records {n}/{N} (scanned)、段棋盘 stages ...、状态账 account emitted ... dup ... lowq ... verify ... failed ...[noise ... absorbed ...][stitched ... threads ...]、LLM 行 {profile} in_flight a/b calls n retries n tok $\uparrow\downarrow$ {cost} p50 {t} + keys {env} ok|cooldown Ns|disabled + breaker {streak}/{threshold}、熔断横幅 Δ breaker open、错误条 errors ... (空为 (none))、终版面板 labelkit run done · {run_id} · {badge} · elapsed {mm:ss} + counts (= report.counts) + stage time (approx: summed on_stage intervals, not report timings)。

generate_only：批进度区退化为 generate ► calls i/N · produced n (calls 实时自 llm.call；produced 于生成相位结束一次更新——它是相位未计量)，批棋盘自再流批次起激活。运行结束：最后一次重绘后定格为静态终版面板 (counts 表 + per-stage 耗时横条 + llm_usage 小表 + rejects/trace 路径行)，scrollback 保留完整日志。

run 之外的面 (U13)：rich 档下 validate --probe 结果表 (仅当 stdout 为 TTY 时渲染，否则保持现行行式——脚本消费不破坏) 与 dry-run 估算摘要以表格呈现 (数值与 plain 逐项一致；plain 档行式输出为逐字节锚——含 v1.8/v1.9 无条件打印的 segment_calls / stitch_calls 行)；rubric --show 恒 plain (stdout 机器消费，不受 console.mode 影响)。

rich 档键盘开关 (一期实施，对齐决策 1.6 U15；生效合取：rich $\wedge$ stdin TTY $\wedge$ console.interactive = true $\wedge$ termios 可用，否则纯渲染；封闭键集，未列键忽略)：

键	行为
? / h	键位帮助展开/收起
l	LLM 面板展开 (每密钥一行：env 名、状态、calls、rate_limited) / 收起
e	最近错误条开/关 (环形最近 5 条 error 事件的 stage + kind——7.6 封闭词表，数据无关)

+ / -	画布行数上限增/减 (4–16)
p	暂停/恢复画布重绘 (日志照常滚动; 调试/复制友好)
q	面板脱离: 余下运行降级 plain (不终止运行、不影响退出码)

两行提示的逐字文案 (2026-08-14 英文化):

```
[?]help [l]LLM expand [e]errors [p]pause [q]detach
keymap ?/h help  l LLM expand (one line per key)  e recent errors  +/- canvas lines(4-16)  p pause repaint  q detach (res
t of the run degrades to plain)
```

终端状态纪律: `tty.setcbreak` (非全 raw——保留 ISIG, **Ctrl-C** 产生 SIGINT 的语义不变, 仍走 3.10.3 优雅中断); 退出/降级/异常路径经 `finally` 恢复 `termios` 属性; 键盘轮询在渲染 `tick` 内非阻塞 `select`, 零新线程。stdin 被占用的代价 (粘贴的后续命令被吞) 以 `console.interactive = false` 回避。

信息纪律与失败语义 (红线, U6/U7): 面板与心跳行 = `stderr` 镜像同级——只显示计数、枚举、profile 名、密钥环境变量名、file:line 结构字段; 不显示 `record id`、`excerpt`、`reason/task_name/critiques` 等任何 LLM 自由文本与输入数据内容。渲染期任何异常自吞 + 一次性 `WARN` + 当场降级 `plain` 续跑, 渲染永不影响退出码与数据产出; 终端宽 < 60 列退化为单行 `\r` 形态。面板为 M12 的第四个纯消费面 (3.12.3 `ProgressListener` 旁路): 零 7.2 事件目录改动、`report.json` 零新键。

7.8 测试要求 (验收级)

层	要求
单元	L1 确定性修复穷举样例集 (围栏/截断/单引号/尾逗号/前后缀噪声); BT 拟合对已知解析解 (两元素、全胜、tie-only) 误差 < 1e-4; 配对采样在固定 seed 下逐字节可复现; UI 配对表 (图 3-1) 全分支: 冲突/缺对/跨目录/前导零。
集成	以 mock provider (录制响应) 跑通 2.3.1 全部组合矩阵; 断言 <code>report</code> 不变量、主输出全行过用户 Schema、 <code>rejects=refs</code> 时文件不含任何输入内容子串。
契约	对真实 API 的 probe 冒烟 (CI 可选); 结构化输出 L0 关闭/开启两态等价性 (最终输出结构一致)。
日志 (v1.1 新增)	<code>trace</code> 每行可被 <code>json.loads</code> 解析且恰含 <code>ts / run_id / batch_no / stage / ev / record_ids / payload</code> 七字段; 对 7.2 事件目录逐事件断言 <code>payload</code> 字段齐全; 首行为 <code>run.start</code> 且携带 <code>trace_schema_version=1</code> ; <code>trace.content="refs"</code> 时 <code>trace</code> 文件不含任何输入内容子串 (与 <code>rejects=refs</code> 同法断言); 注入写失败 (不可写路径 / mock <code>OSError</code>) 断言运行不中断、 <code>warn</code> 恰打印一次、 <code>report.trace.dropped_events</code> 计数正确。
console (v1.10 新增)	定宽渲染快照断言 (<code>Console(width=100, force_terminal=True)</code>), 喂 <code>MetricsSink</code> 计数器状态而非 LLM 响应——不违反真实 LLM 测试纪律): 九态账 / 密钥池三态 / 熔断横幅 / 中断横幅 / 窄终端退化 / <code>generate_only</code> 形态 / <code>l · e</code> 展开态; 回归锚 (三层, U24——实跑逐字节 diff 经 <code>refute</code> 审计证伪: 行携时间戳与 <code>duration_ms</code> 、温度 0 端点非逐字节确定): ① 单元层——固定时钟注入下 <code>plain</code> 行格式纯函数 (<code>console_format</code>) 黄金快照逐字节断言; ② <code>dry-run</code> 层——examples 五目录八工程 (v1.13 起: <code>text</code> 两工程 / <code>ui</code> / <code>stream</code> 两工程 / <code>mix</code> 两工程 / <code>synth-stream</code> ; v1.12 为四目录七工程) <code>--dry-run --console plain</code> 的 <code>stderr</code> 估算行 (无时间戳字段) 逐字节 diff 为空 (八个黄金文件入库、离线套件逐运行钉死—— <code>dryrun-mix.txt</code> / <code>dryrun-mix-text.txt</code> 为 v1.12 新增的 <code>mix</code> 主/姊妹双工程 <code>golden</code> , <code>dryrun-synth-stream.txt</code> 为 v1.13 新增的时间流生成工程 <code>golden</code> ; v1.13 的七个既有 <code>golden</code> 字节不动——估算行格式零改动, 三类生成调用全折入 <code>generate_calls</code> , 3.10.3 时间流生成行; 2026-08-14 代码规则整改把八个黄金文件整体重冻结到英文估算行与英文注记行上——键、键序、数值与行序全部不变, 仅文案语言变, 此后八文件继续逐字节钉死); ③ 实跑层——八工程实跑 <code>stderr</code> 归一化 (时间戳/耗时/token/延迟/计数值 → 占位符, cargo 测试套 [ELAPSED] 遮蔽同法) 后与基线结构等价 (行序与固定文案逐字节一致); <code>log_format="jsonl"</code> 下 <code>stderr</code> 逐行可解析且显式 <code>rich</code> 被拒并 <code>WARN</code> ; 降级注入: 渲染 <code>tick</code> 抛异常 → 运行照常完成、退出码不变、恰一条 <code>WARN</code> 、自动转 <code>plain</code> (<code>rich</code> 档下 <code>emitter</code> 已静态让位——余下 <code>plain</code> 进度/摘要行由渲染器以 <code>console_format</code> 续打, <code>q</code> 脱离同路径); 键盘: 伪 TTY 注入键序断言 <code>q</code> 脱离 / <code>p</code> 暂停期日志照常 / <code>termios</code> 属性退出后逐字节复原 / <code>cbreak</code> 下 <code>Ctrl-C</code> 仍触发 SIGINT 优雅中断; 协议: <code>listener=None</code> 路径零行为变化、全回调 <code>O(1)</code> 无 I/O。

8. 非目标、设计假设与演进路线

8.1 非目标

见 2.1.2 工具级负边界；另注：不承诺跨批分数可比性（pairwise 为批内相对分，3.4.3）；不做多机分布式（单机并发已被 API 限速主导）。

v1.9（线索缝合域）明确不做六条：① **真并发**（同屏双任务/分屏）——单前台屏无真并发 [65]，帧单一归属是手术/归因/守恒的公共地基（帧多标签先例 [72] 经评估否决）；② **跨会话/跨运行缝合**——线索作用域不跨 session、不跨 batch（3.16.4），无持久化红线不变（2.6）；③ **帧级区间树与子任务嵌套校验**——episode 内子任务跨度经需求方 2026-07-16 裁决不做引擎特性（标注层模式：用户 Schema 自声明 `subtasks: [{label, step_range}]`，工具不校验其语义）；④ **自动拆线手术**——verify 对错缝只标 `wrong_stitch` 不重构（3.7.3）；⑤ **在线/增量处理**——批处理工具定位不变；⑥ **完成感知封闭**（收尾动作模式触发线索封闭）——链序上 `extract` 后置，缝合运行时无动作证据、「机械收尾动作模式」不可判而撤除（3.16.4 ①）；若未来「`extract` 先行」次序演进（8.4 M14 行候选）使动作证据先行，可重评（8.4 M16 行演进候选）。

v1.10（console 域）明确不做三条：① **web/hosted viewer**——数据只去配置声明的 LLM 端点、无遥测红线（2.6）；② **面板内数据内容检视**——`trace excerpt / full` 档职责（7.4；面板信息纪律 U6/U22 红线，7.7）；③ **跨运行历史面板/持久化仪表**——无状态原则（2.6）。

v1.13（时间流生成域）明确不做九条（均列 8.4 演进候选或维持既有归属）：① **重放评测回路**——工件可自行当输入重放（3.6.5 / 6.5），但「自动重放并与 `truth` 比对出分」是独立工具的职责（2.1.2 ⑧）；② **UI 模态时间流生成**——本形态恒 `text` 模态（截图无从合成，8.3 O3 维持该辖区）；③ **乱序与缺失两类噪音**——乱序与 M2 `on_disorder` 自相矛盾、缺失对合成无意义（只做插入与重复两类，3.6.5）；④ **交叉并发度 $k > 2$** ——`sessions` 定容下恒 $k \in \{1,2\}$ ；⑤ **蓝图低温分档**——蓝图与帧实现同温（`temperature` 单键）；⑥ **一批多蓝图装箱**——序列一次蓝图调用；⑦ **合成序列上开放** `[frame.annotate]`——帧内容即生成物，再标注一遍无信息增益（与本形态互斥，3.1.4）；⑧ **generate 专属 trace 通道**——两类调用经 `llm.call` 可见（7.2 零增量声明）；⑨ **输出精确定量与补齐回路**——`sequences` 是尝试配额（8.3 O6 维持）。

v1.14（帧类构成档位与时间字段回填域）明确不做十条（均列 8.4 M6 行演进候选或维持既有归属）：① **平面生成形态的档位**——档位附着于蓝图的帧类构成，平面生成无蓝图无帧类，档位无处附着；② **按档绑定 `llm profile`**——会扰动冻结的（`llm, style`）预抽流与密钥引用集语义（3.6.5 抽签消费顺序表）；③ **按档覆盖 `len_range / temperature`**——档位只表达构成，不表达长度与采样参数；④ **档间序约束与 `tier_rank` 的工具侧质量语义**——不校验「高档构成 $\supset$ 低档构成」之类的子集链，也不赋予序数高低任何质量方向（方向归用户，裁决 `tier_rank` 即档位身份）；⑤ **宽松构成「至多这些类」**——现版恒等语义（去掉 `contains` 即得宽松形态），但宽松形态会使高低档的低配产物不可区分、档位身份不可从数据反推（等真实工程需求再裁）；⑥ **自由文本内的时间表述对齐**——把已铺时间语境注入提示词、让 LLM 在句子里说对时间，须改动冻结的抽签消费顺序表（时间轴须先于内容生成铺设），另裁；⑦ **序列级与标注 Schema 侧的时间字段绑定**——标注是抽取产物、非生成侧辖区；⑧ **`frame_gap_s` 的间隔分布形扩展**——现版恒为均匀采样，分布族（BIMP 的对数正态/伽马族 [106]、AT-KDE 的经验分布 [108]）与本版正交；⑨ **语义词表扩词**——四值是冻结闭集，扩词走 `spec` 修订；⑩ **绑定字段数值约束关键字的运行期强制**——现版只保证类型层（`ts` $\Rightarrow$ string、其余 $\Rightarrow$ number）并对多余关键字发 WARN，时间量的值域由时间轴决定（裁决·绑定即剔除）。

v1.15（按类档位表域）明确不做三条（均列 8.4 M6 行演进候选）：① **按类权重速记糖**——形如 `tier_weights = [1, 2]` 的「只改权重、构成沿用全局表」缩写形态：它本质是行级合并的变体，与裁决·表级原子覆盖（本版的地基）冲突；② **跨类 `rank` 对齐声明**——不提供「各类 `rank` 语义须一致」的声明或校验，`tier_rank` 是类内身份、跨类无任何工具语义（v1.14「工具不赋序数以质量方向」的自然延伸）；③ **全局表 $\times$ 类过滤**——形如 `tier_ranks = [1, 3]` 的「从全局表挑子集」形态：与整表覆盖并存会产生第三种合成语义，本版只保留一种。v1.14 的十条非目标照旧停放——其中②（按档绑 `llm profile`）与③（按档覆盖 `len_range / temperature`）在按类面下同样不做：按类覆盖的辖区只是档位表本身（构成与权重），不外扩到序列级的 `profile` 与采样参数。

v1.12（帧粒度域）明确不做七条：① **帧级 `quality/dedup/verify/generate`**——帧粒度仅分类与标注两面（成员帧的治理由 segment 噪声剔除与 episode 级质量门承担）；② **帧多标签与帧级扇出**——帧单一归属是手术/归因/守恒的公共地基（`[frame.classify]` 无 `assignment`，显式书写定向 `CONFIG_ERROR`，3.1.4）；③ **帧级 L2.5 回调**——`output.validator` 仅约束序列级用户 Schema 调用（演进候选 `frame.annotate.validator`，8.4 M5 行）；④ **帧级 `self_consistency`**——成本 $\propto n$ 且投票键须取自帧 Schema 需动投票主干（`[frame.annotate]` 无该键，显式书写定向 `CONFIG_ERROR`）；⑤ **摘要行帧标签回填**——演进候选，爆炸半径已勘明（8.4 M13 行）；⑥ **同内容帧标注备忘录**——演进候选（8.4 M5 行）；⑦ **按序列类分叉的帧类表**——帧类表全局一份，与序列类表相互独立、允许重名、互不约束（5.2）。

v1.16（序列规则与联合规划域）明确不做以下范围：

- 不合成缺失帧、插入中断或模拟丢帧后的业务恢复；`planner` 只冻结完整 `attempt` 的帧类、`owner`、时间轴和 `noise` 槽，LLM 作废后做确定性投影。
- 不提供跨业务的全流顺序声明、跨序列类的 `tier_rank` 对齐语义，也不把有限迹规则扩展成 Allen interval algebra 或可配置的任意区间关系。
- 不提供运行期 fallback、改抽长度、重求 `skeleton`、追加调用补齐 `noise` 或跨运行缓存；`sequences` 仍是尝试配额，实际 `survivor` 可以更少。
- 不把规则、窗口、`correlation`、`planner witness` 或用户 `hook` 输出写进 `truth`、`artifact`、`report` 以外的内容面；不新增 `generate` 专属 `trace` 通道。
- 不放开 UI 模态时间流生成，亦不自动执行 `artifact` 重放并与 `truth` 评分；两项仍归独立工具或后续产品议题。

8.2 设计假设（若不成立需回到设计层）

#	假设	若不成立的影响
A1	UI 树导出为 6.2 可映射的 JSONL（平铺节点行或单行嵌套树）。	需在 M2 增加导出格式适配器（新增子模块，不影响其他模块）。
A2	一个 uitree 文件 = 一屏（与一张截图对应）。	若一文件含多屏序列，需引入「屏内行号」扩展 index 语义（影响 3.2.4 与 6.2）。
A3	所配 VLM 支持单请求多图（pairwise 比较需两组截图）。	不支持时 M4 在 UI 模式自动改用 criteria_per_call="single" + 两图拼接（Pillow 纵向拼接）——已在 M4 留有实现开关，默认不启用。
A4	单次运行 ≤ 50 万条（2.6 内存模型）。	超出需 dedup.scope="batch" 或分目录多次运行。

8.3 开放问题（后续版本议题）

#	议题	现状与触发条件
O1	语义去重（SemDeDup [26]，需 Embedding API）	已于 v1.2 落地为可选第④级（3.3.3， dedup.semantic ），本行保留作决策溯源；默认仍关闭（ dedup.semantic = false ），零 embedding 依赖的默认行为与 v1.0 一致。
O2	跨批可比的 pairwise 分数（锚点样本法：每批混入固定锚点记录参与比较）	QuRating 原文以全局训练分类器回避该问题；运行时替代方案需实验验证后再纳入规格。
O3	UI 模式生成（以现有截图为底、仅生成指令/任务侧文本，AgentTrek 式轨迹合成 [15]）	本版生成仅文本模式；有明确需求后单独立项。v1.8 注记：stream × generate 互斥（ segment.enabled 要求 generate.enabled=false ， 2.3.1）——序列/轨迹合成仍不并入本议题的现版范围；届时须先裁决与 stream 模式的组合语义（互斥放开或串接）。v1.13 部分核销：上句遗留的「互斥放开或串接」之问已由 v1.13 给出答卷—— 第三形态·直装 （时间流生成，3.6.5）：既不放开互斥、也不串接，而是让 M6 直接产出序列信封（流是生成出来的，无须再分段），互斥条文字面维持不动。本议题的存续部分收窄为 UI 模式：文本时序流合成已落地，截图侧合成（AgentTrek 式 [15]）仍待立项——触发条件不变。
O4	断点续跑	与「不存储中间态」冲突，明确排除；超大任务靠分目录运行缓解。
O5	labelkit analyze 子命令：读 trace.jsonl 产出标注质量分析 / rubric 诊断报告（自动计算 7.5 诊断指标、reason 关键词聚类）	本版仅提供 jq 级手工分析（7.5）；trace 事件契约（7.2）稳定运行一个版本后立项。v1.10 注记（U16）：全屏交互 trace 浏览器品类与 textual 渲染库于本议题立项时一并重估（console 面板经验可迁移， docs/dev/SPEC-tui-console.md §5）。
O6	全局精确定量与生成补齐回路（ output.target_count ，输出恰好 N 条）	设计草案：两阶段流水线——第一阶段全量接入 + 去重 + 打分并缓存分数；第二阶段全局 top-K 选出恰好 target_count 条，再仅对选中集执行标注与输出。前提：分数具全局可比性——pointwise 绝对刻度，或 pairwise 经 O2 锚点法校准后方可全局排序。配生成补齐回路：target 未达时从高分种子生成 → 去重 → pointwise 质量门 → 计入，停止条件 = 达标 V max_backfill_rounds V 本轮合格率 < 下限（生成器饱和时合格率持续衰减，即递归自生成数据的 model collapse 现象 [36]，故合格率下限停止条件必不可少）。现状：2026-07-02 评审对齐为演进路线——本版以 quality.selection = "top_ratio" 提供流式批内近似定量（3.4.3），不承诺全局恰好 N 条。触发条件：出现「必须恰好 N 条」的下游需求。v1.7 注记：generate_only 的按类生成配比（每类 standalone_count）经对齐归本议题——属量目标语义，与全局定量一并立项（1.6 v1.7 对齐决策 ③）；v1.7 的按类参数仅为加工条件化（2.1.2 ⑥）。v1.13 窄化注记：时间流生成的按类 sequences（3.6.5）已落地按类量目标的「尝试配额」半边——按类可各自声明尝试条数、类段字典序展开、--limit 在配额层截断；本议题的存续辖区据此收窄为输出侧的精确定量（「每类输出恰好 N 条」）与补齐回路（未达标时续生成的停止条件族）——尝试配额无输出条数保证：蓝图/实现失败、逐帧钩子违规、序列相似度过滤与下游质量门/评审淘汰都会使实际产出低于配额，且本版不补齐（裁决·量目标辖区）。
O7	多 API Key 负载均衡（单 profile 密钥池：最少在途轮换 / 每密钥 429 冷却 / 认证按密钥禁用 / 全池冷却有界驻留）	已于 v1.6 落地（3.9.3 密钥池行、5.1 api_key_envs 、5.2 run.max_park_s 、7.2 三事件、6.4 keys 子块），本行保留作决策溯源（对齐记录见 1.6，2026-07-03）。触发条件即 8.1 所注「单机并发已被 API 限速主导」——无人值守长跑被单密钥用量限额中断。单密钥配置在数据产出、重记账与熔断/退出语义上与 v1.5 一致（429 等待路径修订见 3.9.3 重试行）。业界同构：LiteLLM Router / 网关侧客户端多密钥轮换实践。端点镜像池（多 base_url）经评审明确排除：同 provider+model 的不同部署在 temperature=0 下仍有数值漂移（GPU kernel / batching 差异），会翻转 pairwise 裁决与语义去重边界判定、污染 7.5 同种子翻转率指标；如未来放开须先解决跨部署可比性。
O8	stitch.judges 多模型评审团扩展（缝合判定的跨家族多数决，镜像既有 verify.judges / quality.judges 模式作纯配置扩展）	v1.9 选型记录（1.6，2026-07-16 / T18）：本版采「单模型多次」（stitch.votes，self-consistency [33]）而非「多模型评审团」（PoLL [32]）——缝合的两类误差病理分工明确：漂移（方差病）→ votes 采样多数决；过连接（偏差病）→ 机械先验合取（3.16.4）。评审团修不了过连接：PIRA 消融显示过连接是跨家族共享偏差（GPT 系与 Gemini 系同向 trigger-happy [64]），异构裁判会把共享偏差投成多数 [86]，且实测评审团有效独立票仅 ≈2 [89]；当前部署纪律为单端点单模型。触发条件：第二模型家族进入部署面，且真机门禁审计显示漂移（而非过连接）是漏缝/错缝主体。

8.4 算子算法演进路线（robustness & diversity）

按模块汇总「现行算法 → v1.2 已收录的可选增强 → 演进候选」。v1.2 可选增强均默认关闭、不改变各模块默认行为（配置键规格见 5.1—5.2）；演进候选仅收录有顶刊论文或工业项目背书者，待触发条件出现后立项。

模块	现行算法（默认）	v1.2 已收录的可选增强	演进候选（背书 / 触发条件）
M3 去重	规范化 SHA-256 精确 + MinHash-LSH 近似 (3.3)；UI 模态加 pHash	SemDeDup 语义级可选第④级 [26]： dedup.semantic（默认 false），开启后经 dedup.semantic_embedding 引用的 [embedding.<name>] profile 取向量，余弦相似度 ≥ dedup.semantic_threshold（默认 0.95）判重	子串级精确去重（Lee et al. [3] 的 suffix-array 变体，捕获行内重复长片段）；MinHash 参数自适应（按批内文本长度分布自动调 ngram / num_perm）。触发：短文本上 MinHash 误杀/漏杀率超预期。
M4 打分	pairwise+BT 与 pointwise 双模式 (3.4)，threshold 过滤	多评审团 quality.judges（默认 []；奇数个 profile 多数票 [32]）；双顺序裁决 quality.both_orders（默认 false，开启后每对正反两序各裁决一次以对消位置偏差 [20]，细节见 3.4.3）；批内定量优选 quality.selection = "top_ratio"（3.4.3）	O2 锚点跨批校准（每批混入固定锚点记录）；rubric 自动挖掘（Critic [31]，约 30 对人工偏好即可挖出可解释准则）；评审漂移监测（固定校准集定期回归，比对裁决一致率）。触发：需要跨批可比分数，或 rubric 迭代频繁。
M5 标注	提示词组装单次标注 (3.5.2)；v1.12 增帧级逐帧标注 (3.5.5)	self-consistency 标注 annotate.self_consistency（默认 0=关；n≥3 且为奇数，以 annotate.sc_temperature = 0.7 采样 n 次、字段级多数票聚合 [33]）	best-of-n 拒绝采样（同一记录标注 n 条取评审 top-1，打分器思想同 FineWeb-Edu [11]）。触发：标注一致性仍不达标。 帧级 L2.5 回调 frame.annotate.validator （v1.12 候选）：镜像 output.validator 的帧级同胞——挂接帧 Schema 调用的 L2.5、违规回喂修复环；现版帧标注走内部 Schema 待遇、无 L2.5（3.8.2 路由声明）。触发：帧标注出现 Schema 表达不了的业务约束。 同内容帧标注备忘录 （v1.12 候选）：同一运行内容完全相同的成员帧（文本行 verbatim 重复 / 截图+树同签名）复用同一帧标注结果，省逐帧调用成本；与「无跨运行状态」红线（2.6）相容——备忘录仅进程内存。触发：帧标注成本成为瓶颈且重复帧占比可观测偏高。 按类 L2.5 回调 （v1.13 候选）：output.validator 现为全局唯一——按序列类标注 Schema 已可分叉（3.5.2），但回调仍是一份，类间业务约束不同时只能在回调内自行按 label 分支；候选 = [class.<name>.annotate].validator 的按类覆盖（白名单只增原则）。触发：按类 Schema 的使用工程出现 Schema 表达不了、且各类互不相同的业务约束。

M6 生成	Self-Instruct 式种子自举生成 [18] (3.6.2)； v1.13 增时间流形态 (3.6.5)：蓝图 → 帧实现两阶段合成 [99][100] + 零 LLM 机械交织 [101][102] (装箱/交叉/噪音/重发/ts) + 序列级相似度过滤 [104]	多 LLM 混合 <code>generate.llms</code> (数组, <code>generate.mixture = round_robin weighted</code>) + <code>[[generate.styles]]</code> 风格模板 (3.6.2； <code>persona</code> / 受众×风格分桶的多样性思想 [34][35])； <code>[[generate.stream]]</code> 时间流形态 (v1.13, 默认关——全关时与 v1.12 字节等价)； <code>[[generate.stream.tiers]]</code> 帧类构成档位表与 <code>[[frame.class.*.generate.time_fields]]</code> 时间字段绑定 (v1.14, 均默认不在场)； <code>[[class.<name>.generate.tiers]]</code> 按类档位表 (v1.15, 默认缺省——全缺省时与 v1.14 字节等价, 含报表)	Evol-Instruct 自动深化/扩展算子 [19] (对种子指令做复杂化与广度改写)。触发：生成多样性或难度分布不足。 v1.13 时间流形态的六个演进候选 ： 交叉并发度 $k > 2$ (现版 <code>sessions</code> 定容下恒 $k \in \{1, 2\}$ ；触发：需要三线程以上穿插的评测集)； 蓝图低温分档 (蓝图取低温保结构、实现取高温保多样；现版同温单键；触发：蓝图结构性错误率可观测偏高)； 一批多蓝图装箱 (一次调用产出多条序列的蓝图以摊薄调用成本；触发：蓝图调用占比成为成本瓶颈)； 乱序噪音 (需与 M2 <code>on_disorder</code> 的处置语义先行对齐——现版乱序帧会被摄取侧直接丢弃，注入即自相矛盾；触发：出现「容忍轻度乱序」的重放评测需求，与 M14 行的「有界乱序重排窗」候选配对落地)； 重放评测回路 (自动重放工件并与 <code>truth</code> 比对出分；现版为明确非目标 2.1.2 ⑧，宜作独立工具；触发：合成数据集需要回归门禁)； generate 专属 trace 通道 (现版两类调用经 <code>llm.call</code> 可见、零新通道 7.2；触发：需要逐序列复盘蓝图与实现的对应关系)。 v1.14 两机制的五个演进候选 ： 宽松构成「至多这些类」 (去掉蓝图 Schema 的逐类 <code>contains</code> 即得——现版恒等语义使档位身份可从 <code>members[]</code> 反推对账，宽松形态会让高低档的低配产物不可区分；触发：出现「档位只设上界、不要求全覆盖」的真实工程)； 按档绑定 <code>llm profile</code> / 档案 (现版档位只表达帧类构成， <code>profile</code> 与采样参数均属序列级——按档绑定会扰动冻结的 (<code>llm</code> , <code>style</code>) 预抽流与密钥引用集语义；触发：出现「不同档位用不同模型合成」的成本或质量诉求)； 自由文本内的时间语境对齐 (把已铺 <code>ts</code> 注入提示词、让 LLM 在句子里也说对时间——须先裁决抽签消费顺序表的改动 (时间轴须先于内容生成铺设)；触发：下游需要「文本表述与时间戳一致」的合成数据)； <code>frame_gap_s</code> 的间隔分布形扩展 (现版恒为均匀采样；分布族与经验分布是过程仿真侧的成熟做法——BIMP 的分布族参数 [106]、AT-KDE 的分段 KDE [108]；触发：合成流的时间分布形态成为下游评测的敏感变量)； 语义词表扩词 (现版四值冻结闭集 <code>{ts, gap_prev_s, gap_next_s, elapsed_s}</code> ，扩词走 <code>spec</code> 修订；候选词如「会话内相邻行间隔」「距会话首帧秒」等跨序列口径；触发：出现序内口径表达不了的真实时间语义)。 v1.15 按类档位表的三个演进候选 ： 按类权重速记糖 (<code>tier_weights = [1, 2]</code> 形态——只改权重、构成沿用全局表；它是行级合并的变体，与裁决·表级原子覆盖的地基冲突，故本版不做；触发：真实工程里大量类只想调权重而构成完全一致，整表复制的冗余成为维护负担)； 跨类 <code>rank</code> 对齐声明 (声明并校验「各类同 <code>rank</code> 语义一致」使跨类 <code>rank</code> 可比——现版 <code>rank</code> 是类内身份、跨类无工具语义；触发：下游出现需要跨类按档聚合的评测口径)； 全局表 × 类过滤 (<code>tier_ranks = [1, 3]</code> 从全局表挑子集——与整表覆盖并存会产生第三种合成语义，本版只保留一种；触发：出现「各类只是启用全局表的不同子集」的规模化配置形态)。
M7 校验	单 <code>judge</code> 独立评审 + 有界修复环 [20][21] (3.7)	多评审团 <code>verify.judges</code> (默认 <code>[]</code> ；奇数个 <code>profile</code> 多数票, <code>critiques</code> 合并并标注来源 <code>judge</code> [32], 3.7.2)	评审团分歧驱动的人工抽检队列 (多数票非全票一致的记录进抽检清单，人机对齐界面思想出自 EvalGen [30])。触发：评审团分歧率持续偏高。
M8 结构	L0—L3 四层防线：供应商结构化输出 + 确定性修复 + 有界 LLM 修复环 (3.8)	— (v1.2 无新增)	约束解码引擎本地化 (Outlines / XGrammar 类 <code>grammar</code> 引擎 [23] [24])：自托管推理时以解码期硬约束替代当前面向 API 场景的四层防线。触发：迁移至自托管/本地推理栈。

M13 分类 (v1.7)	LLM 封闭集分类：类别表词表经内部 Schema enum 硬校验，单/多标签可配，失败归兜底类 (3.13)	可选 self-consistency 投票 <code>classify.self_consistency</code> （默认 0=关；n≥3 奇数，single 多数票 / multi 逐标签投票 [33]，3.13.4)	embedding 粗分 + LLM 精分两级分类降本（粗分近邻筛选、LLM 只精判边界样本；触发：分类调用成本成为瓶颈）；开放集 tagging 仅打标不路由 (InsTag 形态 [38]，标签进 <code>_meta</code> 供多样性/复杂度分析；触发：需要超出静态类表的语义分析)；~~按类输出 Schema~~ (v1.13 已核销： <code>[class.<name>.annotate].schema_path / schema_inline</code> 落地为覆盖语义的按序列类标注 Schema——触发条件（单 Schema 的 oneOf 条件子模式表达不了按类差异）由 examples/synth-stream 的两个序列类兑现：购票抽行程要素、智能家居抽设备动作，字段集互不相同。M5 六消费点与 M11 写前终检统一按行取类有效 Schema (3.5.2/3.11.2)，M8 以显式待遇参数保住 L2.5 与 resolved_at 记账 (3.8.2)；帧级标注 Schema 仍按粒度唯一，5.2 白名单表尾行已同步改写)；逐类适用度打分档（0—5 分 + 阈值筛命中集合，替代集合判断；触发：需要可调的多标签灵敏度）；多标签仅打标不扇出档 (labels 全集进 <code>_meta</code>、仍按首标签走单条管线，assignment 枚举留扩展位；触发：出现「要全集标签、不要多行输出」的真实场景，1.6 v1.7 对齐决策 ⑥)；摘要行帧标签回填 (v1.12 候选，裁决·摘要行回填砍掉)：把帧级判决标签回填进成员摘要行 (frame_digest 输出)，使下游 quality/annotate/verify 的序列证据面携带帧类信号——v1.12 审计勘明真实爆炸半径 = quality/annotate/verify/extract 四处渲染点 + CONTRACTS §10 多个逐字节冻结模板（模板改动连带黄金快照与手册重采），收益不成比例而砍掉，帧标签仅经 <code>_meta.stream.members[]</code> 输出 (3.11.2)。触发：出现「下游序列判定必须消费帧类信号」的真实工程且愿承担模板面重冻结成本。
M14 分段 (v1.8)	gap/key/上限规则会话化 (M2 会话流视图，3.2.8) + 三步演绎滑窗裁决（双向上下文概括 → 五值封闭集关系分类 → 演绎查表映射边界/噪声；window=20、重叠 1 帧、确定性缝合，3.14）[47] [48]	~~ segment.use_vision ~~ (v1.11 移除——窗口附图改由 segment.llm 所指 profile 的 supports_vision 能力推导 (parse product vision_resolved，3.14/V1)：树贫瘠场景的表达面 = 选 profile 即选能力 [63]，纯文本裁决 = 把 segment.llm 指向纯文本 profile；存量显式键定向 CONFIG_ERROR, V2)；segment.context 可选域上下文（非边界定义，判据模板内置零配置可用)；segment.strategy="rules" 零 LLM 纯规则档	有界乱序重排窗 (k-帧滑窗重排，容忍采集端轻度乱序——现行为流式单调性校验 + stream.on_disorder；触发：真实输入出现轻度乱序，1.6 v1.8 决策 ⑬)；交错 episode (帧属并行任务而非噪声——RPA 交错例程难变体 [50]，需全局归属模型；触发：审计显示交错形态占噪声主体)；跨段边界仲裁（跨段搬帧修复，v1 只标记——代价是邻段级联重修与乒乓风险；触发：审计显示跨段形态占缺陷主体，S31)；extract-先行次序（先在会话内逐相邻对摘取、再在动作序列上一次分段——GUIDE / Watch & Learn / VideoAgentTrek / OpenCUA 的同行主流次序 [57][58][60][43]；以成本权衡裁决：dedup 前置节省 vs 分段证据质量，非环形依赖，S32；触发：帧摘要证据上的分段质量不达标)；嵌入变点检测 (Embed-KCPD [47]：training-free 核变点检测，仅文本基准验证、GUI 流无先例，需 embedding profile；触发：LLM 分段调用成本成为瓶颈)；k>1 窗口重叠（重叠多帧多数决缝合压接缝误判；触发：接缝帧误判率可观测偏高)。
M15 摘取 (v1.8)	相邻帧对 <s_i, s_{i+1}> LLM zero-shot 摘取（一请求 2 图 + OpenCUA 稳定帧锚定句 [42][43]）+ 树 diff 证据（结构键多重集匹配，代码侧确定性，3.15)；action_type 11 值词表 [45][62]	extract.include_diff（默认 true，可关做 A/B 消融——Sharingan 像素 diff 负结果、结构化树 diff 方向未定 [59]；extract.instruction 域提示（[class.<name>.extract] 可按类覆盖)	文本模态 extract（「转移摘要」弱语义档，v1 仅 UI 序列；触发：文本流工程出现真实需求，1.6 v1.8 决策 ⑦)；缺帧补全 (Repairing Event Logs [51] 先验：缺失事件修复依赖跨轨迹习得的过程模型，v1 仅标记 capture_gap；触发：跨语料过程先验可用)；本地 IDM profile（专训逆动力学模型替代 zero-shot——Watch & Learn 实测专训 91.7% vs zero-shot 70.5% [58]，是「不训练/托管本地模型」负边界 (2.1.2 ①) 的已记录机会成本；触发：extract 错误率成为下游质量主瓶颈且允许自托管推理栈)；完成度末帧图（quality 轨迹打分的 completion 维度附末帧单图，+1 图/episode——忠于 OS-Genesis TRM 原型的输入配置（含末三帧截图）[41]；触发：完成度维与人工判定的失配集中于视觉终态证据)。
M16 缝合 (v1.9)	单调选池 LLM 判定 × 机械先验合取（析取三腿 + stale-gap 降格，bias="conservative") + 有界二遍复评 + 短段救援 + 接缝机械占位 (3.16) [64][74][87]	stitch.votes（默认 1=关；≥3 奇数，verdict, thread_ref 严格多数决 [33]——置信度门槛的正规替代 [79]，3.16.4)；stitch.bias="llm" 纯 LLM 消融档；stitch.rescue_short / stitch.repass 双开关；stitch.stale_gap_steps 时间衰减（双职：先验降格 + 逐出优先腿 [66][81])	stitch.judges 多模型评审团 (O8 选型记录——现版拒绝理由与触发条件见 8.3；镜像 verify.judges 纯配置扩展)；完成感知封闭（收尾动作模式触发封闭——B-1 撤除因 extract 后置无动作证据 (8.1 ⑥)；触发：M14 行「extract-先行次序」候选落地使动作证据先行)；复评面扩展至多碎片线索（现版复评候选仅单碎片线索——双多碎片线索误分裂与池截取排除目标不在修复面内、由真机门禁兜底 (3.16.4 残差声明)；触发：门禁审计显示该形态占漏缝主体)。

上下文预 算 (v1.11)	<code>context_window</code> 声明制 (0=关) + 零依赖启发式估算 + 动态贪心装填 (条数参数降级为上限, <code>w_min</code> 静态护栏/估算上界) + 图片成本测量-反应式三层 (先验装填 → 溢出裁帧保清重试 → 判审裁帧升清重试 → <code>usage</code> 在线校准、批冻结快照) + M9 咽喉终检 (3.9, V6–V21)	<code>default_image_px</code> 图片采样工作点 (默认 0 = 沿用 <code>max_image_px</code> 即 v1.10 行为; <code>max_image_px</code> 升格为升级天花板 + 像素制硬限制域, V18); <code>context_window</code> 打折声明作通用 <code>margin</code> 放大器 (唯一逃生门—— <code>margin</code> / 估算系数 / 阶梯常数冻结于代码不开配置面, V7/V8/V18)	运行中分母修正 (<code>metrics.run_estimate</code> 重复调用通道 / <code>counters</code> 每 tick 拉取通道——V12 已证实机械可行、v1 不用; 触发: <code>w_min</code> 上界与 <code>report.stream.windows</code> 实际窗数长期偏差可观测); 定向区域升清 (裁剪可疑区域而非整帧升清——Ferret-UI 双子图 / DirectX VRS foveation / AwaRes·MEGA-GUI 判审触发裁片升清, [C-52][C-56][C-67] 见 <code>docs/dev/PROPOSAL-context-budget.md</code> ; 触发: 整帧升清仍不足以修复判审失败); per-profile 密度旋钮 (cl100k 旧词表中文 1.25–1.4 t/字不被 CJK×1.0 覆盖的记载局限; 触发: 该类 <code>profile</code> 部署出现且浪费/超窗可观测); 输出侧预算 (<code>num_per_call</code> × 样本长 vs <code>max_output_tokens</code> 的输出预算; 触发: <code>output_truncated</code> 桶占比可观测偏高); 计数 API / usage 文本密度校准回路 (智谱 tokenizer API 抽样校准 + LangChain <code>usage-scaling</code> 同构, [C-63][C-71]; 触发: 文本估算偏差成为主要浪费源——cl100k 缺口的将来闭合路径)。
--------------------------	---	--	---

v1.16 M6 现行算法冻结补充：规则/窗口生效时，M6 采用单一 CP-SAT 联合问题冻结 完整流 skeleton，再调用 LLM 生成 brief 与 payload；无约束时保留 v1.15 默认路径。规划 期为每个实际 attempt 恰抽一次循环长度偏好，联合模型以偏好名次和为主目标、可行 noise 为次目标，一次求解同时定稿长度、帧类 word、session、timestamp 与 noise 槽；不能逐候选 重求、按候选长度分别求解、把求解失败当作随机候选淘汰，也不能重规划或 fallback。planner、M1、estimate 共用问题入口。LLM 作废后的处理是删除 attempt、移除空 session、保留幸存 timestamp 并投影 noise/duplicate。验证顺序、hook 深拷贝、报告恒等式和 artifact 不变面 见 3.6.6、6.4、6.5。

9. 参考文献

- [1] Wettig, A., Gupta, A., Malik, S., Chen, D. QuRating: Selecting High-Quality Data for Training Language Models. ICML 2024. arXiv:2402.09739. 开源: github.com/princeton-nlp/QuRating
- [2] Bradley, R. A., Terry, M. E. Rank Analysis of Incomplete Block Designs: The Method of Paired Comparisons. Biometrika 39, 1952.
- [3] Lee, K. et al. Deduplicating Training Data Makes Language Models Better. ACL 2022. arXiv:2107.06499.
- [4] Chen, D. et al. Data-Juicer: A One-Stop Data Processing System for Large Language Models. SIGMOD 2024 (Industrial). arXiv:2309.02033.
- [5] Argilla. distilabel: Synthetic Data and AI Feedback Framework. 工业开源项目。 github.com/argilla-io/distilabel
- [6] Soldaini, L. et al. Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. ACL 2024. arXiv:2402.00159. 工具链: github.com/allenai/dolma
- [7] OpenAI. Structured Outputs. 平台能力文档 (工业)。 platform.openai.com/docs/guides/structured-outputs
- [8] Liu, J. instructor: Structured Outputs with Validation-Retry.; json-repair. 工业开源库。 github.com/jxn1/instructor / github.com/mangiucugna/json_repair
- [9] NVIDIA. NeMo Curator: Scalable Data Curation for LLMs. 工业开源项目。 github.com/NVIDIA/NeMo-Curator
- [10] Hunter, D. R. MM Algorithms for Generalized Bradley-Terry Models. Annals of Statistics 32(1), 2004.
- [11] Penedo, G. et al. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. NeurIPS 2024 Datasets & Benchmarks. arXiv:2406.17557.
- [12] Refuel AI. Autolabel: Label, Clean and Enrich Datasets with LLMs. 工业开源项目。 github.com/refuel-ai/autolabel
- [13] Baechler, G. et al. ScreenAI: A Vision-Language Model for UI and Infographics Understanding. IJCAI 2024. arXiv:2402.04615.
- [14] GUI-360°: A Comprehensive Dataset and Benchmark for Computer-Using Agents. arXiv:2511.04307. (LLM 驱动的 GUI 数据质量过滤管线)
- [15] Xu, Y. et al. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. ICLR 2025. arXiv:2412.09605.
- [16] Wu, Z. et al. OS-ATLAS: A Foundation Action Model for Generalist GUI Agents. ICLR 2025. arXiv:2410.23218.
- [17] You, K. et al. Ferret-UI: Grounded Mobile UI Understanding with Multimodal LLMs. ECCV 2024. arXiv:2404.05719.
- [18] Wang, Y. et al. Self-Instruct: Aligning Language Models with Self-Generated Instructions. ACL 2023. arXiv:2212.10560.
- [19] Xu, C. et al. WizardLM: Empowering Large Language Models to Follow Complex Instructions (Evol-Instruct). ICLR 2024. arXiv:2304.12244.
- [20] Zheng, L. et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets & Benchmarks. arXiv:2306.05685.
- [21] Madaan, A. et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023. arXiv:2303.17651.
- [22] Bai, Y. et al. Constitutional AI: Harmlessness from AI Feedback. Anthropic, 2022. arXiv:2212.08073.
- [23] Willard, B., Louf, R. Efficient Guided Generation for Large Language Models (Outlines). arXiv:2307.09702. 工业开源: github.com/dottxt-ai/outlines
- [24] Geng, S. et al. JSONSchemaBench: Evaluating Constrained Decoding with LLMs on Efficiency, Coverage and Quality. 2025. openreview.net/forum?id=FKOaJqKoio
- [25] Tam, Z. R. et al. Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of LLMs. EMNLP 2024 Industry. arXiv:2408.02442.
- [26] Abbas, A. et al. SemDeDup: Data-Efficient Learning at Web-Scale through Semantic Deduplication. 2023. arXiv:2303.09540.
- [27] OpenTelemetry Community. OpenTelemetry GenAI Semantic Conventions. (生成式 AI spans/metrics/events 语义约定; Status: Development, 实验性、非 stable) github.com/open-telemetry/semantic-conventions-genai (2026-07-02 访问)
- [28] LangChain. LangSmith: AI Agent & LLM Observability and Evals Platform. 工业平台。 docs.langchain.com/langsmith/observability (2026-07-02 访问)
- [29] Weights & Biases. W&B Weave: Observability and Evaluation Platform for LLM Applications. 工业平台。 docs.wandb.ai/weave (2026-07-02 访问)
- [30] Shankar, S., Zamfirescu-Pereira, J.D., Hartmann, B., Parameswaran, A. G., Arawjo, I. Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences. UIST 2024. arXiv:2404.12272.
- [31] Guo, H., Lv, K., Guo, Q. et al. CritiQ: Mining Data Quality Criteria from Human Preferences. ACL 2025 (Long Papers). arXiv:2502.19279.

- [32] Verga, P., Hofstätter, S., Althammer, S., Su, Y., Piktus, A., Arkhangorodsky, A., Xu, M., White, N., Lewis, P. Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models (PoLL). 2024. arXiv:2404.18796.
- [33] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E. H., Narang, S., Chowdhery, A., Zhou, D. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023. arXiv:2203.12171.
- [34] Ge, T., Chan, X., Wang, X., Yu, D., Mi, H., Yu, D. (Tencent AI Lab). Scaling Synthetic Data Creation with 1,000,000,000 Personas (Persona Hub). arXiv:2406.20094. (截至 2026-07 仍为 arXiv 预印本, 未正式发表)
- [35] Ben Allal, L., Lozhkov, A., van Strien, D. Cosmopedia: how to create large-scale synthetic data for pre-training Large Language Models. Hugging Face Blog (工业), 2024-03-20. huggingface.co/blog/cosmopedia; 数据集 huggingface.co/datasets/HuggingFaceTB/cosmopedia
- [36] Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., Gal, Y. AI models collapse when trained on recursively generated data. Nature 631 (8022), 755–759, 2024. DOI: 10.1038/s41586-024-07566-y. (2025 年有一则 Author Correction, 不影响主结论)
- [37] Su, D. et al. Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset. arXiv:2412.02595. (质量分类 → 分档路由不同合成管线, 产品化于 NeMo Curator [40])
- [38] Lu, K. et al. #InsTag: Instruction Tagging for Analyzing Supervised Fine-tuning of Large Language Models. ICLR 2024. arXiv:2308.07074. (LLM 指令语义打标 + 据标签驱动数据决策)
- [39] Lambert, N. et al. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124. (按核心技能分治的数据构造与 per-skill 合成管线)
- [40] NVIDIA. NeMo Curator: Distributed Data Classification (Domain / Quality / Multilingual 分类器作为管线阶段, 记录级标签字段驱动路由)。工业文档。docs.nvidia.com/nemo/curator (2026-07-07 访问; 工具链主引用见 [9], 此处为分类能力面)
- [41] Sun, Q. et al. OS-Genesis: Automating GUI Agent Trajectory Construction via Reverse Task Synthesis. ACL 2025. arXiv:2412.19723. (reverse task synthesis 三段式与 trajectory reward model; TRM 为 1–5 五级整数分制, 附录 A.3 的 0–5 六级为 LabelKit 家规改制)
- [42] Baker, B. et al. VPT: Video PreTraining — Learning to Act by Watching Unlabeled Online Videos. NeurIPS 2022. arXiv:2206.11795. (逆动力学模型 IDM 给无标签流打伪动作标签); GUI-Shift: K-step GUI Transition. ICLR 2026. arXiv:2505.12493. (IDM 信号在 GUI 域的自监督化, 范式最新形态)
- [43] Wang, X. et al. OpenCUA: Open Foundations for Computer-Use Agents. arXiv:2508.09123. (AgentNet 标注基础设施; Action Reduction 确定性归并 + State-Action Matching 稳定帧锚定)
- [44] Rawles, C. et al. AITW: Android in the Wild — A Large-Scale Dataset for Android Device Control. NeurIPS 2023 Datasets & Benchmarks. arXiv:2307.10088. (hindsight language relabeling: 先有流、后分段再打标)
- [45] Li, W. et al. AndroidControl: On the Effects of Data Scale on Computer Control Agents. NeurIPS 2024 Datasets & Benchmarks. arXiv:2406.03679. (episode/step 两级标注结构与 8 值动作词表; 动作数 = 截图数 - 1)
- [46] Lu, Q. et al. GUI-Odyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. ICCV 2025. arXiv:2406.08451. (跨 App episode 全集; 为跨 App 流引入 RECENT 应用切换动作; GPT-4o 步级语义补注)
- [47] Hearst, M. TextTiling: Segmenting Text into Multi-paragraph Subtopic Passages. Computational Linguistics 23(1), 1997; Gwon & Kim et al. Def-DTS: Deductive Reasoning for Open-domain Dialogue Topic Segmentation. Findings of ACL 2025. arXiv:2505.21033; Embed-KCPD: Kernel Change-Point Detection on Sentence Embeddings. 2026. arXiv:2601.18788. (话题分割谱系: 词汇内聚 → 嵌入变点 → LLM 三步演绎; Def-DTS 消融——半结构 (仅双向概括) 比裸问题差、完整三步最优、边界信号清晰场景裸判决可胜全套; 其按数据集集意图图池为词表按域定制先例)
- [48] Shou, M.Z. et al. Generic Event Boundary Detection: A Benchmark for Event Segmentation (GEBD). ICCV 2021. arXiv:2101.10511. (无词表事件边界的任务化: 5 人多评 + 一致性过滤协议下中等共识可达; "1 level deeper" 粒度锚定与 dominant subject 注意力锚定)
- [49] Lynch, C., Sermanet, P. Language Conditioned Imitation Learning over Unstructured Data (Hindsight Instruction Pairing). RSS 2021. arXiv:2005.07648. (描述后置范式源头: 先有无结构行为流, 事后配「哪条指令使该轨迹最优」)
- [50] Marrella, A. et al. Automated Segmentation of UI Logs (RPA 专著章节, 2020); Leno, V. et al. Discovering Candidate Routines from Unsegmented UI Logs. 2020. arXiv:2008.05782. (无监督 UI 日志例程发现; 显式处理不属任何例程的噪声事件; 交错例程难变体)
- [51] Rogge-Solti, A., van der Aalst, W.M.P., Weske, M. Repairing Event Logs Using Stochastic Process Models / Improving Documentation by Repairing Event Logs. 2013–2014 (流程挖掘). (缺失事件的事后修复依赖跨轨迹习得的过程模型先验: 随机 Petri 网 + trace alignment + 贝叶斯网络补时间戳)
- [52] Yang, A. et al. Vid2Seq: Large-Scale Pretraining of a Visual Language Model for Dense Video Captioning. CVPR 2023. arXiv:2302.14115. (稠密视频描述「先 temporal localization 再 captioning」两段式)
- [53] Lin, T. et al. BSN: Boundary Sensitive Network for Temporal Action Proposal Generation. ECCV 2018. arXiv:1806.02964; Bodla, N. et al. Soft-NMS: Improving Object Detection with One Line of Code. ICCV 2017. (时序候选「宁滥勿缺 + 后段精化/软排除」范式)
- [54] 语音端点检测的 hangover / 边界余量惯例: ITU-T G.729 Annex B VAD; WebRTC VAD; 两遍端点检测 (工业标准). (防切头切尾的边界余量手法, 移植为 verify 评审证据的【边界余量】段)

- [55] Apache Flink [EventTimeSessionWindows](#) / Apache Beam [Sessions](#). 工业标准会话窗口原语。nightlies.apache.org/flink / beam.apache.org (2026-07-13 访问)
- [56] Anthropic. Vision API 文档 (100 图/请求; >20 图时单图任一边 >2000px 为 400 硬拒 (非缩放); 32MB/请求; 服务端降采样档 1568px/2576px)。工业平台文档。platform.claude.com/docs/en/build-with-claude/vision (2026-07-13 访问)
- [57] GUIDE: GUI Trajectory Segmentation and Diagnosis. 2026. arXiv:2604.04399. (GUI 轨迹「纯文本动作序列整段一次调用分段 + 逐段诊断」框架; 3.3k 段 MLLM 质检 99.4% 可用率、50-80 步无衰减; 整体式 judge 随轨迹长度退化的量化数据)
- [58] Watch & Learn: Learning GUI Actions from Unlabeled Screen Recordings. CVPR 2026. arXiv:2510.04673. (zero-shot MLLM 动作标注实测 70.5% vs 专训 IDM 91.7% 的直接对照; 「噪声标注主动伤害下游」结论)
- [59] Sharingan: Extracting User Actions from Desktop Recordings. arXiv:2411.08768. (桌面录屏动作抽取 DF vs DiffF 对照: 显式帧间差异注入反而降性能; 按动作类型精度极不平衡——click P=0.94、drag P=0.40)
- [60] VideoAgentTrek / Video2Action. ICLR 2026. arXiv:2510.19488; Video2GUI. 2026. arXiv:2605.14747. (屏幕流 → 轨迹的两条 2026 工业路线: 专训 7B 边界模型 vs prompted LLM 滑窗——滑窗形态仍是量产 SOTA 之一; 两者均动作先于/伴随分段)
- [61] Web-Shepherd: Advancing PRMs for Reinforcing Web Agents. NeurIPS 2025; GUI-Shepherd. 2025. arXiv:2509.23738; AgentRewardBench. 2025. arXiv:2504.08942. (轨迹判分信度的系统证据: zero-shot judge 高方差、无单一模型通吃、GPT-4o WebArena 轨迹准确率可低至 10%; 检查清单分解是保命组件)
- [62] ByteDance. UI-TARS 统一移动动作空间。工业开源项目。github.com/bytedance/UI-TARS; UIPro: Unifying and Propelling GUI Agents. ICCV 2025. arXiv:2509.17328. (2025 后移动端动作词表的事实并集: 含 drag 与 recent/应用切换)
- [63] Do GUI Agents Believe Their Eyes? 2026. arXiv:2607.04334. (引 CLAY 对 RICO 的人工标注统计: Android 10.6% 结构节点无有效视觉呈现 (ghost node)、37.4% 屏幕含 ghost node——UI 树不可靠性的量化)
- [64] Chai, Y. et al. PIRA-Bench: Proactive Intent Recognition from Ambient Screen Streams. 2026. arXiv:2603.08013. (屏幕流 = 多线索穿插 + 噪声的形式化: $T = U \text{ 任务子轨迹 } \cup \text{ 噪声}$, 任务子轨迹为非连续帧子集; 顺序线程记忆基线 PIRF (无容量上限, 靠反思删除控规模); 噪声消融证实过连接偏差——GPT-5.2 于 PIRF 框架 precision 92.23→50.52 而 recall 83.57→84.54 反升、Gemini-3.1-Pro 85.28→53.05 同向, 原文 "trigger-happy"; 线索级指标 $S_{\text{final}} = F1 \times FPS_{\text{norm}}$ 与负样本协议。注意: 0 被引新基准 (2026-07 复核仍为 0), 自报数字权重打折)
- [65] Hu, D. H., Yang, Q. CIGAR: Concurrent and Interleaving Goal and Activity Recognition. AAAI 2008. (interleaving / concurrent 活动建模的区分; resumption 判定单元 = 「挂起目标尾动作 × 候选恢复首动作」转移对)
- [66] Cook, D. J., Crandall, A. S., Thomas, B. L., Krishnan, N. C. CASAS: A Smart Home in a Box. IEEE Computer 46(7), 2013. (interleaved ADL 数据集系列; 挂起时长分布特征使重叠活动识别 +11.36%——时间衰减先验的实证)
- [67] Leno, V., Polyvyanyy, A., Dumas, M., La Rosa, M., Maggi, F. M. Robotic Process Mining: Vision and Challenges. Business & Information Systems Engineering 62, 2020; Process Mining Handbook ch.16, Springer 2022. (UI 日志 interleaved 解缠 = open challenge; 全局法 (trace alignment / 频繁模式挖掘) 依赖「例程重复」前提, 对一次性自然任务不可迁移)
- [68] Agostinelli, S., Marrella, A., Mecella, M. 任务挖掘学术解缠系列 (Exploring the Challenge of Automated Segmentation in RPA 等, 2021/2024); UiPath Task Mining / Celonis Task Mining / Microsoft Process Advisor 官方产品文档 (2026-07 访问)。(学术系列之外, 三家工业产品均无穿插解缠——采集纪律回避 / 时间邻近分组 / 按任务录制, 产品空白区; 通信类 App 「天然噪声/穿插高发」名单同出其产品文档)
- [69] Song, Y. et al. Ego4D Goal-Step: Toward Hierarchical Understanding of Procedural Activities. NeurIPS 2023 Datasets & Benchmarks. (goal>step>substep 三级层级标注 + [is_continued](#) 续接标志——「平面分段 + 线索身份」表达穿插的直接先例)
- [70] Murty, S. et al. NNetNav: Unsupervised Learning of Browser Agents Through Environment Interaction in the Wild. 2024. arXiv:2410.02907. (自然流 → 事后反推任务标注范式; 「可命名性」剪枝判据——与 OS-Genesis [41] 同范式组)
- [71] Shao, D. et al. FineGym: A Hierarchical Video Dataset for Fine-grained Action Understanding. CVPR 2020; Kuehne, H. et al. The Language of Actions: Recovering the Syntax and Semantics of Goal-Directed Human Activities (Breakfast). CVPR 2014. (视频域层级时间标注范式——三级结构的域外印证)
- [72] Yeung, S. et al. Every Moment Counts: Dense Detailed Labeling of Actions in Complex Videos (MultiTHUMOS). IJCV 123, 2017; Sigurdsson, G. A. et al. Hollywood in Homes: Crowdsourcing Data Collection for Activity Understanding (Charades). ECCV 2016. (帧多标签/重叠活动标注先例——经评估后否决采纳 (帧单一归属是手术/归因/守恒公共地基), 引用记录被拒方案)
- [73] Fine, S., Singer, Y., Tishby, N. The Hierarchical Hidden Markov Model: Analysis and Applications. Machine Learning 32, 1998; Allen, J. F. Maintaining Knowledge about Temporal Intervals. CACM 26(11), 1983. (嵌套结构与 during / overlaps 区间关系的形式语义底座——界定「线索包含碎片、碎片穿插」语义而不引入区间树)
- [74] Saeedi, A., Peukert, E., Rahm, E. Incremental Multi-source Entity Resolution for Knowledge Graph Completion (FAMER). ESWC 2020; Gruenheid, A., Dong, X. L., Srivastava, D. Incremental Record Linkage. VLDB 2014. (增量实体消解的修复证据: 无修复贪心劣于 batch 且次序依赖、n=1 局部重聚类即追平 batch; merge-only 是增量法中质量最差、带修复可达 batch 质量——有界二遍复评的直接依据)

- [75] Kummerfeld, J. K. et al. A Large-Scale Corpus for Conversation Disentanglement. ACL 2019; Zhu, H. et al. Online Conversation Disentanglement with Pointer Networks. EMNLP 2020; Ma, X. et al. Exploring Dialogue Disentanglement with Bipartite Graph Networks. ALTA 2021. (对话解缠谱系: 贪心链接标准解码、并发线程 ≤ 3 占 46.4%、VI / 1-1 overlap / link-F1 指标三件套; 在线顺序解缠端到端先例; 全局二部图匹配仅在图结构已知时胜出)
- [76] IdentifyMe: A Challenging Long-Context Mention Resolution Benchmark. Findings of NAACL 2025; CORRECT-DETECT: Balancing Accuracy and Abstention in LLMs. EMNLP 2025. (LLM 回避「以上皆非」宁可硬连的过连接量化: 准确与弃权此消彼长、默认过度承诺——保守偏置的同向实证)
- [77] Wang, Z. et al. On Position Bias in LLM Multi-candidate Selection. COLING 2025; LLM Pairwise Judgment Sensitivity to Positional and Contextual Structure. 2026. arXiv:2604.18835. (多候选选择式判定的位置偏差与成对判定的位置/上下文结构敏感性——池卡最近活跃降序呈现与候选位置扰动测试的依据)
- [78] Zhang, Y. et al. In-context Clustering-based Entity Resolution with Large Language Models: A Design Space Exploration (LLM-CER). SIGMOD 2026. arXiv:2506.02509; GraphCR: Graph-based Cluster Repair with Active Learning. ACM JDIQ, 2025. DOI: 10.1145/3735511; Alper: Evolving-Graph Probabilistic Label Propagation for Entity Resolution. 2026. arXiv:2605.25814. (全局 LLM 聚类自身需防幻觉护栏 (MDG)、记录集成成显著影响聚类质量——非免费替代; 更重的簇修复机器——已评估、按规模不采: 会话内碎片 $\leq$ 数十, $n=1$ 复评够用且零训练)
- [79] DINCO: Diversity-Normalized Intrinsic Confidence. 2026 (ICLR 2026 投稿). arXiv:2509.25532; ADVICE: Answer-Dependence for Verbalized Calibration. ACL 2026 (Long Papers). (口头置信度系统性过高且分数饱和——"miscalibrated, reporting high confidence on instances with low accuracy"; answer-independence 致 systematic overconfidence; 采样一致性是最强基线之——去 confidence 判定腿与 votes 机制的依据)
- [80] Altmann, E. M., Trafton, J. G. Task Interruption: Resumption Lag and the Role of Cues. CogSci 2004; Trafton, J. G. et al. Preparing to Resume an Interrupted Task. IJHCS 58, 2003. (cue-guided resumption——「返回同一页面」是任务恢复强前兆线索的认知机理)
- [81] Iqbal, S. T., Horvitz, E. Disruption and Recovery of Computing Tasks: Field Study, Analysis, and Directions. CHI 2007. DOI: 10.1145/1240624.1240730. (真实桌面日志: 挂起窗口均值 3 (S.D. $\approx$ 2) ——max_open=4 的实证锚点; 27% 的挂起超过 2 小时才恢复——时间衰减与「封闭 $\neq$ 终结」依据; 切换前收尾动作密集 (段落完成率基线 0.78/min $\rightarrow$ 切换前 10.9–12.8/min, 即时响应情形) ——短段救援机理)
- [82] González, V. M., Mark, G. "Constant, Constant, Multi-tasking Craziness": Managing Multiple Working Spheres. CHI 2004; Mark, G. et al. ECSCW 2005; Leiva, L. et al. Back to the App: The Costs of Mobile Application Interruptions. MobileHCI 2012; Jones, S. L. et al. Revisitation Analysis of Smartphone App Use. UbiComp 2015. (working spheres 多任务粒度人因基线; 手机域中断/回访实证)
- [83] Oliver, N. et al. SWISH: Semantic Analysis of Window Titles and Switching History. IUI 2006; Shen, J. et al. Real-Time Detection of Task Switches of Desktop Users (TaskPredictor). IJCAI 2007; Rath, A. S. et al. 2013. (window title 是任务识别最强单特征 (85.57%); 多窗证据聚合优于单窗——摘要卡证据面依据)
- [84] Log2Plan: Task Group Discovery from GUI Logs with Two-Level Matching. UIST 2025. (embedding 召回 + LLM 精判的两级任务组匹配工业近例)
- [85] Leno, V. et al. Identifying Candidate Routines from Unsegmented UI Logs. ICPM 2020 (例程发现主引用见 [50], 此处为全局法前提面); Routine Identification over Unsegmented UI Logs Revisited. 2025. arXiv:2510.08118. (无分段 UI 日志例程识别全局法的「重复例程」前提——2025 年复核不变)
- [86] LLMs as a Jury: Cross-Model Consensus Can Outperform Process Reward Models for LLM Reasoning. 2026. arXiv:2607.10139. (精确区分两路线: §2 逐字 "self-consistency is better calibrated while the cross-model signal is more accurate"; 自一致性修不了系统性偏差 (模型把自己的错投成多数); 实测错误相关性 within-model 0.68 > same-family 0.52 > cross-family 0.47, 共享错误地板 = "unanimous and wrong")
- [87] Takada, K., Mori, S. Rethinking Dialogue Disentanglement for LLMs via Dialogue-Level Assignment and Subsequent Context (GreedyDisentangle). LaCATODA@AAAI-26, CEUR-WS Vol-4178, 2026. (M16 stitch 单调选池的同任务同形制 SOTA 先例: 簇摘要呈现 + LLM 归簇或判 new + 顺序贪心, 在 Kummerfeld IRC 基准 [75] 全指标超 per-pair 与非 LLM 方法; subsequent-context 显著有效——二遍复评的第三依据. 风险注记: DD-GEPA (arXiv:2606.07894) 报告 ~30B 开源模型上此法性能骤降——所配 profile 需实测门禁兜底)
- [88] Engram: Selective Memory Curation for Long-Horizon Agents. 2026. arXiv:2606.09900; Memori: Structured Summarization Memory. 2026. arXiv:2603.19935; LLM Agent Memory 综述. 2026. arXiv:2603.07670. (精选紧凑上下文准确率反超全量原始历史——+10.4 pt (83.6% vs 73.2%, LongMemEval) 且 token 少 8 倍; 结构化摘要以 ~5% token 胜过其它记忆系统——摘要卡证据面的正面抬升; 综述给出反向风险的正式命名 "summarization drift": 每次压缩静默丢弃低频细节)
- [89] When LLMs Agree, Are They Right? 2026. arXiv:2607.08065 (265k 样本审计); Nine Judges, Two Effective Votes. 2026. arXiv:2605.29800. (前沿模型高自一致致过度自信: GPQA 上 77% 条目自一致 ≥ 0.8 、其中 48% 是错的——自一致性只是条件性代理, votes 治方差 (漂移) 不治偏差 (过连接); 9 裁判 7 家族评审团有效独立票仅 ≈ 2.0 –2.5、聚合算法最多弥合 11% Condorcet 缺口——拒绝评审团路线的实测边界)
- [90] Tian, Y., Zhou, K., Pelleg, D. Characterization and Prediction of Mobile Tasks. ACM TOIS 41(1), 2023. DOI: 10.1145/3522711. (人工标注 1414 个真实手机任务: 仅 22.6% 任务存在穿插、多日志任务均 4.1 logs / 1.7 apps——手机穿插深度比桌面浅, 桌面锚 max_open=4 在移动域为宽松上界的间接佐证)
- [91] LlamaIndex. PromptHelper: `available_context = context_window - num_prompt_tokens - num_output` (负值抛锚)、`DEFAULT_PADDING = 5`、`repack()` 把内容重新装填至最大化利用. 工业开源项目. github.com/run-llama/llama_index (llama-index-core/llama_index/core/indices/prompt_helper.py, 2026-07-22 访问)
- [92] Claude Code auto-compact 触发式反编译记录: 触发点 = `contextWindow - min(maxOutputTokens, 20000) - 13000` ——「预留输出 + 万级固定 buffer」结构 (各来源触发百分比 ~83%/~89.4%/~95% 不一, 结构一致). gist.github.com/sam-saffron-jarvis/9d8e291c4e696ac7948702d6c4884448 (2026-07-22 访问)

- [93] OpenAI. Codex CLI 高级配置: `model_context_window` 用户声明 + 目录钳制 (`context_window.min(max_context_window)`) + 400K = 272K 输入 + 128K 输出预留 + `effective_context_window_percent = 95` 与 90% 触发压缩——声明制 + 输出预留 + 比例边距的完整同构。工业文档。
developers.openai.com/codex/config-advanced; github.com/openai/codex/issues/19185 (2026-07-22 访问)
- [94] QwenLM. Qwen-VL 官方评测口径: `max_frames = 768` 且总视频 token $\leq 24,576$ 双约束; 维护者给出 `max_pixels = 24576 \times 28 \times 28` // `num_frames` ——帧数是上限、预算守恒。github.com/QwenLM/Qwen3-VL/issues/1248 (2026-07-22 访问)
- [95] NVIDIA. NeMo Curator Nemotron-CC 管线 DocumentJoiner: 把相邻短段拼至 `max_segment_tokens` ("maximize input utilization") ——数据管线的 token 装填同类算子 (工具链主引用见 [9], 此处为装填能力面)。github.com/NVIDIA-NeMo/Curator (tutorials/synthetic/nemotron_cc/nemotron_cc_pipelines.py, 2026-07-22 访问)
- [96] BBA: A Buffer-Based Approach to Rate Adaptation (Huang, T.-Y. et al.) . SIGCOMM 2014. yuba.stanford.edu/~nickm/papers/sigcomm2014-video.pdf; BBR: Congestion-Based Congestion Control (Cardwell, N. et al.) . ACM Queue 14(5), 2016.
dl.acm.org/doi/fullHtml/10.1145/3012426.3022184. (measure-don't-model 范式两代表作: BBA 稳态不需容量估计、启动期必须要; BBR windowed-max 带宽 / windowed-min RTT 测量式建模取代丢包反应——V19 在线校准器「窗口化最大值滤波」的直接蓝本)
- [97] Cline. 生产级上下文管理的刻意反应式设计: "deliberate design choice since accurate token counting varies by model/tokenizer ... the first request that exceeds limits will fail"; 图片 10K-30K+ token/张且 usage 滞后一拍; 实证企业网关存在 `usage: null` 响应。工业开源项目 issue 记录。
github.com/cline/cline/issues/6055; issues/7383; issues/9433 (2026-07-22 访问)
- [98] Zoom Eye: 置信度分数驱动的图像树递归变焦 (训练无关; HR-Bench +15.7-17.7%, 8B 超 GPT-4o) . EMNLP 2025 Oral. github.com/om-ai-lab/ZoomEye; V*/SEAL: Guided Visual Search as a Core Mechanism in Multimodal LLMs. CVPR 2024. vstar-seal.github.io (置信度低于阈值即递归切 patch 搜索, 终止 = 命中或达最小粒度; V*Bench 7B+搜索 75.4% vs GPT-4V 55.0%); UI-Zoomer: 置信门控变焦 (训练无关; GUI grounding +4.2-13.4%) . 2026. arXiv:2604.14113. (「低置信 → 定向升清重注」谱系——V21 判审升级路径的学术与 GUI 域背书)
- [99] APIGen-MT: Agentic Pipeline for Multi-Turn Data Generation via Simulated Agent-Human Interplay. NeurIPS 2025. arXiv:2504.03601. (两阶段合成骨架: 先产出并校验任务蓝图 (含可验证的真值), 再由模拟互演把蓝图展开为多轮轨迹——v1.13 时间流形态「蓝图 → 帧实现」两类调用的直接同型先例)
- [100] Yao, L. et al. Plan-and-Write: Towards Better Automatic Storytelling. AAAI 2019. arXiv:1811.05701.; Shah, P. et al. Building a Conversational Agent Overnight with Dialogue Self-Play (M2M). 2018. arXiv:1801.04871.; Rastogi, A. et al. Towards Scalable Multi-Domain Conversational Agents: The Schema-Guided Dialogue Dataset. AAAI 2020. arXiv:1909.05855. (静态计划先行对长文本连贯性的收益对照; M2M / SGD 的「先按 schema 生成结构化真值骨架、再自然语言化」范式 = 零再标注真值保留的方法学来源——v1.13 帧类真值在蓝图层即确定的依据)
- [101] Burattin, A. PLG2: Multiperspective Process Randomization with Online and Offline Simulations. BPM 2016 Demo. arXiv:1506.08415.; Camargo, M., Dumas, M., González-Rojas, O. Automated Discovery of Business Process Simulation Models from Event Logs (Simod). Decision Support Systems 134, 2020. (过程日志生成器: 噪音逐类概率参数 + 多轨迹按时间归并交织的机械形态; Simod 的到达间隔分布族——v1.13 机械交织器「结构维度全部机械化、LLM 只管内容」的工业方法学背书)
- [102] Wu, D. et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ICLR 2025. arXiv:2410.10813.; Kwan, W.-C. et al. MT-Eval: A Multi-Turn Capabilities Evaluation Benchmark for LLMs. EMNLP 2024. arXiv:2401.16745.; Maharana, A. et al. Evaluating Very Long-Term Conversational Memory of LLM Agents (LoCoMo). ACL 2024. arXiv:2402.17753. (长程交互数据集的构造惯例: 干扰/无关轮次的注入位置参数化为可控实验变量、时间事件图先于内容生成——v1.13 噪音掷签位置与 ts 铺设先于内容的次序背书)
- [103] Process-mining 合成日志的真值方法学 (2025): 噪音须在计划层注入方能保留标签溯源链接, 事后加噪会切断真值归属。综述与实践记录 (BPM / Process Mining 社区)。(v1.13 「噪音在蓝图/交织层注入、`truth` 链接保留」的依据——对照事后加噪的不可对账形态)
- [104] Chen, L. et al. AlpaGasus: Training a Better Alpaca with Fewer Data. ICLR 2024. arXiv:2307.08701.; NVIDIA. Nemotron-4 340B Technical Report. 2024. arXiv:2406.11704. (合成数据两条纪律: 过滤优于全量 (9k 精选超 52k 全量)、判定与生成分离 (reward model / judge 独立于生成器) ——v1.13 序列级相似度过滤与判决形评审拒绝采样的背书)
- [105] Dong, Y. et al. SteerLM: Attribute Conditioned SFT as an (User-Steerable) Alternative to RLHF. EMNLP 2023 Findings. arXiv:2310.05344.; Wang, Z. et al. HelpSteer2: Open-source dataset for training top-performing reward models. NeurIPS 2024 Datasets & Benchmarks. arXiv:2406.08673. (显式多维属性条件化生成——属性是可控输入而非隐式偏好, 且属性值随样本保留供下游消费 (偏好对构造、课程学习) ——v1.14 帧类构成档位「档位是显式声明的构成属性 + 档位序号随产物三点落盘」的方法学背书)
- [106] BIMP / QBP Simulator: BPMN 2.0 离散事件业务流程仿真引擎 (到达率、活动历时分布族、资源池为机械参数; 仿真引擎盖章全部时间量)。工业工具, University of Tartu. bimp.cs.ut.ee (2026-08-18 访问; 与 [101] Simod 同族——Simod 发现的仿真模型即由本引擎执行) (v1.14 「时间量归时钟、内容量归 LLM」分工的工业形态背书)
- [107] LogGenerator: 由用户声明的过程模型经离散事件仿真产出合成事件日志的工具形态——全部时间字段由引擎盖章, 模型侧从不自报时间量。过程挖掘社区工具 (2026-08-18 访问) (v1.14 时间字段回填「绑定即剔除 + 机械回填」的同型实现先例)
- [108] Kirchdorfer, L., Özdemir, K. et al. A Divide-and-Conquer Approach for Modeling Arrival Times in Business Process Simulation (AT-KDE). arXiv:2505.22381; 扩展版 Arrival Times in Dynamic Environments: Modeling, Evaluation, and Benchmarking for Business Process Simulation. Process Science, 2026. DOI: 10.1007/s44311-026-00041-z. 开源: github.com/konradoezdemir/AT-KDE (静态到达间隔分布的局限与分段 KDE 的替代方案, 20 个真实流程实测——v1.14 把 `frame_gap_s` 的间隔分布形扩展列为演进候选而非本版内置的依据, 8.4 M6 行)

- [109] JSON Schema. Draft 2020-12 Core / Validation 规范: `contains`、`allOf`、`const` 均为原生关键字 (`contains` 断言数组中至少一个元素满足子模式, 一个 Schema 对象只有一个 `contains` 键位故多条件须分 `allOf` 支)。json-schema.org/draft/2020-12 (2026-08-18 访问; `jsonschema` $\geq$ 4.21 直接可校验, L2 无需翻译层) (v1.14 蓝图覆盖硬约束「enum 给 $\subseteq$ 、contains 给 $\supseteq$ 、合成构成恰等」的规范依据, 3.8.1)
- [110] De Giacomo, G., Vardi, M. Y. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. IJCAI 2013. <https://www.ijcai.org/Proceedings/13/Papers/132.pdf> (v1.16 有限迹、vacuity、终止位置与有限自动机语义; 此前候选材料中的 `/Papers/226.pdf` 非本文链接, 本规格固定为 `/Papers/132.pdf`)
- [111] Declare4Py. Managing Process Models — DECLARE templates. https://declare4py.readthedocs.io/en/latest/tutorials/2.Managing_Process_Models.html (v1.16 的 15 个模板名称与标准有限迹 conformance 语义参考; 本工具不引入 Declare4Py 的运行依赖)
- [112] Burattin, A., Maggi, F. M., Sperduti, A. Conformance Checking Based on Multi-Perspective Declarative Process Models. Expert Systems with Applications 65, 2016. <https://arxiv.org/pdf/1503.04957> (MP-Declare 的 activation、correlation 与 time 条件分工; 本工具不引入其完整表达式语言)
- [113] Dechter, R., Meiri, I., Pearl, J. Temporal Constraint Networks. Artificial Intelligence 49, 1991. <https://cse.unl.edu/~choueiry/Documents/DechterMeiri-AIJ.pdf> (时间约束网络与差分约束语义底座)
- [114] Disjunctive Temporal Problems under Structural Restrictions. AAAI. <https://ojs.aaai.org/index.php/AAAI/article/view/16489> (日历析取导致的组合约束; v1.16 交由同一个 CP-SAT 模型联合求解)
- [115] Google OR-Tools. CP-SAT Solver. https://developers.google.com/optimization/cp/cp_solver (v1.16 结构、时间、日历、会话、交叉与噪音槽联合规划的成熟求解器)
- [116] Google OR-Tools. Python `AddAutomaton` API. https://or-tools.github.io/docs/pdoc/ortools/sat/python/cp_model (同一组位置变量上的有限自动机约束接口; 不构造显式乘积 DFA)
- [117] Google OR-Tools 9.15 PyPI metadata. <https://pypi.org/project/ortools/> (生产依赖精确锁定 `ortools==9.15.6755`, 不按本机可用版本漂移)

附录 A. 系统默认 Rubric 全文

以下为随包分发的三套默认 rubric（`labelkit/data/rubrics/default_text.toml`、`default_ui.toml`、`default_trajectory.toml`（v1.8））完整内容。用户可用 `labelkit rubric --show` 导出为起点。文本 rubric 的四个维度直接取自 QuRating [1]（提示词为其开源提示的中文文化改写）；pointwise 等级采用 FineWeb-Edu 的加性量表形式 [11]。UI rubric 维度综合 GUI 数据管线的质量过滤实践 [13][14][15]。轨迹 rubric（v1.8）的判断来源与背书见 A.3 节注。

A.1 default:text

```
name = "default-text-v1"

[[criteria]]
key = "writing_style"
weight = 1.0
description = "写作水平：表达是否流畅、结构是否清晰、用词是否准确得体。"
pairwise_prompt = "比较两段文本，哪一段的写作水平更高（表达流畅、结构清晰、用词准确）？"
pointwise_levels = [
  "0: 语句不通或大量乱码/模板噪声，基本不可读。",
  "1: 勉强可读，但错漏频繁、结构混乱。",
  "2: 在 1 的基础上，句子基本通顺，但表达平庸、有明显冗余或口水内容。",
  "3: 在 2 的基础上，结构清晰、表达准确，接近正式成文水平。",
  "4: 在 3 的基础上，行文精炼、逻辑连贯，几乎无可挑剔的表达问题。",
  "5: 在 4 的基础上，写作水平出色，可作为高质量语料范例。"]

[[criteria]]
key = "facts_trivia"
weight = 1.0
description = "事实与知识含量：包含多少可核验的事实、知识点或具体信息。"
pairwise_prompt = "比较两段文本，哪一段包含更多具体、可核验的事实与知识？"
pointwise_levels = [
  "0: 无任何事实信息（纯情绪/寒暄/占位内容）。",
  "1: 只有零星、模糊的事实性陈述。",
  "2: 在 1 的基础上，含少量具体信息，但密度低。",
  "3: 在 2 的基础上，事实信息明确且多处出现。",
  "4: 在 3 的基础上，信息密度高、具体且相互支撑。",
  "5: 在 4 的基础上，知识含量丰富准确，具有参考价值。"]

[[criteria]]
key = "educational_value"
weight = 1.0
description = "教育/训练价值：作为模型训练数据能带来多少可学习的能力。"
pairwise_prompt = "比较两段文本，哪一段更有教育价值、更值得用于训练语言模型？"
pointwise_levels = [
  "0: 无学习价值（噪声、纯广告、无意义重复）。",
  "1: 学习价值极低，内容浅表。",
  "2: 在 1 的基础上，有一定可学习内容但组织松散。",
  "3: 在 2 的基础上，内容系统、有清晰的知识或任务示范价值。",
  "4: 在 3 的基础上，示范性强，覆盖推理/解释/结构化表达等能力。",
  "5: 在 4 的基础上，训练价值突出，属稀缺的高质量样本。"]

[[criteria]]
key = "required_expertise"
weight = 1.0
description = "所需专业度：理解该文本所需的领域专业知识深度。"
pairwise_prompt = "比较两段文本，理解哪一段需要更深的领域专业知识？"
pointwise_levels = [
  "0: 无任何专业门槛（日常寒暄级）。",
  "1: 常识即可理解。",
  "2: 在 1 的基础上，涉及少量领域术语。",
  "3: 在 2 的基础上，需要相关领域的系统背景知识。",
  "4: 在 3 的基础上，需要较深的专业训练才能完全理解。",
  "5: 在 4 的基础上，属专家级内容。"]
```

A.2 default:ui

```

name = "default-ui-v1"

[[criteria]]
key = "screenshot_readability"
weight = 1.0
description = "截图可读性：图像清晰、无遮挡/花屏/过度压缩，界面元素可辨认。"
pairwise_prompt = "比较两组屏幕数据，哪一组的截图更清晰可读（无花屏、遮挡、严重压缩伪影）？"
pointwise_levels = [
    "0：图像损坏或几乎不可辨认。", "1：严重模糊/遮挡，仅能辨认轮廓。",
    "2：在 1 的基础上，主要元素可辨认但细节文字难以阅读。",
    "3：在 2 的基础上，界面文字基本可读。",
    "4：在 3 的基础上，全部元素清晰锐利。",
    "5：在 4 的基础上，截图质量无可挑剔。"]

[[criteria]]
key = "tree_screen_consistency"
weight = 1.5
description = "树-图一致性：UI 树节点（角色/文本/边界框）与截图内容相互吻合。"
pairwise_prompt = "比较两组屏幕数据，哪一组的 UI 控件树与截图内容更一致（节点文本、位置与画面吻合）？"
pointwise_levels = [
    "0：树与截图明显不是同一屏。", "1：大量节点在截图中找不到对应。",
    "2：在 1 的基础上，主要控件能对应但坐标/文本多处不符。",
    "3：在 2 的基础上，绝大多数节点与画面吻合。",
    "4：在 3 的基础上，逐节点核对仅个别偏差。",
    "5：在 4 的基础上，树与截图完全一致。"]

[[criteria]]
key = "state_completeness"
weight = 1.0
description = "界面状态完整性：页面加载完成、无骨架屏/空白区/被弹层意外遮挡。"
pairwise_prompt = "比较两组屏幕数据，哪一组的界面状态更完整（加载完成、无骨架屏或异常空白）？"
pointwise_levels = [
    "0：空白页或全骨架屏。", "1：大面积未加载/报错区域。",
    "2：在 1 的基础上，主体内容呈现但有局部缺失。",
    "3：在 2 的基础上，页面完整呈现。",
    "4：在 3 的基础上，状态稳定（无加载中痕迹、toast 残影）。",
    "5：在 4 的基础上，界面状态可作为标准样本。"]

[[criteria]]
key = "interaction_richness"
weight = 1.0
description = "信息与交互丰富度：界面包含的信息量与可交互元素的多样性（对训练 GUI 理解/操作模型的价值）。"
pairwise_prompt = "比较两组屏幕数据，哪一组的界面信息量与可交互元素更丰富、更有训练价值？"
pointwise_levels = [
    "0：空屏/纯壁纸/锁屏等无信息界面。", "1：元素极少（如单按钮启动页）。",
    "2：在 1 的基础上，有少量信息或控件。",
    "3：在 2 的基础上，信息与控件种类中等偏上。",
    "4：在 3 的基础上，界面信息密集、控件类型多样。",
    "5：在 4 的基础上，属高价值复杂界面样本。"]

```

A.3 default:trajectory (v1.8)

序列/轨迹打分的内置 rubric（`labelkit/data/rubrics/default_trajectory.toml`）：`quality.rubric` 缺省（空串）且**打分单元是序列**时自动选用——条件为 `segment.enabled = true`（v1.8 stream 模式）**∨** `generate_stream.enabled = true`（v1.13 时间流生成形态：直装序列同样按轨迹四准则打分，3.6.5）；两模态一致，用户显式选择器恒优先（3.4.3、3.1.4 S29 扩展）。判据说明三则：① 四个准则全部**任务无关**——锚定 episode 的内部结构（终态证据、步骤承接、方向性、离题占比）而非外部任务语义，无需知道任务是什么即可打分；措辞模态中立、不预设步骤在场（extract 关闭时「步骤」读作「帧间变化」，3.4.3）。② completion 与 coherence 源自 OS-Genesis trajectory reward model 的 Completion + Coherence 两维 [41]——TRM 原文为 **1—5 五级**整数分制，本表 **0—5 六级**是 LabelKit 家规（default:text 量表形态）的改制，多出的一级用于区分「彻底不可用」与「早期夭折」。③ purposefulness 与 noise_residue 无 TRM 原文背书、背书分开挂：前者自 Coherence 的 "toward the goal" 语义拆分独立成维，后者源自 RPA UI 日志分割对「不属任何例程的噪声事件」的显式处理（Leno et al. [50]）。


```
name = "default-trajectory-v1"

[[criterial]]
key = "completion"
weight = 1.0
description = "完成度：序列是否推进到可辨识的终态（确认页/成功提示/收尾返回），还是中断在半途。"
pairwise_prompt = "比较两条操作序列，哪一条更完整地到达了流程终态（确认页、成功提示、收尾返回等证据更充分）？"
pointwise_levels = [
  "0：无任何有效推进：停留在起始屏或全为无效重复操作，看不到朝向任何终态的进展。",
  "1：有少量推进但明显夭折：在流程早期即中断，远未接近终态。",
  "2：在 1 的基础上，推进到流程中段，但未帧仍是未完成的中间态（如表单未提交、订单未确认）。",
  "3：在 2 的基础上，主要步骤已完成，但缺少收尾证据（无确认页或成功提示，未步含糊）。",
  "4：在 3 的基础上，末帧呈现明确终态证据（确认页、成功提示、完成回执），仅个别交互处理不干净。",
  "5：在 4 的基础上，全流程完整收尾（含必要的确认或返回入口动作），终态无歧义。"]

[[criterial]]
key = "coherence"
weight = 1.0
description = "连贯性：相邻步骤是否互相承接，有无无法解释的状态跳变或往复抖动。"
pairwise_prompt = "比较两条操作序列，哪一条的步骤衔接更连贯（无跳变、无往复抖动、无不可解释的状态变化）？"
pointwise_levels = [
  "0：步骤间基本无承接关系：大量不可解释的状态跳变，或多数动作无法辨识。",
  "1：偶有承接，但断裂多于连贯，难以还原为一条操作线。",
  "2：在 1 的基础上，主线可辨，但存在多处跳变或往复抖动（同一屏幕反复进出）。",
  "3：在 2 的基础上，主线连贯，仅有一两处轻微跳变或冗余往返，不影响理解。",
  "4：在 3 的基础上，步步承接、方向一致，无可见跳变，仅个别可解释的重复操作。",
  "5：在 4 的基础上，衔接严密无冗余，每一步都是前一步的自然后继。"]

[[criterial]]
key = "purposefulness"
weight = 1.0
description = "目的性：步骤是否构成朝单一目标的持续推进，而非漫游、乱点或多目标混杂。"
pairwise_prompt = "比较两条操作序列，哪一条更像在朝单一明确目标持续推进（而非漫游乱点或多个意图混杂）？"
pointwise_levels = [
  "0：完全漫游：步骤间无方向性，如随机点击或来回浏览，无法归纳出任何目标。",
  "1：隐约有意图但方向频繁变化，无法归纳为单一目标。",
  "2：在 1 的基础上，可归纳出大致目标，但夹杂明显的无关探索或并行意图。",
  "3：在 2 的基础上，主体步骤朝单一目标推进，离题探索占少数且很快回归。",
  "4：在 3 的基础上，几乎每步都在逼近目标（同族屏幕递进、输入逐步收敛），偏离可忽略。",
  "5：在 4 的基础上，路径接近达成该目标的最短操作序列，目的性一目了然。"]

[[criterial]]
key = "noise_residue"
weight = 1.0
description = "噪声残留：段内与主线活动无关的步骤（弹窗、通知、误触、无关屏幕）的残留程度。"
pairwise_prompt = "比较两条操作序列，哪一条残留的无关步骤（弹窗、通知、误触等离题内容）更少？"
pointwise_levels = [
  "0：噪声占主体：一半以上步骤与主线无关，段基本不可用。",
  "1：噪声比例高，主线被反复打断。",
  "2：在 1 的基础上，噪声仍多处出现，但主线整体可辨。",
  "3：在 2 的基础上，仅少数几步离题，且不打断主线理解。",
  "4：在 3 的基础上，至多一处轻微离题（如一闪而过的通知），几乎不影响样本质量。",
  "5：在 4 的基础上，全部步骤均属主线，无任何噪声残留。"]
```